Car Connectivity Consortium

Digital Key Release 3

Technical Specification

Version 1.2.2
(CCC-TS-101)



Copyright © 2011-2023 Car Connectivity Consortium LLC
All rights reserved
Confidential

VERSION HISTORY

Version	Date	Comment
1.0.0	16 Apr 2020	Release 2, BoD Approved Version 1.0
1.1.0	22 Dec 2020	Release 2, BoD Approved Version 1.1
1.0.0	19 May 2021	Release 3, BoD Approved Version 1.0
1.1.0	25 July 2022	Release 3, BoD Approved Version 1.1.0
1.2.0	15 Feb 2023	Release 3, BoD Approved Version 1.2.0
1.2.1	19 July 2023	Release 3, BoD Approved Version 1.2.1
1.2.2	20 Sep. 2023	Release 3, BoD Approved Version 1.2.2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

LEGAL NOTICE

The copyright in this Specification is owned by the Car Connectivity Consortium LLC ("CCC LLC"). Use of this Specification and any related intellectual property (collectively, the "Specification") is governed by; (a) the license terms contained in this Legal Notice and the Car Connectivity Consortium Specification License Agreement (the "License Agreement") for anyone who is not a Member of CCC LLC (a "non-Member"); or (b) the terms contained in this Legal Notice and the CCC LLC Limited Liability Company Agreement (the "LLC Agreement") for any party who is a Member of CCC LLC (a "Member").

Use of the Specification by a non-Member is prohibited unless such non-Member has entered into the License Agreement and downloaded this Specification in accordance with the requirements of CCC LLC. The legal rights and obligations of each Member are governed by the LLC Agreement and their applicable Membership Agreement, including without limitation those contained in Article 10 of the LLC Agreement.

In addition to the terms of the LLC Agreement and its exhibits and appendices, CCC LLC hereby grants each Member a right to use and to make verbatim copies of the Specification for the purposes of implementing the technologies specified in the Specification to their products ("Implementing Products") under the terms of the LLC Agreement (the "Purpose"). Members are not permitted to make available or distribute this Specification or any copies thereof to non-Members other than to their Affiliates (as defined in the LLC Agreement) and subcontractors but only to the extent that such Affiliates and subcontractors have a need to know for carrying out the Purpose and provided that such Affiliates and subcontractors accept confidentiality obligations similar to those contained in the LLC Agreement. Each Member shall be responsible for the observance and proper performance by such of its Affiliates and subcontractors of the terms and conditions of this Legal Notice and the Agreement. No other license, express or implied, by estoppel or otherwise, to any intellectual property rights are granted to Members herein.

THE SPECIFICATION IS PROVIDED "AS IS" WITH NO WARRANTIES, EXPRESS OR IMPLIED, INCLUDING WITHOUT LIMITATION ANY WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NONINFRINGEMENT OF ANY THIRD PARTY INTELLECTUAL PROPERTY RIGHTS, AND COMPLIANCE WITH APPLICABLE LAWS. Notice for Members: Any use of the Specification not in compliance with the applicable terms of this Legal Notice, the LLC Agreement, and Membership Agreement is prohibited and any such prohibited use may result in termination of the applicable Membership Agreement and other liability permitted by the applicable agreement or by applicable law to CCC LLC or any of its members for patent, copyright, and/or trademark infringement.

Notice for Non-Members: Any use of the Specification not in compliance with the applicable terms of this Legal Notice or the License Agreement for non-Members is prohibited and any such prohibited use may result in termination of the non-Member's License Agreement and other liability permitted by the License Agreement or by applicable law to CCC LLC or any of its members for patent, copyright, and/or trademark infringement.

NOTHING IN THE SPECIFICATION CREATES ANY WARRANTIES, EITHER EXPRESS OR IMPLIED, REGARDING SUCH LAWS OR REGULATIONS. ALL LIABILITY, INCLUDING LIABILITY FOR INFRINGEMENT OF ANY INTELLECTUAL PROPERTY RIGHTS OR FOR NONCOMPLIANCE WITH LAWS RELATING TO USE OF THE SPECIFICATION IS EXPRESSLY DISCLAIMED. BY USE OF THE SPECIFICATION, EACH MEMBER AND NON-MEMBER EXPRESSLY WAIVES ANY CLAIM AGAINST CCC LLC AND ITS MEMBERS RELATED TO USE OF THE SPECIFICATION.

Disclaimers for Members: Each Member hereby acknowledges that its Implementing Products may be subject to various regulatory controls under the laws and regulations of various jurisdictions worldwide. Such laws and regulatory controls may govern, among other things, the combination, operation, use, implementation, and distribution of Implementing Products. Examples of such laws and regulatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

controls include, but are not limited to, road safety regulations, telecommunications regulations, technology transfer controls, and health and safety regulations. Each Member is solely responsible for the compliance by their Implementing Products with any such laws and regulations and for obtaining any and all required authorizations, permits, or licenses for their Implementing Products related to such regulations within the applicable jurisdictions. Each Member acknowledges that nothing in the Specification provides any information or assistance in connection with securing such compliance, authorizations, or licenses**.**

Disclaimers for non-Members: Each non-Member that downloads or otherwise obtains access to the Specification acknowledges that its limited copyright license is for the sole purpose of evaluation and such license does not extend to implementations. Products that are the result of implementing the Specification may be subject to various regulatory controls under the laws and regulations of various jurisdictions worldwide. Such laws and regulatory controls may govern, among other things, the combination, operation, use, implementation, and distribution of products. Examples of such laws and regulatory controls include, but are not limited to, road safety regulations, telecommunications regulations, technology transfer controls, and health and safety regulations. Each non-Member that downloads or otherwise obtains access to the Specification is solely responsible for decisions whether to become a Member of CCC LLC and whether to implement products based upon the Specification, and non-Member acknowledges that if such non-Member implements products then it is solely responsible for the compliance with any such laws and regulations and for obtaining any and all required authorizations, permits, or licenses for their products related to such regulations within the applicable jurisdictions. Each non-Member that downloads or otherwise obtains access to the Specification acknowledges that nothing in the Specification provides any information or assistance in connection with securing such compliance, authorizations, or licenses for products that may implement the Specification**.**

CCC LLC reserves the right to adopt any changes or alterations to the Specification as it deems necessary or appropriate.

COPYRIGHT © 2011-2022. CCC LLC.

TABLE OF CONTENTS

VERSION HISTORY **2**
LEGAL NOTICE **3**
TABLE OF CONTENTS **5**
LIST OF FIGURES **17**
LIST OF TABLES **20**
LIST OF LISTINGS **26**
ABBREVIATIONS AND ACRONYMS **29**
DEFINITIONS **32**
NOTATIONS **33**
CODE LISTINGS **35**
1 INTRODUCTION AND SCOPE **36**
2 SYSTEM ARCHITECTURE **37**
2.1 OVERVIEW 37
2.2 HIGH-LEVEL FEATURES 37
2.3 HIGH LEVEL ARCHITECTURE 37
2.4 ACTORS 39
2.4.1 Vehicle *39*
2.4.2 Vehicle NFC Readers [WCC1] *39*
2.4.3 Vehicle Bluetooth LE Module [WCC2/WCC3] *40*
2.4.4 Vehicle UWB Module [WCC3] *40*
2.4.5 Vehicle OEM Server *40*
2.4.6 Key Tracking Server (KTS) *40*
2.4.7 Devices *41*
2.4.8 Device OEM Server *41*
2.4.9 Relay Server *41*
2.5 RELATIONSHIPS 42
2.5.1 Door NFC Reader (3) [WCC1] *42*
2.5.2 Console NFC Reader (4) [WCC1] *42*
2.5.3 Bluetooth LE Interface (11) [WCC2/WCC3] *42*
2.5.4 UWB Interface (12) [WCC3] *43*
2.5.5 Owner-to-Friend Device Link (via (2), (6), (8), and (7)) *43*
2.5.6 Owner or Friend Device to Vehicle OEM Server (10, 9) *43*
2.5.7 Telematics Link (1) *43*
2.5.8 Owner/Friend Device OEM Server Link (2,7) *43*
2.5.9 Vehicle OEM Server to KTS (5) *43*
2.5.10 Owner/Friend Device OEM Server to Vehicle OEM Server (6,8) *44*
2.6 DEVICE STRUCTURE 45

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2.6.1 NFC Component [WCC1] *45*
2.6.2 Bluetooth Module [WCC2/WCC3] *46*
2.6.3 UWB Module [WCC3] *46*
2.6.4 Secure Element (or equivalent) *46*
2.6.5 Digital Key Applet *46*
2.6.6 Digital Key Framework *47*
2.6.7 Vehicle OEM App *47*
2.6.8 Native App *47*
2.7 VEHICLE STATES 47
2.8 DIGITAL KEY USER CLASSES 47
2.8.1 Owner *47*
2.8.2 Friend *48*
2.9 ACCESS PROFILES 48
2.9.1 Owner *48*
2.9.2 Friend *48*
2.10 VERSIONING 48
2.10.1 General *48*
2.10.2 Domain Versions *48*
2.10.3 Domain Wise Version Agreement *50*
2.10.4 Software Updates *53*
2.10.5 Version Numbers *57*
2.10.6 Version Introduction *57*
2.10.7 Version Support *58*
2.10.8 Version Table *59*
2.10.9 Version Deprecation *60*
3 NFC INTERFACE [WCC1] 61
3.1 NFC FUNCTIONAL REQUIREMENTS 61
3.1.1 Vehicle *61*
3.1.2 Device *61*
3.2 NFC PROCEDURES 62
3.2.1 NFC Polling and Link Setup Procedure *62*
3.2.2 NFC Data Transfer Procedure *63*
3.2.3 NFC Link Teardown Procedure *63*
3.2.4 NFC Reset Procedure *64*
4 DATA STRUCTURE 65
4.1 APPLET INSTANCE LAYOUT 65
4.2 DIGITAL KEY STRUCTURE 66
4.3 MAILBOXES 68
4.3.1 Private Mailbox *69*
4.3.2 Confidential Mailbox *74*
4.3.3 Immobilizer Token *75*

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

4.3.4 Mailbox Access Rights ... 76
5 OWNER PAIRING COMMANDS [WCC1/WCC2/WCC3] ... 77
5.1 SUPPORTED COMMANDS FOR OWNER PAIRING ... 77
5.1.1 SELECT Command ... 78
5.1.2 SPAKE2+ REQUEST Command ... 79
5.1.3 SPAKE2+ VERIFY Command ... 81
5.1.4 WRITE DATA Command ... 83
5.1.5 GET DATA Command ... 89
5.1.6 GET RESPONSE Command ... 90
5.1.7 OP CONTROL FLOW Command ... 91
5.2 DATA ELEMENTS ... 93
5.2.1 Certificates ... 93
6 OWNER PAIRING ... 94
6.1 OVERVIEW ... 94
6.2 KEYS AND DATA ... 94
6.3 OWNER PAIRING IMPLEMENTATION ... 94
6.3.1 Phase 0: Preparation ... 96
6.3.2 Phase 1: Initiate Pairing Procedure ... 96
6.3.3 Phase 2: First Session with NFC Reader ... 97
6.3.4 Phase 3: Second Session with NFC Reader ... 109
6.3.5 Phase 4: Finalization of Pairing Procedure (optional) ... 115
6.3.6 Timers ... 118
6.3.7 Pairing Password URL ... 119
7 STANDARD TRANSACTION ... 121
8 FAST TRANSACTION ... 123
9 USER AUTHENTICATION ... 124
9.1 EXPLICIT USER AUTHENTICATION POLICY ... 125
9.2 IMPLICIT USER AUTHENTICATION POLICY ... 125
10 CHECK PRESENCE TRANSACTION ... 127
11 DIGITAL KEY SHARING [WCC1/WCC2] ... 128
11.1 ENCODING ... 128
11.2 SHARING PRINCIPLES ... 128
11.2.1 Definitions ... 128
11.2.2 Principles ... 129
11.3 COMMUNICATION CHANNEL ... 130
11.3.1 Notifications ... 132
11.3.2 Cross-Platform Sharing Invitation ... 132
11.3.3 General API Parameter Definitions ... 132
11.3.4 API Parameter Usage ... 136
11.3.5 Security Considerations ... 138

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

11.3.6 Device PIN ... 138
11.4 CERTIFICATE CHAIN ... 144
11.5 PREREQUISITES ... 144
11.6 KEY CONFIGURATION ... 145
11.7 ERROR CODES ... 145
11.8 KEY SHARING FLOW: STEPS ... 146
11.8.1 Steps 1 and 2 (Owner): Sharing Invitation ... 146
11.8.2 Steps 3 and 4 (Friend): Key Signing Request ... 148
11.8.3 Steps 5 and 6 (Owner): Generate Import Request ... 150
11.8.4 Step 7 (Friend): Endpoint Data Import ... 153
11.8.5 Steps 8 and 9 (Friend): Track Key ... 155
11.8.6 Step 10 (Vehicle OEM Server): Track Key Response ... 155
11.8.7 Step 14 (Friend): First Transaction ... 155
11.9 OWNER DEVICE OEM SERVER NOTIFICATION ... 156
11.9.1 Step 12 (Vehicle OEM Server): eventNotification ... 156
11.9.2 Step 13 (Owner Device OEM Server): eventNotificationResponse ... 156
11.10 KEY TRACKING AND ONLINE ATTESTATION DELIVERY ... 156
11.10.1 Online immobilizer token and slot identifier retrieval ... 158
11.11 VEHICLE ATTESTATION ... 159
11.12 ALGORITHMS ... 159
11.13 VERSIONING ... 161
12 DELEGATED KEY SHARING ... 162
13 KEY TERMINATION AND DELETION [WCC1/WCC2] ... 163
13.1 KEY TERMINATION SCENARIOS ... 164
13.2 FRIEND KEY REMOTE TERMINATION ... 167
13.2.1 REV_100: Friend key termination in vehicle ... 168
13.2.2 REV_110: Friend key termination in owner Vehicle OEM account ... 169
13.2.3 REV_120: Friend key termination in friend Vehicle OEM account ... 169
13.2.4 REV_130: Friend key termination on owner device natively ... 171
13.2.5 REV_140: Friend key termination on owner device in Vehicle OEM app ... 171
13.2.6 REV_150: Friend key termination based on expiry date of the key ... 171
13.2.7 REV_160: Friend key termination by Device OEM (device security issue) ... 173
13.2.8 REV_170: Friend key termination due to remote wipe of device ... 175
13.2.9 REV_180: Friend key termination due to deletion of personal data in friend Vehicle OEM account ... 176
13.3 FRIEND KEY LOCAL TERMINATION ... 177
13.3.1 REV_200: Friend key termination on friend device natively ... 177
13.3.2 REV_210: Friend key termination on friend device in Vehicle OEM app ... 177
13.3.3 REV_220: Friend key termination due to local wipe of device ... 178
13.4 FRIEND KEY REMOTE SUSPEND/RESUME ... 178
13.4.1 REV_300: Friend key temporary suspension in Device OEM account ... 178

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

13.4.2 REV_310: Friend key resume in Device OEM account ... 179
13.4.3 Resume attestation ... 181
13.5 OWNER KEY REMOTE TERMINATION ... 181
13.5.1 REV_400: Owner key deletion in vehicle UI (change device) ... 181
13.5.2 REV_410: Owner key termination by Device OEM (security issue) ... 183
13.5.3 REV_420: Owner key termination due to remote wipe of device ... 183
13.5.4 REV_500: Owner key termination on owner device natively (delete pass for Digital Key) ... 183
13.5.5 REV_510: Owner key termination on owner device in Vehicle OEM app ... 184
13.5.6 REV_520: Owner key termination due to local wipe of device ... 184
13.5.7 REV_600/610: Owner key temporary suspension in device lost mode in Device OEM account ... 184
13.6 OWNER DEVICE UNPAIRING ... 185
13.6.1 REV_700: Unpairing in vehicle UI (sell vehicle) ... 185
13.6.2 REV_710: Unpairing on owner device in Vehicle OEM app ... 187
13.6.3 REV_720: Unpairing in owner Vehicle OEM account (sale of vehicle) ... 188
13.6.4 REV_730: Unpairing based on cancellation or expiry date of the Digital Key service 188
13.6.5 REV_740: Unpairing by Vehicle OEM (vehicle stolen) ... 188
13.6.6 REV_750: Unpairing by Vehicle OEM (security breach in vehicle) ... 188
13.6.7 REV_760: Unpairing by Vehicle OEM (garage service process) ... 188
13.6.8 REV_770: Unpairing due to deletion of personal data in owner Vehicle OEM account 188
13.6.9 REV_800: Unbinding of vehicle from owner account in vehicle UI (unpairing, owner sale of vehicle) ... 189
13.6.10 REV_810: Unbinding of vehicle from owner account in owner Vehicle OEM account (unpairing, owner sale of vehicle) ... 189
13.6.11 REV_820: Unbinding of vehicle from owner account in Vehicle OEM app (unpairing, owner sale of vehicle) ... 189
13.7 TERMINATION IN VEHICLE ... 189
13.7.1 Rules ... 189
13.7.2 Example ... 190
14 AUTHENTICATION AND PRIVACY ... 194
14.1 AUTHENTICATION AND PRIVACY KEYS ... 194
14.1.1 Usage ... 194
14.1.2 Certificate Chains ... 195
14.2 REMOTE TERMINATION REQUESTS ... 197
14.2.1 Example of RTR Signature Verification ... 200
14.3 ENCRYPTION AND SIGNATURE VERIFICATION SCHEMES ... 201
14.4 OEM APP DATA ATTESTATION ... 201
15 DIGITAL KEY APPLET ... 202
15.1 INTRODUCTION ... 202

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

15.2 KEYS AND DATA 202
15.3 APPLET IMPLEMENTATION 203
15.3.1 Introduction 203
15.3.2 Commands 215
15.3.3 Security 257
15.3.4 Optimization 260
15.4 HELPER METHOD 261
15.4.1 Hierarchy 261
15.5 VEHICLE IMPLEMENTATION 272
15.5.1 Security Guidelines 272
15.5.2 Optimizations 272
16 CERTIFICATE 273
16.1 GENERAL 273
16.2 CERTIFICATES AND RELATIONSHIPS 273
16.2.1 [A] – SE Root CA Certificate 276
16.2.2 [B] – SE Root Certificate 276
16.2.3 [C] – Instance CA Attestation (signed by SE Root) per Vehicle OEM 276
16.2.4 [D] – Device OEM CA Certificate 276
16.2.5 [E] – Instance CA Certificate (Signed by Device OEM) per Vehicle OEM 277
16.2.6 [F] – Device OEM CA Certificate (Signed by Vehicle OEM) 277
16.2.7 [G] – Digital Key per vehicle 277
16.2.8 [H] – Digital Key Certificate 277
16.2.9 [J] – Vehicle OEM CA Certificate 277
16.2.10 [K] – Vehicle Public Key Certificate 278
16.2.11 [L] – Digital Key Creation Data 278
16.2.12 [M] – Vehicle OEM CA Certificate (signed by Device OEM) 278
16.2.13 [N] – Instance CA Attestation (self-signed) per Vehicle OEM 278
16.3 CERTIFICATE SIZE RESTRICTIONS 279
16.4 DEVICE CERTIFICATE CHAIN 280
16.5 VEHICLE CERTIFICATE CHAIN 280
16.6 SUPPORTED VERIFICATION CHAIN 280
16.7 OWNER PAIRING CERTIFICATE CHAIN 280
16.7.1 Device and Vehicle 280
16.8 KEY SHARING CERTIFICATE CHAIN 281
17 SERVER-TO-SERVER COMMUNICATIONS AND API 282
17.1 INTRODUCTION 282
17.2 API DESIGN 282
17.3 SECURITY 282
17.4 URL SCHEME 282
17.5 HEALTHCHECK 283

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17.6 USER INTERFACE 283
17.6.1 Description 283
17.7 SERVER APIS 283
17.7.1 API - Track Key 283
17.7.2 Manage Key 290
17.7.3 API – eventNotification 295
17.7.4 API – Recover Key Data 299
17.7.5 API – Healthcheck 304
17.7.6 API – versionUpdate 304
17.8 STRUCTURE DEFINITIONS 307
17.8.1 Request Headers 307
17.8.2 Response Headers 308
17.8.3 Key Type 309
17.8.4 EncryptedDataContainer 309
17.8.5 Unencrypted uiBundle 309
17.8.6 Digital Key Status 311
17.8.7 Action for manageKey 312
17.8.8 Event Type of Notification 312
17.8.9 Event Data of Event Notification 315
17.8.10 Unencrypted Event Notification Data 316
17.8.11 Shared Key 317
17.8.12 Key Configuration 317
17.8.13 Entitlement 318
17.8.14 Unencrypted vehicleMobilizationData Fields 318
17.8.15 Supported Entitlements 319
17.8.16 Device Type 319
17.8.17 UI Elements 319
17.8.18 Activation Options 319
17.8.19 Supported approved sharing methods 320
17.9 HTTP STATUS CODES 320
17.10 SUB STATUS CODES 321
17.11 ECC ENCRYPTION FOR DEVICE AND SERVER 323
17.11.1 Frame/Packet Contents 324
17.11.2 Encryption Process 324
17.11.3 Decryption Process 324
17.11.4 Key Derivation Function 325
17.11.5 Symmetric Encryption Key 326
17.11.6 Initialization Vector Derivation 326
17.11.7 EC Public Key Point Encoding in Uncompressed Form 326
17.12 VERSIONING 326
17.12.1 General 326
17.13 EXAMPLE 326

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17.13.1 Keys ... 326
17.13.2 Message to encrypt ... 327
17.13.3 Encryption process ... 327
17.13.4 Decryption process ... 327
18 SECURITY ... 329
18.1 SPAKE2+ PROTOCOL DESCRIPTION ... 329
18.1.1 General ... 329
18.1.2 Execution ... 329
18.1.3 Verification ... 331
18.1.4 Summary ... 331
18.1.5 SPAKE2+ Constant Definitions ... 332
18.2 PRIVACY PROPERTIES ... 333
18.2.1 NFC Transaction [WCC1/WCC2] ... 333
18.2.2 Vehicle OEM Server ... 333
18.2.3 User Privacy ... 333
18.2.4 Multi-Vehicle OEM Deployment ... 333
18.2.5 Error Handling ... 334
18.3 CRYPTOGRAPHIC ALGORITHMS ... 334
18.4 CRYPTOGRAPHIC PROTOCOLS ... 334
18.4.1 Server Password Generation ... 334
18.4.2 Vehicle-side public EC Point Generation ... 334
18.4.3 Device-side public EC Point Generation ... 334
18.4.4 Vehicle-side computation of Shared Secret ... 335
18.4.5 Device-side computation of Shared Secret ... 335
18.4.6 Derivation of Evidence Keys ... 335
18.4.7 Vehicle-side computation of Evidence ... 335
18.4.8 Device-side computation of Evidence ... 336
18.4.9 Derivation of System Keys [WCC1/WCC2/WCC3] ... 336
18.4.10 Generate Attestation Signature ... 336
18.4.11 Validate Attestation or Certificate ... 336
18.4.12 Secure Channel Command Encryption and Authentication ... 336
18.4.13 Secure Channel Response Encryption and Authentication ... 337
18.5 ERROR HANDLING [WCC1/WCC2/WCC3] ... 337
18.6 SECURE ELEMENT (SE) ... 337
19 BLUETOOTH LOW ENERGY (LE) INTERFACE [WCC2/WCC3] ... 339
19.1 BLUETOOTH LE FUNCTIONAL REQUIREMENTS ... 339
19.1.1 Vehicle ... 339
19.1.2 Device ... 339
19.2 BLUETOOTH LE PROCEDURES ... 340
19.2.1 Owner Pairing Connection Establishment ... 340
19.2.2 Bluetooth Encryption ... 352

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.2.3 Passive Entry: Bluetooth LE Setup: ... *352*
19.3 DK MESSAGE FORMAT ... 356
19.3.1 UWB Ranging Service Message ... *358*
19.3.2 SE Message ... *366*
19.3.3 Supplementary Service Message - Time Sync ... *367*
19.3.4 Supplementary Service Message - First Approach ... *369*
19.3.5 Supplementary Service Message - RKE ... *370*
19.3.6 Vehicle OEM App Message ... *371*
19.3.7 Head Unit Pairing Message ... *371*
19.3.8 DK Event Notification ... *373*
19.4 TIME SYNCHRONIZATION ... 387
19.4.1 Definitions ... *387*
19.4.2 General Description ... *390*
19.4.3 Device Uncertainty ... *393*
19.4.4 "Bluetooth LE Timesync" Procedures. ... *394*
19.4.5 "Bluetooth LE Timesync" message flow examples ... *397*
19.4.6 Conditions for a "Bluetooth LE Timesync" procedure ... *399*
19.5 DIGITAL KEY – FLOWS ... 400
19.5.1 Owner Pairing Flow ... *400*
19.5.2 Head Unit Pairing Flow ... *408*
19.5.3 Passive Entry ... *410*
19.5.4 URSK Management ... *410*
19.5.5 Flow Selection - Establish Secure Ranging ... *413*
19.5.6 Standard Transaction over Bluetooth LE ... *419*
19.5.7 Engine Start ... *421*
19.5.8 Friend - Sharing & First Approach ... *421*
19.5.9 RKE Function Flow ... *424*
19.5.10 Bluetooth LE Activation Flow ... *429*
19.6 DIGITAL KEY - PREFERENCE MANAGEMENT ... 430
19.6.1 Preference Management: Connected Vehicles < Connection Limit ... *430*
19.6.2 Preference Management: Connected Vehicles >= Connection Limit ... *432*
19.7 SUBEVENT HANDLING ... 432
19.7.1 Command Complete SubEvent Code Handling ... *432*
19.7.2 Ranging Session Status Changed SubEvent ... *433*
19.8 ERROR HANDLING ... 435
19.8.1 SE Transaction Recovery ... *435*
20 UWB MAC AND CHANNEL ACCESS [WCC3] ... 437
20.1 UWB MAC ARCHITECTURE ... 437
20.1.1 Overview ... *437*
20.2 MAC TIME GRID ... 439
20.3 MAC TIME GRID SYNCHRONIZATION ... 444

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

20.4 HOPPING FLAG AND ROUND INDEX DETERMINATION 445
20.5 MAC PROTOCOL 447
20.5.1 Ranging Exchange Sequence 447
20.5.2 Ranging Session Setup 452
20.5.3 MAC Control Channel (Over Bluetooth LE) 455
20.5.4 UWB MAC Configuration 456
20.6 STS INDEX INCREMENTATION 458
20.6.1 Rules for STS Index Incrementation 459
20.6.2 STS Incrementation and Packet Mapping within a Ranging Round 459
20.6.3 Calculation of STS Index 460
21 UWB PHY [WCC3] 461
21.1 INTRODUCTION 461
21.2 UWB PHY BLOCK DIAGRAM 461
21.3 UWB PACKET FORMAT 461
21.4 UWB FRAME ELEMENTS 462
21.4.1 BPRF SYNC 463
21.4.2 BPRF SFD 464
21.4.3 Scrambled Timestamp Sequence (STS) 464
21.4.4 BPRF PHR 464
21.4.5 Standard Compliant Data Field 465
21.5 PULSE SHAPE COMBINATIONS 465
21.5.1 Symmetrical Root-Raised-Cosine Pulse – PulseShape 0x0 465
21.5.2 Precursor-Free Pulse – PulseShape 0x1 466
21.5.3 PulseShape Combinations 466
21.6 RANGING MARKER 467
21.7 UWB RF 467
21.7.1 Operating frequency bands and Channel assignments 467
21.7.2 Frequency Source Requirements 467
22 UWB SECURITY [WCC3] 468
22.1 CRYPTOGRAPHY 468
22.1.1 Deriving the STS Sequence from the DRBG 468
22.1.2 SP0 Frames Encryption 468
22.1.3 Key Derivation Functions (KDFs) 469
22.1.4 UWB Tracking Prevention Between Ranging Recoveries 472
22.2 UWB RANGING 473
22.2.1 STS Index Management 473
22.3 UWB MODULE 474
23 REFERENCES 475
APPENDIX A. 477
A.1. Standard Transaction Cryptography Flow 477

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

A.2. Fast Transaction Cryptography Flow *480*
A.3. Standard Transaction Cryptography Flow (with Reader Intent for Fast Transaction) *482*
A.4. X.509 Schemes *486*
APPENDIX B. 491
B.1. Instance Configurations *491*
B.2. Certificate Field Formats *492*
APPENDIX C. 496
C.1. External CA Certificate *496*
C.2. Vehicle OEM Intermediate Certificate *497*
C.3. Vehicle OEM Key/Leaf Certificate *498*
C.4. Vehicle OEM Privacy Encryption Certificate *499*
C.5. Vehicle OEM Signature Verification Certificate *500*
APPENDIX D. OWNER PAIRING TEST VECTORS [TO BE UPDATED] 502
D.1. Derivation of z0, z1 with Scrypt *502*
D.2. Computation of w0, w1 ..... *502*
D.3. Computation of L *503*
D.4. Computation of X *503*
D.5. Computation of Y *504*
D.6. Computation of Z, V (by device) *505*
D.7. Computation of Z, V (by vehicle) *506*
D.8. Derivation of K, CK, SK *508*
D.9. Derivation of Evidence Keys K1, K2 ..... *509*
D.10. Computation of Evidences M1, M2 ..... *509*
D.11. Derivation of System Keys *510*
APPENDIX E. NFC-F SUPPORT [WCC1] 510
E.1. Introduction *510*
E.2. Device Requirement *511*
E.3. Preconditions *512*
E.4. Commands and responses *512*
E.5. Protocol Operation *514*
APPENDIX F. DIGITAL KEY FRAMEWORK API 517
F.1. Overview *517*
F.2. Functional Requirements *517*
APPENDIX G. 518
G.1. Ranging Session Parameter Tables [WCC3] *518*
G.2. Hopping Sequence *519*
G.3. Default Hopping Sequence: *519*
G.4. AES-Based Hopping Sequence: *519*
APPENDIX H. [WCC3] 519
APPENDIX I. [WCC3] 520
I.1. PulseShape 0x0 ..... *520*

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

I.2. PulseShape 0x2 521
APPENDIX J. [WCC3] 523
J.1. UWB Test Vector 523
J.2. UWB Test Vector without hopping 524
J.3. UWB Test Vector with continuous hopping 528

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

LIST OF FIGURES

Figure 2-1: Digital Key Architecture with Actors and Their Relationships 38
Figure 2-2: Device Functional Elements. 45
Figure 2-3: Domain Versions 49
Figure 2-4: D-VS Agreement for Key Tracking 52
Figure 2-5: Version Agreement after Device Software Update 54
Figure 2-6: Example – manageKey() usage based on D-VS agreed version 55
Figure 2-7: V-OD-FW Agreement after Vehicle SW Update 56
Figure 4-1: Applet Instance Layout 65
Figure 4-2: Friend Digital Key Structure and Entitlements Attestation 66
Figure 4-3: Mailboxes Read/Write Permissions 69
Figure 4-4: Owner Device Private Mailbox Signaling 70
Figure 4-5: Slot Identifier Bitmap Assignment 71
Figure 4-6: Owner Device Immobilizer Token Signaling 72
Figure 4-7: Slot Identifier List to Confidential Mailbox mapping 73
Figure 6-1: Owner Pairing NFC Exchanges 95
Figure 6-2: Owner Pairing Flow - Phase 0/1: Preparation/Initiation 96
Figure 6-3: Owner Pairing Flow - Phase 2: First NFC Session 98
Figure 6-4: SPAKE2+ Flow 100
Figure 6-5: Key Creation Data Transfer to Device 101
Figure 6-6: Key Creation Data Transfer to Device 102
Figure 6-7: Key Creation Info Retrieval by Vehicle 104
Figure 6-8: Owner Pairing Flow – Phase 3: Second NFC Session 111
Figure 6-9: Error Management of Different Phases in Owner Pairing 115
Figure 6-10: Owner Pairing Flow – Phase 4: Finalization 116
Figure 6-11: Vehicle and Device Transaction Timeout 119
Figure 7-1: Standard Transaction Flow 122
Figure 8-1: Fast Transaction Flow 123
Figure 10-1: Check Presence Transaction 127
Figure 13-1: REV_100: Friend Key Termination in Vehicle 168
Figure 13-2: REV_100a: Friend Key Termination in Vehicle (Vehicle Required to be Online) 169
Figure 13-3: REV_110/120: Friend Key Termination in Owner/Friend Vehicle OEM Account 170
Figure 13-4: REV_130/140: Friend Key Termination on Owner Device Natively/in Vehicle OEM App 171
Figure 13-5: REV_150: Friend Key Termination Based on Expiry Date of the Key 172
Figure 13-7: REV_160: Friend Key Termination by Device OEM (Device Security Issue) 174
Figure 13-8: REV_160a: Friend Key termination by Device OEM and Friend Device is Offline 175
Figure 13-9: REV_170: Friend Key Termination Due to Remote Wipe of Device 176
Figure 13-10: REV_200/210: Friend Key Termination on Friend Device Natively/in Vehicle OEM App 177
Figure 13-11: REV_220: Friend Key Termination Due to Local Wipe of Device 178
Figure 13-12: REV_300: Friend Key Suspension by Device OEM Account 179
Figure 13-13: REV_310: Friend Key Resume in Device OEM Account or on Device 180
Figure 13-14: REV_310: Friend Key Resume Using ResumeAttestation 180

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-15: REV_400: Owner Key Deletion in Vehicle UI (Change of Device) 182
Figure 13-16: REV_400a: Owner Key Deletion in Vehicle UI (Change of Device) 183
Figure 13-17: REV_600: Owner key suspension due to device reported lost/stolen 184
Figure 13-18: REV_610: Owner key resumption after device reported lost/stolen 185
Figure 13-19: REV_700: Unpairing in Vehicle UI (Sale of Device) 186
Figure 13-20: REV_700a: Unpairing in Vehicle UI (Vehicle Required to be Online) 187
Figure 13-21: REV_710: Unpairing in Vehicle OEM App on Owner Device 188
Figure 13-22: Example Step 1: After Owner Pairing 190
Figure 13-23: Example Step 2: Device Fully Refilled 191
Figure 13-24: Example Step 3: Key Shared with Friend 191
Figure 13-25: Example Step 4: Refill of Shared Slot Identifier 192
Figure 13-26: Example Step 5a: Key Termination in Vehicle 192
Figure 13-27: Example Step 5b: Key Termination in Device 193
Figure 14-1: Message Authentication and Privacy Encryption 194
Figure 14-2: Authentication and Privacy Encryption Certificate Chain 196
Figure 15-1: Authentication Command Flows Over Contactless Interface 206
Figure 15-2: Authentication Command Flows Over Wired Interface 206
Figure 16-1: Variant 1 Certification Chain Model 274
Figure 16-2: Variant 2 Certification Chain Model 275
Figure 19-1: Bluetooth LE Link Layer Connection Establishment. 341
Figure 19-2: L2CAP Connection-Oriented Channel. 341
Figure 19-3: Bluetooth LE Pairing and Encryption Setup. 343
Figure 19-4: Passive Entry Flow Diagram. 353
Figure 19-5: Connection performance in relationship with device, transmitter, and receiver requirements. 355
Figure 19-7: Synchronization Methods. 391
Figure 19-8: Synchronization state between device and anchor. 392
Figure 19-9: Time Synchronization Uncertainty Flow Diagram. 394
Figure 19-10: Successful Bluetooth LE timeSync at BT connection (Procedure 0). 395
Figure 19-11: Successful Bluetooth LE timeSync triggered by vehicle (Procedure 1). 396
Figure 19-12: Unsuccessful Timesync message. 396
Figure 19-13: Bluetooth LE Timesync message flow example 1. 397
Figure 19-14: Time Synchronization flow diagram example 2. 398
Figure 19-15: Bluetooth LE Time Sync message flow example 3. 399
Figure 19-16: Owner Pairing Flow for Bluetooth LE/UWB. 401
Figure 19-17: Bluetooth LE Secure OOB Pairing Prep. 404
Figure 19-18: Capability Exchange. 406
Figure 19-20: Head Unit Pairing Flow Diagram. 409
Figure 19-21: URSK Derivation Flow in a dedicated Standard Transaction 412
Figure 19-22: Flow Selection for Establishing Secure Ranging 414
Figure 19-23: Secure Ranging Setup Flow. 415
Figure 19-24: Sub-Optimal Flow. 416
Figure 19-25: Ranging Session State Machine. 417
Figure 19-26: Ranging Suspend Accepted Flow. 417
Figure 19-27: Ranging Suspend Delayed Flow. 418

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-28: Ranging Recovery Flow. 419
Figure 19-29: Standard transaction over Bluetooth LE. 420
Figure 19-31: Friend First Approach Flow. 423
Figure 19-32: RKE Flow for an Event-based RKE Action. 425
Figure 19-33: RKE Flow for an Enduring RKE action. 427
Figure 19-34: Vehicle Function Status Retrieval and Event-based Update. 429
Figure 19-35: First Approach Activation. 430
Figure 19-36: Device Preference Management. 431
Figure 19-37: Required Capability Exchanged. 432
Figure 19-38: URSK Not Found. 433
Figure 19-40: Recovery Failed Flow. 435
Figure 19-41: URSK Refresh due to Status Word 6484_h in CREATE RANGING KEY Response. 436
Figure 20-1: Digital Key RAN Concept. 438
Figure 20-2: Digital Key One-to-Many Ranging Protocol. 440
Figure 20-3: Overview of MAC Grid (See Table G- 1). 443
Figure 20-4: Overview of Block Synchronization Approach. 445
Figure 20-5: Example of UWB Message Flow for MAC Protocol. 447
Figure 20-6: Example of MAC Parameter Negotiation and Setting. 455
Figure 20-7: SP0 Packet Content. 456
Figure 20-8: Basic Principles of STS Incrementation. 459
Figure 21-1: Block diagram for IEEE 802.15.4z HRP UWB PHY. 461
Figure 21-2: SP3 462
Figure 21-3: SP0 462
Figure 21-4: PHR bit assignment. 464
Figure 21-5: Data Field Encoding for Standard PHY. 465
Figure 22-1: Overview of the KDF used and its relationship. 469

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

LIST OF TABLES

Table 4-1: Signaling Bitmap Decoding 71
Table 4-2: Private Mailbox Content 73
Table 4-3: Confidential Mailbox Content 75
Table 4-4: Mailbox Access Rights 76
Table 5-1: Owner Pairing Command Set 77
Table 5-2: Generic Status Words 77
Table 5-3: Response to SELECT Command 78
Table 5-4: SPAKE2+ REQUEST Command Fields 79
Table 5-5: SPAKE2+ REQUEST Response Fields 79
Table 5-6: SPAKE2+_REQUEST Response Error Status Words 81
Table 5-7: SPAKE2+ VERIFY Command Fields 81
Table 5-8: SPAKE2+ VERIFY Response Fields 82
Table 5-9: SPAKE2+ VERIFY Response Error Status Words 82
Table 5-10: WRITE DATA Response Error Status Words 83
Table 5-11: Objects for Digital Key Creation 83
Table 5-12: Objects for Device Digital Key Certificate 84
Table 5-13: Mailbox Mapping 86
Table 5-14: Device Configuration Data 87
Table 5-15: GET DATA Response Error Status Words 89
Table 5-16: GET DATA Response Decrypted Payload for tag $7F20_h$ 90
Table 5-17: GET DATA Response Decrypted Payload for tag $7F22_h$ 90
Table 5-18: GET DATA Response Decrypted Payload for tag $7F24_h$ 90
Table 5-19: GET DATA Response Decrypted Payload for Tag $D3_h$ 90
Table 5-20: GET RESPONSE Response Error Status Words 91
Table 5-21: OP CONTROL FLOW P1 Parameters 92
Table 5-22: OP CONTROL FLOW P2 Parameters for P1=10_h (continue) 92
Table 5-23: OP CONTROL FLOW P2 Parameters for P1=11 (end with success) 92
Table 5-24: OP CONTROL FLOW P2 Parameters for P1=12 (end with failure) 92
Table 5-25: Certificates 93
Table 6-1: Error SW Condition List 1. 108
Table 6-2: Owner Key Tracking Request 112
Table 6-3: Owner Key Tracking Response Parameters 112
Table 6-4: Recommended Minimum Vehicle and Device Timeout Values 119
Table 6-5: Pairing Password URL Syntax 120
Table 9-1: AUTH0 Command Transaction Type Coding 124
Table 9-2: User Authentication Policies 125
Table 11-4: Key Configuration 139
Table 11-6: Key Creation Request 140
Table 11-7: Key Signing Request 143
Table 11-9: Error Code Message 145
Table 11-10: Error Codes 145

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 11-12: External CA Certificate Container 149
Table 11-13: Instance CA Certificate Container 149
Table 11-14: Endpoint Certificate Container 149
Table 11-16: Import Request 150
Table 11-18: Attestation Package 153
Table 11-19: Key Tracking and Online Attestation Delivery Request 157
Table 11-20: Key Tracking Response 158
Table 11-21: Vehicle Attestation Data Fields 159
Table 11-22: Vehicle Attestation 159
Table 13-1: Vehicle-triggered Key Termination 164
Table 13-2: Device-triggered Key Termination 164
Table 13-3: Vehicle OEM-triggered Key Termination 164
Table 13-4: Device OEM-triggered Key Termination 165
Table 13-6:Detailed Key Termination Use Cases 166
Table 13-7: Resume Attestation 181
Table 14-1: Authentication and Privacy Keys 195
Table 14-2: Remote Termination Request Data Fields 198
Table 14-3: Remote Termination Request Owner Device 198
Table 14-4: Remote Termination Request from Vehicle OEM Server 199
Table 14-5: OEM App Data Attestation Input 201
Table 15-1: Command Availability on Interfaces 204
Table 15-2: Generic Status Words 208
Table 15-3: Coding of the Different Ranges of Class Byte Values 209
Table 15-4: INSTALL for INSTALL Content of Tag C9 209
Table 15-5: Applet Implementation Options 210
Table 15-6: Values for Protocol Data Type A (tag ‘86’) 211
Table 15-7: Values for Protocol Data Type B (tag 87_h) 212
Table 15-9: Coding of the notification 213
Table 15-10: Notification Context 213
Table 15-11: Value and Presence of the Different Fields in the HCI Notification Event 213
Table 15-12: SELECT Response Fields 216
Table 15-13: Endpoint Configuration 216
Table 15-14: Setting of Tag 46_h and Tag 47_h by vehicle for various wireless capability combinations. 218
Table 15-15: Setting of Tag 46_h and Tag 47_h by the owner device for the key sharing. 219
Table 15-16: Endpoint verification information 220
Table 15-17: Endpoint Termination Request 224
Table 15-18: Endpoint Termination Attestation Data Fields 225
Table 15-19: Endpoint Termination Attestation 225
Table 15-20: Deletion Request 226
Table 15-21: AUTHORIZE ENDPOINT Command Payload 227
Table 15-22: AUTHORIZE ENDPOINT Internal Buffer Content Before Processing (offset 0) 231
Table 15-23: AUTHORIZE ENDPOINT Attestation Data Fields 231
Table 15-24: AUTHORIZE ENDPOINT Internal Buffer Content After Processing (offset 0) 231
Table 15-25: View Parameters 233

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-26: View Endpoint Identifier Response in Internal Buffer 234
Table 15-27: View Instance CA Response in Internal Buffer 234
Table 15-28: AUTH0 P1, P2 Parameters 235
Table 15-29: AUTH0 Command Payload 235
Table 15-30: AUTH0 Response Payload 236
Table 15-31: AUTH1 Vehicle Authentication Data Fields 237
Table 15-32: AUTH1 Command Payload 237
Table 15-33: AUTH1 Response Payload Before Encryption 238
Table 15-35: PRESENCE0 Response Payload 239
Table 15-36: PRESENCE1 Command Payload 240
Table 15-37: PRESENCE1 Vehicle Authentication Data Fields 240
Table 15-38: PRESENCE1 Response Payload Before Encryption 240
Table 15-39: READ BUFFER Command Payload for Format 2 241
Table 15-40: Exchange Command Decrypted Payload 243
Table 15-41: Exchange Response Decrypted Payload 244
Table 15-42: CONTROL FLOW P1 Parameters 245
Table 15-43: CONTROL FLOW P2 Parameters 245
Table 15-44: CONTROL FLOW P1/P2 Values for Applet 246
Table 15-45: CREATE ENCRYPTION KEY Command Payload 247
Table 15-46: Encryption Key Attestation Data Fields 247
Table 15-47: Encryption Key Attestation 247
Table 15-48: GET PRIVATE DATA Command Payload 248
Table 15-49: SET PRIVATE DATA Command Payload 249
Table 15-50: SET CONFIDENTIAL DATA Command Payload 250
Table 15-51: SET CONFIDENTIAL DATA Internal Buffer Content Before Processing 250
Table 15-52: SETUP ENDPOINT Command Payload 251
Table 15-53: SETUP INSTANCE Command Payload 253
Table 15-54: SIGN Command Payload 253
Table 15-55: SIGN Command Payload 254
Table 15-56: SIGN Data Fields 254
Table 15-57: SIGN Response 254
Table 15-58: MANAGE UA Command Payload 255
Table 15-59: Delete Ranging Keys Request. 256
Table 15-60: Receiver Endpoint Key Attestation Signed by Sender 268
Table 15-61: Arbitrary Data Attestation 270
Table 19-1: AdvA field of ADV_IND. 345
Table 19-2: AdvData field of ADV_IND. 345
Table 19-3: Definition of IntentConfiguration byte. 345
Table 19-4: Mandatory fields in Pairing Request. 345
Table 19-5: Mandatory fields in Pairing Response. 346
Table 19-6: DK Service UUID. 346
Table 19-7: SPSM Characteristic declaration. 346
Table 19-8: SPSM Characteristic value declaration. 347
Table 19-19: DK Message Format 356

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-20: Message Header definition. 356
Table 19-21: Message Type definition. 356
Table 19-22: Payload Header definition. 357
Table 19-23: Message Type and its associated messages. 357
Table 19-24: Ranging_Capability_RQ message and its parameters. 359
Table 19-25: Definition of the parameters for Ranging_Capability_RQ. 359
Table 19-26: Ranging_Capability_RS message and its parameters. 359
Table 19-27: Definition of the parameters for Ranging_Capability_RS. 360
Table 19-28: Ranging_Session_RQ message and its parameters. 360
Table 19-29: Definition of the parameters for Ranging_Session_RQ. 360
Table 19-30: Ranging_Session_RS message and its parameters. 361
Table 19-31: Definition of the parameters for Ranging_Session_RS. 361
Table 19-32: Ranging_Session_Setup_RQ message and its parameters. 362
Table 19-33: Definition of the parameters for Ranging_Session_Setup_RQ. 363
Table 19-34: Ranging_Session_Setup_RS message and its parameters. 364
Table 19-35: Definition of the parameters for Ranging_Session_Setup_RS. 364
Table 19-36: Ranging_Suspend_RQ message and its parameter. 364
Table 19-37: Definition of the parameter for Ranging_Suspend_RQ. 364
Table 19-38: Ranging_Suspend_RS message and its parameter. 364
Table 19-39: Definition of the parameter for Ranging_Suspend_RS. 365
Table 19-40: Ranging_Recovery_RQ message and its parameter. 365
Table 19-41: Definition of the parameter for Ranging_Recovery_RQ. 365
Table 19-42:Ranging_Recovery_RS message and its parameter. 365
Table 19-43: Definition of the parameter for Ranging_Recovery_RS. 365
Table 19-44: Configurable_Ranging_Recovery_RQ message and its parameter. 366
Table 19-45: Definition of the parameter for Configurable_Ranging_Recovery_RQ. 366
Table 19-46: Configurable_Ranging_Recovery_Response message and its parameter. 366
Table 19-47: Definition of the parameter for Configurable_Ranging_Recovery_Response. 366
Table 19-48: DK_APDU_RQ message and its parameter. 367
Table 19-49: Definition of the parameter for DK_APDU_RQ. 367
Table 19-50: DK_APDU_RS message and its parameter. 367
Table 19-51: Definition of the parameter for DK_APDU_RS. 367
Table 19-52: Time_Sync message and its parameters. 367
Table 19-53: Definition of the parameter for Time_Sync. 368
Table 19-54: First_Approach_RQ message and its parameters. 369
Table 19-55: Definition of the parameter for First_Approach_RQ. 369
Table 19-56: First_Approach_RS message and its parameters. 370
Table 19-57: Definition of the parameter for First_Approach_RS. 370
Table 19-58: RKE_Auth_RQ message and its parameter. 370
Table 19-59: Definition of the parameter for RKE_Auth_RQ. 370
Table 19-60: RKE_Auth_RS message and its parameter. 370
Table 19-61: Pass_through message and its parameter. 371
Table 19-62: Definition of the parameter for Pass_through. 371
Table 19-63: HU_PP message and its parameters. 371

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-64: Definition of the parameter for HU_PP. 371
Table 19-65: HUP_RQ message and its parameters. 372
Table 19-66: Definition of the parameter for HUP_RQ. 372
Table 19-67: HUP_RS message and its parameters. 373
Table 19-68: Definition of the parameter for HUP_RS. 373
Table 19-69: DK Event Notification message and its parameters. 373
Table 19-70: Definition of the parameters for DK Event Notification. 373
Table 19-71: DK SubEvents Category and its parameters. 374
Table 19-72: Definition of Command_Status and its Command_Status codes. 374
Table 19-73: List of Command_Status for Command Complete SubEvent. 374
Table 19-74: Definition of Session_Status and its Session_Status codes. 376
Table 19-75: List of Session_Status for Ranging Session Status Changed SubEvent. 376
Table 19-76: Definition of DR_Intent and DR_Intent codes. 376
Table 19-77: List of DR_Intent for Device Ranging Intent SubEvent. 377
Table 19-78: List of Vehicle Status Changed SubEvent Tags. 378
Table 19-79: Definition of Function ids and their corresponding Action ids. 381
Table 19-80: Definition of Standardized Function/Execution Status Values to Indicate the (Temporary) Unavailability of Function/Execution or Errors. 384
Table 19-81: Definition of RKE Request SubEvent Templates. 385
Table 19-82: Definition of Headunit_Pairing_Status and its Session_Status. 386
Table 19-83: List of Session_Status for Head Unit SubEvent. 386
Table 19-86: 7F49 Template. 403
Table 19-87: URSK storage requirements per Digital Key endpoint. 412
Table 19-88: RKE allowed user authentication configuration. 424
Table 20-1: Example MAC Configuration. 444
Table 20-2: Mapping of Slots to UWB Ranging Packets. 448
Table 20-3: Pre-Poll Request Message and its parameters. 449
Table 20-4: The definition of parameters in the Pre-Poll Message. 449
Table 20-5: Final_Data Message and its parameters. 450
Table 20-6: The definition of parameters in the Final_Data Message. 451
Table 20-7: Ranging Status of a Responder. 452
Table 20-8: The Contents of the MHR Field. 456
Table 20-9: The Contents of the Frame Control Field. 456
Table 20-10: The Contents of the Auxiliary Security Header. 457
Table 20-11: The Contents of the Vendor Specific Header IE. 458
Table 21-1: Supported UWB Configurations. 463
Table 21-2: The Mandatory Length-127 Ternary Code Sequences. 463
Table 21-3: Summary of pulse shapes. 466
Table 21-4: Overview of PulseShape_Combo values and the associated transmit pulse shapes. 466
Table A-1: Issuer and Verifier Rules Common to External CA, Instance CA and Endpoint 489
Table A-2: Issuer and Verifier Rules For Extensions Common to External CA, Instance CA and Endpoint 490
Table A-3: Issuer and Verifier Rules For Extensions Specific to External CA 490
Table A-4: Issuer and Verifier Rules For Extensions Specific to Instance CA 490
Table A-5: Issuer and Verifier Rules For Extensions Specific to Endpoint 491

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table A-6: Issuer and Verifier Rules for Vehicle OEM Root CA certificate 491
Table A-7: Issuer and Verifier Rules for Vehicle Public Key Certificate 491
Table E-1: ENCAPSULATION_CMD format....513
Table E-2: Format of the P1 field....513
Table E-3: ENCAPSULATION_RSP format....513
Table E-4: Format of the P1 field....514
Table E-5: SENSF_RES format 515
Table E-6: Format of RWTI 516
Table G-1: Number of Valid Ranging Rounds per 96 ms. 518

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

LIST OF LISTINGS

Listing 5-1: Vehicle Certificate Extension Schema....84
Listing 5-2: Vehicle Certificate Extension Data....85
Listing 5-3: Vehicle Public Key Certificate Data....85
Listing 11-3: Sharing Invitation....147
Listing 11-4: Key Creation Processing....149
Listing 11-6: Import Request....152
Listing 11-7: Friend, Endpoint Data Import Processing....154
Listing 11-8: First Friend Transaction....155
Listing 11-9: Key Tracking Receipt....158
Listing 11-10: setPrivateMailboxBit....159
Listing 11-11: clearPrivateMailboxBit....159
Listing 11-12: clearPrivateMailboxBytes....160
Listing 11-13:Generate Attestation Signature....160
Listing 11-14: Generate Sharing Password....160
Listing 14-1: Vehicle OEM Privacy Encryption Certificate Extension Schema....197
Listing 14-2: Vehicle OEM Privacy Encryption Certificate Extension Data....197
Listing 14-3: Vehicle OEM Signature Certificate Extension Schema....197
Listing 14-4: Vehicle OEM Signature Certificate Extension Data....197
Listing 14-5: OEM App Data Attestation Processing....201
Listing 15-1: INSTALL for INSTALL Processing....211
Listing 15-2: SELECT Processing....216
Listing 15-3: Endpoint Certificate Extension Schema....220
Listing 15-4: Endpoint Certificate Extension Data....221
Listing 15-5: Endpoint Certificate Data....222
Listing 15-6: CREATE ENDPOINT Processing....223
Listing 15-7: TERMINATE ENDPOINT Processing for Command Format 1....225
Listing 15-8: TERMINATE ENDPOINT Processing for Command Format 2....225
Listing 15-9: DELETE ENDPOINT Processing....226
Listing 15-10: External CA Certificate Extension Schema....227
Listing 15-11: External CA Certificate Extension Data....227
Listing 15-12: External CA Certificate Basic Constraints Extension Data....227
Listing 15-13: External CA Certificate Data....227
Listing 15-14: Instance CA Certificate Extension Schema....229
Listing 15-15: Instance CA Certificate Extension Data....229
Listing 15-16: Instance CA Certificate Data....229
Listing 15-17: AUTHORIZE ENDPOINT Processing....232
Listing 15-18: VIEW Processing....234
Listing 15-19: AUTH0 Processing....236
Listing 15-20: AUTH1 Processing....238
Listing 15-21: PRESENCE0 Processing....239
Listing 15-22: PRESENCE1 Processing....240

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Listing 15-23: READ BUFFER Processing for Command Format 1 241
Listing 15-24: READ BUFFER Processing for Command Format 2 241
Listing 15-25: WRITE BUFFER Processing 242
Listing 15-26: EXCHANGE Processing 244
Listing 15-27: CONTROL FLOW Processing 246
Listing 15-28: CREATE ENCRYPTION KEY Processing 248
Listing 15-29: GET PRIVATE DATA Processing for Command Format 1 249
Listing 15-30: GET PRIVATE DATA Processing for Command Format 2 249
Listing 15-31: SET PRIVATE DATA Processing 249
Listing 15-32: SET CONFIDENTIAL DATA Processing 250
Listing 15-33: SETUP ENDPOINT Processing 252
Listing 15-34: SETUP INSTANCE Processing 253
Listing 15-35: SIGN Processing 254
Listing 15-36: onDeselect Event Processing 255
Listing 15-37: CREATE RANGING KEY processing 256
Listing 15-38: DELETE RANGING KEYS Processing. 257
Listing 15-39: Generate Random 257
Listing 15-40: Generate Key Pair 258
Listing 15-41: Generate Identifier 258
Listing 15-42: Generate Attestation Signature 258
Listing 15-43: Verify Attestation Signature 258
Listing 15-44: Compute Shared Key with Diffie-Hellman 258
Listing 15-45: Key Derivation 259
Listing 15-46: Secure Channel Command Decryption and Authentication 259
Listing 15-47: Secure Channel Response Encryption and Authentication 259
Listing 15-48: Derive KEenc, KEmac 259
Listing 15-49: Confidential Mailbox Encryption and Authentication 260
Listing 15-50: Compute DeviceCryptogram 260
Listing 15-51: framework.createInstance Processing 262
Listing 15-52: framework.getInstance Processing 262
Listing 15-53: framework.view Processing 263
Listing 15-54: framework.deleteInstance Processing 263
Listing 15-55: instance.view Processing 263
Listing 15-56: instance.createEndpoint Processing 264
Listing 15-57: instance.deleteEndpoint Processing 265
Listing 15-58: instance.getEndpoint Processing 265
Listing 15-59: instance.setParameters Processing 266
Listing 15-60: endpoint.setParameters Processing 266
Listing 15-61: endpoint.getCertificate Processing 267
Listing 15-62: endpoint.terminate Processing 267
Listing 15-63: endpoint.authorize Processing 268
Listing 15-64: endpoint.createEncryptionKey Processing 269
Listing 15-65: endpoint.setConfidentialData Processing 269
Listing 15-66: endpoint.getPrivateData Processing 269

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Listing 15-67: endpoint.setPrivateData Processing 270
Listing 15-68: endpoint.sign Processing 270
Listing 15-69: endpoint.auth0 Processing 271
Listing 15-70: endpoint.auth1 Processing 271
Listing 15-71: endpoint.exchange Processing 271
Listing 15-72: endpoint.createRangingKey processing. 272
Listing 17-1: Content of versionUpdate API on Vehicle Software Update 305
Listing 17-2: Content of versionUpdate API after Device Software Update 306
Listing 18-1: Server Password Generation 334
Listing 18-2: Vehicle-side Public Point Generation 334
Listing 18-3: Device-side Public Point Generation 334
Listing 18-4: Vehicle-side Computation of Shared Secret 335
Listing 18-5: Device-side Computation of Shared Secret 335
Listing 18-6: Derivation of Evidence Keys 335
Listing 18-7: Vehicle-side Computation of Evidence 335
Listing 18-8: Device-side Computation of Evidence 336
Listing 18-9: Derivation of System Keys 336
Listing 18-10: Secure Channel Command Encryption and Authentication 336
Listing 18-11: Secure Channel Response Encryption and Authentication 337
Listing A-1: Standard Transaction Cryptography Flow 477
Listing A-2: Fast Transaction Cryptography Flow 480
Listing A-3: Standard Transaction Cryptography Flow (with Reader Intent for Fast Transaction) 482
Listing A-4: X.509 v3 schema 486
Listing A-5: X.509 Key Usage extension 487
Listing A-6: X.509 Basic Constraints extension 488
Listing A-7: X.509 Authority Key Identifier extension 488
Listing A-8: X.509 Subject Key Identifier extension 488
Listing A-9: X.509 ECDSA Signature Format 488

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

ABBREVIATIONS AND ACRONYMS

AES	Advanced Encryption Standard
AGC	Automatic Gain Control
AID	Application Identifier
APDU	Application Protocol Data Unit
Bluetooth LE	Bluetooth Low Energy
BPM	Burst Position Modulation
BPSK	Binary Phase Shift Keying
CA	Certificate Authority
CASD	Controlling Authority Security Domain
CMAC	Cipher-based Message Authentication Code
CoC	Connection Oriented Channel over Bluetooth LE
DK	Digital Key
DRBG	Deterministic Random Bit Generator
dUDSK	Derived UWB Data Secret Key
dURSK	Derived URSK
ECC	Elliptic Curve Cryptography
ECDSA	Elliptic Curve Digital Signature Algorithm
ECIES	Elliptic Curve Integrated Encryption Scheme
ERDEV	Enhanced Ranging Device
GP	GlobalPlatform
HCE	Host Card Emulation; the NFC communication is routed to the framework instead of the SE
HCI	Host Controller Interface
HRP	High Rate Pulse Repetition Frequency
HTTPS	Hypertext Transfer Protocol Secure
ID&V	Identification and Verification (of User Identity)
KDF	Key Derivation Function
KML	Key Management Logic
KTS	Key Tracking Server, operated by Vehicle OEM, privacy protected
L2CAP	Logical Link Control and Adaptation Protocol
Lc	Length Command
Le	Length Expected
LL	Link Layer
LSB	Least Significant Byte
MAC	Message Authentication Code

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1	MFR	MAC Footer
2	MHR	MAC Header
3	MIC	Message Integrity Check
4	MITM	Man-in-the-Middle
5	MSB	Most Significant Byte
6	MSD	Message Sequence Diagram
7	mUPSK	Master UWB Privacy Secret Key
8	mURSK	Master URSK
9	NFC	Near Field Communication
10	NVM	Non-Volatile Memory
11	O2M	One-to-Many
12	OCSP	Online Certificate Status Protocol
13	OEM	Original Equipment Manufacturer
14	OOB	Out-of-Band
15	PAKE	Password Authenticated Key Exchange
16	PHR	Physical Header
17	PHY	Physical Layer
18	PRF	Pulse Repetition Frequency
19	PSDU	Physical Layer Service Data Unit
20	PSM	Protocol/Service Multiplexer
21	QoS	Quality of Service
22	RAN	Ranging Area Network
23	RDEV	Ranging-capable device
24	RFU	Reserved for Future Use
25	RKE	Remote Keyless Entry
26	RS	Reed Solomon
27	RTR	Remote Termination Request
28	TA	Termination Attestation
29	SCA	Side Channel Attack
30	SDS-TWR	Symmetric Double-Sided Two-Way Ranging
31	SE	Secure Element
32	SECDED	Single Error Correct Dual Error Detect
33	SFD	Start of Frame Delimiter
34	SHA	Secure Hash Algorithm
35	SHR	Synchronization Header
36	SP0	STS Packets type 0 (packets with payload and no STS
37	SP3	STS Packets type 3 (packets without PHR, MHR, or payload)
38	SPAKE2+	Simple Password Authenticated Key Exchange

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1	SPSM	Simplified Protocol/Service Multiplexer
2	STS	Scrambled Timestamp Sequence
3	SW	Status Word
4	SYNC	Synchronization Header
5	TLV	Tag Length Value
6	ToF	Time of Flight
7	TTL	Time to Live
8	UA	User Authentication
9	URL	Universal Resource Locator
10	URSK	UWB Ranging Secret Key
11	UWB	Ultra Wide Band

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

DEFINITIONS

Bluetooth Module	An entity managing the Bluetooth communication between device and vehicle as described in this specification. This entity may consist of one or more integrated circuits or may be part of a combo chip solution
Central	The master of the Bluetooth LE connection
Digital Key	Digital credentials used to authenticate access to the vehicle
Endpoint	Digital Key object in the applet
Initiator	An entity that starts the UWB ranging packet exchange by sending a first UWB POLL packet
Owner Pairing	Pairing of an Owner Device with a Vehicle
Peripheral	The slave in the Bluetooth LE connection.
Responder	An entity that responds to a UWB POLL packet
Shared Key	Digital Key shared with a friend device. It is used interchangeably with friend Digital Key
Slot identifier	Value stored in private mailbox to reference a key slot of a Digital Key
Vehicle	A vehicle implementing the Digital Key service
UWB Module	An entity managing the UWB ranging session as described in this specification. This entity may consist of one or more integrated circuits or may be part of a combo chip solution

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

NOTATIONS

Values like $A1_h$ or $FEED_h$ are hexadecimal values using the Most Significant Byte first convention (big-endian).

Values like 1101_b are binary values using Most Significant Bit first convention.

Command and response APDU bytes and buffers are implicitly represented in hexadecimal notation.

Other numerical values like 1234 are decimal representations.

Fields in parentheses (..) are optional or conditional.

The concatenation operation is represented by ||.

ASCII values are represented by “quotes.”

String values shall be UTF-8 encoded unless specified differently.

Strings containing URLs/URIs shall be encoded as per RFC 3986 [27] unless specified differently.

Strings containing date values shall follow ISO-8601[2] definition using date format yyyy-MM-dd'T'HH:mm:ss.SSSZ unless specified differently

Binary data in JSON shall be encoded in hex string format (e.g., “deadbeef”) unless specified differently.

In fields described as a sequence of bits, bit7 is the most significant bit, bit0 is the least significant bit.

The following table shows a list of the parameters/variables used in this specification for the purpose of the UWB MAC layer definition.

Parameter/Variable	Definition	Context
i	Ranging Block index	Ranging session
s	Ranging Round index	Ranging session
m	Slot index	Ranging session
k	Ranging Session index	Ranging session
N^{k}_{Round}	Number of ranging rounds in a ranging block	Ranging session
$Round_Idx^{k}(i)$	Ranging round index in ranging block i in the k-th ranging session that is used for the ranging exchange.	Ranging session
$N^{k}_{\text{Responder}}$	Number of responders	Ranging session
l	Index of responder	Ranging session

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter/Variable	Definition	Context
R_l	Responder with index l=0,1,…, $N^k_{\text{Responder}}$ - 1	Ranging session
UWB_{time0}	Initiator time reference	RAN Global
UWB^k_{time0}	Initiator time reference for the k-th ranging session (by default, this defines the beginning of ranging block 1)	Ranging session
$UWB^k_{time0}(i)$	Initiator time reference for ranging block i for the k-th ranging session	Ranging session
$UWB^k_{time0}(i, Round_Idx^k(i))$	Initiator time reference for ranging exchanges in ranging block i, round $Round_Idx^k(i)$ for the k-th ranging session	Ranging session
STS_Index0^k	First STS index of the k-th ranging session, as negotiated during ranging session setup	Ranging session
$STS_Index^k(i)$	First STS index of ranging block i for the k-th ranging session	Ranging session
$STS_Index^k(i,s)$	First STS index of ranging round s of ranging block i for the k-th ranging session	Ranging session
$STS_Index^k(i,s,TYPE)$	STS index of the slot whose type is TYPE in ranging round s of ranging block i for the k-th ranging session	Ranging session
T^k_{Round}	Ranging round duration	Ranging session
T^k_{Block}	Ranging block duration	Ranging session
$N^k_{\text{Slot_per_Round}}$	Number of slots in a ranging round	Ranging session

1
2
3

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

CODE LISTINGS

The pseudo-code listings presented in this document highlight the important processing steps from a functional perspective. These pseudo-code listings do not represent any specific implementation, but actual implementations shall be functionally equivalent. Operations like basic input validation, length checking, input checking, option checking, or security counter-measures may not be described. Memory or speed optimizations are not described in these listings unless explicitly mentioned.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1 INTRODUCTION AND SCOPE

The Digital Key Technical Specification Release 3 specifies a Digital Key ecosystem using a standardized Digital Key applet, standardized vehicle access protocol, and a scalable architecture to support wide-scale deployment of the Digital Key services across different Vehicle OEMs and Device OEMs. Release 3 is backward compatible with Release 2. Release 3 is not backward compatible with Release 1. Release 1 can be deployed independently of Release 2 or 3.

The Digital Key Technical Specification Release 3 is based on the use of BLE/UWB or NFC as an underlying radio technology to enable Digital Key services. The Digital Key management framework is designed to be radio technologies agnostic, enabling the framework to be extended to support other technologies.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2 SYSTEM ARCHITECTURE

With the exception of Section 2.10, which is normative, this is an informational section, which provides an overview of the features, components, architecture, and operations of the Digital Key system.

2.1 Overview

The proposed system uses asymmetric cryptography to mutually authenticate vehicle and device. The device reveals its identity only to known vehicles.

Public keys are mutually exchanged through pairing of the owner device to the vehicle. The owner can then authorize the use of Digital Keys by friends and family members by signing their public keys.

The system is designed to work fully offline (i.e., no server connection is needed for a vehicle or a device) for all relevant features at the time of execution, such as the owner pairing or lock/unlock vehicle using the Digital Key. Where regulatory or business constraints require online connections, these can be added to the system.

2.2 High-Level Features

The described system complies with the following high-level requirements:

- Security and privacy equivalent to, or better than, physical keys
- Pairing of owner device and vehicle
- Key sharing between owner and friends
- Interoperability across devices and Vehicle OEMs
- Support of multiple Digital Keys from different Vehicle OEMs on a single device
- Enabling of Vehicle OEM to control the issuance of Digital Key and its policy
- Privacy protection against active/passive eavesdroppers

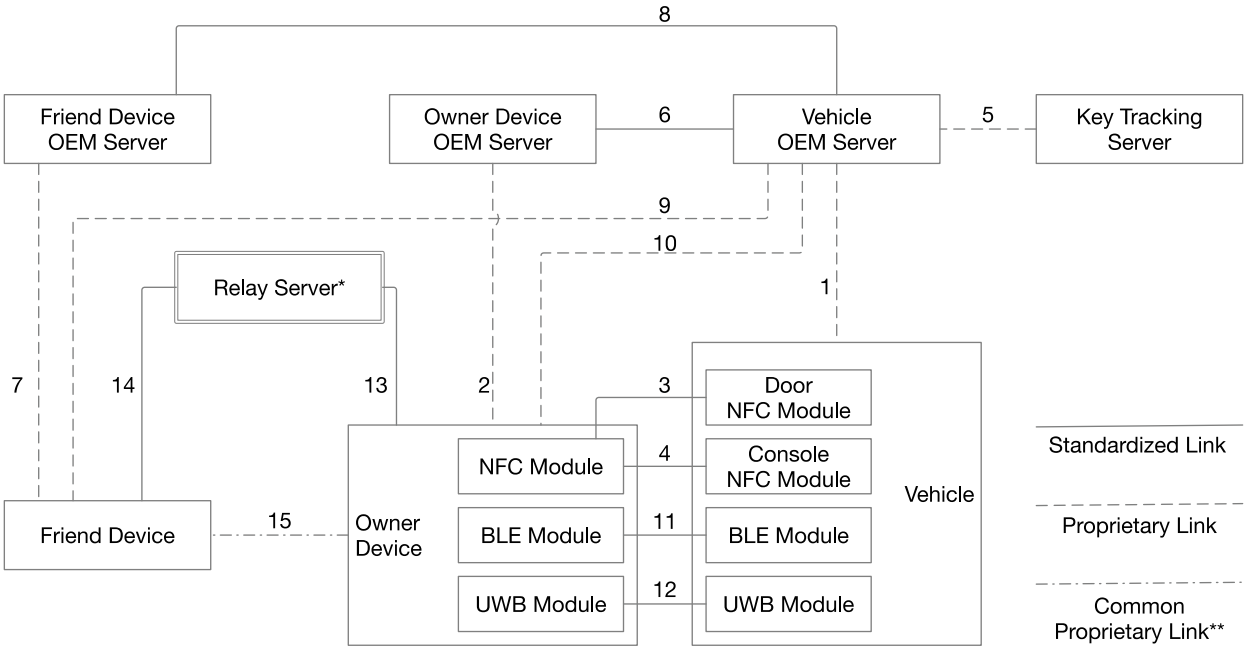
2.3 High level architecture

The Digital Key ecosystem consists of multiple actors that are connected to each other using a combination of standard and proprietary links as shown in Figure 2-1. The standardized links are fully specified in this specification to enable implementation and interoperability. The roles and responsibilities of each entity and their relationships are described in Sections 2.4 and 2.5.

Note: The entities and the proprietary links (dashed lines in Figure 2-1) that are out of scope of this specification are also described to provide an overview of the high-level functionalities of the system.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 2-1: Digital Key Architecture with Actors and Their Relationships



* Relay Server Selected by Owner Device OEM to Share keys with Friend Device from a different Device OEM. Relay Server may be implemented by Device OEM, Vehicle OEM or by a Third Party

** Common Proprietary link is a link that both Owner and Friend device can use for communication. E.g.: iMessage, WhatsApp or Other

In the system, the **vehicle** is linked to the **Vehicle OEM Server** per Telematics Link **(1)**. This link provides a secure communication channel and is fully controlled by the Vehicle OEM.

The **vehicle** is equipped with **NFC Readers** and optional **Bluetooth/UWB modules** to communicate with the devices for owner pairing, vehicle locking/unlocking, and engine start via links (**3**), (**4**) in the case of NFC connectivity, or links (**11**), (**12**) in the case of BLE/UWB connectivity.

The **owner device** communicates with the **owner Device OEM Server** using a proprietary method **(2)**. The **owner device** can also communicate directly with the **Vehicle OEM Server** using a proprietary Vehicle OEM app interface **(10)**. Similarly, the **friend device** can communicate directly with the **Vehicle OEM Server** using a proprietary Vehicle OEM app interface **(9)**.

The (owner/friend) **Device OEM Server** is responsible for managing the life cycle of the Digital Key applet and to update necessary certificates in the owner/friend device via **(2)**/ **(7)**. It can provide services to suspend, restore, and wipe Digital Keys when a device is lost or stolen.

The **Vehicle OEM Server** hosts user accounts and manages ID&V. It also connects to an optional **Key Tracking Server (5)** to register all issued Digital Keys for a vehicle in such a way that privacy of the stored information is preserved.

The **owner Device OEM Server** is not directly connected to the other Device OEM Servers.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The **owner device** can share a Digital Key with the **friend device** (via **(2)**, **(6)**, **(8)** and **(7)**), defining the appropriate access profile and can terminate the shared Digital Key when it is no longer needed. Key sharing can be accomplished either via **(2)** and **(7)**, (most commonly when the Owner Device and Friend device are manufactured by the same OEM), or through a Relay Server (most commonly when the Owner Device and Friend Device are manufactured by different OEMs: via **(13)**, **(14)**). The invitation to obtain a shared key from the relay server shall be sent from the owner device to the friend device via **(15)**. The invitation may use a second factor authentication scheme where the friend proves its identity using schemes defined and required by either Device OEMs (such as Device PIN) or Vehicle OEMs (such as sharing password) as outlined in Section 11.2.2. The **friend device** can host Digital Keys shared by the owner but cannot share the same Digital Key with any other device.

The **friend device** and its **friend Device OEM Server** connection **(7)** support necessary certificate services, as in link **(2)**. Device OEM or Vehicle OEM can facilitate device change. However, this is out of scope.

The interface between the **Vehicle OEM Server** and (owner/friend) **Device OEM Server (6)**/ **(8)** is used to exchange the servers' certificates, key tracking, key termination, and notifications.

All eligible devices contain a certified SE as well as NFC capability to enable them to communicate with the vehicle.

2.4 Actors

In this section, the roles and responsibilities of the entities involved in the Digital Key ecosystem are described. The subsections within 2.4 describe the primary functions of the various actors.

2.4.1 *Vehicle*

- Determine if the owner/friend device is eligible for the Digital Key service before allowing owner pairing or accepting a friend key shared by the owner device.
- If key tracking is required, may provide owner pairing information (owner public key, device information, etc.) to the KTS or verify that owner pairing information was received by the KTS.
- Verify authenticity of the device.
- Authorize any device that proves that it has a valid Digital Key to access the vehicle, and if required by the vehicle, an immobilizer token to start the engine.
- If required, provide a user interface to delete the owner and friend Digital Keys.
- Provide secure processing and storage environment.

2.4.2 *Vehicle NFC Readers [WCC1]*

- Communicate with the owner device for owner pairing and Digital Key transactions (lock/unlock, engine start, etc.).
- Communicate with the friend device for Digital Key transactions.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2.4.3 *Vehicle Bluetooth LE Module [WCC2/WCC3]*

- Communicate with the owner device for owner pairing and Digital Key transactions (lock/unlock, engine start, RKE etc.).
- Communicate with the friend device for first friend transaction and Digital Key transactions
- Communicate with owner or friend device for setup of a secure ranging over UWB
- Communicate with owner or friend device for Remote transactions to allow the device to initiate on-demand features (e.g. Lock/Unlock etc.)
- Communicate with owner or friend device for transmission of Notifications to notify and signal change of state information
- Communicate with owner or friend device for transmission of data of 3rd party vehicle OEM application.

2.4.4 *Vehicle UWB Module [WCC3]*

- Communicate with owner or friend device for secure ranging to securely determine the distance between device and vehicle to support a secure distance measurement for passive entry and passive engine start functionality.

2.4.5 *Vehicle OEM Server*

- Host owner account that links to the owner's vehicle(s).
- Manage Digital Key service subscriptions.
- Sign a shared Digital Key structure for acceptance by vehicle, assuring that business policies are checked and Digital Keys are tracked.
- Provide necessary attestations to the vehicle (when online) so that shared friend Digital Keys are accepted by the vehicle in the first friend transaction.
- Terminate Digital Keys in the vehicle when they have been deleted on a device.
- Sync with the owner device for termination of Digital Keys that is done offline.
- Manage a secure channel to the vehicle.
- Create pairing passwords and provide them to owner device and vehicle.
- Sign vehicle public key.
- Provide necessary certificates to Device OEMs.
- Provide Vehicle OEM and (optionally) Device OEM public keys to vehicle for owner pairing and friend sharing.

2.4.6 *Key Tracking Server (KTS)*

- Record relevant data to be able to assign a tracked Digital Key for a vehicle to a device. The KTS is likely to be managed by the Vehicle OEM.

  The following are the key properties of the KTS:

  - Only accessed for data query when required for legal or insurance reasons or for transmission of friend key information during owner device change (see Section 13.5.1.1)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- Data separation from the Vehicle OEM Server to fulfill privacy requirements of tracked data

2.4.7 *Devices*

- Contain a secure processing and storage environment (SE or equivalent) running a Digital Key applet
- Can take on the role of an owner device and friend device
- Support contactless transactions to lock/unlock vehicle and start the engine
- Support configurable user authentication (e.g., passcode)
- Check service eligibility before allowing owner pairing or acceptance of a friend Digital Key

2.4.7.1 Owner Device

- Implement main features: transaction, owner pairing, Digital Key sharing (sender), and Digital Key termination
- Store necessary certificates for owner pairing and Digital Key sharing
- Terminate Shared Keys by sending termination request to vehicle (via Vehicle OEM Server) and to friend device via Device OEM Server and Vehicle OEM Server
- Provide a common proprietary link (15) from Figure 2-1 for Key Sharing

2.4.7.2 Friend Device

- Implement main features: transaction, Digital Key sharing (receiver), key termination
- Store necessary certificates for Digital Key sharing
- Send termination attestation to Vehicle OEM Server
- Provide a common proprietary link (15) from Figure 2-1 to accept shared keys

2.4.8 *Device OEM Server*

- Load and install the Digital Key instance of the Digital Key applet (if necessary)
- Provide and update necessary certificates in the device
- Allow Digital Key functionality to be temporarily disabled on a lost or stolen device (if online)
- Allow wipe of Digital Keys on a lost or stolen device (if online)

2.4.9 *Relay Server*

- Provides a standardized communication channel to support key sharing between two devices from different (or the same) OEMs. The standardized communication channel is outlined in the Secure Credential Transfer RFC [38].
- Implements push notifications and supports polling mechanisms to keep devices informed during the key sharing process.
- May be implemented by any entity and must be included in the CCC-approved Relay Server providers listed in [35]

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2.5 Relationships

In this section, the relationships between different entities are described. Links referred to in this section (in parentheses) are shown in Figure 2-1.

The following links are standardized according to the protocol, and relevant message formats are defined in this specification:

- **Owner/Friend Device**: Door NFC reader (link 3) as defined in Section 2.5.1
- **Owner/Friend Device**: Console NFC reader (link 4) as defined in Section 2.5.2
- **Owner/Friend Device OEM Server**: Vehicle OEM Server (links 6 and 8) as defined in Section 2.5.10.
- **Owner/Friend Device**: Bluetooth LE Interface as defined in Section 2.5.3.
- **Owner/Friend Device**: UWB Interface as defined in Section 2.5.4.
- **Relay server:** Owner and friend device (links 13 and 14) as defined in Section 11.3

All other links are out of scope of this specification and are described at a high level to provide an overview of the underlying links and their functionality. Standardized information may be transported across out-of-scope links.

2.5.1 *Door NFC Reader (3) [WCC1]*

- Conducts regular transactions (fast or standard) with all devices that have a valid and registered Digital Key
- Conducts first transaction for a friend device during Key Sharing, transmitting necessary attestations so that the vehicle can verify the Digital Key of the friend device
- Implements specific polling to allow automatic selection of the appropriate Digital Key

2.5.2 *Console NFC Reader (4) [WCC1]*

- Conducts (standard or fast) engine start transactions with all devices that have a valid and registered Digital Key
- Authorizes engine start
- Conducts owner pairing transaction with a device to enable it to become the owner device
- Conducts first transaction for a friend device in cases when the vehicle was offline during key sharing, transmitting necessary attestations so that the vehicle can verify the Digital Key of the friend device
- Implements specific polling to allow automatic selection of the appropriate Digital Key (see Section 2.10, NFC Interface)

2.5.3 *Bluetooth LE Interface (11) [WCC2/WCC3]*

- Conducts standard transactions with all devices that have a valid and registered Digital Key
- Conducts owner pairing transaction with a device to enable it to become the owner device
- Conducts first transaction for a friend device
- Conducts setup of the secure UWB ranging session

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- Conducts Remote transactions to allow the device to initiate on-demand features (e.g. Lock/Unlock etc.)
- Transmit Notifications to notify and signal change of state information
- Transmit data of 3rd party vehicle OEM application.

2.5.4 *UWB Interface (12) [WCC3]*

- Conducts secure ranging to securely determine the distance between device and vehicle to support a secure distance measurement for passive entry and passive engine start functionality.

2.5.5 *Owner-to-Friend Device Link (via (2), (6), (8), and (7))*

2.5.5.1 Owner-to-Friend Device Link (via (2), (6), (8), and (7))

- Owner device deletes Digital Keys from the friend device by sending termination and deletion commands
- Friend device sends termination attestation when a Digital Key on the device is terminated

2.5.5.2 Owner-to-Friend Device Link (via (13), (14) and (15))

- Owner device shares Digital Keys with friend device using sequence of communications links: (15), (13), (14)

2.5.6 *Owner or Friend Device to Vehicle OEM Server (10, 9)*

- Links 9 and 10 are realized through the Vehicle OEM app; see Section 2.6.7

2.5.7 *Telematics Link (1)*

- Vehicle OEM managed, proprietary, trusted, and confidentiality-preserving link. Key functions include:
  - Send owner pairing verifier and additional pairing information to the vehicle
  - Obtain signature of the vehicle public key from the Vehicle OEM CA
  - Send Digital Key termination information to delete a specific Digital Key in the vehicle
  - Notify Vehicle OEM Server about Digital Key deletion in vehicle to remove Digital Key from the friend device
  - Register owner Digital Keys with Vehicle OEM Server (KTS)

2.5.8 *Owner/Friend Device OEM Server Link (2,7)*

- Owner/Friend Device authenticates Device OEM Server in a device-specific way
- Device loads/installs Digital Key applet (if not done in factory)

2.5.9 *Vehicle OEM Server to KTS (5)*

- Provide key tracking data to KTS, such as
  - Public key of owner and friend devices

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- Instance CA identifier of hosting SE (owner or friend)
- Anonymized vehicle identifier

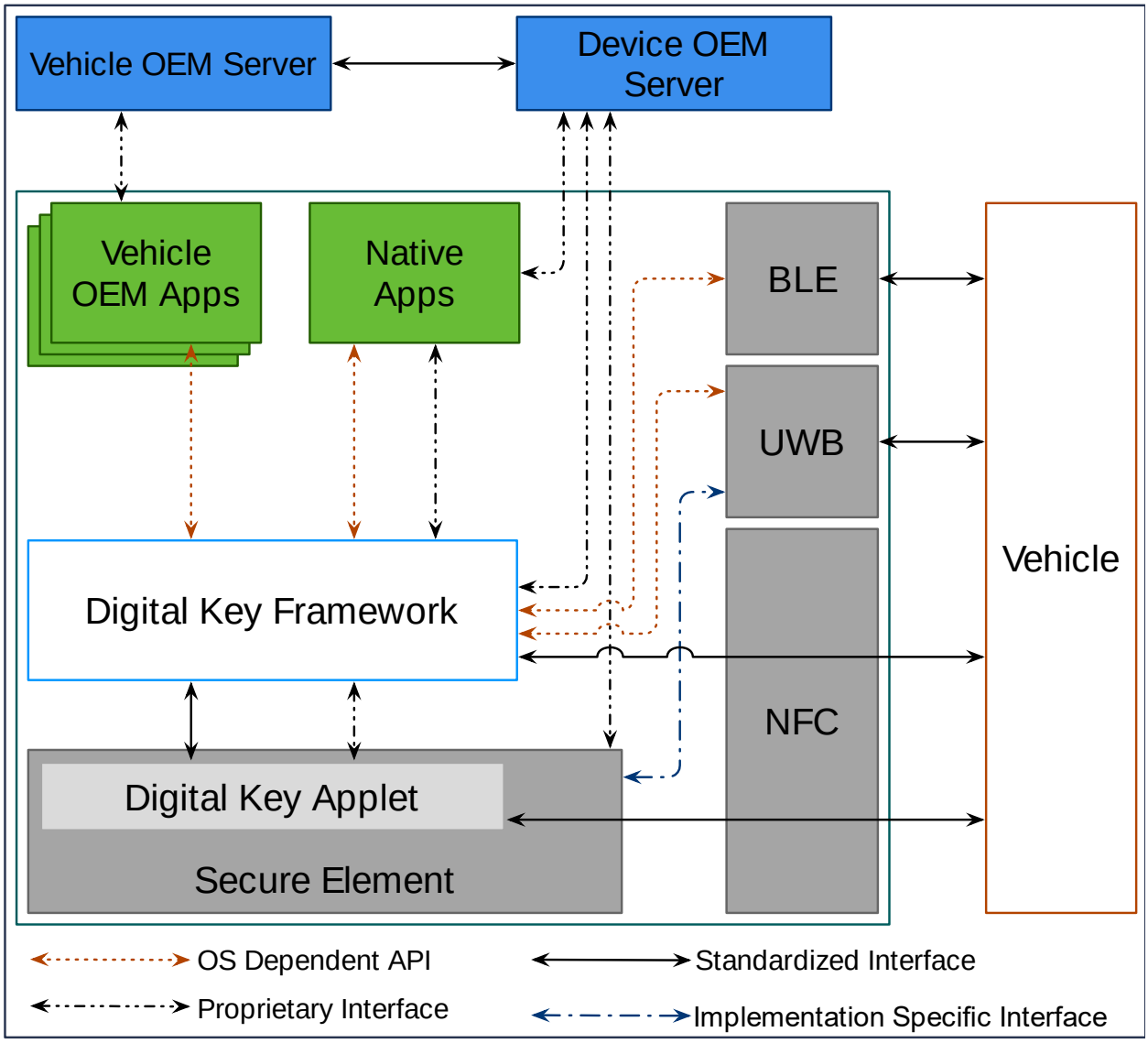
2.5.10 *Owner/Friend Device OEM Server to Vehicle OEM Server (6,8)*

- Establish trust relationship
- Exchange and sign the necessary certificates (see Section 16)
- Support Digital Key sharing
- Support Digital Key tracking
- Support Digital Key termination
- Support notifications

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2.6 Device Structure

Figure 2-2: Device Functional Elements.



On the device side, the following functional elements participate in the system: Secure Element and NFC controller are required components of eligible devices. The SE provides the root of trust, which is the starting point of the trust chain (see Section 16).

2.6.1 *NFC Component [WCC1]*

- Card emulation mode required for contactless transactions
- Host card emulation mode required for owner pairing

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2.6.2 *Bluetooth Module [WCC2/WCC3]*

- Communicate with the vehicle for owner pairing, first friend transaction and Digital Key transactions (lock/unlock, engine start, RKE etc.).
- Communicate with vehicle for setup of a secure ranging over UWB
- Communicate with vehicle for Remote transactions to allow the device to initiate on-demand features (e.g. Lock/Unlock etc.)
- Communicate with vehicle for transmission of Notifications to notify and signal change of state information
- Communicate with vehicle for transmission of data of 3rd party vehicle OEM application.

2.6.3 *UWB Module [WCC3]*

- Communicate with vehicle for secure ranging to securely determine the distance between device and vehicle to support a secure distance measurement for passive entry and passive engine start functionality.

2.6.4 *Secure Element (or equivalent)*

- Java Card (example) and GlobalPlatform [20] support, SE Root based on GP-defined Controlling Authority Security Domain available
- GlobalPlatform Card Specification Amendment C [20] and GlobalPlatform Contactless Extension [21]
- Standard symmetric and asymmetric cryptographic support
- Hosts Digital Key applet
- Ability to distinguish communication between wired interface and contactless interface

2.6.5 *Digital Key Applet*

The Digital Key applet provides the following services:

- Hosts Digital Keys (one applet instance hosts Digital Keys of all Vehicle OEMs)
- Implements relevant (fast and standard) transactions
- Implements Instance CA (see Section 4.1 and Section 16.2.3) to support offline use cases and privacy protection
- Stores immobilizer tokens when required by the vehicle, offline attestations, access profiles, and other data associated with a Digital Key
- Verifies authenticity of the vehicle
- Verifies certificate chain of friend public key

If the Digital Key applet is the SE-centric applet model as defined in Section 15.1, the applet also provides the following service:

- Verifies the Vehicle Public Key Certificate [K]

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2.6.6 *Digital Key Framework*

- Implements main features: owner pairing, Digital Key sharing, and management
- Provides common Digital Key service functionality via a set of OS-specific APIs for Vehicle OEM apps. The APIs are described in documentation of the respective OS platform developer. The functional requirements are defined in Appendix F.

2.6.7 *Vehicle OEM App*

- The Vehicle OEM app is optional. The main features of the app are supported natively by the device.
- May support the same features as the native app plus Vehicle OEM-specific features
- Provides ID&V with Vehicle OEM Server
- Retrieves owner pairing password
- Manages keys with non-standard access profiles

2.6.8 *Native App*

- Provides device-native UI such as Digital Key creation, Digital Key termination and deletion, Digital Key enable/disable, etc.
- Displays a list of all issued owner/friend Digital Keys

2.7 Vehicle States

The vehicle has the following possible internal states:

- **Unpaired:** State in which no owner device is associated with the vehicle. This occurs either at first purchase or when the owner chooses in the vehicle menu to unpair, which leads to the deletion of all Digital Keys (owner and friend).
- **Paired:** State in which an owner device is associated with the vehicle. This state is maintained when the owner deletes the associated owner device (e.g., to change the owner device) without deleting the friend devices (see Section 13.5).
- **Pairing**: The vehicle is in this state when waiting for an owner device to be associated.
- **Blocked**: The vehicle is in this state after a configurable number of failed pairing attempts (Vehicle OEM policy)

2.8 Digital Key User Classes

This section describes the different roles associated with Digital Keys. A single device can hold Digital Keys with different roles for multiple vehicles.

2.8.1 *Owner*

The vehicle accepts only one owner device. When an owner device is associated, the vehicle switches from the unpaired to paired state. The owner has all access rights to the vehicle. The owner's Digital Key may need to be registered in the KTS in order to be accepted by the vehicle.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2.8.2 *Friend*

The vehicle accepts several friend devices. Friend devices may have restricted access rights to the vehicle. These access rights are assigned by the owner using an access profile (see Section 2.9.2) when issuing the Digital Key and are checked by the vehicle and/or Vehicle OEM Server according to the Vehicle OEM policy. The friend Digital Key may need to be registered in the KTS in order to be accepted by the vehicle.

2.9 Access Profiles

For each device associated with the vehicle (owner device and one or more friend devices), the vehicle stores the access profile and corresponding public key.

2.9.1 *Owner*

The owner Digital Key has all access rights with no restrictions.

2.9.2 *Friend*

The owner may choose which profile to grant to the friend during key sharing.

The list of supported access profiles is defined in Section 11. As not all vehicles support all profiles, the vehicle indicates to the owner device which profiles are supported during owner pairing (see Section 6).

2.10 Versioning

2.10.1 *General*

Versioning is required so that devices, vehicles, and servers with different Digital Key specification releases, can still work together.

Version negotiation is used to agree on the highest version supported by all participants of a specific feature or relationship.

2.10.2 *Domain Versions*

Figure 2-3 identifies the domain versions that are relevant for the system.

Domain versions describe logical relationships, not physical APIs, between actors in the system.

All APIs are associated to a particular domain version. Data structures might be associated with one or multiple domain versions, depending on how many entities receive and process the data structure. Data structures can contain elements that are determined by a different domain version than the data structure layout itself. This specification documents those associations where reasonable, so as to allow quick identification of the impact that an API or data structure change has on the relevant domain versions. Some data structures do not list domain version information. In case of a change in those data structures, the relevant domain version(s) need(s) to be identified depending on the nature of the change.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

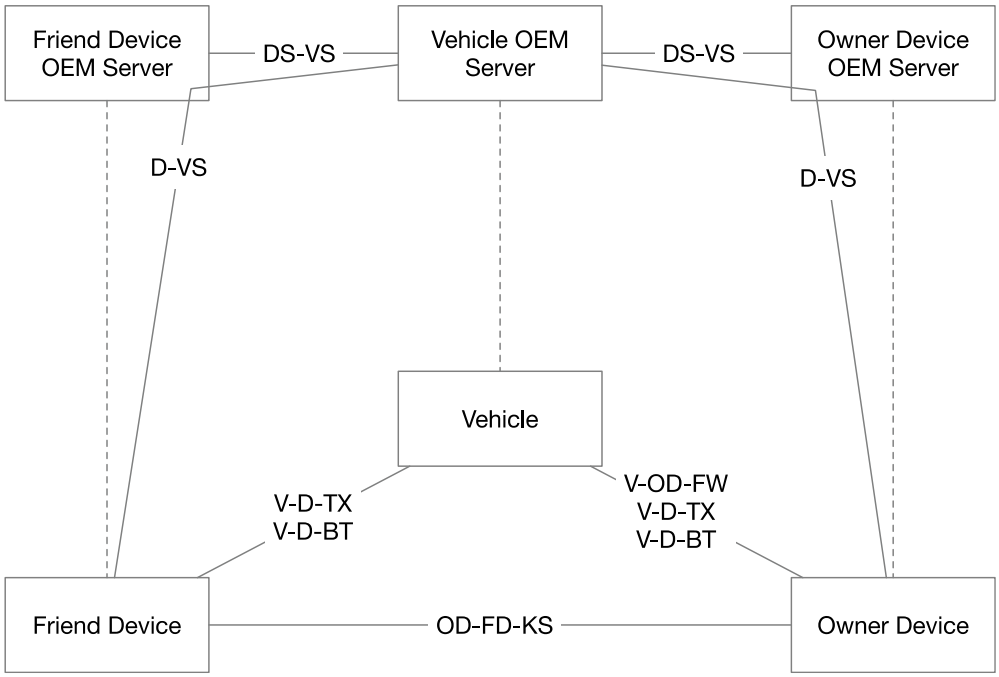
The domain version determining the data structure is provided in the first line (e.g., highest level TLV). Domain versions for data elements are provided with each element in the structure description.

Note that the APIs and data structures do not need to (but can) carry explicit version information, the knowledge of the API and data structure semantics and syntax are implicitly known by sender and receiver based on the associated domain version.

Note also that the relationships described by dashed lines in Figure 2-3 are out of scope of this specification and the representation of these relationships in any subsequent MSDs in this specification is for illustrative purposes only.

Domain versions always include a list of versions from both sides, for example, D-VS-serverList and D-VS-deviceList, or at least an agreed version from the responding side. When a commonly supported version is found through the exchange of lists, then this is the “agreed version”. Where relevant, this clarification is added throughout the specification.

Figure 2-3: Domain Versions



The vehicle-to- device (V-D) relationship is described by two domain versions.

1. (V-D-TX): Version of the transactions (fast, standard, etc.) between device and vehicle.
2. (V-D-BT): This domain version determines the Bluetooth (BT) version for Digital Key used between device and vehicle.

The vehicle-to-owner-device (V-OD) relationship is described by one additional domain version:

1. (V-OD-FW): Version of the framework relationship established at owner pairing and features based on this relationship.

The device-to-vehicle-server (D-VS) relationship and the device-server-to-vehicle-server relationship do not require a differentiation between owner and friend device or owner and friend device server. The versioning is managed independently of the key type.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The vehicle-side lists of V-D-TX, V-OD-FW and V-D-BT obtained by the owner are sent to the friend device during key sharing. This allows Friend device to determine if there will be an agreed V-D-TX and V-D-BT version to connect over Bluetooth LE and to transact with the vehicle. This allows to determine whether a shared key can be accepted or shall be rejected.

The V-D-TX and V-D-BT finally used in transactions between the friend device and vehicle may have different values than the V-D-TX and V-D-BT used in transactions between the owner device and vehicle.

Owner and friend device exchange sharing data to create friend keys. This data is partially generated and consumed by the framework and partially by the applet. Explicit applet domain version agreement is not required as the frameworks of both devices represent the applet and framework version in the key sharing domain version (OD-FD-KS).

Vehicle OEMs provide endpoints URLs to device OEMs to enable routing of device information to the region and environment in which the vehicle is registered based on the ROUTING_INFORMATION obtained during owner pairing and key sharing by the devices.

The vehicle OEM can define a D-VS and a DS-VS version per endpoint URL which allows vehicle servers to run on different SW releases per endpoint. See examples in Section 2.10.4

2.10.3 *Domain Wise Version Agreement*

2.10.3.1 V-OD-FW

The framework version is agreed between vehicle and owner device during owner pairing based on tags $5A_h$ and $5B_h$ in Table 5-3 and Table 5-4 respectively.

V-OD-FW-agreedVersion is provided during key tracking to the vehicle server (see Section 17.7.1.2)

When the device or vehicle SW is updated to support a new V-OD-FW version, then the updated version list is provided via the versionUpdate API (see section 17.7.6.4) to the other side.

2.10.3.2 V-D-TX

The transaction version is agreed between vehicle and owner device during owner pairing based on tags $5C_h$ (both sides) in Table 5-4.

The vehicle list V-D-TX-vehicleList is provided during key sharing in Tag $5C_h$ (DIGITAL_KEY_APPLET_PROTOCOL_VERSIONS) in Table 11-6. This allows the friend device to verify version compatibility with the vehicle before accepting a shared key.

Independent of the V-D-TX version agreed during owner pairing or key acceptance, all devices agree on a version during every fast and standard transaction. This allows updating vehicle or device SW to support higher versions without the need to transfer this knowledge via servers to the other side. See tags $5C_h$ in Table 15-29: AUTH0 Command Payload and Table 15-12: SELECT Response

2.10.3.3 V-D-BT

The Bluetooth version is agreed between vehicle and owner device during owner pairing based on tags $5E_h$ and $5F_h$ in Table 5-4 and Table 5-5 respectively.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The vehicle list V-D-BT-vehicleList is provided during key sharing in tag $5F_h$ in Table 11-6. This allows the friend device to verify version compatibility with the vehicle before accepting or rejecting a shared key.

Independent of the V-D-BT version agreed during owner pairing or key acceptance, all devices agree on a version during every BT connection. This allows updating vehicle or device SW to support higher versions without the need to transfer this knowledge via servers to the other side. The negotiation of versions under the V-D-BT domain during owner pairing and passive entry is described in detail in Section 19.2.1 and Section 19.2.3

2.10.3.4 OD-FD-KS

The key sharing version is agreed between the owner device and the friend device during key sharing based on tags 54_h and 55_h in Table 11-6 and Table 11-7.

The key creation request (Table 11-2) cannot be versioned and must be managed by adding new payloads with new tags if backwards-incompatible changes need to be done in the future. Backwards-compatible changes can be done by adding new tags, as they shall be ignored by earlier versions. It is strongly recommended that devices plan for enough buffer space to receive larger key creation request messages than defined in this version of the specification.

2.10.3.5 D-VS

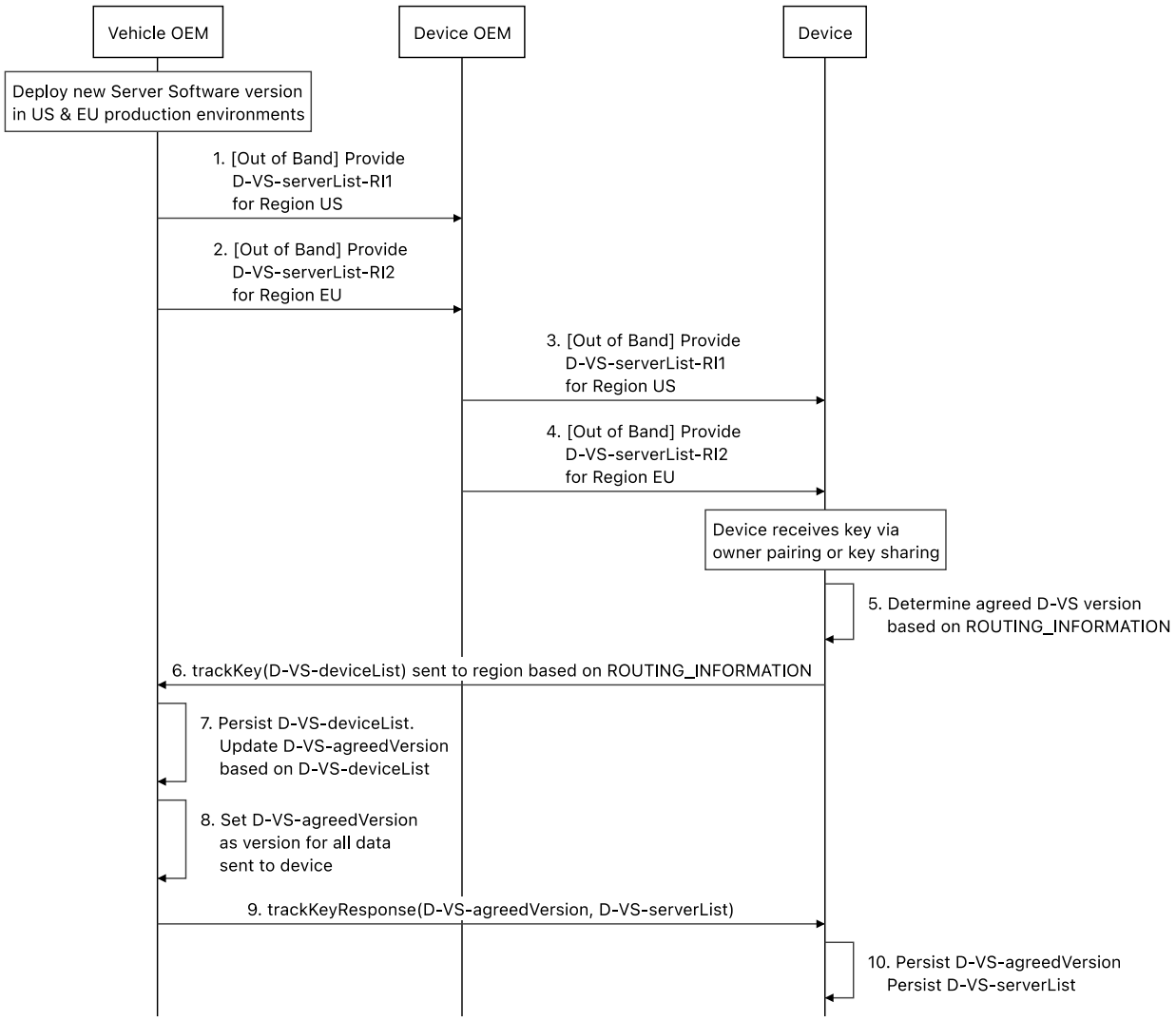
The vehicle server list D-VS-serverList is provided to the devices via a proprietary configuration method of the device OEM.

The device list D-VS-deviceList is provided during key tracking to the vehicle server (see Section 17.7.1.2). The versionUpdate API is not required for this domain version.

One D-VS-serverList can be provided per endpoint URL by the Vehicle OEM to the Device OEM, which corresponds to a specific value of the ROUTING_INFORMATION (RI) provided by the vehicle at owner paring and by the owner device at key sharing.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 2-4: D-VS Agreement for Key Tracking[1]



[1 – 2] VS deploys new SW version for some regions. When deployed, the VS provides the new supported versions to connected DS in a proprietary way (based on the working terms agreed between VS and DS) for each server endpoint identified by the routing information (RI) as per above example (e.g., OEM1.USP.BRD1).

[3 – 4] DS provides the updated version lists in a proprietary way to all devices.

[1] Out-of-band in this figure refers to mechanisms that are outside the scope of the Digital Key Specification. E-mail or other mechanisms may be used for OEM-to-OEM communication compliant with pre-established agreements between device and vehicle OEMs; or Device OEMs may use proprietary APIs to communicate with devices similar to the telematics links deployed by Vehicle OEMs to communicate with Vehicles.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

[5]	ROUTING_INFORMATION is used by the device to determine the agreed D-VS version to send the key tracking request based on e.g., D-VS-server-list-USP.
[6 – 7]	The key tracking request is sent by the device in the newly determined D-VS agreed version. The D-VS server list is determined using the routing information provided during owner pairing. The key tracking request provides the device list for the D-VS version to allow the server to determine the agreed D-VS version for communication with this device (e.g., the key tracking response or a manage key call). Step 6: Although not shown in Figure 2-4, the device also provides the list of supported V-OD-FW versions, including the agreed V-OD-FW version to the VS.
[8]	From this moment on, (including the trackKeyResponse message) the server sends all data in the new agreed D-VS version to this device. See Figure 2-6 as an example.
[9 – 10]	The key tracking response contains the server D-VS version list and the agreed version, which is persisted in the device and should override the relevant configuration that the device has obtained in steps 3 - 4.

2.10.3.6 DS-VS

The device server to vehicle server version is agreed between device OEM and vehicle OEM via a proprietary configuration method. The versionUpdate API is not required for this domain version.

2.10.4 *Software Updates*

A modification of the list of supported domain version values due to a SW update of one or more entities shall lead to a version agreement update between all entities impacted by the version change.

If required, each party needs to be able to handle temporarily different server versions on the other side on a per region basis. Therefore, the version information may need to be connected to a region-specific endpoint or server instance of the other OEM.

It is recommended that each OEM should always try rather to implement changes in a backwards compatible way, especially for server-side updates. Non-backwards compatible changes in server APIs should be avoided. The mechanism to inform OEM server or device counterparts is described in Sections 2.10.3.5 and 2.10.3.6.

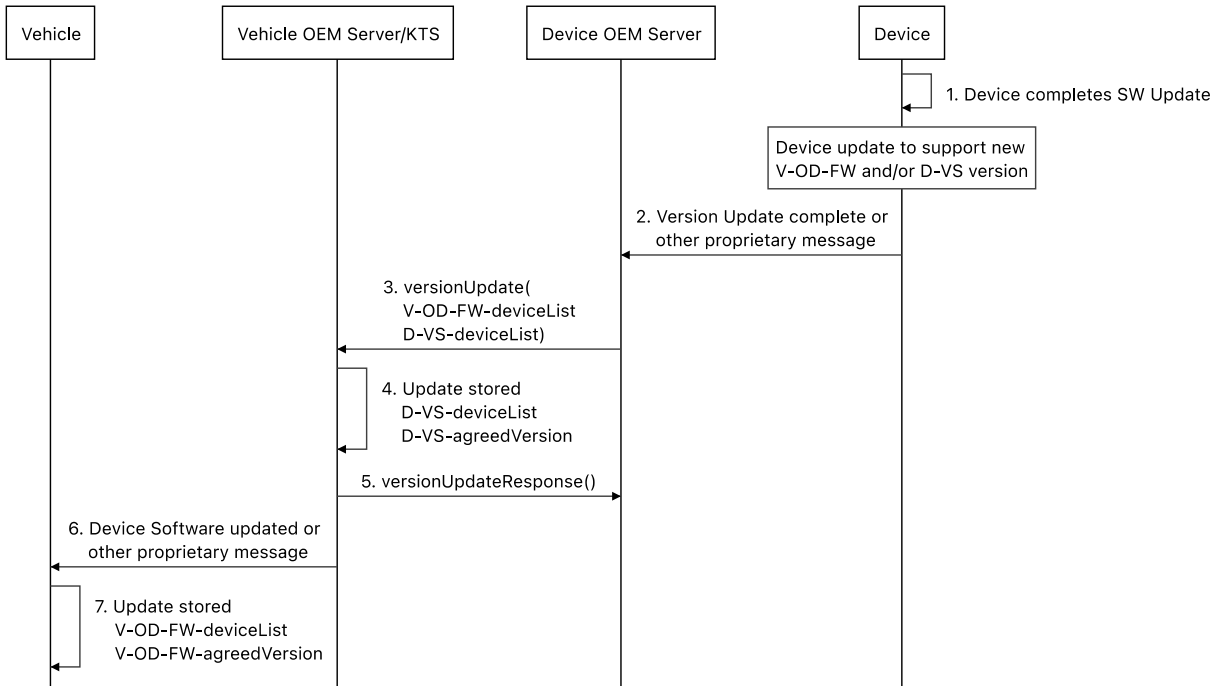
2.10.4.1 Device Software Update

Software updates to the owner or friend device that modify the list of supported V-OD-FW and/or D-VS versions shall trigger the call of the versionUpdate() API to inform the vehicle and vehicle OEM server about the updated device version lists. The call to the vehicle itself might be asynchronous and/or the vehicle might be offline, so the response is not expected to contain

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

vehicle side version information and since the vehicle side version has not changed in this case it does not need to be provided.

Figure 2-5: Version Agreement after Device Software Update



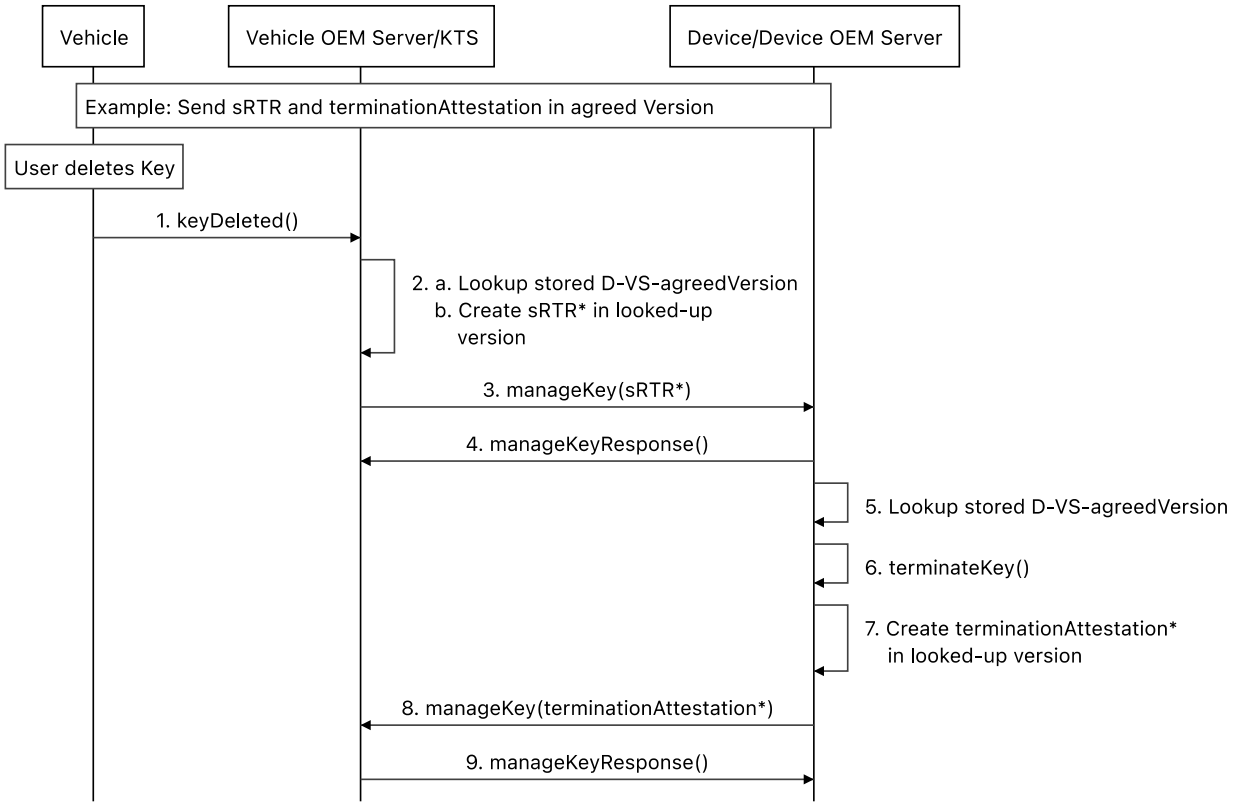
[1 - 2] Device provides updated device version lists to device OEM server. The link between device and device OEM server is proprietary to the device OEM.

[3 – 5] Device OEM server calls versionUpdate API on vehicle OEM server, which determines new agreed version for D-VS. This could result in a new agreed version for V-OD-FW

[6 – 7] Vehicle OEM server provides new V-OD-FW device list to vehicle which determines the new agreed version. The initiation of step [6] is asynchronous with and not contingent on the completion of step [5]. The link between vehicle and vehicle OEM server is proprietary to the vehicle OEM.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 2-6: Example – manageKey() usage based on D-VS agreed version



Data elements exchanged between the server and the device, such as the server remote termination request (sRTR, see Section 14.2) and terminationAttestation are versioned based on the D-VS versions as shown in the example in Figure 2-6. In the example above, sRTR* and terminationAttestation* are versions of the sRTR and the terminationAttestation that are accepted by the device and the server in the agreed D-VS version.

Relationships between device and device OEM server as well as vehicle and vehicle OEM server are proprietary, and their version management is out of scope.

2.10.4.2 Device Server SW Update

In case a device OEM server SW update that modifies DS-VS-serverList the device OEM may notify the vehicle OEM using a proprietary mechanism that is deemed acceptable by both, vehicle and device OEM. The endpoint of the device OEM server that received the update shall increment or rollback the version that is included in the URL, depending on the applicable specific region or instance of the vehicle OEM server and exact API call (e.g., …/2/manageKey). The details of the URL scheme are provided in 17.4.

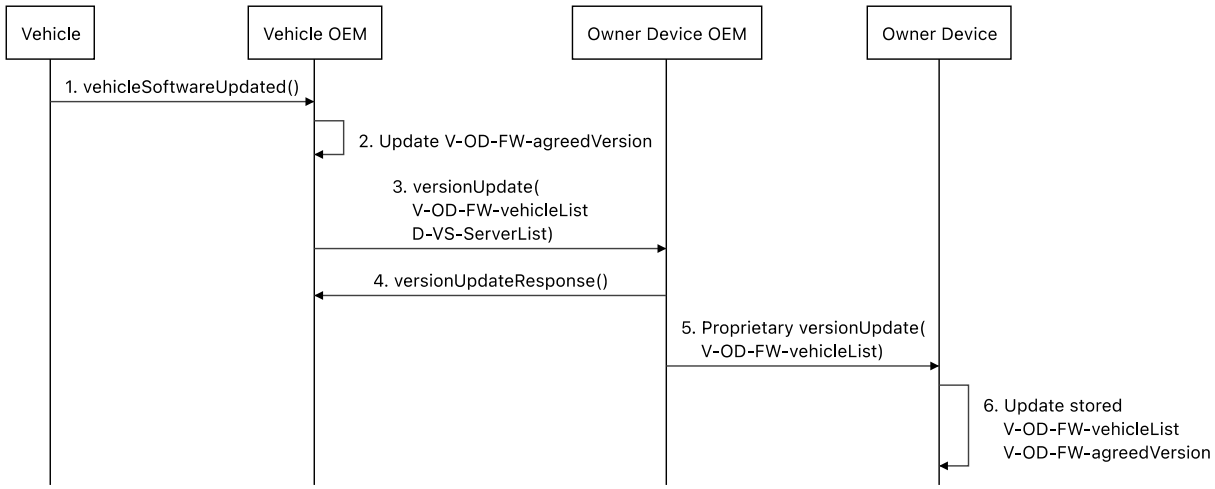
2.10.4.3 Vehicle SW Update

Software updates of the vehicle that modify the list of supported V-OD-FW shall trigger a call of the versionUpdate() API to inform the owner device via the device OEM server about the new vehicle version list (V-OD-FW-vehicleList).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Note: Updates that impact V-D-TX and V-D-BT are agreed during a Fast Transaction or Standard Transaction with the vehicle.

Figure 2-7: V-OD-FW Agreement after Vehicle SW Update



[1] Vehicle SW update is confirmed to the server.

[2] Vehicle server updates agreed V-OD-FW version.

[3 – 4] Vehicle SW versions are sent to device server

[5 – 6] Device server provides new vehicle SW version lists to device using a proprietary message.

Note that an update to a server version that impacts D-VS-serverList is configured in the devices using a proprietary method, as described in Section 2.10.4.4.

2.10.4.4 Vehicle Server SW Update

In case a vehicle OEM server SW update that modifies DS-VS, the vehicle OEM notifies the device OEM using a proprietary mechanism that is deemed acceptable by both device and vehicle OEMs. The endpoint of the vehicle OEM server that received the update shall increment or rollback the version that is included in the URL. This step is dependent on the region-specific server instance, if any, and exact API call (e.g., …/v2/trackKey). The details of the URL scheme are provided in Section 17.4.

In case of a vehicle OEM server SW update that modifies the D-VS-serverList, the vehicle OEM notifies the device OEM in a proprietary way. The device OEM shall propagate this information to relevant devices using proprietary mechanisms (as shown in Figure 2-4), so that devices are made aware in advance of changes to the D-VS-serverList per region, environment, and brand for every vehicle OEM.

The D-VS-serverList is provided in a proprietary way to every device OEM so that devices are already aware of the D-VS-serverList per region, environment, and brand for every vehicle OEM.

Example:

OEM1.**US**P.BRD1: D-VS-serverList = v1.0, v2.0, v2.1; DS-VS = v1.0, v2.0, v2.2

OEM1.**EU**P.BRD1: D-VS-serverList = v1.0, v2.0, v2.1; DS-VS = v1.0, v2.0, v2.2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

OEM1.**CN**P.BRD1: D-VS-serverList = v1.0, v2.0; DS-VS = v1.0, v2.0

Devices that have existing keys for this vehicle OEM re-calculate the agreed D-VS version for each key immediately, whereby the re-calculation is based on the routing information obtained for the corresponding vehicle (either at owner pairing or at friend sharing).

Device to vehicle server communication is then based on the new agreed version. A new key would send the key tracking request based on the updated D-VS agreed version.

2.10.5 *Version Numbers*

Versions consist of a major and a minor version number (e.g., v2.3 with 2 = major and 3 = minor).

Major versions are used for non-compatible versions of a data structure or API, or to add or remove features, data structures and APIs.

Minor versions (with same major version) are compatible but limit the available feature set to the one defined by the lower minor version.

To achieve forward compatibility, data elements in structures/commands/APDUs/API parameters that have been added in higher versions should be ignored by the entity with the lower version for TLV and JSON structures. (e.g., the key creation request data structure (Tag $7F31_h$)).

Entities shall agree on an exact version that is mutually supported. Communication based on different versions used by each entity (e.g., vehicle supporting versions 2.1 and 2.2, device supporting 1.0, 2.0 and 2.3) shall not occur.

Introduction of anchor versions (in section 2.10.6) allow both entities to find a commonly supported version.

2.10.6 *Version Introduction*

With the introduction of the versioning scheme all domain versions unless explicitly stated in Table 2-2 shall be set to 2.0.

The following plan describes the smooth introduction of versioning into the system without breaking compatibility with existing deployments of vehicles, servers and devices.

- All commands, responses, tags or data elements shall be transmitted as described in Digital Key Specification v1.0.0 or v1.1.0 between devices and vehicles if one entity only supports Domain Version 1.0 of V-OD-FW (referred to as SPAKE2+ Protocol version 1.0 in Digital Key Specification v1.0.0 or v1.1.0).
- For all other cases, any attempts to negotiate versions by an entity as defined in this specification shall be gracefully rejected by the other entity.
- All senders should start implementing and using the new data structures and API versions for each relevant domain version 2.0.
- All receivers that gracefully fail attempts to negotiate versions are considered to support the CCC Digital Key Release 3 specifications v1.0.0 and v1.1.0.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- Therefore, in the event that a versioning capable device or vehicle determines that it is interoperating with a non-versioning capable vehicle or device, the sender shall revert to using features defined in CCC Digital Key Release 3 specifications v1.0.0 and v1.1.0.

2.10.7 *Version Support*

Not all entities may be able to support all versions immediately. Therefore, anchor versions that must be supported by all entities, as a coordinated fallback version, shall be used in order to ensure interoperability. Anchor versions are not dedicated parameters, they are regular version values in the domain version lists, which are determined to be mandatory for all entities. An entity shall support all anchor versions prior to its currently supported version. E.g., if the device supports domain version 8.2, it shall support all anchor versions less than and equal to 8.2.

Anchor versions shall be agreed upon in the CCC Technical Working Group and updated in Table 2-2.

The definition of a grace period to support new versions in all entities after a CCC specification release is outside of this specification. Deprecation of specific data structures/versions/APIs shall be documented in this specification

Support period and deprecation rules for older versions are defined outside of this specification.

2.10.7.1 Maintaining compatibility across Versions

To maintain compatibility across differing domain versions supported by devices and vehicles the following rules shall apply:

1. To prevent buffer overflows on the recipient, while also allowing for future incremental additions as needed, a transmitter shall not exceed the maximum size of the SELECT, SELECT Response, SPAKE2+ Request and SPAKE2+ Verify Commands as defined in Section 5.1 by 128 bytes.
2. Using the messaging schemes defined in this specification, a CCC Digital Key Product shall identify the version of the product it is interoperating with. Upon identification of the domain version supported by the DK message recipient, the DK message transmitter shall not transmit a message that includes a tag or data element that has not been defined in the specification version supported by the recipient. The domain versions and associated specification versions within which they are defined are outlined in Table 2-2.
3. Servers shall tolerate the reception of the data elements or server API calls as outlined in Table 2-1, even if they don’t implement versioning as defined in CCC Digital Key Specification v1.2.0.

Table 2-1: Reception of Versioning Parameters by non-versioning capable Servers

Section	API	Recipient	Parameters (JSON)	Notes
17.7.1.2	trackKey	Vehicle OEM Server	vehicleOwnerDeviceFrameworkVersionList deviceVehicleServerVersionList (D-VS-deviceList)	

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17.7.1.3	trackKeyResponse	Device OEM Server	serverSupportedVersions (D-VS-serverList)	If trackKey message does not include vehicleOwnerDeviceFrameworkVersionList or deviceVehicleServerVersionList
17.7.6	versionUpdate	Vehicle OEM Server Device OEM Server		Device OEM calls/tries to call versionUpdate() API on vehicle OEM server Vehicle OEM calls/tries to call versionUpdate() API on Device OEM server

2.10.8 *Version Table*

Table 2-2 provides a list of domain versions along with active current and past values for each domain version. The table lists the following properties for each domain version value:

- Anchor Version (Y/N): whether the domain version value is an anchor version or not
- Specification Version: The version of the Digital Key Specification in which that particular Domain Version value was first introduced
- Data Structure API: New data structures that were introduced in that specification version
- Notes: Informative Description of the Changes introduced in that version of the specification

Table 2-2: Active Version Table

Domain	Domain Version Value	Anchor Version? (Y/N)	Specification Version	Notes
V-OD-FW	2.0	N	1.2.0	- Reuse of SPAKE2+ version Tags for Owner pairing Domain Version (Table 5-4) - Defined Standard and Proprietary access profiles (Table 11-4) - Changed Certificate Version (Tag 41_h) from 01_h to 02_h (Table 11-18) - Converted Key configuration from ASN.1 to BER-TLV - Changed Tag 58_h (Entitlement Data) to Tag 78_h due to change to BER-TLV format (Table 11-18) - Removed variable updateCounter as part of BER-TLV conversion
	1.0	Y	1.1.0/1.0.0	
V-D-BT	2.0	N	1.2.0	- Introduced new Tags $5E_h$ (Table 5-4, Table 11-5) and $5F_h$ (Table 5-5) to track BLE/UWB versioning - Designed new SPSM Version characteristics and GATT based message exchange to communicate version information (Section 19.2.1.7 and Section 19.2.1.8) - Modified Ranging Capability RQ/RS scheme to include versioning information (Section 19.3.1.1 and Section 19.3.1.2)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

	1.0	Y	1.1.0/1.0.0	
V-D-TX	1.0	Y	1.2.0/1.1.0/1.0.0	- Defined protocol to support existing and new Applet versions. Applet version was not incremented in version 1.2.0 of the specification. (Table 15-12, Table 15-29)
D-VS	2.0	N	1.2.0	- Introduced Tag $5F22_h$ to include Subject Key Identifier of certificate used for Signature Verification (Table 14-4) - Created new parameters in trackKey API (Section 17.7.1.2) and trackKeyResponse (Section 17.7.1.3) - Introduced new API - versionUpdate (Section 17.7.6) - Modified Unencrypted Event Notification Data (Section 17.8.10) - Added new Server Sub Status Code 50117 (Section 17.10)
DS-VS	2.0	N	1.2.0	- Negotiation of this domain version is out of scope of this specification
	1.0	N	1.1.0	
OD-FD-KS	2.0	N	1.2.0	- Introduced Tags 54_h (Table 11-6) and 55_h (Table 11-7) to version friend key sharing - Introduced Tags $7F2D_h$ and $7F11_h$ for Sharing Method Attestation (Table 11-16) - Modified Entitlement Data Schema to Match BER-TLV format (Table 11-17)
	1.0	Y	1.1.0	

2.10.9 *Version Deprecation*

Version lists in every entity should be modifiable anytime, even without a SW update. This allows to deprecate any deployed version and prevent it from being used. The mechanism to modify version lists in vehicles, servers and devices is proprietary to device and vehicle OEMs and is out of scope.

Example: Possible handling of flaw in a newer version:

- sRTR has been modified in specification between D-VS versions 2.0 and 2.2.
- sRTR processing with D-VS = 2.2 is flawed in a specific device OS version, but devices with this flaw have already been deployed. All devices also support the anchor version 2.0.
- Version lists in devices supporting this flawed version are updated to not offer version 2.2 anymore, but still support version 2.0.
- The SW update agreement process is executed (as per section 2.10.4) to inform all entities about the version update.
- Servers will now send the sRTR based on anchor version 2.0 to the device, which is able to process it correctly.
- Devices that are fixed (e.g., device OS update installed) will add back D-VS version 2.2 to their version lists so that servers can send sRTR in version 2.2 again.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

3 NFC INTERFACE [WCC1]

This section defines the NFC features that shall be implemented by the devices and how these features shall be operated for the Digital Key use case. This specification requires vehicle and device to support NFC-A technology. Support for NFC-B technology and NFC-F technology is optional on either side; corresponding requirements are only applicable if the device or vehicle is intended to support those technologies.

3.1 NFC functional requirements

This section defines the functional requirements for the NFC implementation on the vehicle and device sides.

3.1.1 *Vehicle*

The vehicle NFC readers shall be compliant with the poller requirements of the NFC Analog Technical Specification [16] for:

- Radio Frequency Power and Signal Interface
- Modulation Poller to Listener – NFC-A
- Load Modulation Listener to Poller (only for NFC-A)

If the vehicle NFC readers are intended to support NFC-B technology, they shall be compliant with the poller requirements for NFC-B as defined in NFC Analog [16]. If the vehicle NFC readers are intended to support NFC-F technology, they shall be compliant with the poller requirements for NFC-F technologies as defined in NFC Analog [16].

The vehicle NFC readers shall be compliant with the Poll Mode requirements relevant for the Type 4A Tag Platform as defined in the NFC Digital Protocol Technical Specification [17]. If the vehicle NFC readers are intended to support NFC-B technology, they shall be compliant with the Poll Mode requirements relevant for the Type 4B Tag Platform Subset as defined in NFC Digital Protocol [17]. If the vehicle NFC readers are intended to support NFC-F technology, they shall be compliant to the Poll Mode requirements relevant for the Type 3 Tag Platform Technology Subset as defined in NFC Digital Protocol [17].

The vehicle NFC readers shall be compliant with the Poll Mode requirements for the ISO-DEP protocol as defined in [17].

3.1.2 *Device*

The device NFC implementation shall be compliant with the listener requirements of the NFC Analog Technical Specification [16] for:

- The Radio Frequency Power and Signal Interface
- Modulation Poller to Listener – NFC-A
- Load Modulation Generic (only for NFC-A)
- Subcarrier Load Modulation NFC-A

The device NFC implementation may be compliant with the listener requirements for NFC-B and/or NFC-F technologies as defined in NFC Analog [16].

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The device NFC implementation shall be compliant with the Listen Mode requirements relevant for the Type 4A Tag Platform defined in NFC Digital Protocol [17]. The device NFC implementation may be compliant with the Listen Mode requirements for the Type 4B Tag Platform and/or the Type 3 Tag Platform as defined in NFC Digital Protocol [17].

The device NFC readers shall be compliant with the Listen Mode requirements for the ISO-DEP protocol as defined in [17].

The device NFC implementation shall be compliant with the generic requirements for Listen Mode as defined in the NFC Activity Technical Specification [18]. The implementation also shall be compliant with the states and transitions of the Listen-Mode State Machine as defined in NFC Activity [18], which are relevant for the Type 4A Tag Platform. The device may implement additional parts of the Listen Mode State Machine, namely the states and transitions relevant for the Type 4B Tag Platform and/or Type 3 Tag Platform.

The device shall configure the Listen-Mode State Machine using the following settings:

- CON_LISTEN_T4ATP shall be enabled.

Other configuration parameter values are implementation specific. If the device is intended to listen for NFC-B technology, it shall enable CON_LISTEN_T4BTP. If the device is alternatively or additionally intended to listen for NFC-F technology, it shall enable CON_LISTEN_T3T.

The device NFC implementation shall support the following power modes:

- **Battery Operational Mode**: The battery of the device has sufficient power to support all its functions.
- **Battery Low Mode**: The battery of the device has reached a threshold at which many functions (e.g. display) are automatically disabled, but the NFC Controller function will still be powered.

The device NFC implementation shall implement means to correctly route traffic addressed to the Digital Key applet or to the Digital Key framework. Routing can be based on the application identifiers contained in the SELECT command defined in [1]. One example of such means is the routing mechanism as defined in the NFC Controller Interface Technical Specification [19], specifically the AID-based Route Selection Process.

If the device is configured to work as a Digital Key, routing to the Digital Key applet shall be enabled in Battery Operational Mode and Battery Low Mode. Additionally, routing to the Digital Key framework shall be enabled during Owner Pairing in Battery Operational Mode.

3.2 NFC Procedures

This section defines how the vehicle operates the NFC interface to detect devices and to establish and end NFC communication with a device.

3.2.1 *NFC Polling and Link Setup Procedure*

To turn on the RF field, the vehicle may perform RF Collision Avoidance as defined in NFC Activity [18].

The vehicle shall run a polling loop that includes the Technology Detection activity as defined in NFC Activity [18], with the following configuration settings:

- CON_POLL_A shall be enabled

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Other configuration parameter values are implementation-specific. If the vehicle intends to poll for NFC-B, it shall enable CON_POLL_B. If the vehicle intends to poll for NFC-F, it shall enable CON_POLL_F.

If the Technology Detection procedure has identified one of these technologies, the vehicle NFC readers shall perform the Collision Resolution activity as defined in NFC Activity [18].

After the Collision Resolution activity, and if an NFC-A or NFC-B device indicating support for the ISO-DEP protocol as defined in NFC Digital Protocol [17], has been identified, the vehicle shall perform the Device Activation activity with the following configuration:

- INT_TECH_SEL shall be set to 000_b for NFC-A or 001_b for NFC-B
- INT_PROTOCOL shall be set to 001_b to activate the ISO-DEP protocol

Other configuration parameter values are implementation-specific. If the vehicle intends to activate a device using NFC-F technology, the vehicle NFC readers shall set INT_TECH_SEL to 010_b and INT_PROTOCOL to 100_b (Type 3 Tag Platform).

Note: The above requirements do not cover the case of multiple identified devices. If multiple devices are identified by the vehicle, the handling is implementation-specific.

3.2.2 *NFC Data Transfer Procedure*

The APDU exchanges defined in this specification shall be performed during the Data Exchange activity as defined in NFC Activity [18].

For devices using NFC-A or NFC-B technology, the vehicle NFC readers shall configure Data Transfer activity as follows:

- INT_PROTOCOL shall be set to 001_b to use the ISO-DEP protocol

If the vehicle intends to activate a device using NFC-F technology, the vehicle NFC readers shall set INT_PROTOCOL to 100_b (Type 3 Tag Platform).

Note: Please refer to Appendix E for details of how to operate the Digital Key contactless transactions over NFC-F.

After successful Device Activation using the ISO-DEP protocol, the vehicle shall operate the ISO-DEP protocol. The ISO-DEP protocol performs error processing in case of timeout or transmission errors. If the ISO-DEP protocol cannot re-establish communication after an error, it will raise an Unrecoverable Timeout Error or Unrecoverable Transmission Error, in which case the vehicle shall perform an NFC reset procedure.

3.2.3 *NFC Link Teardown Procedure*

To tear down an NFC link, the vehicle NFC readers shall perform the Device Deactivation activity as defined in NFC Activity [18].

For devices using NFC-A or NFC-B technology, the vehicle NFC readers shall configure Device Deactivation activity as follows:

- INT_PROTOCOL shall be set to 001_b to deactivate the ISO-DEP protocol

Note: Please refer to Appendix E.5.6 for devices using NFC-F technology.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

3.2.4 *NFC Reset Procedure*

To reset the NFC communication, the vehicle shall perform a reset of the Operating Field as defined in NFC Analog [16] and shall not generate any Operating Field for a time of at least 50 ms.

Afterwards the vehicle should perform an NFC polling and link setup procedure.

Note: The NFC reset procedure uses a longer Operating Field off duration than required for other NFC use cases (the minimum time for an NFC reset defined in [16] is 5.1 ms).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

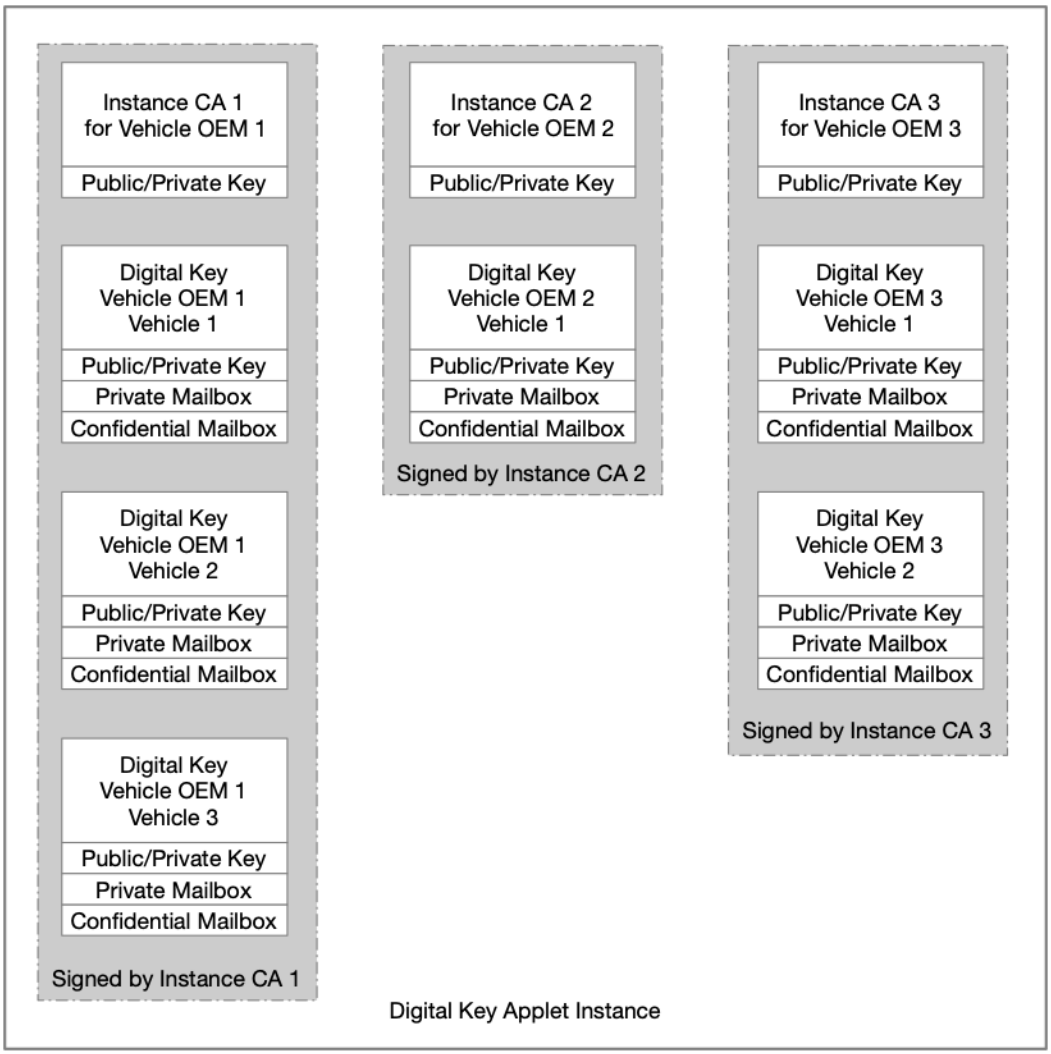
4 DATA STRUCTURE

4.1 Applet Instance Layout

One Digital Key applet instance hosts all the data necessary for a device to perform Digital Key services, as shown in Figure 4-1, including all Digital Keys and one or more Instance CAs.

A Digital Key is represented by a structure and is described in Section 4.2. The **Instance CAs** are intermediate CAs that represent the Device OEM CA and reside within the device. An Instance CA attests to (the public key of) a Digital Key created in the applet instance. Its signature can be verified using the Device OEM CA Certificate. A different Instance CA shall be used for each Vehicle OEM.

Figure 4-1: Applet Instance Layout



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

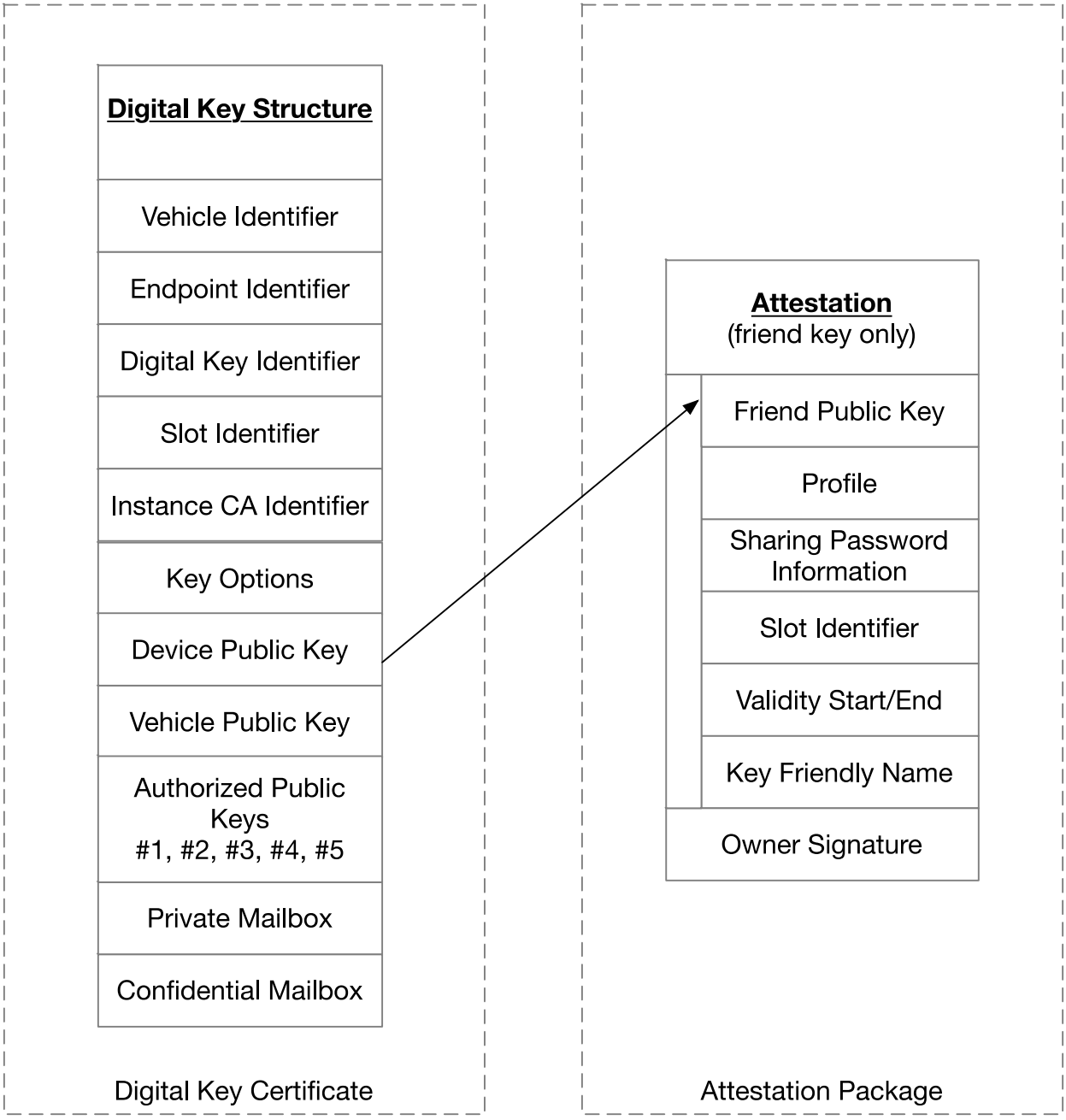
4.2 Digital Key Structure

A Digital Key structure is stored in the applet instance and contains a public/private key pair, a private mailbox, a confidential mailbox, and other elements, as shown in Figure 4-2.

An owner Digital Key consists of the Digital Key structure only. It does not have any limitations in validity and access rights.

A friend Digital Key consists of the Digital Key structure and an entitlements attestation section, which is linked to the friend key by containing the same device public key and by being signed by the owner key.

Figure 4-2: Friend Digital Key Structure and Entitlements Attestation



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The elements of the Digital Key structure are:

- **Vehicle Identifier (also referred to as vehicle_identifier)**: Uniquely identifies the vehicle to which the Digital Key is associated. The vehicle identifier is unique per Vehicle OEM. The vehicle identifier is not identical to the VIN used in the automotive industry. It is transmitted by the vehicle during a contactless transaction.
- **Endpoint Identifier**: Used for device-internal key management. The device creates the endpoint identifier based on the rules given in Appendix B.2. In Section 15, the name "endpoint_identifier" is used for key creation. The identifier is reflected in the subject field of the Digital Key Certificate [H], also referred to as "Digital Key Endpoint Certificate." Rules associated with formatting and parsing of certificates are described in Section A.4.7 of Appendix A.
- **Digital Key Identifier (also referred to as keyID)**: Enables the identification of the Digital Key within the Vehicle OEM Server system. The Digital Key identifier shall be unique per Vehicle OEM. The element is named "subject key identifier" in Section 14, based on the X.509 certificate definition. It is the SHA-1 (see [23] and [24]) hash value over the device public key. See Appendix B.2 for details.
- **Slot Identifier**: A value provided by the vehicle to the owner device to locally identify the key used. The value is transmitted within a contactless transaction. When sharing a key, the owner device or the Vehicle OEM Server provides a slot identifier value to the friend device to be used to create the friend key and, if applicable, to identify the associated immobilizer token.
- **Instance CA Identifier**: Refers to the Instance CA that has signed the Digital Key. It is assigned by the device at Instance CA creation.
- **Key Options**: Indicates which transactions (fast, standard) the key is allowed to perform. There are several more options, such as executing a transaction over the wired interface, etc. Note that those options are not access rights.
- **Device Public Key**: (Also referred to as **device.PK** and **endpoint.PK** in this specification.) Used in the standard transaction. Device public key shall be globally unique. It is generated at endpoint creation and is stored in the vehicle. It is represented by the "subjectPublicKey" field of the endpoint certificate in Section 15. Note that Device.Enc.PK (See Section 14) is different from device public key. Similarly, device private key, device.SK, and endpoint.SK are used interchangeably in this specification.
- **Vehicle Public Key**: (Also referred to as **Vehicle.PK** and **vehicle_pk** in this specification.) Used in the standard transaction. The vehicle public key is the same for all devices associated with the same vehicle.
- **Authorized public keys**: Provided by the vehicle and are used as root in the verification chain of a friend public key at key sharing (see Section 11). In this version of the specification, the vehicle shall include only one single authorized_PK. Usage of additional authorized public keys is left for future use cases.
- **Private Mailbox**: A buffer that allows for the transfer of elements to or from the vehicle during a Digital Key Transaction (see Section 4.3.1 for more details).
- **Confidential Mailbox**: A buffer that allows for the transfer of elements which need confidentiality protection to or from the vehicle during a Digital Key Transaction (see Section 4.3.2 for more details).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

All relevant data elements of the Digital Key shall be verified by the vehicle during the owner pairing procedure (see Section 6) before accepting the device public key, and by the owner device during key sharing (see Section 11) before signing the friend device public key.

Before signing the friend's public key, the owner verifies that the friend's public key pair has been created on an eligible SE. The verification chain starts with an authorized public key in the owner Digital Key structure, which has been provided and is trusted by the vehicle, e.g., the Vehicle OEM CA public key (see Section 16.8).

The elements provided in the attestation package (for friend keys only) are:

- **Friend Public Key**: The public key created by the friend device together with the private key. During key sharing, this public key shall be signed by the owner device together with the other fields in the attestation package.
- **Profile**: Selected by the sender of a Shared Key. It needs to comply with the Vehicle OEM policy and is verified by the vehicle. Digital Keys that do not comply with the Vehicle OEM policy settings are rejected by the vehicle (i.e., their public key is not accepted). The Vehicle OEM policy is out of scope of this specification.
- **Sharing password information**: Contains the seed for the vehicle to generate a sharing password from the shared secret established at owner pairing. It also contains the information regarding whether the owner device policy requires (or not) that the vehicle requests a sharing password to the friend before activating the shared Digital Key.
- **Validity start date**: The earliest date and time the key can be used. The Digital Key may be accepted when presented to a vehicle earlier, but it shall not have effect before the date and time is reached. This field requires an internal time to be available in the vehicle. Accuracy, reliability and security of this internal time are dependent on Vehicle OEM policy and vehicle capabilities.
- **Validity end date**: The latest date and time the key can be used. This field requires an internal time to be available in the vehicle. Accuracy, reliability and security of this internal time are dependent on Vehicle OEM policy and vehicle capabilities.
- **Key friendly name**: A field conveying a user-friendly name for the Digital Key that should be assigned at key sharing (see Section 11). It may be the same name as in the owner's contacts when sharing a Digital Key. The owner should be allowed to edit the name for identification and personalization. For privacy reasons, friendly name should not contain private information such as full name of the friend.

Digital Key elements except private and confidential mailboxes are not changeable during the lifetime of a Digital Key. A Digital Key can be created, terminated, and deleted. In "terminated" state, it is not usable but still able to provide the termination attestation until it is finally deleted from memory. The Digital Key states are managed internally by the applet.

4.3 Mailboxes

The mailbox mechanism allows the SE to store small data buffers accessible by the Digital Key framework and by the vehicle. These data buffers are stored in the SE's non-volatile memory and are read from/written to during transaction, key sharing, and pairing. Each Digital Key (referred to as an Endpoint) created in the applet instance of a device can have two mailbox types (private and confidential) with different security properties and targeted at different usages. The

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

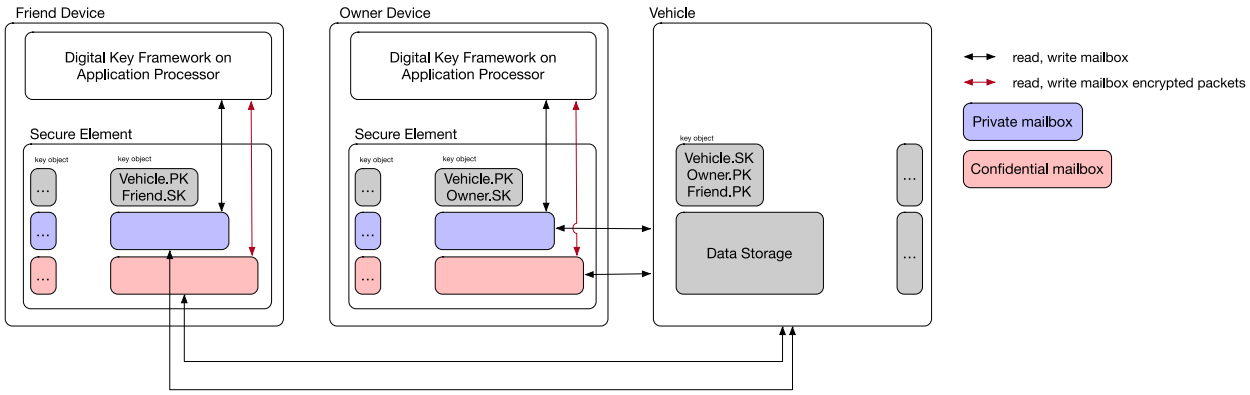
mailboxes support random access, whereby content can be read from/written to using offset/length parameters to access them.

Confidential mailboxes are not shared between Endpoints, and consequently only a vehicle paired to an Endpoint can access the confidential mailbox of such Endpoint. Private mailboxes are not shared among Endpoints, either; however, the Digital Key framework and the vehicle can freely read from and write to their contents.

The confidential mailbox is meant for storage of secret data that is never available in plaintext to the Digital Key framework. In addition, only the associated vehicle has read/write access privileges to the contents of this mailbox. The Digital Key framework can only extract or push portions of the confidential mailbox in an encrypted format. The confidential mailbox is used for storage of immobilizer tokens.

The private mailbox is meant for storage of data accessible in plaintext to the Digital Key framework, the vehicle, and the SE. The use of the private mailbox guarantees that this data is never transferred in plaintext -, and the mailbox access is only possible using the EXCHANGE command. The private mailbox is used for transfer of the attestation package and to indicate which mailbox fields are present/absent (signaling) as described in the following sections.

Figure 4-3: Mailboxes Read/Write Permissions



4.3.1 *Private Mailbox*

The private mailbox can be read from and written to by the Digital Key framework using the device internal wired link as described in Section 15. It can also be read from and written to by the vehicle once the secure channel described in Section 15 is established between a vehicle and device using the Digital Key. Data exchanged with vehicle is always protected by the established secure channel.

The mapping of the private mailbox defines data structures required by the Digital Key framework and provides a bi-directional signaling mechanism to indicate to the receiver that new data is available as described in Figure 4-4.

The private mailbox enables the Vehicle OEM to define its data structures by the signaling mechanism. These data structures are opaque to the Digital Key framework and are only known to the vehicle and a Vehicle OEM app or the Vehicle OEM Server. Arbitrary data that can be stored in and retrieved from the private mailbox associated with each Digital Key includes the signaling bitmap, the slot identifier bitmap, the slot identifiers, the Vehicle OEM proprietary data, and the attestation package, in case a key is presented to a vehicle. The private mailbox

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

format is determined at owner pairing. It shall be the same for owner and friend mailboxes, though certain fields may not be used in friend mailboxes.

The signaling bitmap is required for owner and friend mailboxes. The slot identifier bitmap is used by the owner device to keep track of available slot identifiers and, if applicable, immobilizer tokens for Digital Key sharing. Slot identifiers are only used in the owner mailbox.

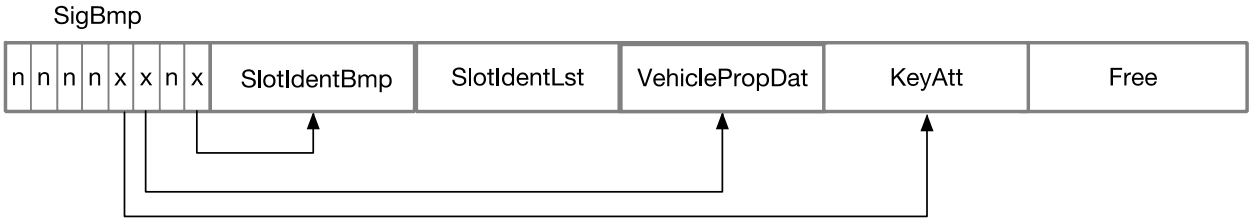
The slot identifiers and, if applicable, the immobilizer tokens are provided to the owner device either by the vehicle or by the Vehicle OEM Server. This is controlled by the vehicle in the SHARING_CONFIGURATION field in Table 5-14 (Tag $7F60_h$ and Tag DA_h).

The slot identifiers and, if applicable, the immobilizer tokens for the friend device are provided by the owner device or by the Vehicle OEM Server. This is controlled by the owner device in the SHARING_CONFIGURATION field in Table 11-6 (Tag $7F60_h$ and Tag DA_h) which derives from the SHARING_CONFIGURATION field in Table 5-14 (Tag $7F60_h$) the owner device received during owner pairing.

Vehicle proprietary data may be used in owner and friend mailboxes. The maximum number usable keys in a vehicle is implementation-specific. The limit of seven slot identifiers for friend keys in the private mailbox refers to the maximum number of shareable friend keys after which the owner is required to obtain new slot identifiers from the vehicle.

Key attestation packages are used with owner and friend mailboxes. Note that these mailboxes have different purposes and structures.

Figure 4-4: Owner Device Private Mailbox Signaling



The **Signaling Bitmap** as contained in the SigBmp field allows the vehicle and the device to inform the other party of changes occurring or actions to be taken on some mailbox field by the other party. It is the responsibility of the vehicle or the device that consumes the bit to reset the bit and delete the data if needed. The signaling relationship and the offsets of the data elements are configured by the vehicle at owner pairing (See Section 6).

The length of this field is 1 byte. This field is big-endian ordered, meaning that bit 0 is the rightmost bit of the field.

The following constants are defined for the rest of this document:

SLOT_IDENTIFIER_BITMAP_BIT = 00_h

SLOT_IDENTIFIERS_BIT = 01_h

VEHICLE_OEM_PROPRIETARY_DATA_BIT = 02_h

ATTESTATION_PACKAGE_BIT = 03_h

SLOT_IDENTIFIER_LENGTH = (VEHICLE_OEM_PROPRIETARY_DATA_OFFSET –

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

SLOT_IDENTIFIERS_OFFSET) / 8

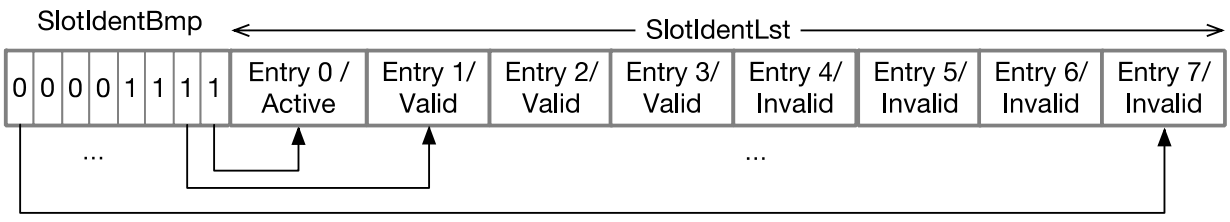
Table 4-1 shows the purpose for each bit in the SigBmp field. The signaling bits may be read and/or written by the vehicle and/or the device.

Table 4-1: Signaling Bitmap Decoding

Bit	Name	Value	Description	Set by
Bit0	SlotIdentBmp	0	All slot identifiers valid	Vehicle
Bit0	SlotIdentBmp	1	Not all slot identifiers valid	Device (when sharing) Vehicle (initial refill at owner pairing)
Bit1	RFU	0	RFU	RFU
Bit1	RFU	1	RFU	RFU
Bit2	VehiclePropDat	0	Device has read vehicle proprietary data	Device
Bit2	VehiclePropDat	1	Vehicle has updated vehicle proprietary data	Vehicle
Bit3	KeyAtt	0	No key attestation package present	Vehicle/Device (attestation package retrieved successfully or is not required any more)
Bit3	KeyAtt	1	Key attestation package present	Vehicle/Device (attestation package written in mailbox)
Bit4	RFU	0	RFU	RFU
Bit4	RFU	1	RFU	RFU
Bit5	RFU	0	RFU	RFU
Bit5	RFU	1	RFU	RFU
Bit6	RFU	0	RFU	RFU
Bit6	RFU	1	RFU	RFU
Bit7	RFU	0	RFU	RFU
Bit7	RFU	1	RFU	RFU

The **Slot Identifier Bitmap** as contained in the **SlotIdentBmp** field signals the validity of a slot identifier in the corresponding entry of the slot identifier, as shown in Figure 4-5.

Figure 4-5: Slot Identifier Bitmap Assignment



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The length of this field is 1 byte. The most significant bit represents the state of entry 7; the least significant bit represents the state of entry 0.

If a bit is set to 0, the slot identifier is invalid. If immobilizer tokens are retrieved from the vehicle (see Table 5-14), in the next standard transaction, the vehicle shall write a new slot identifier value and set the bit to 1 and also fill up a new immobilizer token, if applicable.

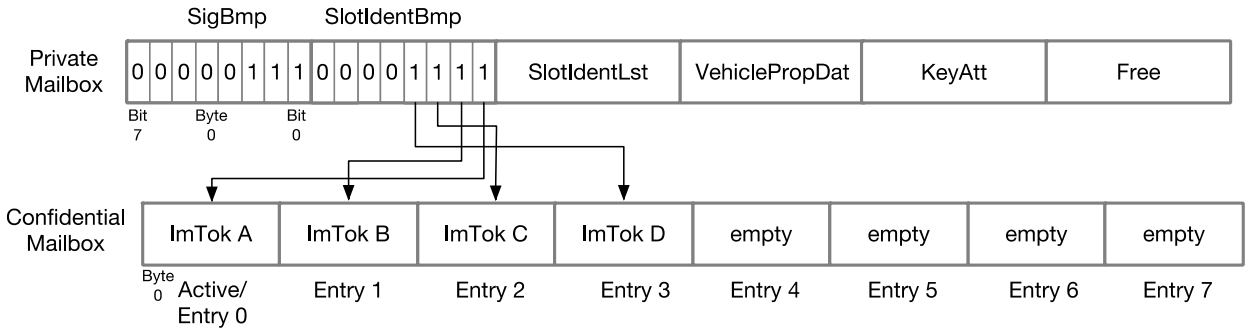
Note: The invalid slot identifier value is still present in the entry. The rules for managing the slot identifier values are described in Section 13.7.

If a bit is set to 1, a valid slot identifier is present in the corresponding entry in the SlotIdentLst field, and the corresponding immobilizer token is also present in the corresponding entry in the confidential mailbox.

Each bit in the SlotIdentBmp shall also indicate the presence of an immobilizer token in the corresponding entry and memory offset of the confidential mailbox (see Table 4-3 and Figure 4-6). This bitmap allows the device and the vehicle to synchronize on the confidential mailbox content, if immobilizer tokens are retrieved from the vehicle.

The active owner immobilizer token (ImTok A) shall always be located in entry 0 (corresponding to memory offset 0). Its presence shall be signaled by setting bit 0 to value 1 in the SlotIdentBmp. Other immobilizer tokens (e.g., entry 1 to 3 in Figure 4-6) are used for friend sharing.

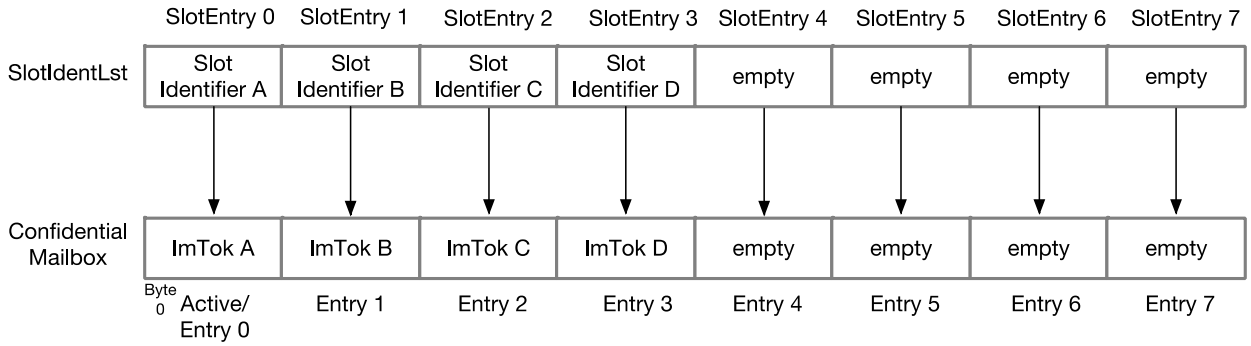
Figure 4-6: Owner Device Immobilizer Token Signaling



The **slot identifiers** as contained in the SlotIdentLst field help the vehicle keep track of which slot identifiers and immobilizer tokens (if slot identifiers and immobilizer tokens are retrieved from the vehicle) have been distributed or are in use. SlotIdentLst field is comprised of eight SlotEntry subfields, and each of the SlotEntry subfields contains a slot identifier value, as shown in Figure 4-7. The length of the SlotIdentLst field shall be eight times the size of a slot identifier. Each slot identifier is mapped to an immobilizer token at the same position index in the confidential mailbox. The length of this data field is defined by (VEHICLE_OEM_PROPRIETARY_DATA_OFFSET – SLOT_IDENTIFIERS_OFFSET).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 4-7: Slot Identifier List to Confidential Mailbox mapping



Slot identifiers are issued by the vehicle or by the Vehicle OEM Server. They are provisioned by the vehicle to the owner device along with associated immobilizer tokens, if immobilizer tokens are retrieved by the vehicle. Alternatively, they are provisioned by the Vehicle OEM Server after owner pairing or key sharing. The vehicle guarantees that slot identifiers are unique over the lifetime of a vehicle.

Slot identifiers and immobilizer tokens are included in the data provided to the friend device when sharing a key (See Section 11).

The **Vehicle OEM proprietary data** as contained in the VehiclePropDat field is arbitrary data specific to each Vehicle OEM.

The length of this data field is defined by (ATTESTATION_PACKAGE_OFFSET – VEHICLE_OEM_PROPRIETARY_DATA_OFFSET). The Vehicle OEM proprietary data is present in the private mailbox until consumed and cleared by the device or by the vehicle.

If the length of this data field is greater than 0, the Vehicle OEM proprietary data should be read by the Digital Key framework within the Digital Key termination process and sent along with the termination attestation to the KTS. The Vehicle OEM proprietary data may not be sent in all cases due to technical constraints.

The **attestation package** as contained in the KeyAtt field can be used for i) providing the Opaque Attestation (see Section 6.3.4.1) to the device, ii) providing the kts-signature, indicating that the owner key was successfully registered at the KTS, to the vehicle (see Section 6.3.4.4) , iii) for supporting friend sharing (providing the key Attestation Package to the vehicle; see Section 11.8.4), as well as for iv) resuming a suspended key using a Resume Attestation provided by the device (see Section 13.4.3).

The attestation package is present in the private mailbox until consumed and cleared by the vehicle or the device (depending on the attestation type). The length of this data field is defined by (MAILBOX_CONTENT_END_OFFSET – ATTESTATION_PACKAGE_OFFSET).

Table 4-2 describes the relationship among data elements (“fields”), memory offsets, and signaling bit. The sizes of the data elements in the private mailbox are defined by offsets, which are provided by the vehicle during owner pairing. Data structure sizes are calculated by subtracting the offset values.

Table 4-2: Private Mailbox Content

Field	Memory Offset	Description	Sig-Bit
SigBmp	SIGNALING_BITMAP_OFFSET	Signaling bitmap	n/a

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Field	Memory Offset	Description	Sig-Bit
SlotIdentBmp	SLOT_IDENTIFIER_BITMAP_OFFSET	Slot identifier bitmap	Bit 0
SlotIdentLst	SLOT_IDENTIFIERS_OFFSET	Vehicle-assigned values to manage immobilizer tokens and termination	Bit 1
VehiclePropDat	VEHICLE_OEM_PROPRIETARY_DATA_OFFSET	Vehicle OEM proprietary data structure	Bit 2
KeyAtt	ATTESTATION_PACKAGE_OFFSET	Cryptographic key attestation package	Bit 3
MbxEnd	MAILBOX_CONTENT_END_OFFSET	Offset of first byte of free memory after last data structure	n/a

The Sig-Bit column in the above table identifies the bit of the signaling bitmap responsible to signal a value change in the associated data structure. The association is configured by the vehicle (see Table 5-13).

SIGNALING_BITMAP_OFFSET shall be set to 0.

SLOT_IDENTIFIER_BITMAP_OFFSET shall be set to 1. The field SlotIdentBmp assigns eight offsets into the confidential mailbox (entries 0–7) to the eight signaling bits.

SLOT_IDENTIFIERS_OFFSET defines the area used by the Vehicle OEM to store eight slot identifiers, which are used for management of slot identifiers and immobilizer tokens if slot identifiers and immobilizer tokens are retrieved from the vehicle. The size of the SlotIdentLst data block shall be a multiple of eight. The size of a slot identifier is determined by the size of tag $4E_h$ in Table 15-13.

VEHICLE_OEM_PROPRIETARY_DATA_OFFSET defines the start position of the structure VehiclePropDat, defined by the Vehicle OEM. This structure is opaque to the Digital Key framework, and part of the structure may be disclosed to the Digital Key framework.

ATTESTATION_PACKAGE_OFFSET defines the start position of the structure KeyAtt used for key sharing on friend devices when the vehicle is offline at the first transaction (see Section 11).

MAILBOX_CONTENT_END_OFFSET indicates the offset of first byte of the free memory after the last data structure of the private mailbox.

The order of the data structures in the mailbox buffer shall be in the order of the associated signaling bits (S-Bit), starting with Bit 0.

The offset values are defined together by both the Vehicle OEM and the Device OEM and are out of scope of this specification.

4.3.2 *Confidential Mailbox*

The confidential mailbox can be read from and written to by the vehicle once the secure channel described in Section 7 or 8 is established between a vehicle and device using the Digital Key. Data exchanged with vehicle is always protected by the established secure channel. The

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

confidential mailbox can also be written to by the Digital Key framework via the internal wired link when data is encrypted with the Endpoint encryption public key. The content of the confidential mailbox may be exported in encrypted form with the encryption public key of another trusted SE if allowed by Endpoint configuration and with user consent.

If required by the Vehicle OEM, the immobilizer token may be stored and retrieved in the confidential mailbox and may be associated with each Digital Key.

The friend confidential mailbox typically contains only one entry corresponding to its active immobilizer token. The active immobilizer token is a secret piece of data to be presented to the vehicle during a Digital Key transaction.

The owner confidential mailbox typically contains eight entries. The first entry (index 0) is dedicated to the active immobilizer token, whereas the other seven entries contain immobilizer tokens to be distributed to friends during sharing operations.

Table 4-3 describes the confidential mailbox memory mapping when immobilizer tokens are required.

Table 4-3: Confidential Mailbox Content

Memory Offset	Description
0	active immobilizer token, 0
IMMOBILIZER_TOKEN_LENGTH	immobilizer token 1
2 × IMMOBILIZER_TOKEN_LENGTH	immobilizer token 2
3 × IMMOBILIZER_TOKEN_LENGTH	immobilizer token 3
4 × IMMOBILIZER_TOKEN_LENGTH	immobilizer token 4
5 × IMMOBILIZER_TOKEN_LENGTH	immobilizer token 5
6 × IMMOBILIZER_TOKEN_LENGTH	immobilizer token 6
7 × IMMOBILIZER_TOKEN_LENGTH	immobilizer token 7

The active immobilizer token shall be present at offset 0 (see Figure 4-6) and is required by the vehicle during a Digital Key transaction. This token shall be present on owner and friend devices if immobilizer tokens are required by the Vehicle OEM.

The immobilizer tokens present in entries 1 to 7 shall be used for distribution during key sharing (see Section 11).

The IMMOBILIZER_TOKEN_LENGTH is provided by the Digital Key framework. It is transmitted in the mailbox configuration table in the first NFC session (see Table 5-13).

4.3.3 *Immobilizer Token*

An optional Vehicle OEM requirement for the device is to store and release an immobilizer token, which is a confidential data element provided to the vehicle when it is requested (e.g., at engine start). The format and content of the immobilizer token are out of scope of this specification.

If implemented, either the vehicle or the Vehicle OEM Server provides the immobilizer tokens to the owner device during or after owner pairing. The immobilizer tokens for the friend devices are provided either by the owner device or the Vehicle OEM Server. The Immobilizer tokens are

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

stored in pre-allocated entries in the confidential mailbox. The confidential mailbox size shall be configured to eight times the size of the immobilizer token.

Immobilizer tokens shall be associated with a slot identifier, as shown in Figure 4-7.

4.3.4 *Mailbox Access Rights*

The following table summarizes access rights for the mailboxes from vehicle and Digital Key framework.

Table 4-4: Mailbox Access Rights

Mailbox	Digital Key Framework	Vehicle
Confidential	write encrypted, export encrypted	read, write
Private	read, write	read, write

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

5 OWNER PAIRING COMMANDS [WCC1/WCC2/WCC3]

This section defines the commands and data elements used for the owner pairing procedure.

The owner device (framework) – vehicle version (V-OD-FW), which is exchanged in the SELECT (5.1.1) and SPAKE2+ REQUEST (5.1.2) commands covers all commands defined in Chapter 5.

The digital key applet protocol versions cover the applet transactions with the vehicle for unlock, engine start, etc., as described in sections 7, 8 and 10. The applet APIs used for endpoint creation, set confidential data, setup endpoint, etc. are managed based on the owner device (framework) – vehicle version.

5.1 Supported Commands for Owner Pairing

Table 5-1 lists all commands defined for the owner pairing flow as described in Section 6.

Table 5-1: Owner Pairing Command Set

Command	Instruction Byte (hex)	Implemented by
SELECT	A4	Digital Key Framework
SPAKE2+ REQUEST	30	Digital Key Framework
SPAKE2+ VERIFY	32	Digital Key Framework
WRITE DATA	D4	Digital Key Framework
GET DATA	CA	Digital Key Framework
GET RESPONSE	C0	Digital Key Framework
OP CONTROL FLOW	3C	Digital Key Framework
See Table 15-1	See Section 15.3.2	Digital Key Applet

Table 5-2 lists the generic status words to be retrieved in case of error during basic input command checking, which apply to all owner pairing commands listed in Table 5-1.

Table 5-2: Generic Status Words

Status Word	Comment
6700_h	wrong length
$6A80_h$	Incorrect parameters in command payload
$6A82_h$	file not found
$6B00_h$	wrong P1 or P2
$6C00_h$	wrong Le
$6D00_h$	wrong INS code
$6E00_h$	wrong CLA code
9000_h	command successfully executed

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

5.1.1 *SELECT Command*

5.1.1.1 Purpose

The vehicle sends the SELECT AID command to the device. The Digital Key framework AID is A0000008094343434444B467631$_h$.

When the Digital Key framework is selected, the device shall respond with the data as described in Table 5-3.

The device shall indicate its current pairing state to the vehicle. The possible states are:

- Not in pairing mode
- Pairing mode started and pairing password entered

The SELECT command used to select the Digital Key applet instance (using Instance AID) is described in Section 15.3.2.1

5.1.1.2 Definition

command: 00 A4 04 00 Lc [Digital_Key_Framework_AID] 00
response: [Table 5-3**]90 00**

Table 5-3: Response to SELECT Command

Tag	Length (bytes)	Description	Field is
5A$_h$	2 × n	n supported vehicle – owner device framework versions (V-OD-FW-deviceList). (ver.high | ver.low).	mandatory
5C$_h$	2 × m	m supported Digital Key applet protocol versions (ver.high | ver.low)	mandatory
D4$_h$	1	00$_h$ = not in pairing mode 02$_h$ = pairing mode started, and pairing password entered	mandatory

5.1.1.3 Usage

The device shall return all supported owner device framework – vehicle (V-OD-FW) and Digital Key applet protocol versions using two bytes per version in big-endian coding.

The reception of at least 16 versions shall be supported by all entities. An entity may transmit up to 16 versions.

Version h.l is represented as ver.high= ‘h’ (one byte) and ver.low= ‘l’ (one byte).

The versions shall be ordered from the highest to the lowest. At least one version shall be provided; otherwise the service is considered as not available.

Domain version V-OD-FW 1.0 maps to SPAKE2+ protocol version 1.0 and Domain version V-D-TX 1.0 maps to Digital Key Applet protocol version 1.0 (coded 0100$_h$). Domain versions and their status as Anchor versions are listed in Table 2-2. The vehicle shall match the versions with its supported versions as described in Section 6.3.3.8.

The device signals if it is in pairing mode or not. If the device is “not in pairing mode,” the vehicle should continue only if the vehicle is in pairing mode itself.

This command shall return a general status word as listed in Table 5-2.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

5.1.2 *SPAKE2+ REQUEST Command*

5.1.2.1 Purpose

In this command, the vehicle shall send the agreed vehicle – owner device framework version list with agreed vehicle – owner device framework (V-OD-FW-agreedVersion) first, all supported Digital Key applet protocol versions, and the Scrypt parameters of the SPAKE2+ protocol. The vehicle shall retrieve the curve point X of the SPAKE2+ protocol in the response. The parameters used in the SPAKE2+ REQUEST command are described in Section 18.

5.1.2.2 Definition

command: 80 30 00 00 Lc [Table 5-4**] 00**
response: [Table 5-5**] 90 00**

Table 5-4: SPAKE2+ REQUEST Command Fields

Tag	Length (bytes)	Description	Field is
$5B_h$	2 x n	Vehicle - owner device framework version list (V-OD-FW-vehicleList) (ver.high | ver.low), agreed version first	mandatory
$5C_h$	2 × m	m supported Digital Key applet transaction (V-D-TX) protocol versions (ver.high | ver.low), agreed version first.	mandatory
$5E_h$	2 × w	w supported BLE (V-D-BT) Supported_DK_Protocol_Version versions (ver.high | ver.low)	Conditional. For WCC2/WCC3 capable vehicles. Shall be included if and only if agreed version in tag $5B_h$ is greater than 1.0[2]
$7F50_h$	32	Scrypt configuration parameters	mandatory
$C0_h$	16	Cryptographic salt, s	mandatory
$C1_h$	4	Scrypt cost parameter, N_{scrypt}	mandatory
$C2_h$	2	Block size parameter, r	mandatory
$C3_h$	2	Parallelization parameter, p	mandatory
$D6_h$	2	Vehicle Brand (see Table 2-1 in [35]), include this tag for all vehicles including vehicle that supports NFC only.	mandatory[3]

Table 5-5: SPAKE2+ REQUEST Response Fields

Tag	Length (bytes)	Description	Field is
50_h	65	Curve point X of the SPAKE2+ protocol, prepended with 04_h as per Listing 18-3	mandatory

[2] The introduction of versioning for Bluetooth subsystem (WCC2) is gated by V-OD-FW version 1.0

[3] Vehicle models launched before 01-01-2021 may not support this. Vehicle models launched after 01-01-2021, including vehicle models only supporting NFC shall support this.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is
$5F_h$	2 or 0	DK (V-D-BT) Protocol Version supported by device from Vehicle List, (as per Table 19-11). Tag with length 0 is returned if no commonly supported version is found.	Conditional. For WCC2/WCC3 capable devices. Present if Tag $5E_h$ was included in SPAKE2+ REQUEST message

5.1.2.3 Usage

Before sending the SPAKE2+ REQUEST command, the vehicle shall check the counter for SPAKE2+ pairing attempts. If the counter indicates 7 pairing attempts have been made, the vehicle shall not send the SPAKE2+ REQUEST command. Instead, the vehicle shall send an OP CONTROL FLOW to abort (error counter limit for wrong pairing password is exceeded, new pairing password needed ($0D_h$) or no specific reason(00_h)). When a new pairing password is generated, the counter shall be reset.

As described in section 2.10.6, the vehicle shall send the V-OD-FW version list with agreed V-OD-FW version, to be used by the owner device.

The vehicle shall also send the list of supported Digital Key applet protocol versions, whereby the first listed version shall be used by the vehicle as the agreed version with the owner device. The whole list of Digital Key applet protocol versions (Tag $5C_h$) shall be included in the Key Creation Request (See Section 11.8.1, DIGITAL_KEY_APPLET_PROTOCOL_VERSION). This allows the determination of the best version to be used for key sharing with friend device. Further, this allows the friend device to determine the best transaction (fast transaction, standard transaction) version with the vehicle (V-D-TX).

The WCC2/WCC3 capable vehicle shall send the version list (tag $5E_h$) to the device in the SPAKE2+ Request. The device sends the selected version, to the vehicle in tag $5F_h$. The set of V-D-BT version lists (tag $5E_h$) obtained from the vehicle shall also be included in the Key Creation Request (see Section 11.8.1). This is done because owner pairing may be first done over the NFC link and Bluetooth activation happens later. If a commonly supported BT protocol version is not found, then the device shall return empty Tag $5F_h$ with length 0. If the owner device does not support Bluetooth, but a friend device does, the domain version V-D-BT becomes applicable. The process of selection of V-D-BT at the time of Bluetooth activation is shown in Figure 19-2. The version selection is described in detail in Section 19.2.1.7.

Reception of at least 16 versions shall be supported by all entities. Transmitters may transmit up to 16 versions.

Version h.l is represented as ver.high=’h’ (one byte) and ver.low=’l’ (one byte).

The first version in the list shall be the agreed version value, the following version values shall be ordered from the highest to the lowest. At least one version shall be provided; otherwise the service is considered as not available.

The vehicle choice for the Digital Key applet protocol version is provided as part of the Digital Key applet transaction (see Section 15).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

All curve points of the SPAKE2+ protocol are defined following x9.63 standard [22] as the byte stream 0x04 || <x> || <y> where <x> and <y> are 32-byte integers in big-endian representation (see Section 18.1).

If the returned X value is at infinity or is not a valid point on the defined ECC curve, the vehicle shall abort the flow and shall send an OP CONTROL FLOW command with P2 value set to $0C_h$ as described in Section 5.1.7.

The number of Scrypt iterations (cost parameters) is a positive 4-byte unsigned integer value that configures the Scrypt function (see Section 18.4) to derive the verifier values in the Vehicle OEM Server and on the device. Other transmitted Scrypt parameters are the block size and the parallelization parameters (see Section 18.1.2). The value part of the Scrypt cost parameter TLV, Block size parameter TLV, and Parallelization TLV is encoded in big-endian format.

If the vehicle does not find any version supported by both sides for either SPAKE2+ or the Digital Key applet protocol or both, the vehicle shall send an OP_FLOW_CONTROL ("Owner Pairing Flow Control") command with an appropriate reason code as defined in Table 5-24, instead of the SPAKE2+ REQUEST command.

If the SPAKE2+ REQUEST command is successfully processed, the vehicle and device shall calculate the shared secret K as per Listing 18-4 and Listing 18-5, respectively.

This SPAKE2+ REQUEST command may return either a general error condition as listed in Table 5-2 or one of the error conditions listed in Table 5-6. No response data shall be returned with an error status word.

Table 5-6: SPAKE2+_REQUEST Response Error Status Words

Status Word	Comment
6985_h	Command used out of sequence
$6A88_h$	Received data invalid or zero
9484_h	Device not ready for pairing

5.1.3 *SPAKE2+ VERIFY Command*

5.1.3.1 Purpose

This command mutually exchanges evidence to prove that the calculated shared secret is equal on both sides.

5.1.3.2 Definition

command: 80 32 00 00 Lc [Table 5-7**] 00**
response: [Table 5-8**] 90 00**

Table 5-7: SPAKE2+ VERIFY Command Fields

Tag	Length (bytes)	Description	Field is
52_h	65	Curve point Y of the SPAKE2+ protocol, prepended with 04_h as per Listing 18-2	mandatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is
57_h	16	Vehicle evidence M[1]	mandatory

Table 5-8: SPAKE2+ VERIFY Response Fields

Tag	Length (bytes)	Description	Field is
58_h	16	Device evidence M[2]	mandatory

5.1.3.3 Usage

Before sending the SPAKE2+ VERIFY command, the vehicle shall increase the counter for SPAKE2+ pairing attempts.

The vehicle shall calculate evidence M[1] as described in Listing 18-7 and send it, together with curve point Y, to the device.

The device shall verify the following:

1. if the received curve point Y is a valid point on the defined ECC curve
2. the received M[1]

If both verifications are successful, the device shall calculate evidence M[2] as described in Listing 18-8 and shall send M[2] back to the vehicle in SPAKE2+ Verify Response.

Only if the vehicle successfully verifies the received M[2], the vehicle shall continue the owner pairing flow.

If any of the above verification fails, i.e., an error occurs, the device shall not calculate M[2] and shall not return M[2] or any other response data except the status word. In this case, the vehicle shall send an OP CONTROL FLOW command to abort the owner pairing with P2 value set to 09_h as described in Section 5.1.7, instead of the next regular command.

The SPAKE2+ VERIFY command may return either a general error condition as listed in Table 5-2 or one of the error conditions listed in Table 5-9.

Table 5-9: SPAKE2+ VERIFY Response Error Status Words

Status Word	Comment
6985_h	Command used out of sequence
$6A88_h$	Verification of evidence failed

The SPAKE2+ VERIFY step leads to secure channel keys used to establish a SCP03 channel between the Framework and the vehicle for the subsequent command exchanges. The used SCP03 channel is established following Listing 18-10 and Listing 18-11. The created secure channel keys shall be reset under the following conditions:

- After successful owner pairing
- When device returns a response other than 9000_h or $61XX_h$ (see Table 5-15)
- A SPAKE2+ REQUEST is sent
- When a tearing of communication occurs
- When the maximum allowed time has expired without successfully terminating owner pairing

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The vehicle needs to start with SPAKE2+ again to establish new keys. Note that the pairing password shall remain the same in those cases. The vehicle shall rotate the pairing password after seven failed attempts.

5.1.4 *WRITE DATA Command*

5.1.4.1 Purpose

This command sends all data needed to generate the Digital Key to the device. It is also used to provide an attestation from the vehicle of the device public key (PK) for key tracking purposes.

5.1.4.2 Definition

The following WRITE DATA command structure shall be used:

command: 84 D4 P1 00 Lc [command_data] [command_mac] 00
response: [response_mac] 90 00

Parameters P1 and P2 are always set to 00, except for the last WRITE DATA command, in which P1 shall be set to 80_h.

The command shall only be allowed to be sent in a secure channel.

The command may return either a general error condition as listed in Table 5-2 or one of the error conditions listed in Table 5-10.

Table 5-10: WRITE DATA Response Error Status Words

Status Word	Comment
6985_h	Command used out of sequence
$6A84_h$	Not enough memory

5.1.4.3 Usage

One or more WRITE DATA commands may be used to write the requested data objects into a buffer in the device.

Table 5-11: Objects for Digital Key Creation

Tag	Length (bytes)	Data Content	Field is
$7F4A_h$	variable	Endpoint creation data, elements from Table 15-13, except the fields listed in the text below	mandatory
$7F4B_h$	variable	DER-encoded X.509 Vehicle Public Key Certificate [K], Listing 5-3	mandatory
$7F4C_h$	variable	DER-encoded X.509 Intermediate Certificate	optional
$7F4D_h$	variable	Mailbox Mapping Table 5-13	mandatory
$7F4E_h$	variable	Device Configuration Table 5-14	mandatory
$5F5F_h$	0	Completion of Digital Key Data Sending	mandatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 5-12: Objects for Device Digital Key Certificate

Tag	Length (bytes)	Data Content	Field is
$5F5A_h$	variable	Attestation of the device PK by the vehicle (opaque)	optional
$5F5F_h$	0	Completion of Digital Key Certificate Sending	mandatory

All required objects of Table 5-11 shall be written to the device in the order given in the table. One or several TLVs are allowed to be written per (sequence of) WRITE DATA command(s).

The TLV $5F5F_h$ shall be written at last to mark the end of the transmitted data sequence. The last WRITE DATA command of this sequence, which contains TLV $5F5F_h$, shall be indicated by setting $P1=80_h$.

All required objects of Table 5-12 shall be written to the device when the vehicle has accepted the device PK. The last WRITE DATA command of this sequence shall be indicated by setting $P1=80_h$.

$5F5F_h$ shall be the last TLV written to mark the end of the transmitted data sequence.

Tag $7F4A_h$ shall contain all data elements from Table 15-13(not including the outer tag $7F27_h$, only the inner tag value $4D_h$, 46_h, etc.), except for the following elements:

- Endpoint identifier (defined by device)
- Instance CA identifier (defined by device)
- Counter limit (deprecated, shall not be used)

If the vehicle_identifier is provided as part of Tag $7F4A_h$, the device shall compare it to the value provided in the Vehicle Public Leaf Certificate [K] and abort the owner pairing if the check fails.

In this version of the specification, the vehicle shall include only one single authorized_PK (inner Tag 49_h). This authorized_PK shall correspond to the Vehicle intermediate CA or the Vehicle OEM CA PK (root).

The maximum command data length shall be 239 bytes: len = [command_data] + [padding] + [command_mac]

len = 239 + 1 + 8 ≦ 255 (ok)

len = 240 + 16 + 8 > 255 (not ok)

[command_data] + [padding] shall be a multiple of the AES block size (16 bytes). The padding scheme is described in [9]. At least one byte padding (80_h) shall be required. The maximum response data length shall be 239 bytes.

The vehicle public key shall be provided as a X.509 certificate signed by the Vehicle OEM CA as described in Listing 5-3.

The X.509 [3] definition of the basic constraints extension is described in Listing A-5.

The X.509 [3] definition of the key usage extension is described in Listing A-4.

Listing 5-1: Vehicle Certificate Extension Schema

```
VehicleCertificateExtensionSchema ::= SEQUENCE
{
  extension_version  INTEGER (1..255)
}
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Listing 5-2: Vehicle Certificate Extension Data

```
vehicle-cert-extension-data VehicleCertificateExtensionSchema ::=
{
  extension_version  1    --value shall be 1
}
```

The Vehicle Public Key Certificate data is described in Listing 5-3. See [3] for details on X.509 v3 certificate format.

Listing 5-3: Vehicle Public Key Certificate Data

```
vehicle-key-cert-data Certificate ::=
{
  tbsCertificate
  {
    version v3, --shall be v3--
    serialNumber ..., --a random integer chosen by the certificate issuer,
    Signature
    {
      algorithm {1 2 840 10045 4 3 2} --OID for ecdsaWithSHA256 (ANSI X9.62 ECDSA algorithm with SHA-256)
    },
    issuer rdnSequence:
    {
      {
        {
          type {2 5 4 3}, --OID for CommonName
          value "..." --shall match the subject of the issuing certificate, shall use PrintableString or UTF8String format

        }
      }
    },
    validity
    {
      notBefore Time: "..." --shall use UTCTime or GeneralizedTime as defined in [3]
      notAfter Time: "..." --shall use UTCTime or GeneralizedTime as defined in [3]
    },
    subject rdnSequence:
    {
      {
        {
          type {2 5 4 3}, --OID for CommonName
          value "Vehicle OEM Identifier" --contains the subject of the certificate, as per Appendix B.2.6
          shall use PrintableString or UTF8String format
        }
      }
    },
    subjectPublicKeyInfo
    {
      algorithm
      {
        algorithm {1 2 840 10045 2 1} --OID for ecPublicKey (ANSI X9.62 public key type)
        parameters {1 2 840 10045 3 1 7} --OID for prime256v1(ANSI X9.62 named elliptic curve)
      },
      subjectPublicKey '04...'H --the public key pre-pended with 04h to indicate uncompressed format
    },
    extensions
    {
      {
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        extnID      {1.3.6.1.4.1.41577.5.1},  --OID for Vehicle Public Key Certificate (see Appendix B.2.2)
        critical    TRUE,
        extnValue   '…'H  --DER encoding for VehicleCertificateExtensionSchema extension as per Listing 5-1
      },
      {
        extnID      {2 5 29 15}, --KeyUsage standard extension
        critical    TRUE,
        extnValue   '03020780'H --DER encoding for KeyUsage, digitalSignature only
      },
      {
        extnID      {2 5 29 19}, --BasicConstraints standard extension
        critical    TRUE,
        extnValue   '3000'H -- DER encoding for cA=FALSE
      },
      {
        extnID      {2 5 29 35}, --OID for AuthorityKeyIdentifier standard extension
        critical    FALSE,
        extnValue   '...'H- DER encoding of an AuthorityKeyIdentifier sequence, containing only a KeyIdentifier element.
-- The KeyIdentifier is an OCTET STRING containing the 160-bit SHA-1 hash of the value of the BIT STRING
subjectPublicKey
                        --from the issuer certificate (excluding the tag, length, and number of unused bits)
      },
      {
        extnID {2 5 29 14}, -- OID for SubjectKeyIdentifier standard extension
        critical FALSE,
        extnValue '…'H --160-bit SHA1 hash of the value of the BIT STRING subjectPublicKey
                        --(excluding the tag, length, and number of unused bits)
      }
    }
  },
  signatureAlgorithm
  {
    algorithm {1 2 840 10045 4 3 2}
  },
  signatureValue '...'H --the certificate signature computed as per [3]
  --ECDSA signature
}
```

The subject field is formatted as described in Appendix B.2.3

The ROUTING_INFORMATION (as used in 11.5) is defined in the common name of the subject field of Listing 5-3, i.e., the subject ID of the Vehicle Public Key Certificate [K] (Appendix B.2.6).

Table 5-13: Mailbox Mapping

Tag	Length (bytes)	Description	Field is
$7F4D_h$	variable	Mailbox mapping	mandatory
$D0_h$	2	SIGNALING_BITMAP_OFFSET as per Table 4-2	mandatory
$D1_h$	2	MAILBOX_CONTENT_END_OFFSET as per Table 4-2	mandatory
$D2_h$	1	IMMOBILIZER_TOKEN_LENGTH as per Table 4-3	optional
$C0_h$	2	Sig-Bit 0: corresponding data offset as per Table 4-2	mandatory
$C1_h$	2	Sig-Bit 1: corresponding data offset as per Table 4-2	conditional

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is
$C2_h$	2	Sig-Bit 2: corresponding data offset as per Table 4-2	conditional
$C3_h$	2	Sig-Bit 3: corresponding data offset as per Table 4-2	conditional
$C4_h$	2	Sig-Bit 4: corresponding data offset as per Table 4-2	conditional
$C5_h$	2	Sig-Bit 5: corresponding data offset as per Table 4-2	conditional
$C6_h$	2	Sig-Bit 6: corresponding data offset as per Table 4-2	conditional
$C7_h$	2	Sig-Bit 7: corresponding data offset as per Table 4-2	conditional

Conditional fields shall be present when the data element designated by the corresponding Sig-Bit is used.

Table 5-14 provides all supplementary configuration information to the device.

Table 5-14: Device Configuration Data

Tag	Length (bytes)	Description	Field is
$7F4E_h$	variable	Device configuration	mandatory
$7F49_h$	variable	7F49 template is used to provide wireless capability (NFC/UWB) and related data; this template shall be included by vehicle. If this tag is not present, only NFC is supported. (equivalent to only 5F50 tag present) See Table 19-86 for details	optional
$7F60_h$	variable	SHARING_CONFIGURATION (for key sharing, see Section 11)	optional
DA_h	1	If this tag is not present, the immobilizer token and slot identifier are retrieved from vehicle. If this tag is present, the following values are assigned: 00_h immobilizer token and slot identifiers are retrieved from vehicle 01_h immobilizer token and slot identifiers are retrieved online 02_h no immobilizer token, and slot identifiers are retrieved from vehicle 03_h no immobilizer token, and slot identifiers are retrieved online 04_h immobilizer token and slot identifiers for owner are retrieved from vehicle and friend immobilizer token and slot identifiers are retrieved online. If this setting is used during owner pairing, the owner device shall use 01_h for friend sharing. 05_h no immobilizer token but with slot ID retrieved from vehicle for owner device; no immobilizer token but with slot ID retrieved online for friend device. If this setting is used during owner pairing the owner device shall use 03_h for friend sharing.	optional

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is
DB_h	0	If this tag is present, owner and friend devices have to obtain a key tracking receipt (owner during Phase 4 as per Section 6.3.5, and friend during Step 7 as per Section 11.8.3.1)	optional
DC_h	0	If this tag is present, vehicle manufacturer supports online certificate delivery feature for key sharing as described in Section 0.	optional
$D7_h$	1	2nd Factor activation required (00_h=not required, 01_h=required for non-approved sharing methods, other=RFU)	optional
$D8_h$	1	SHARING_PASSWORD_LENGTH (vehicle OEM specific as per appendix B.1 If absent, sharing password is not required.	optional
$D9_h$	variable	Supported Digital Key profile list as described in Section 11	optional
Tag $4A_h$ and $4B_h$ of Table 15-52 (SETUP ENDPOINT Command Payload)			optional

If tag $7F60_h$ is absent, the default SHARING_CONFIGURATION is

- DA_h: 00_h
- DB_h: present
- DC_h: not present

If tag $D7_h$ is absent, then no sharing password is required.

Note: Information in Tag $D7_h$ could be overwritten by the presence of the activationRequired field in the key tracking response and/or eventNotification.

Note: Information in Tag $D8_h$ could be overwritten by the presence of the sharingPasswordLength field in activationOptions of the uiBundle sent in the key tracking response and/or eventNotification (Section 17.8.18)

The LONG_TERM_SHARED_SECRET shall be used to generate the password used for key sharing (see LONG_TERM_SHARED_SECRET in Section 11). The LONG_TERM_SHARED_SECRET shall be derived from the SPAKE2+ shared secret (K).

The sharing password length shall be used as a parameter for the derivation algorithm. The supported key profiles list defines the key profiles for key sharing that are supported by the vehicle. The profiles are defined as a list of 1-byte enumeration values corresponding to Table 11-4.

If neither tag $4A_h$ nor $4B_h$ is included in the Device Configuration Data sent by vehicle to device using WRITE DATA command (see step 3 in Figure 6-3), then device shall transmit no data in the AUTH1 response to the vehicle (see step 6 in Figure 6-10 and Section 6.3.4.2).

If tag $4A_h$ or $4B_h$ or both are included in the Device Configuration Data sent by vehicle to device using WRITE DATA command (see step 3 in Figure 6-3), then device shall transmit the corresponding data in the AUTH1 response to the vehicle (see step 6 in Figure 6-8).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

5.1.5 *GET DATA Command*

5.1.5.1 Purpose

This command shall continue to use the established session keys to retrieve all data needed to verify the Digital Key created by the Digital Key framework in the Digital Key applet instance.

5.1.5.2 Definition

command: 84 CA 00 00 Lc [encrypted_tag] [command_mac] 00
response: [response_payload] [response_mac] 90 00 or 61XX

Only one tag shall be requested per GET DATA command. The tags may be requested in arbitrary order. Elements may be omitted if they are not required.

X.509 certificates shall not be wrapped into additional TLV structures, but tags shall be used to refer to the certificates in the command tag list, as indicated below.

Le shall be set to 00_h as the exact length of data depends on the variable certificate field sizes.

GET RESPONSE shall be used if the response status word indicates that more data is to be retrieved.

This command may return either a general error condition as listed in Table 5-2 or one of the error conditions listed in Table 5-15.

Table 5-15: GET DATA Response Error Status Words

Status Word	Comment
$61XX_h$	XX indicates the number of response bytes still available (use GET RESPONSE)
6985_h	Command used out of sequence
$6A88_h$	Reference data not found
6400_h	Digital Key could not be created
6402_h	Requested elements not available

5.1.5.3 Usage

The command requests the transfer of the required data element by providing the corresponding tag. The response data length shall not exceed 239 bytes, as the maximum length of an APDU response shall not exceed 258 bytes (including status word):

len = [response_data] + [padding] + [response_mac] + [status_word]

len = 239 + 1 + 8 + 2 ≦ 258 (ok)

len = 240 + 16 + 8 + 2 > 258 (not ok)

[response_data] + [padding] equals a multiple of the AES block size (16 bytes). At least one byte padding (80_h) is required. The maximum response data length is then 239 bytes.

See Section 18.4.12 for calculation of [command_mac] and [response_mac].

Each element shall be requested in a separate GET DATA command. GET RESPONSE shall be used if the returned data exceeds the GET DATA response data maximum length.

When the response is a TLV structure (e.g., certificate) then the response shall not be wrapped in the requested tag. Otherwise (e.g., string), the response shall be wrapped in the requested tag.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

If all the remaining bytes cannot be sent in the current response, the status word $61XX_h$ shall be used instead of 9000_h.

If XX is between 01_h and EF_h, XX number of bytes (1..239) are still available and shall be sent in the next response (which shall be the last one). Note that XX does not include padding length.

If XX equals 00_h , not all remaining bytes can be sent in the next response.

Values between $F0_h$ and FF_h are not allowed for XX.

Table 5-16: GET DATA Response Decrypted Payload for tag $7F20_h$

Description
Vehicle OEM signed Device OEM CA Certificate (DeviceOEM.PK.VehicleOEMCert), [F] as per Listing 15-12

Table 5-17: GET DATA Response Decrypted Payload for tag $7F22_h$

Description
Device OEM signed Instance CA Certificate, [E] as per Listing 15-15

Table 5-18: GET DATA Response Decrypted Payload for tag $7F24_h$

Description
Digital Key Certificate signed by Instance CA (DigitalKey.DKCert) as per Listing 15-5

Table 5-19: GET DATA Response Decrypted Payload for Tag $D3_h$

Tag	Length (bytes)	Description
$D3_h$	1–30	Friendly name of the owner key (UTF-8) with length of 1 to 30 bytes

The friendly name of the owner Digital Key should be assigned during owner pairing. The friendly name shall be a UTF-8 encoded string. The owner should be allowed to edit the name for identification and personalization. For privacy reasons, the friendly name should not contain private information, such as the full name of the owner.

5.1.6 *GET RESPONSE Command*

5.1.6.1 Purpose

This command shall only be used to retrieve remaining data that could not be fully transmitted in the GET DATA response. This command shall only be accepted if the GET DATA response received previously was with a status word
$61XX_h$

5.1.6.2 Definition

command: 84 C0 00 00 Lc [command_mac] 00
response: [response_payload] [response_mac] 90 00 or 61XX

When the response data is not fully transmitted in the response payload of GET RESPONSE, a status word $61XX_h$ shall be returned. XX_h indicates the number of response bytes still available. When a response data is completely transmitted in the response payload of GET RESPONSE and no more remaining data is to be returned, a status word 9000_h is returned.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

This command may return either a general error condition as listed in Table 5-2 or one of the error conditions listed in Table 5-20.

Table 5-20: GET RESPONSE Response Error Status Words

Status Word	Comment
6985_h	Command used out of sequence

5.1.6.3 Usage

One or more GET RESPONSE commands shall only be used to retrieve all response data that could not be fully transmitted in the GET DATA response.

The command shall only be accepted when sent immediately after a GET DATA or GET RESPONSE response with the status word $61XX_h$.

The vehicle shall send the last GET RESPONSE command when a GET RESPONSE response with XX of 1..239 is received. Values between $F0_h$ and FF_h are not allowed for XX. When XX is 00_h, at least one more GET RESPONSE command shall follow.

5.1.7 *OP CONTROL FLOW Command*

5.1.7.1 Purpose

With this command, the vehicle may control the flow at predefined points and abort the flow at any time while giving a status to the device.

"Continue" and "End" shall be used where specified in the flow diagrams. Status information about the state, successful or unsuccessful end, etc. shall be provided.

5.1.7.2 Definition

This command is always sent outside of the secure channel.

command: 80 3C [Table 5-21**][**Table 5-22 **or** Table 5-23 **or** Table 5-24**]**
response: 90 00

This command may return a general error condition, as listed in Table 5-2.

5.1.7.3 Usage

The vehicle shall not consider the session as terminated before it has received the response to the OP CONTROL FLOW command (applicable for $P1=11_h$ or $P1=12_h$).

The OP CONTROL FLOW response should be kept as short as possible to minimize the risk of false positives when a tearing happens on the response.

When aborted, the vehicle shall perform the NFC link teardown procedure, followed by the NFC reset procedure, before it restarts owner pairing or any other transaction.

P1 indicates the main action triggered by the command; P2 provides the reason code.

The Digital Key Framework shall ignore unknown P1/P2 values, if a P1 value is not known it shall be treated as $P1=10_h$ (continue flow) and ignored.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 5-21: OP CONTROL FLOW P1 Parameters

P1 Value	Description
10_h	continue flow, see reason code for details
11_h	end flow, see reason code for details
12_h	abort flow, see reason code for details

Table 5-22: OP CONTROL FLOW P2 Parameters for P1=10_h (continue)

P2 Value	Description
01_h	key creation data transmitted to device
02_h	key certificate chain received by vehicle
03_h	Waiting for user confirmation on vehicle UI
$0F_h$	Vehicle waiting time extension, keep busy, no action on device side

Table 5-23: OP CONTROL FLOW P2 Parameters for P1=11 (end with success)

P2 Value	Description
11_h	successful end of key creation and verification, key not tracked by vehicle

Table 5-24: OP CONTROL FLOW P2 Parameters for P1=12 (end with failure)

P2 Value	Description
00_h	no specific reason
01_h	no matching SPAKE2+ version found between vehicle and device
02_h	no matching Digital Key applet protocol version found between vehicle and device
03_h	pairing failed due to timeout of timer 1
04_h	pairing failed due to timeout of timer 2
05_h	pairing failed due to timeout of timer 3
06_h	pairing failed due to timeout of timer 4
07_h	pairing failed due to timeout of timer x (spare, not used)
08_h	preconditions for owner pairing not fulfilled
09_h	evidence verification on vehicle side failed
$0A_h$	wrong Digital Key configuration
$0B_h$	certificate verification failed
$0C_h$	curve point X zero or invalid
$0D_h$	error counter limit for wrong pairing password is exceeded, new pairing password needed
$7F_h$	pairing failed due to response data or format error of previous command

The option to end the flow refers to the regular end of the transaction and the possible outcomes (successful, not successful) and the UI actions.

The option to abort the flow refers to the vehicle aborting the flow before its regular end due to unexpected or erroneous behavior.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

5.2 Data Elements

5.2.1 *Certificates*

Table 5-25: Certificates

Tag	Certificate	Reference
$7F20_h$	Device OEM Certificate signed by Vehicle OEM CA	Listing 15-12
$7F22_h$	Instance CA Certificate signed by Device OEM CA	Listing 15-15
$7F24_h$	Digital Key Certificate signed by Instance CA	Listing 15-5
$7F4B_h$	Vehicle Public Key Certificate	Listing 5-3
$7F4C_h$	Vehicle OEM Intermediate Certificate	optional, (Vehicle OEM specific)

The Vehicle OEM intermediate certificate may be required by the Vehicle OEM if the Vehicle OEM CA is not able to sign the Vehicle Public Key Certificate directly.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

6 OWNER PAIRING

6.1 Overview

The owner pairing flow is operated by the Digital Key framework running on the device. As shown in Figure 2-2, the Digital Key framework uses the APDU commands described in Section 5 to manage the configuration of the Digital Key, protected by the SE.

The SE operating system provides the root of trust, which is the starting point in the trust chain as defined in Section 16.1.

The vehicle is able to select the Digital Key applet over NFC using its AID and to select the Digital Key framework using the corresponding AID. The NFC controller may be reconfigured for changing the routing of the communication from the SE to the Digital Key framework and vice versa, based on the selected AID.

A new owner device pairing flow or owner device change does not imply an implicit unpairing, i.e., a new device owner pairing flow only changes the owner's key. Existing shared/friend keys that are already paired and vehicle public keys are not necessarily impacted.

The Digital Key applet instance shall be available on the SE before the time of owner pairing execution.

6.2 Keys and Data

This section defines the key and data elements used in owner pairing:

- **device.PK/device.SK**: Long term key pair generated by device during Digital Key creation. One per Digital Key.
- **vehicle.PK/vehicle.SK**: Key pair generated by the vehicle. Same for all Digital Keys of the vehicle. The lifecycle of this key pair is defined and managed by the Vehicle OEM and is out of scope of this specification.
- **Pairing password**: SPAKE2+ pairing password entered into the device; eight numeric digits (0–9) in UTF-8 format provided by the Vehicle OEM account to authenticate the owner.
- **Kenc**: Derived symmetric key (from SPAKE2+ shared secret), used to encrypt confidential command and response payloads.
- **Kmac**: Derived symmetric key (from SPAKE2+ shared secret), used to calculate command MACs.
- **Krmac**: Derived symmetric key (from SPAKE2+ shared secret), used to calculate response MACs.

6.3 Owner Pairing Implementation

This section describes the phases of the owner pairing flow, which include:

- **Phase 0**: Preparation (see Section 6.3.1)
- **Phase 1**: Initiate pairing procedure on vehicle and device (see Section 6.3.2)
- **Phase 2**: First session with NFC reader (see Section 6.3.3)

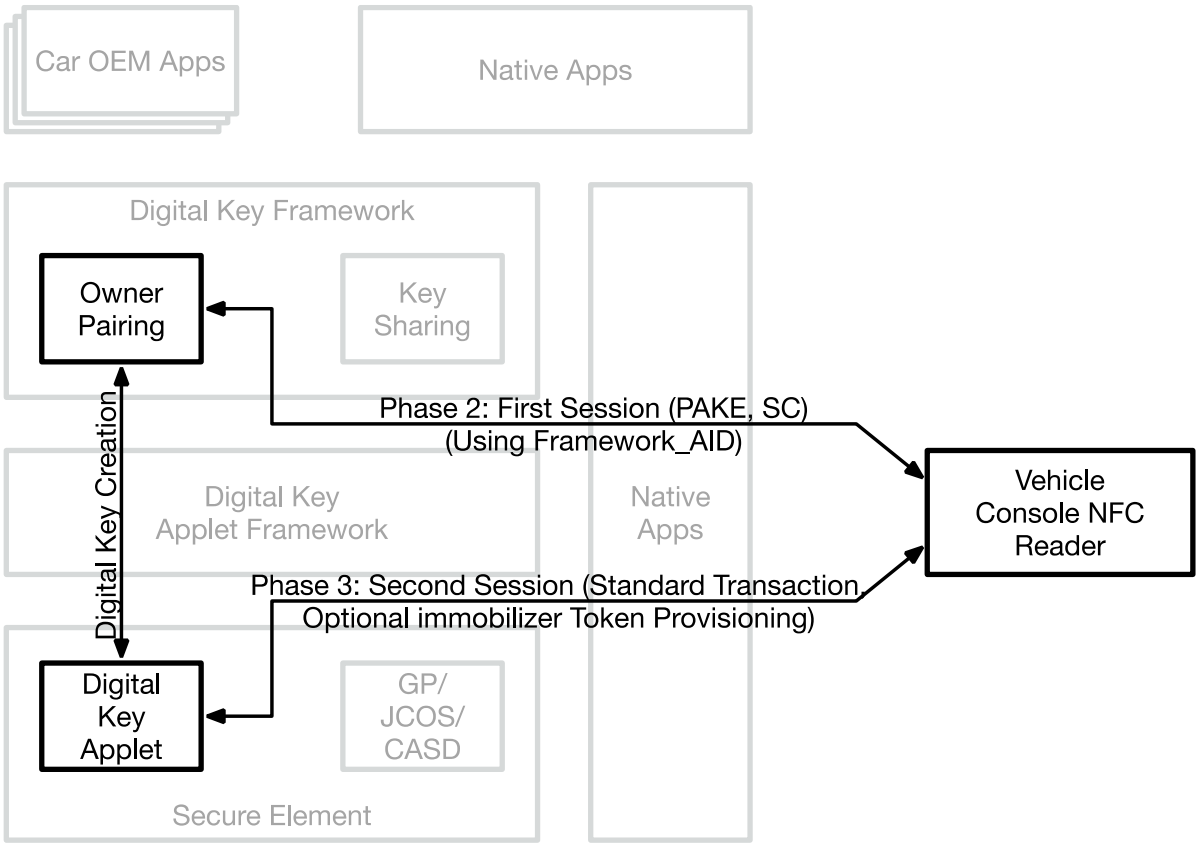
Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- **Phase 3**: Second session with NFC reader (see Section 6.3.4)
- **Phase 4**: Finalization of pairing procedure (see Section 6.3.5)

The owner pairing flow consists of two sessions: the first session (Phase 2), executed with the Digital Key framework, and the second session (Phase 3) with the Digital Key applet, as shown in Figure 6-1. In owner pairing Phase 2, the vehicle configures whether the owner device has to retrieve the immobilizer tokens online from the Vehicle OEM Server or from the vehicle. If online retrieval is configured, the owner immobilizer token is not transferred in the second session with the NFC reader but within the Key Tracking Request/Response to/from the Vehicle OEM Server over the owner Device OEM Server during or after the owner pairing. Figure 6-1 describes the owner pairing process whereby the immobilizer tokens are retrieved from the vehicle.

The first session consists of two NFC transactions. The first transaction is to negotiate protocol versions, execute the SPAKE2+, and transmit all key creation data to the device. The second transaction, which is executed after the creation of the Digital Key in the device, provides the creation attestation and certificate chain to the vehicle.

Figure 6-1: Owner Pairing NFC Exchanges



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

6.3.1 *Phase 0: Preparation*

6.3.1.1 Device Preparation

It is assumed that the device has already installed a Digital Key applet and created an Instance CA per Vehicle OEM prior to owner pairing operation. The Digital Key framework is updated with the list of Vehicle OEM partners.

The Instance CA Certificates are obtained by the Digital Key framework.

See Listing 15-15 for the description of the Instance CA Certificate [E]. All signatures are generated as described in Section 18.4.10.

6.3.1.2 Vehicle Provisioning

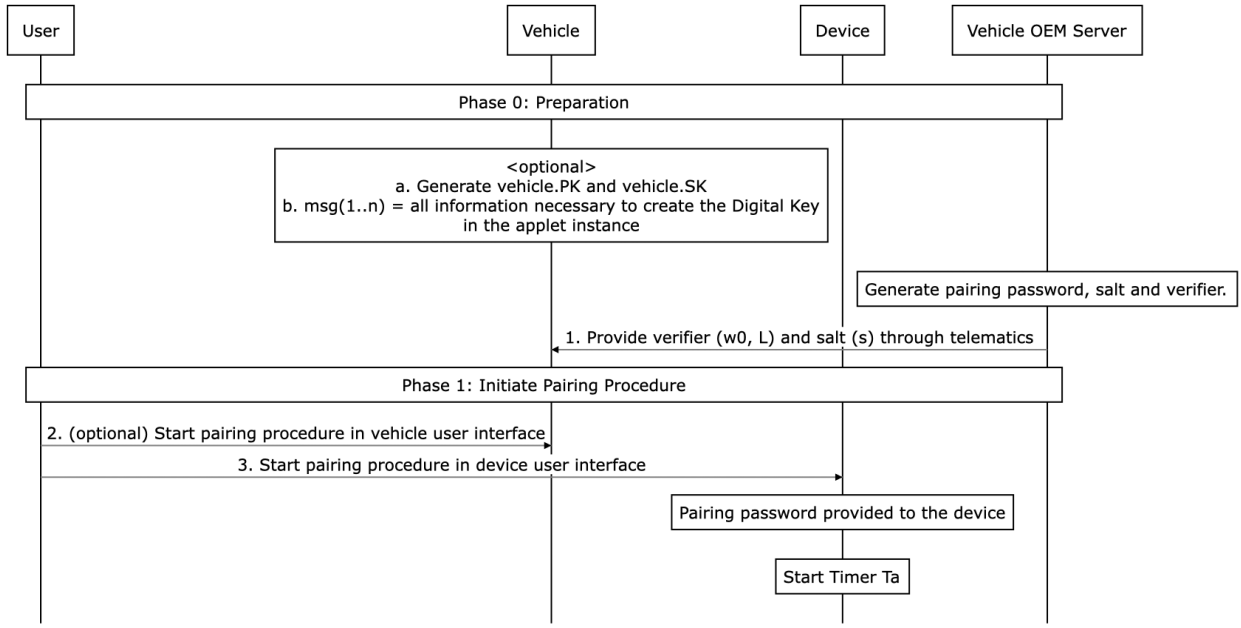
The owner pairing secure channel creation is based on the SPAKE2+ protocol, which is described in Section 18.4.1. The protocol binds the device to the user and is resistant against eavesdropping and MITM attacks. See [10] for more information about the computational and security aspects of the protocol.

Before the owner pairing procedure starts, the Vehicle OEM Server generates the pairing password for the device and the password verifier for the vehicle as described in Listing 18-1, and sends the verifier and salt through a Vehicle OEM proprietary secure channel to the vehicle, as shown in Step 1 of Figure 6-2.

6.3.2 *Phase 1: Initiate Pairing Procedure*

On the vehicle side, the pairing procedure initiation is under the responsibility of the Vehicle OEM. Appropriate owner pairing preconditions need to be met, e.g., the presence of a key fob.

Figure 6-2: Owner Pairing Flow - Phase 0/1: Preparation/Initiation



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The vehicle is either set into pairing mode by the user or tries to select the framework AID on the console NFC reader in the vehicle as long as no owner device is successfully paired.

To start owner pairing mode (Step 3 of Figure 6-2), the device receives the password, either through user input, a URL (see Section 6.3.7), or through an API directly from the Vehicle OEM app, if installed.

The device shall turn off the NFC interface after the pairing procedure has been started in the device (step 3 of Figure 6-2). The NFC link shall be started again after the user has entered the pairing password, or the user interface is dismissed.

NOTE: The deactivation of the NFC interface is introduced to stop additional polling due to external influence on the device or vehicle. For example, with activated NFC interface, while waiting for the input of the User (Step 3 of Figure 6-2), due to movements of the device the vehicle could restart the polling again. This could lead to a restart of the owner pairing while the first one is still running.

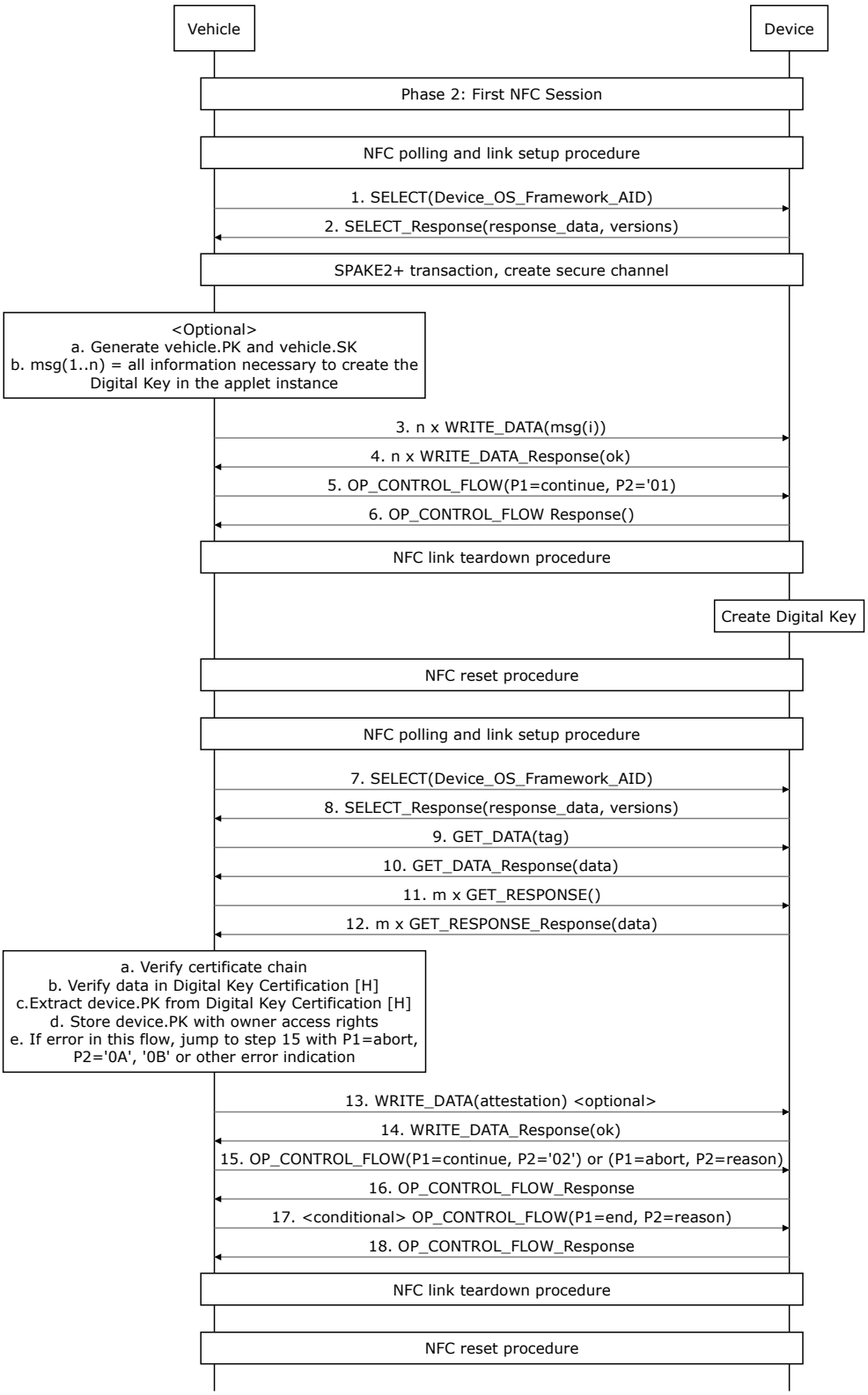
6.3.3 *Phase 2: First Session with NFC Reader*

The first NFC session consists of two distinct NFC transactions.

The first transaction negotiates the protocol versions (see Section 6.3.3.8), mutually authenticates both device and vehicle, establishes a secure channel using SPAKE2+, and transmits all key creation data to the device, as shown in Figure 6-3.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 6-3: Owner Pairing Flow - Phase 2: First NFC Session



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The SPAKE2+ creates a symmetric session key pair used to establish a secure channel between vehicle and device.

Before the second NFC transaction starts, the device creates the Device Key.

In the second NFC transaction, the vehicle reads the key creation data from the device, verifies it and, if successful, stores the device public key. The NFC reset procedure is performed at the end of each transaction.

The following steps or sequence of events takes place in the first NFC session:

6.3.3.1 Step 1: Digital Key Framework Selection

When the device is communicating to the console NFC reader in the vehicle, the vehicle shall select the Digital Key framework using its AID, through the SELECT command. If the vehicle selects the Digital Key Applet AID before the framework AID, then the device shall respond to SELECT Digital Key Applet AID with Status Word (SW) = $6A82_h$. The device shall return all supported SPAKE2+ versions and all Digital Key applet protocol versions to the vehicle through SELECT Response command. This determines the SPAKE2+ version (used for creating secure channel) and Digital Key applet protocol version (for key sharing) to be used on both sides.

The version information shall be checked for compatibility on the vehicle side before continuing as described in Section 6.3.3.8.

The SELECT command is described in Section 5.1.1.

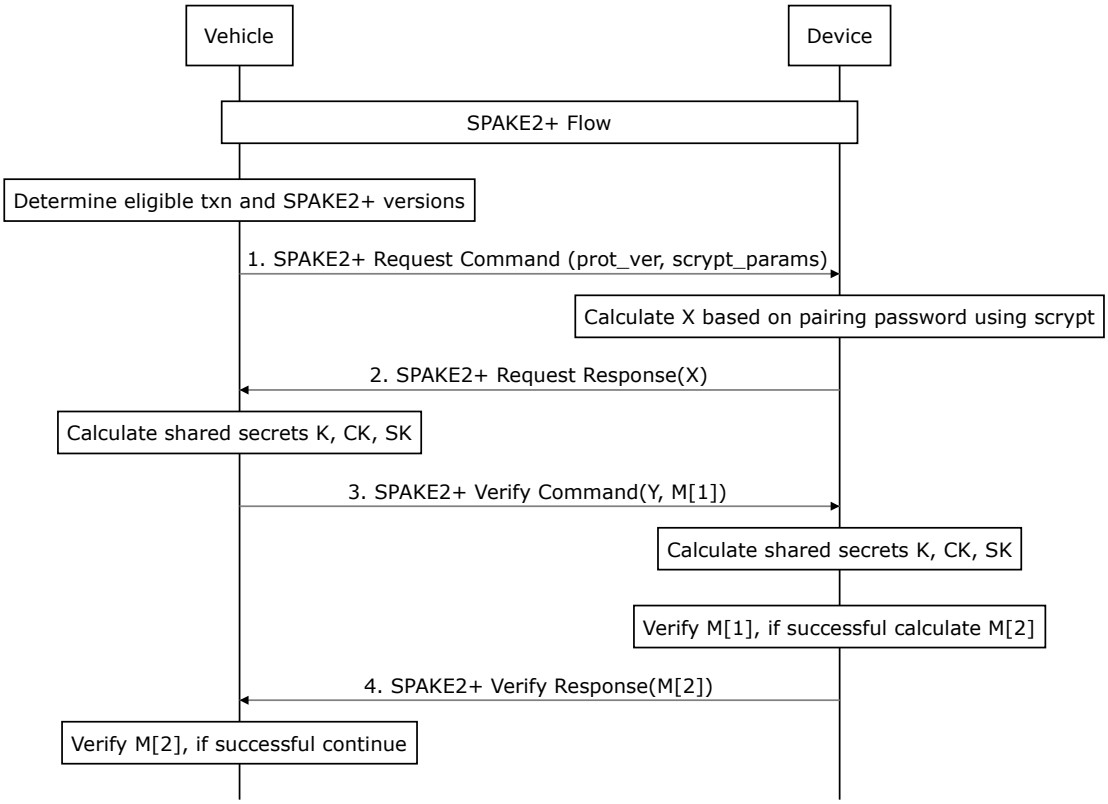
6.3.3.2 Step 2 and 2a: SPAKE2+ Transaction

The vehicle shall select and send to the device the SPAKE2+ protocol version to be used and the list of all supported Digital Key applet protocol versions (as described in Section 6.3.3.8).

The SPAKE2+ transaction is described in [10].

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 6-4: SPAKE2+ Flow



The SPAKE2+ transaction is shown in Figure 6-4. The SPAKE2+ transaction establishes a secure channel for the data exchange between vehicle and device. The secure channel shall remain active across the RF resets.

The secure channel is terminated when the OP CONTROL FLOW command indicating success is sent (Step 17 of Figure 6-3), when an OP CONTROL FLOW command indicates an error, or after any other abortion of the flow. An incorrect decryption or MAC value on APDU commands also terminates the secure channel.

When the vehicle is not ready for owner pairing, it shall use the OP CONTROL FLOW to abort and indicate the condition to the user, such as “preconditions for owner pairing not fulfilled.”

Note: Only commands with the appropriate class byte indication (bit 2 = 1) are part of the secure channel.

6.3.3.3 Steps 3 to 4: WRITE DATA

The vehicle shall send all data required to create a Digital Key through the secure channel to the device using multiple WRITE DATA commands (see Figure 6-5 and Figure 6-6).

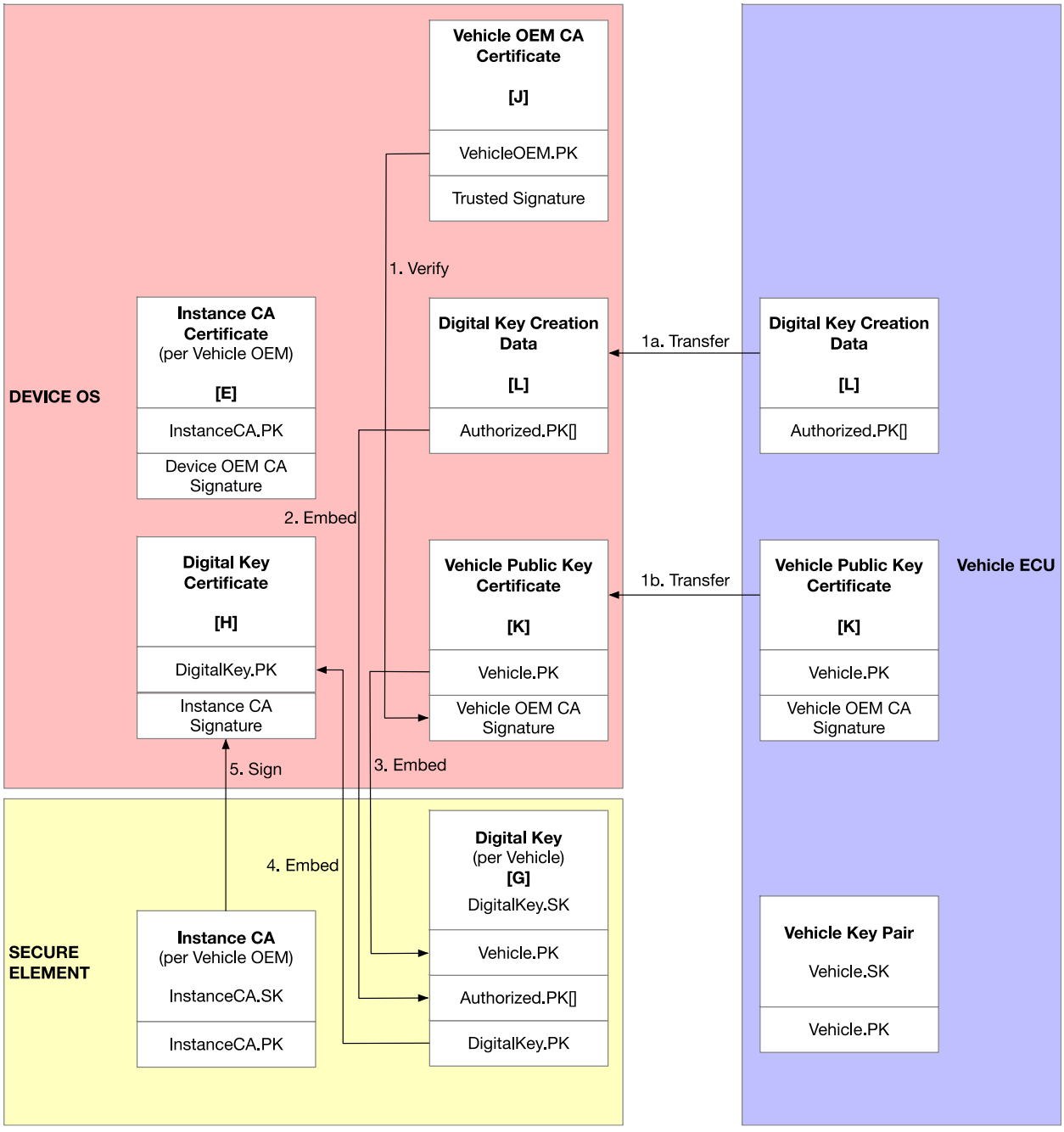
After successful reception, the device shall verify the Vehicle Public Key Certificate [K] as described in Section 6.3.3.10, using one of the following:

- Vehicle OEM CA Certificate [J] (see Figure 6-5)
- Vehicle OEM CA Certificate [M] signed by Device OEM CA [D] (see Figure 6-6)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

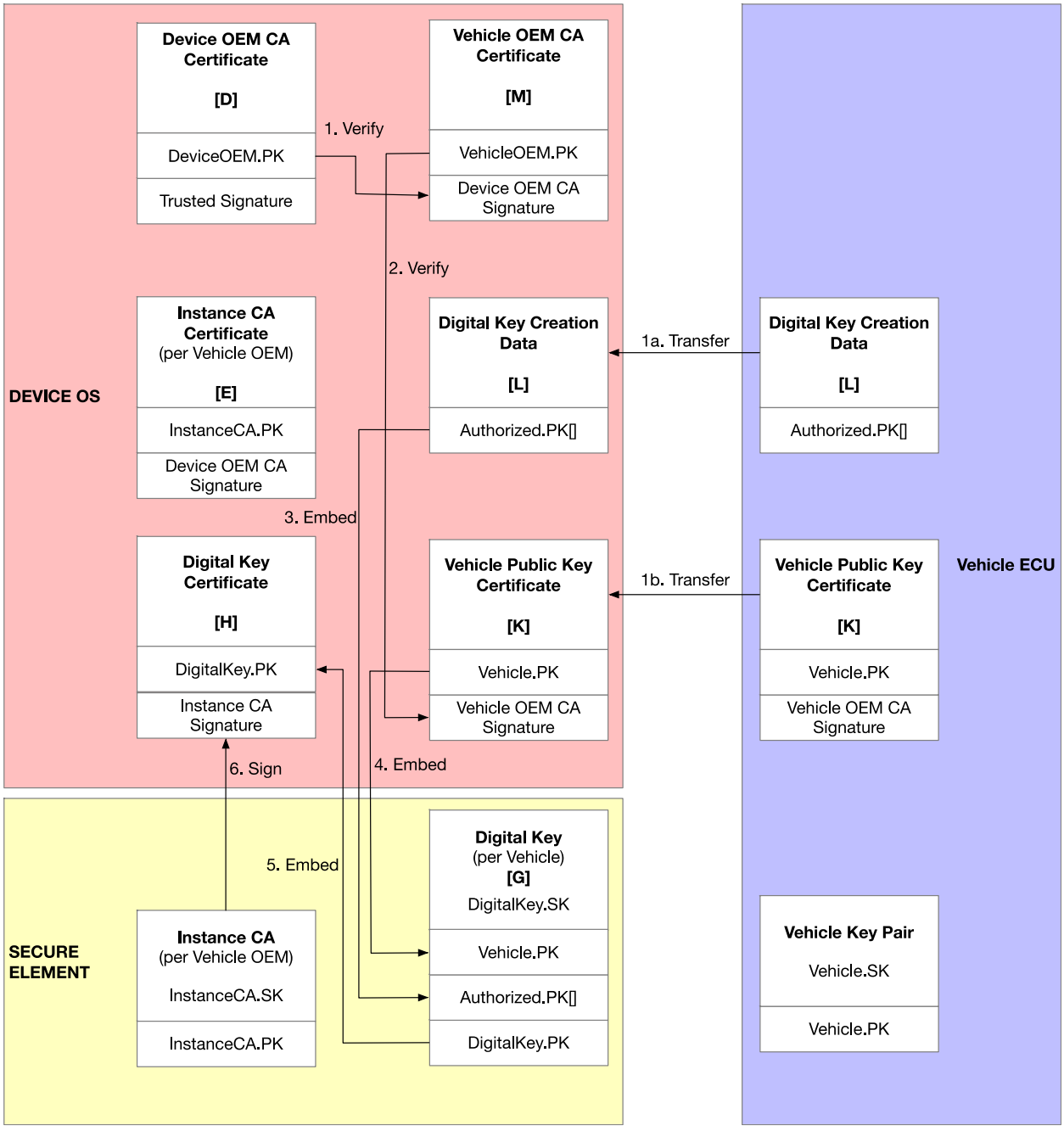
In case of SE-centric applet model implementation, the Digital Key framework may omit the verification since it is conducted by the Digital Key applet during Endpoint creation as described in Section 15.3.2.4.

Figure 6-5: Key Creation Data Transfer to Device



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 6-6: Key Creation Data Transfer to Device



Certificate details are specified in Section 16.

6.3.3.4 Steps 5 to 6: OP CONTROL FLOW

The OP CONTROL FLOW command indicates that all data has been transmitted without error.

The device OS then switches off card emulation and creates the Digital Key as described in section 6.3.3.9.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

6.3.3.5 Step 9 to 12: GETDATA

The derived secure channel shall continue to be used in the second session.

Before sending the GET DATA command, the vehicle shall perform the NFC polling and link setup procedure and reselect the device OS framework AID using SELECT command.

The vehicle shall poll for the device to respond before considering the key creation as failed. See also section 6.3.3.12. For timeouts in case the device is removed from the reader or other connectivity issues occur, see Section 6.3.6.

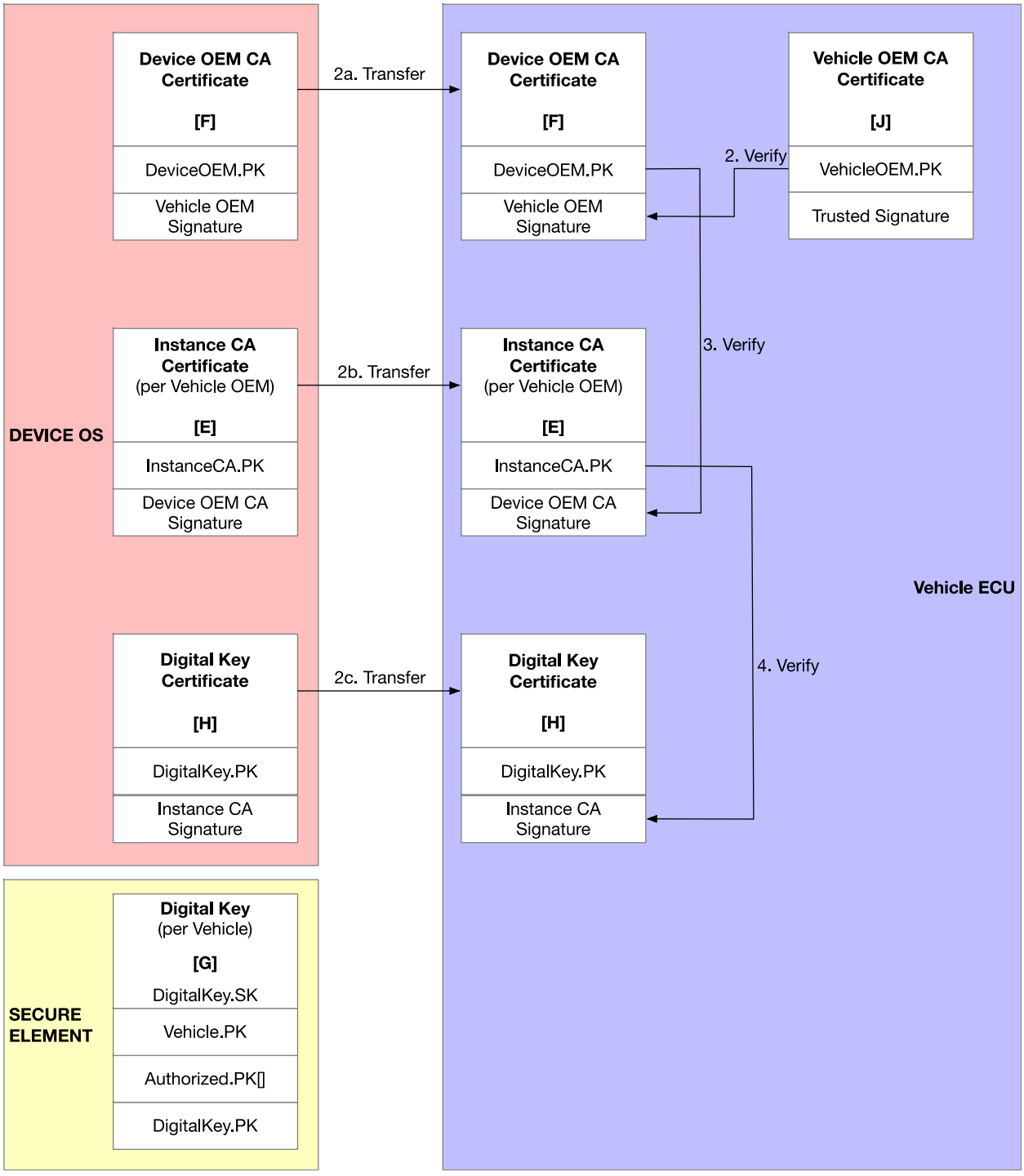
The vehicle shall send GET DATA command to request the Device OEM CA Certificate signed by the Vehicle OEM [F] (see Listing 15-13), the Instance CA Certificate signed by the Device OEM CA [E] (Listing 15-16), and the Digital Key Certificate from the Instance CA [H] (see Listing 15-5).

The vehicle shall verify all certificates and the relevant data elements as described in Section 6.3.3.11. If all the steps are successful, the vehicle shall store the device public key as the owner's public key.

The command GET DATA is described in Section 5.1.5.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 6-7: Key Creation Info Retrieval by Vehicle



Certificate details are specified in Section 16.

6.3.3.6 Step 13 to 14: WRITE DATA

If all verifications were successful, the vehicle may write back to the device with an opaque attestation (see Table 5-12) using WRITE DATA command to confirm the device PK that has

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

been presented to the vehicle. This attestation shall be sent to the KTS when registering the key. The device shall respond with a WRITE DATA response.

If any verification failed, no attestation is written by the vehicle. The vehicle shall send a OP CONTROL FLOW command to the device to abort, with an appropriate error indication as defined in Table 15-27.

6.3.3.7 Step 15 to 18: OP CONTROL FLOW

If vehicle has received all key certificates without error, then it shall send OP CONTROL FLOW ($P1=10_h$, $P2=02_h$) in step 15. Otherwise, the vehicle shall send OP CONTROL FLOW ($P1=12_h$, P2=reason) to abort the owner pairing due to error as indicated by the P2 value.

If the OP CONTROL FLOW in step 15 is with $P1=10_h$, $P2=02_h$, then the vehicle shall send OP CONTROL FLOW ($P1=11_h$, $P2=11_h$) in step 17 to end the owner pairing Phase 2 flow.

6.3.3.8 Checking the Protocol Version

Device and vehicle may support different versions of SPAKE2+ and Digital Key applet protocols. Device and vehicle shall support older but not deprecated protocol versions.

The owner device shall first return all the supported versions of SPAKE2+ and Digital Key applet protocols in the SELECT response. The vehicle shall return the highest matching version of the SPAKE2+ protocol and the agreed version of the Digital Key applet protocol at the beginning of the list, followed by all lower versions supported, ordered from the highest to the lowest. The list of all the supported Digital Key applet protocol versions allows the vehicle and owner device to find a different but compatible version on the friend device for key sharing. An example is given below.

When no matching version can be determined, the owner pairing procedure shall be aborted. In this case, the vehicle should indicate this on its UI and shall send an error message to the device using the OP CONTROL FLOW command with “abort” indication.

Example of supported Digital Key applet protocol versions:

The Digital Key applet protocol versions supported by the vehicle (m supported Digital Key applet protocol versions (ver.high | ver.low)) as contained in SPAKE2+ REQUEST command are received by the device:

- 0104_h
- 0103_h
- 0102_h
- 0101_h
- 0100_h

The Digital Key applet protocol versions supported by the device (m supported Digital Key applet protocol versions (ver.high | ver.low)) as contained in SELECT RESPONSE are received by the vehicle

- 0103_h
- 0102_h
- 0101_h
- 0100_h

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The SELECT RESPONSE APDU would have a TLV with Tag $5C_h$: 5C 08 0103010201010100. The SPAKE2+ REQUEST command would have a TLV with Tag $5C_h$: 5C 0A 01030104010201010100.

6.3.3.9 Creating the Digital Key

The Digital Key framework creates a Digital Key in the SE using the CREATE ENDPOINT command (see Section 15.3.2.4) with the following parameters:

- **vehicle identifier**: Assigned by the vehicle. It is recommended to apply 2 bytes Vehicle Brand identifier (see [35]) and 6 bytes unique identifier (within Vehicle OEM) to the vehicle identifier. As the vehicle identifier is transmitted in the AUTH0 command, it shall be changed when the owner changes (i.e., when the vehicle goes through an "unpaired" state; see Section 2.7 for a description of pairing states) if needed for vehicle privacy.
- **endpoint identifier**: The common name in the subject identifier in the related certificate. It is used to identify the Digital Key. The endpoint identifier is unique for each Digital Key in an applet instance. The format is defined in Appendix B.1.
- **Instance CA identifier**: Determines the Instance CA which signs the Digital Key Certificate after key creation. The value shall match exactly the common name in the **subject identifier of the Instance CA** to be associated (see Listing 15-16). The format is described in Appendix B.
- **Digital Key option group 1 and 2**: Should be set according to the Vehicle OEM policy.
- **protocol version**: The agreed upon Digital Key applet protocol version for the Digital Key.
- **vehicle public key**: The public key of the vehicle. For privacy reasons the key may change when the owner changes (i.e., when the vehicle is going through the "unpaired" state; see Section 2.7).
- **authorized.PK[]**: This array shall contain the Vehicle OEM CA PK. Note that the authorized public key shall not be updated once the Digital Key is created.
- **confidential mailbox size**: Should be set according to the Digital Key structure configuration and the number of immobilizer tokens to be stored, as defined by the Vehicle OEM. When immobilizer tokens are not needed, the size shall be set to 0. The Digital Key framework shall calculate the size of the confidential mailbox as follows:

  confidential mailbox size = (SLOT_IDENTIFIERS_OFFSET – SLOT_IDENTIFIER_BITMAP_OFFSET) × 8 × IMMOBILIZER_TOKEN_LENGTH
- **private mailbox size**: Should be set according to the Digital Key structure and attestation sizes, as defined by the Vehicle OEM and provided by the vehicle in the WRITE DATA command (see Section 4.3.1).
- **slot identifier**: Provided by the vehicle. It is used for Digital Key identification and anti-replay of the deleted Digital Keys as described in Section 13. The vehicle shall provide slot identifier for owner during owner pairing. Depending on vehicle OEM's implementation, the slot identifiers for sharing (for friend) may be retrieved by the owner from vehicle or by the friend online from Vehicle OEM server.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- **counter limit**: Deprecated, shall not be provided.

An owner slot identifier shall in all cases (slot identifier retrieved from vehicle or online) be provided by the vehicle in the endpoint creation data. The value provided here is used for the standard transaction in Phase 3 of owner pairing.

If a Digital Key corresponding to the vehicle identifier already exists in the device, the framework shall terminate the existing Digital Key and send termination attestation along with the Vehicle OEM proprietary data to KTS (if applicable) before creating a new Digital Key.

6.3.3.10 Verifying the Vehicle Public Key Certificate Chain

The device shall parse the certificate chain listed below and validate the critical data elements and the signature before accepting the vehicle public key:

- Vehicle Public Key Certificate format [K]
- Intermediate Certificate (optional)
- Vehicle OEM CA Certificate [J] or Device OEM CA signed Vehicle OEM CA Certificate [M]

All certificates shall be in X.509/ASN.1 format and shall be DER encoded.

The formats of the subject identifier and issuer identifier are defined in Appendix B.1After successful verification of the vehicle public Key Certificate Chain, the owner pairing Phase 2 is completed.

6.3.3.11 Verifying the Endpoint Creation Attestation Chain

The vehicle shall parse the certificate chain listed below and validate all data elements, following the rules defined in Section 15, before accepting the device public key:

- Device OEM CA Certificate (signed by Vehicle OEM) [F]
- Instance CA Certificate (signed by Device OEM) [E]
- Digital Key Certificate [H]

All certificates shall be in X.509/ASN.1 format and shall be DER encoded.

6.3.3.12 Error Handling for the first NFC session

The following errors may appear in the first session of the NFC pairing flow:

- General errors
  - Communication error
    - RF transmission error
    - owner removes device before end of protocol flow, i.e., no deselect command received before link loss
  - commands not received in correct order
  - unknown command received
  - device sends other error status word
  - implementation bug leading to timeout
- SELECT command failure
  - owner pairing not activated on device

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- SPAKE2+ protocol failure
  - wrong password entered on device
  - maximum number of pairing attempts reached (enforced by the vehicle)
  - protocol version mismatch
  - vehicle sends OP CONTROL FLOW command with error indication
- Owner key creation on device side fails
  - Vehicle Public Key Certificate [K] verification failed
  - Digital Key Creation Data inconsistency
  - Digital Key configuration not allowed
  - not enough memory in applet to create the Digital Key

The device shall enforce that the vehicle sends the commands in the specified order. If this does not occur, the device shall abort the transaction with the error code “Command used out of sequence.”

Vehicle and device shall allow up to a maximum of seven retries (using try_cnt, try counter) of the first session. If the maximum number of cumulated failures is reached, vehicle or device shall end the pairing mode and re-provision a new SPAKE2+ verifier.

When no matching protocol versions for SPAKE2+ or Digital Key transaction protocol can be found by the vehicle, it shall send an appropriate OP CONTROL FLOW command (namely, “abort”) and shall not send the SPAKE2+ REQUEST command. In this case the device does not expect additional commands from the vehicle.

After the OP CONTROL FLOW command, the vehicle shall perform the NFC link teardown procedure followed by the NFC reset procedure, before a new or updated device is ready to begin the first step in owner pairing (Phase 2) again.

In the case of a failed transaction due to tearing that cannot be compensated by the contactless protocol, the vehicle shall perform the NFC reset procedure (as defined in Section 3.2.4) and start the NFC polling and link setup procedure.

In case of an application timeout (not on contactless protocol level), the vehicle shall send an OP CONTROL FLOW command indicating a timeout error.

If any error occurs during the first NFC session before key creation, no Digital Key shall be created in the Digital Key applet.

When a communication error occurs, the Digital Key framework shall not delete the Digital Key and shall wait for the vehicle to retry the flow.

If a device sends an error SW (i.e., SW not equal 9000_h or $61XX_h$) which is listed in the table below, the listed behavior shall occur.

Table 6-1: Error SW Condition List 1.

Command	Error SW	Device Action	Vehicle Action
SPAKE2+ VERIFY	$6A88_h$	Stop HCE UI: “Wrong Password”	Reset

When a unlisted error SW is sent, the error counter shall be increased, and the flow shall be retried if the counter has not yet reached the maximum.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

When an error is indicated in an OP CONTROL FLOW command after the Digital Key is created and the vehicle rejects the pairing due to failing verification of the Digital Key Certificate data, the Digital Key framework shall delete the Digital Key that has been created.

On receipt of an OP CONTROL FLOW command indicating a timeout error, if the maximum number of cumulated failures is reached, vehicle or device shall end the pairing mode. The user shall request a restart of the owner pairing procedure for which the Vehicle OEM Server has to re-provision a new SPAKE2+ verifier and password.

When the device detects a link loss or receives an unexpected command without receiving an OP CONTROL FLOW command immediately beforehand, the transaction shall be considered as failed and should be allowed to be repeated a limited number of times (e.g., 5).

When the Digital Key has been created but the Digital Key Certificate [H] has not been requested by the vehicle, the Digital Key shall not be usable.

A policy on the vehicle side shall limit the number of unsuccessful owner pairing attempts per set of pairing password/vehicle verifier to seven as outlined in Section 5.1.2. When a successful pairing is completed or when the maximum number of failed pairing attempts is reached, the vehicle shall delete the verifier immediately. In case of a successful owner pairing, the verifier shall not be updated before the vehicle has verified the key tracking signature unless it was invalidated for a different reason.

When the owner pairing flow is permanently aborted (e.g., via timeout on device, maximum number of retries reached, etc.), the created Digital Key shall be deleted by the device.

During owner pairing Phase 2, the device shall prevent selection of the Digital Key applet by the vehicle and shall respond to SELECT Digital Key applet AID with Status Word (SW) = 6A82.

6.3.4 *Phase 3: Second Session with NFC Reader*

The second NFC session is executed between vehicle and Digital Key applet as described in Figure 6-8.

6.3.4.1 Steps 3 to 12: Standard Transaction

The vehicle shall select the Digital Key applet and execute a standard transaction (step 3–6; see Section 7) using transaction type 07_h (see Table 9-1).

If the vehicle did not write the opaque attestation as shown in step 13 in Figure 6-3 (i.e., no WRITE DATA command is sent by vehicle), then the vehicle shall first write the opaque attestation into the private mailbox. If the immobilizer token is required and retrieved from vehicle, it shall be written into the confidential mailbox of the owner Digital Key in the Digital Key applet.

In order to write the appropriate data structures into the confidential and private mailboxes, the vehicle shall use the EXCHANGE command (see Section 15.3.2.15) with the appropriate parameters to:

- Write the opaque attestation with the tag $5F5A_h$ tag and length into the KeyAtt field in the private mailbox, if the opaque attestation has yet to be written
- Write the owner immobilizer token into the confidential mailbox (if immobilizer tokens are retrieved from the vehicle)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- Write the slot identifier bitmap into the private mailbox (conditional)
- Write the owner slot identifier into the slot identifier list in the private mailbox (conditional)
- Write the Vehicle OEM proprietary data structure into the private mailbox
- Write the signaling bitmap to indicate the update of the above data structures into the private mailbox
- Write immobilizer tokens for sharing into the confidential mailbox (if immobilizer tokens are retrieved from the vehicle and if KTS is not implemented by the Vehicle OEM) (optional)

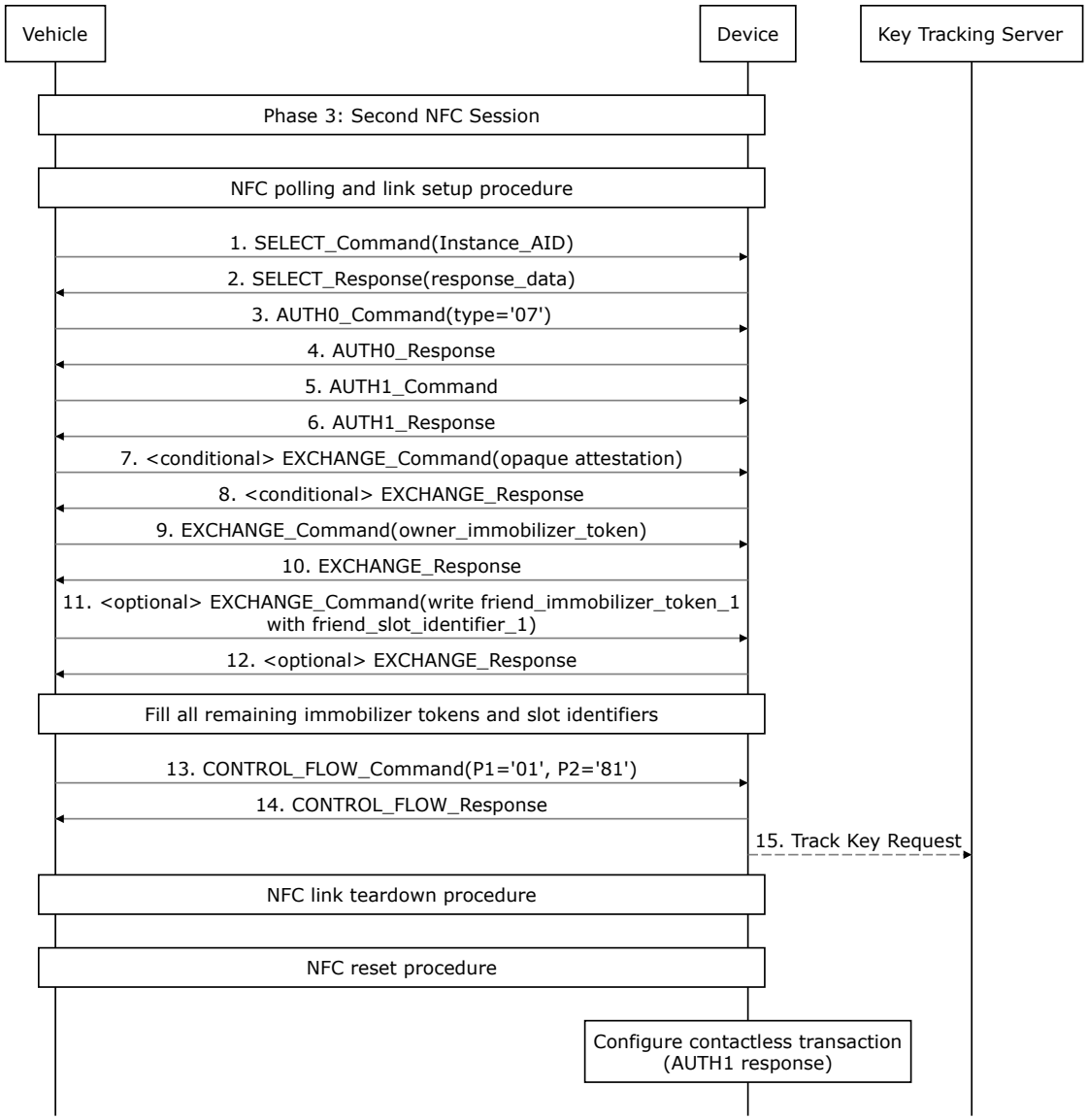
The owner slot identifier is not required as it is included in the endpoint certificate. The vehicle OEM decides whether it needs to be written or not.

It is assumed that no owner access rights need to be provisioned into the Digital Key applet. The vehicle shall grant full access to the owner.

The Digital Key applet shall notify the framework at the end of the transaction, which then shall configure the contactless transaction as described in Section 6.3.4.4.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 6-8: Owner Pairing Flow – Phase 3: Second NFC Session



6.3.4.2 Step 11 and 12: <optional> Provision Disabled Friend Immobilizer Tokens

If the vehicle can provide appropriate security measures and if immobilizer tokens/slot identifiers are provided by the vehicle, the vehicle may write the friend immobilizer tokens and slot identifiers as part of phase 3 (see Figure 6-8). If key tracking is required, the vehicle shall keep them disabled until the key tracking signature has been successfully verified.

6.3.4.3 Steps 13 and 14: CONTROL FLOW

The vehicle shall send the CONTROL FLOW command (see Section 15.3.2.16) to indicate that all data has been written successfully into the mailboxes. The P2 parameters shall be set as follows:

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

P2= 81_h to indicate to the Digital Key applet that vehicle has completed owner pairing phase 3.

At the end of Phase 3, the owner immobilizer token shall be provisioned in the owner device if immobilizer token is retrieved from the vehicle, and the key may or may not yet be tracked by the vehicle.

6.3.4.4 Step 15: Key Tracking Request

If the standard transaction was successful, the owner device shall send an asynchronous message to register the owner key to the KTS. Key registration is optional for the vehicle. If the vehicle attempts to register the key as well, a race condition may occur. Thus, key registration by vehicle is not recommended.

Note: When the standard transaction fails, the device shall not send a key tracking request to the server. The data elements listed in Table 6-2 shall be sent to the KTS by the device.

Table 6-2: Owner Key Tracking Request

Tag	Length (bytes)	Description	Element is
$7F3E_h$	variable	Owner key tracking request	
[E] Instance CA Certificate per Vehicle OEM as per Figure 6-7 and Table 11-13.			mandatory
[H] Digital Key Certificate as per Figure 6-7 and Table 11-14.			mandatory
Endpoint encryption key attestation as per Table 15-47 signed by endpoint private key (present only if immobilizer token is required)			conditional
$D5_h$	variable	Attestation of the device PK by the vehicle (opaque)	mandatory
$D3_h$	variable	Friendly name of the owner key	mandatory
$5F49_h$	65	Device privacy encryption key (Device.Enc.PK)	mandatory
DA_h	variable	Device privacy encryption version Default: “ECIES_v1” (ASCII)	mandatory

Table 6-3: Owner Key Tracking Response Parameters

Parameter	Length	Description
kts-Signature	variable	See Section 17.8.5 unencrypted uiBundle
slotIdentifier	1-8	See Section 17.8.14 vehicleMobilizationDataFields
confidentialMailboxData	variable	See Section 17.8.14 vehicleMobilizationDataFields

All parameters of the Key Tracking Response from the KTS shall not be provided in TLV format. The server API parameters shall be provided as a hex string within the JSON structure defined in the relevant sections referenced in Table 6-3.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

If the slot_identifier is provided, it shall be cryptographically bound to the kts-Signature (by vehicle OEM proprietary methods not defined in this specification) so that the vehicle can verify the authenticity of the slot_identifier.

The content of the kts-Signature is defined by the Vehicle OEM; it is opaque to the device OEM. The signature shall contain a cryptographic confirmation that the owner key has been tracked by the KTS and shall link the signature to the device public key.

The device shall add the tag 45_h and length fields to the kts-Signature (see Section 17.8.5) and stores the TLV structure in the KeyAtt field of the private mailbox (see Table 4-2).

Endpoint encryption key attestation (Tag $7F26_h$) shall be included in Table 6-2 when Tag DA_h in Tag $7F60_h$ in Table 5-14 is set to 01_h.

Owner slot identifier shall be included in the Key Tracking Response as defined in Table 6-3 when Tag DA_h in Tag $7F60_h$ in Table 5-14 is set to 01_h or 03_h.

Owner immobilizer elements (confidentialMailboxData and ephemeralPublicKey parameters) shall be included in the Key Tracking Response as defined in Table 6-3 when tag DA_h in tag $7F60_h$ in Table 5-14 is set to 01_h.

If the owner Key Tracking Response contains a slot identifier, then this slot identifier shall be written to the owner endpoint using the SETUP ENDPOINT command. If no slot identifier is provided in the owner key tracking response, then the slot identifier provided by the vehicle in the endpoint creation data is the one to be used.

All certificates in Table 6-2 shall be in X.509/ASN.1 format and shall be DER encoded.

The opaque attestation is obtained from the KeyAtt field of the private mailbox. Note that sending the attestation as part of the standard transaction is recommended. Only value field of Tag $5F5A_h$ shall be included in the value field of Tag $D5_h$.

See Section 17 for details on the format of the key tracking request.

If immobilizer tokens and/or slot identifiers are retrieved online, the owner device shall send Key Tracking Request to the server (see Section 17.7.1.2) to request owner immobilizer token and/or slot identifier.

6.3.4.5 Configuration of the Contactless Transaction

There are two options to access mailbox data within the standard transaction:

- Transmit the required data as part of the AUTH1 response
- Use the EXCHANGE command (mandatory for the fast transaction)

If the WRITE DATA command received from the vehicle contains tag $4A_h$ and/or tag $4B_h$ in the Device Configuration Data field (see step 3 in Figure 6-3 and Section 5.1.4), the device shall transmit the signaling bitmap and/or owner immobilizer token in the AUTH1 response as described in Table 15-33 and the Digital Key framework shall set the endpoint using the SETUP ENDPOINT command (see Section 15.3.2.21) with the following parameters:

- **confidential offset**: Shall be set to 0 as defined in Table 4-3.
- **confidential length**: Shall be set to IMMOBILIZER_TOKEN_LENGTH as defined in Table 4-3 if immobilizer tokens are used.
- **private offset**: Shall be set to SIGNALING_BITMAP_OFFSET as defined in Table 4-2.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- **private length**: Shall be set to (SLOT_IDENTIFIER_BITMAP_OFFSET – SIGNALING_BITMAP_OFFSET) as defined in Table 4-2.

The other arguments in this method shall not be used.

6.3.4.6 Error Handling for the second NFC session

The following errors that appear in the second session of the NFC pairing flow include:

- General errors that may include
  - Communication error
    - RF transmission error
    - owner removes device before end of protocol flow, i.e., no deselect command received before link loss
  - commands not received in correct order
  - device sends other error status word
  - implementation bug leading to timeout
- SELECT command failure
  - applet not installed
  - device configured for applet when HCE selected over NFC

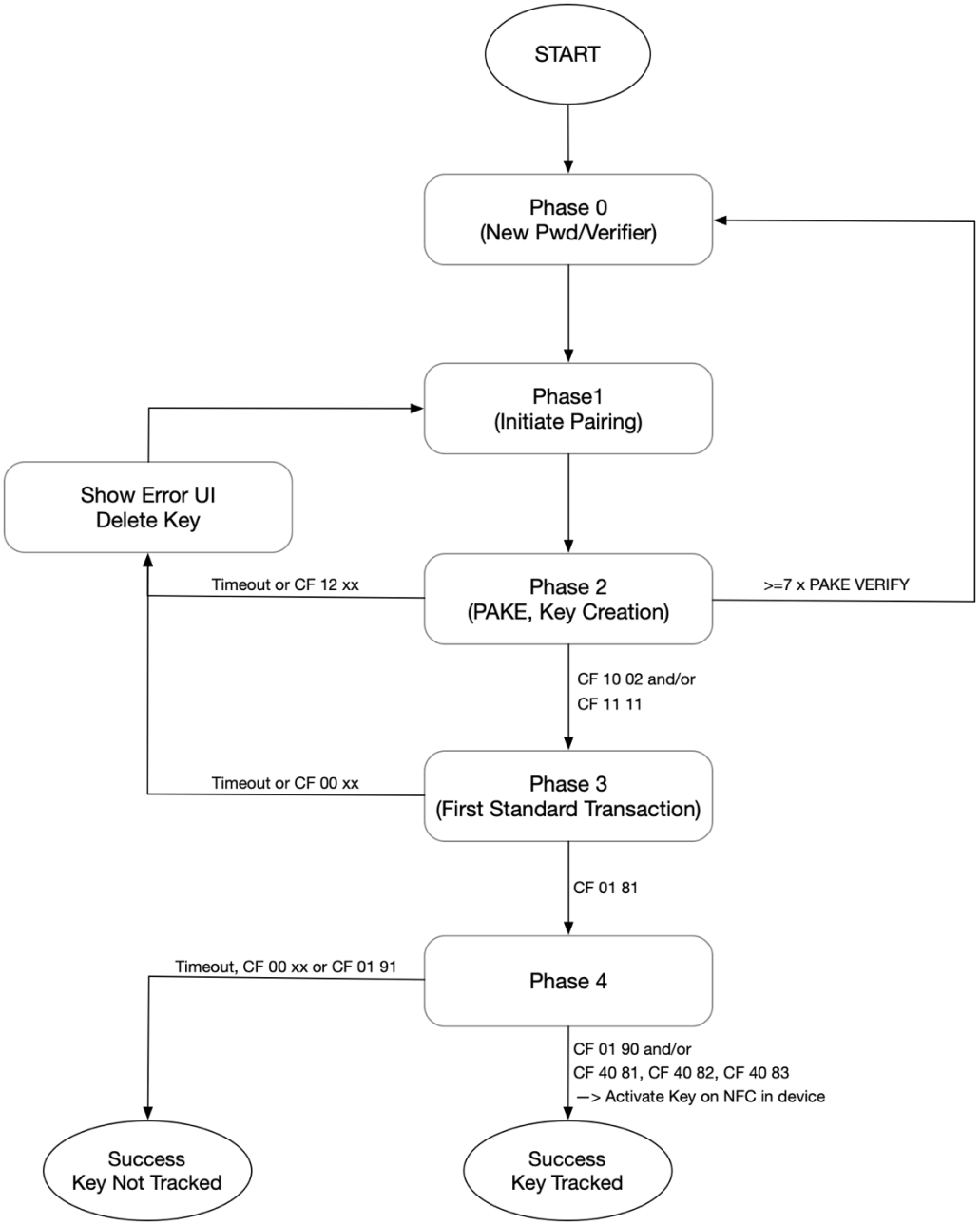
Figure 6-9 shows the error management in owner pairing and the relationship between the different phases. No retry shall be allowed in a specific phase and any error which is not fixed on NFC protocol level shall lead to complete restart.

If the standard transaction (in step 3–6) failed, the endpoint may be deleted.

As soon as the Digital Key is successfully created, it might be usable on the, even if the owner immobilizer token is not yet provisioned. For a consistent user experience, it is recommended to enable the created owner key only when the device has confirmed that the owner immobilizer token and the opaque attestation has been correctly written.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 6-9: Error Management of Different Phases in Owner Pairing



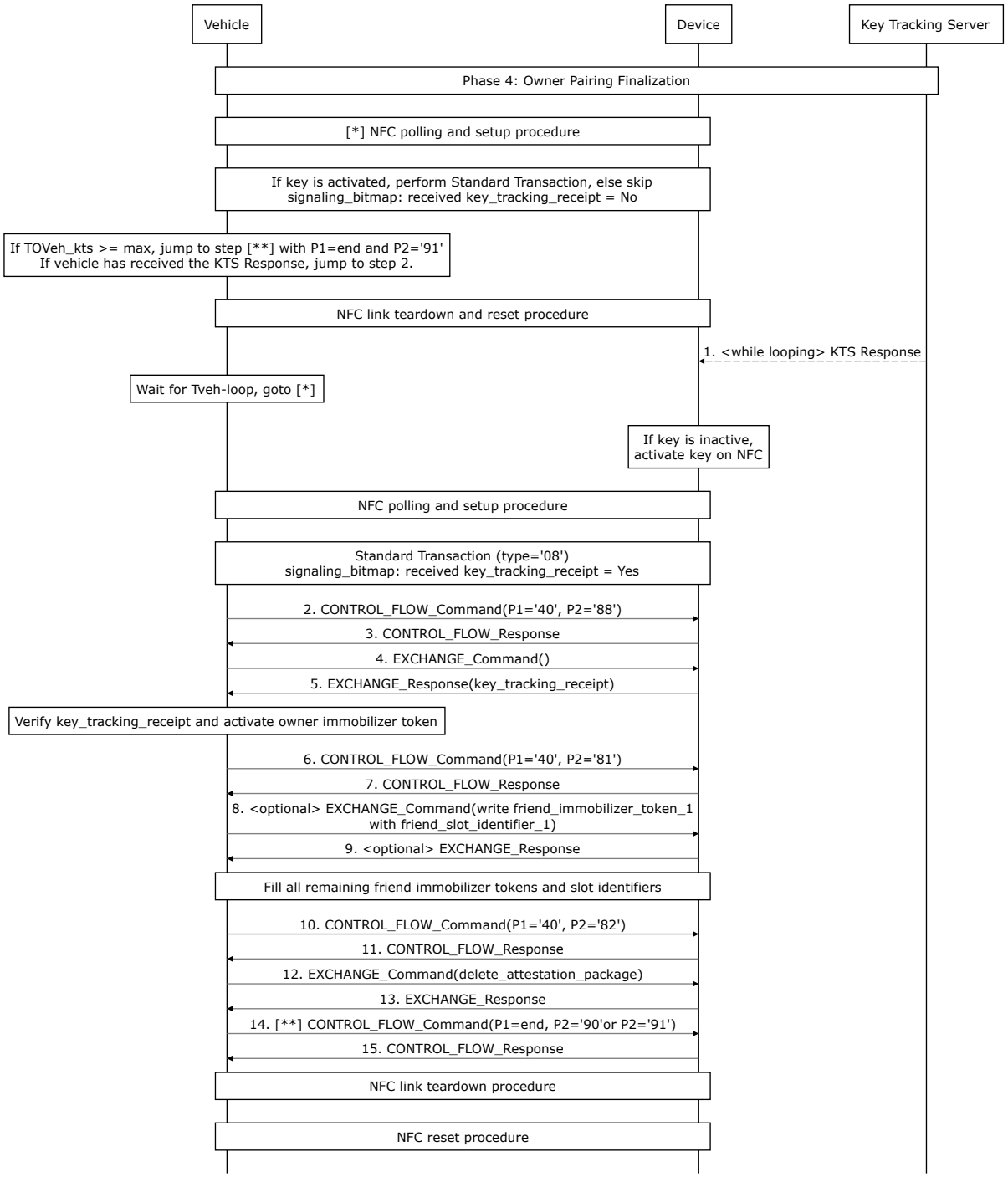
6.3.5 *Phase 4: Finalization of Pairing Procedure (optional)*

The finalization step executes a standard transaction using transaction type 08_h (see Table 9-1) in regular intervals to poll the signaling bitmap indicator for an attestation package in the device.

Figure 6-10 shows the final phase of the owner pairing flow.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 6-10: Owner Pairing Flow – Phase 4: Finalization



6.3.5.1 Before Step 2: Reader Polls for KTS Response

The owner device and (optionally) the vehicle have reached out to the KTS by sending a key tracking request, as defined in Table 6-1 in Phase 2.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

If the device receives a Key Tracking Response from the KTS (see Table 6-3), the device shall store the kts-Signature which is stored in the KeyAtt field in the private mailbox (see Figure 4-4), the corresponding bit in the signaling bitmap (SigBmp) shall be set by the device to indicate to the vehicle the presence of the kts-Signature.

The reader executes standard transactions to check the signaling bitmap in the device. If the kts-Signature is not present or the device does not respond[4], the reader shall try again after a time (Tveh-loop) defined in Section 6.3.6). The reader shall perform the NFC Reset procedure after each standard transaction.

If the reader experiences a loss of connection, it shall perform the NFC Reset procedure and shall restart the NFC Polling and Link Setup procedures.

If the vehicle receives the Key Tracking Response, the vehicle shall first skip the verification of the kts-Signature in the device and then continue by writing the friend immobilizer tokens into the mailbox if immobilizer tokens are retrieved from the vehicle.

If neither vehicle nor device receives a response before TOveh_kts has expired, the vehicle shall not provision friend immobilizer tokens and shall signal the failure to get a KTS signature through a CONTROL FLOW command (see Section 6.3.5.5).

Note: The device may not respond to NFC polling until it has received the key tracking response. The vehicle shall time out (by TOveh_kts) even if the device never responds.

6.3.5.2 Steps 2 to 5: Verification of the Key Tracking Receipt in Device

When the device obtains a KTS signature, the vehicle shall execute a standard transaction and shall send a CONTROL FLOW command (see Section 15.3.2.16) to indicate the status:

1. P1=40h and P2=88h: continue, key tracking response received in device; next step is to read the receipt from the mailbox
2. P1=40h and P2=89h: continue, key tracking response received in vehicle, go directly to verification of key tracking receipt

Steps 2, 3, 4, 5, 10, 11, 12 and 13 shall be executed only when the KTS signature is stored in the device. If a key tracking response is received by the vehicle, only steps 2 and 3 are executed as defined in 2, above.

The vehicle shall then verify the KTS signature.

If the verification of the KTS signatures was successful, the vehicle shall continue with the next step. Otherwise, the vehicle shall abort and execute steps 14 and 15 with an appropriate error indication (see Section 6.3.5.5). If immobilizer tokens are not required, the vehicle can proceed directly to step 10 (see Section 6.3.5.4). Otherwise, the vehicle shall abort and execute steps 14 and 15 with an appropriate error indication.

6.3.5.3 Steps 6 to 9: Provisioning Immobilizer Tokens for Sharing

If offline provisioning of immobilizer token is not required (indicated by the SHARING_CONFIGURATION field in Table 5-14: Tag $7F60_h$ and Tag DA_h), the vehicle can proceed directly to step 10 (see Section 6.3.5.4).

[4] A device may choose not to enable the key on the NFC interface or BLE/UWB interface until the kts-Signature is present in the private mailbox.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

If the vehicle requires offline provisioning of immobilizer token, the vehicle shall indicate the start of the procedure by sending a CONTROL FLOW command (step 6):

P1=40_h and P2=81_h: continue, optional friend immobilizer token refill.

The immobilizer tokens required for key sharing shall be provisioned into the confidential mailbox. Signaling bits and slot identifiers shall be written accordingly using the EXCHANGE command.

6.3.5.4 Steps 10 to 13: Deleting the attestation package

If there is a key tracking receipt present in the private mailbox, which was already verified by the vehicle, the vehicle shall delete the key tracking receipt as follows:

The vehicle shall indicate the deletion of the key tracking receipt by sending a CONTROL FLOW with:

P1=40_h and P2=82_h: continue, attestation package delete start.

The vehicle shall clear the signaling bitmap and delete the attestation package from the private mailbox using an EXCHANGE command, so that the vehicle does not detect the same attestation package a second time, on the next transaction.

6.3.5.5 Steps 14 to 15: CONTROL FLOW

The CONTROL FLOW command indicates the end of the owner pairing flow with the following options:

1. P1=01_h and P2=90_h: end, owner key is tracked, and all data has been written successfully into the mailboxes.
2. P1=01_h and P2=91_h: end, owner key is not tracked, key sharing is not possible, and the owner needs to go online to track the key before using it.

6.3.6 *Timers*

The owner pairing flow is limited by retry counters and by timers on vehicle and device sides.

The vehicle limits the overall time of a pairing attempt, from the first detection of the device responding successfully to the selection of the framework AID until the successful provisioning of the immobilizer token in the Digital Key.

Phases and sessions are limited by shorter timer values, to be able to detect failures more quickly.

Vehicle overall timer TOveh-1 is used to control the owner pairing procedure. This timer is not linked to the device timer TOdev-1.

The device should at least have a timer (TOdev-1) to limit the overall time of the owner pairing procedure until the key and immobilizer token are provisioned. On timeout, the previously created key shall be deleted, and a new entry of the pairing password shall be required.

A timer should not be restarted if it is already running, and the flow passes again through the “Start Timer” state (which is unlikely).

The following scheme provides more details about the timer start and end conditions. It shows the specific timeouts from vehicle and device perspective.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 6-11: Vehicle and Device Transaction Timeout

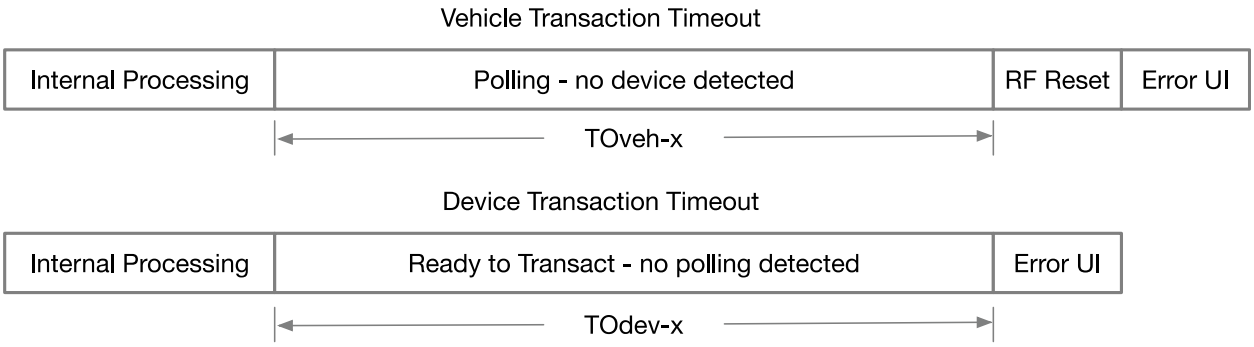


Table 6-4 defines the timer values. The timer numbers refer to Figure 6-11 with a split into vehicle and device side.

Table 6-4: Recommended Minimum Vehicle and Device Timeout Values

Timer	Description	Value
TOveh-1	Overall OP transaction timer (detection of device until final UI)	30 sec
TOdev-1	Overall OP transaction timer (first response to polling until KTS signature not received)	30 sec
TOveh-2	not required	
TOdev-2	Start of pairing mode until detection of polling	Device OEM specific
TOveh-3	Last WRITE DATA command until first response to polling for GET DATA	10 sec
TOdev-3	Endpoint created until detection of polling	5 sec
TOveh-4	Endpoint data processed until first response to polling for first standard transaction	5 sec
TOdev-4	Endpoint data sent to vehicle until detection of polling	10 sec
Tveh-loop	Time of repeated standard transaction to retrieve KTS receipt (Note 1) - this is not a timeout, but a loop timer	1 sec
TOveh-kts	Time until KTS receipt should be received	20 sec

Note 1: The device might not respond to polling until the KTS receipt is received.

6.3.7 *Pairing Password URL*

To start owner pairing mode, the device may receive the pairing password through a URL. The Vehicle OEM may provide the owner with the pairing password as fragment of a URL (e.g. via an email). When the owner consumes this URL on the device, the device will redirect this link to the Framework, which extracts the pairing password from the URL and uses it to start the owner pairing. The URL is not intended to be reached out to. It is used to transport the parameters to be interpreted by the framework.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The URL formatting rules are specified in RFC 3986 [27]. The URL syntax shall take the following form:

scheme://authority/version/redirect?query#fragment

For example:

https://digitalkeypairing.org/v1/0001?technology=NFC,BLE&graphics=0001A0A1A2A3#pwd=12345678

Table 6-5: Pairing Password URL Syntax

Component	Length (character)	Description	Field is
scheme	5	"https": HTTP protocol operating over TLS	mandatory
authority	21	Domain name defined and registered by CCC Domain: digitalkeypairing.org	mandatory
version	2	"v1": Version number to identify the fields in the URL	mandatory
redirect	4	Vehicle Brand Identifier [35]. This is used for server redirection in owner pairing.	mandatory
query		Non-sensitive URL parameters	mandatory
technology	variable	Communication interfaces supported for pairing by the vehicle. This parameter may be "NFC" or "NFC,BLE" If this query is not present, the communication interface is expected to be "NFC"	optional
graphics	12	The NFC reader position graphics identifier of the vehicle which is provided by vehicle OEM. This identifier is given as [Vehicle Brand Identifier || 4-byte vehicle defined graphics identifier], represented as string. The Vehicle Brand Identifier is defined in [35].	optional
fragment		Sensitive URL parameter Device strips the fragment from the URL when using the URL to contact the authority to preserve the confidentiality.	mandatory
pwd	8	pairing password	mandatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

7 STANDARD TRANSACTION

The standard transaction protocol is intended to provide the following properties:

- Mutual authentication
- Forward secrecy
- Tracking resilience
- Integrity and confidentiality

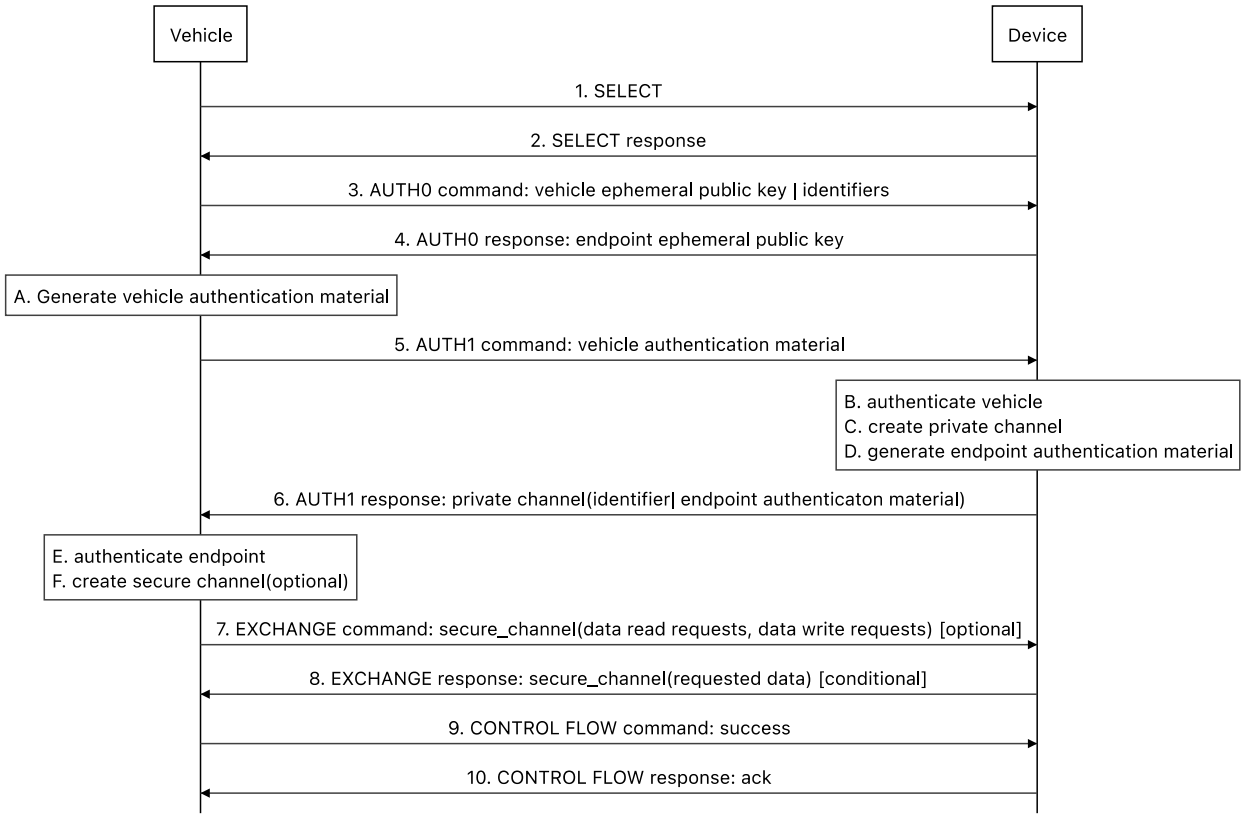
A secure channel between vehicle and device is initiated by generating ephemeral key pairs on the vehicle and device sides. Using a key agreement method, a shared secret can be derived on both sides and used for generation of a shared symmetric key, using Diffie-Hellman and a key derivation function.

The ephemeral public key generated on the vehicle side is signed with the vehicle's private key, vehicle_SK. This results in an authentication of the vehicle by the device. From the device's perspective, this guarantees that no privacy-sensitive data can be leaked by a MITM attack. This principle also allows the device to transmit data to the vehicle without any possibility of leakage by a passive or active eavesdropper.

Finally, the device uses the established secure channel to encrypt its public key identifier along with the signature computed on a vehicle's data-derived challenge and some additional application-specific data. This verification of the device's signature by the vehicle allows the vehicle to authenticate the device.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 7-1: Standard Transaction Flow



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

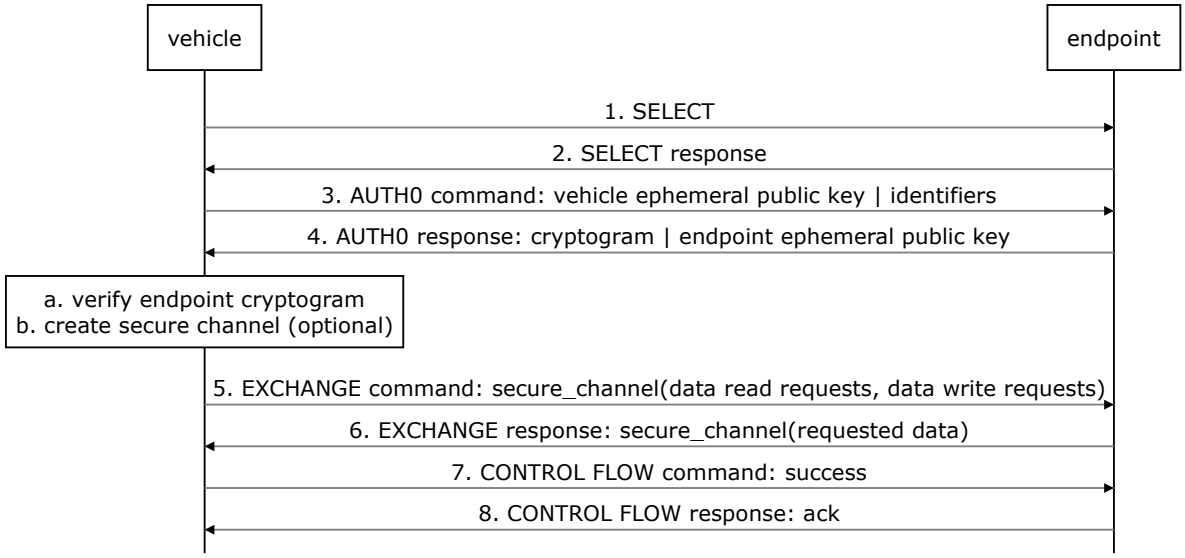
8 FAST TRANSACTION

The fast transaction protocol is intended to provide the following properties:

- Device authentication or Mutual authentication
- Integrity and confidentiality
- Tracking resilience

The device generates a cryptogram based on a secret previously shared during a standard transaction, and this allows the vehicle to authenticate the device. Optionally, a secure channel between vehicle and device is established by deriving session keys from a secret previously shared during a standard transaction and from the ephemeral keys. The ability of the vehicle to establish the secure channel authenticates the vehicle to the device. The shared secret shall always be regenerated in the standard transaction when fast transaction is allowed by the endpoint configuration defined during endpoint creation. Refer to Section 15.3.2.4 (see Table 15-13)

Figure 8-1: Fast Transaction Flow



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

9 USER AUTHENTICATION

Devices shall provide the functionality to enforce user authentication according to their user authentication (UA) policies. The device shall provide at minimum one of the UA policies ([A], [AL], [E], [T]) (see Table 9-2). The user authentication policy is set by the user on the device. UA is enforced by the device only. The conditions for enforcing UA are determined by the selected UA policy, based on the transaction type (see Table 9-2).

The transaction type is set as a domain specific transaction_code value in P2 parameter of the AUTH0 command. Transaction type assignments are specified in Table 9-1.

User authentication may be enabled or disabled individually for each Digital Key. The UA policy is stored as a list of transaction types that require user authentication. For every new Digital Key, the user authentication should be set to Off [O] by default (see Table 9-2). Device should inform Vehicle about the currently used UA policy at BLE connection setup, and also when UA policy is changed on the device (Communication of User Authentication Policy is described in Section 19.3.8.1).

Table 9-1: AUTH0 Command Transaction Type Coding

P2 (transaction_code)	Transaction Type
00_h	RFU
01_h	Door unlock
02_h	Door lock
03_h	First engine start authentication, first contact of a device with engine start rights in a vehicle session (e.g. door is in unlock state before the vehicle session starts and then first engine start in a new vehicle session)
04_h	First engine start authentication, subsequent contact of a device with engine start rights in a vehicle session (e.g. door is in lock state before the vehicle session starts, and door is unlocked before first engine start)
05_h	Other authentication request by vehicle
06_h	User authentication request by vehicle
07_h	First standard transaction at owner pairing (owner immobilizer token provisioning)
08_h	Second standard transaction at owner pairing (read KTS receipt, provision friend immobilizer tokens)
09_h - $0F_h$	RFU
10_h	Derive ranging key (standard transaction only)
11_h	First approach (standard transaction only), i.e. first standard transaction after Bluetooth LE pairing between vehicle and device outside of owner pairing
12_h - FF_h	RFU

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

A vehicle session starts with the first usage of the Digital Key after the end of a preceding vehicle session and ends when the vehicle is locked or after an appropriate timeout of non-usage of the vehicle.

Table 9-2 shows the assignment of policies to the transaction types and the associated resulting user experience.

Table 9-2: User Authentication Policies

Device UA Policy														
Notation	Resulting User Experience	UA based on Transaction Code												
		00_h	01_h	02_h	03_h	04_h	05_h	06_h	07_h	08_h	09_h–$0F_h$	10_h	11_h	12_h–FF_h
Off [O]	UA never requested	-	-	-	-	-	-	X	-	-	-	-	-	-
Access [A]	UA requested on first access or first engine start	X	X	-	X	-	-	X	-	-	X	-	-	X
Access [AL]	UA requested on first access and lock or first engine start and lock	X	X	X	X	-	-	X	-	-	X	-	-	X
Engine [E]	UA on every engine start	X	-	-	X	X	-	X	-	-	X	-	-	X
Always [T]	UA on every usage	X	X	X	X	X	-	X	-	-	X	-	-	X

- **X**: UA to be performed on the device
- -: UA not required for the transaction to be successful

The owner should not be able to restrict the UA setting options of a shared key.

Note: When user authentication is set for a Digital Key, the key may not be usable in Battery Low Mode.

9.1 Explicit User Authentication Policy

To improve security, it may be beneficial to require explicit UA before an action can be triggered. Explicit UA shall require an intent for RKE actions to be registered before explicit UA is enforced. The associated RKE action shall only be performed if the explicit UA has been performed successfully. The time between explicit UA and the action shall not exceed 1 minute except if the device is set to a state where only Digital Key functions can be triggered. Upon leaving this state where only Digital Key functions can be triggered, an explicit UA shall be performed before another action can be performed. Device should inform Vehicle if UA is not required because this state has been entered or if UA is required because this state has been left (Communication of User Authentication Policy is described in Section 19.3.8.1).

9.2 Implicit User Authentication Policy

To improve the user experience, it may be beneficial for the device to allow a single successful UA event to authorize several Digital Key actions within a short period of time (grace period).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

This may be coupled to a UA used for device unlock. If the user device implements such a grace period, it should not exceed 5 minutes. If supported by the device, additional successful UA events should cause this grace period timer to start/restart. The grace period shall expire immediately when the UA becomes invalid, such as when the device is locked. Device should inform Vehicle if UA is not required because the grace period has been entered or if UA is required because the grace period has been left (Communication of User Authentication Policy is described in Section 19.3.8.1).

This implicit UA shall not be used for critical actions, such as endpoint authorization, key sharing, or when a device's UA policy explicitly forbids implicit UA to be used for a transaction type or remote feature execution.

The user should be able to enable or disable usage of this implicit UA. The user may also be able to configure the length of the grace period between zero minutes (disabled) and the maximum time defined above.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

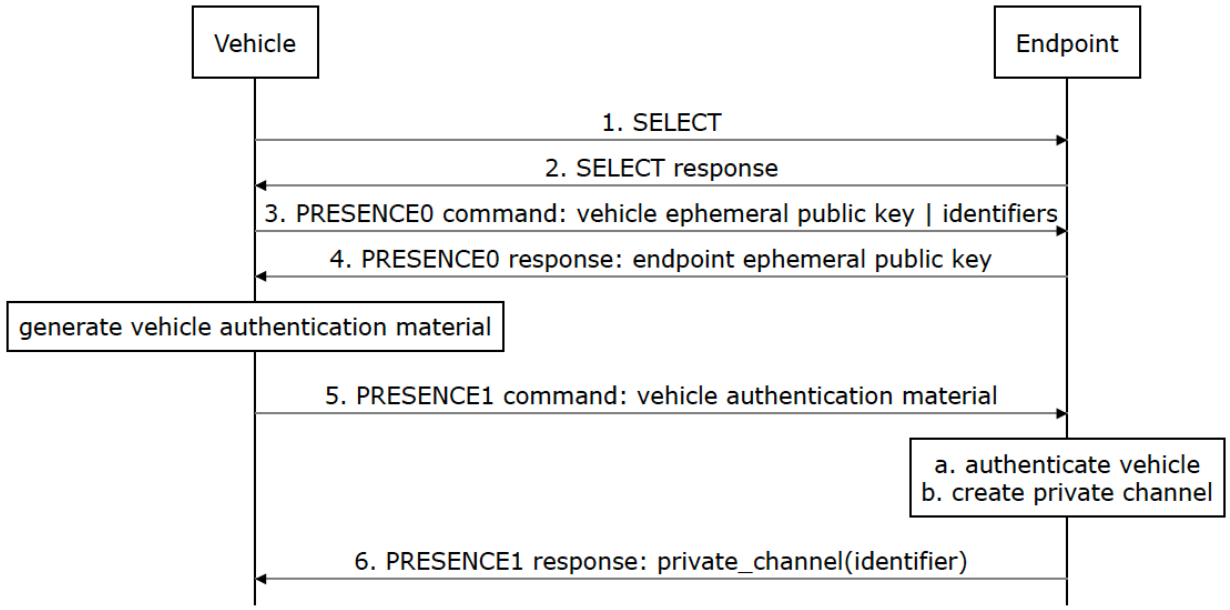
10 CHECK PRESENCE TRANSACTION

The check presence transaction protocol is intended to provide the following properties:

- Vehicle authentication
- Device identification
- Integrity and confidentiality
- Tracking resilience

The mechanism is similar to the standard transaction mechanism described in Section 7, except that the device signature is not sent to the vehicle, and user authentication is disabled. The goal is to allow verification of device presence near the vehicle without requiring user authentication, while preventing tracking.

Figure 10-1: Check Presence Transaction



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

11 DIGITAL KEY SHARING [WCC1/WCC2]

11.1 Encoding

All messages exchanged over the communication channels described in Section 11.2 are compliant with Basic Encoding Rules-TLV(BER-TLV) as outlined in [40]. Certificates are compliant with X.509 format, as per document [3].

The TLV fields shall be ordered as described in this specification. A different field order is considered invalid unless specified otherwise. The nesting level is represented by indentation of tag values in the tag column.

11.2 Sharing Principles

11.2.1 *Definitions*

11.2.1.1 Sharing URL

To initiate the key sharing process, the Owner device requests the Relay Server to create a Mailbox on the relay server. The sharing URL is generated by the relay server and provided to the owner device. The owner device provides the sharing URL to the friend device to initiate the transfer of the DK. The sharing URL shall accept additional parameters. The version is determined by the relay server when generating the URL. The version is determined for use by the relay server and shall not have any impact on the owner or friend device.

11.2.1.2 Sharing Password

If supported by the Vehicle OEM, it is provided to the owner, which provides it to the friend via a dedicated channel. The friend then types this into the vehicle UI for key activation. Vehicle OEM verifies that the correct sharing password was entered. This specification includes sharing password as an *activation option* defined in Section 11.2.1.4 and Section 17.8.18. Hence the sharing password as defined in previous versions of this specification [32] is considered obsolete. Vehicle OEM decides whether to implement this option or not.

11.2.1.3 Device PIN

Can be set by the owner device during the sharing workflow if no sharing password is required by the vehicle OEM. The owner provides the PIN to the friend via a dedicated channel, separate from the channel used to communicate the sharing invitation. The friend then types it into their device UI to continue the sharing workflow. Device OEM verifies if the correct PIN was entered by friend. Device OEM decides whether to implement this option or not.

11.2.1.4 Activation Options

Can be defined by the Vehicle OEM to activate a friend key in the vehicle. The friend can then choose any supported activation option. May contain the sharingPassword as one option. Vehicle OEM decides whether to implement this option or not.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

11.2.1.5 Approved Sharing Methods

These are Device OEM-proprietary sharing channels under full control of the device OEM that have been approved by the vehicle OEM. The security level can be assessed and maintained. When approved as secure by a vehicle OEM then the sharing mechanism doesn't require activation options to be used for a key shared using that method. Vehicle OEMs shall have a way to define the activation method policy for approved sharing methods.

11.2.1.6 Sharing Method Group Identifier

Identifier defined by OEM that implements a proprietary sharing method. Vehicle OEMs use this identifier to define the activation options policy for the referenced sharing method.

11.2.2 *Principles*

The Digital Key system is based on signature verification using asymmetric cryptography. The vehicle unlocks the doors or starts the engine only if a challenge has been signed with a private key corresponding to a public key registered in the vehicle.

After the owner device and the vehicle have been paired as described in Section 6, the owner possesses a private key for which the corresponding public key is stored in the vehicle.

To allow friends to access the vehicle, the owner uses his/her private key to sign friend public keys. On presentation of the owner's signature over the friend's public key, the vehicle will in turn accept the friend as a new vehicle user and store the friend's public key.

Digital key sharing consists of the owner sending an invitation URL to the friend device and a stateful exchange of key creation, key signing, and data import requests. When a secure sharing channel, such as a device OEM-controlled proprietary messaging channel, is used, then invitation and stateful sharing messages can all be sent across this channel. No additional second factor verification (i.e., *activation options* and *device PIN*) of the destination device should be required. These secure sharing methods are named *approved sharing methods* throughout this specification, which means that they are approved by the vehicle OEM for use without additional activation options. The approval process is out of scope of this specification. The list of approved sharing methods can be pre-agreed between vehicle OEM and device OEM, or it can be sent using key tracking and event notifications.

Digital Key Sharing between devices of different device brands (known as cross-platform sharing), consists of:

1. Sending an invitation over a messaging channel that the owner uses regularly with the friend. This provides good insurance that the invitation is received by the intended person. The channel to exchange the stateful sharing messages is defined by [38].
    a. One or more Vehicle OEM-defined *activation options*, such as the *sharing password* along with more options (E.g., presence of a key fob during the first usage of the friend key), shall be applied if required by the vehicle OEM. Vehicle OEM provides these in the trackKey response or eventNotification sent to owner device. Available *activation options* are listed in Section 17.8.18. The list of *activation options* for a given vehicle is communicated to owner and friend device during key tracking. In addition, the owner device provides the list of *activation options* to the friend device during key sharing. The activation options shall be shown in the owner and friend device UIs. Friend can select and use one option from the list of activation options.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

   i. If the sharing password is one of the supported activation options, then the owner device shall make the password available in the device UI along with necessary guidance for usage. In case of a vehicle software update that adds or changes activation options, owner device can be updated using event notifications.
2. If a vehicle OEM decides to not support additional *activation options*, the owner device should propose the use of a *device PIN* (and potentially other "silent" verification methods in the future) to owners that want to add additional verification to their shares. The policy for device OEM-proprietary sharing methods is defined by the owner device OEM.

In some cases, (e.g., when list of *approved sharing methods* has been modified by the vehicle OEM) the owner device shall send a *sharing method attestation* for shares using an approved sharing method to confirm that the owner device uses the updated list of *approved sharing methods*.

The conditions of change of the sharing policy (i.e., adding or removing approved sharing methods from the list) by the vehicle OEM are out of scope of this specification.

The vehicle OEM shall notify the device OEM of a change to the activation policy using either an event notification "SHARING_POLICY_CHANGED" or another device OEM proprietary method.

11.3 Communication Channel

The standardized communication channel between devices and the relay server used by devices and vehicles that implement cross-platform sharing shall use the stateful workflow method defined in [38].

The referenced sharing channel has the following properties:

- Open standard for implementation on all Digital Key eligible devices and servers
- Sharing invitation can be sent over any messaging or chat channel
- Devices connect directly to relay server

In addition, the devices shall only send encrypted sharing data using the encryption key from the sharing invitation.

The communication channel between an owner device and a friend device from the same OEM is implementation-specific and out of scope of this specification but shall be listed as an approved sharing method in Table 11-11, if key activation without vehicle OEM-defined second factor activation option is intended. Figure 11-1 shows the detailed key sharing flow between the owner device and friend device. The process follows the "Stateful Workflow" defined in [38]

Note: In Figure 11-1, the full head arrows represent synchronous requests, dashed arrows represent responses to synchronous requests and open head arrows represent asynchronous messages

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 11-1: Detail of Key Sharing Flow Between Owner and Friend Device After Channel is Established

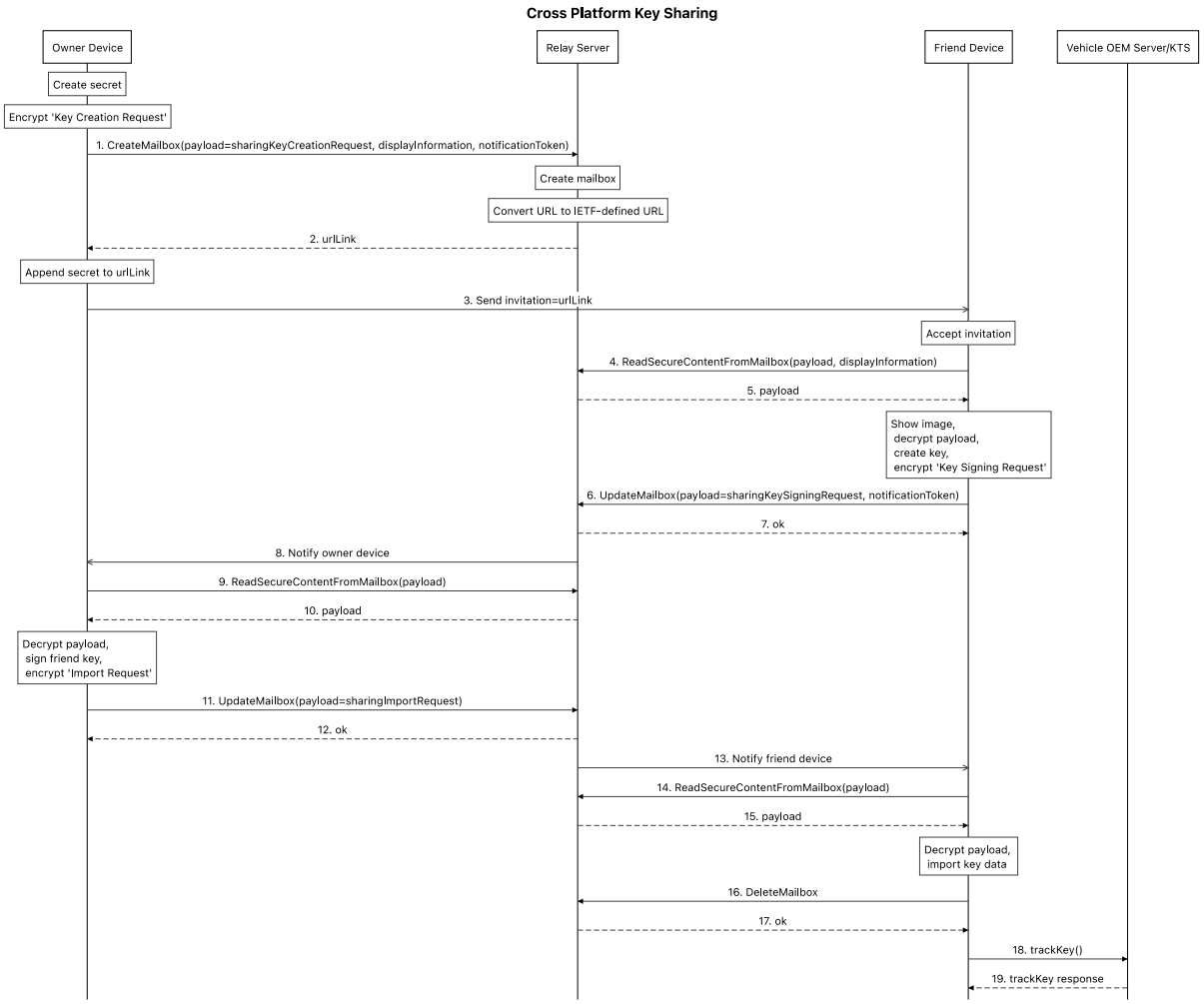


Figure 11-1 does not show all steps or parameters, such as the full *payload* coding or the device OEM server. It is meant to clarify the flow in principle and pointing to the relevant elements.

The exchanges between owner and friend device are described by the stateful workflow method using the following HTTP access methods as defined in [38]. An application running on a device may invoke the following APIs on Relay Server (see Section 11.3.4.1 to 11.3.4.6)

- CreateMailbox
- UpdateMailbox
- DeleteMailbox
- ReadDisplayInformationFromMailbox
- ReadSecureContentFromMailbox
- Relinquish Mailbox

The secure element is not involved in generating key material used for the sharing channel as described in this section.

The key sharing flow and data is described in Section 11.8.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

11.3.1 *Notifications*

The push notification feature shall be implemented by the Relay Server.

Notification tokens as described in [38] should be used by every device OEM.

If the notification tokens are not used, the devices shall implement a polling strategy for Relay Server data. Devices may implement a (backup) polling strategy even if push notifications are used.

The polling strategy should follow these recommendations:

- After sending the invitation, the owner device should poll every 10 seconds for a duration of 3 minutes for the Signing Request (or other messages to be received as per Table 11-3), thereafter it should poll every 30 seconds for the next 10 minutes, thereafter it should poll every 3 minutes for the next hour, thereafter every hour until the mailbox is expired or deleted.
- After accepting the invitation and uploading the signing request to the mailbox, the friend device should poll every 5 seconds for a duration of 1 minute for the Import Request (or other messages to be received as per Table 11-3), thereafter it should poll every 30 seconds for the next 10 minutes, thereafter it should poll every 3 minutes for the next hour, thereafter every hour until the mailbox is expired or deleted.

11.3.2 *Cross-Platform Sharing Invitation*

The owner device generates a Secret and encrypts the payload for the CreateMailbox API using this Secret as described in [38].

The owner device then initiates the sharing session by calling the CreateMailbox API as described in Section 11.3.4.1. The call returns the sharing invitation in form of a URL. The URL is then sent to the friend over any communication channel (e.g., WhatsApp, etc.). The URL formatting is described in [38]. The framework shall append the Secret as a fragment to the URL.

A relay server may be implemented by any CCC member. All CCC-approved relay server URLs shall be listed in [35]. The approval process is defined in a separate document and is controlled by the CCC. All Owner devices shall only use approved relay server URLs. Friend devices shall only accept invitations containing an approved relay server URL. Friend devices shall support all CCC-approved relay server URLs within a reasonable time frame as defined by the CCC in the CCC Program Management Document *[PMD]*.

11.3.3 *General API Parameter Definitions*

The *version* parameter shall be set as described in [38].

The *mailboxIdentifier* shall be used as described in [38].

The *deviceAttestation* should be used to authenticate the owner device if the Relay Server is not hosted by the owner device OEM. In other cases, the device OEM might use proprietary methods to authenticate the device. The friend device does not need to present a *deviceAttestation*.

The *deviceClaim* shall be provided and set by owner and friend device as described in [38].

The payload is encrypted as described in [38]. For Digital Keys payload shall contain the provisioning information as the JSON structure with the keys *format* and *content* as per [38]. For

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Digital Keys the format identifier is *digitalwallet.carkey.ccc*. The *content* data structure is described in Table 11-1.

Table 11-1: content in JSON Format

Key	Type	Max Length (Bytes)	Description	Required
genericSharingData	Dictionary	4096	This is data shared between devices on different device OEM platforms.	Yes
<Device OEM>	Dictionary	4096	Device OEM proprietary provisioning information for the designated device OEM. Ignored by all other device OEMs.	Optional

<Device OEM> is defined in [35].

The owner device can add proprietary data (e.g., “apple”) that a friend device from the same device OEM is able to use, e.g., to improve the user experience. If the friend device is from another device OEM, then the friend device shall ignore the proprietary data. It shall be possible for the owner device to add multiple <Device OEM> fields, each one for a different device OEM, if required.

Note that all fields can be “seen” by friend devices of all platforms.

The genericSharingData is defined as the following JSON structure:

Table 11-2: JSON formatted genericSharingData data structure

Key	Type	Max Length (bytes)	Description	Required
sharingData	String	4096	Sharing data structure (TLV string) as listed in Table 11-3 based on the value of sharingDataType	Yes
sharingDataType	Integer	4	Enum for type of message. See sharingDataType in Table 11-3.	Yes
sharingId	String	64	ID of sharing invitation	Yes
friendKeyId	String	64	Identifier of key on friend device (this is used on owner side to track shared keys)	Conditional. Required for sharingDataType sharingKeySigningRequest.
authType	Array of strings	variable	“VehicleActivation” = activation options “DevicePIN” = PIN entered on friend device Other friend verification options can be added in the future.	Conditional. Required for sharingDataType sharingKeyCreationRequest and if an additional verification method is

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Key	Type	Max Length (bytes)	Description	Required
				required by Owner Device or Vehicle OEM
activationOptions	Array of strings	variable	List of activation options as per section 17.8.18	Conditional. Required for sharingDataType sharingKeyCreationRequest if authType = “VehicleActivation”.
pinLength	Integer	4	Length of device PIN. Absent if device PIN is not used.	Conditional. Required for sharingDataType sharingKeyCreationRequest and if authType = “DevicePIN”.
brand	String	variable	Brand of the vehicle	Conditional. Required for sharingDataType sharingKeyCreationRequest
model	String	variable	Model of the vehicle	Conditional. Required for sharingDataType sharingKeyCreationRequest
vehicleIdentifier	String	variable	Value of the vehicle_identifier	Conditional. Required for sharingDataType sharingKeyCreationRequest
…unknown additional entries shall be ignored, reserved for forward compatibility.				

sharingData contains the key sharing data elements as defined in Table 11-3, depending on the state of sharing.

sharingDataType is defined in Table 11-3.

sharingId is defined by the owner device in the first message to unambiguously identify the sharing session independently of the mailboxIdentifier. The value shall be unique per owner device and per session. Owner and friend device shall use the same value in all messages of the same sharing session. If the owner sends out multiple invitations (e.g., for friend phone and friend watch) then the values of *SharingId* shall be unique per friend device.

friendKeyId shall be sent by the friend device in the key signing request (format as defined in section 2.4 as “digital key identifier”)

authType indicates which of the following additional verification is required. If no method is used, then the field shall be absent. The additional verification methods are listed here:

- *VehicleActivation* usage shall be indicated if both of the following conditions are true:
  - At least one activation option has been provided to the owner device via uiBundle in trackKey response (see section 17.8.5) or eventNotification (see section 17.8.8)
  - A non-approved sharing method has been used

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- *DevicePIN usage shall only be indicated if both of the following conditions are* true:
  - No vehicle activation is required
  - The owner chooses to use the device PIN for a key sharing

It shall be possible to add other friend verification options, which are not part of the current list of activationOptions, in the future.

activationOptions are provided by the vehicle OEM in the trackKey response or eventNotification. Those options are specific to and implemented by the vehicle and may include the sharing password and/or alternatives to key activation via sharing password. Owner and friend device shall indicate them in the UI. If the field is absent, then no additional verification is required by the vehicle OEM. In this case, where no activationOptions are provided by Vehicle OEM, owner or device OEM can still require other verification methods not involving the vehicle OEM, as described in authType.

pinLength shall be a value between 4 and 8 (including 4 and 8), chosen by the owner device. A suitable combination of PIN length and allowed number of attempts to enter the device PIN in the friend device UI shall be chosen by the owner device, following the recommendations in [38].

brand is the string describing the vehicle brand as received by the owner device in trackKeyResponse (as per section 17.7.1.3).

model is the string describing the vehicle model as received by the owner device in trackKeyResponse (as per section 17.7.1.3).

vehicle_identifier is the value as provided in the endpoint configuration (as per section 15.3.2.4 tag $7F27_h$, subtag $4D_h$).

Table 11-3: sharingDataType enum Definition

Key	Value
sharingKeyCreationRequest (Key Creation Request, see Table 11-5)	1
sharingKeySigningRequest (Key Signing Request, see Table 11-7)	2
sharingImportRequest (Import Request, See Table 11-16)	3
sharingOwnerCancel	4
sharingFriendCancel	5
sharingPinReEntryRequest (Table 11-8 in section 11.3.6.2)	6
sharingPinReEntryValue (Key Signing Request, see Table 11-7)	7
…unknown additional entries shall be ignored, reserved for forward compatibility.	

The sharing types *sharingOwnerCancel* and *sharingFriendCancel* shall be used on any error condition instead of the next message of the sharing flow and shall contain an error message (see Table 11-10) with the appropriate error code.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The sharing type *sharingPinReEntryRequest* signals the friend that the entered device PIN was wrong. It contains the number of remaining attempts in the field *sharingData* in TLV format.

The sharing type *sharingPinReEntryValue* contains the key signing request with the new friend device PIN.

Sample Provisioning Information in CreateMailbox Message is shown in Listing 11-1.

Listing 11-1: Sample Provisioning Information in JSON Format

```
{
    "format": "digitalwallet.carkey.ccc",
    "content": {
        "genericSharingData": {
            "sharingData": "7F31xxxx",
// key creation request, this field changes depending on the sharingDataType value
            "sharingDataType": 1,
            "sharingId": "123456789...",
            "friendKeyId": "A2B686DFC..."
            "authType": ["DevicePIN"]
            "pinLength": 6
        },
        "apple": {
            // ...
        }
    }
}
```

11.3.4 *API Parameter Usage*

The usage of API parameters from [38] that are relevant to Digital Key Cross Platform Sharing are described in the following sections.

11.3.4.1 CreateMailbox API Parameters

The payload data contains *format* and *content fields* (see [38]). *content* contains *genericSharingData* and optional device OEM-specific sharing data as described in Table 11-1 *sharingDataType* enum (Table 11-3) shall be set to 1 (sharingKeyCreationRequest) to indicate that the content of s*haringData* is the key creation request as defined in Table 11-6.

The *displayInformation* is determined by the owner device. The format is described in [38].

The *notificationToken* is provided by owner device at the time of mailbox creation and is valid for the lifetime of the mailbox. It should be used by the owner device to receive a notification from the relay server when a Key Signing Request, sharing rejection or subsequent retry of device PIN (in context of Key Signing Request) is received by the Relay Server from the friend device. If notifications are not used, then the owner device shall perform regular polling as described in Section 11.3.1 subsection “Notifications”.

The *mailboxConfiguration* shall be set as follows:

- *accessRights* = “RWD”
- *expiration* = <owner device OEM policy>

The *urlLink* is formatted as described in [37].

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

isPushNotificationSupported shall be set to “true” by all CCC relay servers, i.e., they shall be supported by servers. Devices may choose to not use them although their use is strongly recommended.

11.3.4.2 UpdateMailbox API Parameters

The payload contains *format* and *content*. *content* contains *genericSharingData* and device OEM-specific sharing data as per Table 11-1.

sharingData from a friend device contains

- the Key Signing Request as per Table 11-7, or
- a *sharingFriendCancel* error message.

sharingData from an owner device contains one of the following:

- the Import Request as per Table 11-16, or
- a *sharingOwnerCancel* error message, or
- the Device PIN entry request message

The *notificationToken* should be provided by the friend device to be notified when data has been received by the Relay Server from the owner. The parameters are defined by the friend device. If notifications are not used, then the friend device shall perform regular polling as per Section 11.3.1 subsection “Notifications”.

11.3.4.3 DeleteMailbox API Parameters

The friend device shall delete the mailbox after successfully retrieving the Import Request.

If the owner receives a sharing cancellation from the friend (e.g., friend rejects the invitation), the owner device shall delete the mailbox.

If the friend receives a sharing cancellation from the owner (e.g., owner does not sign the friend key), the friend device shall delete the mailbox.

If the owner wants to cancel a sharing after the key has been signed, in addition to deleting the mailbox a Remote Termination Request shall be used as outlined in Figure 13-4 [REV_140]. The vehicle OEM shall add the friend key ID to a deny-list and reject key tracking (using substatus code 50112), if the friend key is not yet registered.

11.3.4.4 ReadDisplayInformationFromMailbox API Parameters

The *displayInformation* posted during mailbox creation can be retrieved again if required.

11.3.4.5 ReadSecureContentFromMailbox API Parameters

The payload contains the payload uploaded during creation or update of the mailbox according to the descriptions in the respective sections.

The *displayInformation* posted during mailbox creation is retrieved and used to display a placeholder for the final key graphics on the friend device.

The *expiration* of the mailbox indicates the maximum lifetime of the mailbox if it is not deleted before.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

11.3.4.6 RelinquishMailbox API Parameters

This API might be called to allow a device (e.g., a tablet) that receives the sharing invitation but cannot handle a digital key, to release the mailbox link and let another device connect to the mailbox at outlined in [38]. This allows another recipient to claim the mailbox and use the appropriate device to proceed with sharing procedures.

11.3.5 *Security Considerations*

The security considerations in [38] are generic. Some considerations how to apply them in the context of Digital Key:

- The slot identifier ensures that a key creation request can lead to the creation and acceptance of only one key based on this request
- If not using an approved sharing method, sharing should be protected by a second factor, which can be either a vehicle OEM-defined second factor (i.e., activation option) described in section 11.2.1.4and later, if supported by the vehicle, or a device OEM-defined second factor (e.g., the device PIN) as described in section 11.3.6.
- Sharing shall require explicit user authentication (as described in section 9.1 for RKE). Explicit user authentication is the combination of user intent and user authentication.

11.3.6 *Device PIN*

11.3.6.1 Concept

All devices shall support Device PIN that is generated on the owner device and entered on the friend device. Device PIN could be activated by the owner during the sharing workflow if no other vehicle OEM-defined activation option is used, or if the sharing password is not part of the activation options. Device UIs shall guide users to transfer the device PIN over a second channel and guard against the user unintentionally re-using the same channel used to communicate the sharing invitation. The friend then types the PIN into the friend device UI to continue the sharing workflow. The owner device then verifies that the correct PIN was entered into the friend device UI.

The owner device policy determines whether a sharing method requires the device PIN. The owner device policy is determined by the owner device OEM and is out of scope. To ensure a good user experience, it is recommended that multiple activation options that require user interface interaction should not be activated at the same time. For example, sharing password (controlled by vehicle OEM) and device PIN (controlled by device OEM)

Please see Section 11.2 entitled “Principles” about vehicle OEM-defined activation option and device PIN policies.

11.3.6.2 Basic Flow

The owner device generates the reference device PIN (O_PIN). It sets the number of device PIN entry attempts to a device OEM-defined value, considering the *pinLength* as described in Section 11.3.3.

The owner then sends the O_PIN over a second channel to the friend. This step shall be guided by the device UI to ensure that the device PIN is transmitted while complying with restrictions in 11.3.6.1. The owner device decrements the number of attempts by one.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The owner device sends the initial number of attempts in the key creation request via tag 9F17h. The presence of this tag indicates that the owner has requested the entry of device PIN to the friend device.

Table 11-4: Key Configuration

Tag	Length (bytes)	Description	Field is	Domain Version
$7F30_h$	variable	Key Configuration	Mandatory	V-OD-FW
$D0_h$	1	Standard access profiles as defined in Table 11-5	Conditional, present if no proprietary profile is present. Either Tag $D0_h$ or $D1_h$ shall be present	V-OD-FW
$D1_h$	variable	Proprietary access profile	Conditional, present if no standard profile is present. Either Tag $D0_h$ or $D1_h$ shall be present	V-OD-FW
51_h	15 or 13	not_before, DER encoded GeneralizedTime (15 bytes in length) or UTCTime (13 bytes in length) as per RFC 5280 [2]	mandatory	V-OD-FW
52_h	15 or 13	not_after, DER encoded GeneralizedTime (15 bytes in length) or UTCTime (13 bytes in length) as per RFC 5280 [3]	mandatory	V-OD-FW
$D3_h$	variable	Friendly name of the friend key. Limited to 30 characters	mandatory	V-OD-FW

Table 11-5: Standard Access Profiles

Profile	Type	Description
0	Full	Full access and Drive Capabilities
1	Access Only	Only car access, no drive capabilities
2	Access and Drive Restricted	Access and Drive with Restrictions
3	Car Delivery	Car Delivery Profile
4	Valet	Valet Parking Key
5	Vehicle Service	Vehicle Service Key
6 - 255	RFU	

Proprietary access profiles can be coded in a vehicle-specific method within tag $D1_h$.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1 *Table 11-6: Key Creation Request*

Tag	Length (bytes)	Description	Field is	Domain Version
$7F31_h$	variable	Key Creation Request		N/A
endpoint_configuration data field as described in Table 15-13 (without fields endpoint_identifier and instance_CA_identifier)			mandatory	V-OD-FW
key_configuration data field as described in Table 11-4			mandatory	V-OD-FW
59_h	variable	ROUTING_INFORMATION as generated during owner pairing described in Section 5.1.4.3	mandatory	V-OD-FW
$5C_h$	variable	DIGITAL_KEY_APPLET_PROTOCOL_VERSIONS as obtained during owner pairing described in Section 6 (This is the Digital Key applet protocol versions in which the owner received in SPAKE2+ REQUEST during owner pairing)	mandatory	V-OD-FW
$7F60_h$	variable	Friend sharing configuration derived from owner SHARING_CONFIGURATION as generated during owner pairing described in Table 5-14	optional	N/A
DA_h	variable	If this tag is present, friend device shall obtain immobilizer token and/or slot identifier. The following values are assigned: If this tag is present and the value is empty or the value is 00_h: immobilizer token and slot identifier from vehicle via owner device 01_h immobilizer token and slot identifier retrieved online 02_h no immobilizer token required, slot identifier from vehicle via owner device 03_h no immobilizer token required; slot identifier retrieved online	optional	V-OD-FW
DB_h	0	If this tag is present, friend device shall obtain a key tracking receipt as per Sections 11.8.5 and 11.8.6	optional	V-OD-FW
DC_h	0	If this tag is present, Vehicle OEM supports online attestation delivery feature as described in Section 11.10	optional	V-OD-FW
$4A_h$	3	2 bytes offset (unsigned big-endian) || 1 byte length to be returned from confidential mailbox in AUTH1 response as obtained during owner pairing in Section 6	optional	V-OD-FW
$4B_h$	3	2 bytes offset (unsigned big-endian) || 1 byte length to be returned from private mailbox in AUTH1 response as obtained during owner pairing in Section 6	optional	V-OD-FW
$D8_h$	1	SHARING_PASSWORD_LENGTH as obtained during owner pairing in Section 6 (may be used by the friend device UI). If absent, no sharing password is required.	optional	V-OD-FW

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
54_h	2 x m	m supported OD-FD-KS compatibility versions of owner device (ver.high | ver.low), ordered highest to lowest	mandatory	OD-FD-KS
$5E_h$	Var.	V-D-BT DK Protocol Version lists of vehicle (as per Table 5-4)	conditional	V-D-BT
80_h	2 x n	Supported_DK_Protocol_Version	conditional	V-D-BT
…unknown tags under $7F60_h$ shall be ignored, reserved for forward compatibility				
$9F17_h$	1	Number of device PIN entry attempts	Conditional. Required if device PIN is used	OD-FD-KS
$9F18_h$	1	Device PIN length (in digits)	Conditional. Required if device PIN is used	OD-FD-KS
…unknown additional tags under $7F31_h$ shall be ignored, reserved for forward compatibility				

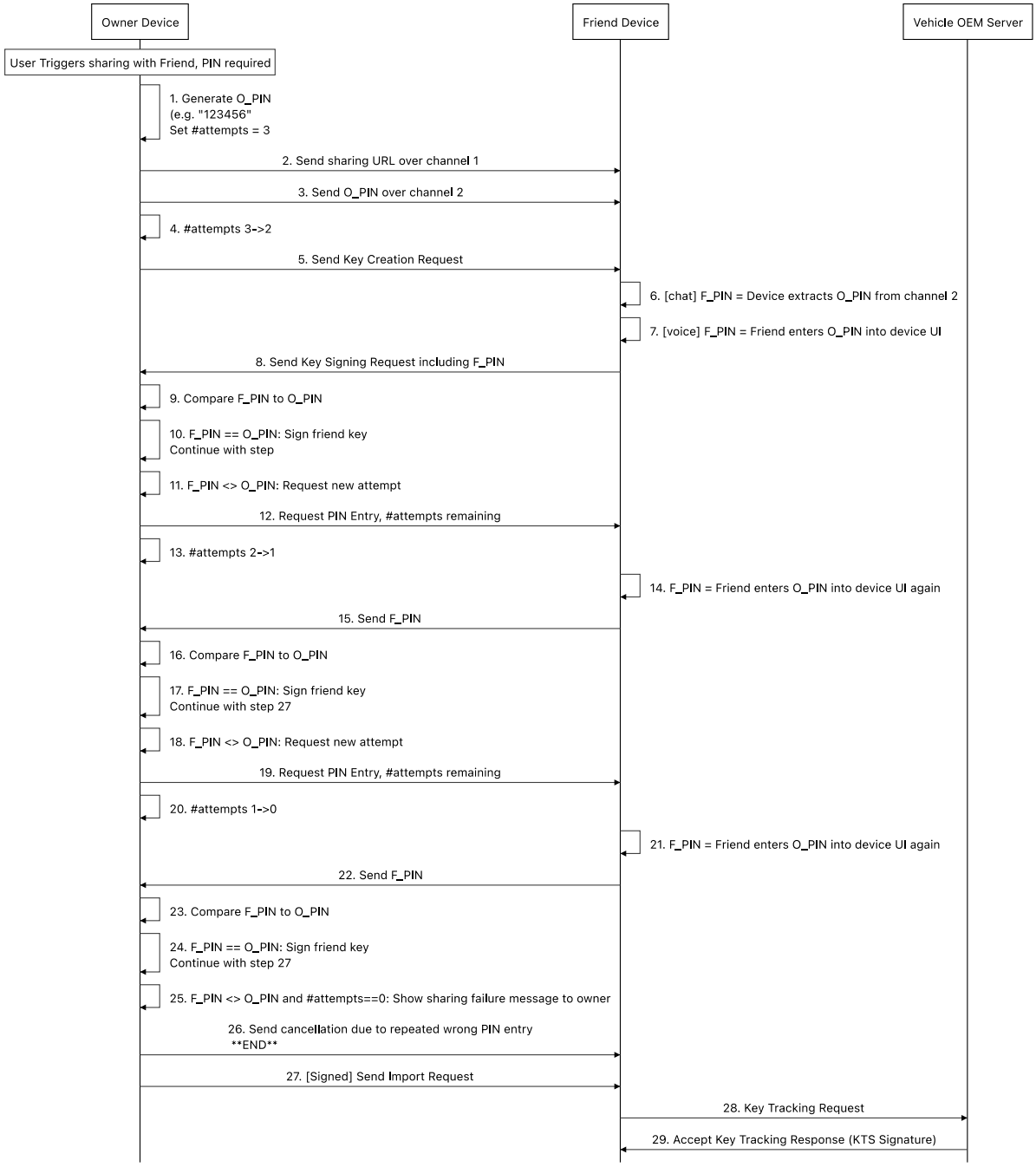
If tag D8h is present, then the sharing password shall be used and tags 9F17h and 9F18h shall not be present. If tag D8h is absent, then tags 9F17h and 9F18h shall indicate the device PIN length in digits to the friend device, if device PIN is required.

If tag $7F60_h$ is absent, then the default friend sharing configuration is:

- DA_h: 00_h
- DB_h: present
- DC_h: not present
- $D8_h$: not present

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 11-2: Device PIN Flow



For a more focused view, Figure 11-2 does not show the device OEM server.

The friend extracts the O_PIN value from the second channel or receives it over a voice call. This value is now named F_PIN. The friend enters the F_PIN into the friend device UI and the friend device adds the F_PIN to the key signing request.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 11-7: Key Signing Request

Tag	Length (bytes)	Description	Field is	Domain Version
$7F36_h$	variable	Friend Key Signing Request		OD-FD-KS
external CA Certificate container as per Table 11-12			mandatory	V-OD-FW
instance CA Certificate container as per Table 11-13			mandatory	V-OD-FW
endpoint certificate container as per Table 11-14 signed by instance CA private key			mandatory	V-OD-FW
endpoint encryption key attestation as described in Table 15-47 signed by endpoint private key			conditional	V-OD-FW
55h	2 x n	n supported OD-FD-KS compatibility versions of friend device (ver.high | ver.low), where n ≥ 16 for receivers and n ≤ 16 for transmitters. The agreed version is first in the list.	mandatory	OD-FD-KS
$5F3F_h$	variable	Friend-entered device PIN (F_PIN)	Conditional. Required if tags 9F17h and 9F18h have been sent in key creation request	OD-FD-KS

The owner device compares the received F_PIN with the O_PIN value. If both values are equal, then the owner device signs the friend key and sends the import request.

If the device PIN values are different, then the owner device sends a sharingPinReEntryRequest message (as per Table 11-8) to the friend. The sharingData structure is defined in Table 11-2.

Table 11-8: Device PIN Entry Request

Tag	Length (bytes)	Description	Field is
$9F17_h$	1	Number of device PIN entry attempts	mandatory

The friend device allows the friend to enter the F_PIN again and sends the input back by sending a sharingPinReEntryValue message (as per Table 11-7) to the owner. The content is identical with the key signing request in Table 11-7, but the sharingDataType indicates that this is a repeated device PIN entry attempt.

The owner compares again F_PIN with O_PIN and, if equal, decides to sign or, if different, to allow a new attempt to the friend device until either a correct F_PIN has been provided or the number of remaining attempts is 0.

If the number of remaining attempts is 0, then the owner sends a cancel message (Table 11-3, *sharingOwnerCancel*) indicating that no valid device PIN has been received in the defined number of attempts.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

If the friend device does not send back tag $5F3F_h$ (F_PIN) in the signing request, then the friend device does not support the device PIN feature. The owner device policy for this case is defined by the device OEM. Options could be to reject sharing to this device or propose to the owner to share without device PIN, providing some security education.

11.4 Certificate Chain

The goal of the certificate chain is to allow the owner's SE to verify that a friend's endpoint is authorized to participate in the sharing protocol. This verification is required to make sure that the friend key pair has actually been generated on an approved SE, and to ensure that any secret data transferred from owner's confidential mailbox to a friend's confidential mailbox shall be handled as per the rules described in Section 4.3.

Each owner endpoint contains one or several authorized_PK set by the vehicle during the owner pairing process as per Section 6. The authorized_PK list contains, at a minimum, the public key of the Vehicle OEM CA. The authorized_PK list may only contain intermediate CA or root CA but no leaf certificates. The External CA is typically the Device OEM CA used to issue the Instance CA Certificate of Digital Key applet. The Instance CA in turn issues the endpoint certificates.

In this version of the specification, only one single authorized_PK shall be set by the vehicle.

The verification of the friend endpoint is handled through the following verification chain. Each element to the right of the arrow is attested by the element above it:

Authorized CA (Vehicle OEM CA)
—> External CA (Device OEM CA)
—> Instance CA
—> Endpoint
—> Endpoint Encryption Key

11.5 Prerequisites

1. The Digital Key applet helper methods described in Section 15.4 or equivalent implementation are available on friend and owner device.
2. Owner and friend devices contain the necessary key material to authenticate remote servers. This key material is stored on the Digital Key framework.
3. Owner and friend Digital Key applet or equivalent implementation possess an Instance CA dedicated to the Vehicle OEM brand involved in sharing. The issuance of Instance CA is described in Section 16.
4. Owner and friend devices have retrieved an External CA certificate as defined in Section 16.2.6. This certificate is stored in the Digital Key framework of both devices.
5. The owner pairing has been successfully executed on the owner device, as per Section 6.
6. If applicable, immobilizer tokens and slot identifiers are available on the owner device's confidential and private mailboxes, as per Section 6.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

7. The following values have been obtained by the owner, as per Section 6 or equivalent preliminary operations. These values are stored on the Digital Key framework:
   (a) IMMOBILIZER_TOKEN_LENGTH
   (b) SIGNALING_BITMAP_OFFSET
   (c) SLOT_IDENTIFIER _BITMAP_OFFSET
   (d) SLOT_IDENTIFIERS_OFFSET
   (e) VEHICLE_OEM_PROPRIETARY_DATA_OFFSET
   (f) ATTESTATION_PACKAGE_OFFSET
   (g) MAILBOX_CONTENT_END_OFFSET
   (h) LONG_TERM_SHARED_SECRET
   (i) SHARING_PASSWORD_LENGTH
   (j) ROUTING_INFORMATION
   (k) DIGITAL_KEY_APPLET_PROTOCOL_VERSIONS
   (l) SHARING_CONFIGURATION
8. Owner and friend OEM Server have implemented the Key Life Cycle Management functions especially key termination and deletion as described in section 13.

11.6 Key Configuration

A TLV encoded schema defines the Digital Key configuration with available access profiles and their encoding. This schema is described in Table 11-4. Note that prior versions of this specification used ASN.1 formatting for the Digital Key configuration.

11.7 Error Codes

The following codes may be sent from owner device to friend device or vice versa if an error occurs during the sharing process. The sharing procedure shall be aborted if an error is returned by friend or owner.

The error codes are returned in the TLV structure as shown in Table 11-9. All error codes that are unknown to the receiver shall be treated as FF_h (unspecified error).

Table 11-9: Error Code Message

Tag	Length (bytes)	Description	Field is	Domain Version
$7F6C_h$	variable	Error Message		OD-FD-KS
$6D_h$	1	Error code as per Table 11-10	mandatory	

Table 11-10: Error Codes

Error code	Description
00_h	Missing mandatory field
01_h	Invalid message structure
02_h	Invalid message content
03_h	Version not supported
04_h	Certificate expired

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Error code	Description
05_h	Invalid certificate structure
06_h	Invalid certificate content
07_h	Invalid certificate chain
08_h	Request rejected
09_h-$1F_h$	RFU
20_h	Sharing cancelled
21_h	Sharing cancelled by owner – no valid Pin received after max number of attempts
22_h	Sharing cancelled by friend
23_h-FE_h	RFU
FF_h	Unspecified error

11.8 Key Sharing Flow: Steps

Figure 11-1 shows the main key sharing flow and describes the content of the data packets exchanged between the owner and friend devices independently of the communication channel or transport layer used to convey these packets.

At the end of this flow, the friend's private mailbox contains an attestation package in which the friend's public key is signed by the owner private key along with a set of entitlements. The friend's confidential mailbox optionally contains an active immobilizer token.

The last step describes how the attestation package is delivered and verified by the vehicle during the first friend transaction.

11.8.1 *Steps 1 and 2 (Owner): Sharing Invitation*

The owner device sends a Key Creation Request containing the required endpoint configuration and the entitlements to be granted. The endpoint_identifier and instance_CA_identifier from field endpoint_configuration shall not be included since their content is defined by the friend device.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 11-11: Approved Sharing Methods

Tag	Length (bytes)	Description	Field is
7F10$_h$	variable	list of approved sharing methods	
61$_h$	variable	Tuple of first sharing method provider identifier and first sharing method group identifier	conditional
40$_h$	2	Sharing method provider identifier as defined in XXX (e.g., 0x0000 = CCC, 0x0001 Apple, 0x0002 = Google, 0x0003 = OEM 3…)	mandatory
41$_h$	variable	Sharing method group identifier defined by the sharing method provider	mandatory
61$_h$	variable	Tuple of second sharing method provider identifier and second sharing method group identifier	optional
40$_h$	2	Sharing method provider identifier as defined in XXX (e.g., 0x0000 = CCC, 0x0001 Apple, 0x0002 = Google, 0x0003 = BMW…)	optional
41$_h$	variable	Sharing method group identifier defined by the sharing method provider	optional

Listing 11-2: Sharing Invitation

```
input: user action
output: key_creation_request
begin
  generate a key_configuration field as per Table 11-4, owner might be prompted with a specific UI for this action.
  key_configuration.updateCounter ⟵ 0

if immobilizer token to be retrieved from the owner device (i.e. Tag DAh in Table 5-14 is not present, or if Tag DAh is present and the corresponding value is empty or is set to 00h)
  slot_identifier ⟵ select the immobilizer token to be shared and save its corresponding slot identifier
else if no immobilizer tokens are used, and slot identifier is retrieved from the owner device (i.e., Tag DA_h in Table 11-5 is present and its value is set to 02_h)
  slot_identifier ⟵ save slot identifier
  generate an endpoint_configuration field as described in Table 15-13 with
    endpoint_configuration ⟵ copy owner's endpoint_configuration
    endpoint_configuration ⟵ remove endpoint_identifier and instance_CA_identifier fields
    endpoint_configuration.option_group_1.bit4 ⟵ 0
    endpoint_configuration.option_group_1.bit5 ⟵ 0
    endpoint_configuration.endpoint_identifier ⟵ see naming rules for friend key (see Section 4.2)

    if confidential_mailbox_size field present in owner's endpoint configuration
      endpoint_configuration.confidential_mailbox_size ⟵ 1 × IMMOBILIZER_TOKEN_LENGTH
    if slot_identifier to be retrieved from the owner device
      endpoint_configuration.key_slot ⟵ slot_identifier

  generate key_creation_request as per Table 11-6 using key_configuration, endpoint_configuration,
    ROUTING_INFORMATION (, friend sharing configuration)

  if sharing password required by vehicle OEM policy or device policy (see Section 11.11)
    sharing_password_seed ⟵ generate random bytes
    generate sharing_password as per Listing 11-12 using
      LONG_TERM_SHARED_SECRET
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        SHARING_PASSWORD_LENGTH
        sharing_password_seed
    store the sharing_password and sharing_password_seed on owner device
    the owner device can display the sharing_password with specific distribution instructions

  send key_creation_request to friend over owner-friend-com
end
```

11.8.1.1 Versioning

If non-backward compatible changes in the key creation request (Table 11-6) are necessary, then a new tag (e.g., $7F33_h$) shall be defined and both data structures shall be sent to the friend device, which can then pick the tag it is able to process.

Related applet commands:

- CREATE ENDPOINT (friend)

For forward compatibility, the friend device shall accept other tags from the owner device under $7F31_h$ in the Key Creation Request. If not known, these tags shall be ignored by the friend device. In future versions, the owner device may send other tags.

If Tag 54_h in Table 11-6 is recognized, then the friend device shall determine a commonly supported key sharing version (OD-FD-KS), send its supported version numbers back in the Key Signing Request (see section 11.8.1.1) and continue the key sharing process according to the determined version number. If the tag is not recognized, the friend device shall behave as prior to version introduction.

The endpoint configuration data shall contain all data elements from older versions to be backward compatible. New tag shall be used if format change or extension is needed.

The friend device shall reject the sharing if the ROUTING_INFORMATION format cannot be handled by the device OEM server, which processes the data. This data format is determined before the sharing starts, so its version or format cannot be changed.

The friend device shall reject the sharing or ask the user to update if DIGITAL_KEY_APPLET_PROTOCOL_VERSIONS are not compatible with the friend applet protocol version. This version list is determined by the vehicle, the device needs to find a matching version from that list.

The friend device shall reject the sharing or ask the user to update if any version of the set of vehicle version lists in tag $5E_h$ is not compatible with the supported friend BT versions.

Note that tags $4A_h$, $4B_h$ and $D8_h$ are not likely to change their format. If this becomes necessary, new tags shall be defined and sent in addition.

11.8.2 *Steps 3 and 4 (Friend): Key Signing Request*

The friend device creates an endpoint using the provided endpoint_configuration, then transfers to the owner a Key Signing Request containing the endpoint certificate along with the associated certificate chain.

The friend device uses the list of authorized_PK provided by the owner in the endpoint_configuration field in order to produce a certificate chain starting from one of these public keys. In this version of the specification, the he created friend endpoint certificate shall not contain any authorized_PK.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 11-12: External CA Certificate Container

Tag	Length (bytes)	Description	Field is
$7F20_h$	variable		
external CA certificate as per Listing 15-13 verifiable by an authorized_PK			mandatory

Table 11-13: Instance CA Certificate Container

Tag	Length (bytes)	Description	Field is
$7F22_h$	variable		
instance CA Certificate as per Listing 15-16 signed by external CA private key			mandatory

Table 11-14: Endpoint Certificate Container

Tag	Length (bytes)	Description	Field is
$7F24_h$	variable		
endpoint certificate as Listing 15-5 signed by instance CA private key			mandatory

Listing 11-3: Key Creation Processing

```
input: key_creation_request
output: friend_key_signing_request
begin
  extract key_configuration, endpoint_configuration from key_creation_request
  check the applet installed on platform supports at least one of the protocol versions present in
DIGITAL_KEY_APPLET_PROTOCOL_VERSIONS
  check proposed entitlements received from owner comply with friend's policy,
    friend may be prompted with a specific UI for this action.
  friend may decline sharing and stop the sharing process at this point.
  check endpoint_configuration complies with friend's policy
  check ability to produce a certificate chain starting from one of the authorized_PK
    present in endpoint_configuration

  execute framework.getInstance as described in Section 15.4.1.3
    output: instance

  generate and add endpoint_identifier, instance_CA_identifier to endpoint_configuration as
    per friend device's local policy
  Remove all element from authorizedPK list from endpoint_configuration
  execute instance.createEndpoint as described in Section 15.4.1.7
    input: endpoint_configuration,
          bool_onlineCertificate = false
    output: endpoint

  execute endpoint.getCertificate as described in Section 15.4.1.12
    input: bool_onlineCertificate = false
    output: endpoint_certificate

  if tag DA_h present in key_creation_request and set to 01_h

    execute endpoint.createEncryptionKey as described in Section 15.4.1.15
      output: encryption_key_attestation

  generate friend_key_signing_request as per Table 11-7
    using external_ca_certificate,
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        instance_ca_certificate,
        endpoint_certificate,
        (encryption_key_attestation)

else
   generate friend_key_signing_request as per Table 11-7
       using external_ca_certificate,
             instance_ca_certificate,
             endpoint_certificate

 send friend_key_signing_request to owner over the friend device to owner device link
end
```

11.8.2.1 Versioning

Related applet commands:

- AUTHORIZE ENDPOINT (owner)

The friend device generates the certificates and attestations in the key signing request (Table 11-7) using the highest version that is compliant with the agreed key sharing version (OD-FD-KS).

It then sends the key signing request with the added list of supported friend versions. The selected, commonly supported version is first in the list.

11.8.3 *Steps 5 and 6 (Owner): Generate Import Request*

The owner generates an attestation over the friend's public key and optionally exports an immobilizer token from the owner endpoint's confidential mailbox. The owner sends an Import Request to the friend.

The owner shall generate a sharing method attestation and include it into the import request unless agreed otherwise with the vehicle OEM.

Table 11-15: Sharing Method Attestation Arbitrary Data to be Hashed and Signed

Tag	Length (bytes)	Description	Field is
61_h	variable	Tuple of used sharing method provider identifier and used sharing method group identifier	conditional
40_h	2	Sharing method provider identifier as defined in XXX (e.g., 0x0000 = CCC, 0x0001 Apple, 0x0002 = Google, 0x0003 = BMW…)	mandatory
41_h	variable	Sharing method group identifier defined by the sharing method provider	mandatory
51_h	20	Friend key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey	mandatory

Table 11-16: Import Request

Tag	Length (bytes)	Description	Field is	Domain Version
$7F32_h$	variable	Mailbox Import Request		OD-FD-KS

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
key_attestation as described in Table 15-60 (containing entitlement data as per Table 11-17, and arbitrary_data in Table 15-23)			mandatory	V-OD-FW
the mailbox_mapping table referenced by tag $7F4D_h$ as described in Table 5-13			mandatory	V-OD-FW
$4A_h$	variable	Encrypted confidential mailbox data	conditional	V-OD-FW
97_h	65	Encryption public key of the owner prepended by 04_h	conditional	V-OD-FW
$7F49_h$	variable	This tag, defined in Table 19-86, provides configuration data for the supported radios	Conditional. Shall be present if the vehicle supports WCC2/WCC3	V-OD-FW
$7F2D_h$	variable	SharingMethodAttestation Arbitrary data attestation (as per Table 15-61) with owner key using arbitrary data = SHA-256 hash value of Table 11-15	optional	
$7F11_h$	variable	Content of Table 11-15 without tag 51_h.	Conditional. Present when $7F2D_h$ is present.	

1

2 *Table 11-17: Entitlement Data*

Tag	Length (bytes)	Description	Field is	Domain Version
Key configuration as per Table 11-4			mandatory	Informative
$4E_h$	variable	slot identifier	Conditional: present if Tag DA_h in Table 11-6 is empty, or set to values 0x00 or 0x02	V-OD-FW
44_h	8	sharing_password_seed	Conditional: present if second factor authentication is required by vehicle OEM	V-OD-FW
$D7_h$	1	second_factor_activation_required (01_h=required for non-approved sharing methods, other=RFU)	Conditional: present if second factor activation is required for non-approved sharing methods by the vehicle OEM	V-OD-FW
$4D_h$	8	vehicle_identifier	Conditional: present if Tag DA_h in Table 11-6	V-OD-FW

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
			is set to values 0x01 or 0x03	

If slotIdentifier is obtained from vehicle (via Owner Device) in keyCreationRequest, then slotIdentifier identified in Table 11-17 shall equal the value set during keyCreationRequest (Table 11-6) and vehicleIdentifier shall be absent.

If friend device obtains the slotIdentifier online in keyCreationRequest then the slotIdentifier shall be obtained later through the Key Tracking Response as per Section 11.10.1

Listing 11-4: Import Request

```
input: friend_key_signing_request
output: import_request
begin
  extract from friend_key_signing_request
    externalCACertificate,
    instanceCACertificate,
    endpointCertificate,
    (encryptionKeyAttestation)

  retrieve key_configuration sent in step1
  verify endpointCertificate has been created as per step1 and step2 rules, otherwise abort procedure

  generate an entitlement_data field as per Table 11-17
If key_slot present in key_creation_request sent in step1
  entitlement_data.slotIdentifier ⟵ key_slot
  entitlement_data.vehicleIdentifier ⟵ vehicle_identifier
  entitlement_data.keyConfiguration ⟵ key_configuration

  if second_factor_authentication_required by Vehicle OEM policy or device policy (see Section 11.12)
    entitlement_data.sharingPasswordSeed ⟵ sharing_password_seed
    entitlement_data.second_factor_authentication_required ⟵ true
  else
    Entitlement_data.second_factor_authentication_required ⟵ false

  execute framework.getInstance as per Section 15.4.1.3
    output: instance

  execute instance.getEndpoint as per Section 15.4.1.9
    output: endpoint

    I ⟵ index of the slot identifier selected in step 1
    clearPrivateMailboxBit(endpoint, i, SLOT_IDENTIFIER_BITMAP) as per Listing 11-9
    setPrivateMailboxBit(endpoint, SLOT_IDENTIFIER_BITMAP_BIT, SIGNALING_BITMAP_OFFSET) as per Listing 11-8

 if immobilizer token required
      immo_token_extraction_offset = i × IMMOBILIZER_TOKEN_LENGTH
      immo_token_extraction_length = IMMOBILIZER_TOKEN_LENGTH
 else
      immo_token_extraction_offset = 0
      immo_token_extraction_length = 0

  execute endpoint.authorize as per Section 15.4.1.14
    input:
     u16_offset = immo_token_extraction_offset
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
      u16_length = immo_token_extraction_length
      bytes_arbitraryData = entitlement_data
      bytes_externalCACertificate = externalCACertificate
      bytes_instanceCACertificate = instanceCACertificate
      (bytes_encryptionKeyAttestation = encryptionKeyAttestation)
      bytes_endpointCreationCertificate = endpointCertificate
    output: bytes_attestation(, bytes_encryptedData, bytes_encSenderePK)

  prepare import_request buffer as per Table 11-16 using
    bytes_attestation,
    (bytes_encryptedData),
    (bytes_encSenderePK)

  send import_request to friend over the owner device to friend device link
end
```

11.8.3.1 Versioning

Related applet commands:

- CREATE ENCRYPTION KEY (owner)
- SET CONFIDENTIAL DATA (friend)

The import request includes data from different sources.

The key attestation format has been pre-agreed between owner device and vehicle (V-OD-FW). Its version is not relevant for the friend device.

The mailbox mapping table is versioned by V-OD-FW, the friend device shall check if the version of the mapping table can be processed.

The confidential mailbox data format and encryption public key is also pre-agreed by V-OD-FW. The encryption algorithm is defined by the SET CONFIDENTIAL DATA API and shall be considered when determining by the OD-FD-KS version.

The configuration data for the supported radios is based on Table 19-86, which supports adding new tags for backward compatibility.

11.8.4 *Step 7 (Friend): Endpoint Data Import*

The key_attestation provided by the owner or the Vehicle OEM Server, shall be written into the friend's private mailbox, and optionally the immobilizer token may be written into the friend's confidential mailbox. The friend's private mailbox has the same mapping as the owner mailbox. The friend's confidential mailbox contains the same or fewer immobilizer tokens than the owner confidential mailbox. The key slot identifier shall only be included by the friend device when slot identifiers are retrieved online.

Table 11-18: Attestation Package

Tag	Length (bytes)	Description	Field is	Domain Version
$7F35_h$	variable	Attestation Package		V-OD-FW
41_h	1	Certificate version = 02_h	mandatory	V-OD-FW
92_h	8	Random as described in Table 15-23	mandatory	V-OD-FW

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
$5A_h$	65	Public key from friend endpoint prepended by 04_h	mandatory	V-OD-FW
78_h	variable	Entitlements data as per Table 11-17	mandatory	V-OD-FW
$9E_h$	64	Signature with owner endpoint private key over Section Table 15-23 as per Listing 11-2	mandatory	V-OD-FW
$4E_h$	1-8	key_slot_identifier – only present if key slot identifiers are retrieved online (i.e., tag DA_h in Table 11-6 is present and set to 01 or 03)	conditional	V-OD-FW
45_h	variable	key_tracking_receipt	conditional	V-OD-FW

Listing 11-5: Friend, Endpoint Data Import Processing

```
input: import_request (Table 11-16)
output: n/a
begin
  extract tags content from import_request as separate buffers:
    key_attestation
    mailbox_mapping
    (encrypted_confidential_mailbox_data),
    (encryption_public_key),

  extract friend private mailbox mapping offsets from mailbox_mapping

  execute framework.getInstance as per Section 15.4.1.3
    output: instance

  execute instance.getEndpoint as per Section 15.4.1.9
    output: endpoint

  if key tracking required as per tag DBh from Table 11-6 or
    online attestation delivery is supported as per tag DCh from Table 11-6
      Execute Listing 11-7 from Section 11.10to retrieve response

  prepare attestationPackage buffer as per Table 11-18 using contents of:
     key_attestation,
    (key_tracking_receipt)

  execute endpoint.setPrivateData as per Section 15.4.1.18
    input: offset = ATTESTATION_PACKAGE_OFFSET, data = attestationPackage

  setPrivateMailboxBit(endpoint, ATTESTATION_PACKAGE_BIT, SIGNALING_BITMAP) as per Listing 11-8

  from that point a notification indicating success of the online attestation delivery can be received from server
    on reception of such notification the following cleanup procedure is executed:
      clearPrivateMailboxBit(endpoint, ATTESTATION_PACKAGE_BIT, SIGNALING_BITMAP) as per Listing 11-9
      execute endpoint.setPrivateData as per Section 15.4.1.18
        input: offset = ATTESTATION_PACKAGE_OFFSET, data = zeroes

  if tag 4Bh was received in Step 1
    execute endpoint.setParameters with the following parameters:
      input: offset_private = offset extracted from tag 4Bh,
           length_private = length extracted from tag 4Bh
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
  if tag DAh from Table 11-6  was present in Step 1, and is empty or equal to 00h or 01h
    execute endpoint.setConfidentialData as per Section 15.4.1.16
      input:
        bytes_encsenderepk = encryption_public_key
        bytes_encdata = encrypted_confidential_mailbox_data
        u16_offset = 0
        u16_length = IMMOBILIZER_TOKEN_LENGTH

  if tag 4Ah was received in Step 1
      execute endpoint.setParameters with the following parameters:
        input: offset_confidential = offset extracted from tag 4Ah,
               length_confidential = length extracted from tag 4Ah
  setPrivateMailboxBit(endpoint, 0, SLOT_IDENTIFIER_BITMAP) as per Listing 11-8

end
```

11.8.5 *Steps 8 and 9 (Friend): Track Key*

If key tracking is required by the Vehicle OEM, the friend device in Step 9 shall send trackKey (see Section 17.7.1.2) to the Vehicle OEM Server via the friend Device OEM Server to obtain a key tracking receipt. Note that step 8 is shown for illustration purposes only.

If slot identifier is retrieved online, the friend device shall keep the key disabled on the NFC and BLE interfaces until a valid slot identifier is available. The key shall be usable on the NFC and BLE interfaces only after a successful trackKeyResponse(see Section 17.7.1.3) containing the slot identifier.

11.8.6 *Step 10 (Vehicle OEM Server): Track Key Response*

In step 10, the Vehicle OEM Server shall send trackKeyResponse (see Section 17.7.1.3) to the friend Device OEM Server. Note that step 11 is shown for illustration purposes only.

11.8.7 *Step 14 (Friend): First Transaction*

This section describes the operations occurring between friend device and vehicle during the first presentation of a friend endpoint on which operations described in Section 11.8.3.1 have been successfully completed.

Listing 11-6: First Friend Transaction

```
begin
  vehicle and device execute the standard transaction as described in Section 7 until
    AUTH1 response from the device
  device AUTH1 response contains: (as described in Section 15.3.2.10)
    the key_slot
    the endpoint_signature
    (confidential_mailbox_subset: offset = 0, length = IMMOBILIZER_TOKEN_LENGTH)
    private_mailbox_subset:offset = SIGNALING_BITMAP_OFFSET, length = 1
  vehicle extracts signaling_bitmap ⟵ private_mailbox_subset[0]
  if immobilizer token required by vehicle
    vehicle extracts immobilizer_token ⟵ confidential_mailbox_subset[0]
  vehicle verifies if ATTESTATION_PACKAGE_BIT in signaling_bitmap is not set
    no attestation package is present, exit and process as regular transaction
  vehicle computes attestation_package_length ⟵ (END_OFFSET–- ATTESTATION_PACKAGE_OFFSET)
  vehicle issues one or multiple EXCHANGE commands to retrieve the attestation package
  if vehicle signature verification of attestation package with owner key is successful
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    vehicle verifies key has not been registered yet, otherwise abort
    vehicle verifies value of slot_identifier is valid, otherwise abort (conditional, this step is not performed if slot identifier is retrieved online)
    vehicle verifies all elements of the attestation package are compliant with its policy, otherwise abort
    vehicle registers endpoint.PK and entitlements
   vehicle verifies the endpoint_signature received in AUTH1
  else
    abort procedure
  vehicle clears ATTESTATION_PACKAGE_BIT in signaling_bitmap
  vehicle clears the attestation package from device private mailbox using the one or multiple EXCHANGE commands

  using value from attestationPackage.second_factor_authentication_required and information present in key tracking receipt
    the vehicle may require the sharing_password generated and verified as
      sharing_password_seed ←vehicle extracts from attestationPackage
      vehicle generates sharing_password as per Listing 11-12 using
        LONG_TERM_SHARED_SECRET
        SHARING_PASSWORD_LENGTH
        sharing_password_seed
      vehicle verifies the sharing_password by requesting friend input on a specific vehicle UI
    if vehicle supports feature from Section 11.11 and sharing_password is successfully presented
      vehicle generates vehicleAttestation as per Section 11.11 and sends it to Vehicle OEM Server
End
```

The vehicle might require additional activation of a shared key to start the engine. In this case the activation options have been provided to the owner in the key tracking request or via event notification. The owner has provided the list of activation options to the friend during key sharing.

11.9 Owner Device OEM Server Notification

Following a successful activation of the shared key, the Vehicle OEM Server shall send an event-Notification SHARED_KEY_ADDED (see Section 17.7.3.2) to the owner Device OEM Server. This may occur before or after the first friend contact.

11.9.1 *Step 12 (Vehicle OEM Server): eventNotification*

In step 12, the Vehicle OEM Server shall send eventNotification (see Section 17.7.3.2) to the owner Device OEM Server.

11.9.2 *Step 13 (Owner Device OEM Server): eventNotificationResponse*

In step 13, the owner Device OEM Server shall send eventNotificationResponse (see Section 17.7.3.3) to the Vehicle OEM Server.

11.10 Key Tracking and Online Attestation Delivery

If required by the Vehicle OEM, a key tracking receipt computed over elements from Table 11-18 shall be verified by the vehicle before accepting a new key. In that case, the key sharing flow requires an additional interaction with a Vehicle OEM Server.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The key tracking receipt may also contain proprietary instructions from the Vehicle OEM requesting the vehicle to verify the sharing password. The Vehicle OEM should consider the consistency for sharing password required in Digital Key creation and key tracking response.

If supported by the Vehicle OEM, the attestation package provided may also be sent directly to the vehicle via the Vehicle OEM telematic link (if the vehicle is reachable) in order to speed up the first friend transaction. This feature is called Online Attestation Delivery.

The cryptographic algorithm and final data elements used by the KTS to produce the receipt are specific to each Vehicle OEM and are out of scope of this specification.

Section 17.7.1 (API – Track Key) describes how the data elements highlighted in the paragraphs above are packaged and sent to the Vehicle OEM. This function assumes the Vehicle OEM Server possesses the owner's public key in order to verify the signature of the incoming packets.

Before the key tracking has taken place, a termination request may be received by the vehicle OEM server. In such a case, a key tracking receipt shall not be sent in response

The Vehicle OEM Server may store the friend's endpoint public key and the Instance CA public key. The Device OEM may store an immutable unique device identifier for a given Instance CA public key, such that if the two parties cooperate, it is possible to link an endpoint public key with an immutable unique device identifier for legal and investigational purposes.

Table 11-19: Key Tracking and Online Attestation Delivery Request

Tag	Length (bytes)	Description	Field is	Domain Version
$7F38_h$	variable	Friend Key Tracking and Online Attestation Delivery Request		V-OD-FW
attestation package as described in Table 11-18 without tag 45_h (key tracking receipt)			mandatory	V-OD-FW
friend endpoint certificate container as per Table 11-14 signed by instance CA			mandatory	V-OD-FW
friend Instance CA Certificate container as per Table 11-13 signed by external CA			mandatory	V-OD-FW
endpoint encryption key attestation as per Table 15-47 signed by endpoint private key (present only if immobilizer token is required)			conditional	D-VS
$5F49_h$	65	Device privacy encryption key (Device.Enc.PK)	mandatory	D-VS
DA_h	variable	Device privacy encryption version, Default "ECIES_v1" (ASCII)	mandatory	D-VS
$7F2D_h$	variable	SharingMethodAttestation Arbitrary data attestation (as per table 15-61) with owner key using arbitrary data = SHA-256 hash value of Table 11-15	Conditional. Friend device shall provide the attestation if it has been received from owner device	D-VS
$7F11_h$	variable	Content of Table 11-15 without tag 51_h.	Conditional: present when $7F2D_h$ is present.	D-VS

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 11-20: Key Tracking Response

Tag	Length (bytes)	Description	Field is	Domain Version
$4E_h$	1–8	Slot_identifier to be used by friend endpoint	conditional	V-OD-FW
45_h	variable	Key tracking receipt	conditional	V-OD-FW
$4A_h$	variable	Encrypted confidential mailbox data (immobilizer token present only if immobilizer token is required)	conditional	D-VS
97_h	65	Ephemeral encryption public key of the Vehicle OEM Server prepended by 04_h (present only if immobilizer token is required)	conditional	D-VS

Endpoint encryption key attestation (Tag $7F26_h$) shall be included in Table 11-19 when Tag DA_h in tag $7F60_h$ in Table 11-6 is set to 01_h.

Friend slot identifier (Tag $4E_h$) shall be included in Table 11-20 when tag DA_h in Tag $7F60_h$ in Table 11-6 is set to 01_h or 03_h.

Friend immobilizer token (Tags $4A_h$ and 97_h) shall be included in Table 11-20 when Tag DA_h in Tag $7F60_h$ in Table 11-6 is set to 01_h.

Listing 11-7: Key Tracking Receipt

```
begin
  device creates friend device to Device OEM Server link
  device prepares message as per Table 11-19
  device sends message over the Device OEM Server link to Vehicle OEM Server link as per Section 17 (API – trackKey)
  Vehicle OEM Server verifies owner signature contained in attestation package
  Vehicle OEM Server optionally tries to push the attestation package via vehicle telematic link
  Vehicle OEM Server optionally computes key tracking receipt as
    per vehicle OEM proprietary specification
  device includes  tag 45h and, if slot identifier is retrieved online 4Eh from Table 11-20 received from server into Table 11-18
    response may be empty if vehicle OEM does not require key tracking receipt
  device updates slot identifier as per Section 15.3.2.21 SETUP ENDPOINT command (if slot identifier is retrieved online)
  device writes confidential mailbox using encrypted confidential mailbox data as per Section 15.3.2.20 SET CONFIDENTIAL
DATA command (if immobilizer token is retrieved online)
end
```

11.10.1 *Online immobilizer token and slot identifier retrieval*

As an optional feature, the immobilizer token and/or the slot identifier may be retrieved by the friend online, together with the Online Attestation package.

In this case, the owner shall not send a slot identifier in the Key Creation Request and shall set the tag DA_h (see Table 11-6) to 01_h to indicate the use of online immobilizer token, or to 03_h to indicate the use of online slot identifier without immobilizer token. After the key signing request, the owner will only use the key identifier to identify the friend key. The Key Tracking Response includes the immobilizer token and/or the slot identifier which has been assigned to this friend key by the Vehicle OEM server. The friend shall store the immobilizer token in the confidential mailbox using the SET CONFIDENTIAL DATA command and/or update the slot identifier using the SETUP ENDPOINT command.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

11.11 Vehicle Attestation

As an optional feature, the vehicle may be able to transmit an attestation to the owner device when a friend enters a correct sharing password in the vehicle. Upon verification of this attestation, the owner device may waive the sharing password requirement for subsequent sharing with the same friend (identified by the same issuing Instance CA) if Device OEM and Vehicle OEM policies allow it. Vehicle OEM should consider the consistency for sharing password required in Digital Key creation and key tracking response.

The vehicle transmits the attestation to the Vehicle OEM Server via the telematic link. The attestation is then transmitted to the owner device via the implemented communication channel.

Table 11-21: Vehicle Attestation Data Fields

Tag	Length (bytes)	Description	Field is
41_h	1	version = 01_h, version of the Vehicle Attestation Data Fields	mandatory
92_h	8	random	mandatory
$5A_h$	65	friend public key	mandatory
44_h	8	sharing_password_seed	mandatory
93_h	4	usage = $E670F31B_h$	mandatory

Table 11-22: Vehicle Attestation

Tag	Length (bytes)	Description	Field is
$7F34_h$	variable	Vehicle Attestation	
key_attestation as described in Table 11-21			conditional
$9E_h$	variable	signature over fields in Table 11-21 using vehicle private key as per Table 11-19	optional

11.12 Algorithms

Listing 11-8: setPrivateMailboxBit

```
input: endpoint, bit_index, offset
output: n/a
begin
  execute endpoint.getPrivateData as per Section 15.4.1.17
    input: offset, length = 1
    output: one_byte

  mask = 1
  mask = mask << bit_index
  one_byte = one_byte | mask

  execute endpoint.setPrivateData as per Section 15.4.1.18
    input: offset, data = one_byte
end
```

Listing 11-9: clearPrivateMailboxBit

```
input: endpoint, bit_index, offset
output: n/a
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
begin
  execute endpoint.getPrivateData as per Section 15.4.1.17
    input: offset, length = 1
    output: one_byte

  mask = 1
  mask = mask << bit_index
  mask = ~mask
  one_byte = one_byte & mask

  execute endpoint.setPrivateData as per Section 15.4.1.18
    input: offset, data = one_byte
end
```

Listing 11-10: clearPrivateMailboxBytes

```
input: endpoint, offset, length
output: n/a
begin
  generate empty_buffer using length
  execute endpoint.setPrivateData as per Section 15.4.1.18
    input: offset, data = empty_buffer
end
```

Listing 11-11:Generate Attestation Signature

```
input: data_fields, private_key
output: signature
begin
  set curve_parameters ⟵"ECC NIST P-256" as per [8]
  signature ⟵ ECDSA(curve_parameters, private_key, data_fields) using SHA-256 as per [8]
  return signature (64 bytes)
end
```

Listing 11-12: Generate Sharing Password

```
input: long_term_shared_secret, sharing_password_length, sharing_password_seed
output: sharing_password
begin
  counter 0 (1 byte)
  filtered_hash []
  while length of filtered_hash < sharing_password_length
    buffer ⟵  counter || long_term_shared_secret || sharing_password_seed
    hash ⟵  SHA-256(buffer)
    character_set_size ⟵ 10 (digits from 0 to 9)
    character_set_filter ⟵ (256–- 256 mod character_set_size)–- 1
    for each byte in hash
      if byte value <= character_set_filter
        append byte to filtered_hash
    if counter <255
       counter ⟵ counter +1
    else
         abort procedure
  for i from 0 to (sharing_password_length–- 1)
      digit ⟵ filtered_hash[i] mod character_set_size
      sharing_password[i] ⟵  toChar(digit)
  return sharing_password (i+1 chars)
end
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

11.13 Versioning

Key sharing data is exchanged via framework APIs and is mostly consumed by the applet. The version information shall combine framework and applet version into OD-FD-KS.

Version agreement does not add another roundtrip to the sharing flow. Instead, the version information is sent by the owner device as part of the key creation request. This implies that the provided key creation data can be consumed by the friend device independently of the supported friend versions. To achieve this, the key creation data must contain all data required by all possibly supported friend device versions.

If, for example, the format of the endpoint identifier changes in a future version, the new endpoint identifier must be identified by a new tag and the owner device must sent old and new identifiers. The friend device can then pick the required format.

Mandatory tags shall not be removed from the key creation request, as the friend device might only support a version where this tag is required to proceed.

The friend device determines the highest commonly supported version from the owner version list and its own version list. It then sends the key signing request in a format supported by the owner device.

The friend device uses the list of vehicle-supported owner pairing versions (V-OD-FW) to determine the format of the key tracking request (Table 11-19). Note that content and format of some fields of the key tracking request are determined by other domain versions.

If no common version can be determined, then both sides shall fall back to the sharing data formats defined before versioning was introduced

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

12 DELEGATED KEY SHARING

Delegated key sharing feature is out of scope of this specification.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

13 KEY TERMINATION AND DELETION [WCC1/WCC2]

Key termination may be triggered in the vehicle, in the Vehicle OEM Server (e.g., account), on the device, and in the Device OEM Server (e.g., remote device wipe, if supported).

A termination attestation should be sent from the device to the KTS whenever the device terminates the Digital Key. Along with the termination attestation, the device should send the Vehicle OEM proprietary data subsection of the private mailbox. The termination attestation and the Vehicle OEM proprietary data may not be sent in all cases due to technical constraints.

If the termination attestation is provided in the manageKey call from the Device OEM server to the Vehicle OEM Server, then the vehicle OEM server should consider the key as “terminated in the device” before attempting to terminate the key in the vehicle.

If the termination attestation is not provided in the manageKey call from the Device OEM server to the Vehicle OEM Server, then the vehicle OEM server shall continue to terminate the key in the vehicle while respecting the vehicle key termination conditions.

The Device OEM shall not terminate an active Digital Key unless one of the following conditions are met:

- The terminate action is initiated by an authenticated user and user intent on the device is confirmed, or
- After the vehicle OEM has signaled to the device OEM that the Digital Key has been deleted in the vehicle

In the case of remote device wipe (REV_170/REV_420), the action shall only be initiated by an authenticated user and Device OEM shall inform user that any keys on the device will not be useable.

To address user safety, a fade-out period may be established by the Vehicle OEM server. The fade out period is defined as the time between the IN_TERMINATION message and the actual termination of the key on the vehicle. The Vehicle OEM server notifies the owner or friend Device OEM server about the start of this period by sending an IN_TERMINATION notification. Further requirements associated with the fade-out period (e.g. use, purpose and duration of the period) are Vehicle OEM specific. A sample implementation of IN_TERMINATION notification is shown in 17.7.7.6. After receiving the IN_TERMINATION message, Device OEM server should notify the user. Further behaviors after reception of the IN_TERMINATION message by the Device OEM server are specific to the Device OEM.

If a fade-out period is required, the Vehicle OEM should notify the device about the start of this period.

If the fade-out period has ended and/or the conditions for a successful termination of the key in the vehicle are met, the key is tracked as terminated in the KTS.

If online, owner and friend devices shall be notified about the successful termination. The device should delete the associated Digital Keys prior to deletion of the instance CA triggered by DELETE CA command.

The detailed termination and deletion flows in the vehicle and device are described in Sections 13.2, 13.3, 13.4, 13.5, and 13.6.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

13.1 Key Termination Scenarios

The termination and suspension use cases defined by CCC are listed in Table 13-1, Table 13-2, Table 13-3, and Table 13-4.

Table 13-1: Vehicle-triggered Key Termination

Use Case ID	Trigger	Owner Key Status	Friend Key Status
R-VT-1	Owner terminates owner key in vehicle	Terminated	-
R-VT-2	Owner terminates friend key in vehicle	-	Terminated
R-VT-3	Owner changes owner device via vehicle	Terminated	-
R-VT-4	Owner/user unpairs vehicle from owner account or resets Digital Key function	Terminated	Terminated

Table 13-2: Device-triggered Key Termination

Use Case ID	Trigger	Owner Key Status	Friend Key Status
R-DT-1	Owner terminates owner key in Vehicle OEM app	Terminated	-
R-DT-2	Owner terminates friend key in Vehicle OEM app	-	Terminated
R-DT-3	Owner terminates owner key in native app	Terminated	-
R-DT-4	Owner terminates friend key in native app	-	Terminated
R-DT-5	Friend terminates friend key in Vehicle OEM app	-	Terminated
R-DT-6	Friend terminates friend key in native app	-	Terminated
R-DT-7	Owner unpairs vehicle from owner account in Vehicle OEM app	Terminated	Terminated

Table 13-3: Vehicle OEM-triggered Key Termination

Use Case ID	Trigger	Owner Key Status	Friend Key Status
R-VOT-1	Owner terminates owner key in Vehicle OEM customer web portal, hotline, or support	Terminated	-
R-VOT-2	Owner terminates friend key in Vehicle OEM customer web portal, hotline, or support	n/a	Terminated
R-VOT-3	Friend terminates friend key in Vehicle OEM customer web portal, hotline, or technical support	n/a	Terminated
R-VOT-4	Owner Digital Key service license expires	Terminated	Terminated
R-VOT-5	Terminate friend key after expiry date (automatic)	n/a	Terminated
R-VOT-6	Owner/legal authorities report a stolen/destroyed vehicle	Terminated	Terminated
R-VOT-7	Garage service process	Terminated	Terminated
R-VOT-8	Security breach on vehicle is detected.	Terminated	Terminated
R-VOT-9	Owner triggers deletion of personal data (owner account) in Vehicle OEM Server	Terminated	Terminated

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Use Case ID	Trigger	Owner Key Status	Friend Key Status
R-VOT-10	Friend triggers deletion of personal data (friend account) in Vehicle OEM Server	n/a	Terminated
R-VOT-11	Owner unpairs vehicle from owner account in Vehicle OEM Server	Terminated	Terminated

Table 13-4: Device OEM-triggered Key Termination

Use Case ID	Trigger	Owner Key Status	Friend Key Status
R-DOT-1	Owner/User wipes device (factory reset)	Terminated	-
R-DOT-2	Friend wipes friend device (factory reset)	n/a	Terminated
R-DOT-3a	Owner reports lost/stolen/rediscovered owner device: suspend/resume keys	Suspended/ Resumed	n/a
R-DOT-3b	Owner reports lost/stolen owner device: remote wipe keys	Terminated	n/a
R-DOT-4a	Friend reports lost/stolen/rediscovered friend device: suspend/resume keys	n/a	Suspended/ Resumed
R-DOT-4b	Friend reports lost/stolen friend device: remote wipe keys	n/a	Terminated
R-DOT-5a	Security breach on owner device is detected	Terminated	Terminated
R-DOT-5b	Security breach on friend device is detected	n/a	Terminated

The use cases in Table 13-1 through Table 13-4 are grouped into more detailed flow diagrams. Table 13-6 provides the mapping. In all the flow diagrams described in Sections 13.2 through 13.6, the exchanges between vehicle and Vehicle OEM Server, device and Device OEM Server, and Vehicle OEM Server and KTS are out of scope of this specification. These exchanges are shown for illustration purposes only. The exchanges between Vehicle OEM Server and Device OEM Server are described by the relevant server APIs (see Section 17). The following APIs are used in key termination procedures:

- **manageKey()** and **manageKeyResponse**: See Section 17.7.2
- **eventNotification()** and **eventNotificationResponse**: See Section 17.7.3

Note: Only relevant parameter(s) in the API that are specific to the particular step in the flow diagram are shown.

All keys are identified by their Digital Key identifier (key id) in the Vehicle OEM Server and Device OEM Server. Digital Key identifiers are not shown explicitly in the flow diagrams unless required for comprehension. Along with the terminationAttestation parameter, the vehicleOEMProprietaryData is also transferred if not explicitly shown in the flow diagrams. Interpretations of the keyID parameter inside of the manageKey() call for action=TERMINATE (as defined in section 17.7.2.2), are described in Table 13-5 below

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 13-5 keyID usage for manageKey() with action=TERMINATE

Request by	ManageKey() Parameters	KeyID Usage
Owner / Friend Device	No RTR included, TA included	Key deleted on device, key to be deleted in vehicle
Device OEM Server	No RTR included, no TA included	key requested to be deleted in vehicle
Owner Device	Device RTR included	keyID = Owner keyID, multiple keys can be deleted in RTR
Vehicle OEM Server	Server RTR included	keyID = one of the keyIDs contained in the RTR

For full parameters of each API call, please refer to the corresponding sections.

Note: Friend key ID is used in all the figures in this section and that the term is synonymous with Digital Key identifier of a friend.

Data elements requiring privacy encryption (See Section 14) are identified by "*" in the flow diagrams. The exact data elements to be encrypted are defined in Section 17.

Messages requiring authentication (See Section 14) are identified by "+" in the flow diagrams. The exact messages and API calls to be signed are defined in Section 17.

Table 13-6:Detailed Key Termination Use Cases

Flow ID	Description	Use Case ID
Friend Key Remote Termination		
REV_100	Friend key termination in vehicle	R-VT-2
REV_110	Friend key termination in owner Vehicle OEM account	R-VOT-2
REV_120	Friend key termination in friend Vehicle OEM account	R-VOT-3
REV_130	Friend key termination on owner device natively	R-DT-4
REV_140	Friend key termination on owner device in Vehicle OEM app	R-DT-2
REV_150	Friend key termination based on expiry date of the key	R-VOT-5
REV_160	Friend key termination by Device OEM (device security issue)	R-DOT-5b
REV_170	Friend key termination due to remote wipe of device	R-DOT-4b
REV_180	Friend key termination due to deletion of personal data in friend Vehicle OEM account	R-VOT-10
Friend Key Local Deletion		
REV_200	Friend key termination on friend device natively	R-DT-6
REV_210	Friend key termination on friend device in Vehicle OEM app	R-DT-5
REV_220	Friend key termination due to local wipe of device	R-DOT-2
Friend Key Remote Suspension		
REV_300	Friend key temporary suspension in device lost mode in Device OEM account	R-DOT-4a
REV_310	Friend key resume from device lost mode in Device OEM account	R-DOT-4a
Owner Key Remote Termination		

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Flow ID	Description	Use Case ID
REV_400	Owner key deletion in vehicle UI (change device)	R-VT-1, R-VT-3
REV_410	Owner key termination by Device OEM (security issue)	R-DOT-5a
REV_420	Owner key termination due to remote wipe of device	R-DOT-3b
Owner Key Local Deletion		
REV_500	Owner key termination on owner device natively (deletion of pass for Digital Key)	R-DT-3
REV_510	Owner key termination on owner device in Vehicle OEM app	R-DT-1
REV_520	Owner key termination due to local wipe of device	R-DOT-1
Owner Key Remote Suspension		
REV_600	Owner key temporary suspension in device lost mode in Device OEM account	R-DOT-3a
REV_610	Owner key resume from device lost mode in Device OEM account	R-DOT-3a
Vehicle Unpairing		
REV_700	Unpairing in vehicle UI (sale of vehicle)	R-VT-4
REV_710	Unpairing on owner device in Vehicle OEM app	R-DT-7
REV_720	Unpairing in owner Vehicle OEM account (sale of vehicle)	R-VOT-11
REV_730	Unpairing based on cancellation or expiry date of the Digital Key service	R-VOT-4
REV_740	Unpairing by Vehicle OEM (vehicle stolen)	R-VOT-6
REV_750	Unpairing by Vehicle OEM (security breach in vehicle)	R-VOT-8
REV_760	Unpairing by Vehicle OEM (garage service process)	R-VOT-7
REV_770	Unpairing due to deletion of personal data in owner Vehicle OEM account	R-VOT-9
Vehicle Unbinding (leads to unpairing)		
REV_800	Unbinding of vehicle from owner account in vehicle UI (unpairing, owner sells vehicle)	R-VT-4
REV_810	Unbinding of vehicle from owner account in owner Vehicle OEM account (unpairing, owner sells vehicle)	R-VOT-11
REV_820	Unbinding of vehicle from owner account in Vehicle OEM app (unpairing, owner sells vehicle)	R-DT-7

13.2 Friend Key Remote Termination

All scenarios in which the friend key termination process is started from a device other than the friend's own device are considered as remote termination cases. The common behavior is that the termination request is issued to the Vehicle OEM Server, which then starts the termination process on the vehicle side.

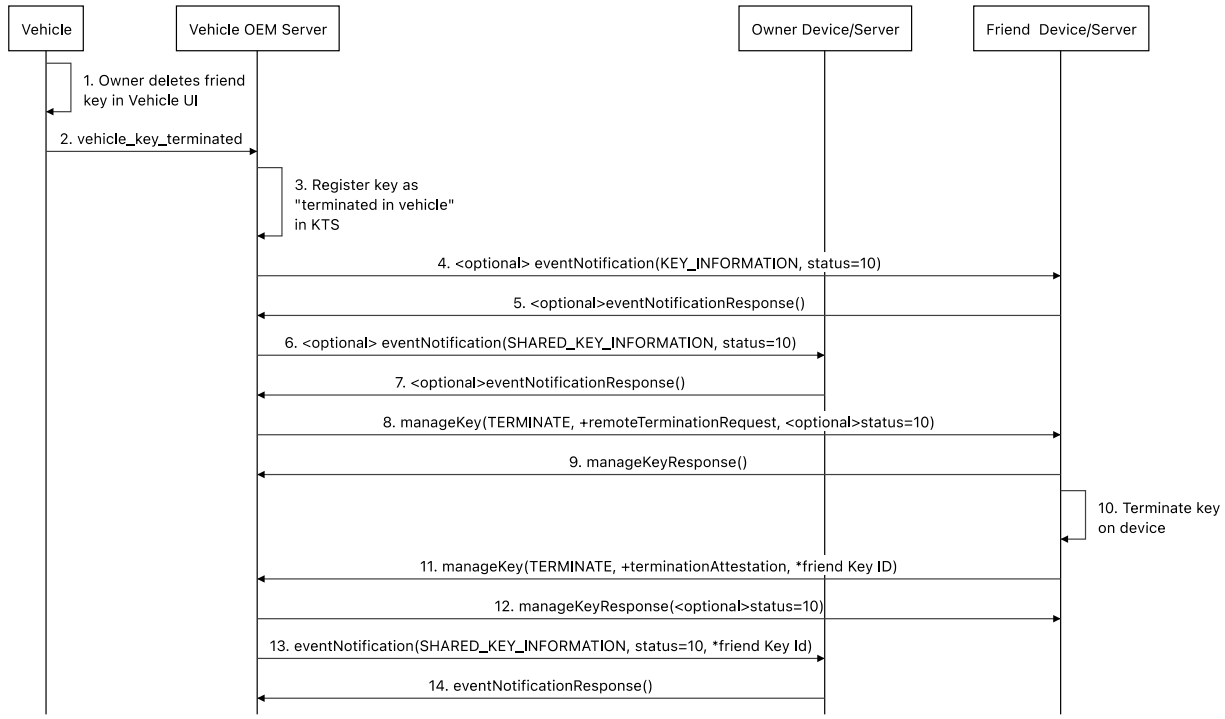
Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

For remote termination, the key is considered as terminated when it is deleted in the vehicle.

13.2.1 *REV_100: Friend key termination in vehicle*

If the owner deletes a friend key in the vehicle, no fade-out is required. The vehicle notifies the devices about the successful deletion via the Vehicle OEM Server (REV_100).

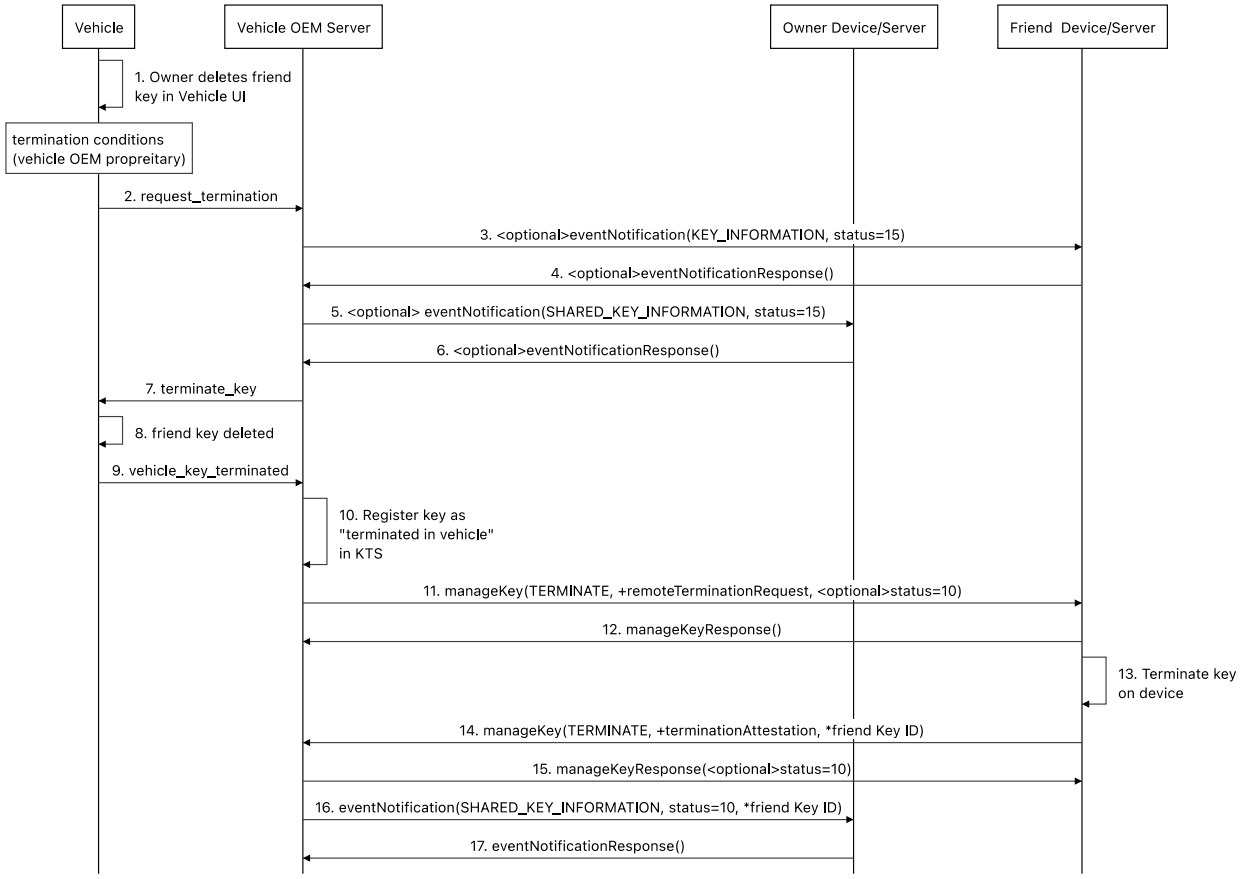
Figure 13-1: REV_100: Friend Key Termination in Vehicle



Alternatively, it may be required for the vehicle to request online deletion. In this case, the flow is similar to remote termination from the Vehicle OEM Server. Devices are informed about the deletion request and the successful deletion (REV_100a)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-2: REV_100a: Friend Key Termination in Vehicle (Vehicle Required to be Online)



The eventNotification (IN_TERMINATION) should be sent in all cases so that the device side does not see any difference between both options.

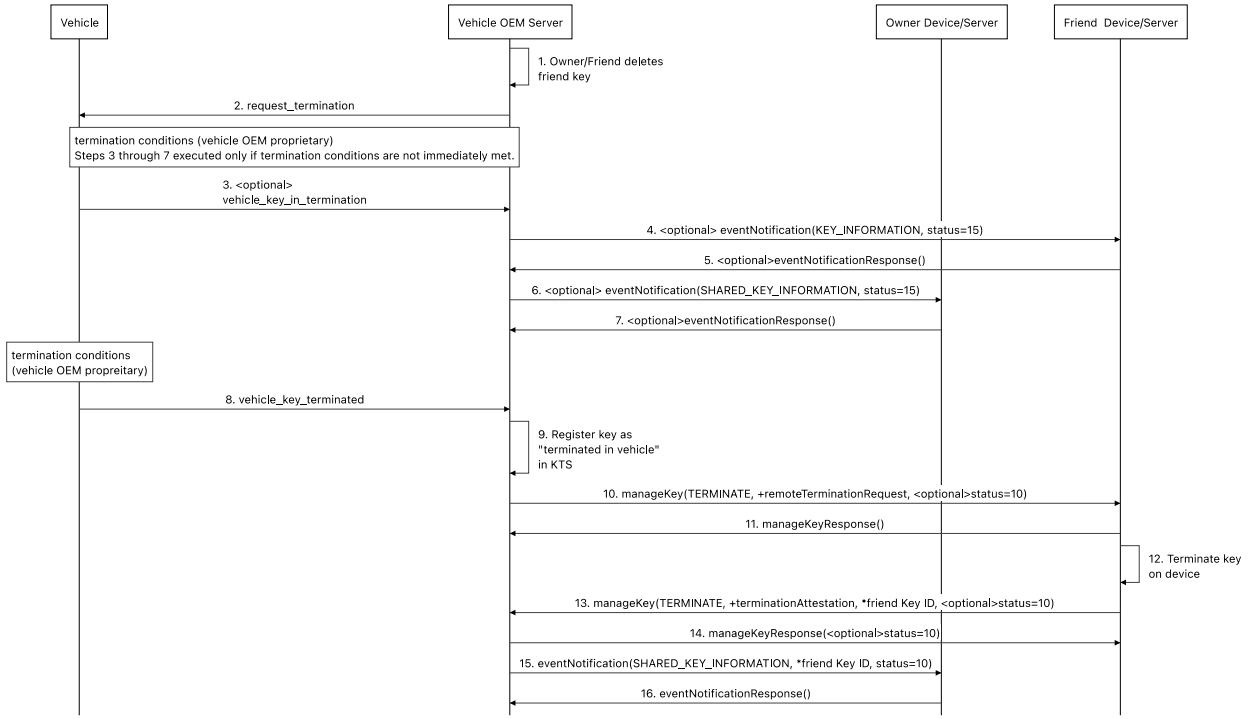
13.2.2 *REV_110: Friend key termination in owner Vehicle OEM account*

13.2.3 *REV_120: Friend key termination in friend Vehicle OEM account*

The same flow shall be executed when either the owner deletes the friend key from the owner's Vehicle OEM account or the friends delete their own key from their Vehicle OEM account. This is shown in Figure 13-3.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-3: REV_110/120: Friend Key Termination in Owner/Friend Vehicle OEM Account

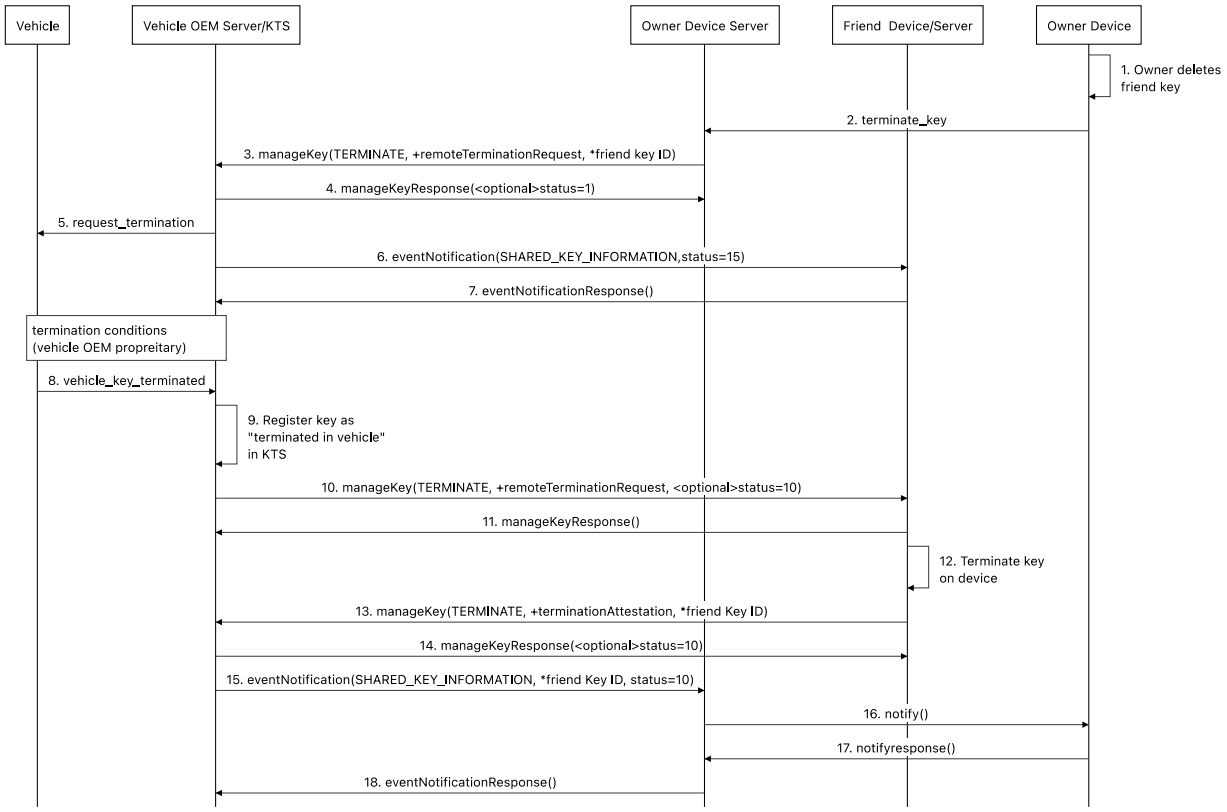


Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

13.2.4 *REV_130: Friend key termination on owner device natively*

13.2.5 *REV_140: Friend key termination on owner device in Vehicle OEM app*

Figure 13-4: REV_130/140: Friend Key Termination on Owner Device Natively/in Vehicle OEM App



In some cases, the owner may terminate a friend key using one of the following methods:

1. Natively through the device operating system (REV_130, see figure 13-4)
2. Using vehicle OEM apps installed on the device which leverage one or more APIs offered by the device operating system (REV_140, see figure 13-4)
3. Using vehicle OEM apps installed on the device which contact the Vehicle OEM server directly. (REV_110)

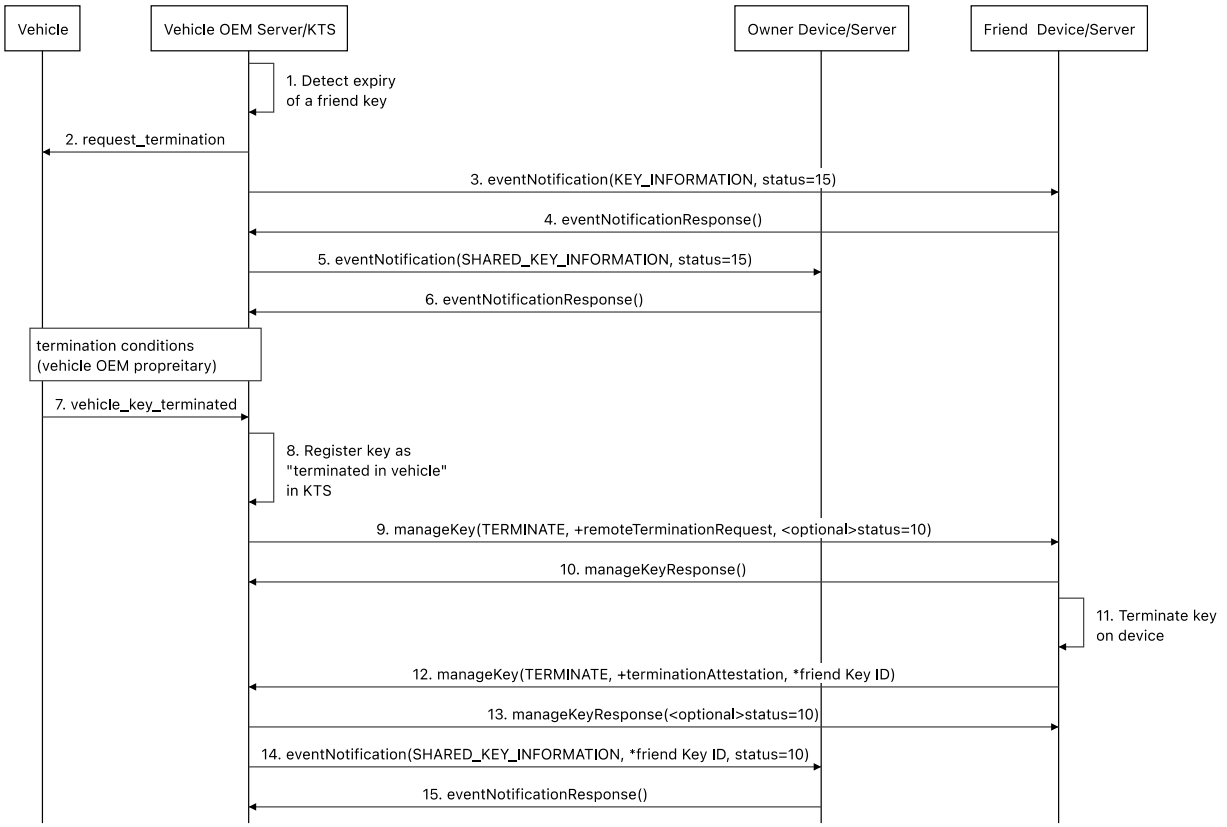
13.2.6 *REV_150: Friend key termination based on expiry date of the key*

If the vehicle or the Vehicle OEM Server detects that a friend key has expired, the key is deleted from the vehicle in the same way as if it were deleted in the vehicle or the Vehicle OEM Server by the owner.

The flow is similar to REV_120: Friend key termination in friend Vehicle OEM account (see Section 13.2.3).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

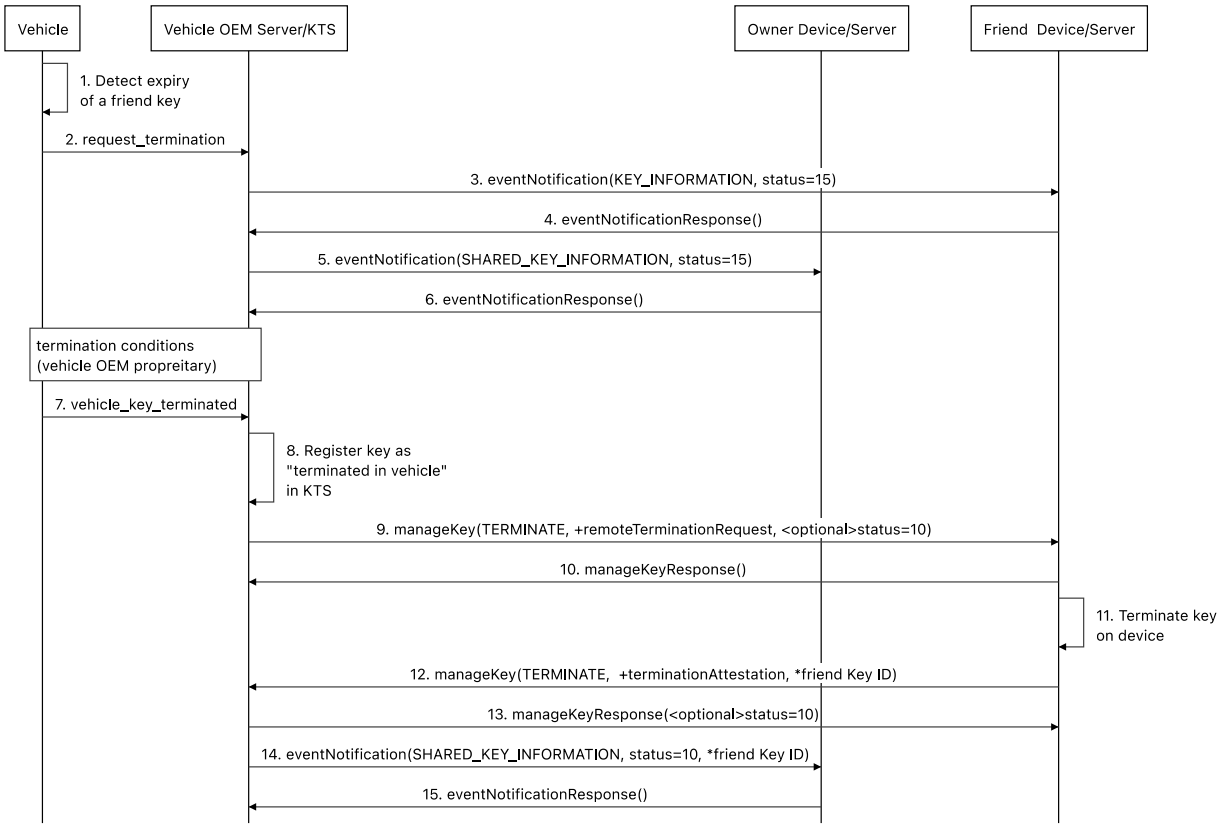
Figure 13-5: REV_150: Friend Key Termination Based on Expiry Date of the Key



The flow depicting Vehicle detection of an expired friend key is shown in Figure 13-6 and is similar to REV_100: Friend Key Termination in Vehicle (Figure 13-1)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-6 Vehicle Detects Expiry of Friend Key



13.2.7 *REV_160: Friend key termination by Device OEM (device security issue)*

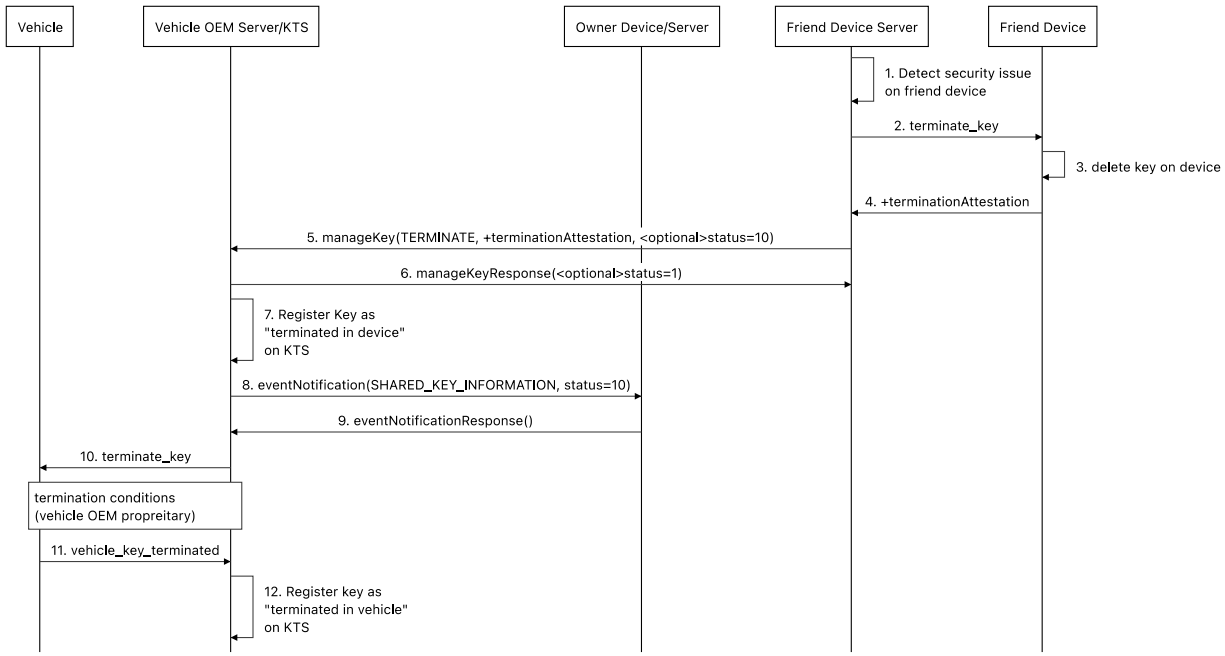
When a security issue on a device or group of devices is detected, the Device OEM removes the Digital Keys based on the nature of the security issue. The details are out of scope of this specification.

In those cases, the keys may be removed immediately from the device, depending on the security issues.

The termination attestation and the Vehicle OEM proprietary data from the device are provided to the KTS.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

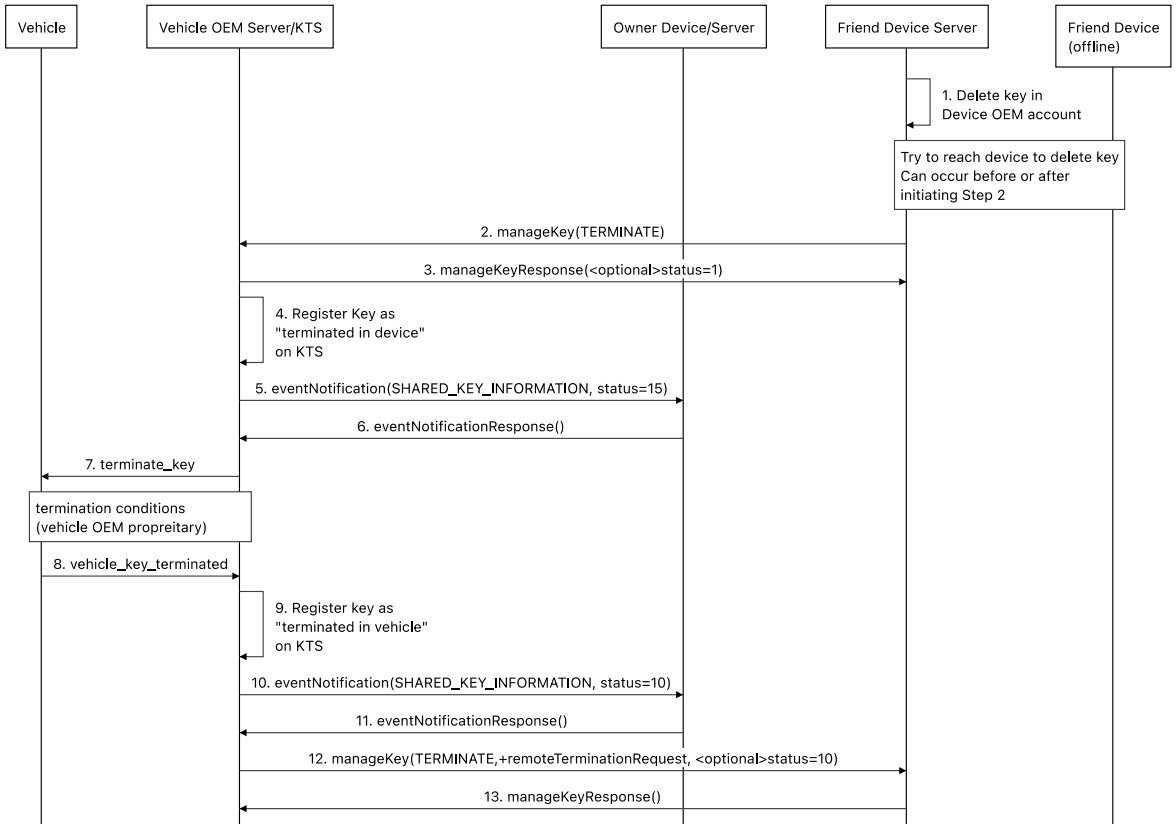
Figure 13-7: REV_160: Friend Key Termination by Device OEM (Device Security Issue)



When the device cannot be reached, the Vehicle OEM Server shall still be notified immediately, as shown in Figure 13-8.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-8: REV_160a: Friend Key termination by Device OEM and Friend Device is Offline



The termination attestation and the Vehicle OEM proprietary data from the device may be provided to the KTS if the device is online.

13.2.8 *REV_170: Friend key termination due to remote wipe of device*

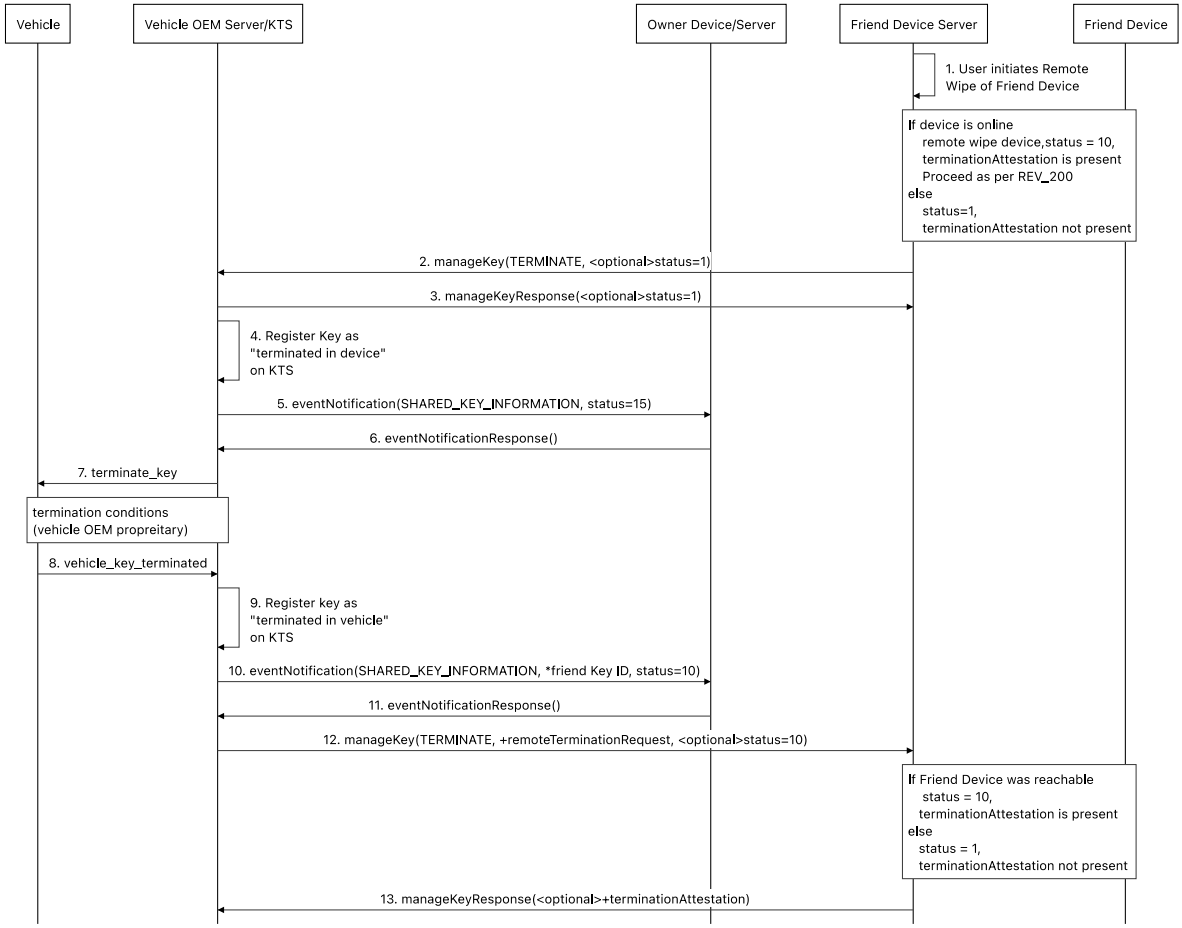
When a friend decides to wipe his/her device remotely, the Device OEM deletes the Digital Keys on the involved devices. The details are out of scope of this specification.

In those cases, the keys are deleted immediately from the friend's device. Vehicle and Vehicle OEM termination conditions and fade-out do not apply.

Note: In the case of a remote wipe of a device, the device may include terminationAttestation and vehicleOEMProprietaryData in manageKey() in Step 3, as shown in Figure 13-9.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-9: REV_170: Friend Key Termination Due to Remote Wipe of Device



In some cases, the friend may terminate the friend key using one of the following methods:

1. Natively through the device operating system (REV_200, see Figure 13-10)
2. Using vehicle OEM apps installed on the device which leverage one or more APIs offered by the device operating system (REV_210, see Figure 13-10)
3. Using vehicle OEM apps installed on the device which contact the vehicle OEM server directly (REV_120)

13.2.9 *REV_180: Friend key termination due to deletion of personal data in friend Vehicle OEM account*

When the friend has a Vehicle OEM account, the shared key is linked with the account. A request to delete the personal data from the account should lead to removal of the key from the vehicle, similar to a key deletion as described in “REV_120: Friend key termination in friend Vehicle OEM account” (see Section 13.2.3).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

13.3 Friend Key Local Termination

All scenarios in which the friend key termination process is started on the friend's own device are considered as local termination cases.

The common behavior is that the termination request should be executed immediately on the friend device. Depending on the connectivity situation, the termination attestation and the Vehicle OEM proprietary data may be sent immediately to the KTS, delayed, or not sent at all.

In cases where the Vehicle OEM Server is not aware of the termination, termination attempt, or loss of the device, the owner or friend (if applicable) should remove the key in the vehicle.

Fade-out or termination preconditions do not apply when termination attestation and the Vehicle OEM proprietary data will be transmitted to the Vehicle OEM Server.

The owner device should be notified about successful registration of the termination. The friend and owner device UI should mark the corresponding key representation as "terminated."

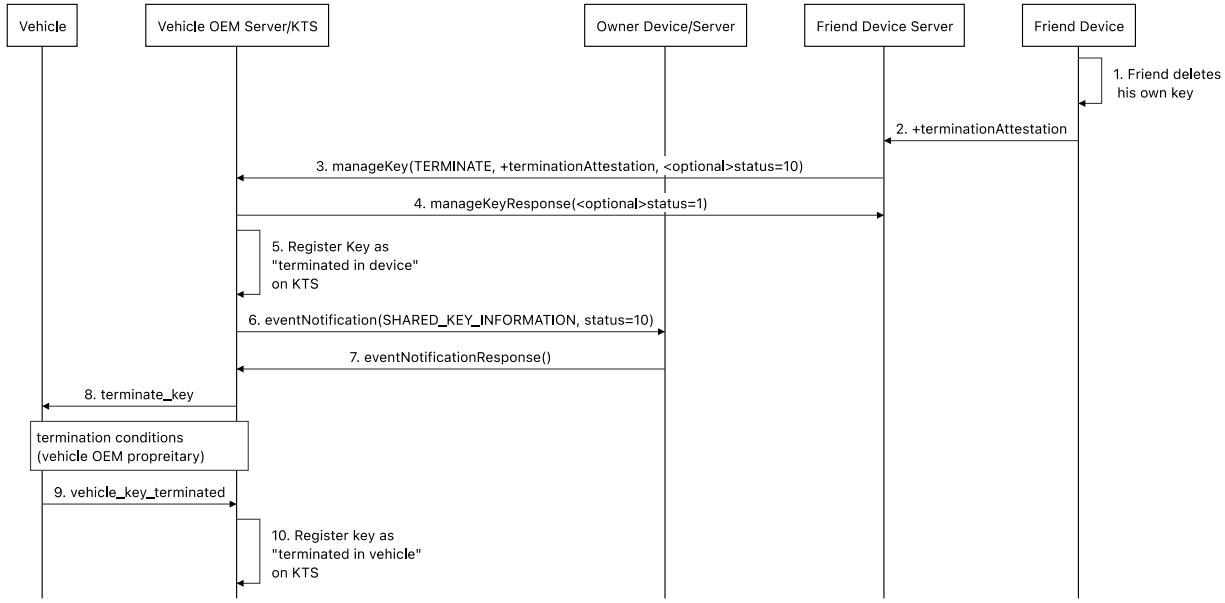
13.3.1 *REV_200: Friend key termination on friend device natively*

13.3.2 *REV_210: Friend key termination on friend device in Vehicle OEM app*

If the key is deleted on the friend device, the termination attestation and the Vehicle OEM proprietary data shall be sent to the friend server as soon as the device is online.

The server flow is exactly the same for native and Vehicle OEM app-initiated deletion.

Figure 13-10: REV_200/210: Friend Key Termination on Friend Device Natively/in Vehicle OEM App



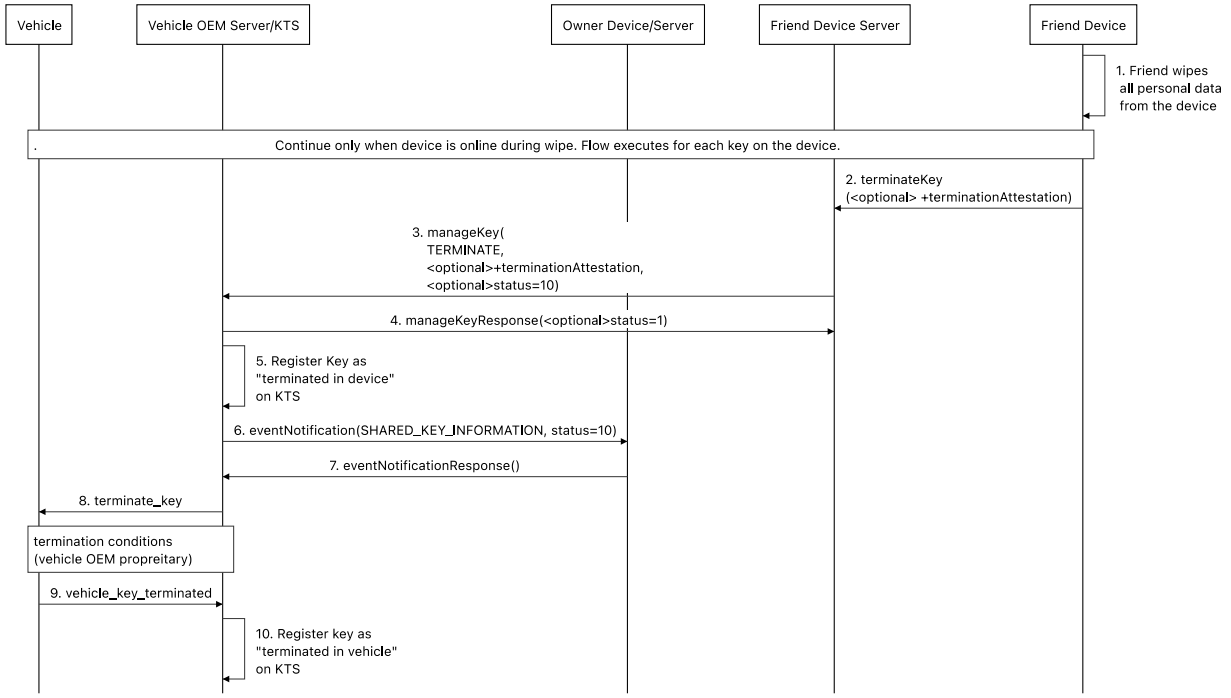
The Vehicle OEM indicates to the vehicle that the key can be terminated immediately when the device key is registered in the KTS as "terminated on device."

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

13.3.3 *REV_220: Friend key termination due to local wipe of device*

If the device is wiped locally, the termination attestation and the Vehicle OEM proprietary data is sent to the friend server on a best-effort basis.

Figure 13-11: REV_220: Friend Key Termination Due to Local Wipe of Device



The Vehicle OEM indicates to the vehicle that the key can be terminated immediately when the device key is registered in the KTS as “terminated on device.”

For each friend key present on the device, the flow shown in Figure 13-11 shall be executed.

The owner key termination due to device wipe is described in Section 13.5.6.

13.4 Friend Key Remote Suspend/Resume

Remote suspend and resume occur when the device is set to lost or stolen mode in the Device OEM Server and when the lost/stolen mode is ended.

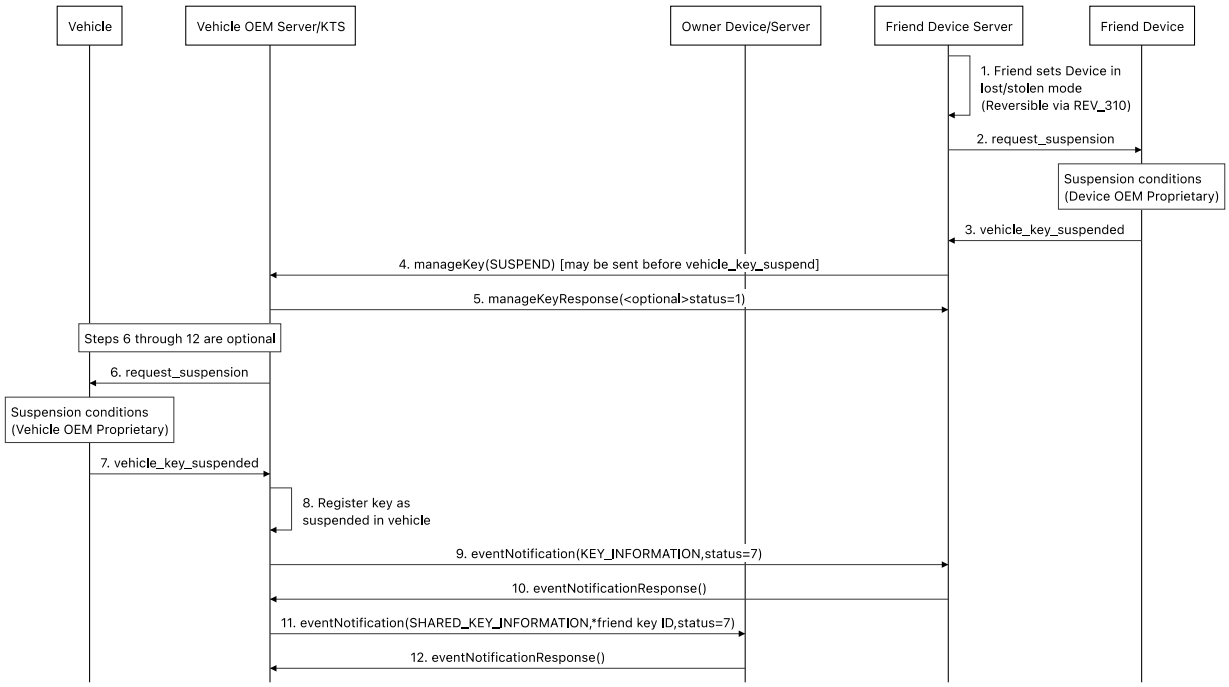
If the device is reachable, the keys are suspended immediately on the device. The suspended status is reversible; keys may be resumed through the Device OEM Server or locally on the device. No cryptographic attestation is available for a key suspended and resumed on the device.

13.4.1 *REV_300: Friend key temporary suspension in Device OEM account*

Suspension of a Digital Key triggered by the Device OEM account is used when the device mode is set to lost or stolen. The Device OEM Server attempts to suspend the key on the device and, in parallel, the Vehicle OEM Server is notified to suspend the keys in the vehicle, as shown in Figure 13-12.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-12: REV_300: Friend Key Suspension by Device OEM Account



The suspension on the device is not tracked in the KTS. The suspension of the key in the vehicle may be tracked by the KTS.

manageKey SUSPEND command communicates the intent of the user to suspend the key to the KTS, but not the suspension status in the device.

The Vehicle OEM notifies the owner and friend when the key is successfully suspended in the vehicle.

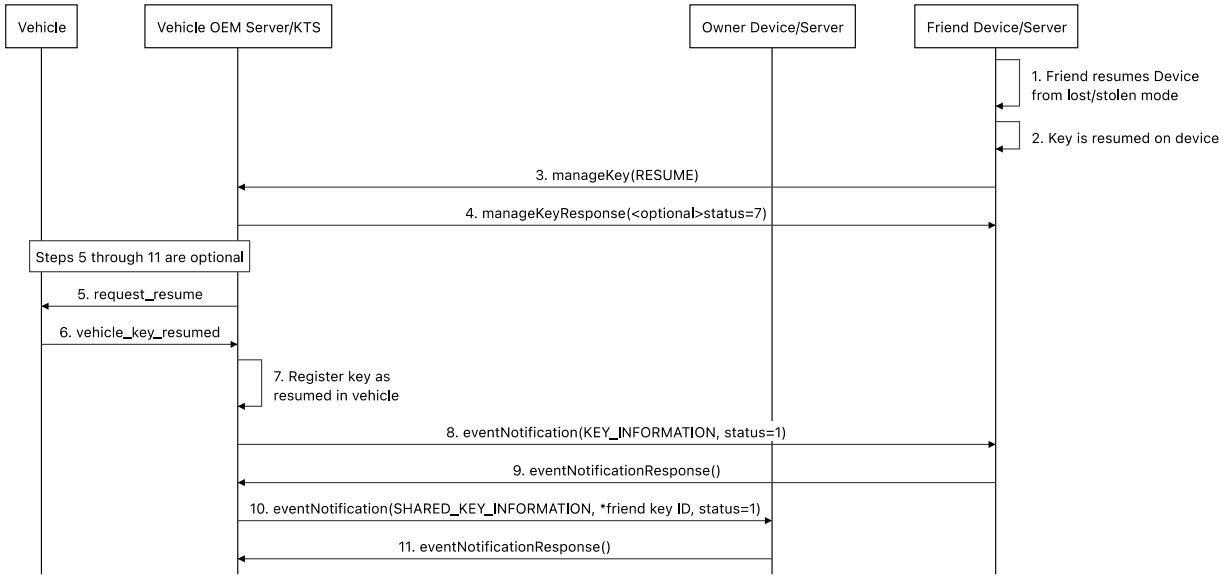
Regardless of vehicle OEM server support of remote suspend/resume or not, the Vehicle OEM server shall response with manageKeyReponse (statusCode 200) if it receives the manageKeyRequest successfully.

13.4.2 *REV_310: Friend key resume in Device OEM account*

In the case where the Vehicle OEM Server supports remote suspend/resume, the suspended key may be resumed in the Device OEM account or on the device itself if it has been recovered, as shown in Figure 13-13. The friend Device OEM Server shall only send manageKey(RESUME) or transmit a resume attestation in the friend private mailbox if it has previously received eventNotification(SUSPENDED).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

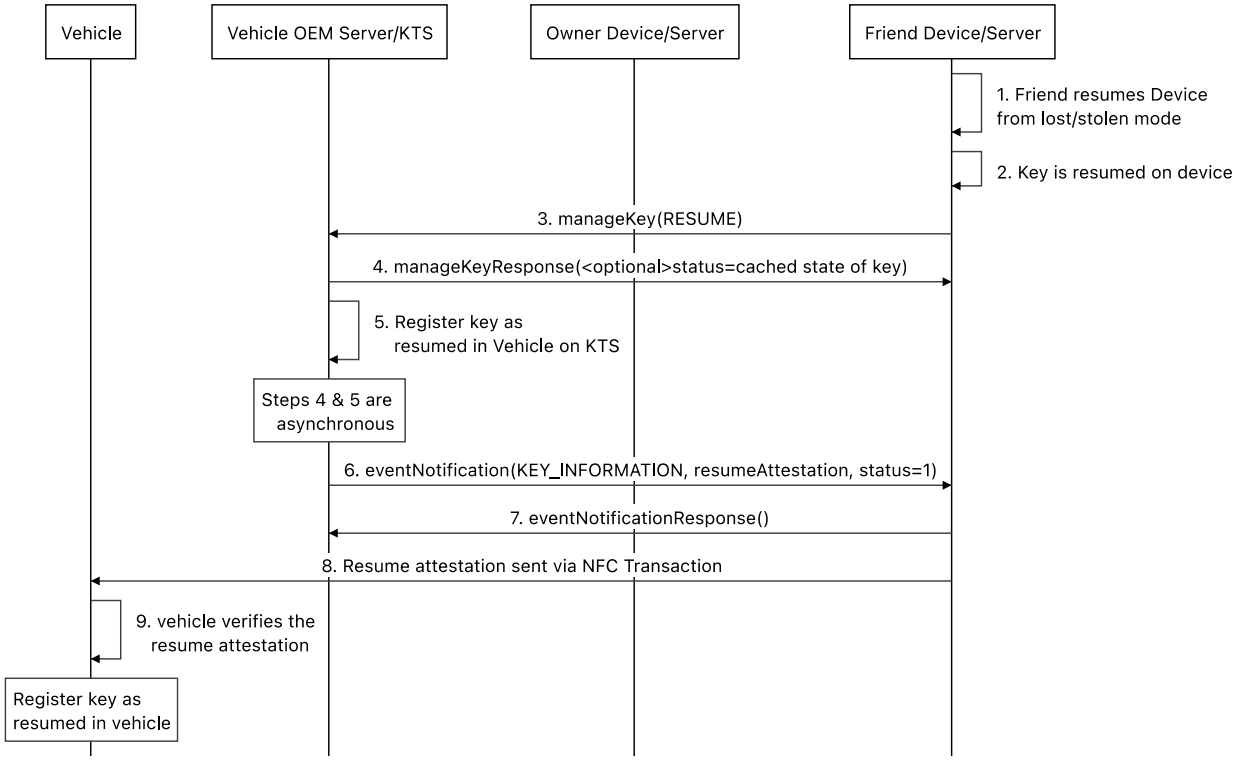
Figure 13-13: REV_310: Friend Key Resume in Device OEM Account or on Device



The device shall be online to resume the key and notify the Vehicle OEM.

If the key is previously tracked as “suspended in vehicle,” the key shall be tracked as “resumed in vehicle” before the vehicle resumes the key to avoid a state in which the key is resumed but still tracked as suspended.

Figure 13-14: REV_310a: Friend Key Resume Using ResumeAttestation



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

When the vehicle is moved into an offline area between suspend and resume, the friend shall be able to provide a resume attestation via NFC transaction to the vehicle, as shown in Figure 13-14.

13.4.3 *Resume attestation*

When a Digital Key has been suspended in a vehicle, it shall be possible to bring a Resume Attestation using the private mailbox of an owner's or friend phone in a similar way as it is done for the first friend transaction.

The Resume Attestation may be used when the vehicle is offline and the owner Digital Key (see Section 13.5.7) or the friend Digital Key (see Section 13.4.1) is in a suspended state.

The device shall add the tag $7F61_h$ and length fields to the resume-Attestation (see Table 13-7) and store the TLV structure in the KeyAtt field of the private mailbox (see Table 4-2).The Resume Attestation shall be written by the Digital Key framework in the private mailbox at offset ATTESTATION_PACKAGE_OFFSET and the ATTESTATION_PACKAGE_BIT signaling bit shall be set.

During a transaction, the vehicle detects the ATTESTATION_PACKAGE_BIT and consumes the attestation. The vehicle then clears the ATTESTATION_PACKAGE_BIT and clears the mailbox area at offset ATTESTATION_PACKAGE_OFFSET.

On reception of a valid Resume Attestation, the vehicle resumes the designated key or does nothing if the Digital Key is already resumed.

Implementation Notes:

If the suspend/resume feature is supported by the Vehicle OEM, the Vehicle OEM must ensure that the Resume Attestation cannot be replayed using a proprietary method.

Table 13-7: Resume Attestation

Tag	Length	Description	Field is	Domain Version
$7F61_h$	variable	resumeAttestation (see Section 17.8.9)		V-OD-FW
vehicle OEM proprietary			mandatory	N/A

13.5 Owner Key Remote Termination

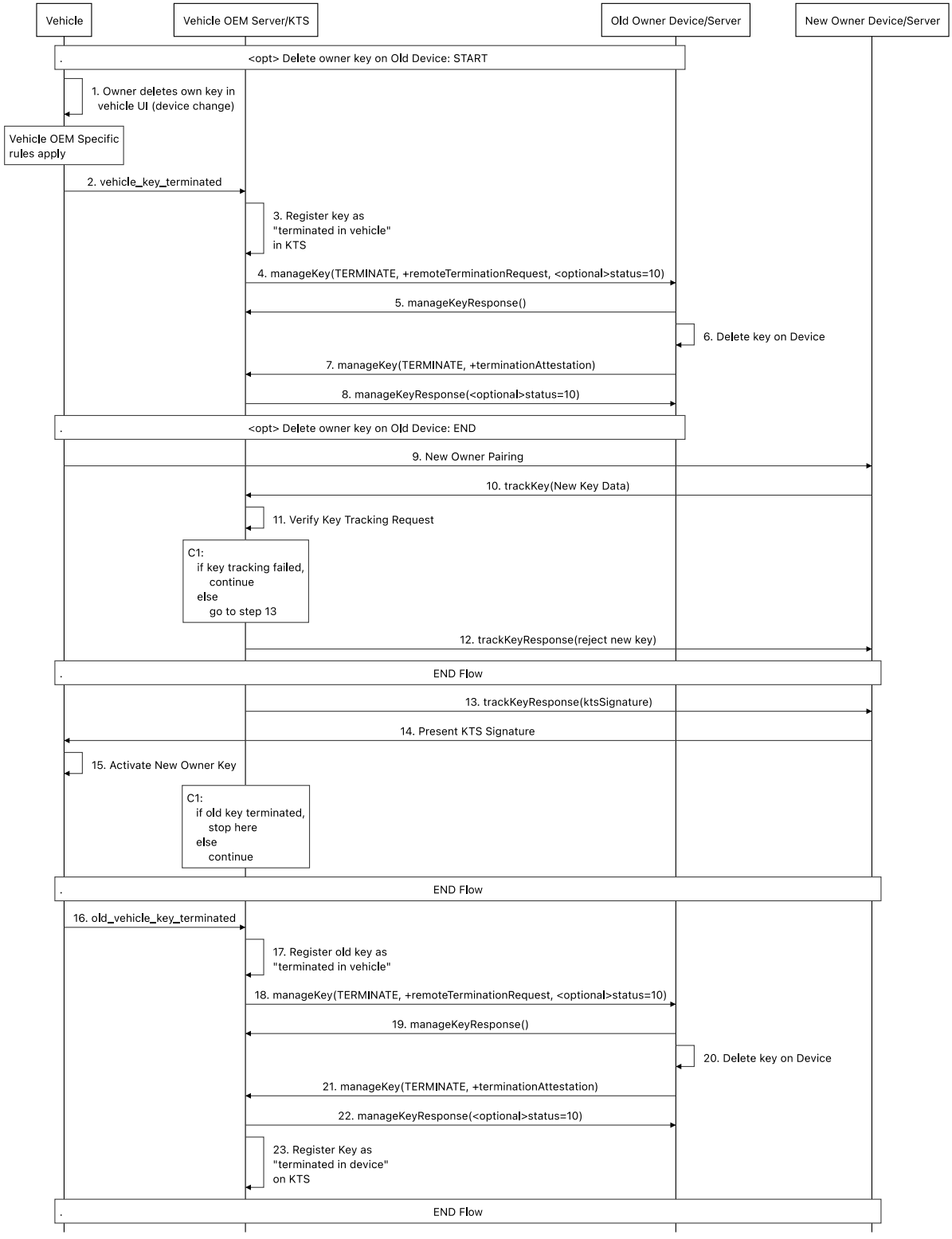
Owner key remote termination shall be handled very similar to friend key remote termination as described in Section 13.2. The friend device is not informed about owner key termination. Owner key termination does not affect the functionality of friend keys for the same vehicle.

13.5.1 *REV_400: Owner key deletion in vehicle UI (change device)*

If an owner device change is triggered in the vehicle, the vehicle deletes the owner key when the necessary pre-conditions are fulfilled. Friend keys shall not be affected. Before the owner device change, the old owner Digital Key may have been terminated.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

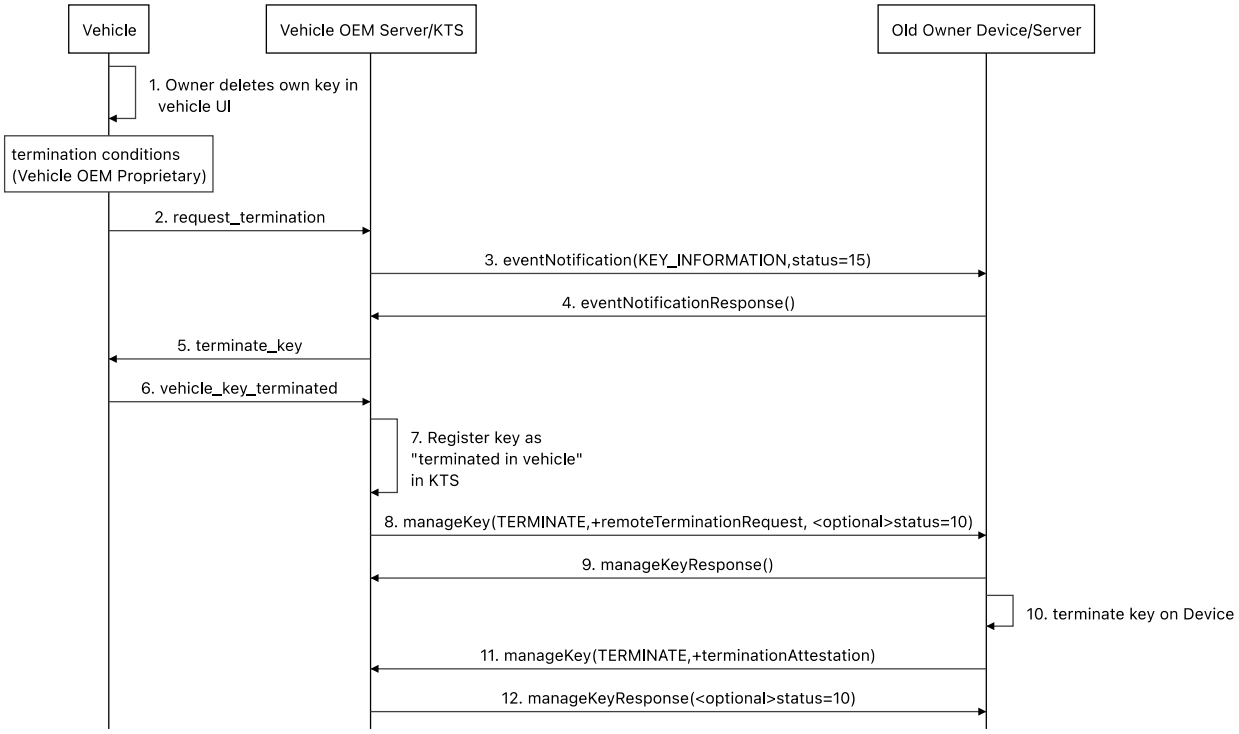
Figure 13-15: REV_400: Owner Key Deletion in Vehicle UI (Change of Device)



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Alternatively, it may be required for the vehicle to request online deletion. In that case, devices are informed about the termination request and the successful termination (See Figure 13-16).

Figure 13-16: REV_400a: Owner Key Deletion in Vehicle UI (Change of Device)



Note: REV_400 and REV_400a are similar to flow REV_100 and REV_100a, respectively, but without involvement of the friend device.

13.5.1.1 Transmission of the shared key information to the new owner device

After the owner device change, information related to the shared keys associated with the owner Digital Key is sent to the new owner device in Step 22 of Figure 13-15 using trackKeyResponse (see Section 17.7.1).

13.5.2 *REV_410: Owner key termination by Device OEM (security issue)*

This flow corresponds to REV_160 with the owner device instead of the friend device. In addition, all friend keys are terminated, and the friend devices are notified.

13.5.3 *REV_420: Owner key termination due to remote wipe of device*

This flow corresponds to REV_170 with the owner device instead of the friend device.

13.5.4 *REV_500: Owner key termination on owner device natively (delete pass for Digital Key)*

This flow corresponds to REV_200 with the owner device instead of the friend device.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

13.5.5 *REV_510: Owner key termination on owner device in Vehicle OEM app*

This flow corresponds to REV_210 with the owner device instead of the friend device.

13.5.6 *REV_520: Owner key termination due to local wipe of device*

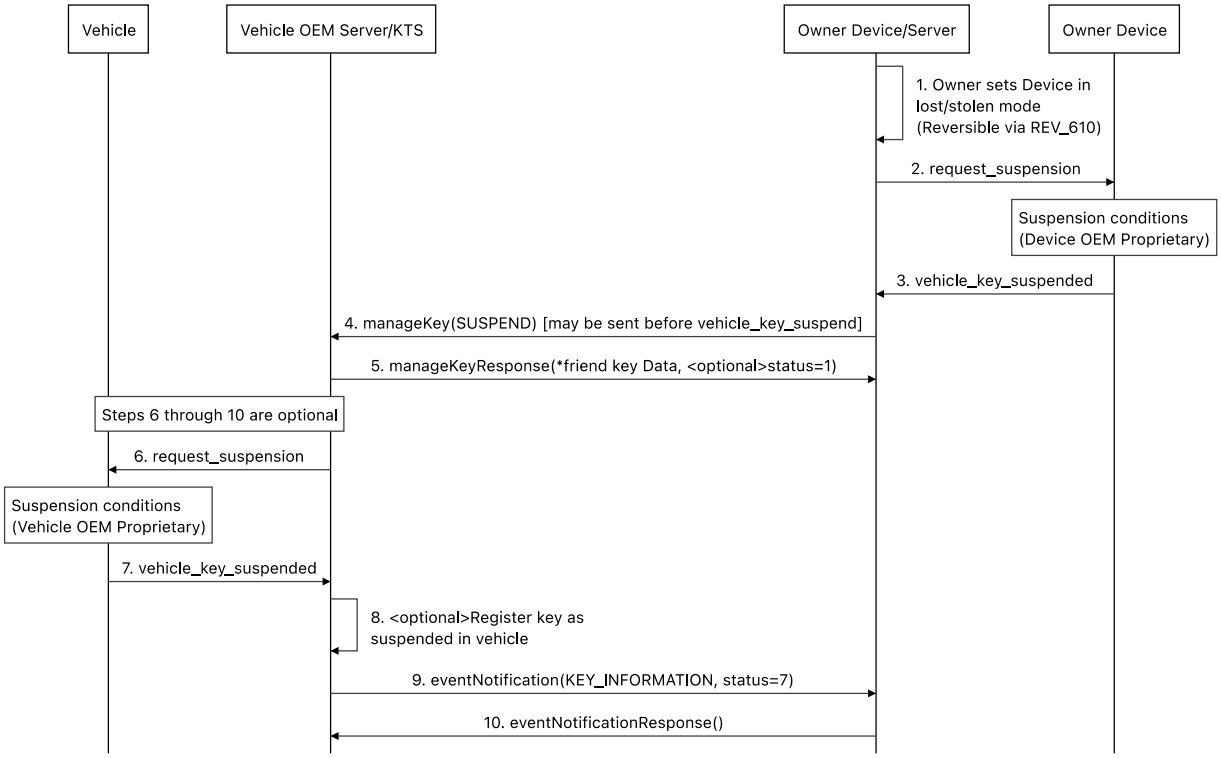
This flow corresponds to REV_220 with the owner device instead of the friend device.

13.5.7 *REV_600/610: Owner key temporary suspension in device lost mode in Device OEM account*

This flow corresponds to REV_300 and REV_310 with the owner device instead of the friend device.

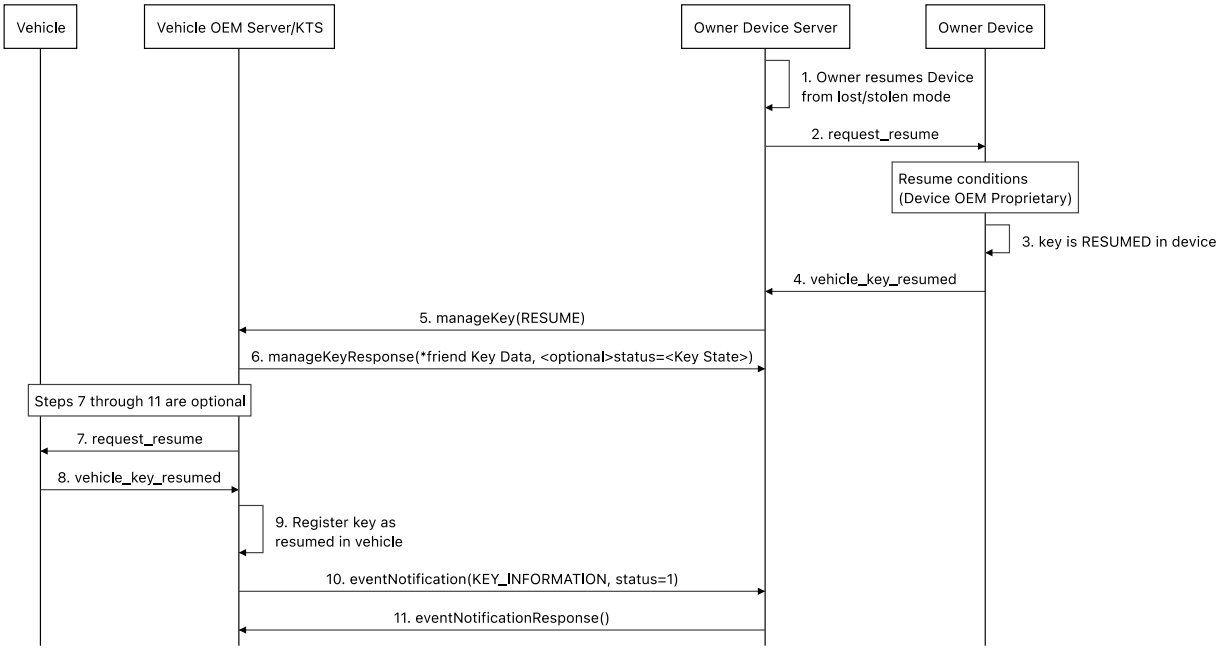
On owner key SUSPEND/RESUME, manageKeyResponse from Vehicle OEM Server shall contain friend Key Data for all active friend keys associated with the Vehicle.

Figure 13-17: REV_600: Owner key suspension due to device reported lost/stolen



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-18: REV_610: Owner key resumption after device reported lost/stolen



13.6 Owner Device Unpairing

If the owner device is unpaired from the vehicle, all friend keys shall be deleted in the vehicle and the friend devices should be notified.

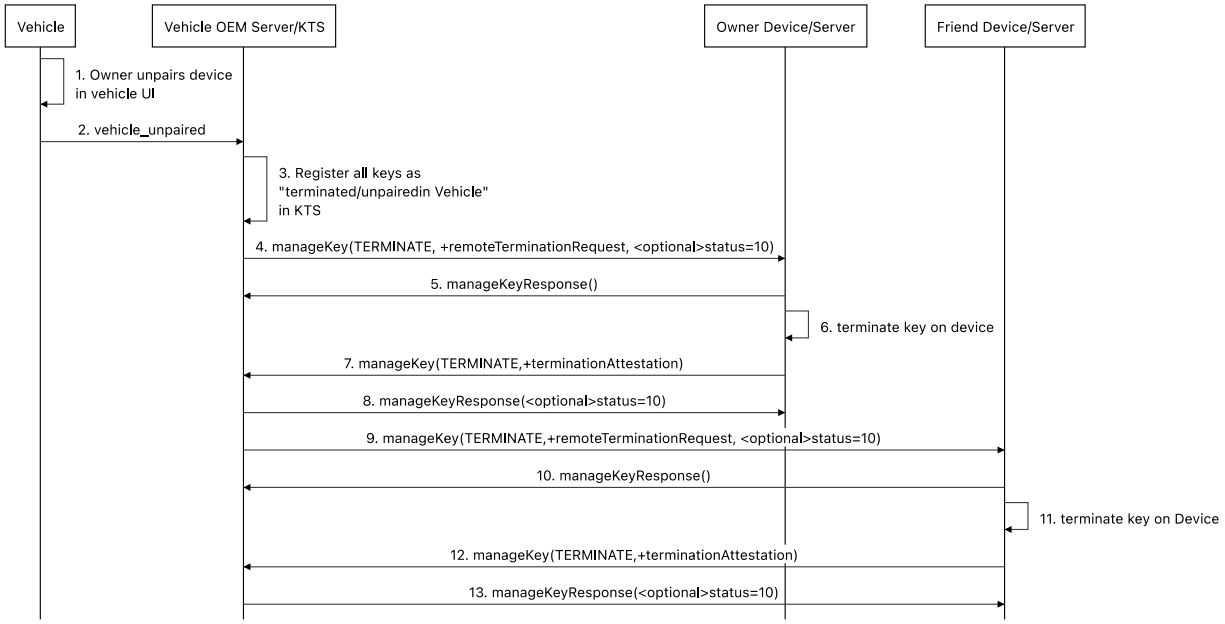
The unpaired state shall be registered in the KTS. Key deletion on the owner and friend devices is optional but recommended.

13.6.1 *REV_700: Unpairing in vehicle UI (sell vehicle)*

If unpairing is started from the vehicle, key deletion is executed under Vehicle OEM-defined conditions, and the owner device is notified.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

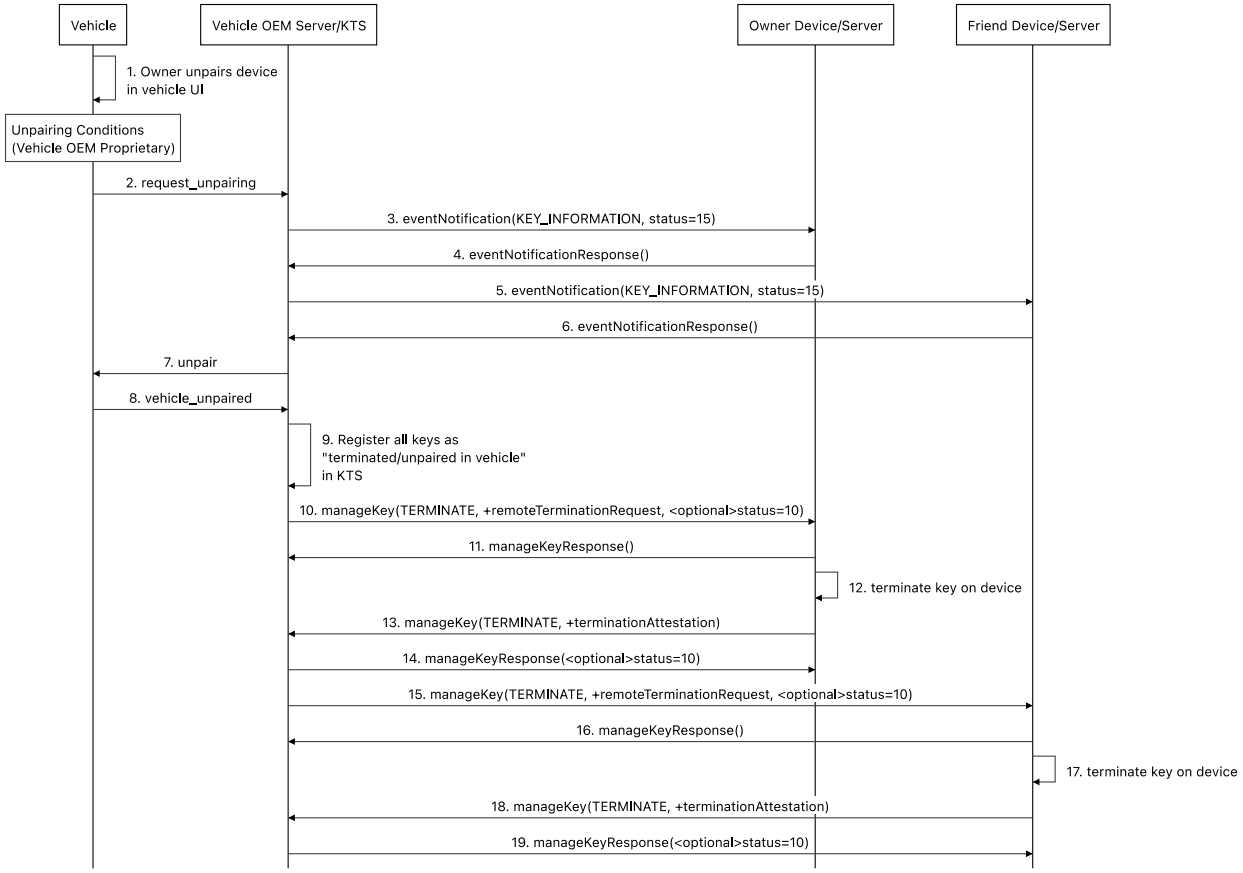
Figure 13-19: REV_700: Unpairing in Vehicle UI (Sale of Device)



Alternatively, it may be required for the vehicle to request online unpairing. In that case, devices are informed about the termination request and the successful termination (See Figure 13-20).
Note: Inability to reach owner or friend device in Steps 3 through 6 does not stop the process

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-20: REV_700a: Unpairing in Vehicle UI (Vehicle Required to be Online)

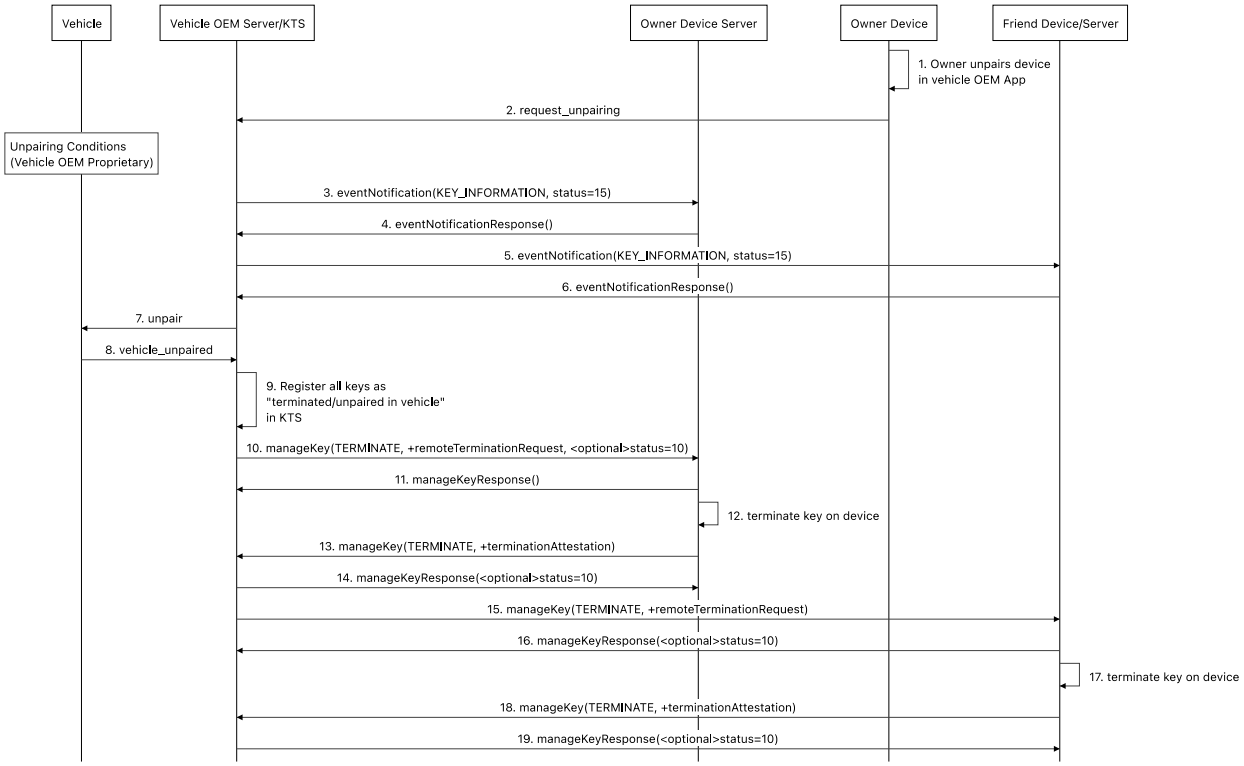


13.6.2 *REV_710: Unpairing on owner device in Vehicle OEM app*

If the owner device is unpaired through the Vehicle OEM app, the app shall not start the unpairing process on the device. The app shall notify the Vehicle OEM Server about the unpairing request, and the Vehicle OEM Server shall execute the unpairing process. Note: Inability to reach friend device in Steps 5 through 6 does not stop the process.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-21: REV_710: Unpairing in Vehicle OEM App on Owner Device



13.6.3 *REV_720: Unpairing in owner Vehicle OEM account (sale of vehicle)*

This flow is similar to REV_710 with the Vehicle OEM Server triggering the unpairing process.

13.6.4 *REV_730: Unpairing based on cancellation or expiry date of the Digital Key service*

This flow is similar to REV_710 with the Vehicle OEM Server triggering the unpairing process.

13.6.5 *REV_740: Unpairing by Vehicle OEM (vehicle stolen)*

This flow is similar to REV_710 with the Vehicle OEM Server triggering the unpairing process.

13.6.6 *REV_750: Unpairing by Vehicle OEM (security breach in vehicle)*

This flow is similar to REV_710 with the Vehicle OEM Server triggering the unpairing process.

13.6.7 *REV_760: Unpairing by Vehicle OEM (garage service process)*

This flow is similar to REV_710 with the Vehicle OEM Server triggering the unpairing process.

13.6.8 *REV_770: Unpairing due to deletion of personal data in owner Vehicle OEM account*

This flow is similar to REV_710 with the Vehicle OEM Server (account) triggering the unpairing process.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

13.6.9 *REV_800: Unbinding of vehicle from owner account in vehicle UI (unpairing, owner sale of vehicle)*

This flow is similar to REV_710 with the vehicle UI triggering the unpairing process.

13.6.10 *REV_810: Unbinding of vehicle from owner account in owner Vehicle OEM account (unpairing, owner sale of vehicle)*

This flow is similar to REV_710 with the Vehicle OEM account triggering the unpairing process.

13.6.11 *REV_820: Unbinding of vehicle from owner account in Vehicle OEM app (unpairing, owner sale of vehicle)*

This flow is similar to REV_710 with the Vehicle OEM app triggering the unpairing process.

13.7 Termination in Vehicle

13.7.1 *Rules*

Digital Key termination on the vehicle side requires protection against replaying all or a part of an earlier key sharing process. The following preconditions shall apply:

- **C1**: The initial slot identifier (SlotID in the example below) values in the private mailbox shall be set to all bytes FF_h.
- **C2**: The initial Slot Identifier Bitmap (SlotID Bitmap in the example below) value is 00_h.

The vehicle shall implement the following rules if Tag DA_h (as defined in Table 5-14) is set to 00_h (slot identifiers and immobilizer tokens) or 02_h (slot identifiers only) are provided by the vehicle:

- **Rule V1**: Accept a key addition only if the friend slot identifier (as described in Section 4.3.1) is verified to be valid for an available key storage in the vehicle.
- **Rule V2**: When a key is deleted in the vehicle (from any interface), delete the data (device PK, etc.) in the corresponding key storage and assign a new slot identifier. The vehicle OEM shall implement appropriate means to ensure replay protection. This new slot identifier is available for future Digital Key sharing. When all bits in the Slot Identifier Bitmap are set to 1, the vehicle shall set the corresponding bit in the signaling bitmap to 0 to indicate that all slot identifiers are written. To avoid inconsistent states, either all of the described operations shall be done in one atomic transaction, or the vehicle shall verify the consistency of all bits and values after each console reader transaction and update accordingly.
- **Rule V3**: On each console reader transaction, the vehicle refills slot identifiers in each entry with bit=0 and sets the bit to 1 in an atomic transaction. The vehicle ensures the uniqueness of the slot identifiers and memorizes the next slot identifier to be written to the device.

All devices shall implement the following rules:

- **Rule D1** (only applies to owner device): Provide a matching pair of slot identifier and, if applicable, immobilizer token from the confidential mailbox to the friend when sharing a

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Digital Key. Set the value in the corresponding bit of the Slot Identifier Bitmap in the private mailbox to 0. Set the corresponding bit in the signaling bitmap to 1 if it was set to 0 before sharing. The device shall always use the lowest value of all slot identifiers for key sharing.

- **Rule D2**: When a key is terminated in the device locally, send the termination attestation and the Vehicle OEM proprietary data to the Vehicle OEM Server to trigger the deletion of the Digital Key in the vehicle.

13.7.2 *Example*

The example below shows the principle of key termination and slot identifier refill as per the above rules. In this example, the following notes apply:

1. Immobilizer tokens are not shown.
2. The vehicle has a number of key stores (Key Storage) in memory (0–10; see Figure 13-22).
3. Key storage 0 is reserved for the owner key.
4. The slot identifiers are refilled by the vehicle starting with “the highest slot identifier value in the private mailbox of the device + 1.”
5. For all steps described in the example, the vehicle shall update the signaling bitmap after writing the slot identifier bitmap and the slot identifiers to signal an update to the owner device.
6. Slot identifiers and immobilizer tokens are provided by the vehicle.

Figure 13-22 shows the device and vehicle memory after owner pairing and before the vehicle has identified the immobilizer tokens and the corresponding slot identifiers for friend sharing (if the immobilizer tokens are retrieved from the vehicle).

Initially all device-side key storages are set to FF_h FF_h (Rule C1). The signaling bitmap shows that slot identifiers need to be refilled.

Figure 13-22: Example Step 1: After Owner Pairing

Owner Device

Framework		Mailbox	
Assigned	**Valid Key List**	**SlotID Bitmap**	**SlotID**
00 00	Owner PK	1	00 00
		0	FF FF
		0	FF FF
		0	FF FF
		0	FF FF
		0	FF FF
		0	FF FF
		0	FF FF

SigBmp

Bit	7	6	5	4	3	2	1	0
Value	x	x	x	x	x	x	x	1

Step 1
Owner Pairing
(Phase 3)

Vehicle

Key Storage	SlotID	Valid Key List
0	00 00	Owner PK
1	00 01*	empty
2	00 02	empty
3	00 03	empty
4	00 04	empty
5	00 05	empty
6	00 06	empty
7	00 07	empty
8	00 08	empty
9	00 09	empty
10	00 0A	empty

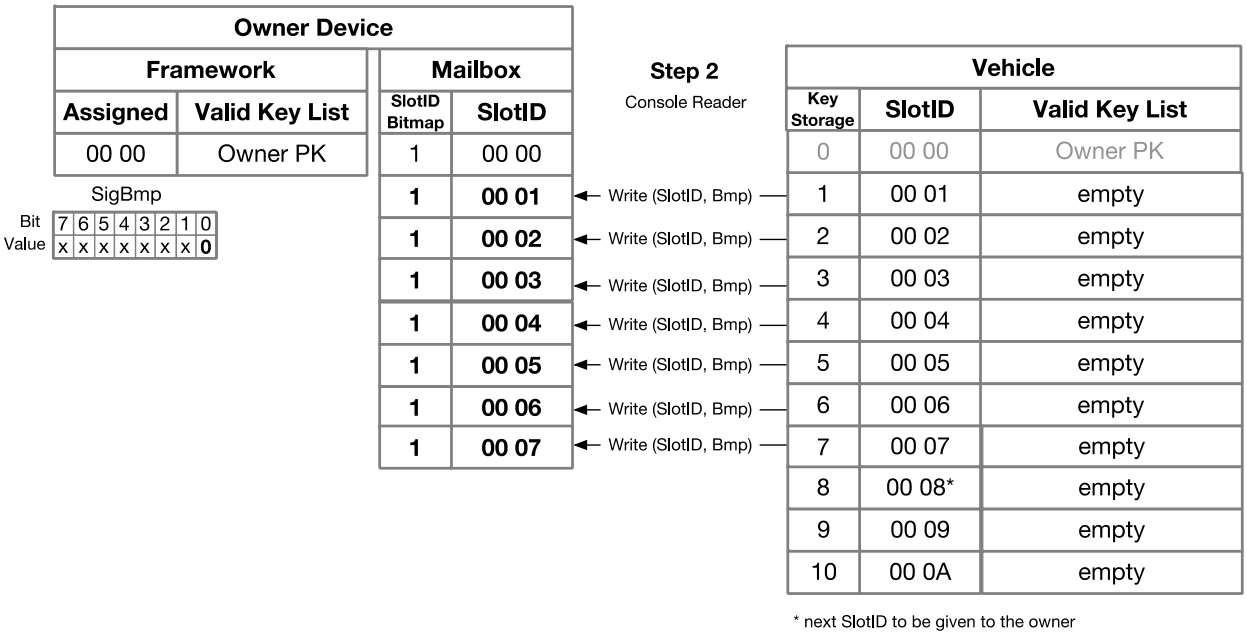
* next SlotID to be given to the owner

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 13-23 shows device and vehicle memory after the vehicle has provisioned the slot identifiers. All entries on the device containing FF_h FF_h have been filled with slot identifier values, and the signaling bitmap has been updated to show that no slot identifier entries are missing in SlotIdentLst (**Rule V3**).

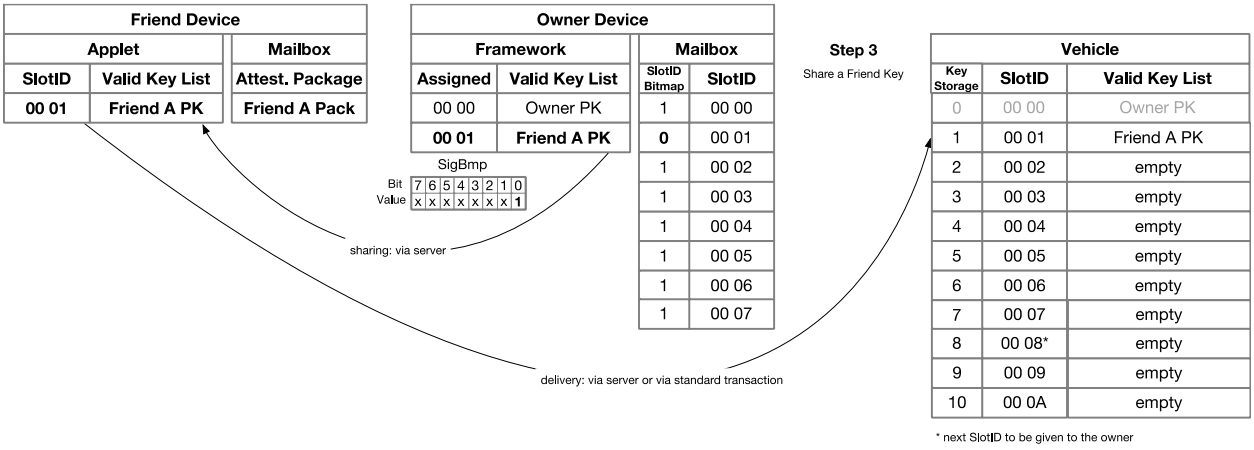
The vehicle memorizes the next slot identifier to be provisioned into the device (*).

Figure 13-23: Example Step 2: Device Fully Refilled



In Figure 13-24, the owner has shared one key with a friend, and the slot identifier bitmap and signaling bitmap have been updated accordingly (**Rule D1**). The friend key is accepted by the vehicle, either via server link or via a standard transaction (**Rule V1**). The slot identifier 00_h 01_h has been used for the friend key (SlotID Bitmap bit= 0). The framework of the owner device memorizes the assigned friend slot identifiers.

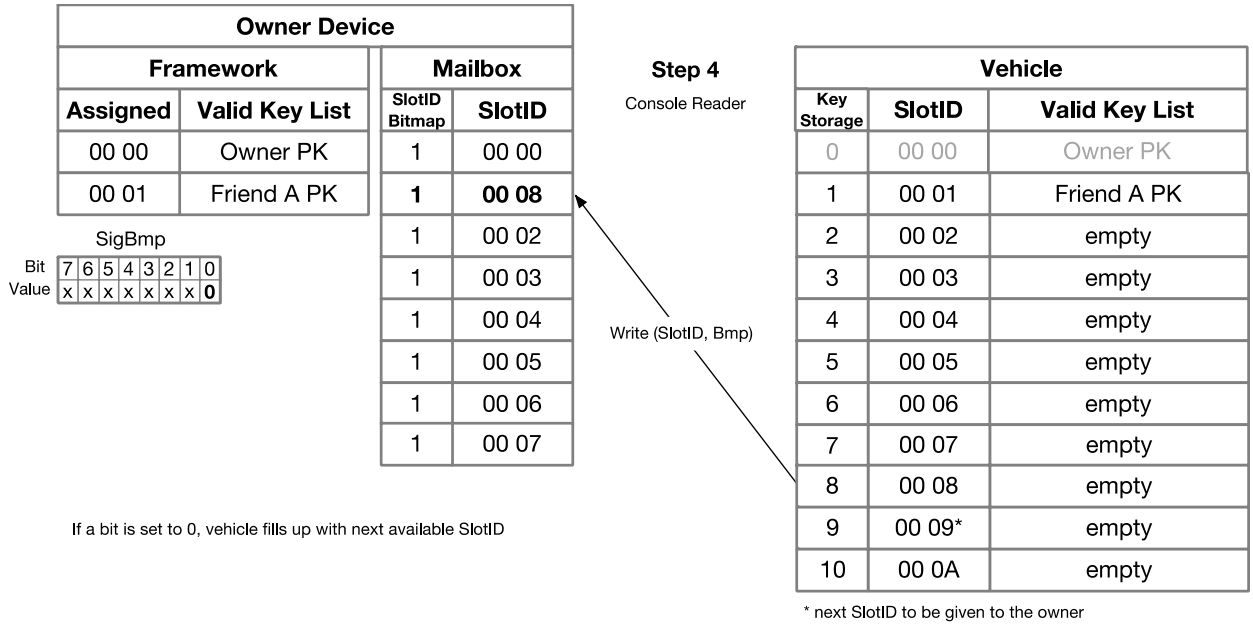
Figure 13-24: Example Step 3: Key Shared with Friend



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

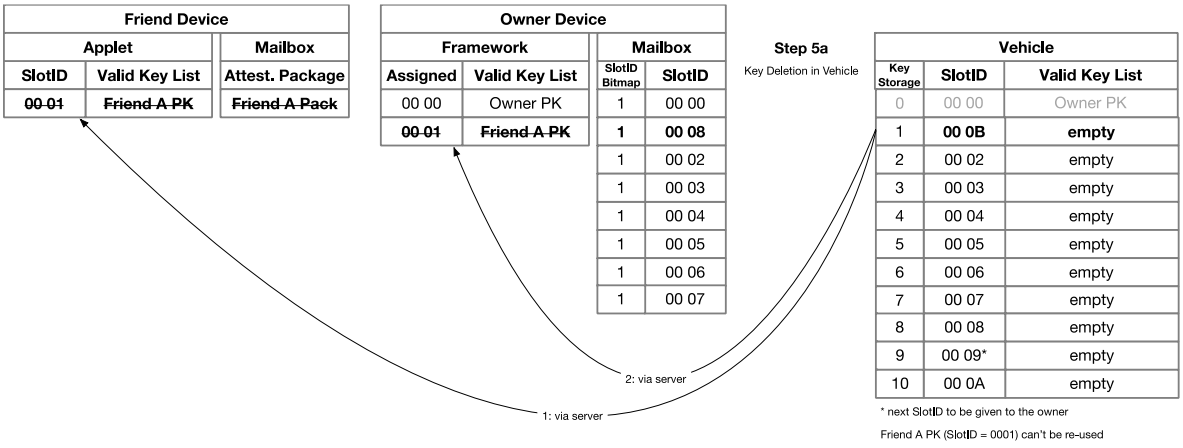
In Figure 13-25, the vehicle has refilled the empty entry in the mailbox with the next available slot identifier (00_h 08_h) and memorizes 00_h 09_h as the next available slot identifier. The corresponding bit in the slot identifier bitmap has been set to 1 and the one in the signaling bitmap has been set to 0 by the vehicle.

Figure 13-25: Example Step 4: Refill of Shared Slot Identifier



In Figure 13-26, the friend key has been deleted in the vehicle (**Rule V2**). The vehicle has emptied the key storage and has assigned slot identifier value 00_h $0B_h$. When the device presents the Digital Key (attestation package) of friend A to the vehicle again, the vehicle rejects the key (**Rule V1**).

Figure 13-26: Example Step 5a: Key Termination in Vehicle

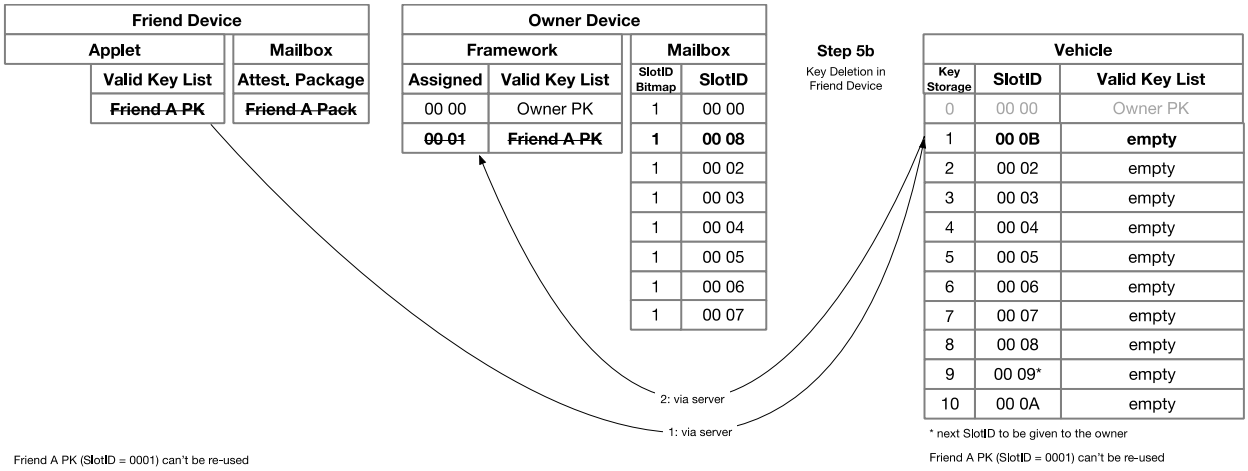


Key termination in the vehicle may be originated in the vehicle, from the Vehicle OEM Server, from the Device OEM Server, or from the owner device. The key termination is propagated to the friend device via the Vehicle OEM Server, which causes remote key deletion on the friend device.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

In an alternative, Figure 13-27, the friend key has been deleted in the device (**Rule D2**). The device deletes the key locally and sends the termination attestation and the Vehicle OEM proprietary data to the Vehicle OEM Server to trigger the deletion of the Digital Key in the vehicle. The vehicle removes the key as for a remote deletion (**Rule V2**).

Figure 13-27: Example Step 5b: Key Termination in Device



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

14 AUTHENTICATION AND PRIVACY

14.1 Authentication and Privacy Keys

14.1.1 *Usage*

Sensitive data exchanged between the device and the Vehicle OEM shall be encrypted. The currently identified data includes:

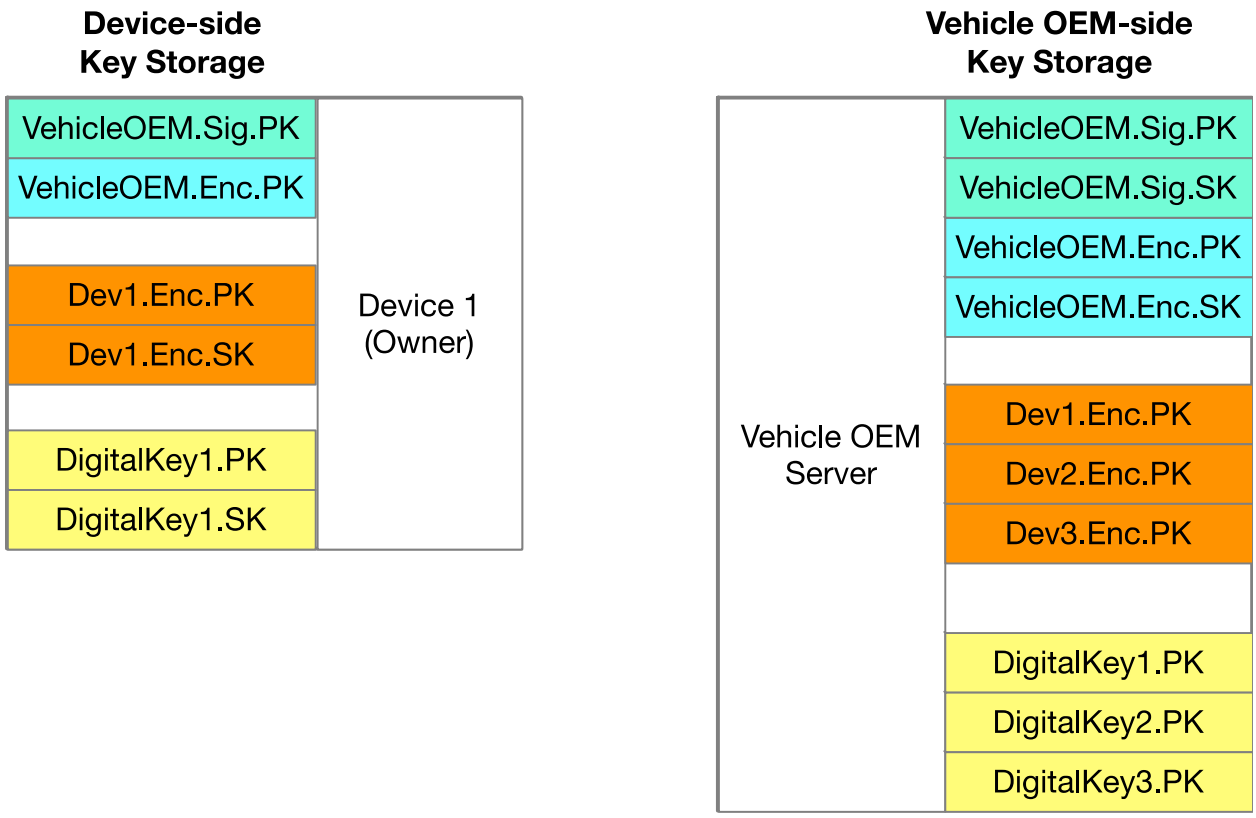
- Key tracking request data (content: attestation package, device encryption public key)
- Notification messages to the owner device that contain friend key information (e.g., the Digital Key identifier of a friend)
- Remote key deletion requests from the owner device that contain friend key information (e.g., the Digital Key identifier of a friend)

Remote key deletion requests that are sent from a device to the Vehicle OEM Server shall be authenticated through a signature.

Key deletion commands that are sent from the Vehicle OEM Server to the target device shall be authenticated through a signature.

Figure 14-1 shows the keys that are used for encryption/decryption and signature verification.

Figure 14-1: Message Authentication and Privacy Encryption



See Table 14-1 for a complete list of involved keys.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 14-1: Authentication and Privacy Keys

Key	Created by	Purpose	Description
VehicleOEM.Sig.SK	Vehicle OEM Server	Vehicle OEM Server signs the data sent to the device	Safely stored in Vehicle OEM Server
VehicleOEM.Sig.PK	Vehicle OEM Server	Device authenticates the data from the Vehicle OEM Server	Certificate pre-stored in device OS
VehicleOEM.Enc.SK	Vehicle OEM Server	Vehicle OEM Server decrypts the data sent from the device	Safely stored in Vehicle OEM Server
VehicleOEM.Enc.PK	Vehicle OEM Server	Device encrypts the data sent to the Vehicle OEM Server	Certificate pre-stored in device OS
Devx.Enc.SK	Device	Device decrypts the data sent from the Vehicle OEM Server	Created at owner pairing/key sharing, safely stored in the device OS if not already existing
Devx.Enc.PK	Device	Vehicle OEM Server encrypts the data sent to the device	Created at owner pairing/key sharing, sent in key tracking request
DigitalKeyx.SK	Device	Device signs the data sent to the Vehicle OEM Server	This is the Digital Key SK, safely stored in the SE
DigitalKeyx.PK	Device	Vehicle OEM Server authenticates the data sent from the device	This is the Digital Key PK

The privacy and authentication keys are used in the following cases, described in detail in Section 17:

1. The key tracking request contains Devx.Enc.PK as part of the encrypted payload. The payload is encrypted using VehicleOEM.Enc.PK with the encryption scheme described in Section 14.3. The Vehicle OEM Server stores Devx.Enc.PK associated with the Digital Key identifier provided in the attestation chain.
2. The remote (Digital Key) deletion request sent from the owner device to the Vehicle OEM Server contains the Digital Key identifiers of owner and friend in a structure described in Section 14.2. The request data is signed using the owner private key (DigitalKeyx.SK) and encrypted using VehicleOEM.Enc.PK, as it contains the Digital Key identifier of a friend. The signature scheme is described in Section 14.3.
3. The key deletion request sent from the Vehicle OEM Server to the target device is signed by the Vehicle OEM Server using the VehicleOEM.Sig.SK. The signature is verified by the target device using VehicleOEM.Sig.PK.
4. The notification which is sent from the Vehicle OEM Server to the owner device is encrypted using Devx.Enc.PK, as it contains the Digital Key identifier of a friend.

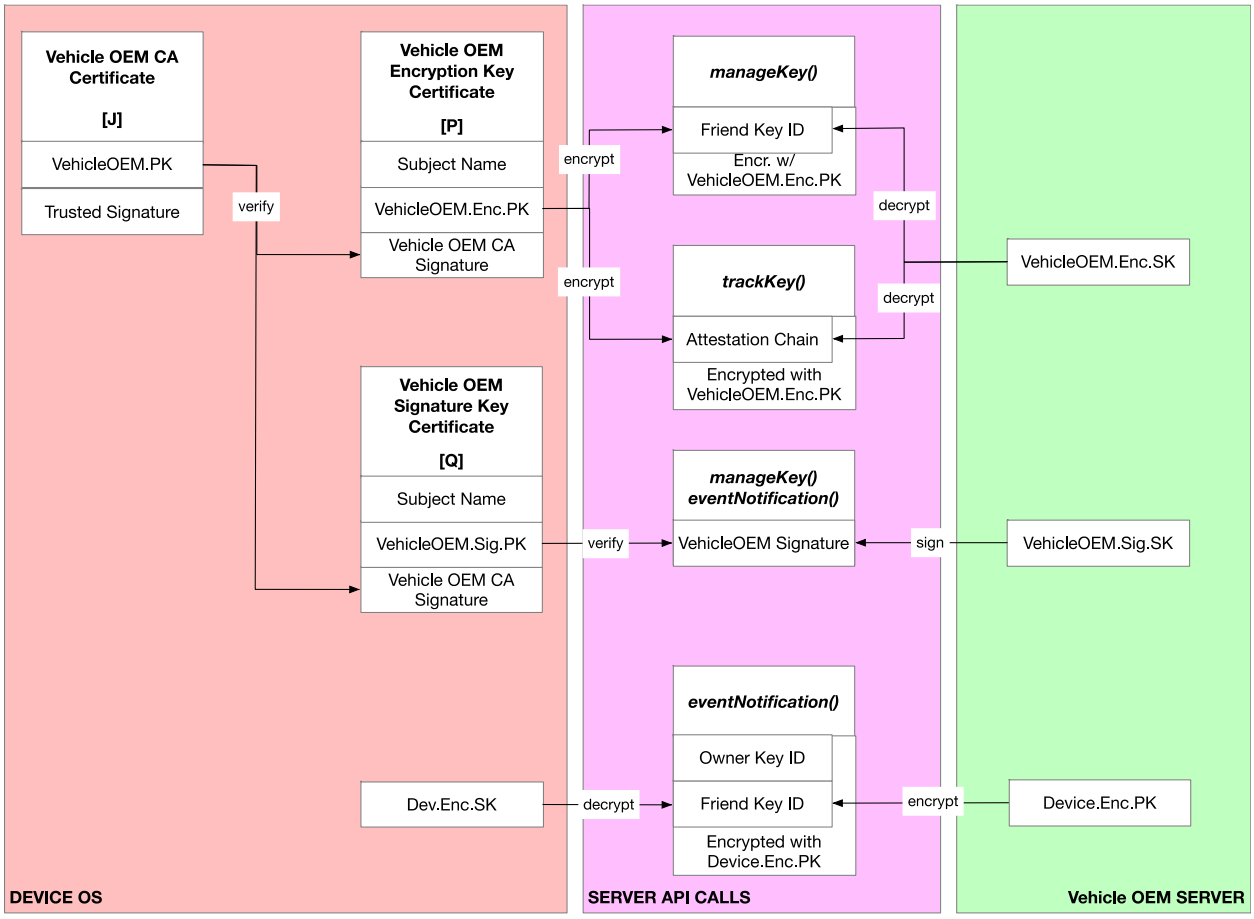
14.1.2 *Certificate Chains*

All public keys used for authentication and privacy are embedded in X.509 certificates except the Device Privacy Encryption Key, and are chained to the trusted root as shown in Figure 14-2. The trust chain of all certificates shall be verified before using them.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The usage of each certificate's public key is described in Table 14-1 and in Section 14.1.1. All private keys are assumed to be stored safely where they have been created.

Figure 14-2: Authentication and Privacy Encryption Certificate Chain



14.1.2.1 [H] – Digital Key Creation Attestation

See Section 16.2.8 for a description of this certificate.

14.1.2.2 [J] – Vehicle OEM CA Certificate

See Section 16.2.9 for a description of this certificate.

14.1.2.3 [P] – Vehicle OEM Privacy Encryption Certificate

This certificate is generated by the Vehicle OEM Server and provided to the Device OEM. Each Device OEM shall receive a different public key in the certificate. The transfer of the certificate from the Vehicle OEM Server via the Device OEM Server to the device is out of scope of this specification. The certificate should have a lifetime defined by Vehicle OEM policy. Revocation may be achieved by stopping the renewal process or through revocation methods (CRL, OCSP), which are out of scope of this specification.

The certificate shall have proprietary extension using the assigned OID listed in Appendix B.2. The extension is defined as shown in Listing 14-1 and Listing 14-2.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Listing 14-1: Vehicle OEM Privacy Encryption Certificate Extension Schema

```
VehicleOEMPrivacyEncryptionCertificateExtensionSchema ::= SEQUENCE
{
  extension_version  INTEGER (1 ..255)
}
```

Listing 14-2: Vehicle OEM Privacy Encryption Certificate Extension Data

```
vehicle-oem-privacy-encryption-cert-extension-data VehicleOEMPrivacyEncryptionCertificateExtensionSchema ::=
{
  extension_version  1    --value shall be 1
}
```

14.1.2.4 VehicleOEM Signature Key Certificate

This certificate is generated by the Vehicle OEM Server and provided to the Device OEM. Each Device OEM shall receive a different public key in the certificate. The transfer of the certificate from the Vehicle OEM Server via the Device OEM Server to the device is out of scope of this specification. The certificate should have a lifetime defined by Vehicle OEM policy. Revocation may be achieved by stopping the renewal process or through revocation methods (CRL, OCSP), which are out of scope of this specification

The certificate shall have a proprietary extension using the assigned OID listed in Appendix B.2.2. The extension is defined as shown in Listing 14-3 and Listing 14-4.

Listing 14-3: Vehicle OEM Signature Certificate Extension Schema

```
VehicleOEMSignatureCertificateExtensionSchema ::= SEQUENCE
{
  extension_version  INTEGER (1 ..255)
}
```

Listing 14-4: Vehicle OEM Signature Certificate Extension Data

```
vehicle-oem-signature-cert-extension-data VehicleOEMSignatureCertificateExtensionSchema ::=
{
  extension_version  1    --value shall be 1
}
```

14.1.2.5 Device Privacy Encryption Key

This key is generated by the device and provided to the Vehicle OEM Server in the TrackKey() API call (see Section 17.7). Each device shall generate a unique device encryption key pair per Digital Key [H].

14.2 Remote Termination Requests

The Remote Termination Request shall be sent when either the owner device or the Vehicle OEM Server requests the deletion of a Digital Key on a target device.

If the owner device originates the Remote Termination Request, the request structure in Table 14-3 shall be used.

If the Vehicle OEM Server or the vehicle itself originate the Remote Termination Request, the request structure in Table 14-4 shall be used. A specifically defined source key id indicates whether the request is originated by the vehicle or the Vehicle OEM Server (owner or friend account, subscription ended, etc.).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

When the key id of the target key is not known to the source device, the target key id tag shall be absent.

If the key id of the target key is known to the source device but not to the server and vehicle, the slot identifier tag shall be used to identify the key to terminate. The corresponding slot identifier shall be deleted in the server and vehicle. When the target device attempts to track the key after its deletion, the track key request shall be rejected.

In some cases, the slot identifier might not be known. In those cases, the key identifier shall be used to identify the key to be deleted.

Table 14-2 describes the data fields of the Remote Termination Request.

Table 14-2: Remote Termination Request Data Fields

Tag	Length (bytes)	Description	Field is	Domain Version
$5F21_h$	20	subject key identifier (key id) of the source Digital Key	mandatory	D-VS
$7F23_h$	variable	list of key identifier and slot identifier pairs of the target Digital Key(s)	mandatory	D-VS
61_h	variable	pair of key identifier and slot identifier of 1st Digital Key	mandatory	D-VS
50_h	20	subject key identifier	conditional	D-VS
57_h	variable	slot identifier	conditional	D-VS
61_h	variable	pair of key identifier and slot identifier of 2nd Digital Key	conditional	
50_h	20	subject key identifier	conditional	
57_h	variable	slot identifier	conditional	
…	…	…	…	

The request shall contain either subject key identifier or slot identifier in element (tag 61_h). It should contain both if they are available.

The request may contain one or more pairs of target slot identifier and target key identifier. The maximum number of keys in the request may be defined by the Vehicle OEM; otherwise it is limited to 16.

The input data fields for signature creation for the remote termination request for the owner device and the Vehicle OEM Server are described in Table 14-3 and Table 14-4, respectively.

The signature creation for the remote termination request is described in Table 14-3.

Table 14-3: Remote Termination Request Owner Device

Tag	Length (bytes)	Description	Field is	Domain Version
$7F40_h$	variable	Remote Termination Request owner device	mandatory	D-VS
content of Table 14-2			mandatory	D-VS

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
SIG-DAT: content of Table 15-56 (Signature Data fields) with: arbitrary_data_ = SHA-256 hash value of Table 14-2			mandatory	V-OD-FW
$9E_h$	64	signature with the private key of the owner Digital Key over fields from SIG-DAT	mandatory	V-OD-FW

Note: The counter_value field (deprecated) in Table 15-56 is not present for SIG-DAT computation.

Table 14-4: Remote Termination Request from Vehicle OEM Server

Tag	Length (bytes)	Description	Field is	Domain Version
$7F39_h$	variable	Remote Termination Request from Vehicle OEM Server	mandatory	D-VS
content of Table 14-2 with: source key id = <vehicle key id>, if originated in vehicle source key id = <vehicle OEM key id>, if originated in Vehicle OEM Server source key id = <owner key id>, if originated by owner device via Vehicle OEM Server				D-VS
$5F22_h$	20	Subject key identifier of the certificate to be used for signature verification (VehicleOEM.Sig.Cert)	Mandatory	D-VS
$9E_h$	64	signature with the private key of the Vehicle OEM Server (VehicleOEM.Sig.SK) over fields from Table 14-2	mandatory	D-VS

The <vehicle OEM key id> is defined as 20 bytes of FF_h. It is used when the key was deleted by the Vehicle OEM (e.g., subscription ended).

The <vehicle key id> is defined as 20 bytes of EE_h. It is used when the key was deleted in the vehicle.

<owner key id> is provided in the remote termination request coming from the owner. It is used when the key was deleted by the owner device.

The <device OEM key id> is defined as 20 bytes of DDh. It is used when the key was deleted by the Device OEM (e.g. remote device wipe).

The signature covers tags $5F21_h$ and $7F23_h$, inclusive of tags and lengths, as defined in Table 14-2.

When receiving a remote termination request from the owner device (Table 14-3), the Vehicle OEM Server shall verify the following before executing the termination request:

- Value of the hash of Table 14-2 matches the arbitrary_data value provided in SIG-DAT
- Signature is correct

When receiving a remote termination request from the Vehicle OEM Server (Table 14-4), the friend device shall verify the following before executing the termination request:

- Source key ID corresponds to the key ID of the originator
- Signature is correct

The Device OEM Server shall ensure the following security properties:

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- Each termination request from a mobile device to the Device OEM Server shall contain a signed terminationAttestation (owner or friend device deletes its own key) or remoteTerminationRequest (owner deletes friend key).
- Each termination request and each termination response from a mobile device to the Device OEM Server shall contain the Vehicle OEM proprietary data subsection of the private mailbox.
- The user can delete only his/her own keys in the device portal, and not the shared keys.
- When keys are deleted in the Device OEM portal (see Table 13-4) and the device could not be reached, the Device OEM Server is not required to provide a remoteTerminationRequest to the Vehicle OEM Server.

14.2.1 *Example of RTR Signature Verification*

RTR –
7F397C5F2114FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF7F2320611E5014444DF
CECAA57A75B4C42D60A7EB1868B793BEDE05706000F000000019E40CD846C5BCFAB51
41CE129A7C197129220EB5747B9A80C1726DEA200FFA42B2178235452B60DD60AA8D7F
9EB43EA486A87C91B9BC9886D83DAE9583F1112D183A

SIG –
CD846C5BCFAB5141CE129A7C197129220EB5747B9A80C1726DEA200FFA42B217823545
2B60DD60AA8D7F9EB43EA486A87C91B9BC9886D83DAE9583F1112D183A

TBS –
5F2114FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF7F2320611E5014444DFCECAA
57A75B4C42D60A7EB1868B793BEDE05706000F00000001

PK -
043517d1a4043c6738a2fc0458fecd4ae94c833b7fac59fbf697ee65979bcd64cbc3ce7323a6c5f0c6
6e9500734e33f0bd93c280b7299fc85674bbc64766492e25

>>> pubkey = bytes.from-
hex('3517d1a4043c6738a2fc0458fecd4ae94c833b7fac59fbf697ee65979bcd64cbc3ce7323a6c5f0
c66e9500734e33f0bd93c280b7299fc85674bbc64766492e25')

>>> data = bytes.from-
hex('5F2114FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF7F2320611E5014444DFCE
CAA57A75B4C42D60A7EB1868B793BEDE05706000F00000001')

>>> sig = bytes.from-
hex('CD846C5BCFAB5141CE129A7C197129220EB5747B9A80C1726DEA200FFA42B2178
235452B60DD60AA8D7F9EB43EA486A87C91B9BC9886D83DAE9583F1112D183A')

>>> vk = ecdsa.VerifyingKey.from_string(pubkey, curve = curve)

>>> vk.verify(sig, data, hashfunc = hashfunc)

True

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

14.3 Encryption and Signature Verification Schemes

The data to be privacy-encrypted is described in Section 17. The encryption scheme to be used is described in Section 17.11. Signatures of the commands shall use ANSI X9.62 ECDSA algorithm with SHA-256 (see [23] and [24]).

14.4 OEM App Data Attestation

OEM App Data Attestation shall be used to obtain a signature over data provided by the OEM App, using the endpoint.SK. In response, all fields of the returned attestation shall be provided to the Vehicle OEM App. For OEM App Data Attestation, a request for User Consent shall be enforced. Enforcement of User Authentication is optional.

Table 14-5 describes the input fields of the OEM App Data Attestation.

Table 14-5: OEM App Data Attestation Input

ASN.1 Tag	Length (bytes)	Description	Field is
04_h	variable	App Bundle Identifier	mandatory
04_h	variable	OEM App Data	mandatory
04_h	variable	Nonce, defined by framework	mandatory

Attestation is generated as described in Listing 14-5.

Listing 14-5: OEM App Data Attestation Processing

```
Input: Table 14-5
output: attestation
begin
  perform user consent
    return error if user did not consent
  perform user authentication if required
  arbitrary_data = SHA-256 hash value of Table 14-5
  send SIGN command to retrieve attestation as described in Table 15-61
  return attestation
end
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

15 DIGITAL KEY APPLET

15.1 Introduction

The Digital Key applet is aimed at providing multi-purpose SE-based transaction mechanisms combined with peer-to-peer key distribution and a data storage system with strong security and privacy properties. Three types of contactless transaction may be used: standard transaction (see Section 7), fast transaction (see Section 8), and check presence transaction (see Section 10).

In this specification, two applet implementation models are provided, depending on the Device OEM's implementation or the Digital Key service deployment model.

- **SE-centric applet model**: For this model, the Device OEM CA Certificate's corresponding public key is protected by the SE, and the non-SE endpoints (e.g., vehicle, server, etc.) are verified by the SE.
- **Framework-centric applet model**: For this model, the Device OEM CA Certificate's corresponding public key is protected by the device OS native key store, and the non-SE endpoints (e.g., vehicle, server, etc.) are verified by the framework.

15.2 Keys and Data

This section defines some of the key and data elements used later in Section 15 to help understand the flow diagrams.

Cryptogram: MAC value calculated over unique public transaction data. The cryptogram proves that the device is in possession of the same symmetric key as the vehicle.

Kpersistent: Symmetric long-term key that is used to derive encryption and MAC session keys. It is stored in NVM on both vehicle and device sides.

Kenc: Derived symmetric key used to encrypt confidential commands and responses payloads

Kmac: Derived symmetric key used to calculate command MACs

Krmac: Derived symmetric key used to calculate response MACs

Kcmac: Derived symmetric key used to calculate cryptograms

Keseed: Symmetric that is used to derive keys for confidential data encryption and MAC

Keenc: Derived symmetric key used to encrypt confidential data

Kemac: Derived symmetric key used to compute MAC on encrypted confidential data

Kdh: Symmetric key generated by a Diffie-Hellman operation

ePK**/eSK**: denotes ephemeral public and private keys

PK**/SK**: denotes public and private keys

transaction_identifier: randomly generated nonce value and used on vehicle and device sides for symmetric key derivation and MAC/signature generation/verification

vehicle_identifier: unique identifier of the vehicle. On device side, it is used to look up the correct endpoint

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

15.3 Applet Implementation

15.3.1 *Introduction*

The following subsections describe internal data structures and APDU commands for the applet. This specification supports only one Digital Key applet protocol version. The Digital Key applet protocol version is set to 0100_h.

15.3.1.1 TLV Field

The various Tag Length Value fields presented in this document shall comply with the BER-TLV format as defined in ISO 7816-4 [1]. The TLV fields shall be ordered as described in this specification; a different field order is considered invalid unless specified otherwise. The nesting level is represented by indentation of Tag values in the Tag column.

15.3.1.2 Applet Memory Types

The naming conventions for the memory types used in the applet are as follows:

- Variable prefixed with “nvm.” Indicates the memory used is persistent after power off.
- Variable prefixed with “cod.” indicates the memory used is volatile and erased after applet deselection (caused by contactless field off or explicit deselection).
- Variable noted as (nvm/cod).object.variable indicates the variable is an object member.
- Variable with neither memory prefix nor object prefix indicates a local volatile variable erased after exiting the current programming context.
- All variables and objects are globally available at the instance level except the local variables defined above.
- Variable suffixed with [] indicates a table of variables.
- Variable suffixed with * indicates all the variables names starting with the string present before *.

15.3.1.3 Endpoint Life Cycle States

state_endpoint_active

In this state, the endpoint accepts all commands allowed by its configuration.

State_endpoint_terminated

In this state, the endpoint only accepts the TERMINATE ENDPOINT and DELETE ENDPOINT commands.

15.3.1.4 Communication Interfaces

The wired interface connects the SE to the device’s Digital Key framework through physical wires inside the device. The contactless interface connects the SE to the NFC reader in the vehicle through radio interface. The notifications sent from the applet to the Digital Key framework (denoted as notify_*) may be implemented as described in Section 15.3.1.9 or using a proprietary method.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1 *Table 15-1: Command Availability on Interfaces*

Command	Wired Interface	Contactless Interface	Comment	Section
SELECT	Yes	Yes		15.3.2.1
CREATE CA	Yes	No		15.3.2.2
DELETE CA	Yes	No		15.3.2.3
CREATE ENDPOINT	Yes	No		15.3.2.4
TERMINATE ENDPOINT	Yes	No		15.3.2.5
DELETE ENDPOINT	Yes	No		15.3.2.6
GET PRIVATE DATA	Yes	No		15.3.2.18
SET PRIVATE DATA	Yes	No		15.3.2.19
AUTHORIZE ENDPOINT	Conditional	No	Availability configured in Section 15.3.2.4	15.3.2.7
SET CONFIDENTIAL DATA	Yes	No		15.3.2.20
CREATE ENCRYPTION KEY	Yes	No		15.3.2.17
VIEW	Yes	No		15.3.2.8
SETUP ENDPOINT	Yes	No		15.3.2.21
SETUP INSTANCE	Yes	No		15.3.2.22
AUTH0	Conditional	Mandatory	Availability configured in 15.3.2.4	15.3.2.9
AUTH1	Conditional	Mandatory	Availability configured in Section 15.3.2.4	15.3.2.10
PRESENCE0	No	Yes		15.3.2.11
PRESENCE1	No	Yes		15.3.2.12
READ BUFFER	Yes	No		15.3.2.13
WRITE BUFFER	Yes	No		15.3.2.14
EXCHANGE	Conditional	Yes	Availability configured in Section 15.3.2.4	15.3.2.15
CONTROL FLOW	Yes	Yes		15.3.2.16
SIGN	Conditional	No	Availability configured in Section 15.3.2.4	15.3.2.23
MANAGE UA	Conditional	No	Availability configured in Table 15-5	15.3.2.24
CREATE RANGING KEY	Conditional	No	Availability depends on platform	15.3.2.26
DELETE RANGING KEYS	Conditional	No	Availability depends on platform	15.3.2.27
GET NOTIFICATION	Optional	No	Availability depends on the implementation	15.3.2.28

2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Implementations may require the use of a SCP such as SCP03 [7] or SCP11 [28] between the Digital Key applet and the Digital Key framework. In these cases, the Digital Key applet shall support usage of a SCP for the following commands: CREATE ENDPOINT, TERMINATE ENDPOINT, DELETE ENDPOINT, GET PRIVATE DATA, SET PRIVATE DATA, VIEW, SETUP ENDPOINT, SETUP INSTANCE, READ BUFFER, WRITE BUFFER, SIGN, AUTHORIZED ENDPOINT, SET CONFIDENTIAL DATA, CREATE ENCRYPTION KEY and MANAGE UA-. This option of the Digital Key applet implementation is called Option A (i.e., DK applet that supports SCP) in this section. The provisioning of secure channel keys is out of scope of this specification. In case the response data does not fit in the response APDU due to secure channel overhead (e.g., due to the addition of padding or response_mac), the applet shall respond with an error status word.

The commands set over the contactless interface are standardized commands. SELECT, AUTH0, AUTH1, PRESENCE0, PRESENCE1, EXCHANGE, and CONTROL FLOW commands are mandatory to be supported; all other commands are optional.

Over the wired interface, implementations may require the support of supplementary logical channels in addition to support for the basic logical channel. This option of the Digital Key applet implementation is called Option B (i.e., DK applet that supports supplementary logical channels) in this section.

15.3.1.5 Command Flows

Figure 15-1 describes the allowed command flows over the contactless interface; other command flows are rejected. The CONTROL FLOW command (which can be used anywhere after the SELECT command) is not represented in this figure.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 15-1: Authentication Command Flows Over Contactless Interface

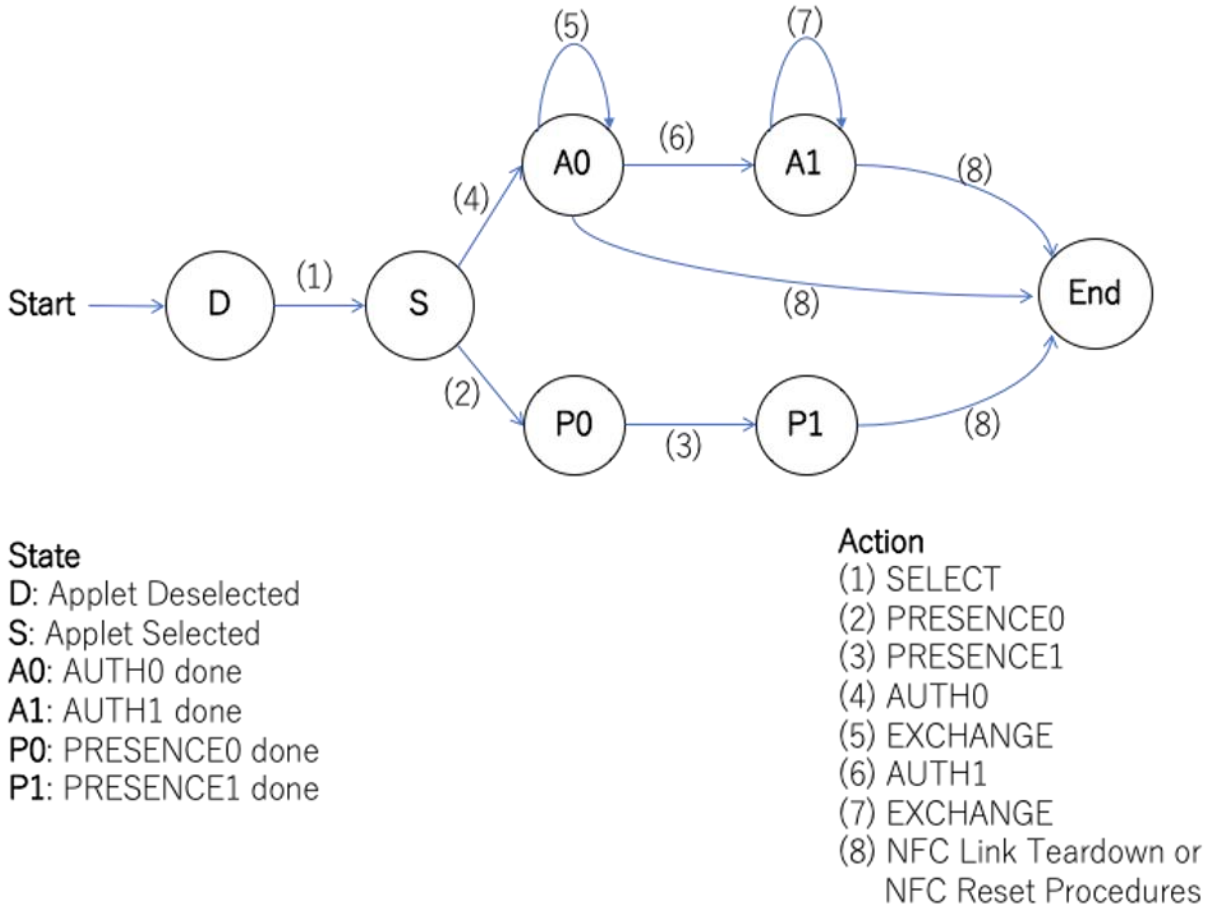
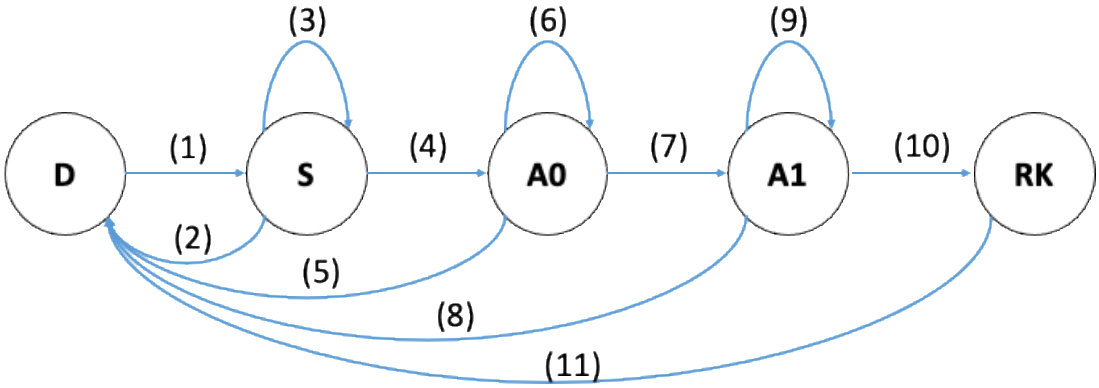


Figure 15-2 describes the allowed command flows over the wired interface. The CONTROL FLOW command (which can be used anywhere after the SELECT command) is not represented in this figure.

Figure 15-2: Authentication Command Flows Over Wired Interface

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential



States
D: Applet Deselected
S: Applet Selected
A0: Auth0 done
A1: Auth1 done
RK: Ranging Key done

Action
(1) SELECT
(2) Deselect
(3) Commands supported on Contact interface excluding AUTH0, AUTH1, EXCHANGE
(4) AUTH0
(5) Deselect
(6) EXCHANGE
(7) AUTH1
(8) Deselect
(9) EXCHANGE
(10) CREATE RANGING KEY
(11) Deselect

In state A0, only AUTH1 and EXCHANGE commands are accepted. In state A1, only EXCHANGE and CREATE RANGING KEY commands is accepted.
GET NOTIFICATION command, if implemented, may be sent at any time after the execution of a command. It has not impact on the state

15.3.1.6 Generic Error Handling

This section applies to all listed commands. It describes the generic status words to be retrieved in case of error during basic input command checking.

The basic input command checking includes checking that an INS is allowed on a given interface, that the CLA is consistent, that P1/P2 bytes have valid values, that Lc is in valid range, and checking the format of the payload.

The basic input command checking is executed before the steps described in the Listings below and can result in the error status words described in Table 15-2.

This specification does not describe all other possible errors that can occur during a command processing when not specified in the command listing. In such cases, the choice of the status word is left to the implementer. However, usage of status word 6400_h is highly recommended to ensure interoperability.

Digital Key applet should return an error if an APDU command includes unexpected or unknown parameters or tags

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-2: Generic Status Words

Status Word	Comment
6400_h	No specific diagnostic
$6A80_h$	Wrong payload format
$6A84_h$	Not enough memory
$6A86_h$	Wrong P1 or P2
$6A88_h$	Reference data not found
6700_h	Wrong command payload length
$6D00_h$	Wrong INS code
$6E00_h$	Wrong CLA code
9000_h	Command successfully executed

15.3.1.7 Class byte coding

The class byte coding of all commands shall conform to Section 5.4 of [1]. The value of the class byte depends on which logical channel (basic or supplementary) the command is addressed for and on whether or not secure messaging is used. As a result:

- When the implementation supports Option B, a range of class byte values is applicable to all the commands available on the wired interface. For commands addressed to the basic logical channel or supplementary logical channels 1, 2, and 3, the class byte coding is defined in Table 2 of [1]. For commands addressed to supplementary logical channels 4 through 19, the class byte coding is defined in Table 3 of [1].
- When the implementation supports Option A, a range of class byte values is applicable to the subset of commands defined in Section 15.3.1.4 when a secure channel is used between the Digital Key applet and the Digital Key framework.

For commands addressed to the basic logical channel or to supplementary logical channels 1, 2, and 3:

- Bit b8 is set to 0 for the SELECT command and to 1 for all other commands defined in this specification.
- Bits b7 to b5 are set to 0.
- When a secure channel is used, bits b4 and b3 are set to 01; otherwise they are set to 00.
- Bits b2 and b1 indicate the logical channel number from 0 to 3.

For commands addressed to the supplementary logical channel 4 through 19:

- Bit b8 is set to 0 for the SELECT command and to 1 for all other commands defined in this specification.
- Bit b7 is set to 1.
- When a secure channel is used, bit b6 is set to 1; otherwise it is set to 0.
- Bit b5 is set to 0.
- Bits b4 to b1 indicate the logical channel number from 4 through 19 (i.e., 0000 for logical channel 4 through 1111 for logical channel 19)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-3 lists the four different ranges of class byte values to be supported for the different commands, depending on the options implemented by the Digital Key applet. The determination of whether a command shall use CLA1, CLA2, CLA3, or CLA4 range is defined in the command description provided in Section 15.3.2.

Table 15-3: Coding of the Different Ranges of Class Byte Values

Range of values of the class byte (CLA)	Over the contactless interface (hex)	Over the wired interface (hex)			
		DK applet implementation does not support Options A or B.	DK applet implementation supports Option A but does not support Option B.	DK applet implementation supports Option B but does not support Option A.	DK applet implementation supports Options A and B.
CLA1	00	00	00	00 – 03 or 40 – 4F	00 – 03 or 40 – 4F
CLA2	N/A	80	80 or 84	80 – 83 or C0 – CF	80 – 87 or C0 – CF or E0 – EF
CLA3	80	80	80 or 84	80 – 83 or C0 – CF	80 – 83 or C0 – CF
CLA4	84	84	84	84 – 87 or E0 – EF	84 – 87 or E0 – EF

15.3.1.8 INSTALL for INSTALL parameters

This command defined in GlobalPlatform specification [20] creates a Digital Key applet instance and allocates the needed memory.

The following values are provided in tag $C9_h$ during the INSTALL for INSTALL command. Refer to GlobalPlatform specification [20] for formatting of this command.

The INSTALL for INSTALL command might include additional parameters, depending on the Device OEM policy for managing the non-volatile memory available for the Digital Key applet (e.g., using the Non-volatile Memory Quota parameters defined in GlobalPlatform [20]).

Table 15-4: INSTALL for INSTALL Content of Tag C9

Tag	Length (bytes)	Description	Field is
$A0_h$	1	install_param_endpoint_count, in range 1–255	mandatory
$A1_h$	1	install_param_instance_CA_count, in range 1–255	mandatory
$A2_h$	2	install_param_internal_buffer_size (MSB|LSB), in range 1024–max (where, “max” is defined by the Device OEM)	mandatory
$A3_h$	2	install_param_max_allocatable_private_mailbox_size (MSB|LSB), in range 0–2048	mandatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is
$A4_h$	2	install_param_max_allocatable_confidential_mailbox_size (MSB|LSB), in range 0–2048	mandatory
$A6_h$	variable	install_param_oid_external_ca	mandatory
$A7_h$	variable	install_param_oid_instance_ca	mandatory
$A8_h$	variable	install_param_oid_endpoint	mandatory
63_h	1	applet_implementation_options	optional

Install_param_endpoint_count

The maximum number of endpoints that can be created on an instance of the applet.

Install_param_instance_CA_count

The maximum number of Instance CA that can be created on an instance of the applet.

Install_param_internal_buffer_size

The size in bytes of the internal buffer described in Section 15.3.2.13. This value is set by the Device OEM according to Device OEM service policy, which is out of scope of this specification.

Install_param_max_allocatable_private_mailbox_size

The maximum size for private mailbox per endpoint. This value is set by the Device OEM according to its service policy, which is out of scope of this specification.

Install_param_max_allocatable_confidential_mailbox_size

The maximum size for confidential mailbox per endpoint. This value is set by the Device OEM according to its service policy, which is out of scope of this specification.

Applet_implementation_options

The applet_implementation_options is encoded as defined in Table 15-5.

If this field is not present, the applet implementation model is the framework-centric applet model, and the SE root of trust certificate chain model is Variant 1, as depicted in Figure 16-1, and the MANAGE UA command is not implemented or disabled.

Note: Tag $A6_h$, $A7_h$ and $A8_h$ have fixed values as defined in Appendix B.2, however they are included here to allow for potential evolutions in the future.

Table 15-5: Applet Implementation Options

b8	b7	b6	b5	b4	b3	b2	b1	Description
-	-	-	-	-	-	-	0	applet_implementation_model is framework-centric applet model
-	-	-	-	-	-	-	1	applet_implementation_model is SE-centric applet model
-	-	-	-	-	-	0	-	SE Root of Trust certificate chain models is Variant 1
-	-	-	-	-	-	1	-	SE Root of Trust certificate chain models is Variant 2
-	-	-	-	-	0	-	-	MANAGE UA command is not available
-	-	-	-	-	1	-	-	MANAGE UA command is available

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

b8	b7	b6	b5	b4	b3	b2	b1	Description
0	0	0	0	0	-	-	-	RFU (0)

Listing 15-1: INSTALL for INSTALL Processing

```
input : install_param *
output : none
begin
    check install_parameters
    atomic start
        allocate cod.internal_buffer using the size defined by install_param_internal_buffer_size
        If not enough memory
            return 6A84h
        reserve space using the number of endpoints defined by
            install_param_endpoint_count,
            install_param_instance_CA_count
        create dummy_endpoint
            generate key pair endpoint_PK, endpoint_SK according to Listing 15-40
            nvmwrite nvm. Dummy_endpoint.SK ← endpoint_SK
            nvmwrite nvm. Dummy_endpoint . Kpersistent , fill with random
            nvmwrite nvm. Dummy_endpoint . kpersistent_established ← true
            nvmwrite nvm.dummy_endpoint.terminated ← false
            nvmwrite nvm.dummy_endpoint.option_group_1 ← 0Fh (b4–b7 are set to zero)
            nvmwrite nvm.dummy_endpoint.option_group_2 ← 00h (b0,b1,b7 are set to zero)
        nvmwrite nvm.supported_protocol_versions_tlv ← TLV as per Table 15-12
    atomic commit
end
```

The following values define the Protocol Data Type A (Card Emulation Mode) tag (86_h) and the Protocol Data Type B (Card Emulation Mode) tag (87_h), which are provided in tag $A0_h$ (Contactless Protocol Parameters tag) of tag EF_h (System Specific Parameters tag) during the INSTALL for INSTALL command. Refer to GlobalPlatform Card Specification Amendment C [20] for formatting of these parameters.

Table 15-6: Values for Protocol Data Type A (tag '86')

Tag	Length (bytes)	Value Description	Value in Protocol Parameter Data (Tag 'A0')	Value in Protocol Parameter Mandatory Mask (Tag 'A1')
80_h	1	Unique Identifier LV structure	00_h	00_h
81_h	1	SAK	20_h	24_h
82_h	2	ATQA	0400_h	$39F0_h$
83_h	1	ATS LV structure	00_h	00_h
84_h	1	FWI, SFGI	72_h	FF_h
85_h	1	CID Support	00_h	00_h
86_h	3	DATA_RATE_MAX	000000_h	$FFFF00_h$

For Type A SFGI value, coexistence issues with legacy NFC services may appear. In such cases, increasing SFGI value could be allowed, with impact only on transaction performance. In any case, this value shall not exceed 8.

Note: LSB byte of the two-byte ATQA value is transmitted first.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-7: Values for Protocol Data Type B (tag 87_h)

Tag	Length (bytes)	Value Description	Value in Protocol Parameter Data (Tag 'A0')	Value in Protocol Parameter Mandatory Mask (Tag 'A1')
80_h	1	PUPI LV structure LV structure	00_h	00_h
81_h	1	AFI	00_h	00_h
82_h	4	ATQB	00000071_h	$000000F1_h$
83_h	1	Higher Layer Response in response to ATTRIB	00_h	00_h
84_h	3	Maximum data rate	000000_h	$FFFF00_h$

NOTE: For some areas, certain contactless services require fixed parameter values. In such cases, corresponding mask values as defined in Table 15-6 and Table 15-7 may be adapted to allow availability of these services concurrently with the DK service.

15.3.1.9 Notification of the Digital Key framework

Notification for APDU commands processed over the contactless interface

This notification mechanism may be used in case the APDU command is processed over the contactless interface. The DK applet uses the Connectivity Service interface defined in ETSI TS 102 705 [25] to request the NFC Controller to create a Host Controller Interface (HCI) event EVT_TRANSACTION (as defined in ETSI TS 102 622 [26]) as follows:

- The field AID (Tag 81_h) contains the AID of the DK applet.
- The field PARAMETERS(Tag 82_h) contains a list of notifications. The coding of each notification is as defined in Table 15-9. These notifications are appended to the list in the sequential order they are generated.

Using this notification mechanism is referred to as Option C in this specification.

Notification for APDU commands processed over the wired interface

This notification mechanism may be used in case the APDU command is processed over the wired interface. The DK framework may issue a GET NOTIFICATION command described in Section 15.3.2.28 to retrieve the notification(s) potentially generated by the DK applet during the processing of the previous APDU command.

Using this notification mechanism is known as Option D in this specification.

Table 15-8: Status Word - Command successfully executed

SW as defined in the listing in Section 15.3.2	SW sent by the Digital Key applet to the Digital Key framework	Behavior of Digital Key framework
SW 9000_h	SW 9000_h if no notification is pending	Forward SW 9000_h to the vehicle
	SW 9100_h if a notification is pending	Forward SW 9000_h to the vehicle

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

SW as defined in the listing in Section 15.3.2	SW sent by the Digital Key applet to the Digital Key framework	Behavior of Digital Key framework
		Issue a GET NOTIFICATION command to retrieve the notification(s)
All other SWs	SW as defined in the listing (notification pending or not)	Forward the received SW to the vehicle Issue a GET NOTIFICATION command to retrieve potentially generated notification(s)

Coding of the notifications

Table 15-9: Coding of the notification

Tag	Length (bytes)	Description	Field is
$7F60_h$	variable	Notification	mandatory
81_h	1	Notification code as defined in Table 15-11	mandatory
$A0_h$	variable	Notification context as defined in Table 15-10	mandatory
$8E_h$	variable	Additional data as defined in Table 15-11	conditional

The notification context is defined in Table 15-10.

The different values of the notification code corresponding to the different notification notify_* are defined in Table 15-11.

The presence and content of the additional data depends on the notification code (see Table 15-11).

Table 15-10: Notification Context

Tag	Length (bytes)	Description	Field is
$A0_h$	variable	Notification context	mandatory
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	conditional
$4D_h$	8	vehicle_identifier	conditional

The presence of the key_identifier and of the vehicle_identifier depends on the notification code (see Table 15-11).

Table 15-11: Value and Presence of the Different Fields in the HCI Notification Event

Notification	Code	Field key_identifier is	Field vehicle_identifier is	Field additional data is
notify_start_of_transaction	00_h	NA	NA	NA
notify_endpoint_not_found_fast	01_h	NA	mandatory	Length: 2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Notification	Code	Field key_identifier is	Field vehicle_identifier is	Field additional data is
				Value: P1, P2 of AUTH0 command
notify_endpoint_not_found_standard	02_h	NA	mandatory	Length: 2 Value: P1, P2 of AUTH0 command
notify_user_authentication_not_performed	03_h	NA	mandatory	Length: 2 Value: P1, P2 of AUTH0 command
notify_standard with nvm.endpoint.identifier	04_h	mandatory	mandatory	Length: 2 Value: P1, P2 of AUTH0 command
notify_fast_kpersistent_established with nvm.endpoint.identifier	05_h	mandatory	mandatory	Length: 2 Value: P1, P2 of AUTH0 command
notify_fast_kpersistent_not_established with nvm.endpoint.identifier	06_h	mandatory	mandatory	Length: 2 Value: P1, P2 of AUTH0 command
notify_vehicle_authentication_failed	07_h	optional	optional	NA
notify_vehicle_authentication_success	08_h	optional	optional	NA
notify_endpoint_not_found_presence	09_h	NA	mandatory	NA
notify_presence with nvm.endpoint.identifier	$0A_h$	mandatory	mandatory	NA
notify_mailbox_written	$0C_h$	optional	optional	NA
notify_deselect with reason	$0D_h$	NA	NA	Length: variable Value: Reason
notify_end_of_transaction with P1, P2 during a fast or standard transaction	$0E_h$	optional	optional	Length: 4 Value: P1, P2 of CONTROL FLOW command, P1, P2 of AUTH0 command
notify_end_of_transaction with P1, P2 outside a fast or standard transaction	$0E_h$	optional	optional	Length: 4

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Notification	Code	Field key_identifier is	Field vehicle_identifier is	Field additional data is
				Value: P1, P2 of CONTROL FLOW command, $FFFF_h$
notify_application_specific with P1, P2 during a fast or standard transaction	$0F_h$	optional	optional	Length: 4 Value: P1, P2 of CONTROL FLOW command, P1, P2 of AUTH0 command
notify_application_specific with P1, P2 outside a fast or standard transaction	$0F_h$	optional	optional	Length: 4 Value: P1, P2 of CONTROL FLOW command, $FFFF_h$
notify_message_in_exchange	10_h	optional	optional	Length: variable Value: Data present in the value part of tag $8E_h$ of the EXCHANGE command
notify_URSK_created	11_h	optional	NA	NA
RFU	12_h–FF_h	-	-	-

15.3.2 *Commands*

15.3.2.1 SELECT command

The AID of the Digital Key applet shall be $A000000809434343444B417631_h$.

This command selects the Digital Key applet instance to be used for the transaction. The instance AID parameter is defined during applet installation (see the INSTALL for INSTALL command in [20]).

The SE should respond with the status word 6985_h if the Digital Key applet is temporarily unavailable.

command: CLA1 A4 04 00 Lc [instance AID] 00
response: [Table 15-12] **90 00**

The CLA1 is as defined in Table 15-3.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-12: SELECT Response Fields

Tag	Length (bytes)	Description	Field is	Domain Version
$5C_h$	variable	A list of supported Digital Key applet transaction (V-D-TX) protocol versions (ver.high | ver.low) ordered from highest to lowest. Each version number is concatenated and encoded on 2 bytes in big-endian notation.	mandatory	V-D-TX

Listing 15-2: SELECT Processing

```
input : none
output : supported_protocol_versions_tlv
begin
  cod.transaction_state ← select_done
  cod.atomic_session ← stopped
  if contactless interface
     sent to digital key framework notify_start_of_transaction
  return nvm .supported_protocol_versions_tlv
end
```

15.3.2.2 CREATE CA command

This command creates an onboard signing authority called Instance CA used for endpoint issuance. The generation and provisioning of this entity is out of scope of this specification. The certificate containing the Instance CA public key shall comply with Listing 15-16. The INS value for CREATE CA command shall be 38_h.

15.3.2.3 DELETE CA command

This command deletes an Instance CA. The method of deletion of this entity is out of scope of this specification. The INS value for DELETE CA command shall be $3A_h$.

15.3.2.4 CREATE ENDPOINT command

The command creates a communication endpoint and provides the corresponding endpoint certificate. A communication endpoint distributes signatures and mailbox content over a secure channel to authenticated parties only.

The endpoint configuration fields described in Table 15-13 may be provided either in the command payload or in the internal buffer using the WRITE BUFFER command described in Section 15.3.2.14. The endpoint creation certificate described in Listing 15-5 is always provided in the internal buffer and is accessible using the READ BUFFER command described in Section 15.3.2.13.

command: CLA2 70 00 00 Lc ([Table 15-13**] [**Table 15-16**] 00)**
response: [response_length] 9000

The CLA2 is as defined in Table 15-3.

Table 15-13: Endpoint Configuration

Tag	Length (bytes)	Description	Field is	Domain Version
$7F27_h$	variable	Endpoint configuration	mandatory	V-OD-FW (Owner Pairing) OD-FD-KS (Key Sharing)
$4D_h$	8	vehicle_identifier	mandatory	V-OD-FW

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
$5F20_h$	1–30	endpoint_identifier, DER encoded PrintableString as per RFC 5280 [3] (without PrintableString tag and length)	mandatory	V-OD-FW
42_h	1–30	instance_CA_identifier, DER encoded PrintableString as per RFC 5280 [3] (without PrintableString tag and length)	mandatory	V-OD-FW
46_h	1	option_group_1 (on bits 0–7): **bit0**: enable(1)/disable(0) Standard transaction allowed on contactless interface **bit1**: enable(1)/disable(0) Fast transaction allowed on contactless interface **bit2**: enable(1)/disable(0) Standard transaction allowed on wired interface **bit3**: enable(1)/disable(0) Fast transaction allowed on wired interface **bit4**: enable(1)/disable(0) AUTHORIZE ENDPOINT command allowed **bit5**: enable(1)/disable(0) Authorizing endpoints which have the AUTHORIZE ENDPOINT command allowed **bit6**: enable(1)/disable(0) EXCHANGE command allowed on wired interface **bit7**: enable(1)/disable(0) EXCHANGE command allowed after AUTH0	mandatory	V-OD-FW
47_h	1	option_group_2 (on bits 0–7): **bit0**: enable(1)/disable(0) SIGN command allowed **bit1**: enable(1)/disable(0) Exported data from confidential mailbox are replaced with random values **bit2**: RFU (shall be set to 0) **bit3**: RFU (shall be set to 0) **bit4**: RFU (shall be set to 0 **bit5**: RFU (shall be set to 0 **bit6**: RFU (shall be set to 0 **bit7**: enable(1)/disable(0) SET CONFIDENTIAL DATA command allowed	mandatory	V-OD-FW
$5C_h$	2	protocol_version	mandatory	V-OD-FW
$5B_h$	65	vehicle_PK prepended by 04_h	mandatory	V-OD-FW
51_h	15 or 13	not_before, DER encoded GeneralizedTime (15 bytes in length) or UTCTime (13 bytes in length) as per RFC 5280 [2]	mandatory	V-OD-FW OD-FD-KS

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
52_h	15 or 13	not_after, DER encoded GeneralizedTime (15 bytes in length) or UTCTime (13 bytes in length) as per RFC 5280 [3]	mandatory	V-OD-FW OD-FD-KS
49_h	variable	authorized_PK[], list of up to 5 public keys prepended by 04_h of 65 bytes length	optional	V-OD-FW
$4A_h$	2	confidential_mailbox_size	optional	V-OD-FW
$4B_h$	2	private_mailbox_size	optional	V-OD-FW
$4E_h$	1–8	key_slot	optional	D-VS
57_h	4	counter_limit, values in range 00000001_h–$FFFFFFFF_h$	deprecated *	N/A

* This field has been deprecated and shall no longer be used.

vehicle_identifier: Field provided by the endpoint creator; this field is used by the applet for fast endpoint lookup during transaction.

endpoint_identifier: Arbitrary field provided by the endpoint creator, describing the subject of the certificate. The endpoint_identifier field typically identifies the entity associated with the public key present in the certificate.

instance_CA_identifier: This field allows for selection of the instance CA used by the SE to sign the endpoint creation certificate. This value is present in the field “subject CommonName” of the corresponding instance certificate.

option_group_1 and option_group_2: For Owner Pairing (Owner Pairing Phase 2), the vehicle shall set these two fields according to Table 15-14.

Table 15-14: Setting of Tag 46_h and Tag 47_h by vehicle for various wireless capability combinations.

Tag	Value	Description
Vehicle supporting WCC1		
46_h	bit0 set to 1	Standard transaction allowed on contactless interface
	bit1 set to 1	Fast transaction allowed on contactless interface
	bit2 set to 0	Standard transaction not allowed on wired interface
	bit3 set to 0	Fast transaction not allowed on wired interface
	bit4 set to 1	AUTHORIZE ENDPOINT command allowed
	bit5 set to 0	Authorizing endpoints which have the AUTHORIZE ENDPOINT command not allowed
	bit6 set to 0	EXCHANGE command not allowed on wired interface
	bit7 set to 0/1	EXCHANGE command allowed (1) or not allowed (0) after AUTH0
Vehicle supporting WCC2 or WCC3		
46_h	bit0 set to 1	Standard transaction allowed on contactless interface
	bit1 set to 1	Fast transaction allowed on contactless interface
	bit2 set to 1	Standard transaction allowed on wired interface

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Value	Description
	bit3 set to 0	Fast transaction not allowed on wired interface
	bit4 set to 1	AUTHORIZE ENDPOINT command allowed
	bit5 set to 0	Authorizing endpoints which have the AUTHORIZE ENDPOINT command not allowed
	bit6 set to 1	EXCHANGE command allowed on wired interface
	bit7 set to 0/1	EXCHANGE command allowed (1) or not allowed (0) after AUTH0
47_h	bit0 set to 1	SIGN command allowed
	bit1 set to 1	Exported data from confidential mailbox are replaced with random values
	bit2 set to 0	RFU
	bit3 set to 0	RFU
	bit4 set to 0	RFU
	bit5 set to 0	RFU
	bit6 set to 0	RFU
	bit7 set to 0/1	SET CONFIDENTIAL DATA command not allowed (0) if immobilizer tokens are not used or allowed (1) if immobilizer tokens are used

The bit7 of tag 46_h, shall not be set to 1 if sensitive data such as immobilizer tokens are exchanged using the EXCHANGE command.

For friend sharing (Key Sharing flow step 1 and 2), the owner device shall set these two fields which is within Key Creation Request according to Table 15-15.

Table 15-15: Setting of Tag 46_h and Tag 47_h by the owner device for the key sharing.

Tag	Value	Description
46_h	bit0 to bit3	As defined in the endpoint configuration (owner device)
	bit4 set to 0	AUTHORIZE ENDPOINT not command allowed
	bit5 to bit7	As defined in the endpoint configuration (owner device)
47_h	bit0 to bit7	As defined in the endpoint configuration (owner device)

protocol_version: Digital Key applet protocol transaction (V-D-TX) version assigned at the time of the endpoint creation. This field is informative and shall not be used by the applet to enforce usage of a specific protocol version. Thus, this shall not prevent the endpoint to be used with other versions of the Digital Key applet protocol version that may be supported by the vehicle and the Digital Key applet later.

vehicle_PK: The public key of the vehicle.

Not_before: The issued certificate start of validity date. The validity date is not required to be verified by the applet.

Not_after: The issued certificate end of validity date. The validity date is not required to be verified by the applet.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Authorized_PK[]: The public keys used by the device to verify other SE's endpoint creation certificates.

Confidential_mailbox_size: If field not present, or if present with confidential_mailbox_size equal to 0000_h, confidential mailbox is not available.

Private_mailbox_size: If field not present, or if present with private_mailbox_size equal to 0000_h, private mailbox is not available.

Key_slot: A field in the endpoint configuration that is populated in the friend endpoint configuration during step 1 of the key sharing (see Section 11.8.1) if slot identifiers are retrieved from the vehicle via the owner device. The value of this field is set to one of the slot identifiers from the SlotIdentLst field. The key_slot can be manually set when a fast search among a large number of public keys is required on the vehicle side. The fast search can be facilitated by attributing key slot numbers instead of relying on the automatic hash-based naming during endpoint creation. If key_slot field is not provided, an automatic slot number is generated as described in Listing 15-6, since the key_slot is mandatory in the AUTH1-Response. In case of slot identifiers provided online, the owner device during key sharing, assigns an automatically generated random slot identifier. The slot identifier is used to derive the Kble_oob, see Sections 19.5.1 and 19.5.8.

Note: A device generated key_slot does not safeguard against attestation replay attacks during key sharing.

This field does not need to be unique among endpoints created on an instance.

This field can be updated after key creation with the SETUP_ENDPOINT command. The value "initial_key_slot" in the endpoint certificate will always remain the one initially assigned during endpoint creation.

Counter_limit: This field is deprecated and shall not be used.

If the applet_implementation_model field is included (i.e., SE-centric applet model) in the INSTALL for INSTALL command, the following additional tags shall appear after the tag $7F27_h$.

Table 15-16: Endpoint verification information

Tag	Length (bytes)	Description	Field is
$7F28_h$	variable	Vehicle OEM CA Certificate (signed by Device OEM)	conditional
$7F4C_h$	variable	DER-encoded X.509 Intermediate Certificate	optional
$7F4B_h$	variable	DER-encoded X.509 Vehicle Public Key Certificate	conditional

Vehicle OEM CA Certificate (signed by Device OEM): this field is provided by the framework and is verified by the SE. The SE also uses this data to verify the Vehicle Public Key Certificate.

It is assumed that Device OEM CA root certificate is stored in the SE for this operation.

Vehicle Public Key Certificate: This field is provided by the vehicle during owner pairing. This field is not present during Digital Key sharing. The SE shall verify the Vehicle Public Key Certificate if this field is present.

Listing 15-3: Endpoint Certificate Extension Schema

```
Schema DEFINITIONS EXPLICIT TAGS
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
BEGIN
EndpointCertificateExtensionSchema ::= SEQUENCE
{
extension_version                INTEGER (1..255),
vehicle_identifier               OCTET STRING (SIZE (8)),
option_group_1                   OCTET STRING (SIZE (1)),
option_group_2                   OCTET STRING (SIZE (1)),
protocol_version                 OCTET STRING (SIZE (2)),
vehicle_PK                       PublicKey,
initial_key_slot                 OCTET STRING (SIZE  (1..8)),
authorized_PK_list               SEQUENCE(SIZE(1..5)) OF PublicKey OPTIONAL,
confidential_mailbox_size        [0] INTEGER (1..65535) OPTIONAL,
private_mailbox_size             [1] INTEGER (1..65535) OPTIONAL,
}

PublicKey ::= SEQUENCE
{
algorithm       AlgorithmIdentifier,
public_key      BIT STRING
}

AlgorithmIdentifier  ::=  SEQUENCE
{
 algorithm       OBJECT IDENTIFIER,
 parameters     ANY DEFINED BY algorithm OPTIONAL
}
END
```

Listing 15-4: Endpoint Certificate Extension Data

```
endpoint-cert-extension-data EndpointCertificateExtensionSchema ::=
{
extension_version       1, --value indicating the extension version, this value is meant to be incremented if the extension fields are changed in
next applet versions.
vehicle_identifier        '...'H, --value as per endpoint configuration
option_group_1            '...'H, --value as per endpoint configuration
option_group_2            '...'H, --value as per endpoint configuration
protocol_version          '...'H, --value as per endpoint configuration
vehicle_PK
{
algorithm
{
algorithm {1 2 840 10045 3 1 7} --OID for prime256v1(ANSI X9.62 named elliptic curve)
},
public_key '04...'H --vehicle_PK pre-pended with 04h
},
initial_key_slot      '...'H, --value as per endpoint configuration
authorized_PK_list
{
{
algorithm
{
algorithm {1 2 840 10045 3 1 7} --OID for prime256v1(ANSI X9.62 named elliptic curve)
},
public_key '04...'H --authorized_PK
},
... --other authorized_PK
},
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
confidential_mailbox_size  ..., --value as per endpoint configuration
private_mailbox_size       ..., --value as per endpoint configuration
}
```

Listing 15-5: Endpoint Certificate Data

```
endpoint-cert-data Certificate ::=
{
  tbsCertificate
  {
    version v3, --shall be v3
    serialNumber ..., --a random integer chosen by the certificate issuer
    signature
    {
      algorithm {1 2 840 10045 4 3 2} --OID for ecdsaWithSHA256 (ANSI X9.62 ECDSA algorithm with SHA256)--
    },
    issuer rdnSequence:
    {
      {
        {
          type {2 5 4 3}, --OID for CommonName
          value "..." --the instance_CA_identifier as per endpoint configuration
                      -- (PrintableString or UTF8String recommended)
        }
      }
    },
    validity
    {
      notBefore Time: "...", --value not_before as per endpoint configuration
      notAfter Time: "..." --value not_after as per endpoint configuration
    },
    subject rdnSequence:
    {
      {
        {
          type {2 5 4 3}, --OID for CommonName
          value "..." --the endpoint_identifier as per endpoint configuration, shall use PrintableString or UTF8String format

        }
      }
    },
    subjectPublicKeyInfo
    {
      algorithm
      {
        algorithm {1 2 840 10045 2 1}, --OID for ecPublicKey (ANSI X9.62 public key type)
        parameters {1 2 840 10045 3 1 7} --OID for prime256v1 (ANSI X9.62 named elliptic curve)
      },
      subjectPublicKey '04...'H --the public key pre-pended with 04 to indicate uncompressed format
    },
    extensions
    {
      {
        extnID     {install_param_oid_endpoint}, --OID for Endpoint certificate
        critical   TRUE,
        extnValue  '...'H --DER encoding for endpoint-cert-extension-data as per Listing 15-4
      },
      {
        extnID     {2 5 29 15}, --OID for KeyUsage standard extension
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        critical    TRUE,
        extnValue   '03020780'H --DER encoding for KeyUsage, digitalSignature
      },
      {
        extnID      {2 5 29 19}, --OID for BasicConstraints standard extension
        critical    TRUE,
        extnValue   '3000'H --DER encoding for cA=FALSE
      },
      {
        extnID      {2 5 29 35}, --OID for AuthorityKeyIdentifier standard extension
        critical    FALSE,
        extnValue   '...''H – DER encoding of an AuthorityKeyIdentifier sequence as defined in [3], containing only
                              – a KeyIdentifier element. The KeyIdentifier is an OCTET STRING containing the
                              – 160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey
                              – from the issuer certificate (excluding the tag, length, and number of unused bits)
      },
      …
    }
  },
  signatureAlgorithm
  {
    algorithm {1 2 840 10045 4 3 2}
  },
  signatureValue '...'H --the certificate signature computed as per RFC 5280 [3].
}
```

Listing 15-6: CREATE ENDPOINT Processing

```
input: Table 15-13, Table 15-16
output: response_length, certificate
begin
  if no payload present in command
    if Table 15-13 data fields not present in cod.internal_buffer
      return 6400h
  if any mandatory field is not present
    return 6400h
  if endpoint_identifier already exists in any of the nvm.endpoint[].identifier
    return 6400h
  if protocol_version from input table is not supported
    return 6400h
  if instance CA referenced by instance_CA_identifier field does not exist
    return 6400h
  if vehicle_identifier is already used by any of the existing of the nvm.endpoint[]
    return 6400h
  if SE-centric applet model
    if Table 15-16 data fields not present in cod.internal_buffer
      return 6400h
    if Table 15-16 data fields are not verified
      return 6400h
  generate key pair endpoint_PK, endpoint_SK according to Listing 15-40
  if key_slot field not provided
    generate key_slot as per Listing 15-41 using subjectPublicKey of the endpoint to be created as input (excluding the tag, length, and number of unused bits)
  generate certificate using Listing 15-5 as per RFC 5280 [3]
  atomic start
    nvmwrite nvm.endpoint.<...> configuration as per Table 15-13 and certificate parts (including certificate signature) needed to re-generate the certificate when retrieved with VIEW command
    nvmwrite nvm.endpoint.identifier ← endpoint_identifier
    nvmwrite nvm.endpoint.key_slot ← key_slot
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    nvmwrite nvm.endpoint.initial_key_slot ← key_slot
30      nvmwrite nvm.endpoint.PK ← endpoint_PK
31      nvmwrite nvm.endpoint.SK ← endpoint_SK
32      nvmwrite nvm.endpoint.Kpersistent, fill with randoms
33      nvmwrite nvm.endpoint.kpersistent_established ← false
34      nvmwrite nvm.endpoint.terminated ← false
35      nvmwrite nvm.endpoint.transaction_code_list_cless ← []
36      nvmwrite nvm.endpoint.transaction_code_list_wired ← []
37    atomic commit

38    store certificate in cod.internal_buffer
39    response_length ← length written in cod.internal_buffer
40    return response_length (2 bytes big-endian)
41  end
```

15.3.2.5 TERMINATE ENDPOINT command

This command sets the endpoint in terminated state and returns a termination attestation. If the endpoint is already in terminated state, the previously computed attestation is returned. The terminated state of an endpoint cannot be reverted; the only possible action is to delete the endpoint.

The following command formats are allowed:

- If no secure channel is to be established between the Digital Key framework and the Digital Key applet, then either command format 1 or command format 2 may be used and implemented by the applet.
- If a secure channel is to be established between the Digital Key framework and the Digital Key applet (option A), command format 2 shall be used and shall be implemented by the applet.

Command format 1:

command: CLA2 72 00 00 Lc [Table 15-17**] 00**
response: [Table 15-19**] 9000**

The CLA2 is as defined in Table 15-3.

Command format 2:

command: CLA2 72 00 00 Lc [Table 15-17**] 00**
response: [response_length] 9000

The CLA2 is as defined in Table 15-3.

Table 15-17: Endpoint Termination Request

Tag	Length (bytes)	Description	Field is	Domain Version
$7F2E_h$	variable	Endpoint Termination Request	mandatory	N/A
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW
91_h	16	termination_requester_nonce, a random value chosen by the termination requester	mandatory	N/A

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
58_h	1–40	arbitrary_data, arbitrary data field chosen by the termination requester	optional	N/A

Table 15-18: Endpoint Termination Attestation Data Fields

Tag	Length (bytes)	Description	Field is	Domain Version
41_h	1	version = 01_h, version of Endpoint Termination Attestation	mandatory	Informative only. N/A
92_h	8	random	mandatory	D-VS
$5F20_h$	1–30	endpoint_identifier, identifier of the terminated endpoint	mandatory	V-OD-FW
$5F49_h$	65	the public key of the terminated endpoint	mandatory	V-OD-FW
91_h	16	termination_requester_nonce	mandatory	D-VS
58_h	1–40	arbitrary_data	optional	D-VS
93_h	4	usage = $D6854CB9_h$	mandatory	D-VS

Table 15-19: Endpoint Termination Attestation

Tag	Length (bytes)	Description	Field is	Domain Version
$7F29_h$	variable	Endpoint Termination Attestation	mandatory	D-VS
Content of Table 15-18			mandatory	
$9E_h$	64	signature with the private key of the terminated endpoint over fields from Table 15-18	mandatory	D-VS

Listing 15-7: TERMINATE ENDPOINT Processing for Command Format 1

```
input: key_identifier
output: termination_attestation
begin
  if key_identifier does not match with any of the existing endpoints
    return 6400h
  if nvm.endpoint.terminated = true
   return nvm.endpoint.termination_attestation
  if UWB present
     delete URSK associated to the key_identifier
  nvm.endpoint.terminated ← true
  generate 8 bytes random as per Listing 15-39
  generate signature over Table 15-18 using key selected from nvm.endpoint.SK according to Listing 15-42
  nvmwrite nvm.endpoint.termination_attestation ← data structure as per Table 15-19
  return nvm.endpoint.termination_attestation
end
```

Listing 15-8: TERMINATE ENDPOINT Processing for Command Format 2

```
input: key_identifier
output: response length, termination_attestation
Begin
  if key_identifier does not match with any of the existing endpoints
    return 6400h
  if nvm.endpoint.terminated = true
    store nvm.endpoint.termination_attestation in cod.internal_buffer
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    response_length ← length written in cod.internal_buffer
    return response_length (2 bytes big-endian)
  if UWB present
    delete URSK associated to the key_identifier
  nvm.endpoint.terminated ← true
  generate 8 bytes random as per Listing 15-39
  generate signature over Table 15-18 using key selected from nvm.endpoint.SK according to Listing 15-42
  nvmwrite nvm.endpoint.termination_attestation ← data structure as per Table 15-19
  store nvm.endpoint.termination_attestation in cod.internal_buffer
  response_length ← length written in cod.internal_buffer
  return response_length (2 bytes big-endian)
End
```

15.3.2.6 DELETE ENDPOINT command

This command will delete an endpoint and release the associated memory.

command: CLA2 74 00 00 Lc [Table 15-20**]**
response: 9000

The CLA2 is as defined in Table 15-3.

Table 15-20: Deletion Request

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW

Listing 15-9: DELETE ENDPOINT Processing

```
input: key_identifier
output: none
begin
  if key_identifier does not match with any of the existing endpoints
    return 6400h
  if UWB present
    delete URSK associated to the key_identifier
  atomic start
    nvmdelete nvm.endpoint
  atomic commit
end
```

15.3.2.7 AUTHORIZE ENDPOINT command

This command signs an endpoint public key (issued by a receiver device) and an arbitrary additional data field using a local endpoint private key (sender device). Optionally, the command can also extract and encrypt a portion of the confidential mailbox to be inserted in the authorized recipient endpoint.

Input and output buffers are transferred using the internal buffer commands READ BUFFER and WRITE BUFFER to avoid APDU length limitations.

The supported verification chains are described in Section 16.6. Chain verification starts from one of the authorized public keys stored in the endpoint and continues from top to bottom with the items in Table 15-22.

This command requires user authentication to be performed on device.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Note: The protocol_version value present in the receiver's endpoint certificate shall not be checked during processing of this command.

command: CLA2 32 00 00 Lc [Table 15-21**] 00**
response: [response_length] 90 00

Table 15-21: AUTHORIZE ENDPOINT Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW
$4A_h$	4	offsetmsb || offsetlsb || lengthmsb || lengthlsb, portion of the confidential mailbox to be exported	optional	V-OD-FW
58_h	variable	arbitrary_data	optional	N/A

Listing 15-10: External CA Certificate Extension Schema

```
ExternalCACertificateExtensionSchema ::= SEQUENCE
{
   extension_version      INTEGER (1..255)
}
```

Listing 15-11: External CA Certificate Extension Data

```
external-ca-cert-extension-data ExternalCACertificateExtensionSchema ::=
{
  extension_version 1      --value shall be 1
}
```

Listing 15-12: External CA Certificate Basic Constraints Extension Data

```
external-ca-cert-basic-constraints BasicConstraints ::=
{
   cA              TRUE,
   pathLenConstraint ...     –value shall be 1 if an Instance CA is conditionally or always present in the chain
                              --value shall be 0 if Instance CA is never present in the chain
}
```

Listing 15-13: External CA Certificate Data

```
external-ca-cert-data Certificate ::=
 {
  tbsCertificate
  {
   version v3, --shall be v3
   serialNumber ..., --a random integer chosen by the certificate issuer
   signature
   {
    algorithm {1 2 840 10045 4 3 2} --OID for ecdsaWithSHA256 (ANSI X9.62 ECDSA algorithm with SHA256)
   },
   issuer rdnSequence:
   {
    {
     {
      type {2 5 4 3}, --OID for CommonName
      value "..." --shall match the subject of the issuer certificate
                  --(shall use PrintableString or UTF8String format)*
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
      }
    }
  },
  validity
  {
    notBefore Time: "...", -- shall use UTCTime or GeneralizedTime as defined in [3]
    notAfter Time: "..." – shall use UTCTime or GeneralizedTime as defined in [3]
  },
  subject rdnSequence:
  {
    {
      {
        type {2 5 4 3}, --OID for CommonName
        value "..." --contains the subject of the certificate
                    --(shall use PrintableString or UTF8String format)*
      }
    }
  },
  subjectPublicKeyInfo
  {
    algorithm
    {
      algorithm {1 2 840 10045 2 1}, --OID for ecPublicKey (ANSI X9.62 public key type)
      parameters {1 2 840 10045 3 1 7} --OID for prime256v1(ANSI X9.62 named elliptic curve)
    },
    subjectPublicKey '04...'H --the public key pre-pended with 04 to indicate uncompressed format
  },
  extensions
  {
    {
      extnID      {1.3.6.1.4.1.41577.5.2}, --OID for external CA Certificate (see Appendix B.2.2)
      critical    TRUE,
      extnValue   '…'H –DER encoding for ExternalCACertificateExtensionSchema extension as per Listing 15-11
    },
    {
      extnID      {2 5 29 15}, --KeyUsage standard extension
      critical    TRUE,
      extnValue   '03020204'H --DER encoding for KeyUsage, CertSign only
    },
    {
      extnID      {2 5 29 19}, --BasicConstraints standard extension
      critical    TRUE,
      extnValue   '...'H --DER encoding for BasicConstraints extension as per Listing 15-12
    },
    {
      extnID      {2 5 29 35}, --OID for AuthorityKeyIdentifier standard extension
      critical    FALSE,
      extnValue   '..."H – DER encoding of an AuthorityKeyIdentifier sequence as defined in [3], containing only
                          – a KeyIdentifier element. The KeyIdentifier is an OCTET STRING containing the
                          – 160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey
                          – from the issuer certificate (excluding the tag, length, and number of unused bits)
    },
    {
      extnID      {2 5 29 14}, --OID for SubjectKeyIdentifier standard extension
      critical    FALSE,
      extnValue   '...'H --160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey
                    --(excluding the tag, length, and number of unused bits)
    },
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    {
      extnID      {...}, --Optional extensions added by the issuer, content not verified by the applet
      critical    FALSE,
      extnValue   '...'H --DER encoding of the extension
    },
    ...
   }
  },
  signatureAlgorithm
  {
    algorithm {1 2 840 10045 4 3 2} --OID for ecdsaWithSHA256 (ANSI X9.62 ECDSA algorithm with SHA256)
  },
  signatureValue '...'H --the certificate signature by the issuer computed as per RFC 5280 [3]
  --ECDSA signature
}
```

* The choice of format between PrintableString and UTF8String in the External CA certificate impacts the format of fields in Vehicle and Device OEM certificates as outlined in Section A.4.7 of Appendix A and Section B.2.5 of Appendix B

Listing 15-14: Instance CA Certificate Extension Schema

```
InstanceCACertificateExtensionSchema ::= SEQUENCE
{
    extension_version         INTEGER (1..255),
    applet_version            OCTET STRING (SIZE 4),
    platform_information      OCTET STRING (SIZE (1..20)) OPTIONAL
}
```

Listing 15-15: Instance CA Certificate Extension Data

```
instance-ca-cert-extension-data InstanceCACertificateExtensionSchema ::=
{
  extension_version          1, --value shall be 1
  applet_version             '...'H --proprietary applet version information
  platform_information       '...'H --optional proprietary platform information
                                    --field typically set by Device OEM Server to
                                    --identify the Secure Element platform
}
```

Note: applet_version and platform_information are immutable values assigned at the time of the Instance CA certificate is issued. It might not reflect the current applet_version or platform_information (e.g., in case of applet or Secure Element platform upgrade).

Listing 15-16: Instance CA Certificate Data

```
instance-ca-cert-data Certificate ::=
{
  tbsCertificate
  {
    version v3, --shall be v3
    serialNumber ..., --a random integer chosen by the certificate issuer
    signature
    {
      algorithm {1 2 840 10045 4 3 2} --OID for ecdsaWithSHA256 (ANSI X9.62 ECDSA algorithm with SHA256)
    },
    issuer rdnSequence:
    {
      {
        {
          type {2 5 4 3}, --OID for CommonName
          value "..." --shall match the subject of the issuer certificate
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
                    --(shall use PrintableString or UTF8String format)*
    }
   }
  },
  validity
  {
   notBefore Time: "...", --shall use UTCTime or GeneralizedTime as defined in [3]
   notAfter Time: "..." -- shall use UTCTime or GeneralizedTime as defined in [3]
  },
  subject rdnSequence:
  {
   {
    {
     type {2 5 4 3}, --OID for CommonName
     value "..." --contains the subject of the certificate (instance_CA_identifier)
                    --(shall use PrintableString or UTF8String format)
    }
   }
  },
  subjectPublicKeyInfo
  {
   algorithm
   {
    algorithm {1 2 840 10045 2 1}, --OID for ecPublicKey (ANSI X9.62 public key type)
    parameters {1 2 840 10045 3 1 7} --OID for prime256v1(ANSI X9.62 named elliptic curve)
   },
   subjectPublicKey '04...'H
   --the public key pre-pended with 04 to indicate uncompressed format
  },
  extensions
  {
   {
    extnID     {install_param_oid_instance_ca}, --OID for Instance CA extension
    critical   TRUE,
    extnValue  '...'H --DER encoding for instance-ca-cert-extension-data as per Listing 15-15
   },
   {
    extnID     {2 5 29 15}, --KeyUsage standard extension
    critical   TRUE,
    extnValue  '03020204'H --DER encoding for KeyUsage, CertSign only
   },
   {
    extnID     {2 5 29 19}, --BasicConstraints standard extension
    critical   TRUE,
    extnValue  '30060101FF020100'H -- DER encoding for BasicConstraints cA=TRUE, pathLenConstraint=0
   },
   {
    extnID     {2 5 29 35}, --OID for AuthorityKeyIdentifier standard extension
    critical   FALSE,
    extnValue  '..."H – DER encoding of an AuthorityKeyIdentifier sequence as defined in [3], containing only
                    – a KeyIdentifier element. The KeyIdentifier is an OCTET STRING containing the
                    – 160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey
                  --from the issuer certificate (excluding the tag, length, and number of unused bits)
   },
   {
    extnID     {2 5 29 14}, --OID for SubjectKeyIdentifier standard extension
    critical   FALSE,
    extnValue  '...'H --160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
                    --(excluding the tag, length, and number of unused bits)
    },
    {
      extnID     {...}, --Optional extensions added by the issuer, content not verified by the applet
      critical   FALSE,
      extnValue  '...'H --DER encoding of the extension
    },
    ...
  }
 },
 signatureAlgorithm
 {
  algorithm {1 2 840 10045 4 3 2}
 },
 signatureValue '...'H --the certificate signature by the issuer computed as per RFC 5280 [3]
}
```

* The selection of format between PrintableString and UTF8String is outlined in Section B.2.4 of Appendix B

Table 15-22: AUTHORIZE ENDPOINT Internal Buffer Content Before Processing (offset 0)

Tag	Length (bytes)	Description	Field is	Domain Version
$7F20_h$	variable	External CA certificate as per Listing 15-13 from the receiver endpoint	optional	V-OD-FW
$7F22_h$	variable	Instance CA Certificate as per Listing 15-16 from the receiver endpoint	optional	V-OD-FW
$7F24_h$	variable	Endpoint Creation certificate as per Listing 15-5 from the receiver endpoint	mandatory	V-OD-FW
Encryption Key attestation as per Table 15-47 from the receiver endpoint			optional	D-VS

Table 15-23: AUTHORIZE ENDPOINT Attestation Data Fields

Tag	Length (bytes)	Description	Field is	Domain Version
41_h	1	version = 01_h, the version of the data structure	mandatory	Informative
92_h	8	random	mandatory	OD-FD-KS
$5A_h$	65	public key from the receiver endpoint creation certificate present in internal buffer	mandatory	OD-FD-KS
58_h	variable	arbitrary_data, field present in command payload	optional	N/A
93_h	4	usage = $91712C44_h$	mandatory	OD-FD-KS

Table 15-24: AUTHORIZE ENDPOINT Internal Buffer Content After Processing (offset 0)

Tag	Length (bytes)	Description	Field is	Domain Version
$9E_h$	64	sender endpoint signature over fields from Table 15-23	mandatory	OD-FD-KS
92_h	8	random	mandatory	OD-FD-KS
97_h	65	encsender_ePK	optional	OD-FD-KS
$4A_h$	variable	encrypted_mailbox || mac	optional	OD-FD-KS

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Listing 15-17: AUTHORIZE ENDPOINT Processing

```
input: internal_buffer, key_identifier(, mailbox_offset, mailbox_length, arbitrary_data)
output: response_length, signature(, encrypted_mailbox, mac, encsender_ePK)
begin
 if key_identifier does not match with any existing endpoint or target endpoint is terminated
  return 6A88h
 if AUTHORIZE ENDPOINT is not allowed on selected sender endpoint as per Table 15-13
  return 6900h
 if ua_performed != true  -- ua_performed true if user authentication performed, false otherwise
  return 6985h
 if cod.internal_buffer is not formatted as described in Table 15-22
  return 6E09h
 if a certificate from chain does not comply with X.509 ASN.1 as per Listing A-4
  return 6400h
 if a certificate from chain cannot be verified as per verifier rules defined in Appendix A.4.7 and data defined in Listing 15-5,
       Listing 15-13, and Listing 15-16
  return 6400h
 if Endpoint certificate configuration has different vehicle_identifier than sender endpoint
  return 6E10h
 if Endpoint certificate any bit from option_group_1 is set while the same bit is cleared on sender endpoint
  return 6E11h
 if Endpoint certificate option_group_1 bit4 is set while bit5 is cleared on sender endpoint
  return 6E12h
 if Endpoint certificate option_group_2 bit0 or bit2 or bit3 or bit5 or bit7 is set while the same bit is
     cleared on sender endpoint
  return 6E13h
 if Endpoint certificate option_group_2 bit1 or bit4 or bit6 is cleared while the same bit is set
     on sender endpoint
  return 6E14h
 if Endpoint certificate configuration has different vehicle_PK than nvm.endpoint.vehicle_PK
  return 6E15h
 if Endpoint certificate contains authorized_PKs that are not part of nvm.endpoint.authorized_PK configuration
  return 6E16h
 if External CA certificate is present in cod.internal_buffer and cannot be verified
    with one of nvm.endpoint.authorized_PK public keys
  return 6E19h
 if Instance CA certificate is present in cod.internal_buffer and cannot be verified
    with one of nvm.endpoint.authorized_PK public keys or with the public key of the External CA certificate
  return 6E20h
 if Endpoint certificate stored in cod.internal_buffer cannot be verified
    with the public key of the Instance CA certificate
    or with one of nvm.endpoint.authorized_PK public keys
    or with the public key of the External CA certificate
  return 6E21h
 generate 8 bytes random as per Listing 15-39 to be added in data_fields according to Table 15-23
 generate attestation signature over fields from Table 15-23 according to Listing 15-42 using nvm.endpoint.SK
 if mailbox_offset and mailbox_length present in command payload
  if Encryption key certificate not present in cod.internal_buffer
   return 6E23h
  if Encryption Key certificate present in cod.internal_buffer cannot be verified with
   the public key of the Endpoint certificate present in cod.internal_buffer
   return 6E24h
  generate ephemeral key pair encsender_ePK, encsender_eSK according to Listing 15-40
  generate KEseed according to Listing 15-44 using encsender_eSK, encreceiver_ePK
  receiver_PK ⟵ public key from receiver endpoint extracted from Endpoint certification present in internal buffer
  derivation ⟵08h
  KEseed_half ⟵ 16 most significant bytes of KEseed
  generate KEenc according to Listing 15-48 using KEseed_half, encsender_ePK, encreceiver_ePK,
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        derivation, receiver_PK
    derivation ⟵ 12h
    KEseed_half ⟵ 16 least significant bytes of KEseed
    generate KEmac according to Listing 15-48 using KEseed_half, encsender_ePK, encreceiver_ePK,
        derivation, receiver_PK
    encrypt and mac the selected confidential mailbox area according to Listing 15-49
    if required by endpoint configuration
      replace the exported portion of confidential mailbox with random bytes
    clear cod.internal_buffer
  fill cod.internal_buffer with signature, random[, encrypted_mailbox, mac, encsender_ePK] as described in Table 15-24
response_length ⟵ length written in cod.internal_buffer
  return response_length (2 bytes big-endian)
end
```

15.3.2.8 VIEW command

This command returns information on endpoints and Instance CA. The response is written in the internal buffer and accessible using READ BUFFER command.

In case the internal buffer size does not allow storage of all requested data elements, the internal buffer is filled with the maximum number of data elements permitted. No truncated data elements are written in the internal buffer.

The applet should respond with a warning SW 6300_h (or any implementation-specific SW in the range $63XX_h$ or $62XX_h$) to indicate that requested data is larger than the internal buffer size, and the length of the data written to the internal buffer which may be read using READ BUFFER command.

key_identifier is defined as the SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits).

command: CLA2 76 P1 P2 Lc (key_identifier) 00
response: [response_length] 9000

The CLA2 is as defined in Table 15-3.

Table 15-25: View Parameters

P1	P2	Lc	Payload	Description	Domain Version
00_h	$00–FF_h$	0	none	Request a list of LV-formatted key identifiers starting from the n-th element in memory, n represented by P2 (dummy endpoints are not returned).	Internal. N/A
01_h	00_h	20	key_identifier	Request endpoint setup information which includes all tags in the Table 15-52	Internal. N/A
02_h	00_h	20	key_identifier	Request endpoint creation certificate. Certificate parts (including certificate signature) may be re-generated.	Internal. N/A
03_h	00_h	0	none	Request current number of endpoints created in instance.	Internal. N/A
04_h	00_h	20	key_identifier	Request endpoint state active 01_h/terminated 00_h.	Internal. N/A
80_h	$00–FF_h$	0	none	Request a list of LV-formatted instance CA identifiers starting from the n-th element in memory, n represented by P2.	Internal. N/A
82_h	00_h	0	none	Request instance additional information.	Internal. N/A
83_h	00_h	0	none	Request current number of Instance CA created in instance.	Internal. N/A

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

P1	P2	Lc	Payload	Description	Domain Version
84_h	00_h	20	key_identifier	Request list of 1-byte transaction codes triggering user authentication on contactless interface; see Table 9-1 and Table 9-2.	Internal. N/A
85_h	00_h	20	key_identifier	Request list of 1-byte transaction codes triggering user authentication on wired interface, see Table 9-1 and Table 9-2.	Internal. N/A

Table 15-26: View Endpoint Identifier Response in Internal Buffer

Length	endpoint_identifier
20	key_identifier[n]
20	key_identifier[n+1]
20	...
20	key_identifier[last available Endpoint index (end of list) OR last Endpoint index that can fit in the internal buffer]

Table 15-27: View Instance CA Response in Internal Buffer

Length	instance_CA_identifier
1–30	instance_CA_identifier[n]
1–30	instance_CA_identifier[n+1]
1–30	...
1–30	instance_CA_identifier[last available Instance CA index (end of list) OR last Instance CA index that can fit in the internal buffer]

Listing 15-18: VIEW Processing

```
input: (key_identifier, instance_CA_identifier)
output: response_length
begin
  if key_identifier or instance_CA_identifier provided as input does not exist
    return 6400h
  if P1=01h or P1=02h or P1=84h or P1=85h
    if target endpoint with matching key_identifier is terminated
      return 6A88h
  if P1 = 01h
    write in cod.internal_buffer endpoint setup information which includes all tags in Table 15-52, tag 4Eh shall be set to the
    current value of the key_slot
  else if P1 = 02 h
    write in cod.internal_buffer endpoint creation certificate as per Listing 15-5
  else if P1 = 03h
    write in cod.internal_buffer current number of endpoints created (1 byte)
  else if P1 = 04h
    if endpoint designated by key_identifier is active
      write in 01h in cod.internal_buffer (1 byte)
    else
      write 00h in cod.internal_buffer (1 byte)
  else if P1 = 82h
    write in cod.internal_buffer instance additional information formatted as per Table 15-4
  else if P1 = 83h
    write in cod.internal_buffer current number of Instance CA created (1 byte)
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
else if P1 = 84h
  cod.internal_buffer <- nvm.endpoint.transaction_code_list_cless
else if P1 = 85h
  cod.internal_buffer <- nvm.endpoint.transaction_code_list_wired
else if P1 = 00h
  write in cod.internal_buffer list of key_identifier starting from n-th element as per Table 15-26 with n=P2
else if P1 = 80h
    write in cod.internal_buffer list of instance_CA_identifier starting from n-th element as per Table 15-27 with n=P2
    response_length = number of bytes written in cod.internal_buffer
if requested data bigger than internal buffer size
  return response_length (2 bytes big-endian), 6300h (or implementation specific warning SW in the ranges of 63XXh or
  62XXh)
else
  return response_length (2 bytes big-endian)
end
```

15.3.2.9 AUTH0 command

This command allows the vehicle to initiate the authentication procedure. In case a fast transaction is requested by the vehicle, a cryptogram is returned, allowing the vehicle to tentatively proceed with a fast transaction.

command: CLA3 80 P1 P2 Lc [Table 15-29**] 00**
response: [Table 15-30**] 9000**

The CLA3 is as defined in Table 15-3.

Table 15-28: AUTH0 P1, P2 Parameters

Description
P1 bits 0–7: bit0 Fast(1)/Standard(0) transaction request bit1 RFU (shall be set to 0) bit2 EXCHANGE commands will(1)/might(0) be sent during current transaction bit3–7 RFU (shall be set to 0) P2 bits 0–7: bit0–7 domain specific transaction_code Note: P1, P2 = FFFFh is reserved (shall not be used in AUTH0)

Table 15-29: AUTH0 Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
5Ch	2	protocol_version, the Digital Key applet transaction (V-D-TX) protocol version (ver.high | ver.low) selected by vehicle, shall be one of the versions proposed by DK Applet in SELECT response	mandatory	V-D-TX
87h	65	vehicle_ePK prepended by 04h	mandatory	V-D-TX
4Ch	16	transaction_identifier (randomly generated)	mandatory	V-D-TX
4Dh	8	vehicle_identifier (see Section 15.2)	mandatory	V-OD-FW

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-30: AUTH0 Response Payload

Tag	Length (bytes)	Description	Field is	Domain Version
86_h	65	endpoint_ePK, the applet generated ephemeral public key prepended by 04_h	mandatory	V-D-TX
$9D_h$	16	cryptogram, the authentication cryptogram returned by the endpoint	conditional	V-D-TX

Cryptogram (Tag 9Dh) shall not be included in the response payload if a standard transaction is requested by the vehicle (AUTH0 with P1 bit0 = 0), otherwise it shall be present.

Listing 15-19: AUTH0 Processing

```
input: protocol_version, P1, P2, vehicle_ePK, transaction_identifier, vehicle_identifier
output: endpoint_ePK, (cryptogram)
begin
  if cod.transaction_state != select_done
    return 6400h
  if protocol_version is not supported (i.e., not part of the nvm.supported_protocol_versions_tlv)
    return 6400h
  cod.current_protocol_version ← protocol_version
  generate an ephemeral key pair cod.endpoint_ePK, cod.endpoint_eSK according to Listing 15-40
  endpoint_ePK ← 04h || cod.endpoint_ePK
  cod.flag ← P1 || P2
  begin constant time or unbiased random time section
    find endpoint for corresponding vehicle_identifier excluding endpoints terminated or with visibility disabled
    if no endpoint found
      endpoint ← dummy_endpoint
      if fast transaction requested
        send to digital key framework notify_endpoint_not_found_fast with P1, P2
      else
        send to digital key framework notify_endpoint_not_found_standard with P1, P2
    if fast transaction requested but not allowed on current interface as per Table 15-13
      endpoint ← dummy_endpoint
  end constant time or unbiased random time section
  if interface is contactless
    if transaction_code from P2 is present in nvm.endpoint.transaction_code_list_cless
      if user authentication not performed
        send to digital key framework notify_user_authentication_not_performed with P1, P2
        endpoint ← dummy endpoint
  else
    if transaction_code from P2 is present in nvm.endpoint.transaction_code_list_wired
      if user authentication is not performed
          send to digital key framework notify_user_authentication_not_performed with P1, P2
          endpoint ← dummy_endpoint
  cod.vehicle_ePK ←  vehicle_ePK
  cod.transaction_identifier ← transaction_identifier
  cod.mac_chaining ← 00000000000000000000000000000000h
  cod.counter ← 00h
  cod.endpoint ← endpoint
  if Standard transaction requested
    cod.transaction_state ← auth0_std_done
    send to digital key framework notify_standard with nvm.endpoint.identifier with P1, P2
    return endpoint_ePK (65 bytes)
  else
     cod.transaction_state ← auth0_fast_done
     if endpoint.kpersistent_established = true
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    send to digital key framework
      notify_fast_kpersistent_established with nvm.endpoint.identifier with P1, P2
  else
    send to digital key framework
      notify_fast_kpersistent_not_established with nvm.endpoint.identifier with P1, P2
  // NOTE: The cryptogram computation below applies
  //   even if endpoint.kpersistent_established is set to false,
  //   as nvm.endpoint.Kpersistent was initialized to a random value at endpoint creation

  interface_byte ← C3h for wired interface or 5Eh for contactless interface
  info ← cod.vehicle_ePK.x || cod.endpoint_ePK.x || cod.transaction_identifier || interface_byte ||
    cod.flag || "VolatileFast" || || 5Ch || 02h || cod.current_protocol_version
  keying_material_length ← 64
  compute derived_keys according to Listing 15-45 using nvm.endpoint.Kpersistent, info, keying_material_length
  KCmac ← subset of derived_keys at offset 0 with length 16
  cod.Kenc ← subset of derived_keys at offset 16 with length 16
  cod.Kmac ← subset of derived_keys at offset 32 with length 16
  cod.Krmac ← subset of derived_keys at offset 48 with length 16

  compute cryptogram according to Listing 15-50 using KCmac
  return endpoint_ePK (65 bytes), cryptogram (16 bytes)
end
```

15.3.2.10 AUTH1 command

This command allows mutual authentication and establishment of a secure channel between the vehicle and device.

This command may be used to optionally pre-compute a ranging key. If this option is implemented, this command shall produce 80 bytes of keying material and the computed ranging key shall subsequently be made available upon reception of the CREATE RANGING KEY command. Otherwise, this command shall produce 48 bytes of keying material only.

command: CLA3 81 00 00 Lc [Table 15-32**] 00**
response: [encrypted_payload] [mac] 9000

The CLA3 is as defined in Table 15-3.

Table 15-31: AUTH1 Vehicle Authentication Data Fields

Tag	Length (bytes)	Description	Field is	Domain Version
$4D_h$	8	vehicle_identifier	mandatory	V-OD-FW
86_h	32	endpoint_ePK.x	mandatory	V-D-TX
87_h	32	vehicle_ePK.x	mandatory	V-D-TX
$4C_h$	16	transaction_identifier	mandatory	V-D-TX
93_h	4	usage = $415D9569_h$	mandatory	V-D-TX

Table 15-32: AUTH1 Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
$9E_h$	64	vehicle_sig computed as per Listing 15-42 using fields from Table 15-31 and vehicle_SK	mandatory	V-D-TX

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-33: AUTH1 Response Payload Before Encryption

Tag	Length (bytes)	Description	Field is	Domain Version
$4E_h$	1–8	key_slot	mandatory	V-OD-FW
$9E_h$	64	endpoint_sig, computed as per Listing 15-42 using fields from Table 15-34 and endpoint_SK	mandatory	V-D-TX
57_h	4	counter_value; this field contains the current endpoint counter value in big-endian	deprecated*	N/A
$4A_h$	variable	confidential_mailbox_data_subset. This tag shall be included if it was included previously in SETUP ENDPOINT command	conditional	V-OD-FW
$4B_h$	variable	private_mailbox_data_subset. This tag shall be included if it was included previously in SETUP ENDPOINT command	conditional	V-OD-FW

* This field has been deprecated and shall no longer be used

Table 15-34: Endpoint Authentication Data Fields

Tag	Length (bytes)	Description	Field is	Domain Version
$4D_h$	8	vehicle_identifier	mandatory	V-OD-FW
86_h	32	endpoint_ePK.x	mandatory	V-D-TX
87_h	32	vehicle_ePK.x	mandatory	N/A
$4C_h$	16	transaction_identifier	mandatory	V-OD-FW
93_h	4	usage = $4E887B4C_h$	mandatory	V-OD-FW

Listing 15-20: AUTH1 Processing

```
input: vehicle_sig
output: encrypted_payload, mac
begin
  if cod.transaction_state != auth0_std_done AND cod.transaction_state != auth0_fast_done
    return 6400h
  if standard transaction not allowed on this interface as per Table 15-13
    return 6900h
  endpoint ⟵ cod.endpoint
  fetch nvm.endpoint.vehicle_PK
  verify vehicle_sig according to Listing 15-43 using nvm.endpoint.vehicle_PK and fields from Table 15-31
  if vehicle_sig is not verified
    send to digital key framework notify_vehicle_authentication_failed
    return 6400h
  send to digital key framework notify_vehicle_authentication_success
  compute cod.Kdh according to Listing 15-44 using cod.transaction_identifier, cod.vehicle_ePK, cod.endpoint_eSK
  interface_byte ⟵ C3h for wired interface or 5Eh for contactless interface
  info ⟵ cod.vehicle_ePK.x || cod.endpoint_ePK.x || cod.transaction_identifier || interface_byte ||
    cod.flag || “Volatile” || 5Ch || 02h || cod.current_protocol_version
  if wired interface
    keying_material_length ⟵ 48 or 80
  else
    keying_material_length ⟵ 48
  compute derived_keys according to Listing 15-45 using cod.Kdh, info, keying_material_length
  cod.Kenc ⟵ subset of derived_keys at offset 0 with length 16
  cod.Kmac ⟵ subset of derived_keys at offset 16 with length 16
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    cod.Krmac ⟵ subset of derived_keys at offset 32 with length 16
    if wired interface and keying_material_length = 80
      cod.URSK ⟵ subset of derived_keys at offset 48 with length 32
    info ⟵ cod.vehicle_ePK.x || cod.endpoint_ePK.x || cod.transaction_identifier || interface_byte ||
      cod.flag || “Persistent” || 5Ch || 02h || cod.current_protocol_version
    keying_material_length ⟵ 32
    compute derived_keys according to Listing 15-45 using cod.Kdh, info, keying_material_length
    Kpersistent ⟵ subset of derived_keys at offset 0 with length 32
    append nvm.endpoint.key_slot to response payload
    atomic start
      if fast transaction allowed by endpoint configuration as per Table 15-13 (option_group_1 bit1 and/or bit3 is set)
        nvmwrite nvm.endpoint.Kpersistent ⟵ Kpersistent
        nvmwrite nvm.endpoint.kpersistent_established = true
    atomic commit
    compute endpoint_sig over fields Table 15-34 according to Listing 15-42 using endpoint.SK
    append endpoint_sig to response payload
    append confidential_mailbox_data_subset portion to response payload as per configuration in Section 15.3.2.21
    append private_mailbox_data_subset to response payload
    encrypt and mac response payload according to Listing 15-47 using cod.Kenc, cod.Krmac, cod.counter, cod.mac_chaining
    cod.transaction_state ⟵ auth1_done
    return mac, encrypted_payload
  end
```

15.3.2.11 PRESENCE0 command

This command allows the vehicle to initiate the Check Presence transaction.

command: CLA3 3D 00 00 Lc [Table 15-29**] 00**

response: [Table 15-35**] 9000**

The CLA3 is as defined in Table 15-3.

Table 15-35: PRESENCE0 Response Payload

Tag	Length (bytes)	Description	Field is	Domain Version
86_h	65	endpoint_ePK, the applet generated ephemeral public key prepended by 04_h	mandatory	V-D-TX

Listing 15-21: PRESENCE0 Processing

```
input: protocol_version, vehicle_ePK, transaction_identifier, vehicle_identifier
output: endpoint_ePK
begin
  if cod.transaction_state != select_done
    return 6400h
  If protocol_version is not supported (i.e., not part of nvm.supported_protocol_versions_tlv)
    return 6400h
  cod.current_protocol_version⟵ protocol_version
  generate an ephemeral key pair cod.endpoint_ePK, cod.endpoint_eSK according to Listing 15-40
  endpoint_ePK ⟵ 04h || cod.endpoint_ePK
  cod.flag ⟵ 5Bh
  begin constant time or unbiased random time section
    find endpoint for corresponding vehicle_identifier excluding endpoints terminated or with visibility disabled
    if no endpoint found
      endpoint ⟵ dummy_endpoint
      send to digital key framework notify_endpoint_not_found_presence
  end constant time or unbiased random time section
  cod.endpoint ⟵ endpoint
  cod.vehicle_ePK ⟵ vehicle_ePK
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
  cod.transaction_identifier ⟵ transaction_identifier
  cod.transaction_state ⟵ presence0_done
  cod.mac_chaining ⟵ 00000000000000000000000000000000h
  cod.counter ⟵ 00h
  send to digital key framework notify_presence with nvm.endpoint.identifier
  return endpoint_ePK (65 bytes)
end
```

15.3.2.12 PRESENCE1 command

This command allows an authenticated vehicle to retrieve the endpoint key_slot.

command: CLA3 3E 00 00 Lc [Table 15-36**] 00**
response: [encrypted_payload] [mac] 9000

The CLA3 is as defined in Table 15-3.

Table 15-36: PRESENCE1 Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
$9E_h$	64	vehicle_sig computed as per Listing 15-42 using fields from Table 15-37 and vehicle_SK	mandatory	V-D-TX

Table 15-37: PRESENCE1 Vehicle Authentication Data Fields

Tag	Length (bytes)	Description	Field is	Domain Version
$4D_h$	8	vehicle_identifier	mandatory	V-OD-FW
86_h	32	endpoint_ePK.x	mandatory	V-D-TX
87_h	32	vehicle_ePK.x	mandatory	V-D-TX
$4C_h$	16	transaction_identifier	mandatory	V-D-TX
93_h	4	usage = 47165979_h	mandatory	V-D-TX

Table 15-38: PRESENCE1 Response Payload Before Encryption

Tag	Length (bytes)	Description	Field is	Domain Version
$4E_h$	1–8	key_slot	mandatory	V-OD-FW

Listing 15-22: PRESENCE1 Processing

```
input: vehicle_sig
output: encrypted_payload, mac
begin
  if cod.transaction_state != presence0_done
    return 6400h
  cod.transaction_state ⟵ cp_done
  endpoint ⟵ cod.endpointh
  verify vehicle_sig according to Listing 15-43 using nvm.endpoint.vehicle_PK and fields from Table 15-37
  if vehicle_sig is not verified
    send to digital key framework notify_vehicle_authentication_failed
    return 6400h
  send to digital key framework notify_vehicle_authentication_success
  compute Kdh according to Listing 15-44 using cod.transaction_identifier, cod.vehicle_ePK, cod.endpoint_eSK
  interface_byte ⟵ 5Eh for contactless interface
  info ⟵ cod.vehicle_ePK.x || cod.endpoint_ePK.x || cod.transaction_identifier || interface_byte ||
    cod.flag || "Volatile" || 5Ch || 02h || cod.current_protocol_version
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
  keying_material_length ⟵ 48
  compute derived_keys according to Listing 15-45 using Kdh, info, keying_material_length
  cod.Kenc ⟵ subset of derived_keys at offset 0 with length 16
  cod.Krmac ⟵ subset of derived_keys at offset 32 with length 16

  append nvm.endpoint.key_slot to response payload as per Table 15-38
  encrypt and mac response payload according to Listing 15-47 using cod.Kenc, cod.Krmac, cod.counter, cod.mac_chaining
  return mac, encrypted_payload
end
```

15.3.2.13 READ BUFFER command

This command is used to read temporarily stored data in an internal buffer. This supplements the APDU buffer for large payloads. The following command formats are allowed:

- If no secure channel is to be established between the Digital Key framework and the Digital Key applet, then either command format 1 or command format 2 may be used and implemented by the applet.
- If a secure channel is to be established between the Digital Key framework and the Digital Key applet (option A), command format 2 shall be used and shall be implemented by the applet.

Command format 1:

command: CLA2 B0 [offsetmsb] [offsetlsb] Le
response: [response] 90 00

The CLA2 is as defined in Table 15-3.

Command format 2:

command: CLA2 B0 [offsetmsb] [offsetlsb] Lc [Table 15-39**] 00**
response: [response] 90 00

The CLA2 is as defined in Table 15-3.

Table 15-39: READ BUFFER Command Payload for Format 2

Tag	Length (bytes)	Description	Field is	Domain Version
80_h	1	1-byte unsigned size of the data that shall be read from the internal buffer	mandatory	Internal. N/A

Listing 15-23: READ BUFFER Processing for Command Format 1

```
input: offsetmsb, offsetlsb, Le
output: response
begin
  if data to be read overflows the cod.internal_buffer size
    return 6400h
  read cod.internal_buffer at offsetmsb || offsetlsb
  return response
end
```

Listing 15-24: READ BUFFER Processing for Command Format 2

```
input: offsetmsb, offsetlsb, size_of_data
output: response
Begin
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
  if size_of_data != 00h
    if size_of_data > remaining size of available data at offset offsetmsb || offsetlsb or size_of_data > maximum response size
          allowed
    return 6400h or 6700h or 6985h
  return response of size_of_data bytes at offset offsetmsb || offsetlsb from the cod.internal_buffer
end
```

15.3.2.14 WRITE BUFFER command

This command is used to temporarily store data in an internal RAM buffer. This supplements the APDU buffer for large payloads.

command: CLA2 D0 [offsetmsb] [offsetlsb] Lc [data]
response: 90 00

The CLA2 is as defined in Table 15-3.

Listing 15-25: WRITE BUFFER Processing

```
input: offsetmsb, offsetlsb, data
output: n/a
begin
  if data to be be written overflows the cod.internal_buffer size
    return 6400h
  write cod.internal_buffer at offsetmsb || offsetlsb with data from command payload
  return
end
```

15.3.2.15 EXCHANGE command

This command reads, writes or sets data from confidential/private mailboxes. This command is available after AUTH0 and/or AUTH1, depending on endpoint creation parameters. The commands and responses are always wrapped in the currently established secure channel.

The command also is used to send notification messages to the Digital Key framework. In such cases, the messages are not stored in mailbox.

In case command MAC verification fails, the applet returns 6982_h, and the atomic session, if any, is aborted.

The set request fills the indicated memory area with the provided single byte value.

Multiple read, write, set, and notify operations can be present in a single EXCHANGE command. Device shall support up to one notify operation to be transferred per EXCHANGE command, if more than one notify operation is sent in an EXCHANGE command the behavior is undefined

All read operations contained in an EXCHANGE command are returned in the order they appear in the request buffer. All write/set operations contained in an EXCHANGE command are written atomically and in the order they appear in the request buffer if the Atomic Session indicator flag is set to 0. Write/set operations are atomic per group of EXCHANGE commands processed as part of an atomic session.

An atomic session spanned over multiple commands is executed by setting the Atomic Session indicator flag to 1 on a series of consecutive EXCHANGE commands. The session is closed, and all data written in NVM by setting the Atomic Session indicator flag to 0 on the last EXCHANGE command of the series.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

When multiple EXCHANGE commands are part of an atomic session, all written values are effectively readable only after successful execution of a command with the Atomic Session indicator flag set to 0. All read operations occurring before that point return the old value.

If the transaction has been started over the wired interface, as conditionally permitted depending on endpoint configuration, the Digital Key framework shall avoid prematurely interrupting the transaction, e.g., by reselecting the applet. However, if it hasn't been notified of transaction end after some unusually long time period (i.e., timeout), the Digital Key framework may decide to forcefully end the transaction, e.g., by reselecting the applet.

command: CLA4 C9 00 00 Lc [encrypted_command_payload] [command_mac] 00
response: [encrypted_response_payload] [response_mac] 90 00

The CLA4 is as defined in Table 15-3.

Table 15-40: Exchange Command Decrypted Payload

Tag	Length (bytes)	Description	Field is	Domain Version
option byte: bit0 Atomic Session indicator start(1)/stop(0), other bits RFU (left to 0)			mandatory	V-D-TX
88_h	3	offsetmsb || offsetlsb || length, read request in private mailbox	optional	V-D-TX
89_h	3	offsetmsb || offsetlsb || length, read request in confidential mailbox	optional	V-D-TX
$8A_h$	variable	offsetmsb || offsetlsb || data, write request in private mailbox	optional	V-D-TX
$8B_h$	variable	offsetmsb || offsetlsb || data, write request in confidential mailbox	optional	V-D-TX
$8C_h$	4	offsetmsb || offsetlsb || length || value, set request in private mailbox	optional	V-D-TX
$8D_h$	4	offsetmsb || offsetlsb || length || value, set request in confidential mailbox	optional	V-D-TX
95_h	5	offsetmsb || offsetlsb || lengthmsb || lengthlsb || value, set request in private mailbox	optional	V-D-TX
96_h	5	offsetmsb || offsetlsb || lengthmsb || lengthlsb || value, set request in confidential mailbox	optional	V-D-TX
$8E_h$	variable	notify Digital Key framework with data present in this field. Data present in this field is stored in the Digital Key framework to be transmitted to the Vehicle OEM app. The maximum size supported for the value field of this TLV object is 211 bytes (size available for the *notify_message_in_exchange* notification to the Digital Key framework). This is reduced by 40 bytes if the EXCHANGE command also includes one or more write or set requests (to manage the *notify_mailbox_written* notification). In any case, the size of this value field is limited by the available payload in the EXCHANGE command.	optional	V-D-TX
...other read, write or set requests			optional	

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1 *Table 15-41: Exchange Response Decrypted Payload*

Length (bytes)	Description	Field is	Domain Version
variable	data read from request 1	conditional	V-D-TX
variable	data read from request 2	conditional	V-D-TX
...data read from request n		conditional	

2 *Listing 15-26: EXCHANGE Processing*

```
1  input: encrypted_command_payload, command_mac
2  output: encrypted_response_payload, response_mac
3  begin
4   if cod.counter = FFh
5     cod.transaction_state ← select_done
6     cod.atomic_session ← stopped
7     return 6900h
8
9   if cod.transaction_state != exchange_done and cod.transaction_state != auth1_done
      and cod.transaction_state != auth0_fast_done
10      return 6400h
11
12  if cod.transaction_state == auth0_fast_done and EXCHANGE not allowed after AUTH0
13    return 6400h
14
15  cod.counter ⟵ cod.counter + 1
16  endpoint ⟵ cod.endpointh
17  verify and decrypt command according to Listing 15-46
18    inputs: cod.Kenc, cod.Kmac, cod.counter, cod.mac_chaining
19    output: decrypted_command_payload, mac_chaining_out
20  if command mac verification fails
21   cod.transaction_state ← select_done
22   cod.atomic_session ← stopped
23    return 6982h
24  cod.mac_chaining ⟵ mac_chaining_out
25  cod.transaction_state ⟵ exchange_done
26  if total requested output data in decrypted_command_payload is more than 239 bytes
27    return 6400h
28  if write/read/set requests present in decrypted_command_payload are out of mailbox boundaries
29    return 6400h
30  if notification requests are present in command payload
31    send to digital key framework notify_message_in_exchange
32  execute all notification requests from Table 15-40
33  execute all read requests from Table 15-40 on endpoint mailboxes
34  append read data to response payload for all read requests as per Table 15-41 from endpoint mailboxes
35  if Atomic Session indicator bit = 1
36    cod.atomic_session ⟵ started
37  if cod.atomic_session = started
38    if all write/set requests exceeds cod.commit_buffer size
39       cod.atomic_session ⟵ stopped
40       return 6A84h
41    store all write/set requests from Table 15-40 in cod.commit_buffer
42    if Atomic Session indicator bit = 0
43      atomic start
44        if cod.commit_buffer is not empty
45          execute all write/set requests pending in cod.commit_buffer on endpoint mailboxes
46          send to digital key framework notify_mailbox_written
47      atomic commit
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    cod.atomic_session ← stopped
  else
    atomic start
      if decrypted_command_payload contains write or set requests
        execute all write/set requests from Table 15-40 on endpoint mailboxes
        send to digital key framework notify_mailbox_written
    atomic commit
    compute and return encrypted_response_payload and response_mac according to
    Listing 15-47 using cod.Kenc, cod.Krmac, cod.mac_chaining, cod.counter
  end
```

15.3.2.16 CONTROL FLOW command

This command allows the vehicle to indicate the final success or failure of the transaction or to signal application-specific codes.

The P2 field is used for vehicle error codes, domain specific codes, or timing information.

The vehicle error codes are hints provided by the vehicle to the device in case of a transaction failure. The decision to send such error codes is vehicle implementation-specific.

The variables P1 and P2 are version controlled under the V-D-TX domain version.

command: CLA3 3C [Table 15-42**] [**Table 15-43**]**
response: 90 00

The CLA3 is as defined in Table 15-3.

Table 15-42: CONTROL FLOW P1 Parameters

P1 value	Description
00_h	transaction finished with failure
01_h	transaction finished with success
40_h	application specific
other values	RFU

Table 15-43: CONTROL FLOW P2 Parameters

P2 value	Description
00_h	with P1 = 00_h, 01_h, or 40_h, no information provided
00_h–$7F_h$	Only with P1 = 00_h, this range is used for the following vehicle error codes: 00_h no information provided 01_h public key not found 02_h public key expired 03_h public key not trusted 04_h invalid signature 05_h invalid channel 06_h invalid data format 07_h invalid data content 08_h–$7F_h$ RFU
80_h–FF_h	With P1 = 00_h, 01_h, or 40_h, this valid range is shown in Table 15-44

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1 *Table 15-44: CONTROL FLOW P1/P2 Values for Applet*

P1	P2	Description	Reference
00_h	$A0_h$	Key deleted in/not known to vehicle. Vehicle was offline during key deletion in vehicle or unknown key without attestation package detected	See Note 4
00_h	$B0_h$	Cannot lock the vehicle because not all doors/trunk are closed	See Note 4, 5
00_h	$B1_h$	Cannot lock the vehicle because it is not in parking state (e.g. engine is still running/vehicle is still in driving state)	See Note 4, 5
01_h	$B0_h$	First approach successful	Section 19.5.8.2
01_h	81_h	Successful end, vehicle has completed owner pairing phase 3	Section 6.3.4.3
01_h	90_h	End, key is tracked, and all data have been written successfully into the mailboxes	Section 6.3.5.5
01_h	91_h	End, key is not tracked, key sharing is not possible, and the owner needs to go online to track the key before using it	Section 6.3.5.5
40_h	81_h	Optional token refill start	Section 6.3.5.2
40_h	82_h	Continue, Attestation package delete start	Section 6.3.5.4
40_h	83_h	Optional service data write start	See Note 1
40_h	88_h	Continue, key tracking response received in device, next step is to read the receipt from the mailbox	Section 6.3.5.2
40_h	89_h	Continue, key tracking response received in car, go directly to verification of key tracking receipt	Section 6.3.5.2
40_h	90_h	Extend reader processing time (do nothing on device)	See Note 2
40_h	$A0_h$	Long processing time (UI indication)	See Note 3

Note 1: Vehicle OEM may use this option for providing application-specific service data after engine start sequence.

Note 2: Extended reader processing time is a command that can be sent by the reader to prevent a reader-side timeout. It has no effect on the device.

Note 3: When the vehicle executes operations that take longer than the usual UI-tolerated times, it should send this indication to the device. The device can show an appropriate UI, which is out of scope of this specification.

Note 4: Used to indicate unsuccessful completion of transaction or inability to perform intended action due to a conflicting state on the vehicle.

Note 5: Typically, only relevant for NFC transactions.

2 *Listing 15-27: CONTROL FLOW Processing*

```
1  input: P1, P2
2  output: none
3  begin
4    if cod.transaction_state = auth0_std_done or cod.transaction_state = auth1_done or cod.transaction_state =
                 auth0_fast_done or cod.transaction_state = exchange_done or cod.transaction_state = create_rk_done
5      if P1 == 00h
6        cod.transaction_state ← select_done
7        cod.atomic_session ← stopped
8        send to digital key framework notify_end_of_transaction with P1, P2 of CONTROL FLOW, P1, P2 of AUTH0 command
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    else if P1 == 01h
      cod.transaction_state ← select_done
      cod.atomic_session ← stopped
      send to digital key framework notify_end_of_transaction with P1, P2 of CONTROL FLOW, P1, P2 of AUTH0 command
    else
      send to digital key framework notify_application_specific with P1, P2 of CONTROL FLOW, P1, P2 of AUTH0 command
  else
    if P1 == 00h or P1 == 01h
      send to digital key framework notify_end_of_transaction with P1, P2 of CONTROL FLOW, outside of a fast or standard
            transaction
    else
      send to digital key framework notify_application_specific with P1, P2 of CONTROL FLOW, outside of a fast or standard
            transaction
  End
```

15.3.2.17 CREATE ENCRYPTION KEY command

Create a confidential mailbox encryption key attestation in order to allow data insertion in the endpoint. This encryption key is stored in NVM and valid for only one usage with the SET CONFIDENTIAL DATA command. After single usage with the SET CONFIDENTIAL DATA command, the key is erased from internal memory.

The attestation containing the encryption public key is stored in the internal buffer, accessible with the READ BUFFER command.

command: CLA2 8A 00 00 Lc [Table 15-45**] 00**
response: [response_length] 90 00

Table 15-45: CREATE ENCRYPTION KEY Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW

Table 15-46: Encryption Key Attestation Data Fields

Tag	Length (bytes)	Description	Field is	Domain Version
41_h	1	version = 01_h, version of Encryption Key Attestation	mandatory	N/A. Informative Only
92_h	8	random	mandatory	N/A. Informative Only
$5F49_h$	65	encreceiver_ePK, the encryption ephemeral public key prepended by 04_h	mandatory	N/A. Informative Only
$5D_h$	20	authority_key_identifier, 160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey from the issuer (endpoint) certificate (as defined in Listing 15-5 excluding the tag, length, and number of unused bits)	mandatory	N/A. Informative Only
93_h	4	usage = $49A8A741_h$	mandatory	N/A. Informative Only

Table 15-47: Encryption Key Attestation

Tag	Length (bytes)	Description	Field is	Domain Version
$7F26_h$	variable	Encryption Key Attestation	mandatory	N/A. Informative Only

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
content of Table 15-46			mandatory	N/A. Informative Only
$9E_h$	64	Receiver endpoint signature over fields from Table 15-46 with endpoint.SK	mandatory	N/A. Informative Only

Listing 15-28: CREATE ENCRYPTION KEY Processing

```
input: key_identifier
output: response_length, encryption_key_attestation
begin
  if key_identifier does not match with any of the existing endpoints or target endpoint is terminated
    return 6A88h
  generate a key pair encreceiver_ePK, encreceiver_eSK according to Listing 15-40
  atomic start
    store nvm.endpoint.encreceiver.eSK using encreceiver_eSK
    nvm.endpoint.encreceiver.isActive ⟵ true
  atomic commit
  generate signature using nvm.endpoint.SK and fields from Table 15-46 according to Listing 15-42
  generate data field encryption_key_attestation as per Table 15-47
  store encryption_key_attestation in cod.internal_buffer
  response_length  ⟵ length written in cod.internal_buffer
  return response_length (2 bytes big-endian)
end
```

15.3.2.18 GET PRIVATE DATA command

Retrieve data in the private mailbox. The following command formats are allowed:

- If no secure channel is to be established between the Digital Key framework and the Digital Key applet, then either Command Format 1 or Command Format 2 may be used and implemented by the applet.
- If a secure channel is to be established between the Digital Key framework and the Digital Key applet (option A), command format 2 shall be used and shall be implemented by the applet.

Command Format 1:

command: CLA2 78 [offsetmsb] [offsetlsb] Lc [Table 15-48**] Le**
response: [response] 90 00

The CLA2 is as defined in Table 15-3.

Command Format 2:

command: CLA2 78 [offsetmsb] [offsetlsb] Lc [Table 15-48**] 00**
response: [response] 90 00

The CLA2 is as defined in Table 15-3.

Table 15-48: GET PRIVATE DATA Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
80_h	1	1-byte unsigned size of the data that shall be read from the internal buffer	mandatory for command format 2 not required for command format 1	V-OD-FW

Listing 15-29: GET PRIVATE DATA Processing for Command Format 1

```
input: key_identifier, offset_msb, offset_lsb, Le
output: response
begin
  if key_identifier does not match with any existing endpoints or target endpoint is terminated
    return 6A88h
  return response of Le bytes at offset offsetmsb || offsetlsb from the private mailbox
end
```

Listing 15-30: GET PRIVATE DATA Processing for Command Format 2

```
input: key_identifier, offset_msb, offset_lsb, size of_data
output: response
begin
  if key_identifier does not match with any existing endpoints or target endpoint is terminated
    return 6A88h
  return response of size_of_data bytes at offset offset_msb || offset_lsb from the private mailbox
end
```

15.3.2.19 SET PRIVATE DATA command

Store data in the private mailbox.

command: CLA2 7A [offsetmsb] [offsetlsb] Lc [Table 15-49**]**

response: 90 00

The CLA2 is as defined in Table 15-3.

Table 15-49: SET PRIVATE DATA Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW
$4B_h$	variable	data, field to be written in private mailbox	mandatory	V-OD-FW

Listing 15-31: SET PRIVATE DATA Processing

```
input: key_identifier, offsetmsb, offsetlsb, data
output: none
begin
  if key_identifier does not match with any existing endpoints or target endpoint is terminated
    return 6A88h
  atomic start
    write command data in NVM private mailbox at offsetmsb || offsetlsb
  atomic commit
  return
end
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

15.3.2.20 SET CONFIDENTIAL DATA command

Store data in the confidential mailbox. The appropriate data fields as described Table 15-51, shall have been previously loaded in the internal buffer at offset 0 using the WRITE BUFFER command. The CREATE ENCRYPTION KEY shall have been executed for this endpoint, and the encryption key shall still be active.

command: CLA2 7C [offsetmsb] [offsetlsb] Lc [Table 15-50**]**
response: 90 00

Table 15-50: SET CONFIDENTIAL DATA Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW

Table 15-51: SET CONFIDENTIAL DATA Internal Buffer Content Before Processing

Tag	Length (bytes)	Description	Field is	Domain Version
97_h	65	encsender_ePK	mandatory	N/A. Informative Only
$4A_h$	variable	encrypted_mailbox || mac	mandatory	N/A. Informative Only

Listing 15-32: SET CONFIDENTIAL DATA Processing

```
input: key_identifier, offsetmsb, offsetlsb
output:
begin
  if key_identifier does not match with any existing endpoints or target endpoint is terminated
    return 6A88h
  if command is not allowed as per Table 15-13
    return 6900h
  if nvm.endpoint.encreceiver.isActive ⟵ false
    return 6400h
  if cod.internal_buffer does not contain data described in Table 15-51
    return 6400h
  nvm.endpoint.encreceiver.isActive ⟵ false
  generate KEseed according to Listing 15-44 using nvm.endpoint.encreceiver.eSK, encsender_ePK
  derivation ⟵ 08h
  KEseed_half ⟵ 16 most significant bytes of KEseed
  generate KEenc according to Listing 15-48 using KEseed_half, encsender_ePK, nvm.endpoint.encreceiver_ePK,
    derivation, nvm.endpoint.PK
  derivation ⟵ 12h
  KEseed_half ⟵ 16 least significant bytes of KEseed
  generate KEmac according to Listing 15-48 using KEseed_half, encsender_ePK, nvm.endpoint.encreceiver_ePK,
    derivation, nvm.endpoint.PK
  verify and decrypt the selected confidential mailbox area according to Listing 15-49
  if verification fails
    randomize nvm.endpoint.encreceiver.eSK
    return 6982h
  atomic start
    randomize nvm.endpoint.encreceiver.eSK
    write command data in NVM confidential mailbox at offsetmsb || offsetlsb
  atomic commit
  clear cod.internal_buffer
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
return
end
```

15.3.2.21 SETUP ENDPOINT command

This command is used to change the default private/confidential mailbox content returned in AUTH1 response; by default, no mailbox content is returned in the AUTH1 response.

This command is used to enable/disable individual endpoint visibility. When visibility is disabled, the endpoint cannot be found during the AUTH0 command. Consequently, no transaction can be executed with the endpoint. Other management commands remain available.

A newly created endpoint is visible by default.

Persistent configurations are applied immediately upon request (overriding current volatile configuration, if any) and are reapplied on each SE power cycle.

Volatile configurations are applied immediately upon request.

When persistent and volatile configurations are present in the same command, the configurations are applied in the order mentioned in Listing 15-33.

command: CLA2 7E 00 00 Lc [Table 15-52**]**
response: 90 00

The CLA2 is as defined in Table 15-3.

Table 15-52: SETUP ENDPOINT Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW
$4A_h$	3	2 bytes offset (unsigned big-endian) || 1 byte length to be returned from confidential mailbox in AUTH1 response (persistent)	optional	N/A. Informative Only
$4B_h$	3	2 bytes offset (unsigned big-endian) || 1 byte length to be returned from private mailbox in AUTH1 response (persistent)	optional	N/A. Informative Only
82_h	1	00_h (disable) / 01_h (enable) endpoint visibility contactless interface (persistent)	optional	V-D-TX
83_h	1	00_h (disable) / 01_h (enable) endpoint visibility on contactless interface (volatile)	optional	V-D-TX
84_h	0–16	List of 1-byte transaction codes triggering a user authentication if present in AUTH0 P2 over contactless interface (persistent), see Table 9-1, Table 9-2	optional	V-D-TX
85_h	0–16	List of 1-byte transaction codes triggering a user authentication if present in AUTH0 P2 over wired interface (persistent), see Table 9-1, Table 9-2	optional	V-D-TX
$4E_h$	0–8	key_slot to be used by friend endpoint (1–8 byte) If tag present but empty, key_slot will be restored to initial_key_slot assigned during key creation.	optional	V-OD-FW

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
$9B_h$	1	00_h (disable) / 01_h (enable) endpoint visibility on wired interface (persistent)	optional	V-D-TX
$9C_h$	1	00_h (disable) / 01_h (enable) endpoint visibility on wired interface (volatile)	optional	V-D-TX

Listing 15-33: SETUP ENDPOINT Processing

```
input: key_identifier, Table 15-52
output:
begin
  if key_identifier does not match any of the existing endpoints or target endpoint is terminated
    return 6A88h
  if offset/length out of mailbox boundaries
    return 6400h
  if the selected mailbox areas sizes do not fit in AUTH1 response
    return 6400h
  if one of the transaction_code present in nvm.endpoint.transaction_code_list_cless is not present in tag 84h
    if user authentication has not been performed
      return 6982h
  if one of the transaction_code present in nvm.endpoint.transaction_code_list_wired is not present in tag 85h
    if user authentication has not been performed
      return 6982h
  atomic start
    store the persistent endpoint configuration from command payload
    if tag 4Eh is present
      if tag 4Eh is empty
        nvmwrite nvm.endpoint.key_slot ← nvm.endpoint.initial_key_slot (restore initial value from key creation)
      else
        nvmwrite nvm.endpoint.key_slot ← key_slot (assign new value provided in tag 4Eh)
  atomic commit
  apply the persistent endpoint configuration from command payload
  apply the volatile endpoint configuration from command payload
  return
end
```

15.3.2.22 SETUP INSTANCE command

This command is used to enable/disable endpoint visibility of all endpoints on the contactless interface. If visibility is disabled, the endpoints appear as non-existent on the contactless interface.

A newly created endpoint is enabled by default. Persistent configurations are applied immediately upon request (overriding current volatile configurations, if any) and are reapplied on each SE power cycle.

Volatile configurations are applied immediately upon request. When persistent and volatile configurations are present in the same command, the configurations are applied in the order mentioned in Listing 15-34 (SETUP INSTANCE processing).

command: CLA2 34 00 00 Lc [Table 15-53**]**
response: 90 00

The CLA2 is as defined in Table 15-3.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-53: SETUP INSTANCE Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
82_h	1	00_h (disable) / 01_h(enable) all endpoints visibility on contactless interface (persistent)	optional	V-D-TX
83_h	1	00_h (disable) / 01_h(enable) all endpoints visibility on contactless interface (volatile)	optional	V-D-TX
$9B_h$	1	00_h (disable) / 01_h(enable) all endpoints visibility on wired interface (persistent)	optional	V-D-TX
$9C_h$	1	00_h (disable) / 01_h(enable) all endpoints visibility on wired interface (volatile)	optional	V-D-TX
90_h	0	If this tag is present, all volatile configurations are cleared, and the current persistent configurations are applied.	optional	V-D-TX

Listing 15-34: SETUP INSTANCE Processing

```
input: Table 15-53
output:
begin
  atomic start
    store the persistent endpoint configuration for all endpoints in the instance
  atomic commit
  apply the persistent instance configuration for all endpoints in the instance
  apply the volatile instance configuration for all endpoints in the instance
  return
end
```

15.3.2.23 SIGN command

This command signs an arbitrary data field using the private key of the selected endpoint. The command can be used with or without performing a user authentication; a different “usage” value is included in the signature by the SE to differentiate these two contexts.

command: CLA2 30 [Table 15-54**] 00 Lc [**Table 15-55**] 00**
response: [Table 15-57**] 90 00**

Table 15-54: SIGN Command Payload

P1 value	Description	Domain Version
00_h	User authentication is required	V-D-TX
01_h	User authentication is not required	V-D-TX
02_h-FF_h	RFU	N/A

The CLA2 is as defined in Table 15-3.

If P1 value is set to 0 (i.e., user authentication is required), the device will perform user authentication. The Digital Key applet obtains information on whether or not the user authentication was performed in a proprietary manner and uses the appropriate usage value for SIGN response (see Listing 15-35).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-55: SIGN Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the endpoint certificate that issues the attestation (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW
58_h	32	Arbitrary data to be signed	mandatory	N/A. Informative Only

Table 15-56: SIGN Data Fields

Tag	Length (bytes)	Description	Field is	Domain Version
41_h	1	Version = 01_h, version of Signature Data Fields	mandatory	V-D-TX
92_h	8	Random	mandatory	N/A. Informative Only
$5D_h$	20	key_identifier, 160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey from the endpoint certificate that issued the attestation (excluding the tag, length, and number of unused bits)	mandatory	V-OD-FW
58_h	32	arbitrary_data (provided in command)	mandatory	N/A. Informative Only
57_h	4	counter_value of the endpoint	deprecated*	N/A
93_h	4	usage = $D074DA4F_h$, if user authentication was performed usage = $FC6F4C17_h$, if user authentication was not performed	mandatory	N/A. Informative Only

* This field has been deprecated and shall no longer be used.

The following table describes the content of the SIGN response.

Table 15-57: SIGN Response

Tag	Length (bytes)	Description	Field is	Domain Version
41_h	1	version = 01_h, version of Signature Data Fields	mandatory	V-D-TX
92_h	8	Random	mandatory	N/A. Informative Only
$5D_h$	20	key_identifier	mandatory	V-OD-FW
57_h	4	counter_value, this field contains the current endpoint counter value in big-endian	deprecated*	N/A
$9E_h$	64	Signature over fields from Table 15-56 with the endpoint private key	mandatory	N/A. Informative Only

* This field has been deprecated and shall no longer be used.

Listing 15-35: SIGN Processing

```
input: P1, key_identifier, arbitrary_data
output: signature, random, key_identifier, version, (counter)
begin
  if key_identifier does not match any of the existing endpoints or target endpoint is terminated
    return 6A88h
  if SIGN not allowed on this endpoint as per Table 15-13
    return 6900h
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
if P1 == 00h
  if user authentication is not performed
    return 6985h
  usage D074DA4Fh
else if P1 == 01h
  usage FC6F4C17h
else
  return 6A86h
version ← 01h
generate 8 bytes random as per Listing 15-39 to be added in data_fields according to Table 15-56
generate data_fields to be signed according to Table 15-56
   version, usage, key_identifier, random, arbitrary_data
generate signature according to Listing 15-42 using data_fields and nvm.endpoint.SK of the current endpoint
return signature, random, key_identifier, version
end
```

15.3.2.24 MANAGE UA command

This command provides the user authentication status. Implementing this command is optional and the user authentication status may be provided using a proprietary method.

command: CLA2 A0 00 00 Lc [Table 15-58**] 00**
response: 90 00

The CLA2 is as defined in Table 15-3.

Table 15-58: MANAGE UA Command Payload

Tag	Length (bytes)	Description	Field is	Domain Version
80_h	1	User authentication status 00_h (user not authenticated) / 01_h (user authenticated)	mandatory	V-OD-FW
81_h	variable	Discretionary data	optional	V-OD-FW
$A1_h$	variable	TLV-structured discretionary data	optional	V-OD-FW

The applet shall ignore tags 81_h and $A1_h$ if it has no interpretation for them

15.3.2.25 onDeselect() Event

This event is called on explicit or implicit deselection of the applet.

Event processing pseudo-code:

Listing 15-36: onDeselect Event Processing

```
input: reason
output: none
begin
  if contactless interface
    (optional) send to digital key framework notify_deselect with reason
end
```

15.3.2.26 CREATE RANGING KEY command

This command generates a ranging key (pre-derived URSK) and makes it available to a UWB along with arbitrary data. This command shall only be executed after successfully receiving AUTH1 Response. If the creation of pre-derived URSK is not possible (e.g. two pre-derived URSKs already exist), the status word 6484_h is returned.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The arbitrary_data field (up to 127 bytes) is optional and reserved for future usage.

command: CLA2 71 00 00 Lc [arbitrary_data]
response: 90 00

Listing 15-37: CREATE RANGING KEY processing.

```
input: arbitrary_data
output: none
begin
  if UWB not supported by platform
    return 6D00h
  if cod.transaction_state != auth1_done and cod.transaction_state != exchange_done
    return 6400h
  cod.transaction_state ⟵ create_rk_done

  if cod.URSK not initialized
    interface_byte ⟵ C3h
    info ⟵ cod.vehicle_ePK.x || cod.endpoint_ePK.x || cod.transaction_identifier || interface_byte ||
      cod.flag || "Volatile" || 5Ch || 02h || cod.current_protocol_version

    keying_material_length ⟵ 80
    compute derived_keys according to Listing 15-45 using cod.Kdh, info, keying_material_length
    URSK ⟵ subset of derived_keys at offset 48 with length 32
  else
    URSK ⟵ cod.URSK

  key_identifier ⟵ 160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey from the currently
                   selected endpoint certificate (excluding the tag, length, and number of unused bits)
  Make URSK, cod.transaction_identifier, key_identifier, arbitrary_data available
  if storing URSK fails because queue is full
    return 6484h
  if URSK is not available for other reason
    return 6400h
  send to digital key framework notify_URSK_created
  return 9000h
end
```

15.3.2.27 DELETE RANGING KEYS command

This command is optionally supported to delete either a given pre-derived URSK or all pre-derived URSKs associated to an endpoint. This command shall not affect the URSK used in the active ranging session.

command: CLA2 41 00 00 Lc [Table 15-59**]**
response: 9000

The CLA2 is as defined in Table 15-3.

Table 15-59: Delete Ranging Keys Request.

Tag	Length (bytes)	Description	Field is	Domain Version
CFh	4	UWB session ID	conditional	V-D-BT

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag	Length (bytes)	Description	Field is	Domain Version
50_h	20	key_identifier, SHA-1 hash of the value of the BIT STRING subjectPublicKey of the target endpoint (excluding the tag, length, and number of unused bits)	conditional	V-OD-FW

Listing 15-38: DELETE RANGING KEYS Processing.

```
input: uwb_session_ID, key_identifier
output: none
begin
 if uwb_session_ID is present
  if uwb_session_ID does not match with any existing secure ranging session
   return 6A88h
  if uwb_session_ID matches the session id of a currently active secure ranging session
   return 6400h
  atomic start
   delete pre-derived URSK (and related data) associated with uwb_session_ID
   atomic commit
   return 9000h
 if key_identifier is present
  if key_identifier does not match with any of the existing endpoints
   return 6A88h
 atomic start
  delete all pre-derived URSKs (and related data) associated with key_identifier except if URSK used
  for a currently active secure ranging session
 atomic commit
 return 9000h
end
```

15.3.2.28 GET NOTIFICATION command

Implementing this command is optional and the notification(s) may be provided using a proprietary method. If Option D is supported, this command shall be implemented as defined in Section 15.3.1.9, in order to retrieve the notification(s) that may have been generated during the processing of the previous APDU command over the wired interface and the same logical channel. **command: CLA2 A1 00 00 00**
response: [list of notification TLVs ($7F60_h$)] 90 00

The CLA2 is as defined in Table 15-3.

The command response contains a list of notifications. The coding of each notification is as defined in Table 15-9.These notifications are appended to the list in the sequential order they are generated. If no notification has been generated during the execution of the previous APDU command, the response shall be empty.

15.3.3 *Security*

15.3.3.1 Cryptography Algorithms

Listing 15-39: Generate Random

```
input: length
output: random
begin
 select random generator compliant with 'AIS31' standard
 random = buffer filled with random bytes for the size indicated in input parameters
 return random
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
end
```

Listing 15-40: Generate Key Pair

```
input: none
output: ecc.PK.x, ecc.PK.y, ecc.PK, ecc.SK
begin
  set curve parameters ← 'ECC NIST P-256' as per [8]
  generate ecc.PK.x, ecc.PK.y, ecc.SK
  ecc.PK = (ecc.PK.x, ecc.PK.y)
  return ecc.PK.x (32 bytes), ecc.PK.y (32 bytes), ecc.SK (32 bytes)
end
```

Listing 15-41: Generate Identifier

```
input: buffer
output: identifier
begin
  hash ← SHA-1(buffer)
  identifier = 6 first bytes of hash
  return identifier
end
```

Listing 15-42: Generate Attestation Signature

```
input: data_fields, private_key
output: signature
begin
  set curve_parameters ← 'ECC NIST P-256' as per [8]
  signature ← ECDSA(curve_parameters, private_key, data_fields) using SHA-256 as per [8]
  return signature (64 bytes)
end
```

Listing 15-43: Verify Attestation Signature

```
input: data_fields, public_key, signature
output: boolean
begin
  hash ← SHA-256(data_fields)
  set curve parameters ← 'ECC NIST P-256' as per [8]
  boolean ← Verify(public_key, hash, signature)
  return boolean (true/false)
end
```

Listing 15-44: Compute Shared Key with Diffie-Hellman

```
input: ePK, eSK, (transaction_identifier)
output: Kdh
begin
  Compute the steps indicated by Section 4.3 of BSI TR-03111 [6] with the following mapping:
  KeyAgreementProtocol : ECKA-DH
  (p, a, b, G, n, h) : ECC NIST P-256 Curve parameters as per [8]
  d̂ : eSK
  P̂: ePK
  S_AB : key agreement output (shared secret point)
  Z_AB : computed from S_AB as per Section 3.1.3 of BSI TR-03111 [6]
  key derivation input : Z_AB
  Key Derivation Function: KDFX9.63 as per Section 4.3.3 of BSI TR-03111 [6]
  K : 256
  H : SHA-256
  SharedInfo : (transaction_identifier)
  KeyData : Kdh (32 bytes)
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
end
```

Listing 15-45: Key Derivation

```
input: input_keying_material, info, keying_material_length
output: derived_keys
begin
  Compute the steps indicated by Section 2 of RFC 5869 [13] with the following mapping:
  Hash : SHA-256
  IKM : input_keying_material
  salt : NULL
  info : info
  L : keying_material_length
  OKM : derived_keys
end
```

Listing 15-46: Secure Channel Command Decryption and Authentication

```
input: encrypted_command_payload, Kmac, Kenc, counter, mac_chaining_in
output: decrypted_command_payload, mac_chaining_out
begin
  Compute the steps indicated by Section 6.2.6 of GPC_SPE_014 [7] with the following mapping:
  CommandDataField - Plain Text : payload
  PaddedCounterBlock : 00000000000000000000000000000000h || counter (1 byte, up to max FFh)
  S-ENC : Kenc
  CipheredCommandDataField : encrypted_command_payload
  CommandDataField - PlainText : decrypted_command_payload

  Compute the steps indicated by Section 6.2.4 of GPC_SPE_014 [7] with the following mapping:
  Variationfromspec : MAC plaintext payload only includes command data field (CLA, INS, P1, P2, Lc, Le are omitted)
  CommandDataField : encrypted_command_payload
  MACChainingValue : mac_chaining_in (16 bytes)
  NewMACChainingValue : mac_chaining_out (16 bytes)
  S-MAC : Kmac
end
```

Listing 15-47: Secure Channel Response Encryption and Authentication

```
input: payload, Krmac, Kenc, counter, mac_chaining_in
output: encrypted_payload, mac
begin
  Compute the steps indicated by Section 6.2.7 of GPC_SPE_014 [7] with the following mapping:
  ResponseDataField – PlainText : payload
  PaddedCounterBlock : 80000000000000000000000000000000h || counter (1 byte, up to max FFh )
  S-ENC : Kenc
  CipheredResponseDataField : encrypted_payload

  Compute the steps indicated by Section 6.2.5 of GPC_SPE_014 [7] with the following mapping:
  Variationfromspec : MAC plaintext payload only includes MAC Chaining Value || encrypted payload, (SW is omitted)
  ResponseDataField : encrypted_payload
  MACChainingValue : mac_chaining_in (16 bytes)
  S-RMAC : Krmac
  R-MAC : mac (8 bytes)
end
```

Listing 15-48: Derive KEenc, KEmac

```
input: KEseed_half, sender_ePK, receiver_ePK, derivation, receiver_PK
output: KEenc, KEmac
begin
  Compute the steps indicated by Section 4.1 of NIST SP800-108r1 [4] with the following mapping:
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

PRF : CMAC as defined by NIST SP800-38B using AES-128 block cipher [5]
h : 128
r : 8
K_{IN} : KEseed_half
Label : 0000000000000000000000_h concatenated with 08_h /12_h for KEenc/KEmac generation
Context : receiver_ePK.x || sender_ePK.x || receiver_PK.x
L : 0080_h (AES-128)
PRF (K_{IN}, *Label* || 00_h || *L* || *i* || *Context*) : concatenation order
K_{OUT} : KEenc(16 bytes) or KEmac(16 bytes)
end

Listing 15-49: Confidential Mailbox Encryption and Authentication

input**:** payload, KEenc, KEmac
output**:** encrypted_payload, mac
begin
Compute the steps indicated by Section 6.2.6 of GPC_SPE_014 [7] with the following mapping:
CommandDataField - PlainText : payload
PaddedCounterBlock : $00000000000000000000000000000001_h$
S-ENC : KEenc
CipheredCommandDataField : encrypted_payload

Compute the steps indicated by Section 6.2.4 of GPC_SPE_014 [7] with the following mapping:
Exception: The header [84_h Ins P1 P2 Lc] shall not be included in the CMAC input,
only MAC Chaining Value and payload are used as input to the CMAC block.
CommandDataField : encrypted_payload
MACChainingValue : $00000000000000000000000000000000_h$
S-MAC : KEmac
C-MAC : mac (8 bytes)
end

Listing 15-50: Compute DeviceCryptogram

input**:** KCmac, endpoint_PK.x, vehicle_PK.x, transaction_identifier, vehicle_identifier
output**:** cryptogram
begin
Compute the steps indicated by Section 4.1 of NIST SP800-108r1 [4] with the following mapping:
PRF : CMAC as defined by NIST SP800-38B using AES-128 block cipher [5]
h : 128
r : 8
K_{IN} : KCmac
Label : 0000000000000000000000_h concatenated with 32_h
Context : vehicle_PK.x || endpoint_PK.x || transaction_identifier || vehicle_identifier
L : 0080_h (AES-128)
PRF(K_{IN}, *Label* || 00\hex || *L* || *i* || *Context*) : concatenation order
K_{OUT} : cryptogram(16 bytes)
end

15.3.4 *Optimization*

This section provides a non-exhaustive list of optimizations that may improve the applet performance. These optimizations depend on platform capabilities. The section is provided for informational purposes only.

1. The ephemeral public key generated during the AUTH0 command could be pre-generated at the end of each transaction (onDeselect). In order to avoid transaction delays in case of communication errors, several pre-generated ephemeral keys could be used until fallback to runtime generation. For transactions over wired interface, ephemeral key

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

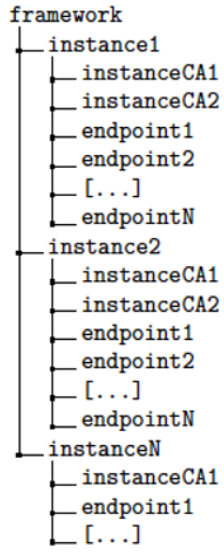
pairs should be generated at runtime instead of from the pool of pre-generated keys because of more relaxed timing constraints.

2. An ephemeral public key generated during the AUTH0 command could be re-used for a limited amount of time (e.g., a few seconds) in order to avoid transaction-time key pair generation on early failed transactions. However, an ephemeral key pair should be deleted after being used in an ECDH operation.
3. The generation of the endpoint signature could be done in parallel with reader processing in the vehicle after sending the AUTH0 response.
4. The next Kpersistent could be generated in parallel with reader processing in the vehicle after sending the AUTH0 response.
5. The Kmac, Kenc, and Krmac could be generated in parallel with reader processing in the vehicle after sending the AUTH0 response.
6. The AUTH1 response wrapped in the secure channel could be prepared in parallel with reader processing in the vehicle after sending the AUTH0 response.

15.4 Helper Method

This section describes groups of APDU commands to be referred as a single functional block in other specifications. The section is provided for informational purposes only.

15.4.1 *Hierarchy*



15.4.1.1 Types

For the purpose of description, variable names are prefixed with the following types:

- **bool_**: A boolean
- **bytes_**: An array of 8-bit bytes
- **uX_**: An unsigned value of X bits
- **obj**: An object

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- **type[]**: A list of objects of the defined type

15.4.1.2 framework.createInstance

Create a new instance of the Digital Key applet. The instance can contain several endpoints.

Inputs:

bytes_AID: The instance AID.

u32_endpointCountMax: The maximum number of endpoints.

u32_instanceCACountMax: The maximum number of Instance CAs.

u32_internalBufferSize: The size of internal buffer.

u32_maxAllocatablePrivateMailboxSize: The maximum private mailbox size of an endpoint.

u32_maxAllocatableConfidentialMailboxSize: The maximum confidential mailbox size of an endpoint.

Outputs:

obj_instance: An object representing the Digital Key applet instance created

Listing 15-51: framework.createInstance Processing

```
input: bytes_AID, u32_endpointCountMax, u32_instanceCACountMaxu32_internalBufferSize
output: obj_instance
begin
  request instance provisioning to the Device OEM Server with input parameters
  create a handle representing the Digital Key applet instance, obj_instance.
  return obj_instance
end
```

15.4.1.3 framework.getInstance

Retrieve an object representing the Digital Key applet instance.

Inputs:

bytes_AID: The instance AID

Outputs:

obj_instance: An object representing the selected instance

Listing 15-52: framework.getInstance Processing

```
input: bytes_AID
output: obj_instance
begin
  create a handle representing the Digital Key applet instance, obj_instance
  return obj_instance
end
```

15.4.1.4 framework.view

Get AID list of installed instance.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Inputs:

n/a

Outputs:

bytes_instanceAID[]: AID list of the instances installed

Listing 15-53: framework.view Processing

```
input: n/a
output: bytes_instanceAID[]
begin
  return installed instance AID list
end
```

15.4.1.5 framework.deleteInstance

Delete a Digital Key applet instance and all related resources. The method described is currently not called in this specification. It is included for completeness and potential usage by other specifications.

Inputs:

obj.instance: Object representing the instance

Listing 15-54: framework.deleteInstance Processing

```
input: obj.instance
output: n/a
begin
  request instance deletion to the Device OEM Server with input parameters
end
```

15.4.1.6 instance.view

Retrieve the instance related information. This method is called over an instance object (obj_instance).

Outputs:

u8_endpointCount: The number of endpoints created.

u8_endpointCountMax: The maximum number of endpoints

u16_internalBufferSize: The size of the internal buffer

bytes_keyIdentifier[]: A list of key identifiers present on the instance

bytes_instanceCAIdentifier[]: A list of Instance CA identifiers present on the instance

Listing 15-55: instance.view Processing

```
input: obj_instance
output: u8_endpointCount,
        u8_endpointCountMax,
        u16_internalBufferSize,
        bytes_keyIdentifier[],
        bytes_instanceCAIdentifier[]
begin
  retrieve instance informations using command in Section 15.3.2.8
  return u8_endpointCount,
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        u8_endpointCountMax,
        u16_internalBufferSize,
        bytes_keyIdentifier[],
        bytes_instanceCAIdentifier[]
end
```

15.4.1.7 instance.createEndpoint

Create an endpoint on the selected instance. This method is called over an instance object (obj_instance).

If bool_onlineCertificate is set, the endpoint creation certificate is signed by the Device OEM CA instead of the Instance CA. The device needs to be able to reach the Device OEM Server for this feature to be available.

Inputs:

bytes_vehicleIdentifier: The vehicle identifier

bytes_endpointIdentifier: The endpoint_identifier

bytes_instanceCAIdentifier: The identifier of the Instance CA to be used for key attestation

u8_optionGroup1: The option group 1

u8_optionGroup2: The option group 2

u16_protocolVersion: The Digital Key applet protocol version

bytes_vehiclePK: The vehicle's public key

bytes_notBefore: DER encoded GeneralizedTime.

bytes_notAfter: DER encoded GeneralizedTime.

bytes_authorizedPK[]: List of public keys of the authorized CAs (n × 65 bytes, n < 6).

u16_confidentialMailboxSize: The confidential_mailbox_size (optional)

u16_privateMailboxSize: The private_mailbox_size (optional)

bytes_keySlot: The key slot (optional)

u32_counterLimit: The signature counter limit (deprecated)

Outputs:

obj_endpoint: An object representing the created endpoint

Listing 15-56: instance.createEndpoint Processing

```
input: input list
output: obj_endpoint
begin
  generate input_data as per Table 15-7
  if input_data > 255
    send the WRITE BUFFER command as described in Section 15.3.2.14 with input input_data
    send the CREATE ENDPOINT command as described Section 15.3.2.4 without payload
  else
    send the CREATE ENDPOINT command as described Section 15.3.2.4 with input_data as payload
  create a handle representing the created endpoint, obj_endpoint
  return obj_endpoint
end
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

15.4.1.8 instance.deleteEndpoint

Delete an endpoint on a specific instance. This method is called over an instance object (obj_instance).

This method is currently not called in this specification. It is included for completeness and potential usage by other specifications.

Inputs:

bytes_keyIdentifier: The key identifier

Listing 15-57: instance.deleteEndpoint Processing

```
input: bytes_keyIdentifier
output: n/a
begin
  send the DELETE ENDPOINT command as described in Section 15.3.2.6 with input_data as payload
  delete endpoint corresponding data on digital key framework
end
```

15.4.1.9 instance.getEndpoint

Obtain a handle on an endpoint. This method is called over an instance object (obj_instance).

Inputs:

bytes_keyIdentifier: The key identifier

Outputs:

obj_endpoint: An object representing the endpoint

Listing 15-58: instance.getEndpoint Processing

```
input: bytes_keyIdentifier
output: obj_endpoint
begin
  create a handle representing the endpoint, obj_endpoint.
  return obj_endpoint
end
```

15.4.1.10 instance.setParameters

Set instance parameters. This method is called over an instance object (obj_instance).

This method is currently not called in this specification. It is included for completeness and potential usage by other specifications.

Inputs:

bool_class_visibility_persistent: If set to false, all endpoints are invisible from the contactless interface. This configuration is persistent across SE reboots.

bool_class_visibility_volatile: If set to false, all endpoints are invisible from the contactless interface. This configuration is valid for the current power on session.

bool_wired_visibility_persistent: If set to false, all endpoints are invisible from the wired interface on AUTH0 command. This configuration is persistent across SE reboots.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

bool_wired_visibility_volatile: If set to false, all endpoints are invisible from the wired interface on AUTH0 command. This configuration is valid for the current power on session.

Listing 15-59: instance.setParameters Processing

```
input: obj_instance, parameters
output: n/a
begin
  set instance parameters using command as described in Section 15.3.2.22
end
```

15.4.1.11 endpoint.setParameters

Set endpoint parameters. This method is called over an endpoint object (obj_endpoint). Note that all inputs listed below in this subsection are optional.

Inputs:

u16_offset_confidential: The offset from which data contained in confidential mailbox is returned in AUTH1.

u8_length_confidential: The length of confidential data returned in AUTH1.

u16_offset_private: The offset from which data contained in private mailbox are returned in AUTH1.

u8_length_private The length of private data returned in AUTH1.

bool_cless_visibility_persistent: If set to false, the endpoint is not visible from the contactless interface. This configuration is persistent across SE reboots.

bool_cless_visibility_volatil: If set to false, the endpoint is not visible from the contactless interface. This configuration is valid for the current power-on session.

bytes_cless_transaction_codes: List of 1-byte transaction codes triggering a user authentication if present in AUTH0 P2 over contactless interface; see Table 9-1 and Table 9-2.

bytes_wired_transaction_codes: List of 1-byte transaction codes triggering a user authentication if present in AUTH0 P2 over contactless interface; see Table 9-1 and Table 9-2.

bool_wired_visibility_persistent: If set to false, the endpoint is not visible from the wired interface on AUTH0 command. This configuration is persistent across SE reboots.

bool_wired_visibility_volatile: If set to false, the endpoint is not visible from the wired interface on AUTH0 command. This configuration is valid for the current power-on session.

Listing 15-60: endpoint.setParameters Processing

```
input: obj_endpoint, parameters
output: n/a
begin
  if bytes_cless_transaction_codes is missing any transaction code present in current configuration
    perform user authentication
  if bytes_wired_transaction_codes is missing any transaction code present in current configuration
     perform user authentication
  set endpoint parameters using command in Section 15.3.2.21
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
end
```

15.4.1.12 endpoint.getCertificate

Retrieve an endpoint certificate. This method is called over an endpoint object (obj_endpoint).

Input:

bool_onlineCertificate request: Attestation created by Device OEM CA (true) or by Instance CA (false).

Output:

bytes_endpointAttestation: An endpoint certificate as per Listing 15-5.

Listing 15-61: endpoint.getCertificate Processing

```
input: bool_onlineCertificate
output: bytes_endpointCertificate
begin
  if bool_onlineCertificate = true
    if endpoint certificate issued by device OEM exists
      return endpoint certificate issued by device OEM stored in digital key framework
  else
    send VIEW command to retrieve endpoint certificate
    return endpoint certificate issued by instance CA
end
```

15.4.1.13 endpoint.terminate

Terminate an endpoint and obtain a termination attestation. This method is called over an endpoint object (obj_endpoint).

Inputs:

bytes_nonce: Termination nonce (16 bytes)

bytes_arbitrary_data: Arbitrary data (optional, 1–40 bytes).

Output:

obj_attestation: A key termination attestation

Listing 15-62: endpoint.terminate Processing

```
input: bytes_nonce, (bytes_arbitrary_data)
output: bytes_terminationAttestation
begin
  send TERMINATE ENDPOINT command to retrieve attestation as described
    In Table 15-19 using bytes_nonce (and bytes_arbitrary_data)
  return bytes_terminationAttestation
end
```

15.4.1.14 endpoint.authorize

Sign an externally received endpoint attestation along with arbitrary data and optionally export confidential mailbox data. This method is called over an endpoint object (obj_endpoint).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 15-60: Receiver Endpoint Key Attestation Signed by Sender

Tag	Length (bytes)	Description	Field is
$7F25_h$	variable	Receiving Endpoint Key Attestation	mandatory
content of Table 15-23			mandatory
$9E_h$	64	Signature with sender endpoint private key over Table 15-23	mandatory

Inputs:

u16_offset: Offset of confidential mailbox to be retrieved (optional)

u16_length: Length of confidential mailbox to be retrieved (optional)

bytes_arbitraryData: Arbitrary data to be signed (optional)

bytes_externalCertificate: External CA certificate of the receiver as per Listing 15-13 (optional)

bytes_instanceCertificate Instance: CA Certificate of the receiver as per Listing 15-16 (optional)

bytes_encryptionKeyAttestation: Encryption key attestation of the receiver as per Table 15-47 (optional)

bytes_endpointCreationCertificate: Endpoint creation certificate of the receiver as per Listing 15-5

Outputs:

bytes_attestation: Attestation signed by the sender endpoint containing arbitrary data and public key of the receiving endpoint

bytes_encryptedData: Data encrypted for the receiver confidential mailbox

bytes_encSenderePK: Encryption public key of the sender

Listing 15-63: endpoint.authorize Processing

```
input: (u16_offset, u16_length, bytes_arbitraryData, bytes_instanceCertificate,
      bytes_encryptionKeyAttestation, bytes_externalCertificate,) bytes_endpointCreationCertificate
output: bytes_attestation, bytes_encryptedData, bytes_encSenderePK
begin
  perform user authentication
  send WRITE BUFFER command in order to send bytes_instanceCertificate,
       bytes_encryptionKeyAttestation, bytes_endpointCreationCertificate
  send AUTHORIZE ENDPOINT command with u16_offset, u16_length, bytes_arbitraryData
  bytes_attestation = prepare table as per  Table 15-60
  bytes_encryptedData = content of tag 4Ah from Table 15-24
  bytes_encSenderePK = content of tag 97h from Table 15-24

  return bytes_attestation, bytes_encryptedData, bytes_encSenderePK
end
```

15.4.1.15 endpoint.createEncryptionKey

Retrieve a single usage encryption key for confidential mailbox writing by other entities. This method is called over an endpoint object (obj_endpoint).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Output:

bytes_attestation: Attestation containing encryption key as per Table 15-47

Listing 15-64: endpoint.createEncryptionKey Processing

```
input: none
output: bytes_attestation
begin
  send CREATE ENCRYPTION KEY command to retrieve attestation as described in Section 15.3.2.17
  return attestation
end
```

15.4.1.16 endpoint.setConfidentialData

Write data in the confidential mailbox. This method is called over an endpoint object (obj_endpoint).

Inputs:

bytes_encsenderepk: Public ephemeral key of the sender

bytes_encdata: Encrypted data to be written in private maibox

u16_offset: Offset of data to be written

Listing 15-65: endpoint.setConfidentialData Processing

```
input: bytes_encsenderpk, bytes_encdata, u16_offset, u16_offset
output: n/a
begin
  send WRITE BUFFER command to write bytes_encsenderpk and  bytes_encdata in internal buffer as described in Section 15.3.2.14
  send SET CONFIDENTIAL DATA command to write in confidential mailbox as described in Section 15.3.2.20
end
```

15.4.1.17 endpoint.getPrivateData

Retrieve data from the private mailbox. This method is called over an endpoint object (obj_endpoint).

Inputs:

u16_offset: Offset of private mailbox to be retrieved

u16_length: Length of private mailbox to be retrieved

Listing 15-66: endpoint.getPrivateData Processing

```
input: u16_offset
output: u16_length
begin
  execute one or multiple GET PRIVATE DATA command as described in Section 15.3.2.18 to retrieve the requested data
end
```

15.4.1.18 endpoint.setPrivateData

Write data in the private mailbox. This method is called over an endpoint object (obj_endpoint).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Inputs:

u16_offset: Offset to start writing in private mailbox

bytes_data: Data to be written in private mailbox

Listing 15-67: endpoint.setPrivateData Processing

```
input: u16_offset, bytes_data
output: n/a
begin
  Send SET PRIVATE DATA command to write in private mailbox as described in Section 15.3.2.19
end
```

15.4.1.19 endpoint.sign

Obtain a signature on arbitrary data. This method is called over an endpoint object (obj_endpoint).

Table 15-61: Arbitrary Data Attestation

Tag	Length (bytes)	Description	Field is
$7F2D_h$	variable	Arbitrary Data Attestation	mandatory
Content of Table 15-56			mandatory
$9E_h$	64	signature with endpoint.SK over fields from Table 15-56	mandatory

Inputs:

bytes_arbitraryData: Arbitrary data to be signed

Outputs:

bytes_attestation: An attestation as described in Table 15-61

Listing 15-68: endpoint.sign Processing

```
input: bytes_arbitraryData
output: bytes_attestation
begin
  perform user authentication if required
  send SIGN command to retrieve attestation as described in Table 15-61
  return attestation
end
```

15.4.1.20 endpoint.auth0

Start the mutual authentication procedure. This method is called over an endpoint object (obj_endpoint).

Inputs:

bytes_epkinput: Ephemeral public key of the vehicle

bytes_transactionIdentifier: Transaction identifier

bytes_vehicleIdentifier: Vehicle identifier

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Outputs:

bytes_epkoutput: Ephemeral public key of the endpoint

u16_protocolversion: Digital Key applet protocol version supported by endpoint

Listing 15-69: endpoint.auth0 Processing

```
input: bytes_epkinput, bytes_transactionIdentifier, bytes_vehicleIdentifier
output: bytes_epkoutput, u16_protocolversion
begin
  send SELECT command using the AID of the obj_endpoint to retrieve u16_protocolversion
  send AUTH0 command as described in Section 15.3.2.9 using bytes_epkinput, bytes_transactionIdentifier,
bytes_vehicleIdentifier
    P1 parameters are Standard Transaction requested, User Authentication not requested and
    retrieve bytes_epkoutput
  return bytes_epkoutput, u16_protocolversion
end
```

15.4.1.21 endpoint.auth1

The second step of the mutual authentication procedure, endpoint.auth0 shall have been executed previously. This method is called over an endpoint object (obj_endpoint).

Input:

bytes_signature: Signature of the third party entity wishing to authenticate

Output:

bytes_encryptedPayload: Encrypted payload

Listing 15-70: endpoint.auth1 Processing

```
input: bytes_signature
output: bytes_encryptedPayload
begin
  send AUTH1 command using bytes_signature as described in Section 15.3.2.10
  return bytes_encryptedPayload
end
```

15.4.1.22 endpoint.exchange

Read and write data in mailboxes upon establishment of a secure channel, provided that endpoint.auth0 (and endpoint.auth1 depending on endpoint configuration) have been successfully executed. This method is called over an endpoint object (obj_endpoint).

Input:

bytes_encryptedInput: Encrypted payload

Output:

bytes_encryptedOutput: Encrypted payload

Listing 15-71: endpoint.exchange Processing

```
input: bytes_encryptedInput
output: bytes_encryptedOutput
begin
  send EXCHANGE command using bytes_encryptedInput as described in Section 15.3.2.15
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
  return bytes_encryptedOutput
end
```

15.4.1.23 endpoint.createRangingKey

Generate a ranging key and make it available to UWB. Shall only be called after successfully receiving AUTH1 Response.

Inputs:

bytes_arbitraryData: arbitrary data.

Listing 15-72: endpoint.createRangingKey processing.

```
input: bytes_arbitraryData
output: none
begin
  send CREATE RANGING KEY command using bytes_arbitraryData as described in Section 15.3.2.26
end
```

15.5 Vehicle Implementation

15.5.1 *Security Guidelines*

The following items describe important implementation guidelines for the vehicle.

1. During the standard transaction, the vehicle shall always verify the endpoint signature (endpoint_sig) before any other operation. A successful verification of this signature is the only way for the vehicle to authenticate the endpoint.
2. During the standard transaction, the vehicle shall never modify its persistent memory before having successfully verified the endpoint signature (endpoint_sig).
3. During the standard transaction, the vehicle shall never write data in the endpoint confidential or private mailbox before having successfully verified the endpoint signature (endpoint_sig).
4. During the standard and fast transactions, the vehicle shall verify that the use of contactless interface is indicated in the response to the AUTH0 and AUTH1 command when the transaction is performed over the NFC interface.[WCC1]

15.5.2 *Optimizations*

The following items describe recommended implementation guidelines for the vehicle.

1. One or several vehicle ephemeral keys can be pre-generated such that a fresh ephemeral key is immediately ready when the next transaction starts.
2. If a vehicle only supports the fast transaction, a random number can be generated instead of an ephemeral key pair.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

16 CERTIFICATE

16.1 General

The PKI model described in this section is based on PKI mechanisms to build a chain of trust down to a single Digital Key. The following certificate chains are defined:

1. Device certificate chain
2. Vehicle certificate chain
3. Owner pairing key attestation chain
4. Key sharing key attestation chain

The objective is to provide full offline capability through the verification chain.

Certificate expiry shall be checked by the KTS. If technical capabilities allow, the Digital Key applet, vehicle, and owner device should also verify certificate expiry.

In this specification, two variants of the SE root of trust certificate chain model are provided.

- **Variant 1**: SE root of trust based on CASD: In this model (see Figure 16-1), the SE root of trust is managed by the SE Root CA implemented by the CASD.
- **Variant 2**: SE root of trust based on DK applet associated security domain: In this model (see Figure 16-2), the SE root of trust is managed using the secure channel of the security domain associated with the DK applet.

The provisioning of the root of trust in the SE is out of scope of this specification.

16.2 Certificates and Relationships

Figure 16-1 provides an overview of the overall certification chains when Variant 1 is implemented.

The Device OEM may choose to directly embed the Vehicle OEM CA Certificate [J] in the device OS as depicted in Figure 16-1. Alternatively, the Device OEM may choose to embed the Vehicle OEM CA Certificate signed by the Device OEM CA [M] in the device OS.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 16-1: Variant 1 Certification Chain Model

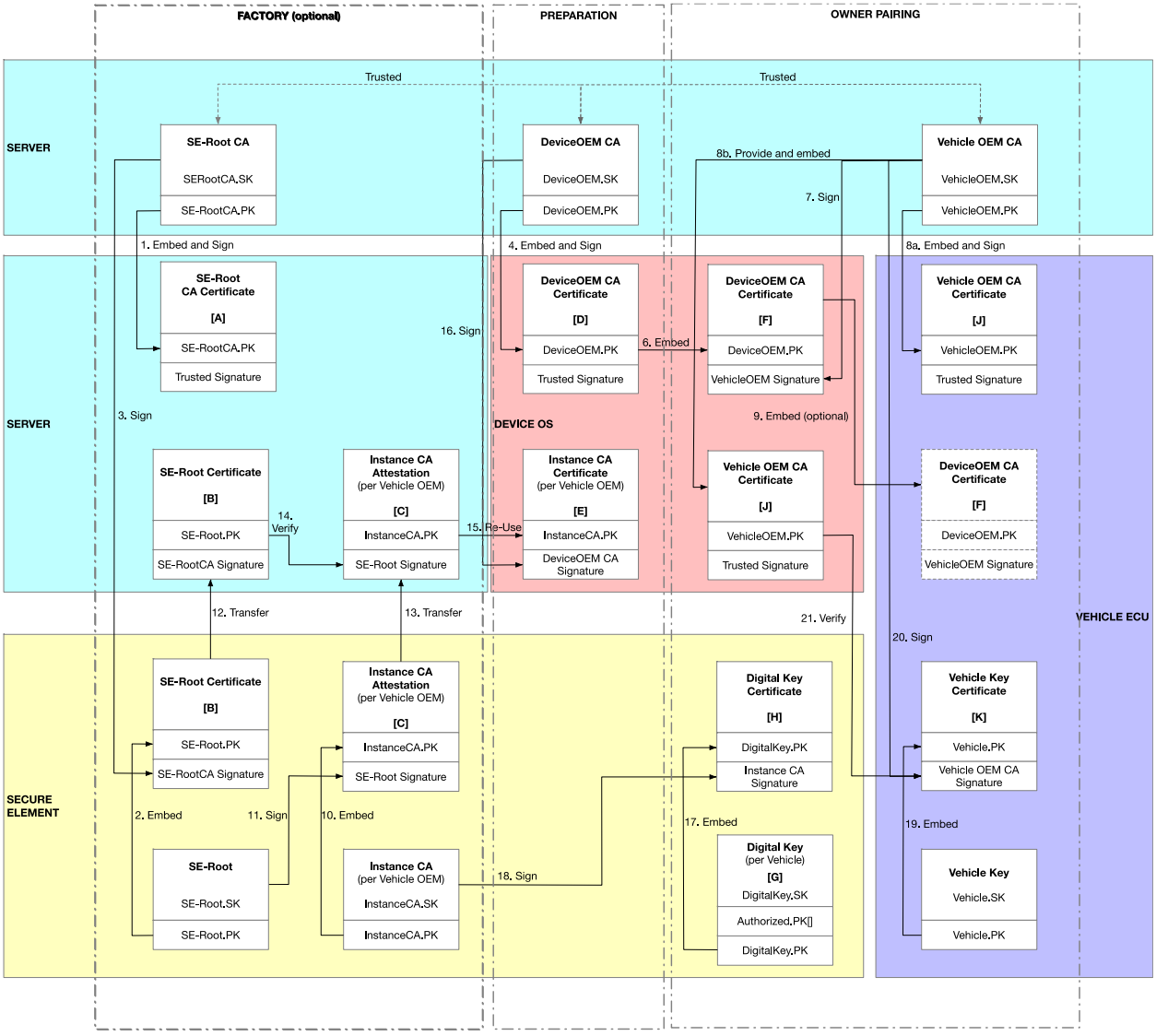
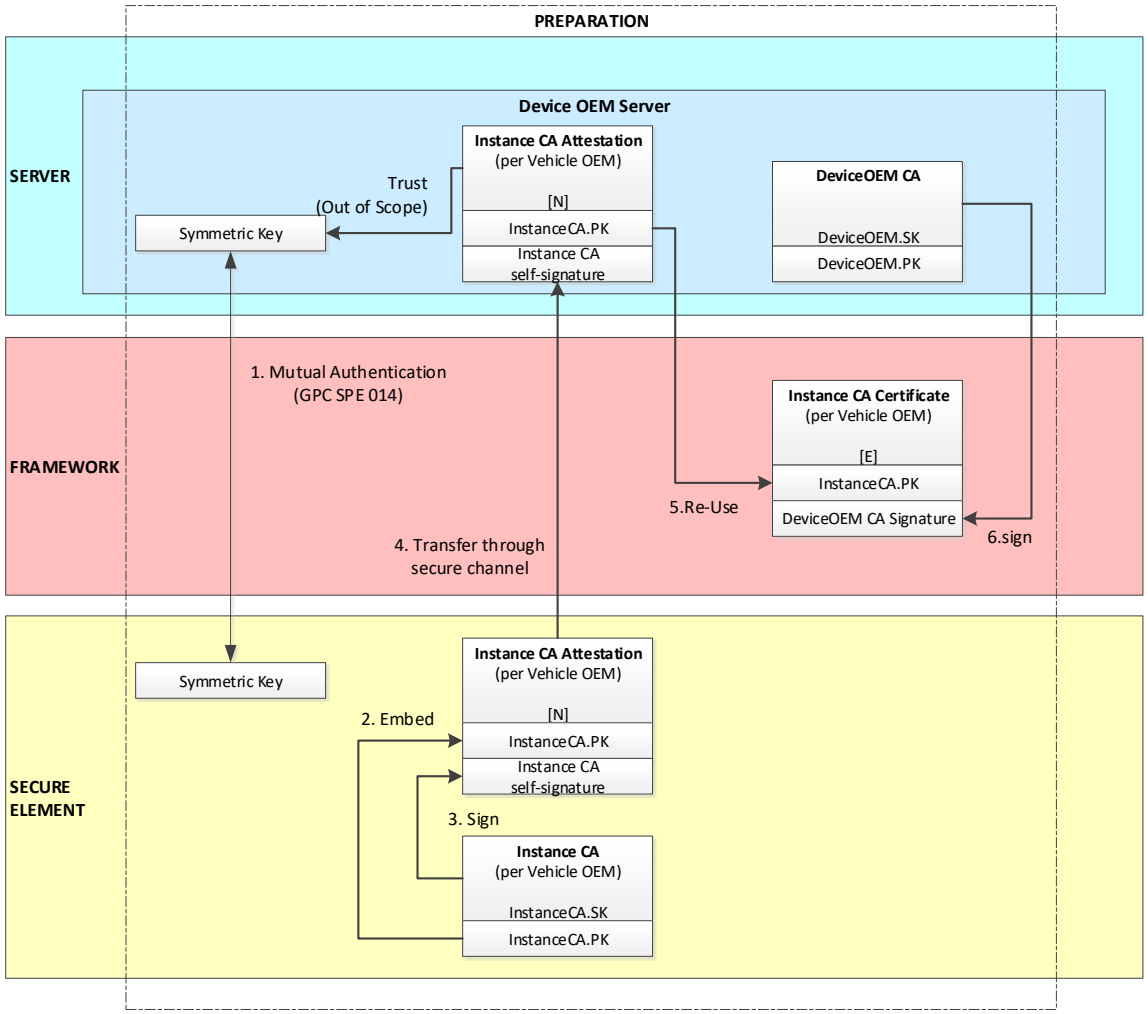


Figure 16-2 provides an adaptation when Variant 2 is implemented.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 16-2: Variant 2 Certification Chain Model



[A] - SE Root CA Certificate – Variant 1
[B] - SE Root Certificate – Variant 1
[C] - Instance CA Attestation (signed by SE Root) per Vehicle OEM – Variant 1
[D] - Device OEM CA Certificate
[E] - Instance CA Certificate (signed by Device OEM CA) per Vehicle OEM
[F] - Device OEM CA Certificate (signed by Vehicle OEM CA)
[G] - Digital Key per Vehicle
[H] - Digital Key Certificate
[J] - Vehicle OEM CA Certificate
[K] - Vehicle Public Key Certificate
[L] - Digital Key Creation Data
[M] - Vehicle OEM CA Certificate (signed by Device OEM CA)
[N] - Instance CA Attestation (self-signed) per Vehicle OEM – Variant 2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

16.2.1 *[A] – SE Root CA Certificate*

This certificate applies to Variant 1.

This certificate is provided by the entity that controls the root key pair in the SE, e.g., the SE manufacturer. It is trusted by the Device OEM CA.

The SE Root CA private key has signed the SE Root Certificates [B] embedded in the SE. Those certificates include immutable identifiers that could allow device tracking.

It is mandatory for every Digital Key-eligible Device OEM to be able to validate SE root signatures from the SEs on its devices.

16.2.2 *[B] – SE Root Certificate*

This certificate applies to Variant 1.

The SE Root Certificate [B] is generated securely and attests to the identity of the SE. [B] is signed by the SE Root CA. [B] is used to verify the Instance CA Attestation [C] created by the Digital Key applet instance. If [C] is successfully verified by [B], the Device OEM signed Instance CA Certificate [E] is issued by the Device OEM Server and stored in the device OS of the corresponding device.

Within the SE, [B] is stored in the CASD. The provisioning of the [B] in the SE is out of scope of this specification.

16.2.3 *[C] – Instance CA Attestation (signed by SE Root) per Vehicle OEM*

This certificate applies to Variant 1.

The Instance CA Attestation [C] is generated after an Instance CA is created by the Digital Key applet instance. [C] includes the public key of the Instance CA and is signed by the private key of the SE root. One Instance CA is created per Vehicle OEM. The Device OEM CA verifies and signs [C] with its private key to issue a new Instance CA Certificate [E] (See Section 16.2.5). [E] is verifiable using the Device OEM CA Certificate [D] and Vehicle OEM signed Device OEM CA Certificate [F].

Within the SE, the private key of the SE root is stored in the CASD. The provisioning of this private key is out of scope of this specification.

The signature of [C] by the private key of the SE root may rely on a proprietary API implemented by the CASD or may rely on the CASD signature service available through AuthoritySignature interface as defined in [15].

16.2.4 *[D] – Device OEM CA Certificate*

This certificate is the Device OEM root of trust for the Digital Key system. This certificate is trusted by the Device OEM and all Vehicle OEMs. The Vehicle OEM receives a CSR from the Device OEM in order to countersign the Device OEM CA Certificate [D]. This then becomes a Vehicle OEM signed Device OEM CA Certificate [F] with the same Device OEM public key.

Depending on the Device OEM's implementation or the Digital Key service deployment model, the Device OEM CA Certificate [D] may be stored in the device OS or in the SE.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

16.2.5 *[E] – Instance CA Certificate (Signed by Device OEM) per Vehicle OEM*

The Device OEM-signed Instance CA Certificate [E] contains the public key of the Instance CA Attestation [C] (Variant 1) or [N] (Variant 2) and is signed by the private key of the Device OEM CA. [E] allows the verification of the Instance CA Attestation using a Device OEM CA Certificate [F]. Using the Device OEM CA signature removes the requirements of otherwise having to trust the SE Root CA. It also anonymizes the static device information to external parties. During the owner paring, [E] is transferred to the vehicle and is used to verify the Digital Key Certificate [H].

The certificate should have a short lifetime and should be renewed often. Revocation may be achieved by stopping the renewal process or through revocation methods (CRL, OCSP), which are out of scope of this specification.

16.2.6 *[F] – Device OEM CA Certificate (Signed by Vehicle OEM)*

The Vehicle OEM CA public key is embedded in the Vehicle OEM CA Certificate [J] in the vehicle (for owner pairing) and in authorized PK in the owner SE (for key sharing). This Vehicle OEM CA public key is used to verify the Vehicle OEM signed Device OEM CA Certificate [F] (note that [F] is also referred to as an “external CA certificate” in Section 15). [F] is then used to verify the Device OEM signed Instance CA Certificate [E]. [F] shall be stored in the device OS of all devices eligible for Digital Keys.

16.2.7 *[G] – Digital Key per vehicle*

[G] is the Digital Key generated by the Digital Key applet. There is one Digital Key per vehicle. During owner pairing, all Digital Key elements are provided by the vehicle and transferred to the device.

16.2.8 *[H] – Digital Key Certificate*

When a Digital Key is created, the applet signs the public key of the Digital Key structure using the corresponding Instance CA’s private key (see Section 4.1) to create the Digital Key Certificate [H]. The certificate is required for owner pairing and key sharing.

16.2.9 *[J] – Vehicle OEM CA Certificate*

This certificate is the Vehicle OEM root of trust for the Digital Key system. [J] is trusted by all Device OEMs and the corresponding Vehicle OEM. The private key corresponding to the Vehicle OEM CA Certificate is used to sign the Device OEM CA Certificate [D] to become [F]. [F] is transferred to the vehicle in order to verify the certificate chain at owner pairing, in which [F] is first verified by [J]. This leads to the Digital Key attestation as shown in Figure 6-7.

The public key of this certificate is also embedded in the owner’s Digital Key (as authorized PK). It is used during key sharing to verify the certificate chain, i.e., to validate the friend’s public key origin.

The Device OEM may embed [J] on the device and use it to validate the authenticity of the vehicle public key (i.e., Vehicle.PK) contained in [K].

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

16.2.10 *[K] – Vehicle Public Key Certificate*

Before accepting the Digital Key Creation Data [L] from the vehicle during owner pairing, the device verifies the origin of the vehicle public key (i.e., Vehicle.PK) using the Vehicle Public Key Certificate [K]. The Vehicle.PK is attested to by the Vehicle OEM CA Certificate [J].

Note: The device uses the Vehicle OEM CA Certificate [J] or Device OEM signed Vehicle OEM CA Certificate [M] to verify [K] as described in Sections 16.2.9 and 16.2.12, respectively, before using [K] to verify Digital Key Creation Data [L].

16.2.11 *[L] – Digital Key Creation Data*

The Digital Key Creation Data [L] (not shown in Figure 16-1) is described in Figure 6-5.

The authorized public keys are the root of trust for key sharing.

Authorized Public Key #1: Vehicle OEM CA PK (mandatory)

This is used as the root for the key sharing validation chain. It cannot be updated once the Digital Key is created.

16.2.12 *[M] – Vehicle OEM CA Certificate (signed by Device OEM)*

The device may use the Device OEM signed Vehicle OEM CA Certificate [M] to validate the authenticity of the vehicle public key (i.e., Vehicle.PK) contained in the Vehicle Public Key Certificate [K]. The root of the validation chain is the Device OEM CA Certificate [D], which is trusted by the device. The public key of the Device OEM is used to validate the signature of [M].

16.2.13 *[N] – Instance CA Attestation (self-signed) per Vehicle OEM*

This attestation applies to Variant 2.

[N] is generated when a new Instance CA is created in the Digital Key applet. [N] contains the public key of the Instance CA in the Digital Key applet and is self-signed by the Instance CA using its private key, instanceCA.SK.

[N] is retrieved and verified by the Device OEM Server, and then the Device OEM Server issues a new Instance CA Certificate [E] by signing the InstanceCA.PK with the DeviceOEM.SK.

When implementing Variant 2, the SE root of trust is ensured by the following mechanisms:

- The Digital Key applet generates the Instance CA Attestation [N].
- The Device OEM Server opens a secure channel with the Digital Key applet to retrieve [N].
- One of the SCPs defined by GlobalPlatform (e.g., SCP03) shall be used. This secure channel shall be set to provide, at a minimum, integrity and data origin authentication on the APDU responses (i.e., requesting R-MAC with or without R-ENCRYPTION in parameter “i” when SCP03 is used).
- The Digital Key applet relies on its security domain for the implementation of the secure channel.
- The secure channel keys used by the security domain (i.e., AES keys when SCP03 is used) are specific to a given SE and are trusted by the Device OEM Server.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- When the Device OEM Server analyzes the response of the Digital Key applet, a successful verification of the R-MAC ensures that the data received over this secure channel has been generated by the expected Digital Key applet in the SE.

Figure 16-2 depicts the adaptation of the certificate chain model for Variant 2.

After successful verification of the Instance CA Attestation [N], the Instance CA Certificate [E] is issued by the Device OEM Server.

16.3 Certificate Size Restrictions

This section defines X.509 certificate size restrictions to ensure compatibility with embedded system (memory) constraints. Maximum certificate sizes are chosen to allow the addition of proprietary, non-critical extensions in future certificate versions.

External CA Certificate, issued by Vehicle OEM

The maximum size of the DER-encoded certificate as per RFC 5280 [3] is 650 bytes.

The length of commonName (CN) fields for subject and issuer is limited to 30 bytes, independently of the chosen value type.

Instance CA Certificate

The maximum size of the DER-encoded certificate as per RFC 5280 [3] is 650 bytes.

The length of commonName (CN) fields for issuer is limited to 30 bytes independently of the chosen value type.

Note: The subject CN field size is limited by the TLV definition in section “endpoint configuration” of the applet description (see Table 15-13).

Endpoint Certificate

The certificate shall include all authorized public keys that the vehicle provides.

In this version of the specification, the owner endpoint certificate shall be created with exactly one authorized public key (enforced by car); the friend endpoint certificate shall be created with exactly zero authorized public keys (enforced by owner phone DK framework).

The applet allows for up to five authorized public keys per endpoint; usage of more than one authorized public key at the secure element level is kept open for future use cases.

The maximum size, of the DER-encoded certificate as per RFC5280 [3], shall be 650 bytes when holding one authorized public key. The maximum size shall be 970 bytes when holding five authorized public keys.

Note: The subject CN field size is limited by the TLV definition in section “endpoint configuration” of the applet description (see Table 15-13).

Encryption Key Attestation

The size of this attestation (not an X.509 certificate) is invariable.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

16.4 Device Certificate Chain

The root of trust is moved from the SE Root to the Device OEM. The Device OEM then trusts and attests to signatures done by the on-device Instance CA.

⟶ attest/verify

⇒ trust

Instance CA chains:

SE Root CA Certificate ⟶ Instance CA Attestation

Device OEM CA ⇒ SE Root CA

Device OEM CA ⇒ Instance CA

Device OEM CA Certificate ⟶ Instance CA Certificate

16.5 Vehicle Certificate Chain

The vehicle public key is attested to by a Vehicle OEM CA Certificate in the device OS. This certificate is trusted by the Device OEM.

⟶ attest /verify

⇒ trust

Device OEM CA ⇒ Vehicle OEM CA

Device OEM embeds Vehicle OEM CA Certificate in device

Vehicle OEM CA Certificate ⟶ Vehicle Public Key Certificate

16.6 Supported Verification Chain

The following chain shall be supported by Digital Key applet for key sharing:

Authorized PK ⟶ External CA certificate ⟶ Instance CA Certificate⟶ endpoint

16.7 Owner Pairing Certificate Chain

At owner pairing, the vehicle verifies the authenticity and the correctness of the key created in the SE. The key properties, including the device public key, are attested to by the Digital Key certification chain, which is rooted in the vehicle.

16.7.1 *Device and Vehicle*

⟶ attest /verify

⇒ trust

Vehicle OEM CA ⇒ Device OEM CA

Device OEM CA ⇒ Instance CA Certificate

Vehicle OEM CA Certificate ⟶ Device OEM CA Certificate ⟶ Instance CA Certificate⟶ Digital Key Certificate

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

16.8 Key Sharing Certificate Chain

Before issuing the Digital Key to the friend, the owner device verifies the authenticity of the public key created in the friend device. The friend device may be a different device brand than the owner device.

It is assumed that the vehicle has provided to the owner SE the Vehicle OEM CA PK, stored safely in the authorized_PK structure within the Digital Key Creation Data [L]. For verification, the friend device provides the Instance CA Certificate [E] signed by the Device OEM and the friend Device OEM CA Certificate [F] signed by the Vehicle OEM.

⟶ attest /verify

⇒ trust

The owner device verifies:

Vehicle OEM CA PK ⟶ friend Device OEM CA Certificate ⟶ friend Instance CA Certificate ⟶ friend Digital Key Certificate

The owner device signs the friend Digital Key Certificate using the owner's private key. The vehicle verifies the owner signature before accepting the friend's public key when presented by the friend device or online through the Vehicle OEM Server:

Owner PK⟶ friend Digital Key Certificate

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17 SERVER-TO-SERVER COMMUNICATIONS AND API

17.1 Introduction

This section describes the API specification for communications between Device OEM Server and Vehicle OEM Server.

Request and response bodies shall be formatted as JSON. The communication protocol used for all interfaces shall be HTTPS. All strings shall be UTF-8 encoded (Unicode Normalization Form C (NFC)).

17.2 API Design

- All APIs shall be implemented as HTTPS POST requests unless otherwise specified.
- Request and response payloads shall consist of simple JSON based data structures that allow for future expansion. The main data types used are array, dictionary, boolean, and string.
- Error codes are specified at three levels:
    - Transport level error codes
        - HTTP error code as described in Section 17.9
    - Standardized application layer error codes
        - Status code field in common response header; values defined in Section 17.9.
    - Detailed error code
        - Optional Sub Status code field in common response header as described in Section 17.10

17.3 Security

Client application shall be hosted on user's devices. Device OEM Server and Vehicle OEM Server shall be hosted in secure data centers.

Server APIs shall be supported only over https. Sensitive data elements, where applicable, shall be protected with additional encryption protocols.

Server APIs shall be supported only over https with mutual authentication, i.e., 2-way TLS.

17.4 URL Scheme

All server services URIs shall conform to the following scheme:

scheme://host:port/{service}/{version}/apiName

where:

- **scheme** = https
- **host:port** = Host name and Port number for the endpoint.
- **service** = Target application that hosts the API endpoint.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- **version** = Version number of the schema. Current major version is v1.
- **apiName** = Name of the API being called

An API version shall be included in the URI for all interfaces. The first supported version is v1. The version shall be incremented by 1 for major API changes or backward incompatible iterations on existing APIs.

17.5 Healthcheck

The healthcheck is a way to determine server availability of Device OEM and Vehicle OEM Servers.

17.6 User Interface

17.6.1 *Description*

User interfaces are created and controlled by Device OEM entities. Each Device OEM can customize the user experience as per its platform.

The Vehicle OEM provides all necessary visual elements for a specific vehicle/model to the Device OEM in an offline process, prior to going live. Details of the visual elements are out of scope of the specification and should be negotiated between the Vehicle OEMs and Device OEMs. An identifier, which can be chosen by the Vehicle OEM, is assigned for UI elements. This identifier is called uiIdentifier.

To ensure uniqueness across Vehicle OEMs, a prefix may be used. The uiIdentifier should be generic and the system should not be able to identify owner or friend using uiIdentifier.

An example of an uiIdentifier for a specific vehicle: “VehicleOEMNameModelYearColor.” The UI elements are stored on the Device OEM Server to ensure the best availability at runtime.

During different types of provisioning (owner pairing, key sharing), the uiIdentifier is provided to the Device OEM Server by the Vehicle OEM Server. The Device OEM may then select the predefined UI elements. It is the responsibility of the Device OEM to display the UI elements appropriate to device-specific parameters.

17.7 Server APIs

17.7.1 *API - Track Key*

This is a generic API offered by the Vehicle OEM Server to track a Digital Key. This API should have idempotent behavior. Idempotency in this case means that if trackKey is called for a keyID that is already registered, the API call shall be successful and return with a usual response. The metadata of the key (e.g., the key status as outlined in Section 17.8.6), as well as the metadata of the system (e.g., the activation options) shall not be idempotent. This information needs to be returned up to date. This API is used in owner pairing and key sharing as described in Section 6.3.4.4 and Section 11.8.5, respectively.

17.7.1.1 End Point

HTTPS POST /{service}/{version}/trackKey

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17.7.1.2 trackKey ()

Parameter	Type	Max Length (bytes)	Description	Required	Domain Version
encryptionCertChain	List of String	4096	Base 64 encoded array of X509 certificates in DER format representing a certificate chain for encrypting uiBundle from Vehicle OEM Server to Device OEM Server. Encryption of trackKeyResponse() is performed using the first certificate in the list. This is a chain of trust established between vehicle and Device OEMs and is out of scope for this specification.	Mandatory	DS-VS
encryptionVersion	String	32	Defines the version of encryption to be used for encryption from Vehicle OEM Server to Device OEM Server. Value is ECIES_v1	Mandatory	DS-VS
keyID	String	128	The unique identifier for a key. Computed as the 160-bit SHA-1 hash of the value of the bit string subjectPublicKey from the keyData (excluding the tag, length, and number of unused bits).	Mandatory	V-OD-FW
keyType	Enum (see Section 17.8.3)	32	OWNER or SHARED. See Section 17.8.3	Mandatory	V-OD-FW
deviceType	Enum	32	Type of device on which key is being provisioned.	Mandatory	D-VS
accountIdHash	String	128	Hash of a Device OEM account id. For the same Device OEM account, accountIdHash should be the same throughout the life of the account.	Mandatory	DS-VS
keyData	Encrypted-DataContainer (see Section 17.8.4)	4096	Encrypted from Device to Vehicle OEM Server. Encrypted using VehicleOEM.Enc.PK as defined in Section 14. Refer to Section 6.3.4.4 (see Table 6-2) and Section 11.10 (see Table 11-19)	Mandatory	Encryption: DS-VS Data: V-OD-FW, D-VS
vehicleOwnerDeviceFrameworkVersionList (V-OD-FW-versionList)	String	512	Contains vehicle owner device framework version V-OD-FW list with V-OD-FW-agreedVersion first	Conditional: Present if request sent by owner device	D-VS
deviceVehicleServerVersionList (D-VS-deviceList)	String	512	Contains the sorted list of all versions the device supports for communication with the vehicle OEM server (D-VS)	Mandatory	D-VS

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17.7.1.3 trackKeyResponse ()

Parameter	Type	Max Length (bytes)	Description	Required	Domain Version
responseHeader	Response-Header Object	1024	The common response header	Mandatory	DS-VS
uiBundle	EncryptedData-Container (see Section 17.8.4)	4096	Specified in Section 17.8.5. The encrypted uiBundle, encrypted from Vehicle OEM Server to Device OEM Server	Mandatory	Encryption: DS-VS Data: V-OD-FW
Vehicle-Mobilization-Data	EncryptedData-Container (see Section 17.8.4)	4096	Encrypted from Vehicle OEM Server to device. Encrypted using Device.Enc.PK as defined in Section 14. Refer to Unencrypted vehicleMobilizationData (see Section 17.8.14)	Conditional. Required if immobilizer token or slot identifier or both are retrieved online.	Encryption: DS-VS Data: V-OD-FW
eventType	enum	64	SHARED_KEY_ADDED from eventType	Conditional. Required only if a tracked owner key has shared keys associated with it, and if owner migrates to new device and tracks new owner key for the vehicle.	Encryption: DS-VS
eventData	Map	2048	eventData used for SHARED_KEY_INFORMATION event.	Conditional. Required only if a tracked owner key has shared keys associated with it and if owner migrates to new device and tracks new owner key for the vehicle.	V-OD-FW

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required	Domain Version
status	Integer	16	Status of the keyId that was sent in the trackKey Request (See Section 17.8.6)	Optional	DS-VS
brand	String	32	The brand of the vehicle.	Mandatory	D-VS
model	String	32	The model of the vehicle.	Mandatory	D-VS
serverSupportedVersions (D-VS-serverList)	String	512	Contains the sorted list of all versions the vehicle OEM server supports for communication with the device (D-VS). First version is the agreed version selected by the server, then all other versions are listed in descending order	Mandatory	D-VS

All version lists shall be coded as hex strings, identical to tags $5A_h$, $5B_h$ and $5C_h$ in sections 5.1.1 and 5.1.2.

All version lists shall be sorted in descending order. Where indicated, the first version is the agreed version, then the remaining list shall be sorted in descending order.

17.7.1.4 Sample trackKey

```
HTTP POST /{service}/{version}/trackKey
x-requestId: 80BCCEF8F59127A6D1BB5CFEB534D95
x-device-oemId: device OEM A
{
    "encryptionCertChain"    : [ "MIIBx...pglH","nMIIBx...pglH"  ],
    "encryptionVersion"      : "ECIES_v1",
    "keyID"                  : "394858302023",
    "keyType"                : "OWNER | SHARED",
    "deviceType"             : "PHONE",
    "accountIDHash"          :
"C8645830202EFEB53427A6D75F15C85E78A5195307E2351858349AB9",
    "keyData"                : {
        "version"            : "ECIES_v1",
        "ephemeralPublicKey" :
"04613197827d91806d630bc4adff44686b012316eb03825f2d6587ffd58d32f4522ada80cc9
3679e1a316dc0729ebf8172fd41f0c0c1bdda01126f1a6186b2a008",
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        "publicKeyHash"      :
"d6e3bbc95bf11d533677a1aa052d7caef29fd76a02e43e3609579966168d7d19",
        "data"               :
"5GRH7/ecb85HFsTalxn3IdeT7ARtfFZn2AuMft1p...IkcchjFBLJTGm9qvJtxK/3EdWob+iFS9
FwHeKSjq8MxdNgQ1rj4fq6CzQfY8O9",
        "vehicleOwnerDeviceFrameworkVersionList" : "02000100",
        "deviceVehicleServerVersionList"    : "02000201"
     }


}
```

17.7.1.5 Sample trackKeyResponse

```
HTTP 200
x-responseId                 : 80BCCEF8F59127A6D1BB5CFEB534D95

{
    "responseHeader"         : {
        "statusCode"         : "200"
    },
    “status”                 : 1,
    "uiBundle"               : {
        "version"            : "ECIES_v1",
        "ephemeralPublicKey" :
"806d6306b04613197827d91012316eb03bc4adff4468825f2d6587fada80cc93679e1afd58d
32f4522172fd41f0c0c1b1126f1316dc0729ebf8a6186b2a008dda0",
        "publicKeyHash"      :
"1d533677a1aa052dd6e3bbc95bf17cae02e4f29fd76a8d7d193e360957996616",
        "data"               :
"lxn3IdeT5b85HFsTa7ARtfFZGRH7/ecn2AuMft1p...K/3EdWob+iFS9m9qvJtxCzQfY8O9FwHe
KSjq8MxdNgQ1rj4fq6IkcchjFBLJTG"
    }
    "eventType"              : "SHARED_KEY_INFORMATION",
    "eventData"              : {
      "sharedKeys"   : "3",
      "shareableKeys": "7",
      "keyValidTo"   : "2021-02-19T23:05:17.462Z",
      "keyValidFrom" : "2019-02-19T23:05:17.462Z",
      "reason"       : "shared key data is needed",
      "encryptedData": {
         "version"           : "ECIES_v1",
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
          "ephemeralPublicKey":
"0982340982347508934763728327423734689234823575089328327423734689234823476372832742374237",
          "publicKeyHash"     :
"509823475023734689234823850982347502373468923943468923482347637283",
          "data"              :
"ydST1ZG5Q3cflzfM1qjjwpmOmWQHta...SBBiIAGK3PUbdE2IF/MmPefY9bsH"
          }
       },
    "brand"           : "xyz",
    "model"           : "abc"
       "vehicleMobilizationData"      : {
         "version"           : "ECIES_v1",
         "ephemeralPublicKey":
"0982340982347508934763728327423734689234823575089328327423734689234823476372832742374237",
         "publicKeyHash"     :
"509823475023734689234823850982347502373468923943468923482347637283",
         "data"              :
"ydST1ZG5Q3cflzfM1qjjwpmOmWQHta...SBBiIAGK3PUbdE2IF/MmPefY9bsH"
         },
    "serverSupportedVersions” : "02000100"
}
```

Sample of Unencrypted uiBundle data

```
{
    "ktsSignature"      :
"3045022017B5FBBCF2F475F15C85E78A519530792211CC4DA50EC0BBDCC42C7C28FA780E022100D20D47E8624C851473602A02BFCD4F4C0DB02C2FA1A4A1E56A9530019CE289D4",
    "keyValidFrom"      : "2019-03-06T18:00:49.239Z",
    "keyValidTo"        : "2022-03-05T18:00:49.239Z",
    "sharedKeys"        : "3",
    "shareableKeys"     : "7",
    "uiIdentifier"     : "35E3406EFDA511E88EB2F2801F1B9FD1",
    "sharingPasswordRequired"       : false,
    "entitlement"                   : {
        "value"                     : 0,
        "description"               : "full",
        "longDescription"           : "full access and drive capabilities"
    },
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    "supportedEntitlements"         : {
        "entitlements" : [
            {
                "value"             : 0,
                "description"       : "full",
                "longDescription"   : "full access and drive capabilities"
            },
            {
              "value"             : 1,
              "description"       : "accessOnly",
              "longDescription"   : "only vehicle access , no drive
capability"
            },
            {
              "value"             : 2,
              "description"       : "accessAndDriveRestricted",
              "longDescription"   : "access and drive with restrictions"
            },
            {
                "value"             : 3,
              "description"       : "vehicleDelivery",
              "longDescription"   : "vehicle delivery profile"
            },
            {
              "value"             : 4,
              "description"       : "valet",
              "longDescription"   : "valet parking key"
            },
            {
                "value"             : 5,
                "description"       : "vehicleService",
              "longDescription"   : "service key"
            },
            {
                "value"             : 6,
                "description"       : "custom entitlement description",
              "longDescription"   : "custom entitlement long description"
            }
        ]
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    }
  }
}
```

Sample of Unencrypted vehicleMobilizationData (See section 17.8.14)

```
{
    "slotIdentifier"           : "11223344",
    "confidentialMailboxData"  : "3ZCVBjxM2 ... Cwk265HB+sB9",
    "ephemeralPublicKey"       : "
04F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446CAD19D2010
62DD1C7CB0BB293BF16A4BEFB2ED500977E7197E01F26906E39B5F"
}
```

Note: Please refer to Section 17.11 for encryption scheme and other details.

17.7.2 *Manage Key*

This is a generic API offered by the Device OEM Server and Vehicle OEM Server to manage the lifecycle of a Digital Key. This API is used in key sharing and key termination as described in Section 11 and Section 13, respectively.

17.7.2.1 End Point

HTTP POST /{service}/{version}/manageKey

17.7.2.2 manageKey()

Parameter	Type	Max Length (bytes)	Description	Required	Domain Version
keyID	String	128	The unique identifier for a key.	Mandatory	V-OD-FW
action	enum	128	Action of manage key (See Section 17.8.7).	Mandatory	DS-VS
termination-Attestation	String	4096	Termination attestation of a key when owner Digital Key or shared Digital Key is terminated on own device. Signed by DigitalKeyx.SK, as defined in Section 14.	Conditional: provided by owner/friend device whenever possible	D-VS

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required	Domain Version
deviceRemote-Termination-Request	EncryptedData-Container (see Section 17.8.4)	4096	Remote termination request for termination of a remote key when shared key is terminated from owner device. Signed by DigitalKeyx.SK as defined in Section 14.	Conditional: provided by owner device	D-VS
serverRemote-Termination-Request	String (see Section 14.2)	4096	Remote termination request for termination of a remote key when owner key or shared key is terminated from Vehicle OEM Server. Signed by VehicleOEM.Sig.SK as defined in Section 14.	Conditional: provided by KTS	D-VS
vehicleOEM-Proprietary-Data	String	4096	Subsection of the private mailbox of a Digital Key containing the Vehicle OEM proprietary data when owner or shared key is terminated on the device (local or remote termination).	Conditional: Provided by the device whenever possible	n/a

17.7.2.3 manageKeyResponse()

Parameter	Type	Max Length (bytes)	Description	Required	Domain Version
Response-Header	Response-Header Object	1024	The common response header	Mandatory	DS-VS
status	integer	16	Status of the keyId that was sent in the manageKey request (See Section 17.8.6)	Optional	DS-VS
Encrypted-Data	Encrypted-Data-Container	4096	Decrypted contents format is specified in Section 17.8.10. Used to send the latest status of all shared keys to owner device.	Conditional. Required if manageKey called with deviceRemote-TerminationRequest or if	D-VS

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required	Domain Version
	(See Section 17.8.4)		When this response is a result of owner deleting shared key, the decrypted value shall contain the respective shared key with a status for InTermination	requested action is SUSPEND or RESUME.	

17.7.2.4 Sample manageKeyRequest for owner/friend deleting own key

```
HTTP POST /{service}/{version}/manageKey
x-requestId : 80BCCEF8F59127A6D1BB5CFEB534D95
x-device-oemId: Device OEM A
{
    "keyID"                    :   "4938205039594",
    "action"                   :   "TERMINATE",
    "terminationAttestation"   :
"5CFEB534D8F5CEFA6D1B780BCB5CFEB534D95912CEFA6D1B780BCB5CFE"
"vehicleOEMProprietaryData"    : "5CEFA6D1B780BCB5CFEB534D"
}
```

Note: For terminationAttestation data structure, see Section 15.3.2.5, Table 15-18, and Table 15-19.

17.7.2.5 Sample manageKeyRequest for owner deleting Shared Key

```
HTTP POST /{service}/{version}/manageKey
x-requestId: 80BCCEF8F59127A6D1BB5CFEB534D95
x-device-oemId: device OEM A
{
    "keyID"                    :   "4938205039594",
    "action"                   :   "TERMINATE",
    "deviceRemoteTerminationRequest"  :   {
        "version"                  : "ECIES_v1",
        "ephemeralPublicKey"       :
"16eb03825f2d97827d91806d630bc4adff44686587ffd58d32f4522ada80cc01126f1a6186b
2a00893679e1abf8172fd41f0c0c1bdda3b0046131123616dc0729e",
        "publicKeyHash"            :
"677a1aa052d7caef29d6e3bbc95bf11d43e3609579966168d7d533fd76a02e19",
        "data"                     :
"kE8/Rs9sNo4olc6s3Qej7mXhAtDPcp ...1Bm/r7bKNX6m365vnIDEDc"
    }
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
}
```

Note: For deviceRemoteTerminationRequest data structure, see Section 14.2.

17.7.2.6 Sample manageKeyRequest for vehicle deleting Shared Key

```
HTTP POST /{service}/{version}/manageKey
x-requestId: 80BCCEF8F59127A6D1BB5CFEB534D95
x-device-oemId: device OEM A
{
    "keyID"                    :   "4938205039594",
    "action"                   :   "TERMINATE",
  "serverRemoteTerminationRequest":
"3AEBF536D2F5CEFA6D1C780BCA9CEBF534E90734CEFA2D1B780ACB8EBC"
}
```

Note: For serverRemoteTerminationRequest data structure, see Section 14.2.

17.7.2.7 Sample manageKeyResponse for Owner/Friend deleting own key

```
HTTP 200
x-responseId : 80BCCEF8F59127A6D1BB5CFEB534D95

{
    "responseHeader" : {
        "statusCode" : "200"
    },
    "status"          : 10
}
```

17.7.2.8 Sample manageKeyResponse for Owner deleting shared key

```
HTTP 200
x-responseId : 80BCCEF8F59127A6D1BB5CFEB534D95

{
    "responseHeader" : {
        "statusCode" : "200"
    },
    "status"          : 1,
   "encryptedData"  : {
        "version"            : "ECIES_v1",
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        "ephemeralPublicKey" :
"09823409823475089347637283274237346892348235750893283274237346892348234763728237463728348235",
        "publicKeyHash"      :
"509823475023734689234823850982347502373468923943468923482347637283274237346892348235",
        "data"               :
"ydST1ZG5Q3cflzfM1qjjwpmOmWQHta...SBBiIAGK3PUbdE2IF/MmPefY9bsH”
      }
}
```

Sample unencrypted encryptedData value, specified at Unencrypted Event Notification Data (See Section 17.8.10)

```
{
    "keyID"          : "483729239475C864E4F",
    "version"        : "1.0",
    "sharedKeysData" : [
        {
            "keyID"            : "292348379464E4F75C8",
            "keyConfiguration" : {
                "friendlyName" : "Jon's Phone",
                "rights"       : 0,
                "keyValidFrom" : "2021-02-19T23:05:17.462+0000",
                "keyValidTo"   : "2021-02-19T23:05:17.462+0000"
            },
            "status"           : 1,
            "groupIdentifier"  : "E24A5978-66BB-4810-A29B-E4B193D5076F",
            "deviceType"       : "PHONE"
        },
        {
            "keyID"            : "498277430125B8F85F1",
            "keyConfiguration" : {
                "friendlyName" : "Jon's Watch",
                "rights"       : 0,
                "keyValidFrom" : "2021-02-19T23:05:17.462+0000",
                "keyValidTo"   : "2021-02-19T23:05:17.462+0000"
            },
            "status"           : 1,
            "groupIdentifier"  : "E24A5978-66BB-4810-A29B-E4B193D5076F",
            "deviceType"       : "WATCH"
        },
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        {
            "keyID"            : "276459836197C6A34D2",
            "keyConfiguration" : {
                "friendlyName" : "Jane's Phone",
                "rights"       : 0,
                "keyValidFrom" : "2021-02-19T23:05:17.462+0000",
                "keyValidTo"   : "2021-02-19T23:05:17.462+0000"
            },
            "status"           : 15,
            "groupIdentifier"  : "D308745E-35E3-4FCD-B121-D159457B79C6",
            "deviceType"       : "PHONE"
        }
    ]
}
```

17.7.3 *API – eventNotification*

This is a generic API offered by the Device OEM Server and the Vehicle OEM Server to communicate different events on a Digital Key.

17.7.3.1 End Point

HTTP POST /{service}/{version}/eventNotification

17.7.3.2 EventNotification ()

Parameter	Type	Max Length (bytes)	Description	Required
keyID	String	128	The receiver's unique key identifier.	Conditional
eventType	enum	64	The type of the event. Specified in Section 17.8.8.	Mandatory
eventData	Map	2048	Contextual based on event type. Specified in Section 17.8.9.	Mandatory

17.7.3.3 EventNotificationResponse ()

Parameter	Type	Max Length (bytes)	Description	Required
response-Header	Response-Header	1024	The common response header	Mandatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required
status	Integer	16	Status of the keyId from the eventNotification request (see Section 17.8.6)	Optional (if response is sent from vehicle OEM)

17.7.3.4 Sample eventNotificationRequest to send owner key information to owner Device OEM Server

```
HTTP POST /partner/v1/eventNotification
x-requestId : 80BCCEF8F59127A6D1BB5CFEB534D95
x-vehicle-oemId: Vehicle OEM A
{
    "keyID"             : "483729239475C864E4F",
    "eventType"         : "SUBSCRIPTION_CHANGED",
    "eventData"         : {
        "sharedKeys"        : "3",
        "shareableKeys"     : "7",
        "status"           : 1,
        "keyValidTo"        : "2021-02-19T23:05:17.462Z",
        "keyValidFrom"      : "2019-02-19T23:05:17.462Z",
        "reason"            : "key subscription is changed"
    }
}
```

17.7.3.5 Sample eventNotificationRequest to send shared key information to owner Device OEM Server

```
HTTP POST /partner/v1/eventNotification
x-requestId   : 80BCCEF8F59127A6D1BB5CFEB534D95
x-vehicle-oemId       : Vehicle OEM A
{
    "keyID"                 : "483729239475C864E4F",
    "eventType"             : "SHARED_KEY_INFORMATION",
    "eventData"             : {
        "sharedKeys"        : "3",
        "shareableKeys"     : "7",
        "status"            : 1,
        "keyValidTo"        : "2021-02-19T23:05:17.462Z",
        "keyValidFrom"      : "2019-02-19T23:05:17.462Z",
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        "reason"             : "shared key is added",
              "encryptedData"        : {
              "version"              : "ECIES_v1",
              "ephemeralPublicKey"   :
"0982340982347508934763728327423734689234823575089328327423734689234823476372832742374237346892348235",
              "publicKeyHash"        :
"509823475023734689234823850982347502373468923943468923482347637283278",
              "data"                 :
"ydST1ZG5Q3cflzfM1qjjwpmOmWQHta...SBBiIAGK3PUbdE2IF/MmPefY9bsH"
      }
      }
}
```

Sample of unencrypted encryptedData value, specified at Unencrypted Event Notification Data (See Section 17.8.10):

```
{
   "keyID": "483729239475C864E4F",
   "version": "1",
   "sharedKeysData": [
       {
                      "keyID"                : "292348379464E4F75C8",
                      "keyConfiguration"     : {
                             “friendlyName”  : “Jon’s Phone”,
                             “rights”        : 0,
                             “keyValidFrom”  : “2021-02-19T23:05:17.462Z”,
                             “keyValidTo”    : “2021-02-19T23:05:17.462Z”
                             },
                      "status"               : 1
                      "groupIdentifier"      : “E24A5978-66BB-4810-A29B-
E4B193D5076F”,
                      “deviceType”           : “PHONE”
       },

       {
              "keyID"                : "379292464E4F75C8348379",
              "keyConfiguration"     : {
                             “friendlyName”  : “Jane’s Phone”,
                             “rights”        : 0,
                             “keyValidFrom”  : “2021-02-19T23:05:17.462Z”,
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
                              “keyValidTo”    : “2021-02-19T23:05:17.462Z”
                              },
              "status"                : 6
                     "groupIdentifier"      : “D308745E-35E3-4FCD-B121-
D159457B79C6”,
                     “deviceType”           : “PHONE”
       }
   ]
}
```

17.7.3.6 Sample eventNotificationRequest to friend Device OEM Server

```
HTTP POST /partner/v1/eventNotification
x-requestId : 80BCCEF8F59127A6D1BB5CFEB534D95
x-vehicle-oemId: Vehicle OEM A
{
    "keyID"                : "483729239475C864E4F",
    "eventType"            : "IN_TERMINATION",
    "eventData"            : {
        "keyValidTo"       : "2021-02-19T23:05:17.462Z",
        "keyValidFrom"     : "2019-02-19T23:05:17.462Z",
        “status”           : 15,
        "reason"           : "shared key being terminated"
    }
}
```

17.7.3.7 Sample eventNotificationRequest to send model field to Owner or Friend Device Server

```
HTTP POST /partner/v1/eventNotification
x-requestId : 80BCCEF8F59127A6D1BB5CFEB534D95
x-vehicle-oemId: Vehicle OEM Name
{
    "keyID"     : "483729239475C864E4F",
    "eventType" : "UI_ELEMENTS_UPDATED",
    "eventData" : {
        "keyValidTo"   : "2021-02-19T23:05:17.462+0000",
        "keyValidFrom" : "2019-02-19T23:05:17.462+0000",
        “status”       : 1,
        "reason"       : "Model is updated",
        "uiElements"   : {
            "model" : "Model Name"
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        }
    }
}
```

17.7.3.8 Sample eventNotificationResponse

```
HTTP 200
x-responseId : 80BCCEF8F59127A6D1BB5CFEB534D95
{
    "responseHeader" : {
        "statusCode" : "200"
    }
}
```

17.7.4 *API – Recover Key Data*

This is a generic API offered by the Vehicle OEM Server to retrieve metadata of active keys. This API should only be called if the device loses some of the information listed in this section and shall not be used for regular sync. Vehicle OEM specific request policy may apply (out of scope of this CR). It is recommended that device OEM Server should store metadata of active keys to avoid calling the API in order to reduce the load on Vehicle OEM Server.

Attention: key metadata returned using this API might differ from what was known to the device OEM at the time of trackKey. If metadata returned is indeed different, the device OEM shall use the returned values from then on.

17.7.4.1 End Point

HTTP POST /{service}/{version}/recoverKeyData

17.7.4.2 recoverKeyData()

Parameter	Type	Max Length (bytes)	Description	Required
encryption-CertChain	List of String	4096	Base 64 encoded array of X509 certificates in DER format representing a certificate chain for encrypting uiBundle from Vehicle OEM Server to Device OEM Server. Encryption of recoverKeyDataResponse()	Mandatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required
			is performed using the first certificate in the list. This is a chain of trust established between vehicle and Device OEMs and is out of scope for this specification.	
encryption-Version	String	32	Defines the version of encryption to be used for encryption from Vehicle OEM Server to Device OEM Server. Value is ECIES_v1	Mandatory
keyID	String	128	The unique identifier for a key.	Mandatory

1 *17.7.4.3 recoverKeyDataResponse()*

Parameter	Type	Max Length (bytes)	Description	Required
response-Header	Response-Header Object	1024	The common response header	Mandatory
uiBundle	Encrypted Data-Container (see Section 17.8.4)	4096	Specified in Section 17.8.5. The encrypted uiBundle, encrypted from Vehicle OEM Server to Device OEM Server. Attention: KtsSignature is not part of this object.	Mandatory
eventType	enum	64	SHARED_KEY_INFORMATION from eventType	Conditional. Required only if the key id in the request is an owner key and has shared keys associated with it.
				Conditional. Required only if the key id in

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required
eventData	Map	2048	eventData used for SHARED_KEY_INFORMATION event.	the request is an owner key and has shared keys associated with it.
status	Integer	16	Status of the keyId from the request (see Section 17.8.6)	Mandatory
brand	String	32	The brand of the vehicle.	Mandatory
model	String	32	The model of the vehicle.	Mandatory
instanceFreshness	Integer	1	Owner Device Instance CA freshness requirement for Key Sharing in hours. No freshness check required if tag is absent or 0.	Conditional. Required only if the key id in the request is an owner key.

17.7.4.4 Sample recoverKeyData

```
HTTP POST /{service}/{version}/ recoverKeyData
x-requestId: 80BCCEF8F59127A6D1BB5CFEB534D95
x-device-oemId: device OEM A
{
    “encryptionCertChain”    : [ “MIIBx…pglH”,”nMIIBx…pglH”  ],
    “encryptionVersion”      : “ECIES_v1”,
    “keyID”                  : “394858302023”
}
```

17.7.4.5 Sample recoverKeyDataResponse

```
HTTP 200
x-responseId                 : 80BCCEF8F59127A6D1BB5CFEB534D95

{
    "responseHeader"         : {
        "statusCode"         : "200"
    },
    "uiBundle"               : {
        "version"            : "ECIES_v1",
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        "ephemeralPublicKey" :
"806d6306b04613197827d91012316eb03bc4adff4468825f2d6587fada80cc93679e1afd58d
32f4522172fd41f0c0c1b1126f1316dc0729ebf8a6186b2a008dda0",
        "publicKeyHash"      :
"1d533677a1aa052dd6e3bbc95bf17cae02e4f29fd76a8d7d193e360957996616",
        "data"               :
"lxn3IdeT5b85HFsTa7ARtfFZGRH7/ecn2AuMft1p...K/3EdWob+iFS9m9qvJtxCzQfY8O9FwHe
KSjq8MxdNgQ1rj4fq6IkcchjFBLJTG"
    }
    "eventType"              : "SHARED_KEY_INFORMATION",
    "eventData"              : {
      "sharedKeys"    : "3",
      "shareableKeys": "7",
      "keyValidTo"    : "2021-02-19T23:05:17.462Z",
      "keyValidFrom"  : "2019-02-19T23:05:17.462Z",
      "reason"        : "shared key data is needed",
      "encryptedData": {
         "version"           : "ECIES_v1",
         "ephemeralPublicKey":
"0982340982347508934763728327423734689234823575089328327423734689234823476372
832742373468923482235",
         " publicKeyHash"    :
"5098234750237346892348238509823475023734689239434689234823476372832 7",
         "data"              :
"ydST1ZG5Q3cflzfM1qjjwpmOmWQHta...SBBiIAGK3PUbdE2IF/MmPefY9bsH"
         }
      },
    "status"                : 1,
    "brand"                 : "xyz",
    "model"                 : "abc",
    "instanceFreshness"     : 48
}
```

Sample of Unencrypted uiBundle data

```
{
    "keyValidFrom"      : "2019-03-06T18:00:49.239Z",
    "keyValidTo"        : "2022-03-05T18:00:49.239Z",
    "sharedKeys"        : "3",
    "shareableKeys"     : "7",
    "uiIdentifier"     : "35E3406EFDA511E88EB2F2801F1B9FD1",
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    "sharingPasswordRequired"       : false,
    "entitlement"                   : {
        "value"                     : 0,
        "description"               : "full",
        "longDescription"           : "full access and drive capabilities"
    },
    "supportedEntitlements"         : {
        "entitlements" : [
            {
                "value"             : 0,
                "description"       : "full",
                "longDescription"   : "full access and drive capabilities"
            },
            {
              "value"             : 1,
              "description"       : "accessOnly",
              "longDescription"   : "only vehicle access , no drive
capability"
            },
            {
              "value"             : 2,
              "description"       : "accessAndDriveRestricted",
              "longDescription"   : "access and drive with restrictions"
            },
            {
                "value"             : 3,
              "description"       : "vehicleDelivery",
              "longDescription"   : "vehicle delivery profile"
            },
            {
              "value"             : 4,
              "description"       : "valet",
              "longDescription"   : "valet parking key"
            },
            {
                "value"             : 5,
                "description"       : "vehicleService",
              "longDescription"   : "service key"
            },
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
            {
                "value"             : 6,
                "description"       : "custom entitlement description",
              "longDescription"   : "custom entitlement long description"
            }
        ]
    }
  }
}
```

Note: Please refer to Section 17.11 for encryption scheme and other details.

17.7.5 *API – Healthcheck*

This is a generic API offered by the Device OEM Server and the Vehicle OEM Server to determine the availability of the corresponding server. This API does not require any headers.

17.7.5.1 End Point

HTTP GET /{service}/healthcheck

17.7.5.2 Input

N/A

17.7.5.3 Output

N/A

17.7.5.4 Sample healthcheckRequest

HTTP GET /partner/healthcheck

17.7.5.5 Sample healthcheckResponse

HTTP 200

17.7.6 *API – versionUpdate*

This API is used for the exchange and update of vehicle and device versions.

Upon an update (either device or vehicle), Device, Vehicle and Servers determine the highest common supported version (D-VS, V-OD-FW) for subsequent communication.

If after the update of a vehicle SW, the list of supported versions affecting the device has been modified, this API should be called by the vehicle OEM server. In this case, the vehicle OEM determines the timing of when this API is called.

If after the update of a device SW, the list of supported versions affecting the vehicle or vehicle server has been modified, this API should be called by the device OEM server. In this case, the device OEM determines the timing of when this API is called.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Note that due to the asynchronous call chain (vehicle or device might be offline) the version agreement occurs on both sides without providing the agreed version back to the caller.

If a a versionUpdate API call is received by an OEM server which supports Domain Version DS-VS 2.0 of higher, but the associated target entity (e.g., vehicle) supports only Domain Version V-OD-FW v1.0, that OEM server shall respond with success to the versionUpdate API call.

All version lists within JSON shall coded as hex strings, identical to tags $5A_h$ and $5B_h$ as outlined in Sections 5.1.1 and 5.1.2.

The following sections lists all scenarios in which the API is used and provides the list of parameters to be included.

17.7.6.1 *Call Scenario: Vehicle Software Update impacting V-OD-FW*

- Called by VS for every owner key for the vehicle at the called Device OEM server. Refer to Figure 2-7.
- Call parameters
  - *keyID*
  - *deviceVehicleServerVersionList* (D-VS-serverList, should not change)
  - *vehicleOwnerDeviceFrameworkVehicleVersionList* (V-OD-FW-vehicleList)
- Response parameters
  - success/failure

Listing 17-1: Content of versionUpdate API on Vehicle Software Update

```
{
   "updateScenario": "VEHICLE_SW_UPDATE",
   "keyID": "3ED46B38D85BCC0A1F31C9BAC509116BC09CC3D2",
   "vehicleOwnerDeviceFrameworkVehicleVersionList": "02000201",
   "deviceVehicleServerVersionList": “02000201”
}
```

17.7.6.2 Call Scenario: Device SW update

- Called by DS (device OEM server) for every key on the updated device at the called Vehicle OEM server
- Parameters
  - *keyID*
  - *deviceVehicleServerVersionList* (D-VS-deviceList)
  - *vehicleOwnerDeviceFrameworkDeviceVersionList* (V-OD-FW-deviceList)
- Response parameters
  - success/failure

Refer to Figure 2-5

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Listing 17-2: Content of versionUpdate API after Device Software Update

```
{
   "updateScenario": "DEVICE_SW_UPDATE",
   "keyID": "3ED46B38D85BCC0A1F31C9BAC509116BC09CC3D2",
   "deviceVehicleServerVersionList": "02000201",
   "vehicleOwnerDeviceFrameworkDeviceVersionList": "02000201"
}
```

17.7.6.3 End Point

HTTP POST /{service}/{version}/versionUpdate

17.7.6.4 versionUpdate()

The caller of this API can provide a keyID and associated version lists as described in individual call scenarios above. An example of a full HTTP request is provided below

```
HTTP POST /{service}/{version}/versionUpdate
x-requestId: 80BCCEF8F59127A6D1BB5CFEB534D95
x-device-oemId: device OEM A
{
      "updateScenario": "DEVICE_SW_UPDATE",
      "keyID": "3ED46B38D85BCC0A1F31C9BAC509116BC09CC3D2",
      "deviceVehicleServerVersionList": "2021",
      "vehicleOwnerDeviceFrameworkDeviceVersionList": "202224"
}
```

Parameter	Type	Max Length (bytes)	Description	Required
updateScenario	String	64	Identifies the scenario from the above list. Possible identifiers are: VEHICLE_SW_UPDATE (17.7.6.1), DEVICE_SW_UPDATE (17.7.6.2)	Mandatory
keyID	String	128	keyID for which the version information apply.	Conditional. To be used in scenarios 17.7.6.1, 17.7.6.2
deviceVehicleServer VersionList	String	512	Contains the sorted list of all versions the device/vehicle supports for communication with the vehicle/device server (D-VS, D-VS).	Conditional. Present for DEVICE_SW_UP DATE (17.7.6.2) and

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required
				VEHICLE_SW_UPDATE(17.7.6.1)
vehicleOwnerDeviceFrameworkVehicleVersionList	String	512	Contains the sorted list of all versions the vehicle supports for communication with the device (V-OD-FW).	Conditional. Present for VEHICLE_SW_UPDATE (17.7.6.1), for owner device only
vehicleOwnerDeviceFrameworkDeviceVersionList	String	512	Contains the sorted list of all versions the device supports for communication with the vehicle (V-OD-FW).	Conditional. Present for DEVICE_SW_UPDATE (17.7.6.2), for owner device only

17.7.6.5 versionUpdateResponse()

versionUpdateResponse does not contain version information.

Response body example:

```
HTTP 200
x-responseId                : 80BCCEF8F59127A6D1BB5CFEB534D95
{
    "responseHeader"        : {
        "statusCode"        : "200"
    }
}
```

17.8 Structure Definitions

17.8.1 *Request Headers*

All requests by the Vehicle OEM Server and Device OEM Server server shall contain HTTP request headers as defined below

value	Description
x-requestId	All requests by Device OEM Servers and Vehicle OEM Servers shall have an HTTP header “x-requestId”. The value shall be a UUID of length 36 containing hyphens.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

value	Description
x-timestamp	All requests by Device OEM Servers and Vehicle OEM Servers shall have an HTTP header “x-timestamp”. Timestamp should be specified in Unix timestamp. The value shall be given in milliseconds. The value shall not be modified in case of request retry. This is used to identify outdated requests in case of retry.
x-device-oemId	All requests by Device OEM Servers shall have an HTTP header “x-device-oemId”. Value is the name of the Device OEM.
x-vehicle-oemId	All requests by Vehicle OEM Servers shall have an HTTP header “x-vehicle-oemId”. Value is the Vehicle OEM identifier (See Table 2-1 of [35]).
x-responseId	The corresponding response to the API shall have HTTP header “x-responseId”, which should echo the value of “requestId” in the request header. This is used to identify the request associated to the response for a particular API request and response pair. The x-responseId shall be sent by Device OEM Server and Vehicle OEM Server.
x-device-oem-host	The FQDN of the Device OEM’s endpoint. Vehicle OEM uses this information to initiate communication to Device OEM for a specific keyID. The “x-device-oem-host” shall be sent by the Device OEM Server
x-user-identifier	Device OEM server may include a HTTP header “x-user-identifier”. Value is • generated by the device OEM server and stored by Vehicle OEM servers • a AES-GCM encrypted base64 encoded string representation of a hash value generated using a pseudonymized device side userId. • up to 128 bytes long If included by Device OEM server, then all server API messages transmitted to device OEM server by Vehicle OEM server shall include the value.

17.8.2 *Response Headers*

All responses by the Vehicle OEM Server and Device OEM Server shall contain header data of type “ResponseHeader” within the payload. Contents keys are defined below

Parameter	Type	Max Length (bytes)	Description	Required
statusCode	String	64	Status code used to indicate an error, or “200” for success.	Mandatory
subStatus-Code	String	64	Conveys failure codes from downstream entities or for more granular conveyance of specific error conditions.	Optional
Status-Message	String	256	Not parsed programmatically. Example: “Downstream system offline”	Optional

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17.8.3 *Key Type*

Key	Type	Max Length (bytes)	Description	Required
OWNER	String	32	owner key type	N/A
SHARED	String	32	shared key type	N/A

17.8.4 *EncryptedDataContainer*

Parameter	Type	Max Length (bytes)	Description	Required
version	String	10	Identifier for the algorithm specified in this document. Example: “ECIES_v1”.	Mandatory
Ephemeral-PublicKey	String	Variable	Hex-encoded sender's ephemeral public key. This should be from an ephemeral key pair generated for a single message. Format for encoding the sender public key: concat(0x04, x, y) (This will be 65 bytes total for a key generated based on named curve secp256r1 - 1.2.840.10045.3.1.7)	Mandatory
Public-KeyHash	String	Variable	Hex-encoded recipients’ key agreement public key fingerprint. Fingerprint generation algorithm: sha256Hash(toUncompressedRawECPublicKey-Format(recipientKAPublicKey)). SHA-256 hash of the uncompressed key agreement public key (concat(0x04, x, y)) of the receiving system used in ECIES_v1 Key Agreement	Mandatory
data	String	Variable	Base64-encoded encrypted data. data=base64Encode(encrypt(unencryptedData, symmetricEncryptionParams))	Mandatory

17.8.5 *Unencrypted uiBundle*

Parameter	Type	Max Length (bytes)	Description	Required
kts-Signature	String	256	Signature created by key tracking server (KTS); see Section 6.3.4.4.	Conditional. Required only in trackKeyResponse

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required
uiIdentifier	String	32	Identifier referencing visual elements that are required to create user interface for Digital Key. The definition and encoding of the string value is agreed between vehicle and device OEMs individually and is out of scope of this specification.	Mandatory
sharing-Password-Required	boolean	16	Indicates if sharing password is required	Conditional. Required if activationRequired is not present.
activationRequired	boolean	16	Indicates that a 2nd factor is required to activate a shared key	Optional. Transmitted in trackKeyResponse
activationOptions	See Section 17.8.20	N/A	Describes vehicle OEM-defined activation options for an inactive shared key	Conditional: Required if vehicle supports at least an activation option.
approvedSharingMethods	See Section 17.8.21	N/A	List of SharingMethodGroups accepted by vehicle OEM as secure sharing methods that do not require 2nd factor activation	Optional, only for owner keys
entitlement	See Section 17.8.13	N/A	Describes entitlement supported for the key being tracked, maps to the “profiles” field as described in Section 11.6, Table 11-5.	Mandatory
supported-Entitlements	See Section 17.8.15	N/A	Describes list of all the vehicle supported entitlements. Entitlement in the supportedEntitlements list maps to the “profiles” field as described in Section 11.6, Table 11-5	Conditional. Required for owner keys

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required
keyValid-From	String	32	Key validity start date. Time should be specified in UTC ISO-8601 Format yyyy-MM-dd'T'HH:mm:ss.SSSZ.	Required for shared key Optional for owner key
keyValidTo	String	32	Key validity end date. Time should be specified in UTC ISO-8601 Format yyyy-MM-dd'T'HH:mm:ss.SSSZ.	Required for shared key Optional for owner key
sharedKeys	String	4	Number of keys shared by the owner.	Conditional. Required for owner keys
shareable-Keys	String	4	Number of keys available for sharing.	Conditional. Required for owner keys

17.8.6 *Digital Key Status*

A Digital Key can have the states in the following table. Throughout the lifetime of a Digital Car Key its state will change according to the following:

- After creation the key is in the state InActive. If no further activation is necessary, the key is in Active state right after creation.
- After activation (vehicle OEM policy applies) the key is in the status Active.
- If the key is set to lost mode by the user, its state changes to Suspend: Lost Mode. (Attention: This does not need to be supported by all vehicle OEMs.)
- If the key is deleted by a party that is not able to delete the key right away (fade out period applies), it changes its status into InTermination.
- As soon as the key is deleted by either the keyholding device itself or by the vehicle, so that the key is not usable anymore, the state of the key changes into its final state deleted (Terminated: By Issuer). The list of SharedKeys (e.g., returned in a eventNotificationData (encryptedData)) shall not contain any key in “Terminated”(10) state.

Status ID	Status	Type	Max Length (bytes)	Description	Required
1	Active	Integer	32	Key is active and is usable.	N/A

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Status ID	Status	Type	Max Length (bytes)	Description	Required
7	Suspend: Lost Mode	Integer	32	Key is suspended due to lost mode and is unusable.	N/A
10	Terminated: By Issuer	Integer	32	Key is terminated by issuer (Vehicle OEM Server) and is unusable.	N/A
14	InActive	Integer	32	Key is inactive	N/A
15	InTermination	Integer	32	Key is being terminated.	N/A

17.8.7 *Action for manageKey*

Action	Type	Max Length (bytes)	Description	Required
TERMINATE	enum	32	Terminate the key	N/A
SUSPEND[1]	enum	32	Suspend the key	N/A
RESUME[1]	enum	32	Resume the key	N/A

[1]: SUSPEND and RESUME actions are only applicable in manageKey API calls from Device OEM server to Vehicle OEM Server.

17.8.8 *Event Type of Notification*

Name	Description	Reason for the event to be triggered
IN_TERMINATION	Key is being terminated	When owner or shared key is being terminated from local/remote source, this notification is sent from the Vehicle OEM Server to owner or friend Device OEM Server. Digital Key is not terminated yet. This eventType is not necessary if KEY_INFORMATION is used.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Name	Description	Reason for the event to be triggered
SUSPENDED	Key is suspended	When the vehicle suspends owner or shared key, this notification is sent from the Vehicle OEM Server to owner or friend Device OEM Server. Digital Key is suspended. This eventType is not necessary if KEY_INFORMATION is used.
RESUMED	Key is resumed	When the vehicle resumes the owner or shared key, this notification is sent from the Vehicle OEM Server to owner or friend Device OEM Server. Digital Key is resumed. This eventType is not necessary if KEY_INFORMATION is used.
KEY_INFORMATION	Key Data is needed	This eventType should be used whenever there is information of the own key sent with no regards of the actual action.
SHARED_KEY_INFORMATION	Shared Key Data is needed	This eventType should be used whenever there is information of shared keys sent to the owner with no regards of the actual action.
SHARED_KEY_-IN_TERMINATION	Shared key is being terminated	When shared key is being terminated from local/remote source, this notification is sent from the Vehicle OEM Server to the owner Device OEM Server. Digital Key is not terminated yet. This eventType is not necessary if SHARED_KEY_INFORMATION is used.
SHARED_KEY_-TERMINATED	Shared key is terminated	When shared key is terminated (after fade-out period), this notification is sent from the Vehicle OEM Server to the owner Device OEM Server. Digital Key is terminated. This eventType is not necessary if SHARED_KEY_INFORMATION is used.
SHARED_KEY_-SUSPENDED	Shared key is suspended	When the vehicle suspends a shared key, this notification is sent from the Vehicle OEM Server to the owner Device OEM Server. Shared key is suspended. This eventType is not necessary if SHARED_KEY_INFORMATION is used.
SHARED_KEY_-RESUMED	Shared key is resumed	When the vehicle resumes a shared key, this notification is sent from the Vehicle OEM Server to the owner Device OEM Server. Shared key is resumed.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Name	Description	Reason for the event to be triggered
		This eventType is not necessary if SHARED_KEY_INFORMATION is used.
RESUMING	Digital Key is being resumed	When the vehicle is resuming all access, this notification is sent to all devices with this Digital Key for the vehicle. Digital Key is not yet resumed on the vehicle. This eventType is not necessary if SHARED_KEY_INFORMATION is used.
SHARED_KEY_-ADDED	Shared key is tracked and activated in device	When a shared key has been successfully tracked in the vehicle OEM server and activated on the device (key is in status “active”), this notification is sent from the Vehicle OEM Server to the owner Device OEM Server.
SUBSCRIPTION_-CHANGED	Key subscription changed	When Digital Key service subscription changes in Vehicle OEM Server, this notification is sent from the Vehicle OEM Server to owner or friend Device OEM Server. This may occur, e.g., when the Digital Key service is renewed.
SHARING_-PASSWORD_-REQUIRED	Password is required for key sharing	When the policy for the sharing password changes in Vehicle OEM Server, this notification is sent from the Vehicle OEM Server to owner or friend Device OEM Server.
SHARING_POLICY_CHANGED	Sharing policy for shared key required	When the policy for the approved sharing methods changes in Vehicle OEM Server, this notification is sent from the Vehicle OEM Server to owner Device OEM Server (if no proprietary method is used).
ACTIVATION_OPTIONS_CHANGED	Activation options for vehicle have changed	When the available activation options change in the vehicle (e.g. SW update), this notification is sent from the Vehicle OEM Server to owner Device OEM Server.
ENTITLEMENTS_-UPDATED	Entitlements are updated	When entitlements supported by the vehicle are updated in the Vehicle OEM Server and vehicle. These entitlements are then available to the owner for future key sharing operations; existing shared keys are not affected.
UI_ELEMENTS_-UPDATED	UI elements are updated	When UI elements such as model is updated in Vehicle OEM Server and propagated to device for existing Digital Key.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Name	Description	Reason for the event to be triggered
VEHICLE_-ATTESTATION	Vehicle Attestation Notification	When vehicle transmits an attestation to the owner device in case a friend enters a correct sharing password in the vehicle. Implementation of this event type is optional for device OEM.

17.8.9 *Event Data of Event Notification*

Name	Type	Description	Required
status	Integer	State of the Digital Key (see Section 17.8.6)	Conditional. Only if the digital key already exists (e.g., not during CREATE_SHARED_KEY)
keyValidFrom	String	Key validity start date. Time should be specified in UTC ISO-8601 Format yyyy-MM-dd'T'HH:mm:ss.SSSZ.	Conditional. Only if the digital key already exists (e.g., not during CREATE_SHARED_KEY)
keyValidTo	String	Key validity end date. Time should be specfied in UTC ISO-8601 Format yyyy-MM-dd'T'HH:mm:ss.SSSZ.	Conditional. Only if the digital key already exists (e.g., not during CREATE_SHARED_KEY)
reason	String	Reason that triggered event. For informational purposes only; not displayed in UI.	Mandatory
uiElements	UIElements (see Section17.8.17)	Updated UI elements	Conditional. Required if eventType is UI_ELEMENTS_-UPDATED
sharedKeys	String	Number of keys shared by the owner	Conditional. Required if eventNotification is sent to owner Device OEM Server and if it contains shared key information
shareableKeys	String	Number of keys available for sharing	Conditional. Required if eventNotification is sent to owner Device OEM Server and if it contains shared key informatio

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Name	Type	Description	Required
resume-Attestation	String	Resume attestation for resuming a Digital Key as defined in Section 13.4.3	Conditional. Required only for eventNotification, eventType RESUMING
encryptedData	Encrypted-Data-Container (see Section 17.8.4)	Used to send shared key information to owner device. Encrypted from Vehicle OEM Server to Device. Encrypted using Devx.Enc.PK" as defined in Section 14.1.	Conditional. Required if eventNotification is sent to owner Device OEM Server and if it contains shared key information
supported-Entitlements	See Section 17.8.15)	Describes list of all the vehicle supported entitlements	Conditional. Required if entitlements are updated in Vehicle OEM Server and vehicle
activationOptions	See Section 17.8.18	Describes activation options for an inactive shared key	Conditional: Required for ACTIVATION_OPTIONS_CHANGED event
approvedSharingMethods	See Section 17.8.19	List of SharingMethods accepted by car OEM as approved sharing method	Conditional: Required for SHARING_POLICY_CHANGED event

17.8.10 *Unencrypted Event Notification Data*

Parameter	Type	Max Length (bytes)	Description	Required
keyID	String	128	The unique identifier for an owner key	Mandatory
Shared KeysData	List of Shared Key (see Section 17.8.11)	N/A	List of all associated shared keys	Mandatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Type	Max Length (bytes)	Description	Required
vehicle-Attestation	String	N/A	For a description of the TLV encoded attestation, refer to Table 11-22	Conditional. Required if vehicle OEM server supports this optional feature as described in Section 11.11 and is sending VEHICLE_ATTESTATION notification when a friend has entered the correct sharing password in the vehicle.

17.8.11 *Shared Key*

Parameter	Type	Max Length (bytes)	Description	Required
keyID	String	128	The unique identifier for a shared key	Mandatory
Key-Configuration	See Section 17.8.12	4096	Shared key configuration	Mandatory
status	Integer	16	Status of a shared key (see Section 17.8.6)	Mandatory
groupIdentifier	String	4096	UUID per friend per given vehicle, generated by Vehicle OEM. This field is a unique identifier over (vehicle, accountIdHash).	Mandatory
deviceType	enum	32	Type of device on which key is being provisioned.	Mandatory

17.8.12 *Key Configuration*

Key	Type	Max Length	Description	Required
friendlyName	String	30	The friendly friend's name to be displayed to the user	Mandatory
rights	Integer	16	Integer indicating which access profile the shared key has; see Table 11-5.	Mandatory

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

keyValidFrom	String	32	Key validity start date. Time should be specified in UTC ISO-8601 Format yyyy-MM-dd'T'HH:mm:ss.SSSZ.	Mandatory
keyValidTo	String	32	Key validity end date. Time should be specified in UTC ISO-8601 Format yyyy-MM-dd'T'HH:mm:ss.SSSZ.	Mandatory

17.8.13 *Entitlement*

Entitlement is also referred to as access profile as described in Section 11.6, Table 11-5.

Name	Type	Description	Required
value	Integer	Value of the access profile of the shared key; see Table 11-5.	Mandatory
description	String	A short English description used to look up localized description strings for this Digital Key on the device, e.g., accessOnly.	Mandatory
long-Description	String	A long form description of the entitlement. This should be a one-sentence description of the entitlement (aka access profile), e.g., only vehicle access, no drive capability.	Mandatory

17.8.14 *Unencrypted vehicleMobilizationData Fields*

Parameter	Type	Max Length (bytes)	Description	Required
slotIdentifier	String	8	The slot identifier to assign for the key Table 6-3 and Table 11-20)	Conditional. Required if slot identifier is retrieved online
confidential-MailboxData	String	Variable	Encrypted confidential mailbox data (see Table 6-3 and Table 11-20)	Conditional. Required if immobilizer token is retrieved online
ephemeral-PublicKey	String	Variable	Ephemeral encryption public key of the Vehicle OEM server (see Table 6-3 and Table 11-20)	Conditional. Required if immobilizer token is retrieved online

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17.8.15 *Supported Entitlements*

Name	Type	Description	Required
entitlements	List of entitlement	Describes the list of all vehicle supported entitlements	Mandatory

17.8.16 *Device Type*

Key	Type	Max Length (bytes)	Description	Required
PHONE	String	32	phone	N/A
WATCH	String	32	watch	N/A
OTHER	String	32	other	N/A

17.8.17 *UI Elements*

Name	Type	Description	Required
model	String	The vehicle model description to be updated.	Yes

17.8.18 *Activation Options*

Activation options should be indicated on owner and friend device UI during the key sharing process. The friend can then choose any supported activation option.

Activation options are an extension and should be used instead of the “sharing password required” indication in the uiBundle (see section 17.8.6).

Name	Type	Description	Required
activationOptionsList	List of strings	Provides the list of all supported 2nd factor activation options for activation of a shared key (17.8.21)	Conditional: Required only if more options than the if sharing password and/or other activation options are supported by the vehicle
sharingPasswordLength	String	String representation of Vehicle OEM specific SHARING_PASSWORD_LENGTH as per Appendix B.1	Optional: To be included if SHARING_PASSWORD_LENGTH needs to be updated

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The following activation options may be supported by the vehicle. Device UIs shall support all listed options.

Activation options	Description	Required
sharingPasswordActivation	Friend key can be activated by entering the sharing password in the vehicle UI	Optional
ownerKeyActivation	Friend key can be activated by presenting the owner key to the vehicle when the friend key is used for the first time	Optional
friendKeyActivation	Friend key can be activated by presenting another active friend key to the vehicle when the friend key is used for the first time	Optional
keyFobActivation	Friend key can be activated by presenting the key fob to the vehicle when the friend key is used for the first time	Optional

The text to describe each activation option in the UI is defined by the device OEM.

17.8.19 *Supported approved sharing methods*

Name	Type	Description	Required
approved SharingMethods	List of strings	Describes the list of all approved sharing method group identifiers as described in Table 11-11(list of string encoded hex values of tags 61h, without tag and length of the subtags, e.g., [“4002000041020001”,”4002000041020001”])	Optional

17.9 HTTP Status Codes

Status Code	Description	Caller should retry
200	Request was received and processed	No
301	A resource is moved permanently, retry on new location	Yes
302	A resource is moved temporarily, retry on new location	Yes

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Status Code	Description	Caller should retry
400	Bad Request	No
404	Malformed or unknown URI	No
500	Internal server error	Yes
503	Service unavailable	Yes

The default “Caller should retry” policy of a given status code is shown in this section, which shall be followed unless otherwise specified with a sub-status code described in Section 17.10.

17.10 Sub Status Codes

Status Code	Description	HTTP Status Code	Caller should retry	Note
50000	Unexpected error	500	Yes	Shall not be used unless a more appropriate sub-status code has not been defined
50001	Downstream systems unavailable	500	Yes	Shall be used when systems on callee side are temporarily unavailable for unknown reasons and a later retry will likely succeed. There will be no immediate retries
50002	OEM scheduled downtime	503	No	Shall be used when systems on callee side are temporarily unavailable for unknown reasons and a later retry will likely succeed. There will be no immediate retries
50009	No associated user	404	Yes	Shall be used when there is no user account registered.
50110	Unknown key id supplied	404	No	Shall be used when the referred key id is not known (or has already been deleted) on callee side. If possible a more detailed sub-status code shall be used to refer to either owner or friend key id.
50111	Unknown owner Digital Key identifier supplied	404	No	Shall be used when the owner key id is not known (or has already been deleted) on callee side

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Status Code	Description	HTTP Status Code	Caller should retry	Note
50112	Unknown Digital Key identifier of a friend supplied	404	No	Shall be used when the friend's key id is not known (or has already been deleted) on callee side
50113	Maximum sharing key limit reached	500	No	Shall be used when the received key tracking request refers to a key that would exceed the vehicle's capacity of storing keys (or hit a business limit). This error should be avoided by restricting the number of shared keys to the allowed limit on device side.
50114	Cryptography error occurred	400/ 500	No	Shall be used when any cryptographic element (signature) in the supplied data is wrong.
50115	Feature not supported	500	No	Shall be used when the caller data refers to a feature that is not supported on callee side
50116	Conflicting Key Slot Identifier	400	N/A	Shall be used when there is a mismatch between the key id and the expected slot identifier in the trackKey request (e.g., slot identifier was used with another key id before)
50117	Version not supported	400/ 500	No	Shall be used when the version sent in the request is not supported by the recipient
50119	Invalid or expired Sharee Instance CA Certificate	400	No	Shall be used when Instance CA Certificate from Sharee has expired or is invalid
50120	Invalid or expired Device OEM Root Certificate	400	No	Shall be used when root certificate of the Device OEM has expired or is invalid
50121	Invalid or expired Vehicle OEM Root Certificate	400	No	Shall be used when root certificate of the Vehicle OEM has expired or is invalid

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Status Code	Description	HTTP Status Code	Caller should retry	Note
50122	Owner Pairing not possible: vehicle is not ready	400	No	Shall be used in case the vehicle is unmapped from account after owner pairing but before key tracking
50123	Friend Tracking failed: Owner signature not as expected	400	No	Shall be used when owner signature is invalid
50124	Friend tracking failed: No owner key found or in invalid state	400	No	Shall be used when owner key is in invalid state

Wherever the specification allows multiple HTTP Status Codes, caller (e.g., Vehicle OEM) server should choose the HTTP Status Code that fits the specific error (5xx for errors within Vehicle OEM server, 4xx for errors on caller's side, e.g., Device OEM server).

Status code 50000 Unexpected error - shall not be used unless a more appropriate sub-status code has not been defined. In this case concerned parties are encouraged to file Change Requests against this specification to add a dedicated sub-status code for the new error and not use "unexpected error" anymore. Callers shall accept even unknown sub-status codes and map them to "unexpected error" internally to avoid the use of versioning to introduce new error codes.

The caller shall retry any request satisfying one or more of the following criteria:

- Received an HTTP status code that allows a retry (see table above)
- Request timed out or otherwise failed at the network level such that no HTTP response was received or processed

Requests may be retried a maximum of three times. Retry attempts shall adhere to the following guidelines:

- First retry shall occur a pseudorandom amount of time Δt after the original attempt, where Δt is between 0s–1s inclusive.
- Second retry shall occur a pseudorandom amount of time Δt after the first retry attempt, where Δt is between 1s–5s inclusive.
- Third retry shall occur a pseudorandom amount of time Δt after the second retry attempt, where Δt is between 5s–30s inclusive.

17.11 ECC Encryption for Device and Server

This section defines the specifics of the ECIES_v1 algorithm for ECDHE encryption and decryption.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

17.11.1 *Frame/Packet Contents*

This following information shall be included in the frame/packet sent from the encrypting/sending party to the decrypting/receiving party:

1. Algorithm Identifier

   The identifier for the algorithm specified is “ECIES_v1”.

2. Sender Key Agreement Public Key

   The sender's key agreement public key shall be from an ephemeral key pair generated for a single message.

   The format for encoding the sender public key: concat(0x04, x, y)

   (This will be 65 bytes total for a key generated based on named curve secp256r1 and its OID value 1.2.840.10045.3.1.7.

3. Recipient Key Agreement Public Key Fingerprint

   The recipient’s key agreement public key fingerprint. The fingerprint shall be generated using algorithm: SHA-256 hash of the uncompressed key agreement public key (i.e., concat(0x04, x, y)) of the receiving party used in ECIES_v1 Key Agreement.

fingerprint = sha256Hash(toUncompressedRawECPublicKeyFormat(recipientKAPublicKey));

4. Encrypted Data

   The binary encrypted data.

17.11.2 *Encryption Process*

The encrypting system shall follow the steps below to encrypt the data:

1. Generate sender's ephemeral EC keypair on named curve ephemeral for the message. secp256r1 and its OID 1.2.840.10045.3.1.7.
2. Using recipient's encryption public key and the ephemeral private key, generate the shared secret using Elliptic Curve Diffie-Hellman key agreement algorithm with OID 1.3.132.1.12.
3. Derive the symmetric key as described in Key Derivation Function (see Section 17.11.4)
4. Derive the initialization vector for symmetric encryption as described in initialization vector derivation (see Section 17.11.6).
5. Encrypt data using all 128 bits of the derived symmetric key using AES–128 id-aes128-GCM with its OID 2.16.840.1.101.3.4.1.6 with no padding, the initialization vector, and no associated authentication data. The 16-byte GCM authentication tag shall be appended to the cipher text.

17.11.3 *Decryption Process*

Receiving system shall follow the steps below to decrypt the encrypted data:

1. Validate that the sender's ephemeral public key is valid on the named curve secp256r1 and its OID 1.2.840.10045.3.1.7.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2. Using receiving system's encryption private key (identified by recipient fingerprint) and the sender's ephemeral public key, generate the shared secret using Elliptic Curve Diffie-Hellman key agreement algorithm with OID 1.3.132.1.12.
3. Derive the symmetric key as described in Key Derivation Function (see Section 17.11.4).
4. Derive the initialization vector for symmetric encryption as described in Initialization vector derivation (See Section 17.11.6).
5. Decrypt the data using AES-128 id-aes128-GCM with its OID 2.16.840.1.101.3.4.1.6 with no padding, the initialization vector, and no associated authentication data. The 16-byte GCM authentication tag shall be appended to the cipher text.

17.11.4 *Key Derivation Function*

This subsection defines the algorithm for deriving the symmetric key from a shared secret and other parameters.

Derive the keying material according to the algorithm defined in Section 3.6 of [22], using the ANSI X9.63 Key Derivation Function described in Section 3.6.1 of [22].

```
Z = sharedSecretFromKeyAgreement(myECPrivateKey, otherPartyECPublicKey);
sharedInfo = computeSharedInfo();
counter = 0x00000001; // 4 bytes
keyingMaterial = Hash(Z | counter | sharedInfo);
```

where

senderKeyAgreementPrivateKey is generated in item 2 of Section 17.11.1

recipientKeyAgreementPublicKey is defined in item 3 of Section 17.11.1.

computeSharedInfo is defined in Section 17.11.4.1.

with the following parameters:

Input name	Value
H (Hash function)	SHA-256
keydatalen (Length of symmetric key)	16
Z (Shared secret)	The shared secret calculated using the ECDH key agreement keys
SharedInfo	The shared information containing the Shared Info sequence

17.11.4.1 Shared Info

Input name	Required	Value
Sender Key Agreement Public Key	Mandatory	The senders' ephemeral key agreement public key. Format: toUncompressedRawECPublicKeyFormat(sender.ephemeralKeyAgreement-PublicKey)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Recipient Key Agreement Public Key	Mandatory	The recipient's key agreement public key Format: toUncompressedRawECPublicKeyFormat(recipient.keyAgreementPublicKey)

17.11.5 *Symmetric Encryption Key*

The first 16 bytes of the 32-byte keying material output from the KDF shall be used as the symmetric key for encryption.

17.11.6 *Initialization Vector Derivation*

The last 16 bytes of the 32-byte keying material output from the KDF shall be used as IV.

17.11.7 *EC Public Key Point Encoding in Uncompressed Form*

To encode the EC public key point in uncompressed form, follow Elliptic-Curve-Point-to-Octet-String Conversion in Section 2.3.3 of [22].

17.12 Versioning

17.12.1 *General*

Server APIs send and receive data that can originate from another server or from a device connected to the other server (if other server is device OEM server).

A new column in the data description tables of the APIs indicate the domain version that a data element depends on.

The negotiation of the API version used between the Device OEM Server and the Vehicle OEM Server (DS-VS) is outside of this specification.

17.13 Example

This section shows the step-by-step output of the encryption of a sample message using sample sender and recipient keys.

17.13.1 *Keys*

Party	Key Type	Value
Sender	**Key Agreement Key**	
	Private Key Hex (S)	53994c02ca9f6d1afbda1742f43d3c17dedfaf3367727208fe9ccc50a33824a0
	Public Key Hex (0x04 | x | y)	0474542541492424edc34f33ba94e61bf718f33ca393d1bf0816f156e4a26687 3f59564aad1a95c9fd8971b527171784e390d9137d037fe2ae30490b1ed1d73aa3
Recipient	**Key Agreement Key**	

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

	Private Key (S)	6c300cac339b4c7084e34175e2e6f5e1d1b135d158bd578c2b1af870facaef17
	Public Key Hex (0x04 | x | y)	04ad2d126c3f8f85bf5796f6ab849b57a35133fed491eced0254ed6a1169d928 1f98018f1aa71d222874d72f47b0b28d84dacb8801b96e815c06f1151210cc7090
	Public Key Fingerprint Hex	ac0095a2121357c69d64213137973222d9873bc80dbcc6ea044d6db3ec5bfbcb

17.13.2 *Message to encrypt*

```
{ "id": "Hello", "value": "World" }
```

17.13.3 *Encryption process*

Data	Format	Value
Algorithm Identifier	String	ECIES_v1
Derived Shared Secret	Hex	a6c3021dc18ad03959768250e872818585264fbdd1b6de6ed32a7eb16f19f858
KDF Shared Info	Hex	0474542541492424edc34f33ba94e61bf718f33ca393d1bf0816f156e4a26687 3f59564aad1a95c9fd8971b527171784e390d9137d037fe2ae30490b1ed1d73a a304ad2d126c3f8f85bf5796f6ab849b57a35133fed491eced0254ed6a1169d9 281f98018f1aa71d222874d72f47b0b28d84dacb8801b96e815c06f1151210cc 7090
Keying Material	Hex	58a5f8b53ff010a62eb2e98303f7d2d088b35fa8b758797777dbafa81df3bc9f
Derived AES Key	Hex	58a5f8b53ff010a62eb2e98303f7d2d0
Initialization Vector	Hex	88b35fa8b758797777dbafa81df3bc9f
Encrypted Data	Hex	46e9fbeb564e5eba68c094da08172bac8299bd268749763224dea8a59313d548 963dafbc3dcc7ea206646edeb4469e5d6eed38

17.13.4 *Decryption process*

Data	Format	Value
Algorithm Identifier	String	ECIES_v1
Derived Shared Secret	Hex	a6c3021dc18ad03959768250e872818585264fbdd1b6de6ed32a7eb16f19f858

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

KDF Shared Info	Hex	0474542541492424edc34f33ba94e61bf718f33ca393d1bf0816f156e4a2 66873f59564aad1a95c9fd8971b527171784e390d9137d037fe2ae30490b 1ed1d73a a304ad2d126c3f8f85bf5796f6ab849b57a35133fed491eced0254ed6a116 9d9 281f98018f1aa71d222874d72f47b0b28d84dacb8801b96e815c06f11512 10cc 7090
Keying Material	Hex	58a5f8b53ff010a62eb2e98303f7d2d088b35fa8b758797777dbafa81df3b c9f
Derived AES Key	Hex	58a5f8b53ff010a62eb2e98303f7d2d0
Initialization Vector	Hex	88b35fa8b758797777dbafa81df3bc9f
Decrypted Data	String	{ "id": "Hello", "value": "World" }

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

18 SECURITY

18.1 SPAKE2+ Protocol Description

18.1.1 *General*

This section explains the principles of the SPAKE2+ protocol. Refer to Section 18.4 for implementation details.

The system uses SPAKE2+, which is an ECC-based pairing algorithm protocol, to mutually authenticate two entities based on the knowledge of a password provided on the client side (i.e., device) and a verifier, permanently stored in the server (i.e., vehicle). For more detailed information, see [10].

The NIST P-256 curve shall be used [8].

The Vehicle OEM Server should generate the password and provision the necessary elements (w0, L; see 18.1.2) into the vehicle well before start of the owner pairing, so that pairing is possible even when the vehicle is offline at the moment of owner pairing.

The Scrypt key derivation is executed in the server and on the device, which allows the server and device to adapt the derivation <> parameters over time to counter increases in attacker performance.

The password pwd should be provided to the owner through the Vehicle OEM account, protected by the login credentials known only by the owner. The password is UTF-8 encoded and should be passed into the scrypt function in this coding.

All values are assumed to be in big-endian byte order. The x and y random generator shall have uniform distribution over the required range and be cryptographically secure.

18.1.2 *Execution*

The Vehicle OEM Server derives L and w0 from the password as follows:

z0=left 40 bytes of Scrypt(pwd, s, Nscrypt, r, p, dkLen)

z1=right 40 bytes of Scrypt(pwd, s, Nscrypt, r, p, dkLen)

It shall derive w0 and w1 using the method FIPS 186-4, B.5.1 Per-Message Secret Number Generation Using Extra Random Bits:

w0 = (z0 mod (n-1)) + 1 with n being the order n of base point G as defined for NIST P-256

w1 = (z1 mod (n-1)) + 1 with n being the order n of base point G as defined for NIST P-256

w0 and w1 shall not equal.

Then, it shall calculate:

$L = w1 \times G$

with G as defined for the chosen elliptic curve.

The scrypt algorithm is described in [11].

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The Scrypt function is defined with the following parameters:

- **Salt**: 16 bytes, randomly generated for each new verifier as per [12]
- **Cost parameter Nscrypt**: 4096 or higher
- **Block size r**: 8
- **Parallelization parameter p**: 1
- **Output length dkLen**: 80

The Vehicle OEM Server then provides salt, L, and w0, which constitute the verifier, securely and confidentially to the vehicle.

This specification does not specify the means to upgrade these parameters to faster hardware in the future. A simple solution would be for the server to provide these parameters to the vehicle too.

When the pairing starts or re-starts after an erroneous attempt, the vehicle shall randomly generate a valid scalar y on the chosen curve, and then calculate:

$$Y = y \times G + w0 \times N$$

where N is a point on the used elliptic curve as specified in Section 18.1.5. When the maximum number of failed pairing attempts is reached (see the usage of SPAKE2+ REQUEST command in Section 5.1.2), a new password and verifier shall be generated to allow to retry pairing.

Y shall not be the point at infinity and shall be on the chosen curve. A new y shall be generated, and a new Y shall be calculated until the requirements for Y are fulfilled before continuing the protocol.

The vehicle shall transmit Y to the device together with the Scrypt configuration parameters.

When owner pairing is started, the password is provided to the device to be paired, which allows it to calculate:

z0 = left 40 bytes of Scrypt(pwd, s, Nscrypt, r, p, dkLen)

z1 = right 40 bytes of Scrypt(pwd, s, Nscrypt, r, p, dkLen)

using s, Nscrypt, r, and p transmitted by the vehicle. dkLen is implicitly known on both sides.

It shall derive w0 and w1 using the method FIPS 186-4, B.5.1 Per-Message Secret Number Generation Using Extra Random Bits:

w0 = (z0 mod (n-1)) + 1 with n being the order n of base point G as defined for NIST P-256

w1 = (z1 mod (n-1)) + 1 with n being the order n of base point G as defined for NIST P-256

Then the device shall randomly generate a valid scalar x on the chosen curve and then calculate:

$$X = x \times G + w0 \times M$$

where M is a point on the chosen elliptic curve as specified in Section 18.1.5. A new x shall be generated and X shall be calculated on each new pairing attempt, within the limits of a maximum number of retries until a new password needs to be generated by the server.

X shall not be the point at infinity and shall be on the chosen curve. A new x shall be generated, and a new X shall be calculated until the requirements for X are fulfilled before continuing the protocol.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Device and vehicle then shall exchange X and Y through the commands defined in Section 5.
On reception of X, the vehicle shall compute the shared secret curve points Z and V as follows:
$Z = y \times (X - w0 \times M)$
$V = y \times L$
On reception of Y, the device shall compute the shared secret curve points Z and V as follows:
$Z=x \times (Y - w0 \times N)$
$V=w1 \times (Y - w0 \times N)$
Both vehicle and device then shall calculate the shared secret K as follows:
K = SHA-256(len(X) || X || len(Y) || Y || len(Z) || Z || len(V) || V || len(w0) || w0)
where len(str) denotes the length of a string in bytes, represented as an 8-byte little-endian number.
The shared secrets CK and SK are derived from K:

CK = K [0:128]

SK = K [128:128] 5

The LONG_TERM_SHARED_SECRET is derived based on Listing 18-9.

18.1.3 *Verification*

The verification of the derived keys shall be done by calculating a check value on either side which are mutually exchanged and verified.
The vehicle shall calculate the verification value M[1] as

K1=HKDF(CK, “ConfirmationKeys” || TLV $5B_h$ || TLV $5C_h$, [0:128]), where ConfirmationKeys is a defined static string (see Listing 18-6).

M[1]=C-MAC(K1, X) using K1 as secret key, using CMAC-AES-128 as defined in [RFC4493]

and shall send it over to the device.
The device shall verify M[1] using the received M[1] from the vehicle and, if successful, shall calculate the verification value M[2] as

K2=HKDF(CK, “ConfirmationKeys” || TLV $5B_h$ || TLV $5C_h$, [128:128]) (see Listing 18-6)

M[2]=C-MAC(K2, Y) using K2 as secret key, using CMAC-AES-128 as defined in [RFC4493]

and shall send it over to the vehicle.
The protocol succeeds when the vehicle successfully validates M[2].

18.1.4 *Summary*

To summarize, the following elements are part of the protocol:

- **s** (16 bytes): Salt generated by the Vehicle OEM Server

[5] The notation used is [start_index: number_of_bits]

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- **pwd**: Password generated by the Vehicle OEM Server, available in the owner's Vehicle OEM account
- **z0** (40 bytes): Part of verifier including FIPS extra bits, derived from the password using a one-way function
- **z1** (40 bytes): Value including FIPS extra bits derived from the password using a one-way function
- **w0** (32 bytes): Part of verifier, derived z0
- **w1** (32 bytes): Value derived from z1
- **L**: curve point, part of verifier, calculated by the Vehicle OEM Server in uncompressed format
- **x** (32 bytes): Random scalar generated by device on each pairing attempt
- **y** (32 bytes): Random scalar generated by vehicle on each pairing attempt
- **M**: Known constant curve point in uncompressed format
- **N**: Known constant curve point in uncompressed format
- **X**: Intermediate public curve point exchanged between device and vehicle in uncompressed format
- **Y**: Intermediate public curve point exchanged between vehicle and device in uncompressed format
- **Z**: Secret curve point generated on both sides from public values and constants in uncompressed format
- **V**: Secret curve point generated on both sides from public values and constants in uncompressed format
- **K** (32 bytes): Shared SPAKE2+ secret
- **CK** (16 bytes): Shared SPAKE2+ secret (as derived from K) to derive confirmation keys
- **SK** (16 bytes): Shared SPAKE2+ secret (as derived from K) to derive system keys
- **K1** (16 bytes): Vehicle-side secret key for evidence calculation M[1]
- **K2** (16 bytes): Device-side secret key for evidence calculation M[2]
- **Kenc**, **Kmac**, **Krmac**: Secure channel keys
- **LONG_TERM_SHARED_SECRET**: Long-term shared secret for further key derivations

For all operations, all curve points shall be represented in x9.63 standard format as the byte stream 0x04 || <x> || <y> format where <x> and <y> are each 32-byte integer in big-endian representation (65 bytes).

x and y shall be newly generated after each failing pairing attempt.

18.1.5 *SPAKE2+ Constant Definitions*

The following NIST P-256 (SECP256r1) curve parameters are defined in [10]

M=

04

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

88 6e 2f 97 ac e4 6e 55 ba 9d d7 24 25 79 f2 99
3b 64 e1 6e f3 dc ab 95 af d4 97 33 3d 8f a1 2f
5f f3 55 16 3e 43 ce 22 4e 0b 0e 65 ff 02 ac 8e
5c 7b e0 94 19 c7 85 e0 ca 54 7d 55 a1 2e 2d 20

N=
04
d8 bb d6 c6 39 c6 29 37 b0 4d 99 7f 38 c3 77 07
19 c6 29 d7 01 4d 49 a2 4b 4f 98 ba a1 29 2b 49
07 d6 0a a6 bf ad e4 50 08 a6 36 33 7f 51 68 c6
4d 9b d3 60 34 80 8c d5 64 49 0b 1e 65 6e db e7

18.2 Privacy Properties

18.2.1 *NFC Transaction [WCC1/WCC2]*

The vehicle identifier is transmitted from the vehicle to the device in the AUTH0 command (see Section 15.3.2.9). It is recommended to change the vehicle identifier when the vehicle owner changes to avoid any privacy issues.

18.2.2 *Vehicle OEM Server*

The Vehicle OEM Server shall be designed in a way to separate business-relevant data from the key tracking data, which may be stored in a KTS for example.

Business data, such as number of purchased mobile keys, expiration of the mobile key subscription, etc., may be exploited to enforce the Vehicle OEM business policy.

Data stored in the KTS shall be privacy protected. The data shall not be used for business reasons.

When a vehicle is stolen (or in similar cases), only the KTS-data for the affected vehicle is to be disclosed, on specific request.

18.2.3 *User Privacy*

The Vehicle OEM-controlled data going into the Digital Key instance shall be chosen and protected in a way such that the privacy of device users is protected in all use cases.

18.2.4 *Multi-Vehicle OEM Deployment*

The device shall enforce that a specific Vehicle OEM application can only access Digital Key data and functionality within the Digital Key applet instance that belongs to the same Vehicle OEM.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

18.2.5 *Error Handling*

On every attempt to execute the owner pairing flow, the device shall generate a new public key pair (device.PK/device.SK).

18.3 Cryptographic Algorithms

All cryptographic elements that are stored in or received from the Digital Key applet shall respect the formats defined in Section 4.

18.4 Cryptographic Protocols

18.4.1 *Server Password Generation*

The Server generates the password for vehicle and device in the following way:

Listing 18-1: Server Password Generation

```
input: Scrypt parameters
output: w0, w1, L, s, pwd
begin
  generate random salt s
  generate random password pwd
  z=Scrypt(pwd, s) with parameters as defined in Section 18.1.2
  z0=left 40 bytes of z
  z1=right 40 bytes of z
  w0=(z0 mod (n-1)) + 1 with n being the order n of base point G as defined for NIST P-256
  w1=(z1 mod (n-1)) + 1 with n being the order n of base point G as defined for NIST P-256
  L=w1×G with G as defined for NIST P-256
  provide pwd to device (through web interface or Vehicle OEM app)
  send s, w0 and L securely to vehicle (through Vehicle OEM proprietary link)
end
```

18.4.2 *Vehicle-side public EC Point Generation*

Listing 18-2: Vehicle-side Public Point Generation

```
input: w0
output: Y, y
bBegin
  generate random scalar y on chosen curve
  Y=y×G+w0×N
  return Y, which is sent to the device during owner pairing
end
```

18.4.3 *Device-side public EC Point Generation*

Listing 18-3: Device-side Public Point Generation

```
input: w0
output: X, x
begin
  generate random scalar x on chosen curve
  X=x×G+w0×M
  return X, which is sent to the vehicle during owner pairing
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
end
```

18.4.4 *Vehicle-side computation of Shared Secret*

Listing 18-4: Vehicle-side Computation of Shared Secret

```
input: X, w0, L, y
output: K
begin
  Z=y×(X-w0×M)
  V=y×L
  K=SHA-256(len(X) || X || len(Y) || Y || len(Z) || Z || len(V) || V || len(w0) || w0)
  where len(str) denotes the length of a string in bytes, represented as an 8-byte little-endian number.
OKM : [0:128] = CK, [128:128] = SK (all 16-byte)
end
```

18.4.5 *Device-side computation of Shared Secret*

Listing 18-5: Device-side Computation of Shared Secret

```
input: Y, w0, w1, x
output: CK, SK
begin
  Z=x×(Y-w0×N)
  V=w1×(Y-w0×N)
  K=SHA-256(len(X) || X || len(Y) || Y || len(Z) || Z || len(V) || V || len(w0) || w0)
  where len(str) denotes the length of a string in bytes, represented as an 8-byte little-endian number.
OKM : [0:128] = CK, [128:128] = SK (all 16-byte)
end
```

18.4.6 *Derivation of Evidence Keys*

Listing 18-6: Derivation of Evidence Keys

```
input: CK
output: K1, K2
begin
  Compute the steps indicated by IETF RFC5869 [13] with the following mapping:
  IKM  : CK
  L : 32
  Salt : NULL
  Info : "ConfirmationKeys" || TLV 5B_h || TLV 5C_h see Table 5-4
  Both TLVs include tag, length and data field.
  Hash : SHA-256
  OKM : [0:128] = K1, [128:128] = K2 (all 16-byte)
  Notation is [start_index: number_of_bits]
end
```

18.4.7 *Vehicle-side computation of Evidence*

Listing 18-7: Vehicle-side Computation of Evidence

```
input: K1, X
output: M[1]
begin
  compute M[1] = CMAC(K1, X) using K1 as secret key, using CMAC-AES-128 as defined in [RFC4493]
  return M[1]
end
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

18.4.8 *Device-side computation of Evidence*

Listing 18-8: Device-side Computation of Evidence

```
input: K2, Y
output: M[2]
begin
  compute M[2] = CMAC(K2, Y) using K2 as secret key, using CMAC-AES-128 as defined in [RFC4493]
  return M[2]
end
```

18.4.9 *Derivation of System Keys [WCC1/WCC2/WCC3]*

Derivation of system keys Kenc, Kmac, Krmac for secure channel and of a long-term symmetric secret for further key derivations. If the vehicle is WCC2 or WCC3, and if the device is either WCC2 or WCC3 capable or device is capable of deriving Kble_intro and Kble_oob_master, then additionally Kble_intro and Kble_oob_master shall be derived. This algorithm is not part of SPAKE2+ but of the applicative part of the system.

Listing 18-9: Derivation of System Keys

```
input: SK, Flag_Kble_intro_Kble_oob_master_support
output: Kenc, Kmac, Krmac, LONG_TERM_SHARED_SECRET
If (Flag_Kble_intro_Kble_oob_master_support == true): Additional output: Kble_intro, Kble_oob_master
begin
  Compute the steps indicated by IETF RFC5869 [13] with the following mapping:
    IKM : SK
    L : if (Flag_Kble_intro_Kble_oob_master_support == true) 96 else 64
    Salt : NULL
    Info : "SystemKeys"
    Hash : SHA-256
    OKM : [0:128] = Kenc, [128:128] = Kmac,  [256:128] = Krmac, [384:128] = LONG_TERM_SHARED_SECRET (all 16-byte)
      if (Flag_Kble_intro_Kble_oob_master_support == true): [512, 128] = Kble_intro; [640,128] = Kble_oob_master (all 16 bytes)
  Notation is [start_index: number_of_bits]
end
```

18.4.10 *Generate Attestation Signature*

The signature is calculated with ECDSA according to [8] using NIST P-256 curve and SHA-256. See Listing 15-42.

18.4.11 *Validate Attestation or Certificate*

See Listing 15-43.

18.4.12 *Secure Channel Command Encryption and Authentication*

Listing 18-10: Secure Channel Command Encryption and Authentication

```
input: payload, Kmac, Kenc
output: encrypted_payload, mac
begin
  Compute the steps as indicated in Section 6.2.6 of GPC_SPE_014 [7] with the following mapping:
  Command Data Field - Plain Text : payload
  Padded Counter Block : 00000000000000000000000000000000h || 1-byte counter (01h for first command, up to max FFh)
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

S-ENC : Kenc
Ciphered Command Data Field : encrypted_payload
MACChainingValue : $00000000000000000000000000000000_h$ on first command or
MAC Chaining value of previous command
S-MAC : Kmac
C-MAC : mac (8 bytes)
end

18.4.13 *Secure Channel Response Encryption and Authentication*

Listing 18-11: Secure Channel Response Encryption and Authentication

input: payload, Krmac, Kenc
output: encrypted_payload, mac
begin
Compute the steps indicated in Section 6.2.7 of GPC_SPE_014 [7] with the following mapping:
Response Data Field - Plain Text : payload
Padded Counter Block: $80000000000000000000000000000000_h$ ||
1-byte counter_value used in command
S-ENC : Kenc
Ciphered Response Data Field : encrypted_payload
Response Data Field : encrypted_payload
MAC Chaining Value : 16-byte MAC Chaining Value from command
S-RMAC : Krmac
R-MAC : mac (8 bytes)
end

18.5 Error Handling [WCC1/WCC2/WCC3]

Vehicle and device should limit the time between starting of the pairing mode and the start of the contactless transaction. After expiration of this time, the vehicle and device need to be brought back to pairing mode by the user.

The overall time for the pairing transaction should be limited on vehicle and device side. When the vehicle and device are brought into pairing mode and the pairing transaction starts (i.e., successful SELECT command/response exchange), the transaction time should be limited. Then, either the pairing was successful, or the pairing process will be aborted by the vehicle and device and needs to be restarted by the user. A new pairing password shall be entered again on the device side.

The time to present the device again after a RF reset should be limited. The vehicle should abort the pairing process if a device is not presented within this time limit after the restart of the reader field.

18.6 Secure Element (SE)

Secure Element (SE): As defined within CCC, is embedded technology that provides a tamper-resistant secure implementation aimed at providing isolation, integrity, confidentiality, secure provisioning, secure storage, and secure processing for the Digital Key applet including its high-value keying material while being isolated from the rich OS and application processor. An implementation shall provide hardware and software protection against physical and logical attacks and unsafe use during the full lifecycle of the secure implementation, the Digital Key applet, and their assets.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tamper-resistant secure hardware: Hardware designed to isolate and protect embedded software and data by implementing appropriate security measures. The hardware and embedded software must be resistant against high attack potential and must meet the requirements as defined by the Digital Key certification which include resistance to physical tampering scenarios. The acceptable scenarios are described in the latest Security IC Platform Protection Profile or in other alternatives identified by the Digital Key certification.

The transport channel between a SE and the NFC component shall protect confidentiality and integrity. The SE shall provide a secure binding between these components that shall be resistant to manipulation after production. This is referred to as a hardware root of trust. It shall be prevented that any code running on the application processor has access to or inject data on this channel. The Secure Element shall have the ability to distinguish communication between wired interface and contactless interface.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19 BLUETOOTH LOW ENERGY (LE) INTERFACE [WCC2/WCC3]

The Secure Ranging Service provides the Bluetooth LE framework necessary to discover, manage, and control UWB-based ranging between a device and a vehicle. Bluetooth LE, Secure Element, and UWB are core to the Digital Key solution. Bluetooth LE offers secure data exchange between device and vehicle which then enables Secure Element to offer mutual authentication and data sharing over a secure channel with the Vehicle. The Bluetooth LE channel is also required to establish and or manage the Secure Ranging service.

19.1 Bluetooth LE Functional Requirements

The Bluetooth LE Controller and Host for both the device and vehicle shall be compliant to the mandatory capabilities of the Bluetooth Core Specification 5.0 or later [30] and shall support the following additional capabilities:

- LE L2CAP Connection Oriented Channel Support
- LE Privacy
- LE Secure Connections

The Bluetooth LE Controller and Host for both the device and vehicle may optionally support the following additional capabilities:

- Long Range (LELR)
- LE Advertising Extensions (mandatory, if LE Long Range is supported)

Vehicles and devices shall use “LE security mode 1/Level 4” or/and “Secure Connections Only mode,” see Vol 3, Part C, Section 10 of [30]. During a connection, the device and vehicle may use the Feature Exchange Procedure to signal the LELR support based on the “LE Coded PHY” bit defined in [30].

19.1.1 *Vehicle*

The Vehicle shall support the GAP Peripheral role.

The vehicle shall support privacy in the advertising state as defined in Volume 6, Part B, Section 6 of [30]. The vehicle shall use resolvable private addresses for advertising events as specified in Volume 6, Part B, Section 6.2 of [30].

If the vehicle supports multiple Bluetooth LE controllers, all controllers shall use the same identity address and resolution key. Each controller may have a different random resolvable address at any time.

The vehicle shall support GATT in the server role.

19.1.2 *Device*

The device shall support the GAP Central role.

The device shall support resolvable private addresses used in advertisements by the vehicle.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The device shall initiate a connection to the vehicle when it receives the advertising packet from a paired vehicle. A connection interval of 30ms is recommended when connecting to a vehicle.

The device shall support GATT in the client role.

19.2 Bluetooth LE Procedures

19.2.1 *Owner Pairing Connection Establishment*

This is the flow for owner pairing connection establishment using Bluetooth out-of-band (OOB) pairing. The Bluetooth LE Setup is divided into the following two subsections:

- Bluetooth LE Link Layer Connection Establishment & Bluetooth LE Owner Pairing GATT Flow
- Bluetooth LE Pairing & Encryption Setup

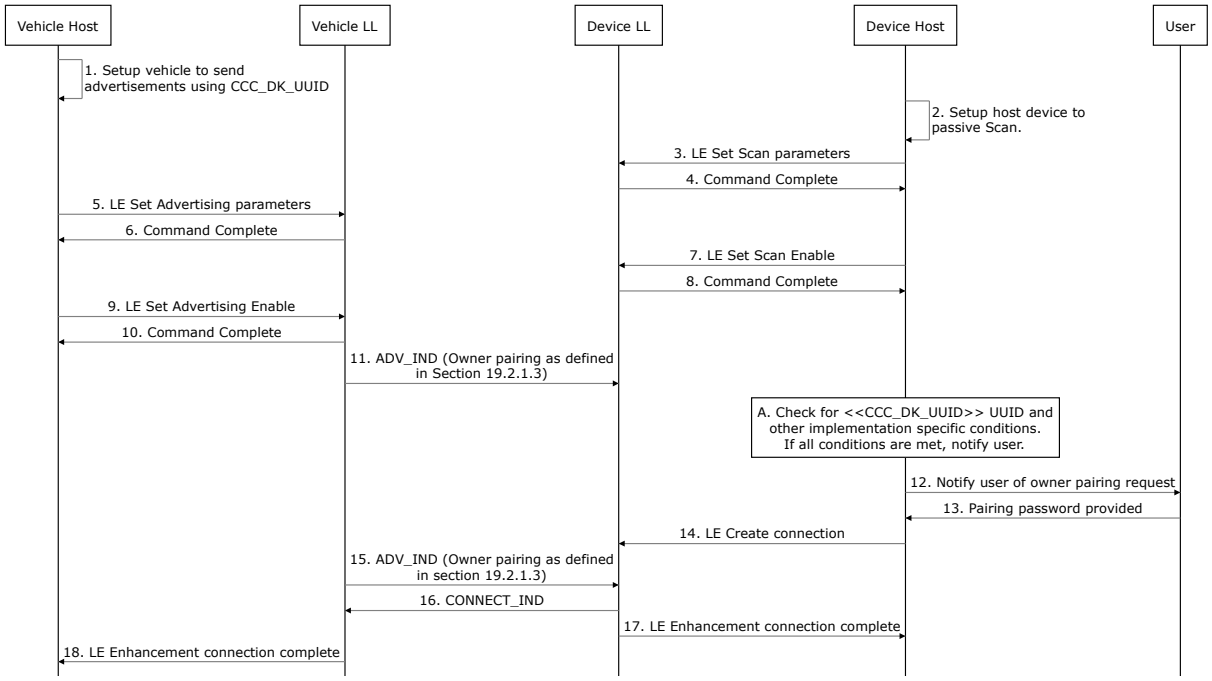
A vehicle that does not support secure ranging shall not allow owner pairing over the Bluetooth LE interface and shall not broadcast the Owner Pairing Advertisement. Owner pairing shall start over NFC as described in Section 6 and followed by Bluetooth LE Activation flow as described in Section 19.5.10.

19.2.1.1 Bluetooth LE Link Layer Connection Establishment & Bluetooth LE Owner Pairing GATT Flow

In Figure 19-1, the vehicle begins by sending ADV_IND with CCC_DK_UUID as the advertising payload (steps 11 and 15). The Vehicle LL shall be in the advertising state with its filtering policy set to accept all connection requests. The device begins passive scanning. The Device LL shall be in the scanning state with its filter policy set to accept all advertisements. Once the Device LL receives an advertisement, it forwards that to the device host. The Device Host shall check if the CCC_DK_UUID is contained in the advertisement payload (box A). If the CCC_DK_UUID is contained in the advertising payload, the user is notified. If the user accepts the owner pairing requests, the user provides the pairing password. The Device LL shall enter initiating state with the filter policy set to the advertiser's address after step 14. Upon receiving the next same advertisement from the Vehicle LL, the Device LL shall send a connection request (CONNECT_IND as in step 16).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-1: Bluetooth LE Link Layer Connection Establishment.



In Figure 19-2, the Device Host (acting as the GATT client) initiates service discovery with the Vehicle Host (acting as the GATT server) to get the DK Service and DK Service UUID_SPSM DK_VERSION characteristic in order to establish the L2CAP connection for the DK Service.

Figure 19-2: L2CAP Connection-Oriented Channel.

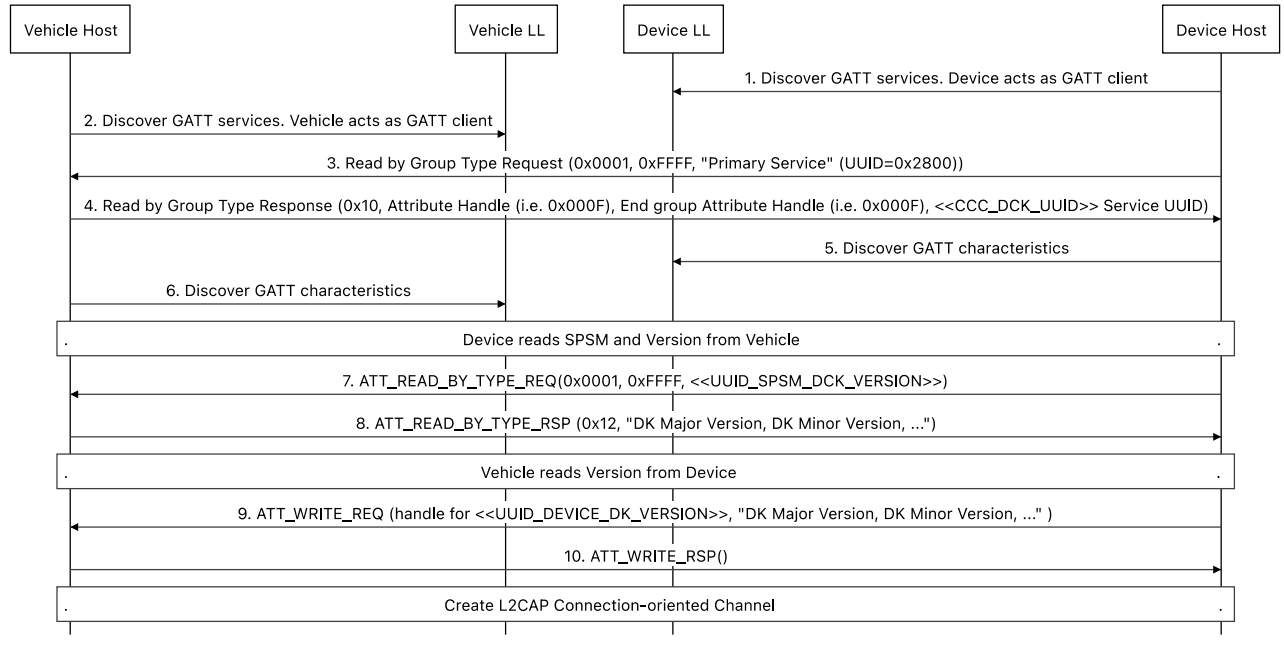


Figure 19-2 is an example in which the device reads the SPSM value from the vehicle's GATT Server. The device may read the SPSM value in other ways, as defined in the Volume 3 Part G of [30].

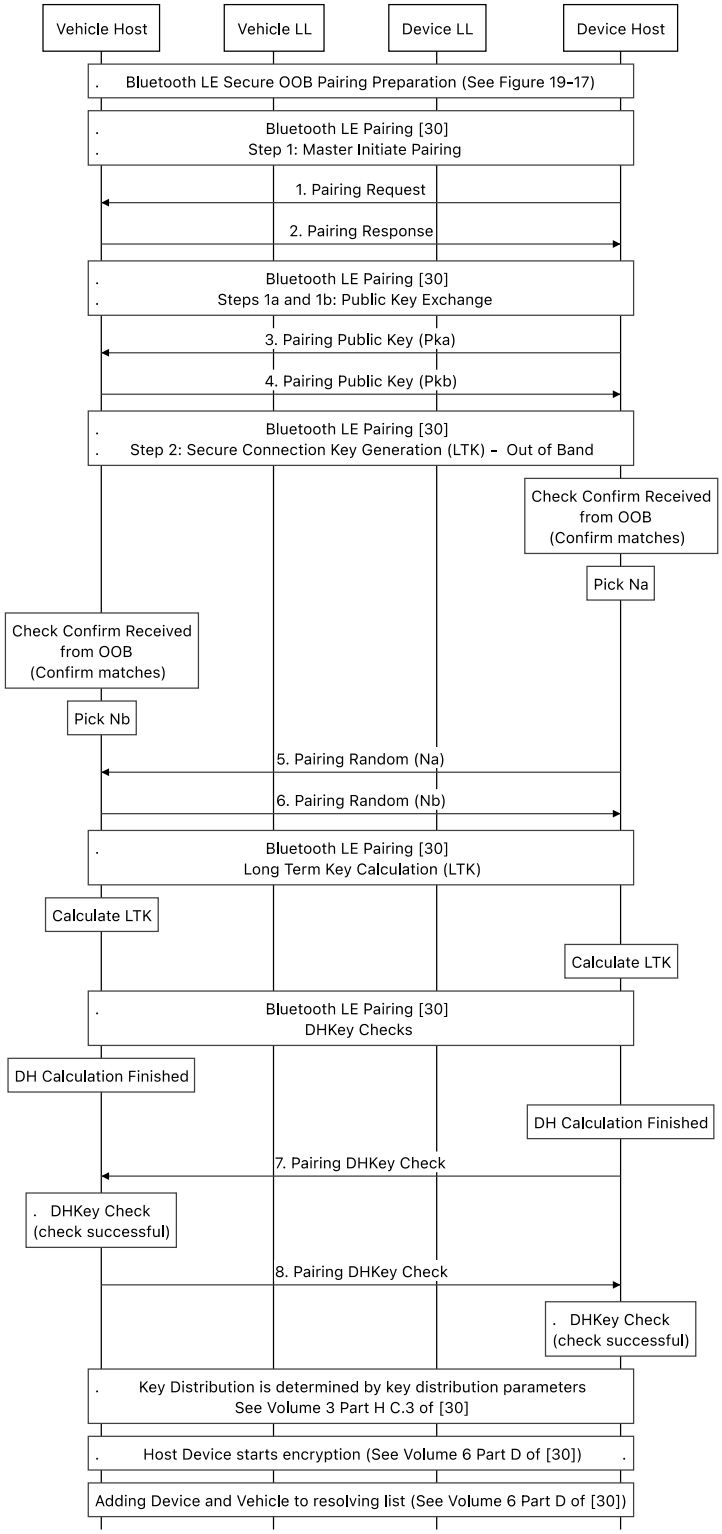
Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

For more details on Box B and the associated message flow, please refer to Volume 3 Part A [30].

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.2.1.2 Bluetooth LE Pairing and Encryption Setup

Figure 19-3: Bluetooth LE Pairing and Encryption Setup.



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

In Figure 19-3, the setup begins with the Bluetooth LE Secure OOB Pairing Preparation procedure as defined in Section 19.5.1 and shown in Figure 19-17.

During this procedure the device has generated its public key (PKa) and secret key (SKa) and has received the vehicle's Bluetooth address (BTAddrB), the vehicle's commitment value (Cb), and random number (rb) over a secured channel. The vehicle has generated its public key (PKb) and secret key (Skb) and received the device's Bluetooth address (BTAddrA) and the device's commitment value (Ca) and random number (ra) over a secured channel.

The procedure continues with the following:

- **Bluetooth LE Pairing: Master Initiated Pairing** (Pairing Phase 1, Step 1) as defined in [30]. The Bluetooth LE Pairing Request PDU is sent from the device to the vehicle. The Bluetooth LE Pairing Response PDU is sent from the vehicle to the device.
- **Bluetooth LE Pairing: Public Key Exchange** (Pairing Phase 1, step 1a and 1b) Vol 2, Part H, Section 7.1 in [30]. A Bluetooth LE Pairing Public Key PDU is sent from the device to the vehicle and a Bluetooth LE Pairing Public Key PDU is sent from the vehicle to the device. Both the device and vehicle begin their DHKey generation.
- **Bluetooth LE Pairing: Secure Connection Key Generation** (LTK) – OOB (Pairing Phase 2, Authentication Stage 1, Step 2) in [30]. Both the device and vehicle verify if the confirm value received as part of the OOB Pairing Preparation procedure matches. Both the device and vehicle generate a random nonce (Na and Nb). The device sends its nonce (Na) to the vehicle using the Bluetooth LE Pairing Random PDU and the vehicle sends its nonce (Nb) to the device using the Bluetooth LE Pairing Random PDU.
- **Bluetooth LE Pairing: LTK calculation** (Pairing Phase 2, Authentication Stage 2) as described in Vol 3, Part H, Section 2.3.5.6.5 in [30]. Once the DHKey generation is completed on the device and vehicle, the device and vehicle calculate their LTK.
- **Bluetooth LE Pairing: DHKey Checks.** The device sends the check value (Ea) using the Pairing DHKey Check PDU to the vehicle. The vehicle sends the check value (Eb) using the Pairing DHKey Check PDU to the device. Both the device and vehicle verify the values.
- **Bluetooth LE Pairing: Key Distribution** (Pairing Phase 3) as described in Vol 3, Part H, Section 2.4.3.2 in [30].
- **Device and vehicle enabling encryption** as described in Vol 3, Part H, Section 2.4.4.2 in [30]. The device and vehicle add each other to their private address resolving list as described in Vol 6, Part B, Section 4.7 in [30].

The messages in the box "Host Device starts encryption" and "Adding device and vehicle to resolving list" are out of scope of this specification. These are shown for illustration purposes only. Please refer to Vol 6 Part D of [30].

19.2.1.3 Owner Pairing Advertising

For owner pairing, only the Legacy LE 1M PHY shall be used. The Advertisement (ADV_IND) for owner pairing shall follow the Section 2.3.1.1 of Volume 6 Part B of [30].

Event Type: Connectable and scannable undirected

The ADV_IND consists of Advertising Address and Advertising Data as shown in Table 19-1 and Table 19-2, respectively:

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-1: AdvA field of ADV_IND.

Field	Length (bytes)
Advertising Address (AdvA)	6

Table 19-2: AdvData field of ADV_IND.

Field	Length (bytes)	Description	Value (hex)
Length	1	Length of AD Type and AD Data	0x03
AD Type	1	16-bit service UUID	0x03
AD Data	2	<<CCC_DK_UUID>>	0xFFF5
Length	1	Length of AD Type and AD Data	0x14
AD Type	1	Service data -128bit UUID	0x21
AD Data	16	CCCServiceDataIntent UUID	0x5810bbc0-b499-11e9-a2a3-2a2ae2dbcce4
AD Data	1	IntentConfiguration	0x1 (Default)
AD Data	2	Vehicle Brand Identifier	AS defined in Table 2-1 of [35]

The usage of CCC_DK_UUID: The Assigned Value is provided by Bluetooth SIG, Inc. and may only be used by its members in compliance with all terms and conditions of use issued by Bluetooth SIG, Inc. For more information visit https://www.bluetooth.com/specifications/assigned-numbers.

The IntentConfiguration is used to allow separate user flows on the device, depending on whether the user has started owner pairing in the vehicle or not. The byte is defined as described in Table 19-3:

Table 19-3: Definition of IntentConfiguration byte.

Field	Description	Bit index
Intent	0: Intent from vehicle - user has actively started owner pairing in vehicle 1: No intent from vehicle	0
Reserved for future use	0: Set to 0 and shall be ignored by receiver	1:7

The vehicle brand identifier is specified in Table 2-1 in [35].

19.2.1.4 Pairing Request Definition:

The following fields are mandatory for the Pairing Request (see Volume 3, Part H, Section 3.5.1 of [30]):

Table 19-4: Mandatory fields in Pairing Request.

Field	Value (hex)	Definition
IO Capability	0x00 – 0xFF	See Vol 3, Part H, Table 3.4 in Section 3.5 of [30]
OOB data flag	1	OOB Authentication data from remote device present

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Field	Value (hex)	Definition
AuthReq	1	Bonding

19.2.1.5 Pairing response definition:

The following fields are mandatory for the Pairing Response (see Volume 3, Part H, Section 3.5.2 of [30]):

Table 19-5: Mandatory fields in Pairing Response.

Field	Value (hex)	Definition
IO Capability	0x00 – 0xFF	See Vol 3, Part H, Table 3.4 in Section 3.5 of [30]
OOB data flag	0x1	OOB Authentication data from remote device present
AuthReq	0x1	Bonding
Maximum Encryption Key Size	0x10	
Initiator Key Distribution	0xF0	
Responder Key Distribution	0xF0	

19.2.1.6 DK Service:

The vehicle shall support the DK Service. This shall be a primary service and there shall only be one instance of this service.

Table 19-6: DK Service UUID.

Field	Value
DK Service UUID	<<CCC_DK_UUID>>

19.2.1.7 Vehicle PSM Characteristic:

This characteristic shall return the Simplified Protocol/Service Multiplexer (SPSM) used for the L2CAP channel from the vehicle.

The device shall read the SPSM from the vehicle upon connection. The vehicle manufacturer shall pick an SPSM. For LE Credit-based Connections, a dynamically allocated SPSM value (i.e., the non-SIG assigned SPSM value in 0x0080-0x00FF) is used.

During passive entry and owner pairing, the device shall upon Bluetooth LE connection:

- Read the characteristics by UUID (with the UUID_SPSM or UUID_SPSM_DK_VERSION). The SPSM value shall use big-endian byte order.
- Write the characteristics by UUID (for UUID_DEVICE_DK_VERSION)

Table 19-7: SPSM Characteristic declaration.

Attribute type	Attribute value		Attribute permission
0x2803 – UUID for «Characteristic»	Characteristic Properties = 0x02	0xMMMM = Handle of Characteristic Value D3B5A130-9E23-4B3A-8BE4-6B1EE5F980A3 – UUID_SPSM	Read only, No Authentication, No Authentication

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-8: SPSM Characteristic value declaration.

Attribute type	Attribute value	Attribute permission
D3B5A130-9E23-4B3A-8BE4-6B1EE5F980A3 – UUID_SPSM	uint16 – SPSM value	Read only, No Authentication, No Authentication

19.2.1.8 DK Version Characteristic:

Table 19-9: Vehicle SPSM and DK version Characteristic Declaration

Attribute type	Attribute value			Attribute permission
0x2803 – UUID for «Characteristic»	Characteristic Properties = 0x02	0xMMMM =Handle of Characteristic Value	AE285B91-6D23-23F1-CA12-6B1EE5B780A3 – UUID_SPSM_DK_VERSION	Read only, Authenticated encryption required

Table 19-10: Vehicle SPSM and DK Version Characteristic Value Declaration

Attribute type	Attribute value	Attribute permission
D3B5A130-9E23-4B3A-8BE4-6B1EE5B780A3 – UUID_SPSM_DK_VERSION	Refer to Table 19-13 (Attribute value definition for Vehicle_SPSM_And_DK_Version_Characteristic)	Read only, Authenticated encryption required

Table 19-11: Device Selected DK Version Characteristic Declaration

Attribute type	Attribute value			Attribute permission
0x2803 – UUID for «Characteristic»	Characteristic Properties = 0x02	0xMMMM = Handle of Characteristic Value	BD4B9502-3F54-11EC-B919-0242AC120005 – UUID_DEVICE_DK_VERSION	Write only, Authenticated encryption required

Table 19-12: Device Selected DK Version Characteristic Value Declaration

Attribute type	Attribute value	Attribute permission
BD4B9502-3F54-11EC-B919-0242AC120005 – UUID_DEVICE_DK_VERSION	Refer to Table 19-14 (Attribute value definition for Device_Selected_DK_Version_Characteristic)	Write only, Authenticated encryption required

Table 19-13: Attribute Value Definition for Vehicle SPSM and DK Version Characteristic

Attribute value	Length (Bytes)	Description
SPSM value	2	SPSM Value.
Supported_DK_Protocol_Version_Len	1	Length byte for supported DK Protocol versions.
Supported_DK_Protocol_Version	2 × n	DK_Protocol_Version is 2 Byte field (Major:Minor); minor version byte changes

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

		whenever there is a change to DK protocol flows or message definitions. Major version byte changes when backward compatibility is broken. The vehicle shall return all supported DK Protocol Version using two bytes per version in big endian coding. n is the number of protocol versions supported by the vehicle The DK Protocol Version shall be ordered from the highest to the lowest version. In this specification the vehicle shall support the DK Protocol Version 1.0 (coded 0100_h). The device shall select the highest DK Protocol Version commonly supported by device and vehicle as described in Selected_DK_Protocol_Version attribute (See Table 19-14) The first supported DK Protocol Version is 0100_h.
Features Supported length	1	1 byte is the length in bytes for the Features Supported Field per DK protocol version
Features Supported	k	k is the number of bytes needed to convey the bitfield for the list of features supported. Refer to Table 19-15.

Table 19-14: Device Selected DK Version Characteristic

Attribute value	Length (Bytes)	Description
Selected_DK_Protocol_Version	2	Device-selected highest DK Ranging Protocol Version commonly supported by device and vehicle. In this specification, the device shall support the DK Ranging Protocol Version 1.0 (coded 0100_h). Attribute may not be included if Supported_DK_Ranging_Protocol_Version_Len = 0x00. Refer to Table 19-13.
Features Supported length	1	Length in bytes for the Features Supported Field
Features Supported	k	k is the number of bytes needed to convey the bitfield for the list of features supported in common by vehicle and device. Refer to Table 19-15.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The device shall respond with the latest version supported. DK Protocol Major Version and DK Protocol Minor Version attribute values shall be encoded as big endian.

DK Protocol Major Version may not be backward compatible with previous DK Protocol Major Versions.

DK Protocol Minor Version shall be compatible with previous DK Protocol Minor Version for the same DK Protocol Major Version.

If UUID_SPSM_DK_VERSION is not available on vehicle, it shall be interpreted that the vehicle supports DK Protocol Version 1.0 (i.e., DK Protocol Major Version is 1 and DK Protocol Minor Version is 0).

If the device supports DK Protocol version 1.0, it shall request the UUID_SPSM from the vehicle.

If the device supports DK Protocol Version higher than 1.0:

- Device shall request the UUID_SPSM_ DK_VERSION characteristic from the vehicle. If the vehicle doesn't support "UUID_SPSM_DK_VERSION", the device shall ask for "UUID_SPSM" since the vehicle only supports DK Protocol version 1.0.
- Device shall use the highest version commonly supported by both vehicle and device.
- If vehicle supports V-D-BT version higher than 1.0, device shall write the Selected_DK_Protocol_Version and Features_Supported to the Vehicle UUID_DEVICE_ DK_VERSION characteristic.

If the vehicle supports DK Protocol Version 1.0, it shall implement UUID_SPSM. If the vehicle that does not support UUID_SPSM_DK_VERSION and the UUID_DEVICE_DK_VERSION characteristic receives a request for the UUID_SPSM_DK_VERSION or UUID_DEVICE_DK_VERSION from a device, it shall respond with the error code "Attribute not found" as indicated by Bluetooth specification (refer to Bluetooth core specification v5.0 Vol 3, Part F 3.4.4.1).

If the vehicle supports DK Protocol Version higher than 1.0:

- Vehicle shall implement the UUID_SPSM and the UUID_SPSM_DK_VERSION characteristic

The total size of the attribute value for UUID_SPSM and UUID_SPSM_DK_Version shall be less than 512 bytes. If this value is exceeded, it shall keep the most recent anchor versions followed by additional supported versions to make it fit within 512 Bytes.

Features Supported definition:

Features Supported shall be defined as follow:

Table 19-15: Supported Feature Definition Bitmap

Attribute type/ Feature Supported	Attribute value	Description
Bit[0]	0 or 1	1_b: TimeSync Procedure 0 Supported Optional for vehicle, See 19.4.4.1
Bit[1]	0 or 1	1_b: TimeSync Procedure 1 Supported

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

		Optional for vehicle, See 19.4.4.2
Bit[2]	0 or 1	1b: URSK derivation via Standard Transaction for other purposes Optional for device and vehicle, See 19.5.6.1
Bit[3]	0 or 1	1b: Head Unit Pairing Optional for device and vehicle, See 19.5.2
Bit[4]	0 or 1	1b: LE Coded PHY Passive Entry Advertising Optional for device and vehicle, See 19.2.3.1

19.2.1.9 Vehicle Bluetooth Tx Power Level Characteristic:

The Vehicle Bluetooth Tx Power Level characteristic is an optional attribute provided by the vehicle. The Vehicle Bluetooth Tx Power Level characteristic shall return the current radiated transmit power of a Bluetooth LE module in dBm [see GATT specification, supplement version 4].

Table 19-16 Vehicle Bluetooth Tx Power Level Characteristics

Attribute type	Data Type	Size (in octets)
0x2A07 – UUID for «Tx Power Level»	sint8	1

The device should read the Vehicle Bluetooth Tx Power level from the vehicle upon connection.

During passive-entry, the device should read the characteristics by UUID (with the UUID for “Tx power level”) at reconnection. The Bluetooth LE module on the vehicle should keep the same radiated transmit power during the entire time of the Bluetooth LE connection.

19.2.1.10Vehicle Antenna Identifier Characteristic:

The Vehicle antenna identifier represents a unique identifier for every physical Bluetooth LE antenna located in the vehicle that establishes a connection to a device. The antenna identifier is defined as a unique value that correspond to a unique physical Bluetooth antenna in the vehicle. This shall remain a unique antenna identifier even if a different device connects to that antenna. The device should read the “Vehicle Antenna Identifier” from the vehicle upon connection.

In the case a Bluetooth module has multiple antennas, it can be considered as single unique single antenna. Also, if multiple Bluetooth modules share multiple antennas, then it can also be considered as a single antenna.

The GATT characteristic instance should be unique per client connecting to the vehicle if the vehicle supports multiple concurrent Bluetooth LE connections.

During passive-entry, the device should read the characteristics by UUID (with the UUID_ AntennaIdentifier) at reconnection.

Table 19-17: Vehicle Antenna Characteristic declaration

Attribute type	Attribute value	Attribute permission

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

0x2803 – UUID for «Characteristic»	Characteristic Properties = 0x02	0xMMMM = Handle of Characteristic Value	c6d7d4a1-e2b0-4e95-b576-df983d1a5d9f – UUID_ AntennaIdentifier	Read only, No Authentication, No Authorization

1
2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-18: Vehicle Antenna Characteristic value declaration

Attribute type	Attribute value	Attribute permission
c6d7d4a1-e2b0-4e95-b576-df983d1a5d9f – UUID_ AntennaIdentifier	Uint16 – Vehicle Antenna Identifier	Read only, No Authentication, No Authorization

19.2.2 *Bluetooth Encryption*

The following requirements apply to Bluetooth encryption:

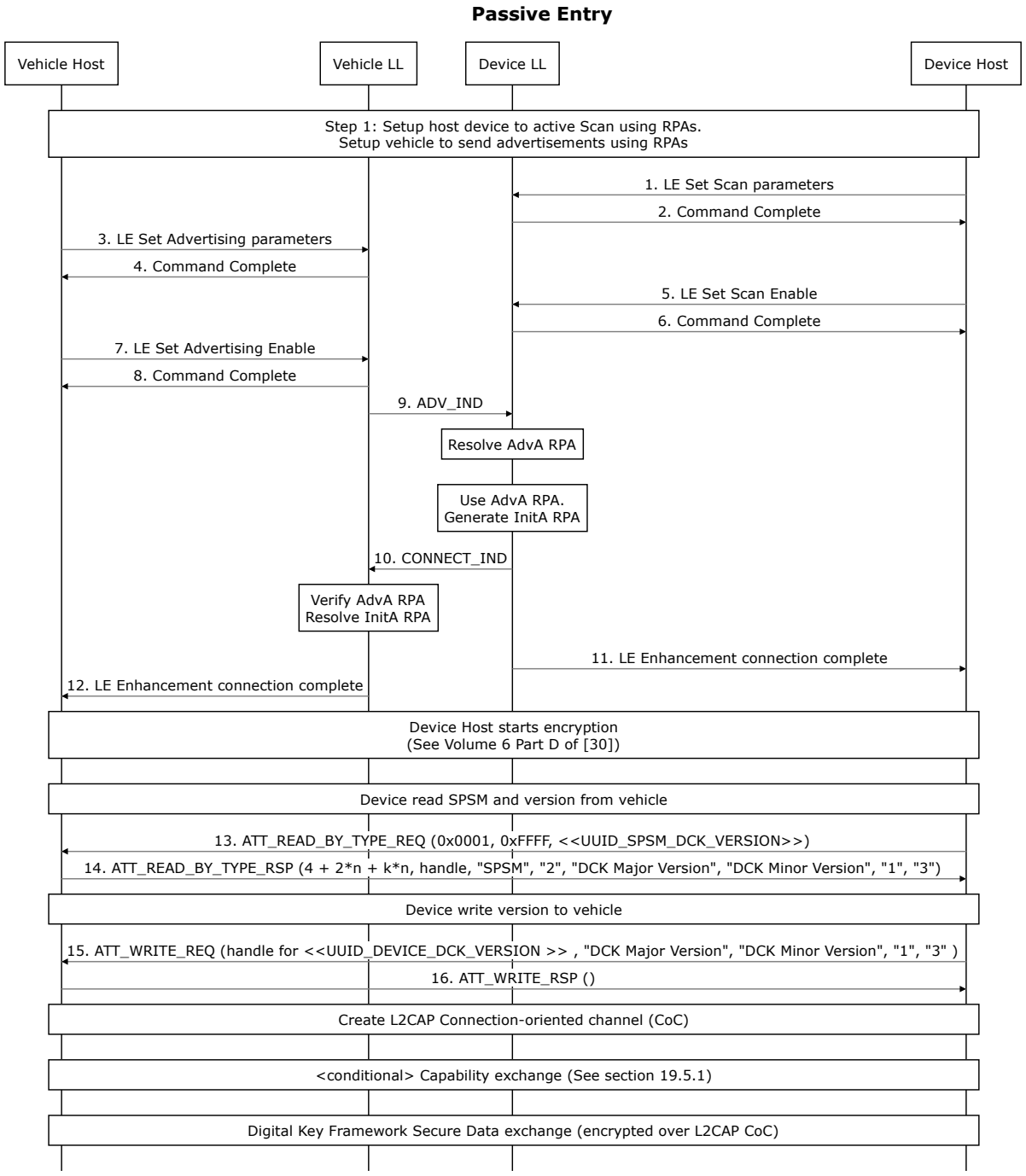
- The device (central) shall request encryption and encryption shall successfully complete before L2CAP connection establishment is initiated, except for owner paring and first friend approach.
- The vehicle shall trigger a disconnect 5 seconds after an unencrypted L2CAP connection establishment if no First_Approach_RQ (see Section 19.3.4.1) or Request_owner_pairing Command Complete SubEvent notification (see Table 19-72) has been received during the first approach and owner pairing, respectively.
- The device and vehicle should not terminate the L2CAP connection used for DK Service while the BLE connection is alive. In case the L2CAP connection used for DK Service is terminated while the Bluetooth LE connection is alive, the device shall attempt to re-establish the DK Service L2CAP connection once conditions permit.

19.2.3 *Passive Entry: Bluetooth LE Setup:*

Once the Bluetooth LE Pairing and Encryption of the Bluetooth LE setup is completed, the subsequent connection to the vehicle shall use the passive entry flow.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-4: Passive Entry Flow Diagram.



For passive entry, Capability Exchange is conditional upon whether either the vehicle or device has an updated DK Protocol Version, UWB Configuration Identifier, PulseShape Combinations, or a combination of these parameters since the last Capability Exchange, as described in Section 19.5.1.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.2.3.1 Passive Entry Advertising:

The vehicle shall advertise on the LE 1M PHY.

When advertising on the LE 1M PHY, the connectable and scannable undirected event shall be used, see Volume 6, Part B, Section 4.4.2.7 in [30]. The advertising interval shall be 42.5ms.

The vehicle may advertise on both the LE Coded PHY with S=2 coding and on the LE 1M PHY. If both PHYs are used, the advertising for each shall occur consecutively.

When advertising on the LE Coded PHY, the connectable undirected event shall be used, see Volume 6, Part B, Section 4.4.2.3 in [30]. The advertising interval shall be 84ms.

For both advertisements, the ADV_IND shall contain AdvA but no advertising data (AdvData).

A paired device shall be able to connect to both the owner pairing advertising and the passive entry advertising to perform the passive entry flows. For known devices connecting to passive entry as well as owner pairing advertisement, the vehicle shall be able to perform the Passive Entry Flow.

19.2.3.2 Passive Entry L2CAP Connection

Upon establishment of the LE link layer connection, the device host shall establish an L2CAP connection using the protocol specified in Section 19.2.1.1.

19.2.3.3 Connection Performance Recommendations

In this section, the connection performance parameters and their values are recommended. The exact values are refined and finalized in the certification/test plan specification, which form the certification requirement. All TBD values in this section will be finalized/measured during interoperability event.

- **Device requirement:**

  During the approach of the user toward the vehicle, the last Bluetooth message before the UWB pre-poll should be sent by 3 (TBD_1) meters for 95 (TBD_2) percentiles. This shall be based on the following assumptions:

  - The maximum approach walk speed is 2.1m/s and the test start at 6 (TBD_3) meters.
  - The conditions defined in certification/test plan specification.
  - Device starts Bluetooth LE scanning at the start of test.
- **Transmitter requirements:**
  - Bluetooth LE Advertisement packet (from vehicle), measured with reference measurement setup, at 10 meters (TBD_5) should be equal or greater than -80 dBm (TBD_4) with a free-space LOS path loss and common test case condition (see note below).
  - The radiated power output power level (including antenna gain) of a CONNECT_IND packet from mobile device should be equal or greater than -80 dBm (TBD_6) at 10 meters (TBD_5) with a free-space LOS path loss and common test case condition (see note below).

Note: Reference measurement setup and common test case condition are defined in certification/test plan specification.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

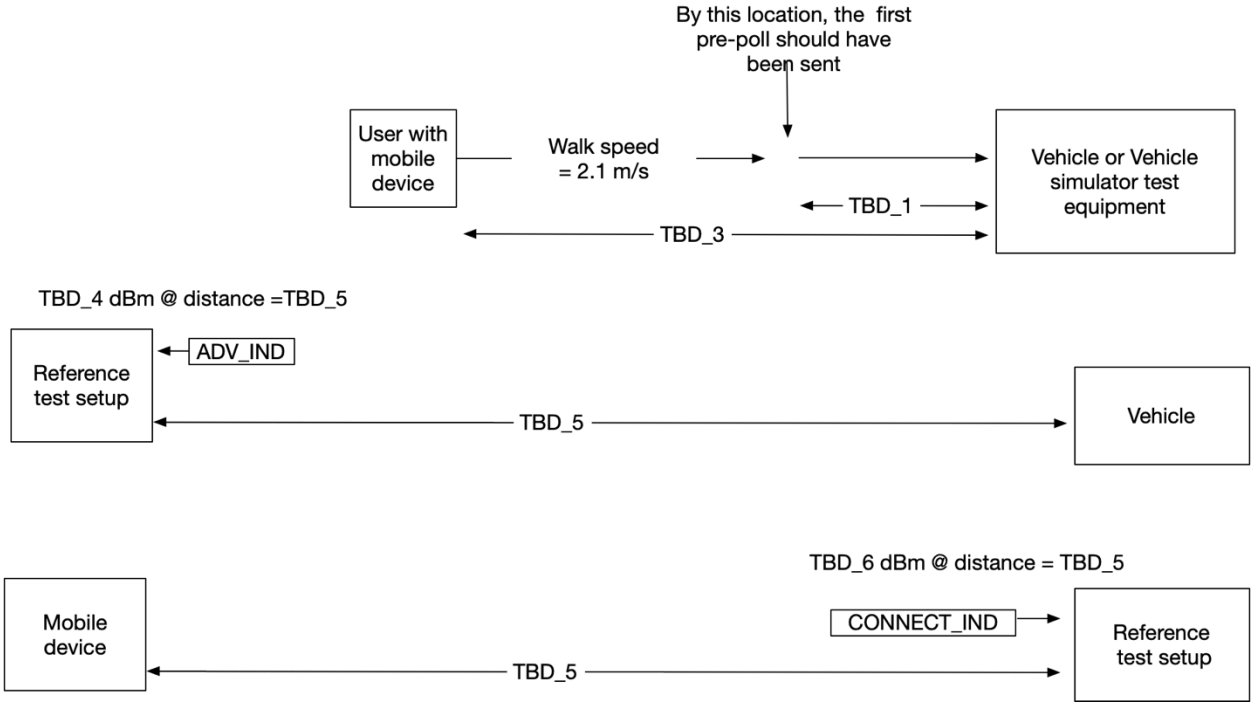
- **Receiver requirements:**
  The actual sensitivity level, as defined in Version 5.2 Vol 6 Part A Section 4 of [30]:
  - Should be equal or better than -85dBm for un-coded PHYs and -88dBm for LE Coded PHY with S=2 coding for the device (see note below).
  - Should be equal or better than -85dBm for un-coded PHYs and -88dBm for LE Coded PHY with S=2 coding for the vehicle (see note below).

Note: Reference measurement setup and common test case condition are defined in certification/test plan specification.

- **General requirements:**
  Test should be done using the optimal ranging setup flow with a previously derived URSK

Any other losses in the Bluetooth link such as vehicle body or interior should be kept at a minimum or at best avoided.

Figure 19-5: Connection performance in relationship with device, transmitter, and receiver requirements.



19.2.3.4 Vehicle Requirements

Maximum advertising intervals:

Maximum number of advertising events per time:

Locked vehicles shall transmit within 504ms (6 periods of 84ms) up to a maximum of

- (a) 24 LE 1M + 12 LELR or
- (b) 36 LE 1M + 6 LELR or
- (c) 48 LE 1M (zero LELR)

advertising events (See Section 19.2.3.1).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Unlocked vehicles shall transmit within 504ms (6 periods of 84ms) up to a maximum of

- (a) 48 LE 1M + 24 LELR or
- (b) 72 LE 1M + 12 LELR or
- (c) 96 LE 1M (zero LELR)

advertising events (See Section 19.2.3.1).

19.3 DK Message Format

The DK messages shall be exchanged over L2CAP using the DK Service SPSM.

The device shall retrieve the DK Service SPSM from the vehicle's GATT server and establish an LE L2CAP credit-based connection to the vehicle's SPSM for use by the DK Protocol (See Figure 19-2).

The DK message format is shown in Table 19-19.

Table 19-19: DK Message Format.

Message Field	Location
Message Header	Byte 0
Payload Header	Byte 1
Length	Byte [3:2]
Data	N

The Message Header field is one byte long and indicates the Message Type of the message being sent. For example, if the message being sent is a Select APDU wrapped within DK_APDU_RQ (see Section 19.3.2.1 for more details), the Message Header field indicates whether this message is an SE message for standard transaction or Framework message for owner pairing, for which the associated message processing differs by the type of message. On the other hand, the Payload Header field is 1 byte long and communicates the message such as DK_APDU_RQ.

The Message Header is defined in Table 19-20 and the Message Type is defined in Table 19-21. Bit [7:6] of the Message Header are reserved for future use and shall be set to 0.

Table 19-20: Message Header definition.

Message Header	Bit field
Message Type	Bit [5:0]
RFU	Bit [7:6]

Table 19-21: Message Type definition.

Message Type definition Bit [5:0]	Value (decimal)
Framework message	0
SE message	1
UWB Ranging Service message	2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Message Type definition Bit [5:0]	Value (decimal)
DK Event Notification	3
Vehicle OEM App message	4
Supplementary Service message	5
Head Unit Pairing message	6
Reserved	7–63

The Payload Header field is defined in Table 19-22:

Table 19-22: Payload Header definition.

Payload Header	Bit field
Message ID	Bit [7:0]

The Length field is two bytes long and indicates the size of the Data field in the message coded in Big Endian format. All the DK message fields shall be encoded in Big Endian format, except for data type fields already defined by Bluetooth Specification or otherwise explicitly defined in this specification. Those data types ordering defined by Bluetooth Specification shall remain unchanged.

The Data field is variable in length and its data structure definition is defined in Sections 19.3.1 to 19.3.8 for each message ID.

Table 19-23 shows the categorization of DK message under different Message Type.

Table 19-23: Message Type and its associated messages.

Message Type	Message	APDUs
Framework	DK_APDU_RQ DK_APDU_RS (As defined in Section 19.3.2)	SELECT, SPAKE2+ REQUEST, SPAKE2+ VERIFIER, WRITE DATA, GET DATA, GET RESPONSE, OP CONTROL FLOW, See Table 5-1
SE	DK_APDU_RQ DK_APDU_RS (As defined in Section 19.3.2)	SELECT, AUTH0, AUTH1, EXCHANGE CONTROL FLOW CREATE RANGING KEY See Table 15-1
UWB Ranging Service	Ranging_Capability_RQ Ranging_Capability_RS Ranging_Session_RQ	N/A

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Message Type	Message	APDUs
	Ranging_Session_RS Ranging_Session_Setup_RQ Ranging_Session_Setup_RS Ranging_Suspend_RQ Ranging_Suspend_RS Ranging_Recovery_RQ Ranging_Recovery_RS Configurable_Ranging_Recovery_RQ Configurable_Ranging_Recovery_RS (As defined in Section 19.3.1)	
DK Event Notification	Ranging_Event (As defined in Section 19.3.8)	N/A
Vehicle OEM App	Pass_Through (As defined in Section 19.3.6)	N/A
Supplementary Service	Time_Sync (As defined in Section 19.3.3) First_Approach_RQ First_Approach_RS (As defined in Section 19.3.4) RKE_Auth_RQ RKE_Auth_RS (As defined in Section 19.3.5)	N/A
Head Unit Pairing	Head_Unit_Pairing_Preparation Head_Unit_Pairing_RQ Head_Unit_Pairing_RS (As defined in Section 19.3.7)	N/A

Below is an example on how to encode an APDU message before transporting it over Bluetooth LE by using DK Message. The same can also be used to decode the incoming message.

```
Message: Select APDU message (See Section 15.3.2.1) over Bluetooth LE with
AID as 0xA0000008094343434444B417631.
Message Type in Message Header: 0x01 (SE)
Message ID in Payload Header: 0x0B (DK_APDU_RQ, 11)
Data: 0x00A404000DA0000008094343434444B41763100
(SELECT APDU command)
DK Bluetooth LE Payload: 0x010B001300A404000DA0000008094343434444B41763100
```

19.3.1 *UWB Ranging Service Message*

This section details the Ranging Service messages to negotiate, initiate and complete UWB ranging between the device and the vehicle.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.3.1.1 Ranging Capability Request (RC-RQ)

This message is sent by the vehicle to the device during ranging capability exchange. The Ranging Capability Request message and its parameters are defined in Table 19-24 and Table 19-25, respectively.

Table 19-24: Ranging_Capability_RQ message and its parameters.

Message	Message ID	Parameters
Ranging_Capability_RQ	0x01	Supported_DK_Protocol_Version_Len, Supported_DK_Protocol_Version, Supported_UWB_Config_Id_Len, Supported_UWB_Config_Id Supported_PulseShape_Combo_Len Supported_PulseShape_Combo

Table 19-25: Definition of the parameters for Ranging_Capability_RQ.

Parameters	Length (bytes)	Description
Supported_DK_Protocol_Version_Len	1	Length byte for supported DK protocol versions
Supported_DK_Protocol_Version	2 × n	DK_Protocol_Version is 2 Byte field (Major:Minor); as defined in Table 19-13. If a DK_Protocol_Version was selected as part of the GATT version negotiation, this field shall contain only the selected DK_Protocol_Version. The first supported DK Protocol Version is 0100_h
Supported_UWB_Config_Id_Len	1	Length byte for supported UWB Configuration Identifiers
Supported_UWB_Config_Id	2 × m	UWB_Config_Id is a 2 Byte field which is an identifier for the supported UWB configuration. This configuration includes the supported PHY layer parameters. See Section 21.4 for the supported UWB PHY layer configurations. m is the number of UWB configs supported by vehicle
Supported_PulseShape_Combo_Len	1	Length byte for the supported PulseShape_Combos
Supported_PulseShape_Combo	1 × *l*	PulseShape_Combo is a 1 Byte field which is an identifier for the supported initiator/response, transmit/receive pulse shape combinations. See Section 21.5 for the supported subset. *l* is the number of PulseShape_Combos supported by the vehicle.

19.3.1.2 Ranging Capability Response (RC-RS)

The message is sent by the device to the vehicle in response to a Ranging Capability Request message (RC-RQ) during ranging capability exchange. The Ranging Capability Response message and its parameters are defined in Table 19-26 and Table 19-27, respectively.

Table 19-26: Ranging_Capability_RS message and its parameters.

Message	Message ID	Parameters
Ranging_Capability_RS	0x02	Selected_DK_Protocol_Version, Selected_UWB_Config_Id Selected_PulseShape_Combo

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-27: Definition of the parameters for Ranging_Capability_RS.

Parameters	Length (bytes)	Description
Selected_DK_Protocol_Version	2	Highest DK Protocol Version commonly supported by device and vehicle defined in Table 19-14. The first supported DK Protocol Version is 0100_h. For DK Protocol Version higher than version 1.0, please refer to Section 19.2.1.7. If supported, the DK Protocol Version selected during GATT negotiation shall be used by device.
Selected_UWB_Config_Id	2	Device-selected UWB config id from the list of supported common UWB config ids between device and vehicle.
Selected_PulseShape_Combo	1	Device-selected single pulse shape combination from the list of common supported ones between device and vehicle.

19.3.1.3 Ranging Session Request (RS-RQ)

This message is sent by the vehicle to the device to start the ranging session setup handshake procedures as defined in Section 20.5.2. The Ranging_Session_RQ message and its parameters are defined in Table 19-28 and Table 19-29, respectively

The vehicle shall use the Selected_DK_Protocol_Version, Selected_UWB_Config_id and Selected_PulsShape_Combo as selected by the device during ranging capability exchange (see Figure 19-18).

If the vehicle has an updated DK Protocol Version or UWB Configuration Identifier or PulseShape Combinations or combination of these parameters, the vehicle shall perform Capability Exchange with the device again as described in Section 19.5.1.

Table 19-28: Ranging_Session_RQ message and its parameters.

Message	Message ID	Parameters
Ranging_Session_RQ	0x03	Selected_DK_Protocol_Version, Selected_UWB_Config_id, UWB_Session_Id, Selected_PulseShape_Combo, Channel_Bitmask

Table 19-29: Definition of the parameters for Ranging_Session_RQ.

Parameters	Length (bytes)	Description
Selected_DK_Protocol_Version	2	Selected protocol version from ranging capability exchange (RC-RQ, RC-RS)
Selected_UWB_Config_Id	2	Selected UWB config id from ranging capability exchange (RC-RQ, RC-RS)
UWB_Session_Id	4	The least significant 4 bytes of the transaction_identifier field. For more details on transaction_identifier, See Table 15-27. Vehicle selects the transaction_identifier based on which pre-derived URSK it chooses to use for establishing UWB ranging session. The least significant 4 bytes of that transaction_identifier is then used as UWB_Session_Id. Device uses UWB_Session_Id for URSK retrieval and session management.
Selected_PulseShape_Combo	1	Device-selected single pulse shape combination from the list of common supported ones between device and vehicle.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameters	Length (bytes)	Description
Channel_Bitmask	1	A bitmap equaling the bitwise “OR” of supported UWB channels Bit 0: UWB channel 5 supported Bit 1: UWB channel 9 supported Bit 2: RFU (shall be set to 0) Bit 3: RFU (shall be set to 0) Bit 4: RFU (shall be set to 0) Bit 5: RFU (shall be set to 0) Bit 6: RFU (shall be set to 0) Bit 7: RFU (shall be set to 0)

19.3.1.4 Ranging Session Response (RS-RS)

This message is sent by the device according to Section 20.5.2. The Ranging Session Response message and its parameters are defined in Table 19-30 and Table 19-31, respectively. If the device has an updated DK Protocol Version or UWB_Config_Id or PulseShape Combinations or combination of these parameters which is different than the ones it received in Ranging Session Request, the device shall respond with DK Event Notification with Subevent_Category set to $0x01_h$ and the corresponding command status code set to $0x02_h$, to initiate Capability Exchange as described in Section 19.5.1.

Table 19-30: Ranging_Session_RS message and its parameters.

Message	Message ID	Parameters
Ranging_Session_RS	0x04	RAN_Multiplier, Slot_BitMask, SYNC_Code_Index_BitMask, Selected_UWB_Channel, Hopping_Config_Bitmask

Table 19-31: Definition of the parameters for Ranging_Session_RS.

Parameters	Length (bytes)	Value	Description
RAN_Multiplier	1	Range = 1 to 255 T_Block RAN = RAN_Multiplier × 96ms Time Range = 96ms to 24480 ms	RAN multiplier for setting the ranging block duration. See Section 20.5.2 for the definition ranging block.
Slot_BitMask	1	A bitmap equaling the bitwise “OR” combination of 0x01, 0x02, 0x04, 0x08, 0x10, 0x20,0x40, 0x80.	Bitmap of supported values of Slot durations as a multiple of T_{Chap}, $N_{Chap_per_Slot}$ as defined in Section 20.5.2. Each “1” in this bit map corresponds to a specific value of $N_{Chap_per_Slot}$ where: 0x01 = “3” 0x02 = “4” 0x04= “6” 0x08 =“8” 0x10 =“9” 0x20 = “12” 0x40 = “24” and 0x80 is reserved. Note that 0x40 is used for testing only

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameters	Length (bytes)	Value	Description
SYNC_Code_Index_BitMask	4	A bitmap equaling the bitwise “OR” combination of 0x00000001, 0x00000002, 0x00000004, … 0x40000000, 0x80000000	Bitmap of SYNC code indices that can be used. The position of each “1” in this bit pattern corresponds to the index of a SYNC code that can be used, where: 0x00000001 = “1” 0x00000002 = “2” 0x00000004 = “3” 0x00000008 = “4” … 0x40000000 = “31” 0x80000000 = “32” Refer to [31] and Section 21.4.1 for SYNC code index definition.
Selected_UWB_Channel	1	“0x01 for channel 5” and “0x02 for channel 9”	Selected UWB channel for requested session. See Section 21.7.1.
Hopping_Config_Bitmask	1	0-255	[b7 b6 b5] bitmask of hopping modes, the device offers to use in this ranging session 100 No Hopping 010 Continuous Hopping 001 Adaptive Hopping The mandatory mode is 010 corresponding to Continuous Hopping mode. The support of 100 or 001 is optional. [b4 b3 b2 b1 b0] bit mask of hopping sequences the device offers to use in this ranging session. b4=1 is always set since the default hopping sequence and support thereof is mandatory and defined in Appendix A. b3=1 is set when the optional AES-based hopping sequence, which is defined in Appendix G, is supported. All remaining bits reserved for future hopping sequences.

19.3.1.5 Ranging Session Setup Request (RSS-RQ)

This message is sent by the vehicle as part of the ranging session setup handshake according to Section 20.5.2. The Ranging Session Setup Request message and its parameters are defined in Table 19-32 and Table 19-33, respectively.

Table 19-32: Ranging_Session_Setup_RQ message and its parameters.

Message	Message ID	Parameters
Ranging_Session_Setup_RQ	0x05	Session_RAN_Multiplier, Number_Chaps_per_Slot, Number_Responders_Nodes, Number_Slots_per_Round, SYNC_Code_Index, Selected_Hopping_Config_Bitmask See Section 20 for the definition of these parameters.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1 *Table 19-33: Definition of the parameters for Ranging_Session_Setup_RQ.*

Parameters	Length (bytes)	Value	Description
Session_RAN_Multiplier	1	Range = 1 to 255 T_Block_S = Session_RAN_Multiplier × 96 ms Time Range = 96ms to 24480 ms	Session specific RAN multiplier. Value selected by vehicle must be greater than or equal to RAN_Multiplier sent by mobile device.
Number_Chaps_per_Slot	1	3, 4, 6, 8,9, 12, or 24 Note that the last value is used only for testing	Selected Slot duration as a multiple of T_{Chap}, $N_{Chap_per_Slot}$ as defined in Section 20.
Number_Responders_Nodes	1	1-255	Number of logical responder nodes participating in this ranging session as selected by the vehicle. See [31]for the definition of a logical node.
Number_Slots_per_Round	1	6,8,9,12,16,18,24,32,36,48,72, or 96	Selected number of slots per round
SYNC_Code_Index	4	A bitmap equaling the bitwise "OR" combination of 0x00000001, 0x00000002, 0x00000004, … 0x40000000, 0x80000000	Bitmap of SYNC code indices that the vehicle can use (this may be a smaller set of the SYNC codes support by the vehicle). The position of each "1" in this bit pattern corresponds to the index of a SYNC code that can be used, where: 0x00000001 = "1" 0x00000002 = "2" 0x00000004 = "3" 0x00000008 = "4" … 0x40000000 = "31" 0x80000000 = "32" See [31] and Section 21 for SYNC code index definition.
Selected_Hopping_Config_-Bitmask	1	0-255	Vehicle shall send a bitmask indicating the selected hopping mode and the selected hopping sequence (in case of continuous or adaptive hopping). This bitmask shall meet the following rules: - For [b7 b6 b5], only one of those bits shall be set to 1 - For [b5 b4 b3 b2 b1 b0], at max one of the bits is set to 1 to signal the selected hopping sequence and the rest of the bits are set to 0. Examples of a valid Selected_Hopping_Config_BitMask [10000000] no hopping mode [01010000] continuous hopping with default hopping sequence [00110000] adaptive hopping with default hopping sequence [01001000] continuous hopping with AES-based optional hopping sequence [00101000] adaptive hopping with AES-based optional hopping sequence

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.3.1.6 Ranging Session Setup Response (RSS-RS)

This message is sent by the device to the vehicle to complete the ranging session setup handshake according to Section 20.5.2. The Ranging Session Setup Response message and its parameters are defined in Table 19-34 and Table 19-35, respectively.

Table 19-34: Ranging_Session_Setup_RS message and its parameters.

Message	Message ID	Parameters
Ranging_Session_Setup_RS	0x06	STS_Index0, UWB_Time0, HOP_Mode_Key, SYNC_Code_Index See Section 20 for the definition of these parameters

Table 19-35: Definition of the parameters for Ranging_Session_Setup_RS.

Parameters	Length (bytes)	Value	Description
STS_Index0	4	0 - 0x3FFFFFFF	Starting STS index
UWB_Time0	8	0 - 0xFFFFFFFFFFFFFFFF	Starting time reference (resolution 1 microsecond) on UWB Device Clock of ranging session (see Section 20.3).
HOP_Mode_Key	4	0 - 0xFFFFFFFF	Key to generate default or AES based hopping sequence. In case of no hopping mode, this parameter shall be set to 0.
SYNC_Code_Index	1	1-32	Selected SYNC code index

19.3.1.7 Ranging Suspend Request Message (RSD-RQ)

This message is used to suspend an active ranging session for a given UWB_Session_Id. The Ranging Suspend Request message and its parameters are defined in Table 19-36 and Table 19-37, respectively.

Table 19-36: Ranging_Suspend_RQ message and its parameter.

Message	Message ID	Parameters
Ranging_Suspend_RQ	0x07	UWB_Session_Id

Table 19-37: Definition of the parameter for Ranging_Suspend_RQ.

Parameters	Length (bytes)	Description
UWB_Session_Id	4	The UWB Session ID of the currently active UWB ranging session.

19.3.1.8 Ranging Suspend Response Message (RSD-RS)

This message is used as a successful response to Ranging Suspend Request (RS-RQ). The Ranging Suspend Response message and its parameters are defined in Table 19-38 and Table 19-39, respectively.

Table 19-38: Ranging_Suspend_RS message and its parameter.

Message	Message ID	Parameters
Ranging_Suspend_RS	0x08	Suspend_Response

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-39: Definition of the parameter for Ranging_Suspend_RS.

Parameters	Length (bytes)	Description
Suspend_Response	1	0x00 - For accepted suspend request 0x01 - To delay the suspend and keep ranging active

19.3.1.9 Ranging Recovery Request Message (RR-RQ)

This message is sent by vehicle to recover a suspended ranging session for a given UWB_Session_Id. The Ranging Recovery Request message and its parameters are defined in Table 19-40 and Table 19-41, respectively. The device and vehicle shall use the Session_RAN_Multiplier negotiated prior to ranging suspension (see Section 19.3.1.5).

Table 19-40: Ranging_Recovery_RQ message and its parameter.

Message	Message ID	Parameters
Ranging_Recovery_RQ	0x09	UWB_Session_Id

Table 19-41: Definition of the parameter for Ranging_Recovery_RQ.

Parameters	Length (bytes)	Description
UWB_Session_Id	4	The UWB Session ID of a previously suspended UWB ranging session.

19.3.1.10 Ranging Recovery Response Message (RR-RS)

This message is sent by device as a successful response to vehicle's Ranging Recovery Request (RR-RQ). The Ranging Recovery Response message and its parameters are defined in Table 19-42 and Table 19-43, respectively.

Table 19-42:Ranging_Recovery_RS message and its parameter.

Message	Message ID	Parameters
Ranging_Recovery_RS	0x0A	STS_Index0, UWB_Time0

Table 19-43: Definition of the parameter for Ranging_Recovery_RS.

Parameters	Length (bytes)	Description
STS_Index0	4	Starting STS index
UWB_Time0	8	Starting time reference (resolution 1 microsecond) on UWB Device Clock of ranging session (see Section 20.3).

19.3.1.11 Configurable Ranging Recovery Request Message (CRR-RQ)

This message is sent by vehicle to recover a suspended ranging session for a given UWB_Session_Id. The Configurable Ranging Recovery Request Message and its parameters are defined in Table 19-44 and Table 19-45, respectively. This message enables the vehicle to request a different RAN Multiplier value than the one selected prior to the suspension (i.e. Session_RAN_Multiplier). For example, the vehicle may request an aggressive frequency to support higher ranging rate for critical use cases such as door handle touch sensor.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-44: Configurable_Ranging_Recovery_RQ message and its parameter.

Message	Message ID	Parameters
Configurable_Ranging_Recovery_RQ	0x12	UWB_Session_Id Requested_RAN_Multiplier

Table 19-45: Definition of the parameter for Configurable_Ranging_Recovery_RQ.

Parameters	Length (bytes)	Description
UWB_Session_Id	4	The UWB Session ID of the currently active UWB ranging session.
Requested_RAN_Multiplier	1	Requested RAN Multiplier for the ranging session to be recovered. The Requested_RAN_Multiplier shall be different than the Session_RAN_Multiplier prior to ranging suspension (see Section 19.3.1.5).

19.3.1.12 Configurable Ranging Recovery Response Message (CRR-RS)

This message is sent by device as a response to CRR-RQ. The Configurable Ranging Recovery Response message and its parameters are defined in Table 19-46 and Table 19-47, respectively.

Table 19-46: Configurable_Ranging_Recovery_Response message and its parameter.

Message	Message ID	Parameters
Configurable_Ranging_Recovery_RS	0x13	Selected_RAN_Multiplier STS_Index0, UWB_Time0

Table 19-47: Definition of the parameter for Configurable_Ranging_Recovery_Response.

Parameters	Length (bytes)	Description
Selected_RAN_Multiplier	1	Selected RAN Multiplier for the ranging session to be recovered. The Selected_RAN_Multiplier shall be equal or greater than the Requested_ RAN_Multiplier
STS_Index0,	4	Starting STS index
UWB_Time0	8	Starting time reference (resolution 1 microsecond) on UWB Device Clock of ranging session (see Section 20.3).

The Selected_RAN_Multiplier shall be used during the recovered ranging session and shall remain valid until the ranging session ends or the ranging session is suspended. When the ranging session is recovered again, the RAN_Multiplier value for the given ranging session shall revert to the default Session_RAN_Multiplier for that ranging session.

19.3.2 *SE Message*

19.3.2.1 APDU Command Encapsulation

The DK_APDU_RQ message is an encapsulation of the APDU command specified as available on the wired interface in Table 15-1. The message and its parameters are defined in Table 19-48 and Table 19-49, respectively.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-48: DK_APDU_RQ message and its parameter.

Message	Message ID	Parameter
DK_APDU_RQ	0x0B	APDU command

Table 19-49: Definition of the parameter for DK_APDU_RQ.

Parameter	Length (bytes)	Description
APDU command	Variable	List of APDU commands are described in Table 19-23. Allowed class byte values are limited to 00_h (for SELECT command), 80_h (for commands with non-secure messaging) or 84_h (for commands with secure messaging).

If the Digital Key framework has selected the Digital Key applet over a supplementary logical channel, it shall update the class byte of the APDU command accordingly before forwarding the APDU command to the Digital Key applet.

19.3.2.2 APDU Response Encapsulation

The DK_APDU_RS message is an encapsulation of the APDU response. Each DK_APDU_RQ message requires a corresponding DK_APDU_RS message. The message and its parameter are defined in Table 19-50 and Table 19-51, respectively.

Table 19-50: DK_APDU_RS message and its parameter.

Message	Message ID	Parameter
DK_APDU_RS	0x0C	APDU response

Table 19-51: Definition of the parameter for DK_APDU_RS.

Parameter	Length (bytes)	Description
APDU response	Variable	The APDU response corresponding to the APDU command

19.3.3 *Supplementary Service Message - Time Sync*

This message is used by device to provide Bluetooth LE Timesync payload which includes, the DeviceEventCount, the UWB Device Time timestamp and the UWB Device Time uncertainty.

The following two conditions shall trigger a timestamp (see Section 0):

- **Procedure 0**: After a CONNECT_IND, the device shall send a Time_Sync message
- **Procedure 1**: Once Bluetooth LE connection has been established, the vehicle may trigger a new Bluetooth LE event-based timestamp by requesting “LE Set PHY”. Device shall use the anchor point of the connection event used for “LL_PHY_UPDATE_IND” Link Layer message to generate a new time synchronization timestamp and send it to vehicle using this message.

Table 19-52 and Table 19-53 describe the parameters meaning based on which condition the timesync message was triggered.

Table 19-52: Time_Sync message and its parameters.

Message	Message ID	Parameters
Time_Sync	0x0D	DeviceEventCount,

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Message	Message ID	Parameters
		UWB_Device_Time, UWB_Device_Time_Uncertainty, UWB_Clock_Skew_Measurement_available, Device_max_PPM, Success, RetryDelay

1 *Table 19-53: Definition of the parameter for Time_Sync.*

Parameters	Length (bytes)	Value	Description
DeviceEventCount	8	0 – 0xFFFFFFFFFFFFFFFE	Procedure 0: Event count is 0 or in case of retransmission, close to 0. Procedure 1: Event count value used for the LL_PHY_UPDATE_IND Link layer message. A value of 0xFFFFFFFFFFFFFFFF indicates DeviceEventCount is unavailable on the device.
UWB_Device_Time	8	0 – 0xFFFFFFFFFFFFFFFF	UWB Ranging clock value in microseconds. This is running clock and shall remain valid for the entirety of the session.
UWB_Device_Time_Uncertainty	1	0 – 0xFF	Time uncertainty between the anchor point of connection event and UWB Clock domain mapping. Format is log 2 accuracy of unsigned value in microseconds divided by 8: Value [µsec] = 2^(UWB_Device_Time_Uncertainty /8) The value range which can be represented is 1µsec (=2^{0}µsec) to 1.094 hour (=$2^{255/8}$ µsec)
UWB_Clock_Skew_Measurement_available	1	0 – 0x1	This is used to inform the vehicle if the device has a clock system which allows the vehicle to estimate and correct for the UWB clock skew, based on DeviceEventCount and UWB_Device_Time information. In particular this requires that the device UWB clock is available at Procedure 0 and Procedure 1. Value = 1: UWB clock skew measurement is available Value = 0: UWB clock skew measurement is NOT available.
Device_max_PPM	2	0-0xFFFF	Worst case clock skew of device UWB clock (with respect to nominal time). Format: Unsigned value representing (single sided) clock skew. Shall be interpreted that clock skew could be either positive or negative. Units is PPM (parts per million).
Success	1	0-0x2	0: The “Bluetooth LE Timesync” procedure has failed and the timesync message parameters are invalid. The vehicle should ask again to perform the “Bluetooth LE Timesync” after RetryDelay 1: The “Bluetooth LE Timesync” procedure on device side is successfully able to map the UWB clock and all parameters in the timesync message are valid. 2: The “Bluetooth LE Timesync” procedure has failed and the Procedure 0 is not available. This only applies to Procedure 0.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameters	Length (bytes)	Value	Description
RetryDelay	2	0-0xFFFF	Unsigned 16bits. Value in milliseconds for minimum delay required by device until the vehicle should trigger a new “Bluetooth LE Timesync”.

19.3.4 *Supplementary Service Message - First Approach*

First Approach messages enable Bluetooth LE OOB Secure LE pairing for both owner and friend devices. These messages are used to share OOB Bluetooth LE data via application level encrypted channel through Kble_intro and Kble_oob as described in Bluetooth LE Secure OOB Pairing Prep (see Figure 19-17).

19.3.4.1 First Approach Request Message (FA-RQ)

This message is sent by the device to the vehicle to share device OOB data. The device OOB data comprises a combination of two encrypted payloads i.e. E1_Payload and E2_Payload. The E1_Payload and E2_Payload are encrypted using Kble_intro and Kble_oob, respectively. The message and its parameters are defined in Table 19-54 and Table 19-55, respectively.

Table 19-54: First_Approach_RQ message and its parameters.

Message	Message ID	Parameters
First_Approach_RQ	0x0E	E1_Payload IV1, Tag1, E2_Payload IV2, Tag2

Table 19-55: Definition of the parameter for First_Approach_RQ.

Parameters	Length (bytes)	Description
E1_Payload	8	E1_Payload is an encrypted (AES-128-CCM) DK_Identifier using Kble_intro key. If slot_identifiers are provided by the vehicle, the DK_Identifier is an 8-byte zero padded slot_identifier based on the slot_identifiers provided by the vehicle. If slot identifiers are provided online, the DK_Identifier is an 8-byte zero padded slot_identifier ased on the random slot_identifier generated by the owner device during key_sharing.
IV1	8	IV used by device for encrypting E1_Payload using Kble_intro
Tag1	4	AES-CCM authentication tag for E1_Payload using Kble_intro
E2_Payload	38	E2_Payload is encrypted (AES-128-CCM) BTAddrA (6 Byte) || Ca (16 Byte) || ra (16 Byte) using Kble_oob as shared secret. The BTAddrA, Ca and ra are Bluetooth address of the device at connection establishment, Calculated Value on the device, and Random Value on the device, respectively (see Figure 19-17).
IV2	8	IV used by device for encrypting E2_Payload using Kble_oob
Tag2	4	AES-CCM authentication tag for E2_Payload using Kble_oob

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.3.4.2 First Approach Response Message (FA-RS)

This message is sent by the vehicle to the device to share vehicle OOB data which is encrypted by Kble_oob key. The message and its parameters are defined in Table 19-56 and Table 19-57, respectively.

Table 19-56: First_Approach_RS message and its parameters.

Message	Message ID	Parameters
First_Approach_RS	0x0F	E_Payload IV, Tag

Table 19-57: Definition of the parameter for First_Approach_RS.

Parameters	Length (bytes)	Description
E_Payload	38	E_Payload is encrypted (AES-128-CCM) BTAddrB (6 Byte) || Cb (16 Byte) || rb (16 Byte) using Kble_oob as shared secret. The BTAddrB, Cb and rb, are the Bluetooth address of the vehicle at connection establishment, Calculated Value on the vehicle, and Random Value on the vehicle, respectively (see Figure 19-17).
IV	8	IV used by vehicle for encrypting vehicle OOB data using Kble_oob.
Tag	4	AES-CCM authentication tag for E_Payload using Kble_oob

19.3.5 *Supplementary Service Message - RKE*

19.3.5.1 RKE_Auth_RQ

This message is sent by vehicle upon receiving RKE action SubEvent (See Section 19.5.9).

The content of the RKE_Auth_RQ and its parameter are shown in Table 19-58 and Table 19-59, respectively.

Table 19-58: RKE_Auth_RQ message and its parameter.

Message	Message ID	Parameter
RKE_Auth_RQ	0x14	RKE Challenge

Table 19-59: Definition of the parameter for RKE_Auth_RQ.

Parameter	Length (bytes)	Description
RKE Challenge	16	Vehicle unique challenge generated per RKE request to avoid replay attack.

19.3.5.2 RKE_Auth_RS

This message is sent by the device in response to the RKE_Auth_RQ.

The content of the RKE_Auth_RS is shown in Table 19-60.

Table 19-60: RKE_Auth_RS message and its parameter.

Message	Message ID	Parameters
RKE_Auth_RS	0x15	All fields contained Table 15-61.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Note 1: The content of the arbitrary_data field (Tag 58_h) in Table 15-56 is described in Section 19.5.9.

Note 2: The content of the usage field (Tag 93_h) in Table 15-56 is set to $D074DA4F_h$ or $FC6F4C17_h$.

19.3.6 *Vehicle OEM App Message*

This message is used by the vehicle OEM's app or the vehicle to exchange messages with each other over the same L2CAP channel used in the Digital Key Service. Digital Key framework is agnostic of this message and its data structure. It is simply acting as pass-through. The message and its parameters are defined in Table 19-61 and Table 19-62, respectively.

Table 19-61: Pass_through message and its parameter.

Message	Message ID	Parameter
Pass_through	0x10	Payload

Table 19-62: Definition of the parameter for Pass_through.

Parameter	Length (bytes)	Description
Payload	Variable	Proprietary payload known to vehicle OEM's app and vehicle. Length of the payload depends on the device/vehicle payload size over Bluetooth LE limitations.

19.3.7 *Head Unit Pairing Message*

Head Unit Pairing messages enable head unit pairing between vehicle and device as part of the owner pairing as described in Section 19.5.1.

19.3.7.1 Head_Unit_Pairing_Preparation Message (HU-PP)

The message and its parameters are defined in Table 19-63 and Table 19-64, respectively.

Table 19-63: HU_PP message and its parameters.

Message	Message ID	Parameter
Head_Unit_Pairing_Preparation (HU-PP)	0x16	BT_Pairing_Configuration, BD_ADDR_Head_Unit, Vehicle capabilities

Table 19-64: Definition of the parameter for HU_PP.

Parameter	Length (bytes)	Description
BT_Pairing_Configuration	1	BitMask for supported capabilities: OOB_Configuration: Bit[0] : Value = 0. The vehicle head unit only supports OOB communication from vehicle head unit to device. Bit[0] : Value = 1. The vehicle head unit supports OOB communication for both directions. Bit[1-7]: Reserved
BD_ADDR_Head_Unit	6	Bluetooth Device Address of the head unit as defined in Volume 2 Part H of [30]
Vehicle capabilities	2	BitMask for supported capabilities on vehicle

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Length (bytes)	Description
		Bit [1]: Value = 0: Head unit Bluetooth Classic (A2DP and HFP) pairing not supported Bit [1]: Value = 1: Head unit Bluetooth Classic (A2DP and HFP) pairing supported Bit [2]: Value = 0: Apple Carplay not supported Bit [2]: Value = 1: Apple Carplay Supported Bit [3]: Value = 0: Android Auto Not Supported Bit [3]: Value = 1: Android Auto Supported Bit[4:15]: Reserved

19.3.7.2 Head Unit Pairing Request Message (HUP-RQ)

This message is sent by the device to the vehicle. The message and its parameters are defined in Table 19-65 and Table 19-66, respectively.

Table 19-65: HUP_RQ message and its parameters.

Message	Message ID	Parameter
Head_Unit_Pairing_RQ (HUP-RQ)	0x17	User Intent, BD_ADDR_Device, Device Capabilities, PairingDataR192, PairingDataR256, PairingDataC192, PairingDataC256

Table 19-66: Definition of the parameter for HUP_RQ.

Parameter	Length (bytes)	Description
User Intent	2	BitMask for user intent coming from device. Bit [0]: Value =0: Intent from device Bit [0]: Value =1: No intent from device Bit [1]: Value = 0: Head unit Bluetooth Classic usage not intended from user. Bit [1]: Value = 1: Head unit Bluetooth Classic usage intended from user Bit [2]: Value = 0: Apple Carplay usage not intended from user Bit [2]: Value = 1: Apple Carplay usage intended from user Bit [3]: Value = 0: Android Auto usage not intended from user Bit [3]: Value = 1: Android Auto usage intended from user Bit [4-15]: Reserved
BD_ADDR_Device	6	Bluetooth Device Address of the device as defined in Volume 2 Part H of [30].
Device Capabilities	2	BitMask for supported capabilities on device Bit [1]: Value = 0: Head unit Bluetooth Classic (A2DP and HFP) not supported Bit [1]: Value = 1: Head unit Bluetooth Classic (A2DP and HFP) supported Bit [2]: Value = 0: Apple Carplay not supported Bit [2]: Value = 1: Apple Carplay supported Bit [3]: Value = 0: Android Auto not supported Bit [3]: Value = 1: Android Auto supported Bit[4-15]: Reserved
PairingDataR192	16	Simple Pairing Randomizer R-192*
PairingDataR256	16	Simple Pairing Randomizer R-256*

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameter	Length (bytes)	Description
PairingDataC192	16	Simple Pairing Hash C-192*
PairingDataC256	16	Simple Pairing Hash C-256*
* See volume 2 Part H Section 7.2.2 of [30]		

19.3.7.3 Head Unit Pairing Response Message (HUP-RS)

This message is sent by the vehicle to the device. The message and its parameters are defined in Table 19-67 and Table 19-68, respectively.

Table 19-67: HUP_RS message and its parameters.

Message	Message ID	Parameter
Head_Unit_Pairing_RS (HUP-RS)	0x18	BD_ADDR_Head_Unit, PairingDataR192, PairingDataR256, PairingDataC192, PairingDataC256

Table 19-68: Definition of the parameter for HUP_RS.

Parameter	Length (bytes)	Description
BD_ADDR_Head_Unit	6	Bluetooth Device Address of the head unit as defined in Volume 2 Part H of [30].
PairingDataR192	16	Simple Pairing Randomizer R-192 *
PairingDataR256	16	Simple Pairing Randomizer R-256 *
PairingDataC192	16	Simple Pairing Hash C-192 *
PairingDataC256	16	Simple Pairing Hash C-256 *
* See volume 2 Part H Section 7.2.2 of [30]		

19.3.8 *DK Event Notification*

DK Event Notifications are used to encapsulate events that are generated by the device or vehicle participating in the ranging exchange. The message and its parameters are defined in Table 19-69 and Table 19-70, respectively.

Table 19-69: DK Event Notification message and its parameters.

Message	Message ID	Parameter
DK Event Notification	0x11	Subevent_Category, Subevent_Code

Table 19-70: Definition of the parameters for DK Event Notification.

Parameter	Length (bytes)	Description
Subevent_Category	1	DK SubEvent category which are listed in the table below
Subevent_Code	Variable	SubEvent code as defined below per category.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.3.8.1 DK SubEvent Category

Table 19-71 defines categories of DK SubEvents supported.

Table 19-71: DK SubEvents Category and its parameters.

DK SubEvent Category	SubEvent_Category	Parameter
Command Complete SubEvent	1	Command_Status
Ranging Session Status Changed SubEvent	2	Session_Status
Device Ranging Intent SubEvent	3	DR_Intent
Vehicle Status Change SubEvent	4	Vehicle_Status
RKE Request Subevent	5	Requested_Action
Head Unit Pairing SubEvent	6	Headunit_Pairing_Status
Device UA Policy SubEvent	7	UA Policy (see Table 19-84)
Device UA Status SubEvent	8	UA Status (see Table 19-85)
RFU	0x09 - 0xFF	None

19.3.8.2 DK SubEvent Codes Per Category

Command Complete SubEvent

The Command Complete SubEvent is used to send appropriate code upon processing of last message. Table 19-72 and Table 19-73 detail the Command_Status codes applicable to the 'Command Complete SubEvent' category. For detailed flow on Command_Status SubEvents, see Section 19.7.1.

Table 19-72: Definition of Command_Status and its Command_Status codes.

Parameter	Length (bytes)	Value	Description
Command_Status	1	0x00 - 0xFF	See Table 19-73 for all command status codes

Table 19-73: List of Command_Status for Command Complete SubEvent.

Command_Status	Code	Description
Deselect_SE	0x00	Vehicle shall send this SubEvent to deselect the Digital Key applet. Upon receiving this, the device shall deselect the Digital Key applet. For each secure element transaction over Bluetooth LE (see SE message in Section 19.3.2), it shall be terminated by this command regardless of the transaction is successful or fail.
BLE_pairing_ready	0x01	Vehicle shall send this SubEvent code when its ready for Bluetooth LE pairing during Owner Pairing
Require_capability_exchange	0x02	Device shall send this SubEvent to signal a mismatched ranging capability or an update to the device capability. Upon receiving this, the Vehicle shall trigger ranging capability exchange (see Section 19.7.1.1)
Request_standard_transaction	0x03	Device shall send this SubEvent to signal the vehicle in all cases during owner pairing and first friend approach where a standard transaction is expected. (e.g. if key_tracking_receipt is available in OP). Upon Receiving this, the vehicle should start a standard transaction.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Command_Status	Code	Description
		If a vehicle has not received the Request_standard_transaction SubEvent (e.g. due to transmission error) or if the vehicle is busy (e.g. vehicle is still busy processing the preceding action), then the vehicle should start the standard transaction after a timeout or when the processing is done, respectively. If the vehicle initiates a standard transaction but the device becomes not ready, the device may respond with a Device_busy SubEvent or may not respond during the command timeout. The vehicle shall retry the standard transaction until the device responds. A timeout shall be applied if the device does not respond.
Request_owner_pairing	0x04	Device shall send this SubEvent to request owner pairing
Reserved	0x05- 0x7F	Reserved for future use
General_error	0x80	Device or vehicle may send this SubEvent to signal an unknown error. Upon receiving this, the vehicle or device may retry or abort.
Device_SE_busy	0x81	Device may send this SubEvent to signal the SE is temporarily unavailable. The device may retry internally to get SE resources before sending this. Upon receiving this, the vehicle may retry the digital key flow.
Device_SE_transaction_state_lost	0x82	Device may send this SubEvent to signal standard transaction state was lost. Upon receiving this, the vehicle may retry the transaction
Device_busy	0x83	Device may send this SubEvent to signal device temporal not able to respond to the sent request. Upon receiving this, the vehicle may retry the action.
Command_temporarily_blocked	0x84	Device shall send this SubEvent when the device UA state and UA policy do not allow the requested command.
Unsupported_channel_bitmask	0x85	Device shall send this SubEvent if the channels provided in the channel bitmask are currently not supported by the device
OP_Device_not_inside_vehicle	0x86	Vehicle shall send this SubEvent when the device was not found inside the vehicle during owner pairing. This error shall cause the vehicle and device to abort owner pairing
Device_resource_available	0x87	The device sends this subevent to signal to the vehicle that it recovered from a state where access to device internal resources was blocked. The vehicle may use this information to retry a flow that failed due to that previous device state.
Reserved	0x88- 0xFB	Reserved for future use
OOB_mismatch	0xFC	Device or vehicle may send this SubEvent to signal the incompatible OOB data exchanged during Bluetooth LE pairing. This is an error code for debugging purposes.
BLE_pairing_failed	0xFD	Device or vehicle may send this SubEvent to signal the failure in Bluetooth LE pairing. This is an error code for debugging purposes.
FA_crypto_operation_failed	0xFE	Device or vehicle may send this SubEvent to signal the failure in First approach due to cryptography. This is an error code for debugging purposes.
Wrong_parameters	0xFF	Device or vehicle may send this SubEvent to signal the use of unsupported message or format. This is an error code for debugging purposes.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Ranging Session Status Changed SubEvent

The Ranging Session Status Changed SubEvent is generated to indicate the start, progress and completion of an UWB related action. These SubEvents can be generated by both the device and the vehicle. The Session_Status codes are described in Table 19-74 and Table 19-75.

Table 19-74: Definition of Session_Status and its Session_Status codes.

Parameter	Length (bytes)	Value	Description
Session_Status	1	0x00 - 0xFF	See Table 19-75 for all session status codes.

Table 19-75: List of Session_Status for Ranging Session Status Changed SubEvent.

Session_Status	Code	Description
Ranging_session_URSK_refresh	0x00	Vehicle may send this SubEvent to request clean-up for pre-derived URSKs.
Ranging_session_URSK_not_found	0x01	Device shall send this SubEvent to signal, the device failed to find URSK for this UWB_Session_Id
Ranging_session_not_required	0x02	Device shall send this SubEvent when it detects the user does not intend to approach the vehicle e.g device is moving away from the vehicle. Upon receiving this subEvent, the vehicle may immediately suspend the ranging session.
Ranging_session_secure_ranging_failed	0x03	Vehicle shall send this SubEvent to signal the Vehicle failed to establish secure ranging.
Ranging_session_terminated	0x04	Vehicle or device may send this SubEvent if it has stopped the ranging session (e.g. due to URSK TTL expiration)
Ranging_session_recovery_failed	0x06	Device shall send this SubEvent to signal it failed to recover ranging for this UWB_Session_Id
Ranging_session_suspended	0x07	Vehicle or device shall send this SubEvent only if it is suspending the current ranging session without allowing the responder to delay the suspension (e.g. UWB has temporarily become unavailable). When vehicle or device receives this, it shall suspend its current ranging session as well without sending a Ranging_Suspend_RQ or a Ranging_session_suspended SubEvent.
Reserved	0x08-0xFF	Reserved for future

Device Ranging Intent SubEvent Codes

This set of SubEvents are generated by the device to notify the vehicle that the user may be approaching the vehicle and ranging may be required. The DR_Intent codes are described in Table 19-76 and Table 19-77.

Table 19-76: Definition of DR_Intent and DR_Intent codes.

Parameter	Length (bytes)	Value	Description
DR_Intent	1	0x00-0xFF	Low_approach_confidence, Medium_ approach_confidence, High_ approach_confidence, Reserved

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-77: List of DR_Intent for Device Ranging Intent SubEvent.

DR_Intent	Code	Description
Low_approach_confidence	0x00	Device notifies vehicle of user approaching with low confidence
Medium_ approach_confidence	0x01	Device notifies vehicle of user approaching with medium confidence
High_ approach_confidence	0x02	Device notifies vehicle of user approaching with high confidence
Reserved	0x03- 0xFF	Reserved for future use

The device shall use the following definition of Confidence levels to select DR_Intent. The setting of the DR_Intent is up to the device OEM implementation. When the vehicle is in low power mode, the vehicle may decide to ignore ranging intent with low or medium approach confidence. When the vehicle is not in low power mode, and if the vehicle is ready to perform the secure ranging operation (e.g., no vehicle internal conflict), the vehicle shall initiate the secure ranging regardless of the DR_Intent confidence level.

To enable secure ranging based on a Device Ranging Intent, the device shall send a Device Ranging Intent SubEvent with at least low_approach_confidence between the point in time a Bluetooth LE connection with the vehicle is established and the point in time the vehicle triggers secure ranging (e.g. user touches the door handle).

The performance requirements from Section 19.2.3.3 shall apply when the vehicle starts the Ranging Session Setup flow after receiving a Device Ranging Intent SubEvent

Approach detection probability:

- Describes the probability an intent was sent in the last 2 to 20 seconds before the customer touched the door
- P(Approach detection) = P(in last [2s,20s] an intent was sent | Customer touches door handle

False approach detection probability:

- Describes the probability that a ranging intent was sent, but the customer has not arrived at the vehicle touching the door handle within the following 20s
- P(False approach detection) = P(Customer has not touched the door handle within the last 20s | ranging intent received 20s ago)

Definition of Confidence levels:

- Low_approach_confidence:
  1. Approach detection probability: >90%, and
  2. False approach detection probability: as low as possible
- Medium_ approach_confidence:
  3. Approach detection probability: as high as possible, and
  4. False approach detection probability: <75%
- High_ approach_confidence
  5. Approach detection probability: as high as possible, and

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

6. False approach detection probability: <50%

Vehicle Status Changed SubEvent

The Vehicle Status Changed SubEvent is used to inform the device about state changes of vehicle functions as shown in Table 19-78. The Digital Key framework on the device shall store the latest values of received function state changes as long as the BLE connection is established.

Table 19-78: List of Vehicle Status Changed SubEvent Tags.

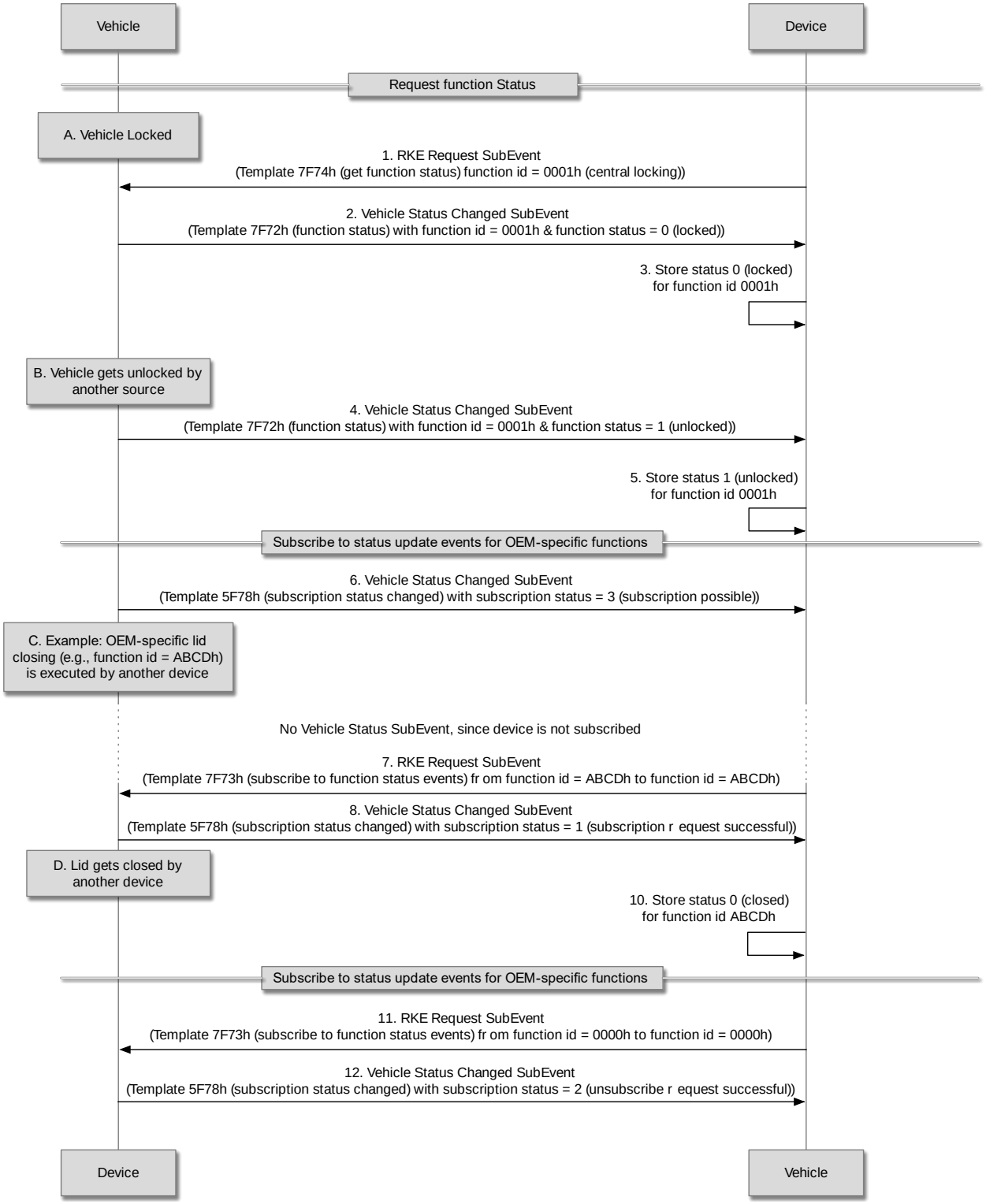
The content of the Vehicle Status Changed SubEvent message is encoded as TLV format as defined in [1]				Length (bytes)	Description	Field is
7F71h				10	Last requested action execution status for function / action id. Tag is only present when an RKE function was triggered beforehand or an automatic action was triggered due to the location / movement of the device (e.g., walk away locking).	C
	80h			2	Function id	M
	81h			1	Action id	M
	82h			1	Execution Status 00h: Function successfully executed 01h: Execution started 02h: Execution stopped successfully (upon user stop request) 03h: Execution ended (e.g., limit reached) 04h–0Fh: Reserved 10h: Execution not possible or cancelled - unspecific reason 11h: Execution not possible due to vehicle state 12h: Authentication Error 13h: Device response timeout 14h: Device out of execution range 15h–4Fh: Reserved 50h–EFh: Execution not possible or cancelled with OEM specific meaning. The meaning of these unsuccessful status codes is proprietary and may be used to send a more specific error reason to the vehicle OEM app. The definition is up to the vehicle OEM. F0h–FFh: Standardized error codes defined inTable 19-80.	M
7F72h				variable	Vehicle function status summary. The tag is present, when the vehicle sends at least one status message.	C
	30h			variable	Sequence of vehicle function status	M
		A0h		variable	Vehicle function status	M
			80h	2	Function id	M
			83h	1	Function status	M
			C0h	variable	Vehicle OEM proprietary data (max. 64 bytes)	O
7F75h				variable	Request confirmation for enduring RKE action. Only present, when enduring RKE action is in progress.	C
	80h			2	Function id	M
	81h			1	Action id	M

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The content of the Vehicle Status Changed SubEvent message is encoded as TLV format as defined in [1]				Length (bytes)	Description	Field is
	87h			variable	Arbitrary data with a maximum size of 64 bytes.	O
5F78h				1	Subscription status changed, only present, when the vehicle changes the subscription state 0: Unsubscribed all, no subscription possible 1: Subscription request successful 2: Unsubscribe request successful 3: Subscription possible/resubscription required	5F78h

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-6: RKE SubEvent Subscription Process



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag $7F71_h$ shall always be sent together with $7F72_h$ in a single SubEvent message to the device that triggered the action causing the vehicle state to change. An exception is when the vehicle state takes more than 250ms to change after the action received. In this situation, $7F71_h$ and $7F72_h$ can be sent separately in two different subevent messages. When sent together, the templates order shall be always $7F71_h$ followed by $7F72_h$. In case of automatic actions triggered due to the location of the device, the device that triggered a state change (i.e., 'central locking' or 'lock and secure') will receive $7F71_h$ and $7F72_h$ as a single SubEvent message with the appropriate action id 0x50(lock) and 0x51(unlock). Other BLE connected devices (not used in decision making) will only receive the $7F72_h$ template in the SubEvent notification. The vehicle can make an exception for devices in the vicinity as described below.

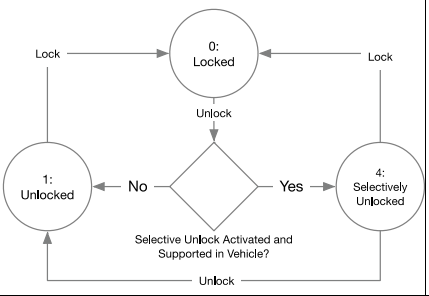
For central locking or lock and secure, the template $7F71_h$ can be included with $7F72_h$ to inform devices that are in the vicinity of the vehicle but did not directly cause the event. This shall only be done for devices located in zones relevant for passive entry ('vicinity').

The $7F71_h$ template shall not be included for below conditions:

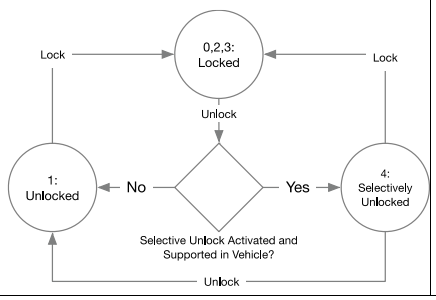
- Lock upon car in motion
- Car lock/unlock from inside by the user

Table 19-79 lists the functions and its corresponding action ids and/or status values. Depending on the usage of the particular ID, it may not provide the corresponding action id or status value. For certain actions that represent automatically executable functions, e.g., due to device position or movement, the vehicle may send an execution status without an explicit RKE trigger. An example of this is using it to signal when execution of an automatic action is unsuccessful (e.g., walk away lock with vehicle state not allowing to lock the vehicle).

Table 19-79: Definition of Function ids and their corresponding Action ids.

Function id	Function	Possible action ids	Possible function status values (Note 1)	Event-based or enduring action	Device (Note 2)	Vehicle (Note 2)
0000_h-$000F_h$ Central locking category (vehicle may support up to one of the following options)						
0001_h	Central locking	0: Lock 1: Unlock 50: Vehicle-triggered Lock (only for execution status) 51: Vehicle-triggered Unlock (only for execution status)	0: Locked 1: Unlocked 4: Selectively Unlocked. Only driver door unlocked (with global unlock available on unlock action) Vehicle State Machine:  	Lock, Unlock: Event; Walk Away Lock, Approach Unlock: only execution status	M	C, only available if 0002_h is not available
0002_h	Lock and secure	0: Lock and secure 1: Unlock	0: Locked and secured 1: Unlocked	Lock and secure, Unlock, Lock	M	O

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Func-tion id	Function	Possible action ids	Possible function status values (Note 1)	Event-based or enduring action	Device (Note 2)	Vehicle (Note 2)
		2: Lock with partial arming of alarm system 50: Vehicle-triggered Lock & Secure (only for execution status) 51: Vehicle-triggered Unlock (only for execution status)	2: Locked with partially armed alarm system 3: Locked with partial arming available 4: Selectively Unlocked. Only driver door unlocked (with global unlock available on unlock action) Vehicle state machine:  	with partial arming of alarm system: Event; Walk Away Lock, Approach Unlock: only execution status		
0003$_h$ … 000F$_h$	Reserved for standardization					
0010$_h$	Driving Readiness		0: No Driving Readiness 1: Driving Readiness (e.g., engine “running” for ICE vehicles)	Event	-	M
0011$_h$	Vehicle low power Mode		0: Normal power mode 1: Low power mode	Event	-	O
0012$_h$	Low Fuel Status		0: Fuel not low 1: Fuel Low	-	-	O
0013$_h$ … 001F$_h$	Reserved for standardization					
0100$_h$	Remote Climatization	0: Stop Climatization 1: Start Climatization	0: Remote Climatization not active 1: Remote Climatization active	Event	O	O
0101$_h$	Panic alarm	0: Mute panic alarm 1: Trigger panic alarm	0: Panic alarm not active 1: Panic alarm active	Event	M	O
0102 $_h$	Fuel Lid	1: Release	0: Closed 1: Open	Event	O	O
0103$_h$ … 010F$_h$	Reserved for standardization					
0110$_h$-011F$_h$ Rear Vehicle Storage Variants category (the vehicle should support one of the following options if the vehicle has a trunk)						
0110$_h$	Manual Trunk/Decklid/ Liftgate/ Tailgate Controls	1: Release	0: Closed 1: Open	Event	O	O
0111$_h$	Power Trunk/Decklid/ Liftgate/	0: Close 1: Open	0: Closed 1: Open	Open: enduring without confirmation,	O	O

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Function id	Function	Possible action ids	Possible function status values (Note 1)	Event-based or enduring action	Device (Note 2)	Vehicle (Note 2)
	Tailgate Controls (with confirmation for close)			Close: enduring with confirmation		
0112h	Power Trunk/Decklid/ Liftgate/ Tailgate Controls (without confirmation for close)	0: Close 1: Open	0: Closed 1: Open	enduring without confirmation	O	O
0113h … 011Fh	Reserved for standardization					
0120h-012Fh Front Vehicle Storage Variants category (the vehicle should support one of the following options if the vehicle has a frunk)						
0120h	Manual "Frunk" Front Trunk Controls Traditional vehicle body style	1: Release	0: Closed 1: Open	Event	O	O
0121h	Power "Frunk" Front Trunk Controls with confirmation Traditional vehicle body style	0: Close 1: Open	0: Closed 1: Open	Open: enduring without confirmation, Close: enduring with confirmation	O	O
0122h	Power "Frunk" Front Trunk Controls without confirmation Traditional vehicle body style	0: Close 1: Open	0: Closed 1: Open	Open/Close: enduring without confirmation	O	O
0123h … 012Fh	Reserved for standardization					
0130h-0140h Vehicle Power Doors Variants category (if supported by vehicle)						
0130h	Power Front Left with Confirmation	0: Close 1: Open	0: Closed 1: Open	enduring with confirmation	O	O
0131h	Power Front Left without confirmation	0: Close 1: Open	0: Closed 1: Open	enduring without confirmation	O	O
0132h	Power Front Right with Confirmation	0: Close 1: Open	0: Closed 1: Open	enduring with confirmation	O	O

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Func-tion id	Function	Possible action ids	Possible function status values (Note 1)	Event-based or enduring action	Device (Note 2)	Vehicle (Note 2)
0133_h	Power Front Right without confirmation	0: Close 1: Open	0: Closed 1: Open	enduring without confirmation	O	O
0134_h	Power Rear Left with Confirmation	0: Close 1: Open	0: Closed 1: Open	enduring with confirmation	O	O
0135_h	Power Rear Left without confirmation	0: Close 1: Open	0: Closed 1: Open	enduring without confirmation	O	O
0136_h	Power Rear Right with Confirmation	0: Close 1: Open	0: Closed 1: Open	enduring with confirmation	O	O
0137_h	Power Rear Right without confirmation	0: Close 1: Open	0: Closed 1: Open	enduring without confirmation	O	O
0138_h … $013F_h$	Reserved for standardiz-ation					
0140_h-0150_h Vehicle Power Windows / Roof (if supported by vehicle)						
0140_h	Power Windows	0: Close 1: Open	0: Closed 1: Open	enduring with confirmation	O	O
0141_h	Power roof	0: Close 1: Open	0: Closed 1: Open	enduring with confirmation	O	O
0142_h … $05FF_h$	Reserved for standardiz-ation					
0600_h to $FFFF_h$	Vehicle OEM-proprietary functions					
Note 1: Possible function status values: Range from 00_h to EF_h, for values $F0_h$ to FF_h (see Table 19-80) **Note 2**: Device and Vehicle codes: Mandatory, Optional, Conditional						

Generic function status values shall be used to indicate errors or the absence of a certain function. These function status values are defined in Table 19-80.

Table 19-80: Definition of Standardized Function/Execution Status Values to Indicate the (Temporary) Unavailability of Function/Execution or Errors.

Function status value	Possible function status values
$F0_h$	Function not supported by vehicle
$F1_h$	Action not supported by vehicle
$F2_h$	Function temporarily not available
$F3_h$	Action temporarily not available

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Function status value	Possible function status values
$F4_h$	Function not purchased
$F5_h$	Action not purchased
$F6_h$	Insufficient entitlements
$F7_h$	Use of wrong RKE security policy for requested action
$F8_h$ to FD_h	Reserved
FE_h	Generic error, no further description available
FF_h	Reserved, shall be used if action is supported but function has no explicit status

RKE Request SubEvent

The RKE Request SubEvent's content, described in Table 19-81, is encoded as TLV. The RKE Request SubEvent may be sent:

1. in response to user's request to perform one of the defined RKE features (tag $7F70_h$ and tags $7F76_h$/$7F77_h$ for enduring actions).
2. to subscribe to vehicle function status updates (tag $7F73_h$).
3. to get the current vehicle functions' status (tag $7F74_h$).

Table 19-81: Definition of RKE Request SubEvent Templates.

Tag			Length (bytes)	Description	Field Is
$7F70_h$			7	Request RKE action. Only present, when RKE action shall be triggered.	C
	80_h		2	Function id	M
	81_h		1	Action id	M
$7F73_h$			variable	Subscribe to vehicle function status events. Unsubscribing from all status updates is possible by setting "from function id" and "to function id" to 0. Tag is only present, when subscription change shall be performed.	C
	30_h		variable	Sequence of function ids. Only present, if vehicle has subscription change to the list of function status	C
		84_h	2	From function id	M
		85_h	2	To function id	M
	86_h		0	If present, vehicle shall send an immediate update of supported function states in this range.	C
$7F74_h$			variable	Get status update for function ids. Only present, when explicit status updates shall be requested.	C
	30_h		variable	Sequence of function ids. Only present, if status for a list of function ids shall be requested.	C
		80h	variable	Function id	M
	86_h		0	If present, vehicle shall send a full status update of all supported functions and their corresponding states.	C
$7F76_h$			variable	Continue enduring RKE action. Only present, when enduring RKE action is in progress.	C
	80_h		2	Function id	M
	81_h		1	Action id	M
	88_h		variable	Arbitrary data (max. 64 byte)	O
$7F77_h$			7	Stop enduring RKE action. Only present, when the enduring RKE action in progress shall be stopped.	C
	80_h		2	Function id	M

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Tag			Length (bytes)	Description	Field Is
	81_h		1	Action id	M

Head Unit SubEvents

The Head Unit SubEvent is generated to indicate the final success or failure of the transaction or to signal application-specific codes to the device. The Head Unit SubEvent codes are described in Table 19-82 and Table 19-83.

Table 19-82: Definition of Headunit_Pairing_Status and its Session_Status.

Parameter	Length (bytes)	Value	Description
Headunit_Pairing_Status	1	0x00 - 0xFF	See Table 19-83 for all head unit pairing status codes

Table 19-83: List of Session_Status for Head Unit SubEvent.

Session_Status	Code	Description
Headunit_pairing_success	0x00	Head Unit Pairing is successful
Headunit_pairing_fail	0x01	Head Unit Pairing fails
Reserved	0x02-0xFF	Reserved for future

Device UA Policy SubEvent

The Device UA Policy SubEvent is used by Device to inform Vehicle about the current used UA Policy on the Device. When the key is created, the UA Policy is set to Off [O] by default. At BLE connection setup Device should inform Vehicle if the UA Policy has changed to other value than Off [O]. During BLE connection, Device should also inform Vehicle if UA Policy has changed.

Table 19-84: List of UA Policies

UA Policy	Value	Description
Off [O]	0x00	UA never requested (default)
Access [A]	0x01	UA requested on first access or first engine start
Access [AL]	0x02	UA requested on first access and lock or first engine start and lock
Engine [E]	0x03	UA on every engine start
Always [T]	0x04	UA on every usage
others	0x05 – 0xFF	RFU

Device UA Status SubEvent

If UA Policy is Off [O], Device UA Status SubEvent should not be sent. If it’s sent then it should be ignored by Vehicle.

If UA Policy is not Off [O], Device UA Status SubEvent is used by Device to inform Vehicle about the current UA Status on the Device. The UA Status is ON by default.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 19-85: List of UA Policies

UA Status	Value	Description
OFF	0x00	Device doesn't require UA for a requested DK function due to: - Device is in a state where only DK function can be triggered (see section 9.1) or - grace period is active during implicit UA (see section 9.2)
ON	0x01	Device might require UA for a requested DK function (default).
others	0x02 – 0xFF	RFU

19.4 Time Synchronization

The time synchronization shall be mandatory for the device. The time synchronization support is strongly recommended for the vehicle due to performance gains such as improved latency and power saving benefit for the vehicle.

19.4.1 *Definitions*

The following definitions are used for this subsection:

19.4.1.1 Device UWB Clock and UWB Device Time

Device UWB Clock is defined as the UWB clock of the device as it appears to the vehicle.

UWB Device Time is defined as the timestamp on the Device UWB Clock.

The appearance of the Device UWB Clock to a vehicle can be defined or undefined:

1. "Defined"
   - Device UWB Clock becomes "defined" to the vehicle, once a successful Bluetooth LE Timesync procedure with valid UWB_Device_Time has been performed
   - Device UWB Clock becomes "defined" to the vehicle, once an UWB packet (of a ranging session with this device) has been received
   - Once defined, vehicle expects that UWB device clock is operated in a consistent manner and the characteristics and requirements below hold
2. "Undefined"
   - Device UWB Clock appears "undefined" to the vehicle prior to a successful Bluetooth LE Timesync.
   - Note: This is the case if the Device UWB clock has not been started.
   - Device UWB Clock becomes "undefined", if there is no UWB and no Bluetooth LE connection for more than 30 seconds.
   - Device UWB Clock becomes "undefined", if a ranging session is suspended.
   - Note: A ranging session may be suspended automatically, if Bluetooth LE connection is lost for more than 30 seconds.
   - Device UWB Clock becomes "undefined", if a ranging session is terminated.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- Note: Session is terminated if 12 hours lifetime is exceeded or the STS-Index range is exhausted.

Characteristics of Device UWB Clock

1. UWB clock is UWB ranging session specific.

   Note: Device may use a common clock for all its ranging sessions and maintain this clock even if a session is suspended.

2. The Device UWB Clock may have any value when entering “defined” state (within its specified range). For example, the device may set UWB_Device_Time=0 upon first valid Bluetooth LE Timesync procedure.
3. During an active ranging session, the Device UWB Clock is identical to the initiator clock as specified in Section 20.2.
4. Suspending a ranging session shall put the UWB clock into an undefined state and reset it to an arbitrary value for session recovery.

   Note: UWB_Time0 for resuming a ranging session may be smaller than UWB_Device_Time at ranging session suspend.

5. Device UWB clock should have a granularity of 1 µsec or smaller, considering that the format in the timesync message and the RSS-RS / RR-RS/ CRR-RS message is defined with 1 µsec resolution.
6. The range of the UWB clock is limited to 0 … 2^64-1 µsec due to the format in the timesync message and the RSS-RS / RR-RS/ CRR-RS message.
7. Device UWB clock does not depend on the existence of a BT connection as long the ranging session is not suspended.

   Note: A short Bluetooth reconnection that does not trigger a ranging suspension during an active ranging session shall trigger Bluetooth LE Timesync procedure 0. But the short Bluetooth reconnection shall not impact the operational state of Device UWB clock (while in “defined” mode).

Requirements for Device

1. Device shall be able to map a timing reference (see Section 19.3.3) onto the Device UWB Clock and generate a UWB_Device_Time timestamp. The term “to map” here means, to take a timestamp from the Device UWB Clock in the moment of the Bluetooth LE Timesync defined timing reference on the air interface, or equivalently, compute the value that was held by the Device UWB Clock during the timing reference by means of an OEM specific method.
2. Device shall support at least one successful Bluetooth LE Timesync procedure before session start/resume.
3. Device shall support all vehicle triggered timesync procedures (Bluetooth LE Timesync procedure 1) during an active session.
4. All (valid) UWB_Device_Time values provided in the timesync messages shall be consistent clock states in accordance with the MAC grid as specified in Section 20.2.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- *Note*: In particular, the value of the UWB clock at session start shall be UWB_Time0 (as announced in the RSS-RS/RR-RS message).

5. UWB_Device_Time shall be presented as the clock state of a 64-bit counter with 1µsec resolution; the 64-bit counter shall wrap around at 2^{64}-1.

Tolerance of Device UWB Clock frequency:

1. Before session start (or before session resume):
   - Maximum tolerance of UWB clock frequency is Device_max_PPM, as included in timesync message
2. After session start (during active ranging session):
   - Maximum tolerance of UWB clock frequency is Device_max_PPM, as included in timesync message and shall be less than ±100 ppm as specified in Section 6.9.7.2 of [33].

19.4.1.2 Terminology: Clock Offset, Skew, Drift

The terminology used in this section follows Section 10 of [37].

Clock state: Value of a clock x at time t=T

Clock offset: Clock offset is the difference in clock states between a clock x and the true time at an arbitrary point in time T.

Relative clock offset: Relative clock offset is the difference in clock states between a clock x and a clock y, at an arbitrary point in time T.

Clock skew: Clock skew is the difference in clock frequency to the nominal clock frequency.

- Clock skew can be viewed as first derivative in time of clock offset.
- Clock skew normalized to the nominal frequency and multiplied by 10^6 is “clock skew in ppm”.

Relative clock skew: Relative clock skew is the difference in clock frequencies of clock x and clock y.

- Relative clock skew can be viewed as the first derivative in time of relative clock offset (the difference in clock frequencies at certain point in time).

Clock drift: Clock drift is the derivative of clock skew (the variation of clock frequency over time). Clock drift is assumed to be negligible for the purpose of this specification.

Figures of merit for timesync

- Vehicle UWB Clock (the UWB clock in a vehicle anchor = responder) is clock x.
- Device UWB Clock is the clock y, and is the reference clock that defines the UWB MAC grid, and to which the responder needs to synchronize.
- Timesync procedure is used to determine the relative clock offset between Device UWB clock and Vehicle UWB clock for the time of the BLE event on the air interface

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- The worst-case clock skew Device_max_PPM of the device UWB clock (relative to true time) and the worst-case clock skew of the vehicle can be used to determine an upper bound for the relative clock offset of a future event (like session start).
- Vehicle may estimate the relative clock skew and compensate for it, to minimize the relative clock offset of a future event (like session start); for the time interval of interest the clock drift is assumed to be zero.

19.4.2 *General Description*

The Bluetooth connection serves as a common time domain between the vehicle and device and is used to synchronize the Bluetooth-connected UWB transceiver by mapping a shared BT event into the UWB clock domain. Synchronization methods are shown in Figure 19-7.

UWB Packet reception:

The reception of an UWB packet by the vehicle should allow to perform a better synchronization than the Bluetooth LE Timesync synchronization method described in this chapter.

Because this gives an anchor the precise position within the MAC grid (see Section 20.3), which it may be used to adjust the relative clock offset between device and anchor. This subsection describes the “Bluetooth LE Timesync” synchronization which happens before the reception of an UWB packet by the vehicle.

Note: An anchor may set its UWB-RX into scanning mode and wait for the reception of a packet without using any “Out of band” synchronization. This is always possible, at the cost of increased power consumption at the vehicle and other potential constraints.

Bluetooth LE Timesync:

In the absence of UWB Packet reception, this section describes a time synchronization over Bluetooth.

Time synchronization over Bluetooth

Time synchronization over Bluetooth, referred as “Bluetooth LE Timesync”, is an out-of-band synchronization method that is carried out between the device and one particular anchor. There are two types of “Bluetooth LE Timesync” procedure (see to Section 19.4.4):

- “Bluetooth LE Timesync at Bluetooth connection”, also referred as “Procedure 0”
- “Bluetooth LE Timesync triggered by vehicle”, also referred as “Procedure 1”

References in this chapter to “Bluetooth LE Timesync” apply to both “Bluetooth LE Timesync at Bluetooth connection” and “Bluetooth LE Timesync triggered by vehicle”

Vehicles may use “Bluetooth LE Timesync” to synchronize on the start of the UWB ranging session.

After start of the session the vehicle may use “Bluetooth LE Timesync” to re-synchronize on the MAC grid if UWB connection was lost for a longer period.

Vehicle internal synchronization

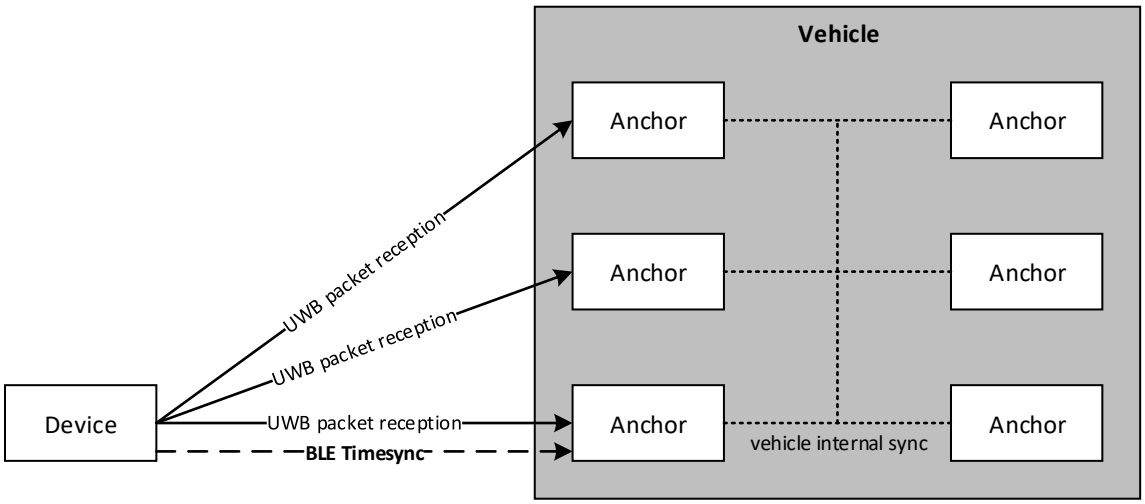
Vehicle internal synchronization may be used to distribute device synchronization (achieved by a particular anchor based on UWB packet reception or “Bluetooth LE Timesync”) between the

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

anchors. The mechanism to achieve the vehicle internal synchronization is implementation specific and is out of scope for this specification.

The different synchronization methods are shown in Figure 19-7.

Figure 19-7: Synchronization Methods.



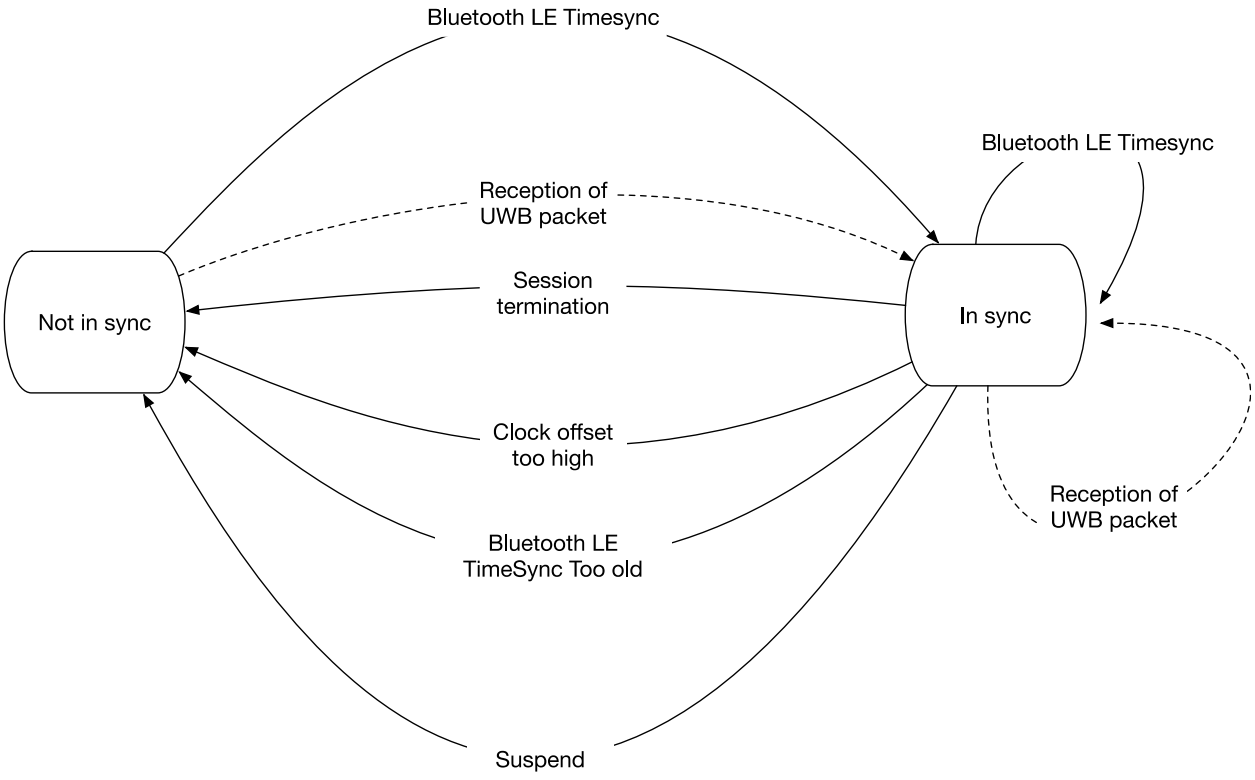
There are two states for the time synchronization between device and anchor as shown in Figure 19-8.Those states are only known by the vehicle.

- "not in sync": Clock offset of anchor UWB clock vs device UWB clock is unknown, or potential offset is larger than 1ms
- "in sync": Clock offset of anchor UWB clock vs device UWB clock is better than 1ms

Note: The limit for the relative clock accuracy that defines the "in sync" state is up to the vehicle implementation.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-8: Synchronization state between device and anchor.



Note: This state diagram defines the pre-conditions for time synchronization and assumes a vehicle implementation which establishes this time synchronization in all anchors. In a real implementation there could be further boundary conditions, that might hinder time synchronization, although the conditions from this state diagram are given.

Note: Reception of an UWB packet shall lead immediately to the “In sync” state. This synchronization via reception of an UWB packet is indicated as dashed line in the state diagram is referred as “In-band” UWB synchronization. However, the scope of this Section is “Out-of-band” UWB synchronization via Bluetooth LE Timesync.

If “in sync”, the vehicle shall be able to predict events like session start or a position within the MAC grid (for an active ranging session) with some uncertainty.

The uncertainties are caused by clock skew between device and vehicle UWB clock, UWB_Device_Time uncertainty in case of “Bluetooth LE Timesync”, and uncertainties on vehicle side.

If no ranging session was started or recovered within 10 seconds after a successful Bluetooth LE Timesync, the Device UWB Clock becomes “Not in sync”.

The vehicle shall only trigger “Bluetooth LE Timesync” if conditions in condition set A or condition set B is met:

Condition Set A:

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1. Condition A.1: A ranging session was needed in the last 150s or is currently running. This can be signaled by Device Ranging Intent as sent by device to vehicle, user physical intent on vehicle (e.g. touching door handle, engine start), or when Session Ranging Setup is to be performed (e.g. during owner pairing, first friend approach, passive entry).
2. Condition A.2: Previous successful “Bluetooth LE TimeSync” was done (either by device or vehicle) more than 150s ago.

Condition Set B:

1. Condition B.1: A ranging session is needed. This can be signaled by Device Ranging Intent as sent by device to vehicle, user physical intent on vehicle (e.g. touching door handle, engine start), or when Session Ranging Setup is to be performed (e.g. during owner pairing, first friend approach, passive entry).
2. Condition B.2: No active ranging session is on-goingNone of the vehicle’s UWB anchors produced a valid ToF measurement within 10 seconds since the last “Bluetooth LE TimeSync”
3. Condition B.3: Previous successful “Bluetooth LE TimeSync” was done (either by device or vehicle) more than 10s ago.

Device clock skew is upper bounded by the parameter Device_max_PPM.

The vehicle may estimate the relative UWB clock skew (UWB clock frequency offset between vehicle and device) using the information from the most recent and one or more previous timesync procedures. In this case, the vehicle may use the estimated clock skew and compensate for it, instead of applying an upper bound with Device_max_PPM.

After the first successful UWB packet is received by vehicle, the vehicle should use this to resynchronize the UWB clocks between device and vehicle. Therefore, the vehicle should apply worst-case PPM based on the smallest duration between:

- Latest successful UWB packet exchange until now
- Latest time sync until now

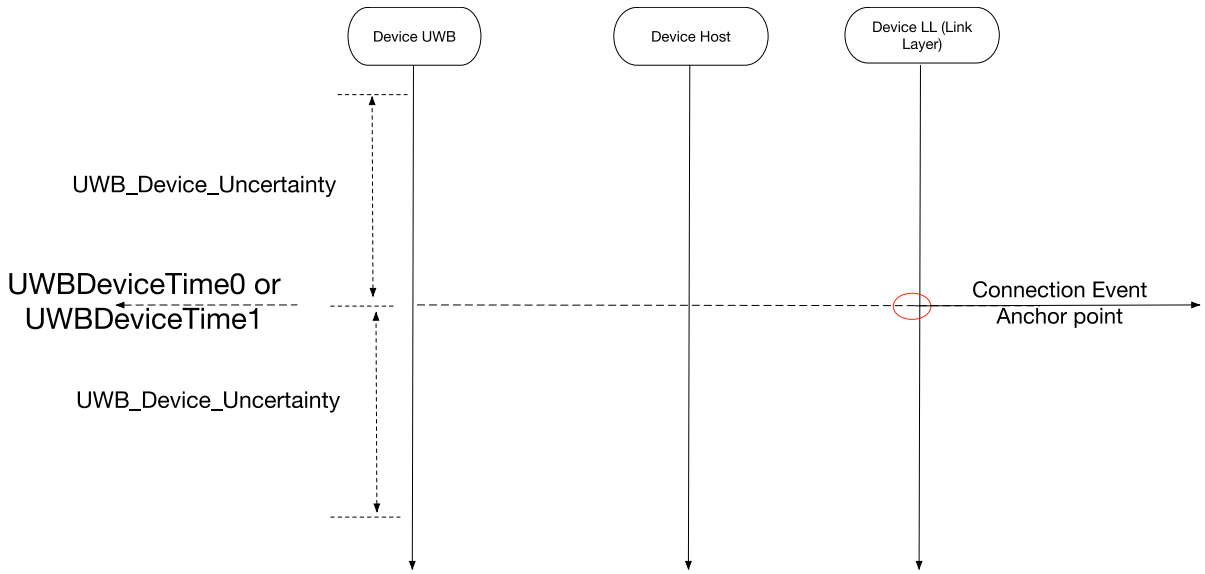
19.4.3 *Device Uncertainty*

UWB_Device_Uncertainty is the time uncertainty between the connection event Anchor point and UWB Clock domain mapping (see Section 19.3.3 for definition)

Figure 19-9 shows the uncertainties on the device side.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-9: Time Synchronization Uncertainty Flow Diagram.



19.4.4 *"Bluetooth LE Timesync" Procedures.*

In this subsection, the following times are defined:

- *UWBDeviceTime0*: UWB time clock domain captured at Procedure 0.
- *UWBDeviceTime1*: UWB time clock domain captured at Procedure 1.
- *DeviceEventCount0*: Event count for the connection interval after Bluetooth connection establishment. This is the first connection interval, and the value shall be 0. UWBDeviceTime0 is taken at the anchor point of this connection interval.
- *DeviceEventCount1*: Event count for the connection interval containing the LL_PHY_UPDATE_IND message sent from device to the vehicle during the "Bluetooth LE TimeSync" procedure 1. UWBDeviceTime1 is taken at the anchor point of this connection interval.

19.4.4.1 Procedure 0: Bluetooth LE Timesync at Bluetooth Connection

The host (device) shall capture the anchor point of the first available connection event after the Bluetooth connection establishment. The device shall map this point in time also into its UWB clock domain, (which corresponds to UWBDeviceTime0) and the device shall send a timesync message over Bluetooth LE. In case of retransmission for the connection event 0, the vehicle should consider only the latest successful LL message.

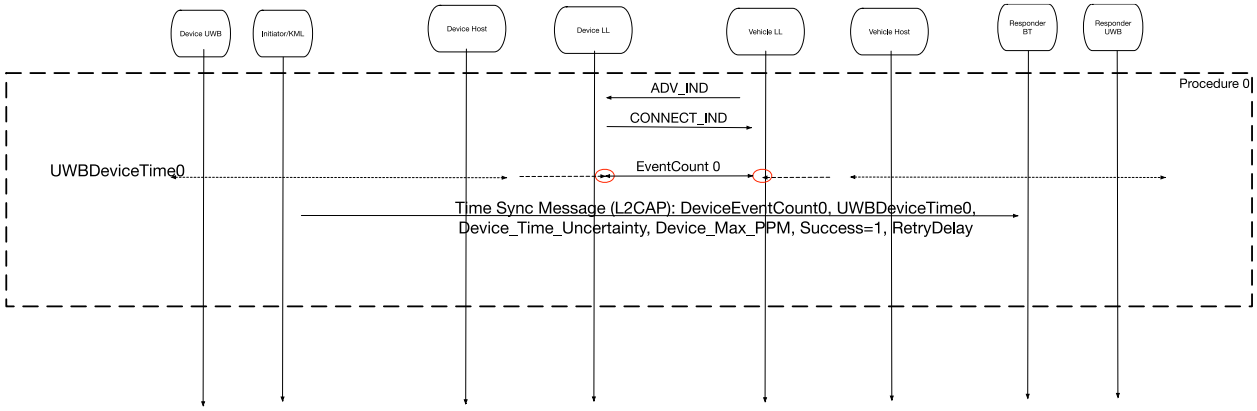
The timesync message is defined in Section 19.3.3.

The timesync message parameter DeviceEventCount should have a value of 0 or close to it.

A successful "procedure 0" shall return a timesync message parameter Success value of 1. The timesync message parameter RetryDelay has no meaning.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-10: Successful Bluetooth LE timeSync at BT connection (Procedure 0).



19.4.4.2 Procedure 1: Bluetooth LE Timesync Triggered by Vehicle

When the vehicle determines a time synchronization is required, the host (vehicle) shall send an "LE Set PHY" command which triggers sending the "LL PHY REQ" to the device. The device should respond to the "LL PHY REQ" by sending "LL_PHY_UPDATE_IND".

The vehicle may trigger the procedure 1 before or after the ranging session start. The vehicle shall trigger the procedure 1 only after Bluetooth connection is established and encryption is completed.

The time synchronization process on the device side is initiated once the device receives the message "LL_PHY_REQ" sent from the vehicle. The time synchronization shall happen at the anchor point of the connection event that contains the response message "LL_PHY_UPDATE_IND" of the device. This time as DeviceEventCount1.

The device shall map this point in time into its UWB clock domain (UWBDeviceTime1). The exact mechanism of mapping the timing reference is out of scope for this specification. In case of retransmission for "LL_PHY_UPDATE_IND", the vehicle should consider only the latest successful "LL_PHY_UPDATE_IND" LL message.

After mapping the timing reference into its UWB clock domain, the device shall send a timesync message over Bluetooth LE. The timesync message is defined in Section 19.3.3.

The vehicle should map the timing reference (anchor point) of a connection event into its UWB clock domain (UWBVehicleTime1).

The vehicle maps its UWB clock in two different ways:

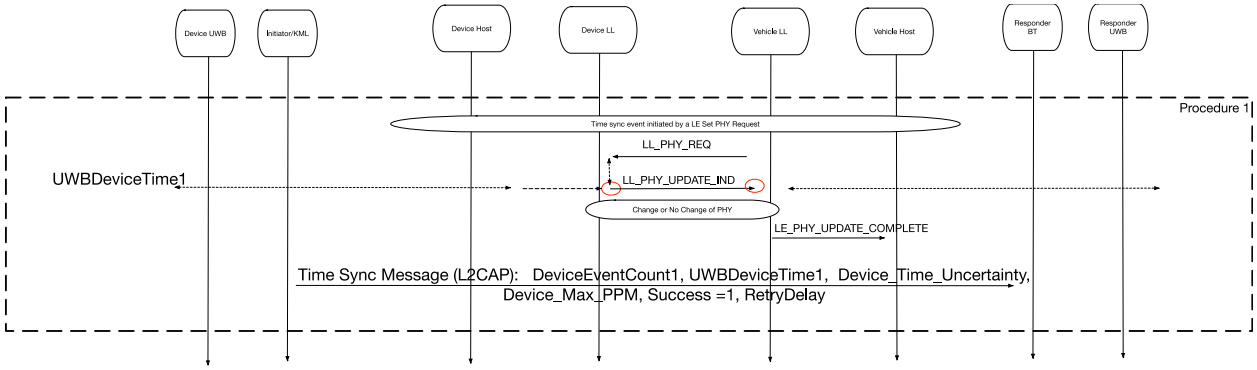
1. **Same connection event as device**: Vehicle has mapped the anchor point of the same connection event as referenced in the timesync message.
    a. In this case the vehicle may directly use UWB_Device_Time from the timesync message to determine the synchronization offset:
        i. SyncOffset = UWBDeviceTime1 – UWBVehicleTime1
    b. The use of DeviceEventCount is not necessary.
2. **Different connection event as device**: Vehicle has mapped the anchor point of a different connection event VehicleEventCount1
    a. In this case the vehicle shall use DeviceEventCount to correct for the different points in time and determine the synchronization offset:

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

i. SyncOffset = UWBDeviceTime1 + (VehicleEventCount1 – DeviceEventCount1) × Connection Interval – UWBVehicleTime1

b. Clock skew between device UWB and BLE clock domain can be neglected, if the delta of the event counts for mapping is small.

Figure 19-11: Successful Bluetooth LE timeSync triggered by vehicle (Procedure 1).



19.4.4.3 Unsuccessful "Bluetooth LE Timesync"

An Unsuccessful "Bluetooth LE Timesync", shown in Figure 19-12, is defined when the success field in the Time_Sync message (see Section 19.3.3.) is set to 0.

The device may return an unsuccessful "Bluetooth LE Timesync" for the following reasons:

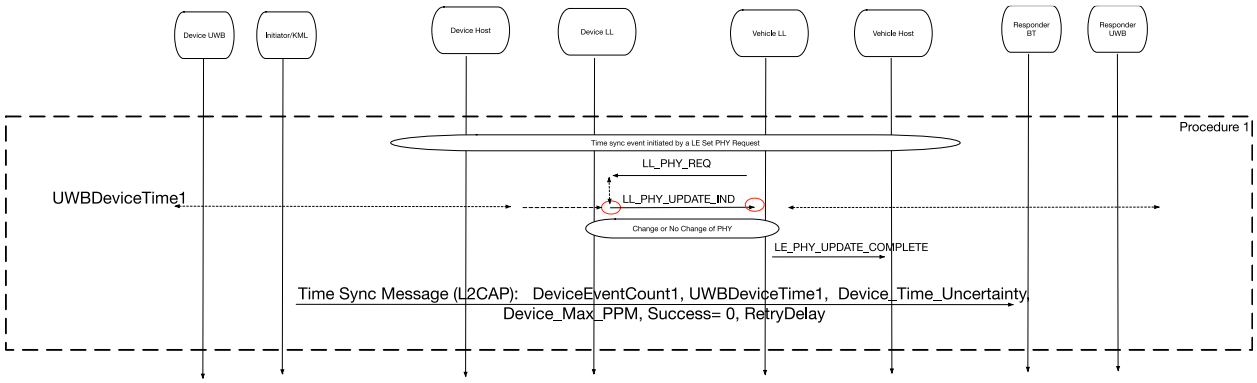
- Device UWB Clock is not available
- Device is unable to map the timing reference to its UWB clock domain.

If the success field is 0, the vehicle should initiate the "Bluetooth LE Timesync" procedure after the delay stated in the field "RetryDelay".

In this case, the following parameters in the time Sync message are undefined or not available:

- UWB_Device_time
- DeviceEventCount1
- UWB_Device_time_uncertainty
- Device_max_PPM

Figure 19-12: Unsuccessful Timesync message.



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

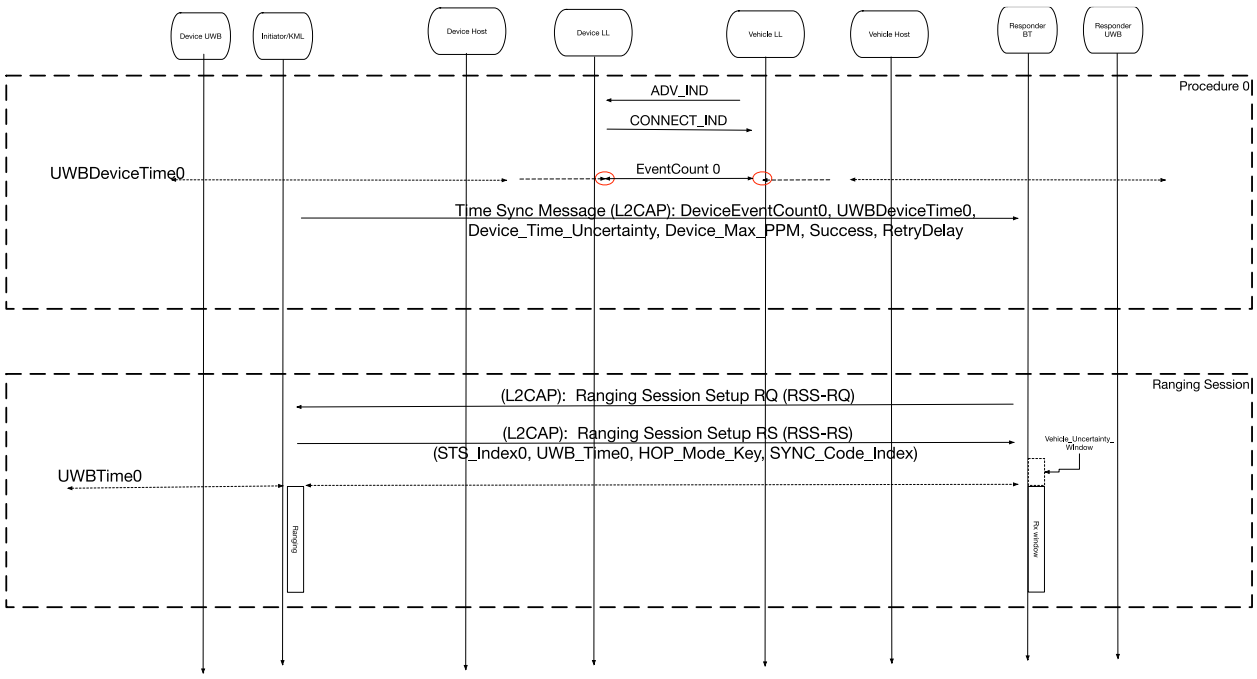
19.4.5 *"Bluetooth LE Timesync" message flow examples*

There are multiple combinations possible for Procedure 0 and Procedure 1 before a ranging session is setup.

19.4.5.1 Example 1: "Bluetooth LE Timesync" with Procedure 0 only

In Example 1, shown in Figure 19-13, the procedure 0 is successful and procedure 1 is omitted.

Figure 19-13: Bluetooth LE Timesync message flow example 1.



19.4.5.2 Example 2: "Bluetooth LE Timesync" with Procedure 0 and Procedure 1

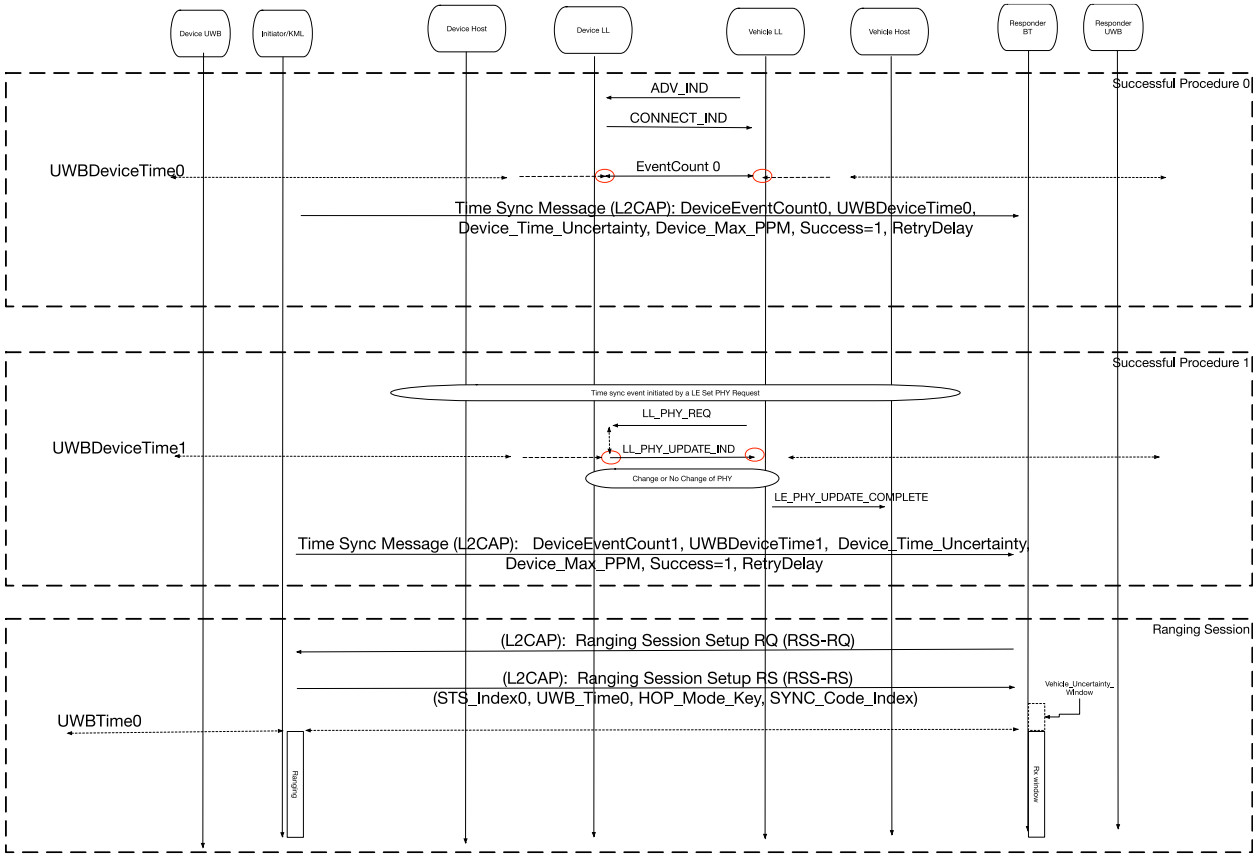
In Example 2, both Procedure 0 and Procedure 1 are successful.

The internal operations of the device and vehicle for obtaining UWB timestamps according to the BLE timestamps are implementation specific.

Figure 19-14 describes the "Bluetooth LE Timesync" message flow with an example of internal operations. In this figure, the ranging session is described as a block as illustration only. It is possible for the vehicle to initiate Procedure 1 any time during the ranging session.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-14: Time Synchronization flow diagram example 2.



19.4.5.3 Example 3: "Bluetooth LE Timesync" with Procedure 1 only

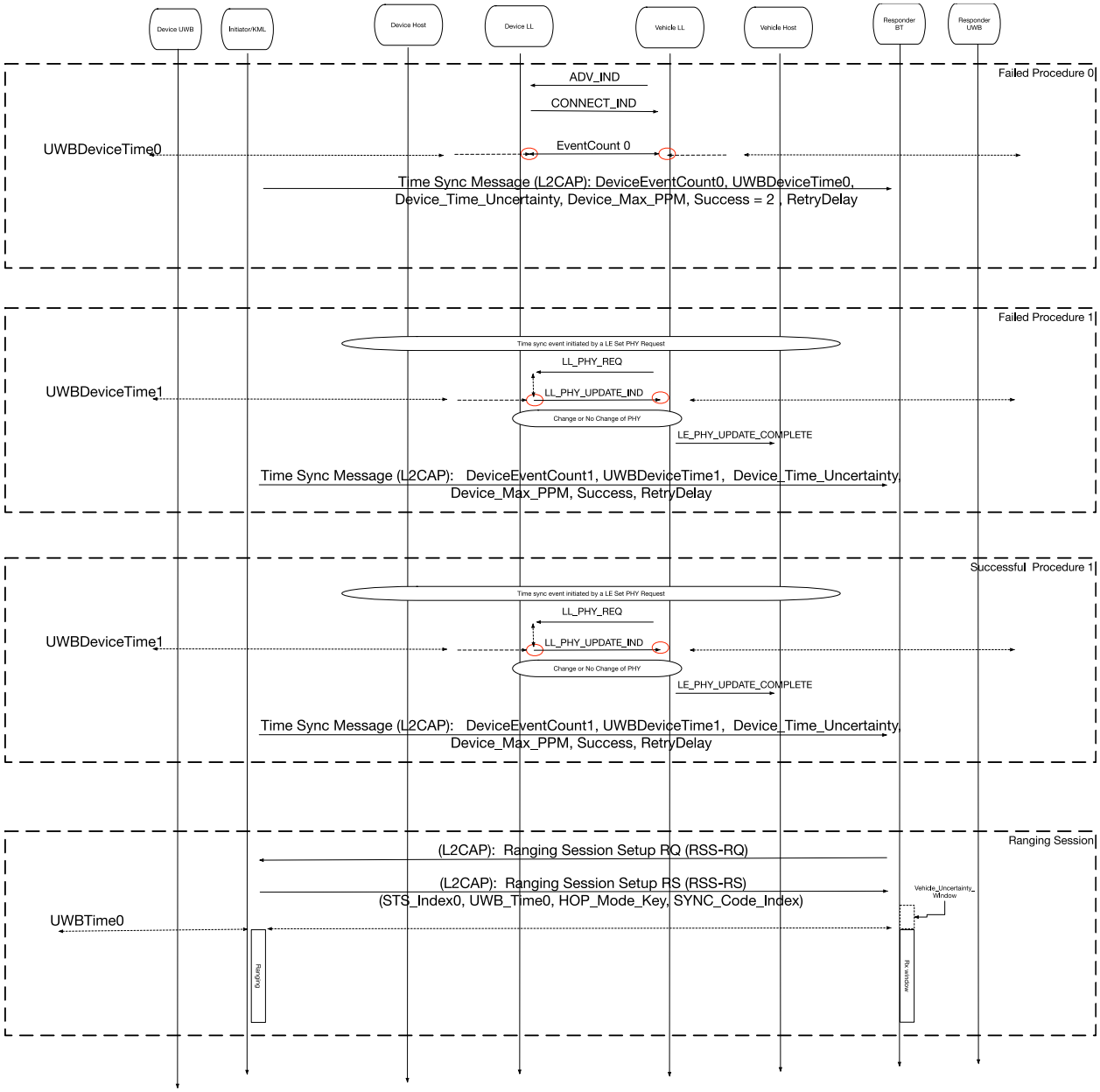
In Example 3, shown in Figure 19-15, the Procedure 0 is unsuccessful and only the second Procedure 1 is successful.

The second Procedure 1 shall be initiated after the retryDelay provided in the first Failed Procedure 1.

If the device supports Procedure 0, it should reply with Success = 2 (the "Bluetooth LE Timesync" procedure has failed and the Procedure 0 is not available) to avoid confusion for the vehicle.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-15: Bluetooth LE Time Sync message flow example 3.



19.4.6 *Conditions for a "Bluetooth LE Timesync" procedure*

A successful "Procedure 1" is defined as the grouping of the following LL messages and DK messages:

- LL_PHY_REQ initiated by vehicle
- LL_PHY_UPDATE_IND sent by device
- Time_Sync message with success field = 1 (see Section 19.3.3).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Frequency of Procedure 1:

- If consecutive successful “Procedure 1” is equal or more than 60 seconds apart, the device shall support it.
- If consecutive successful “Procedure 1” is less than 60 seconds apart, device support is optional.

There should be at least one successful procedure before the Ranging Session Setup Request and Ranging Recovery Request messages.

UWB_Time0 (provided in the Ranging Session Setup Response or Ranging Recovery Response message) shall be consistent in time to the reference UWB_Device_Time. Since Device UWB clock enters "defined" mode after successful Procedure 1, any Bluetooth reconnection after the LL_PHY_UPDATE_IND and before the ranging shall not impact the UWB_Time0.

19.5 Digital Key – Flows

19.5.1 *Owner Pairing Flow*

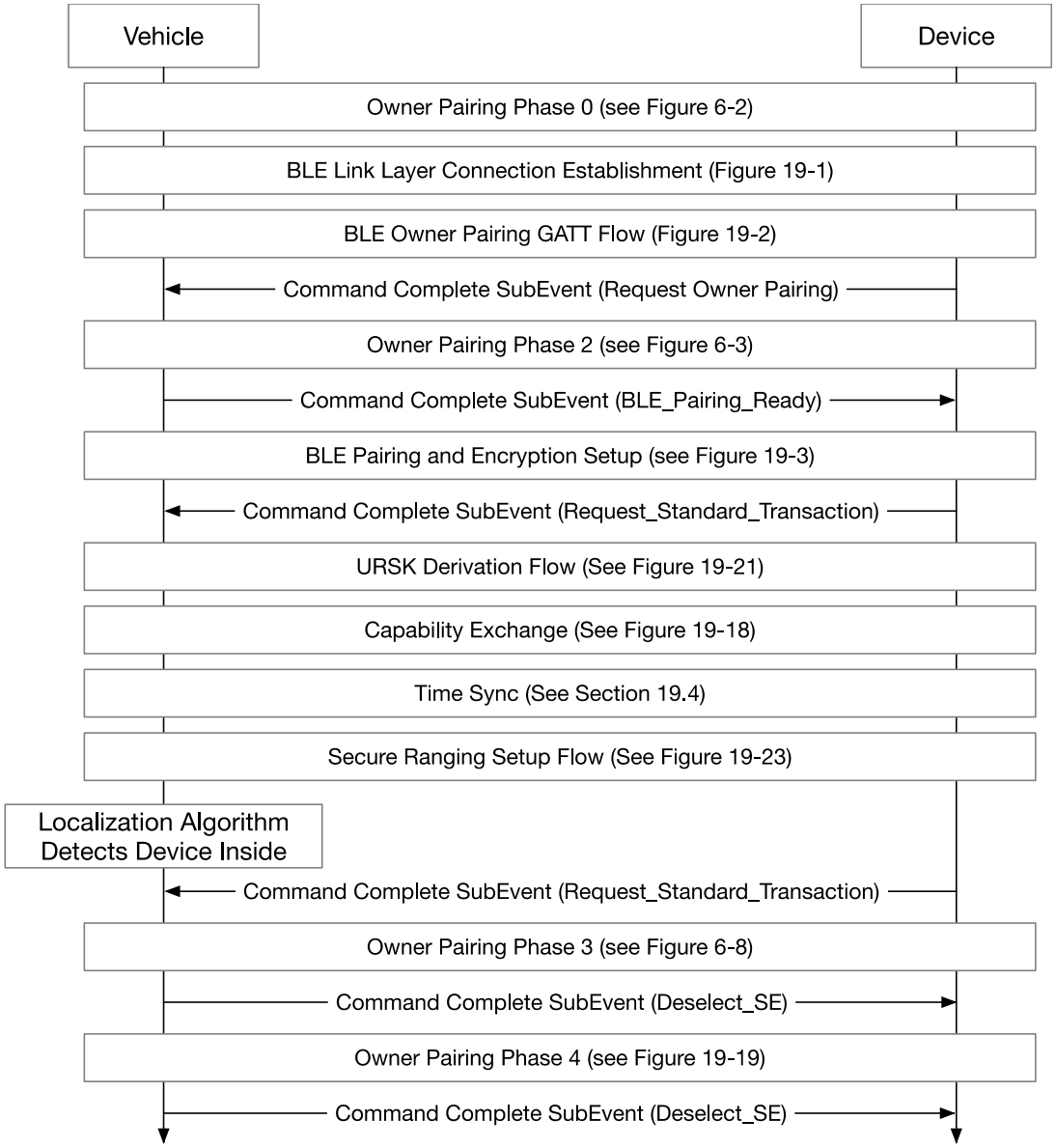
Owner pairing can either be triggered automatically by the vehicle initiating Owner Pairing Connection Establishment as defined in Section 19.2.1 or manually by user going into the vehicle system menu and initiating owner pairing. In either case, before initiating owner pairing process, vehicle ensures that all of its policies have been met. Below are some of the examples for what these policies might include:

1. Physical key fob inside the vehicle
2. No owner device paired with vehicle
3. Digital Key feature has been enabled

Owner pairing over Bluetooth LE leverages similar exchange that takes place over NFC with minor modifications and additional steps to the flow as described in this section and shown in Figure 19-16. NFC related procedures (NFC polling and link setup, NFC link teardown, NFC reset) in Owner Pairing Phase 2, Phase 3 and Phase 4 shall not apply to owner pairing over Bluetooth LE. For Bluetooth LE/NFC, the owner pairing may start over NFC as described in Section 6 and followed by Bluetooth LE Activation flow as described in Section 19.5.10. Owner pairing shall be performed over NFC if the vehicle or device does not support secure ranging.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-16: Owner Pairing Flow for Bluetooth LE/UWB.



Owner Pairing Phase 0: Preparation

This phase prepares device and vehicle for owner pairing as described in Section 6.3.1.

Bluetooth LE Link Layer Connection Establishment

In this setup process, the device goes through Bluetooth LE connection, discover DK services and characteristics and setup L2CAP channel as described above in Figure 19-1.

Bluetooth LE Owner Pairing GATT Flow

In this setup process, device goes through Bluetooth LE Owner Pairing GATT Flow as described in Figure 19-2.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Request Owner Pairing

After connection to the vehicles Bluetooth LE owner pairing advertisement, the device shall send the “Request_Owner_Pairing” SubEvent notification (see Table 19-73) to start owner pairing.

Owner Pairing Phase 2: SPAKE2+ Flow and Certificate Exchange

This phase enables SPAKE2+ based secure channel and certificate exchange between vehicle and device. For more details on Phase 2, refer to Section 6.3.3. To perform Phase 2 exchange over Bluetooth LE, each APDU command and response shall be encoded as below.

Message Type: Framework message

Message:

APDU Command: DK_APDU_RQ (see Section 19.3.2.1)

APDU Response: DK_APDU_RS (see Section 19.3.2.2)

In Owner Pairing Phase 2, both device and vehicle go through SPAKE2+ flow (see Figure 6-3 and Figure 6-4) to derive system keys and OOB Bluetooth LE keys as defined below. More details on SPAKE2+ and system keys derivation (HKDF functions) see Section 18.

Below is the set of derived system keys.

```
Keys = HKDF-SHA256(96, K, “SystemKeys”)      // (output_len,
                                             // input_keying_material, info)
Kenc = Keys[0:15]                            // first 16 bytes
Kmac = Keys[16:31]                           // second 16 bytes
Krmac = Keys[32:47]                          // third 16 bytes
LONG_TERM_SHARED_SECRET = Keys[48:63]        // fourth 16 bytes
Kble_intro = Keys[64:79]                     // fifth 16 bytes
Kble_oob_master = Keys[80:95]                // sixth 16 bytes
```

Out of these keys, Kble_oob_master and Kble_intro are UWB solution specific keys:

1. Kble_oob_master is used as shared secret between device and vehicle which is used to derive Kble_oob to encrypt OOB pairing data during First Approach
2. Kble_intro is used to encrypt DK_Identifier during First Approach

In addition, the vehicle shall provide its “Wireless Capabilities” as part of 7F49 template defined in Table 19-86. The following wireless capability combinations (WCC) as utilized for DK Vehicle and Device operation are defined:

- WCC1: NFC only
- WCC2: NFC and Bluetooth LE (NFC + RKE Functions)
- WCC3: NFC, Bluetooth LE, and UWB (NFC + RKE Functions + Passive/Location-based Functions)

Devices and Vehicles may contain any combination of radios not used for DK operations.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Device shall store Kble_oob_master, Bluetooth Random Static Address of the vehicle and IRK of vehicle in its database. This information shall be used during owner pairing, NFC to UWB migration of owner device and friend sharing.

Table 19-86: 7F49 Template.

Template			Length (bytes)	Description	Field is
$7F49_h$			variable	7F49 template is used to provide wireless capability (NFC/UWB) and related data; this template shall be included by vehicle.	M
	$5F50_h$		0	This tag shall be included if and only if the vehicle supports NFC. Vehicle and Device shall always support NFC capability. If this tag is present, tags 5F51 and/or 7F52 may be present.	M
	$5F51_h$		0	This tag shall be included if and only if the vehicle supports UWB. If this tag is present, tags 5F50 and 7F52 shall be present.	O
	$7F52_h$		variable	This tag (Bluetooth LE) shall be included if and only if either of the WCC shown below are supported by the vehicle: WCC / NFC / UWB / BLE WCC2 / Supported / Not Supported / Supported WCC3 / Supported / Supported / Supported Length for usage during Owner pairing: 28 bytes Length for usage during Friend sharing: 64 bytes	C
		$D0_h$	16	IRK of Bluetooth LE of the vehicle. If 7F52 tag is present, this tag shall be included	C
		$D1_h$	6	Bluetooth Random Static Address of the vehicle. If 7F52 tag is present, this tag shall be included	C
		$D2_h$	0	This Tag shall be included if vehicle supports LELR.	O
		$D3_h$	16	Kble_oob is a 16-byte key used to encrypt/decrypt (AES-128-CCM) Bluetooth LE Secure OOB Pairing data which is exchanged as part of First_Approach_RQ/RS. This key is derived by owner device and vehicle and unique per Digital Key. This tag shall not be used during owner pairing, when transferring the "Wireless capabilities" from vehicle to device. This tag shall only be used for friend sharing.	C
		$D4_h$	16	Kble_intro is a 16-byte key used to encrypt/decrypt (AES-128-CCM) DK_Identifier exchanged as part of First_Approach_RQ. This key is derived by both device and vehicle as part of owner pairing flow (see Section 19.5.1) and is same across all friends. This tag shall only be used for friend sharing.	C

Additional tags sent by the vehicle in 7F49 template shall be ignored by the device.

The vehicle that supports Bluetooth LE/UWB shall also set the corresponding bits in Tag 46_h and Tag 47_h as defined in Table 15-14.

Bluetooth LE Pairing & Encryption Setup Procedure

The Bluetooth LE Secure OOB Pairing Prep flow, shown in Figure 19-17, enables secure sharing of OOB data between device and vehicle to enable Bluetooth LE Secure OOB pairing.

Message Type: Supplementary Service message

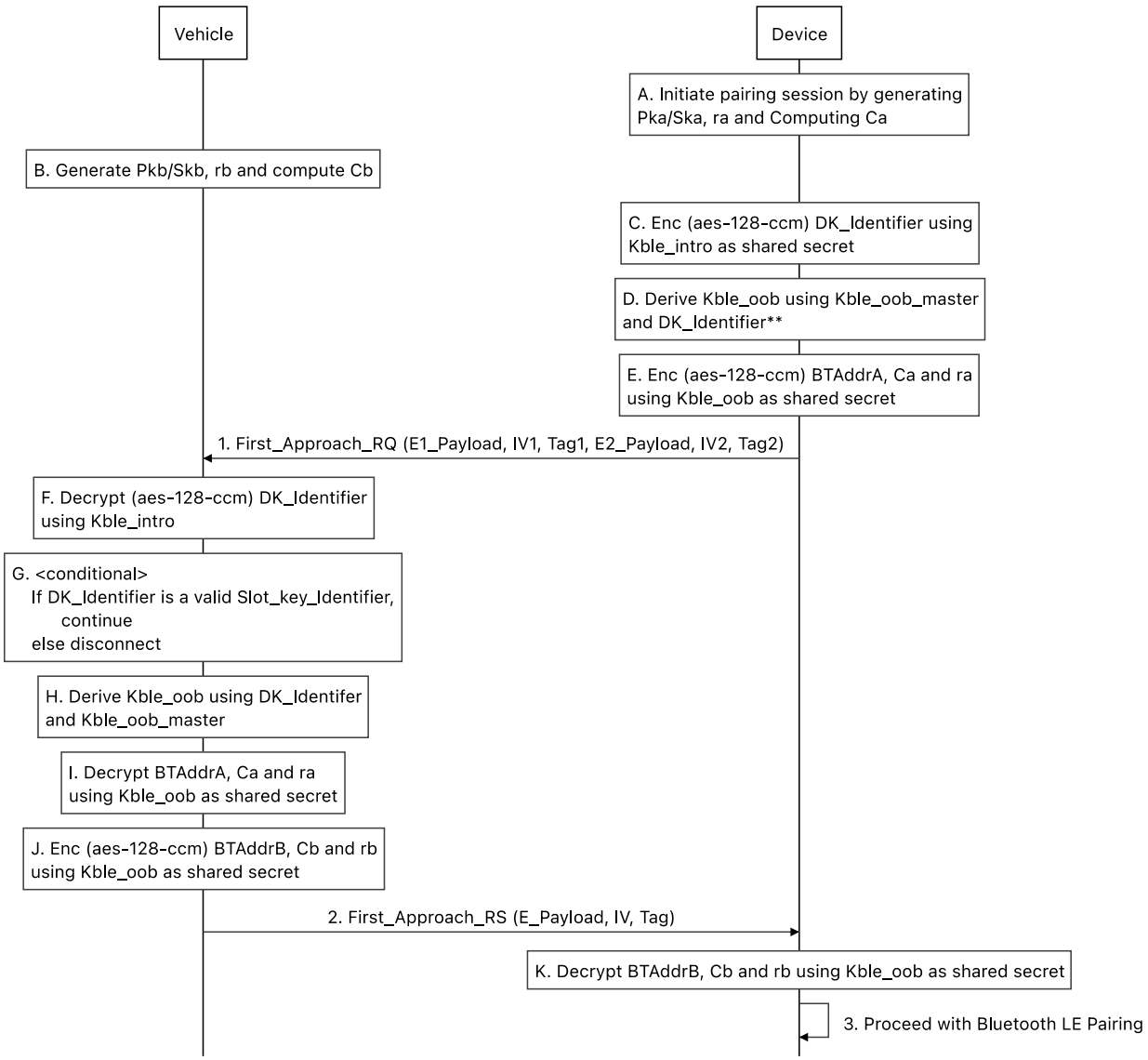
Message:

First_Approach_RQ (see Section 19.3.4.1)

First_Approach_RS (see Section 19.3.4.2)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-17: Bluetooth LE Secure OOB Pairing Prep.



** A friend device does not derive Kble_oob but receives it from the Owner device instead.

Elements of the Bluetooth LE Secure OOB Pairing Prep flow:

- The BTAddrA and BTAddrB are the Device Bluetooth Address and Vehicle Bluetooth Address respectively, used during pairing (See Vol 3, Part H, Section 2.2.7 of [30]).
- For box F, vehicle shall use AES-128-CCM with Kble_intro as shared secret and 8-byte IV provided by device to decrypt DK_Identifier
- For box G, vehicle shall react depending on Tag DAh configuration in Table 5-14: Device Configuration Data.

  If Tag DA_h in Table 5-14 is not present, or set to values 00_h or 02_h, vehicle shall validate that decrypted DK_Identifier value belongs to a valid slot.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

If Tag DA_h in Table 5-14 is present and set to values 01_h, 03_h, 04_h, 05_h, vehicle shall not validate that decrypted DK_Identifier value belongs to a valid slot in this step. The vehicle shall validate the slot identifier during the first standard transaction after Bluetooth LE pairing instead.

- For box H, vehicle shall use HKDF function with decrypted DK_Identifier and Kble_oob_master to derive Kble_oob.
  - Kble_oob = HKDF-SHA256(16, Kble_oob_master, DK_Identifier) // (output_len, input_keying_material, info)
  - Where DK_Identifier is an 8-byte padded slot_identifier. For example, if the slot_identifier is 6-byte, it will have 0x0000 added as prefix to make it 8-byte.
- For box I, vehicle shall use AES-128-CCM with Kble_oob as shared secret and 8byte IV to decrypt device OOB data required for Bluetooth LE Secure OOB pairing (BTAddrA || Ca || ra)
- For box J, vehicle shall use AES-128-CCM with Kble_oob as shared secret and 8-byte IV to encrypt vehicle OOB data required for Bluetooth LE Secure OOB pairing (BTAddrB || Cb || rb)
- For box K, device shall use AES-128-CCM with Kble_intro as shared secret and 8-byte IV provided by device to decrypt DK_Identifier
- Once device and vehicle have successfully performed Bluetooth LE Secure OOB Pairing Prep, device shall initiate Bluetooth LE Pairing procedure by sending pairing request as described in Section 19.2.1.4. Post successful pairing, device shall setup Bluetooth LE encryption before moving to next step.
- The Bluetooth LE Pairing data shall be persisted for device and vehicle at the same point in time the endpoint is persisted on either side. Any failure before that point shall lead to rolling back the Bluetooth LE pairing data to be able to perform a retry.
- Any Bluetooth LE messages being exchanged over-the-air as part of this flow shall not make use of DK message defined in Section 19.3.

URSK Derivation Flow

Once the Bluetooth LE encryption has been setup, if the vehicle supports UWB, the vehicle shall execute the standard transaction flow for deriving URSK. (The URSK Derivation flow is shown in Figure 19-21)

To derive URSK over Bluetooth LE, each APDU command and response shall be encoded as shown below.

Message Type: SE message

Message:

APDU Command Encapsulation: DK_APDU_RQ (see Section 19.3.2.1)

APDU Response Encapsulation: DK_APDU_RS (see Section 19.3.2.2)

Capability Exchange

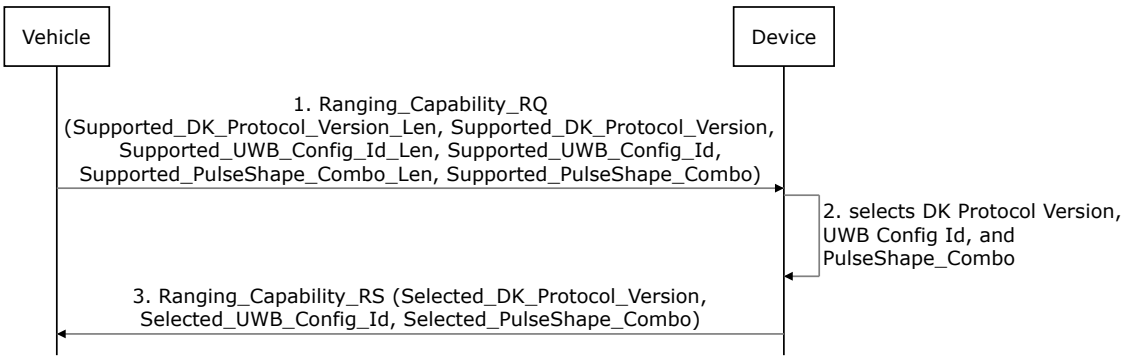
Capability Exchange for owner pairing shall be performed if either the vehicle or device has an updated DK Protocol Version, UWB Configuration Identifier, PulseShape Combinations, or a combination of these parameters, which is different than the one the vehicle and device

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

supported during the last Capability Exchange. The Capability Exchange is shown in Figure 19-18.

The vehicle initiates the Capability Exchange by sending Ranging_Capability_RQ (see Section 19.3.1.1) to the device. Upon receiving the Ranging_Capability_RQ, the device shall respond by sending Ranging_Capability_RS (see Section 19.3.1.2).

Figure 19-18: Capability Exchange.



The device triggers the Capability Exchange by sending DK Event Notification with Subevent_Category set to $0x01_h$ and the corresponding command status code set to $0x02_h$. Upon receiving the DK Event Notification, the vehicle shall initiate the Capability Exchange as shown in Figure 19-18.

Time Sync

If the Device UWB Clock is “Not in sync” while Bluetooth LE connected with the device, the vehicle shall trigger the Bluetooth LE Timesync procedure 1 if all conditions are met (see Section 19.4.2).

Secure Ranging Setup Flow

Once the Capability Exchange is done, vehicle shall initiate secure ranging setup flow. In this flow, the vehicle and device go through a few exchanges to finalize the ranging parameter values required to set up the secure ranging. As part of this flow, the vehicle shall notify the device on which URSK to use by sending Ranging_Session_RQ with its UWB_Session_Id.

To set up secure ranging over Bluetooth LE, each message shall be encoded as below:

Message Type: UWB Ranging Service message

Message:

- Ranging Service Message: RS_RQ (see Section 19.3.1.3)
- Ranging Service Message: RS_RS (see Section 19.3.1.4)
- Ranging Service Message: RSS_RQ (see Section 19.3.1.5)
- Ranging Service Message: RSS_RS (see Section 19.3.1.6)
- DK Event Notification (see Section 19.3.8)

Figure 19-23 illustrates the Secure Ranging Setup flow.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The vehicle shall perform secure ranging and detect that the device is inside the vehicle before proceeding to Owner Pairing Phase 3. If the device is not detected inside of the vehicle or other ranging failure occurred, the vehicle shall send an appropriate SubEvent code defined in Table 19-72 or Table 19-74 and abort the owner pairing attempt.

Owner Pairing Phase 3

To perform Owner Pairing Phase 3 exchange (see Section 6.3.4) over Bluetooth LE, each APDU command and response shall be encoded as below.

Message Type: SE message

Message:

APDU Command Encapsulation: DK_APDU_RQ (see Section 19.3.2.1)

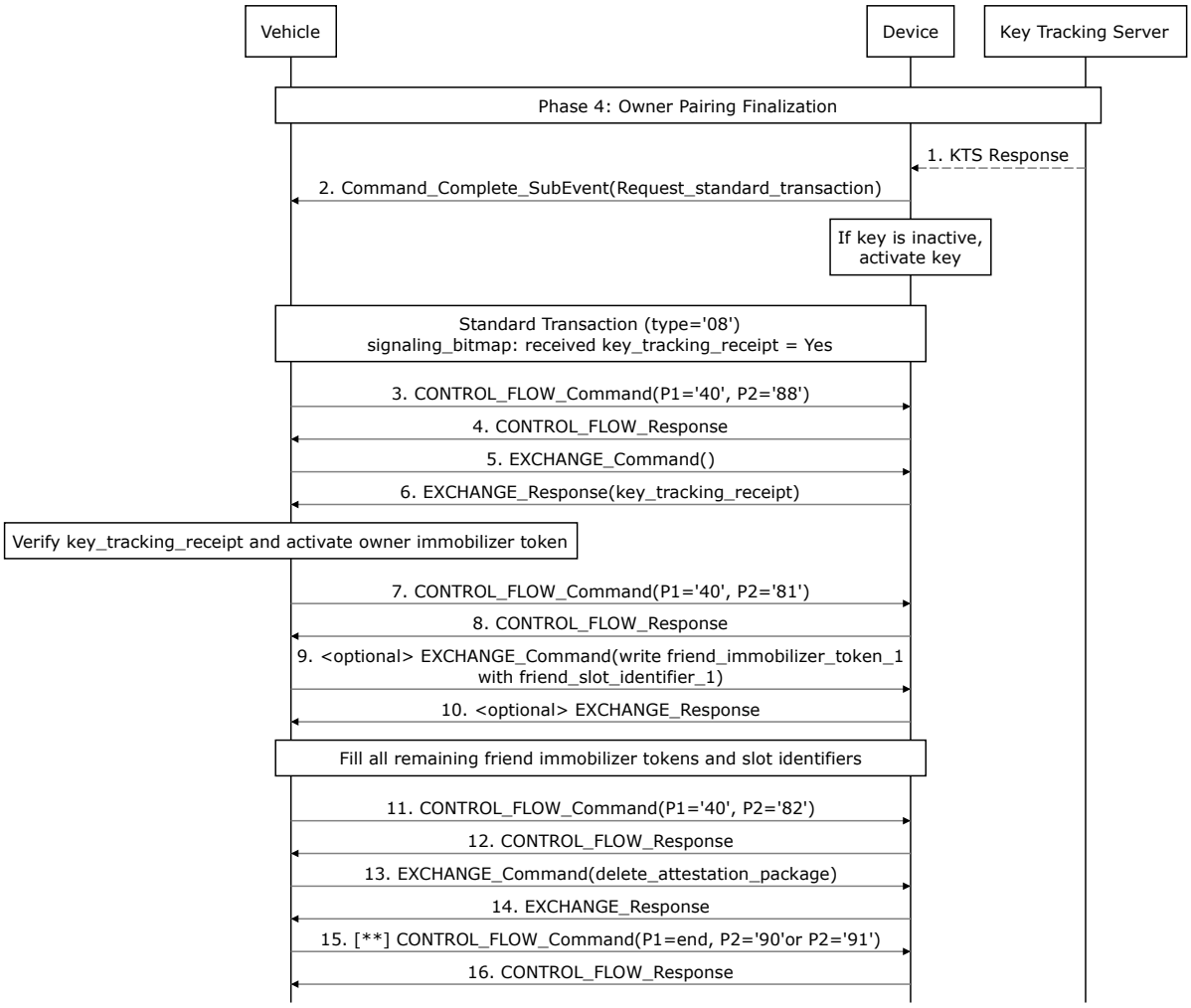
APDU Response Encapsulation: DK_APDU_RS (see Section 19.3.2.2)

Owner Pairing Phase 4

Owner Pairing Phase 4 exchange (see Section 6.3.5) over Bluetooth LE shall be modified as shown in Figure 19-19. If KTS is supported, the device shall wait for KTS the response before sending the Request_standard_transaction subevent to the vehicle to trigger standard transaction.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-19: Owner Pairing Phase 4 over Bluetooth LE



Each APDU command and response shall be encoded as below

Message Type: SE message

Message:

APDU Command Encapsulation: DK_APDU_RQ (see Section 19.3.2.1)

APDU Response Encapsulation: DK_APDU_RS (see Section 19.3.2.2)

19.5.2 *Head Unit Pairing Flow*

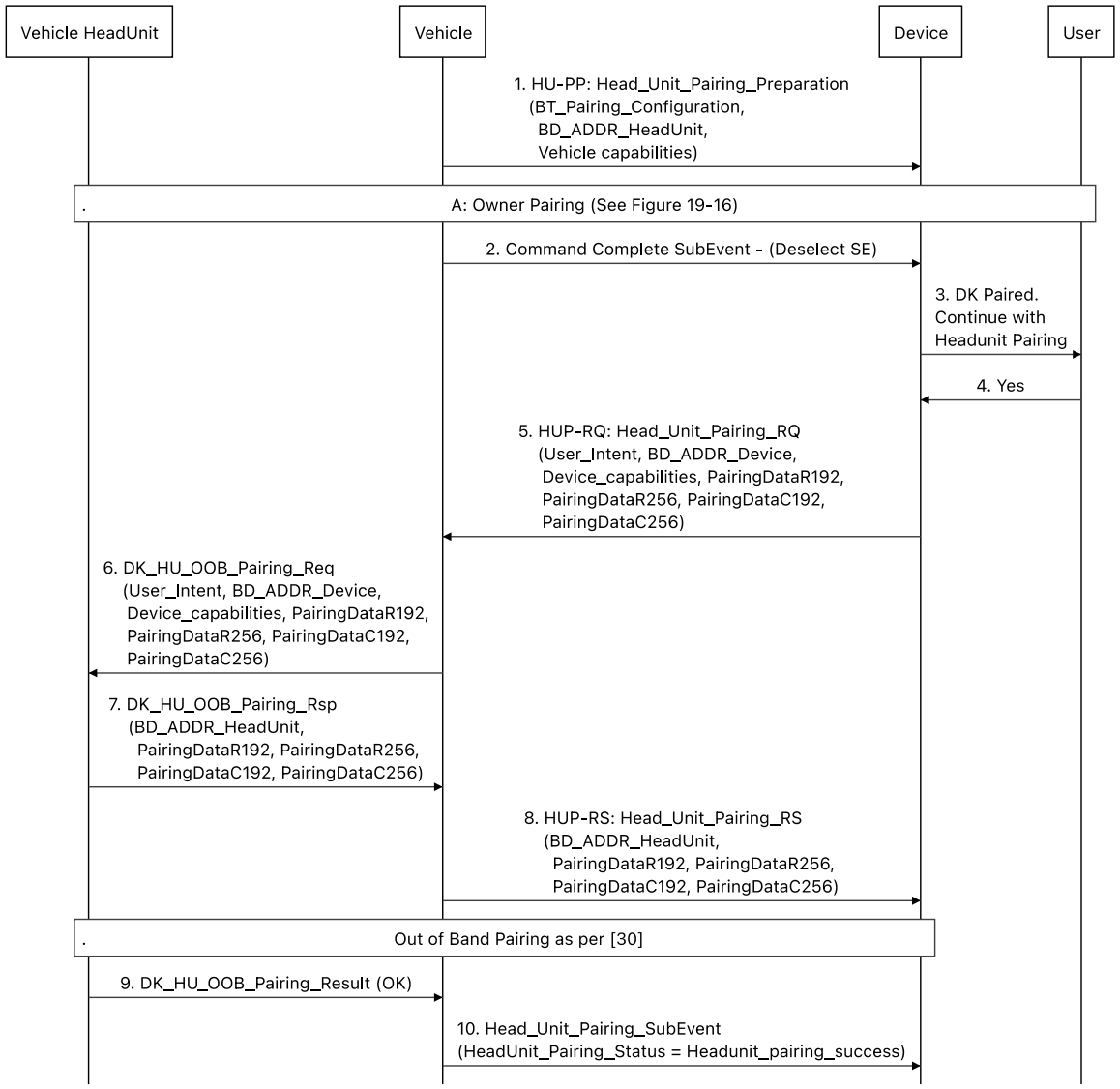
This section describes the Head Unit Pairing flow once the DK pairing is completed. This feature, referred as “Head Unit Pairing”, is optional for both the device and vehicle. This feature only applies if the device’s Bluetooth supports BR/EDR (classic), and the vehicle Bluetooth module for DK is different from that of the head unit (i.e. in a different subsystem or a physical implementation). The Head Unit Pairing flow is shown in Figure 19-20.

The Head Unit Pairing Preparation (HU-PP) message is used for head unit capability discovery and initiating the Head Unit Pairing.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The vehicle may send HU-PP either before or after owner pairing is completed. Sending the HU-PP prior to owner pairing completion allows the device to display a single message at the Subevent Notification about DK pairing complete and head unit capability. If the Head Unit Pairing is supported by the vehicle, the vehicle shall send the HU-PP within 2 seconds from the Command Complete Subevent (Deselect_SE) at the end of owner pairing Phase 4 i.e., step 5 in Figure 19-16. If the Head Unit Pairing is not supported or for all subsequent head unit (re)pairing the device and vehicle shall follow the Bluetooth pairing and encryption setup as defined in [30].

Figure 19-20: Head Unit Pairing Flow Diagram.



Note that the order of HU-PP message and owner pairing subsection (refer to Box A in Figure 19-20) is an example and can be interchanged.

If a device supports Head Unit Pairing, the device shall send HUP-RQ in step 5 (see Section 19.3.7.2). The vehicle shall respond with HUP-RS in step 8 (see Section 19.3.7.3) to complete

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

the Head Unit Pairing. If the device does not support Head Unit Pairing, the flow in Figure 19-20 completes after step 2 and subsequent steps are not required.

If Headunit pairing (step 10) is not finished within 60 seconds from the moment the message HUP-REQ has been sent, the device may retry the head unit pairing from step 3.

Note that DK_HU_OOB_Pairing_Req and DK_HU_OOB_Pairing_Result messages in step 6 and 7 are implementation-specific by the vehicle OEM and are only provided as an example.

For Friend First Approach (Section 19.5.8.2), the message HU-PP shall be sent after the "secure ranging setup flow" is completed.

The flow diagram in Figure 19-20 shows the case where OOB communication is possible from both directions. The vehicle shall configure the parameter BT_Pairing_Configuration in message 1 to indicate to device if the vehicle head unit supports both directions or a single direction (vehicle head unit to device) for the OOB communication.

If OOB communication is possible only in the direction from vehicle head unit to device, the parameters PairingDataR192, PairingDataR256, PairingDataC192, and PairingDataC256 in step 6 should be ignored by the vehicle head unit (see volume 2 Part H Section 7.2.2 of [30].

19.5.3 *Passive Entry*

After owner pairing has been successfully performed, upon each approach, at minimum secure ranging flow shall be exercised to establish secure ranging before enabling any of the passive entry features/actions such as 'Welcome Lights', 'Unlock', 'Lock', etc. Once the secure ranging has been established and device has been localized, vehicle may decide to initiate any of the above actions, as per its policy or requirements.

For passive entry, one of the following conditions shall trigger the vehicle (except if the vehicle is in low power mode, or vehicle is not ready, for example, due to internal conflict) to establish secure ranging with the device:

1. Device sends Device Ranging Intent SubEvent with at least low_approach_confidence.
2. The vehicle determines that there is user intent to use vehicle features that require secure ranging with the device, e.g., user touches door handle

The URSK is needed before secure ranging can be established. The vehicle may have a pre-derived URSK or derive a new URSK as needed. The URSK confidentiality and integrity shall be preserved during the whole lifetime of the URSK.

19.5.4 *URSK Management*

Vehicle may derive and store a new URSK by:

- Executing a dedicated transaction flow (i. e. with a AUTH0 command indicating a transaction_code value 10_h) as shown in Figure 19-21.

  OR

- By adding a CREATE RANGING KEY command to a Standard Transaction flow intended for another purpose like start engine (i.e., with a AUTH0 command indicating a transaction_code value not equal to 10_h) as shown in Figure 19-30; see section 19.5.6.1. Each time this flow is executed successfully, a unique URSK is derived and stored indexed using a UWB_Session_Id. This UWB_Session_Id is the least significant 4 bytes

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

of the transaction_identifier, a 16 bytes random number generated by the vehicle for each AUTH0 and sent to the device as part of AUTH0 command along with other parameters. For more details, see Section 15.3.2.9.

To derive URSK over Bluetooth LE, each APDU command and response shall be encoded as below.

Message Type: SE message

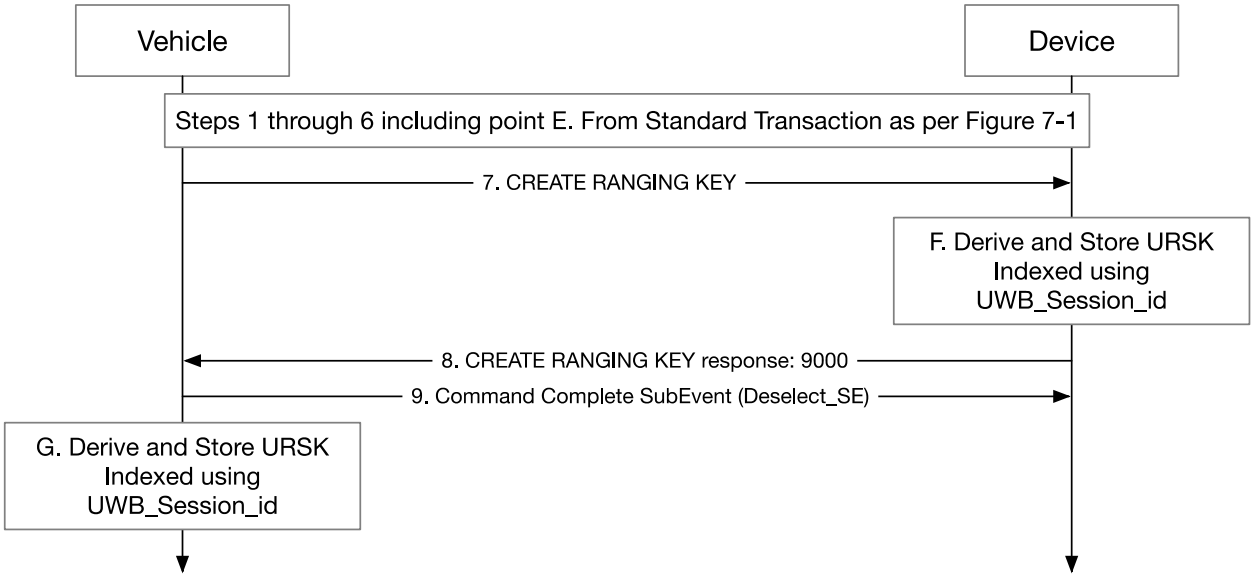
Message:

APDU Command Encapsulation: DK_APDU_RQ (see Section 19.3.2.1)

APDU Response Encapsulation: DK_APDU_RS (see Section 19.3.2.2)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-21: URSK Derivation Flow in a dedicated Standard Transaction



To find details on vehicle's processing for box A, E and H, see Section 7 and Section 15.3.2.26. For box G and H, once the URSK has been successfully derived, it shall be stored in secure storage indexed using UWB_Session_Id.

Vehicles capable of secure ranging shall support the derivation and storage of at least one pre-derived URSK per Digital Key. Devices capable of secure ranging shall support the derivation and storage of two pre-derived URSKs per Digital Key. Each of these pre-derived URSKs is indexed using its associated UWB_Session_Id.

The vehicle shall use the Secure Ranging Setup Flow (see Figure 19-23) to activate a pre-derived URSK. There shall be at most one active URSK per Digital Key, and this active URSK is discarded anytime another one is activated.

After a pre-derived URSK is activated, the vehicle shall request derivation and storage of another pre-derived URSK. The vehicle shall control when this additional URSK derivation occurs to minimize user impact (for example, after passive entry completed) but is recommended to occur shortly after URSK activation. Table 19-87 shows the URSK storage requirements.

Table 19-87: URSK storage requirements per Digital Key endpoint.

URSK Types	Definition	Vehicle URSK Storage	Device URSK Storage
Pre-derived	URSK that has not been activated using the Secure Ranging Setup Flow and has only been kept in secure storage. TTL is not defined until key is activated.	At least 1	2
Active	URSK activated using the Secure Ranging Setup Flow and has neither exhausted its STS_Index max incrementation nor has an expired TTL.	1	1

The URSK shall be discarded if one of the following conditions is met:

1. The STS_Index reaches its maximum value of 2^31-1.
2. The STS_Index was lost, and it cannot be ensured that a previously used STS_Index won't be used again

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

3. The URSK TTL (Time-To-Live) has expired. The vehicle shall enforce an URSK TTL lower or equal to 12 hours (vehicle OEM specific). The URSK TTL starts when the first dURSK is derived. The first dURSK shall be derived immediately before sending or after receiving Ranging Session Setup Response for the device or vehicle, respectively.
4. A new URSK is activated through Secure Ranging Setup Flow.

19.5.5 *Flow Selection - Establish Secure Ranging*

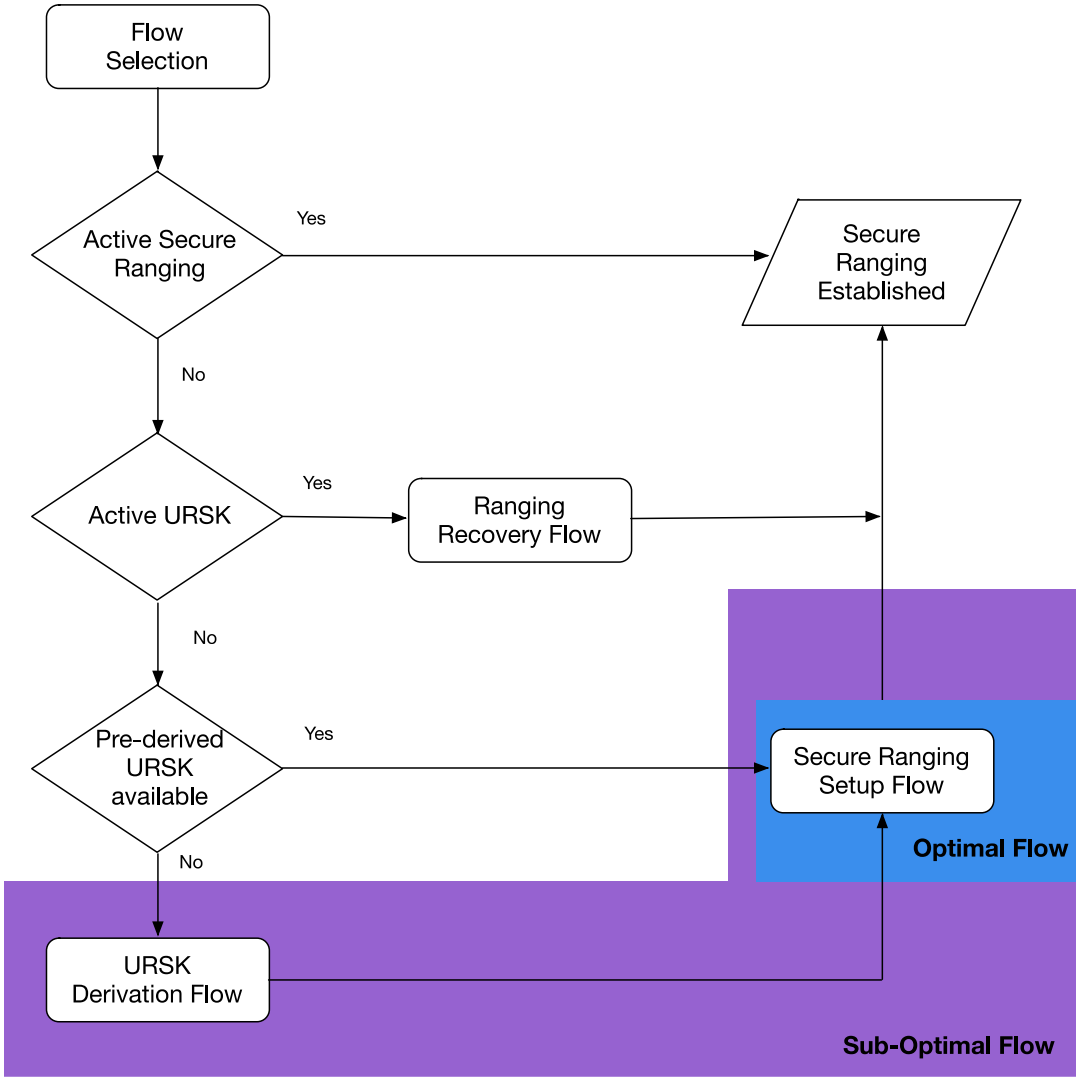
The vehicle has three options to establish secure ranging:

1. Optimal Flow
2. Sub-Optimal Flow
3. Ranging Recovery Flow

To decide which flow needs to be exercised for establishing secure ranging, vehicle shall go through below flow selection. In the following, a ranging session is considered as an Active Ranging Session when the vehicle and the device are exchanging UWB secure ranging messages using the same URSK. This flow is shown in Figure 19-22.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-22: Flow Selection for Establishing Secure Ranging



If the vehicle needs to localize the device, the vehicle shall first check which flow should be initiated for establishing secure ranging. For this, the vehicle shall first check if it already has an active ranging session. If it does, vehicle shall use that session to localize the device.

If there is no active ranging session, the vehicle shall then check whether an active URSK exists. If it does, the vehicle shall follow the ranging recovery flow to recover the ranging session (using its UWB_Session_Id) associated with that active URSK.

If there is no active URSK, the vehicle shall check whether a pre-derived URSK exist. If it does, the vehicle shall follow optimal flow to establish secure ranging.

If there are no active or pre-derived URSKs, the vehicle shall fallback to sub-optimal flow.

If there is an active ranging session, and if the URSK TTL at the vehicle is about to expire or vehicle decides to use a URSK with shorter TTL for engine start (see Section 19.5.6.1), the vehicle may setup a new secure ranging session with a pre-derived URSK. When the pre-derived URSK becomes active, the vehicle and the device shall discard the previously active URSK (if

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

any). For more details on ranging recovery flow, optimal flow and sub-optimal flow, refer to below subsections.

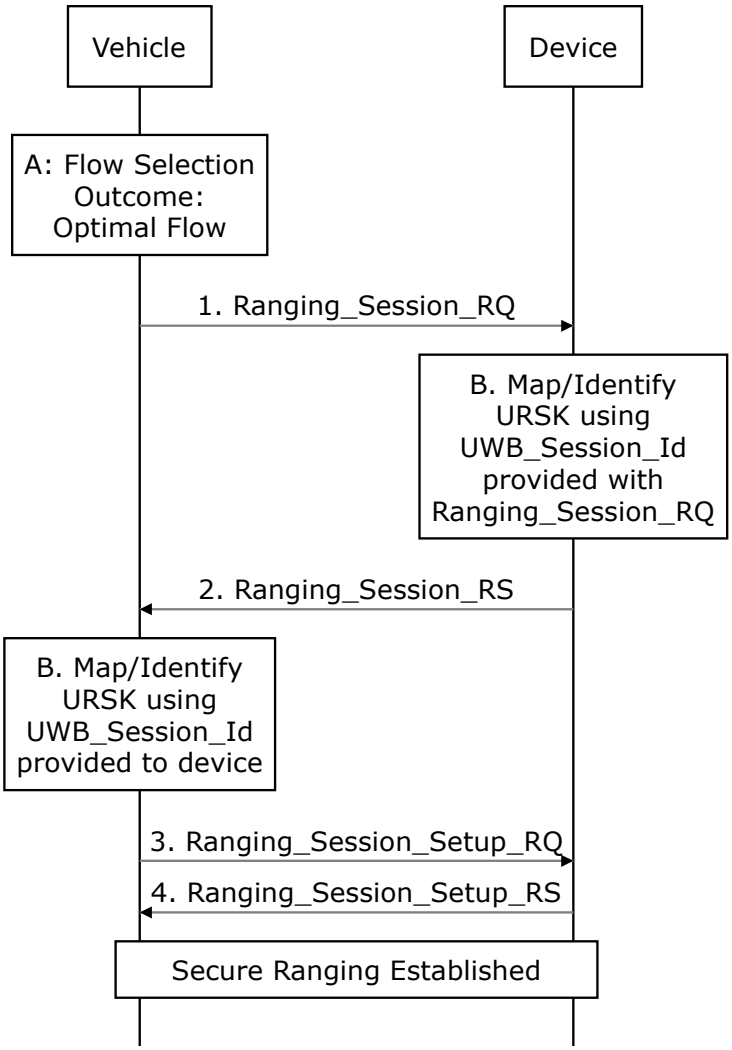
Optimal Flow

Optimal flow shall be exercised by vehicle, if it has a pre-derived URSK available. Optimal flow simply follows Secure Ranging Setup flow using a pre-derived URSK. If the vehicle uses this flow to activate a pre-derived URSK, the vehicle and device shall discard the currently active URSK (if any) for the associated Digital Key before the new URSK is activated.

To setup secure ranging, each message shall be encoded with Message Type: UWB Ranging Service message

The Secure Ranging Setup flow is shown in Figure 19-23.

Figure 19-23: Secure Ranging Setup Flow.



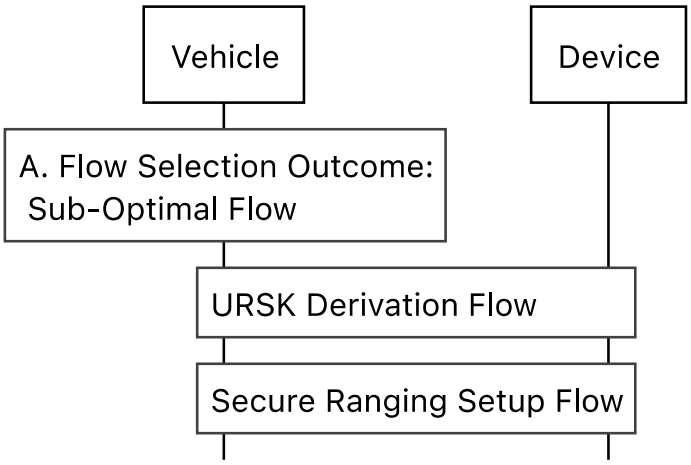
Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Sub-Optimal Flow

Sub-Optimal flow shall be exercised if a vehicle has no pre-derived URSK available. Sub-Optimal flow is a combination of URSK Derivation flow and Secure Ranging Setup flow.

Figure 19-24 illustrates the use of Sub-Optimal flow, in the absence of pre-derived URSK.

Figure 19-24: Sub-Optimal Flow.



In addition, if the device fails to activate an URSK as part of Optimal flow and no more pre-derived URSKs are available or when vehicle URSKs are out of sync with device URSKs (refer Section 19.8), vehicle shall fall back to Sub-Optimal flow.

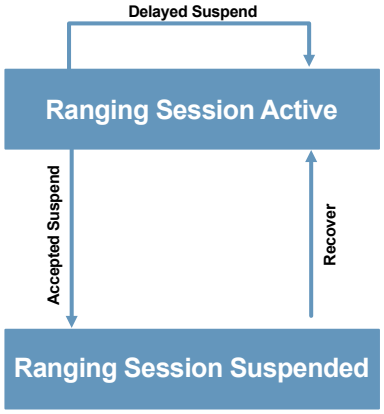
Once secure ranging has been established by either using Optimal or Sub-Optimal flow, vehicle may localize the device. Once device localized, vehicle may perform any of the passive entry features/actions.

Ranging Recovery Flow

Once a ranging session has been established, it can be active for an extended period of time as long its URSK TTL has not expired or STS_Index has not reached max limit. However, as shown in Figure 19-25, either the vehicle or device may put an active ranging session in a suspended state for power optimization.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-25: Ranging Session State Machine.



The decision on when to send a Ranging Suspend Request is up to the sender requester. However, the receiver may choose to delay the suspension request for a limited period of time. For example, device may request to put the ranging session to suspended state, if it does not detect any motion for extended period of time. On the other hand, vehicle may respond back with the request to delay the suspension, if it strongly believes ranging session is either still needed or will be in immediate future.

Figure 19-26 illustrates Ranging Suspend Accepted flow when requested by the device.

Figure 19-26: Ranging Suspend Accepted Flow.

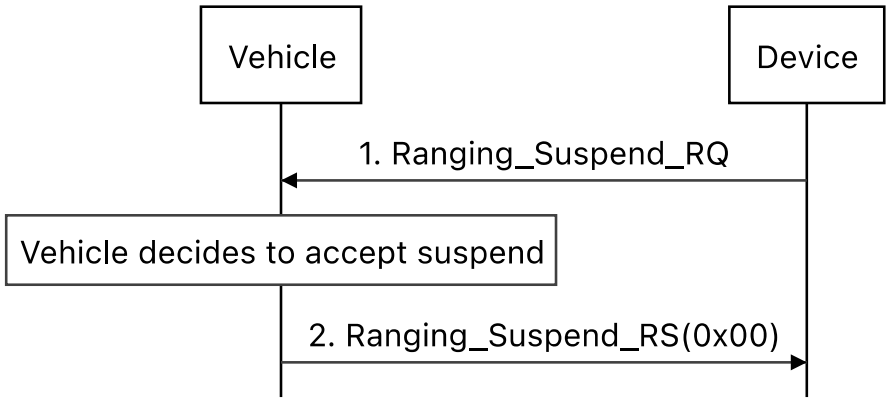
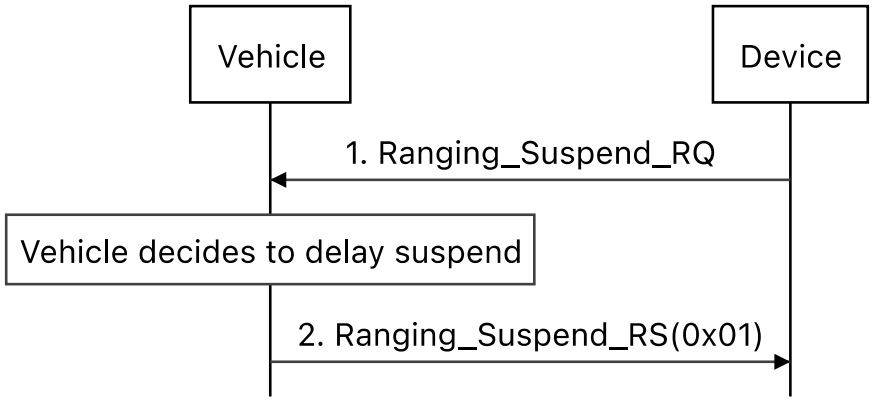


Figure 19-27 illustrates Ranging Suspend Delayed flow when requested by the device.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-27: Ranging Suspend Delayed Flow.



When in suspended mode, vehicle may send a Ranging Recovery Request message (see Section 19.3.1.9) to the device. The device may trigger ranging recovery by sending Device Ranging Intent SubEvent to the vehicle

The Ranging Recovery flow offers a low latency based secure ranging which requires minimal exchange and no new URSK.

However, before initiating Ranging Recovery flow, vehicle shall meet the following requirements:

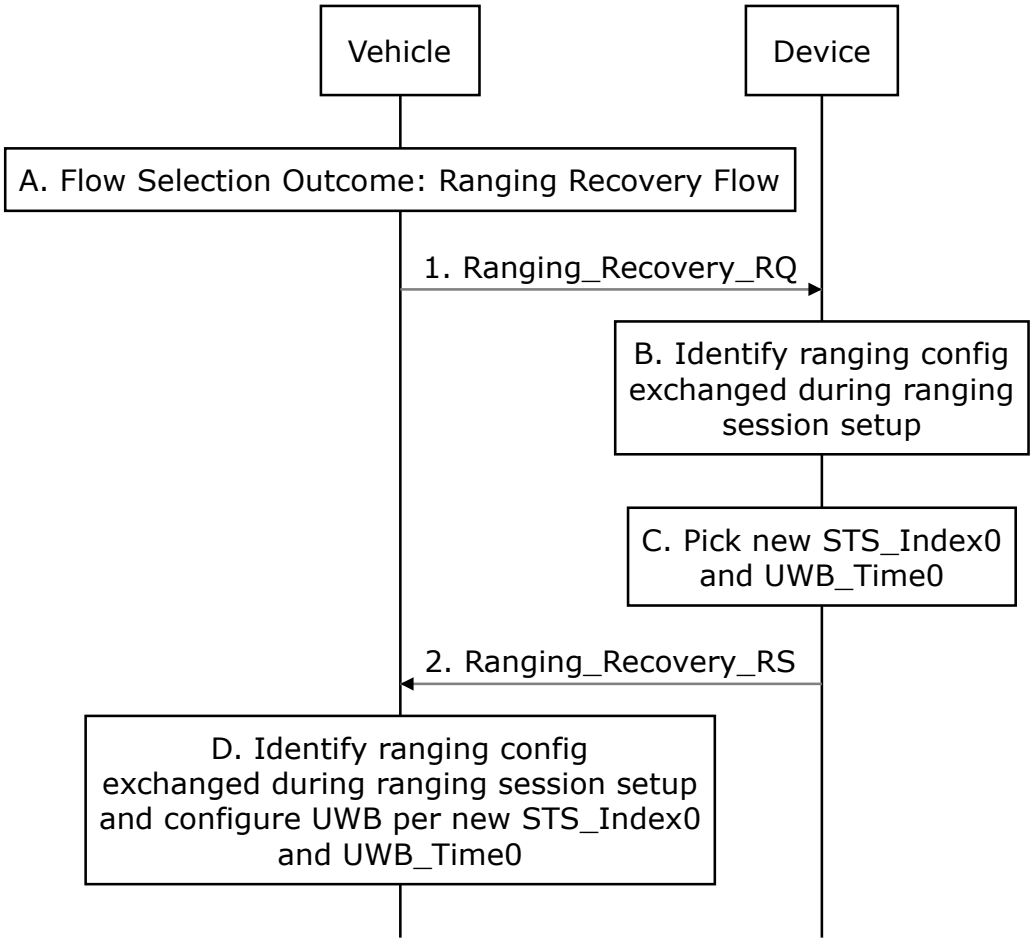
1. Has an active URSK (TTL has not expired) associated with the suspended ranging session in which the vehicle intends to recover
2. No change to ranging configurations of the suspended ranging session is needed, such as change of frequency, # of anchors, etc.

If vehicle needs a secure ranging, the vehicle shall check whether there is any suspended ranging session for the connected device that meets both of the above requirements. If it does, the vehicle shall initiate Ranging Recovery flow. When device receives the Ranging Recovery Request, it shall identify the same set of configurations used to establish the ranging session for the provided UWB_Session_Id. Device shall then pick and send a new UWB_Time0 and STS_Index0.

Figure 19-28 illustrates Ranging Recovery flow.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-28: Ranging Recovery Flow.



For box A, refer to Section 19.5.5.

For box D, vehicle shall select previously negotiated UWB config parameters for UWB_Session_Id associated with recovering ranging session.

Vehicle Low Power State

When the vehicle is in low power state, it may not start ranging after establishing a Bluetooth LE connection. In such case the vehicle may send a vehicle state SubEvent notification indicating its "Low power mode". After receiving this notification, the device shall be responsible to start ranging using the Ranging Intent SubEvent.

The device may immediately send the Ranging Intent or at a later point with higher confidence, when a vehicle approach has been detected.

19.5.6 *Standard Transaction over Bluetooth LE*

If vehicle requires a standard transaction (e.g. in order to request an immobilizer token), the standard transaction protocol as shown in Figure 7-1, followed by a Command Complete SubEvent with Command_Status Deselect_SE, shall be performed.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Each APDU command and response shall be encoded as below.

Message Type: SE message

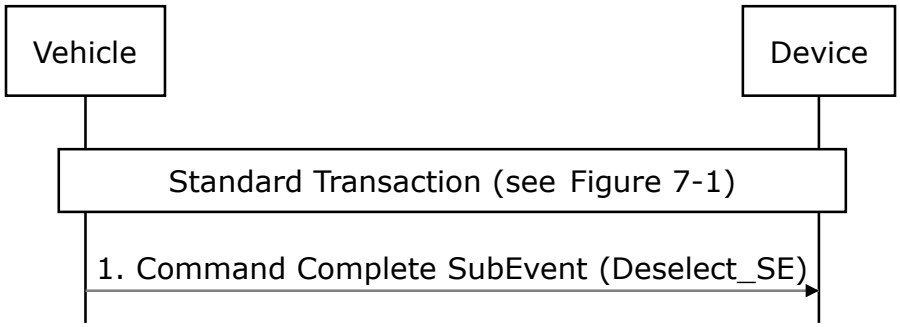
Message:

APDU Command Encapsulation: DK_APDU_RQ (see Section 19.3.2.1)

APDU Response Encapsulation: DK_APDU_RS (see Section 19.3.2.2)

Figure 19-29 illustrates the standard transaction flow.

Figure 19-29: Standard transaction over Bluetooth LE.



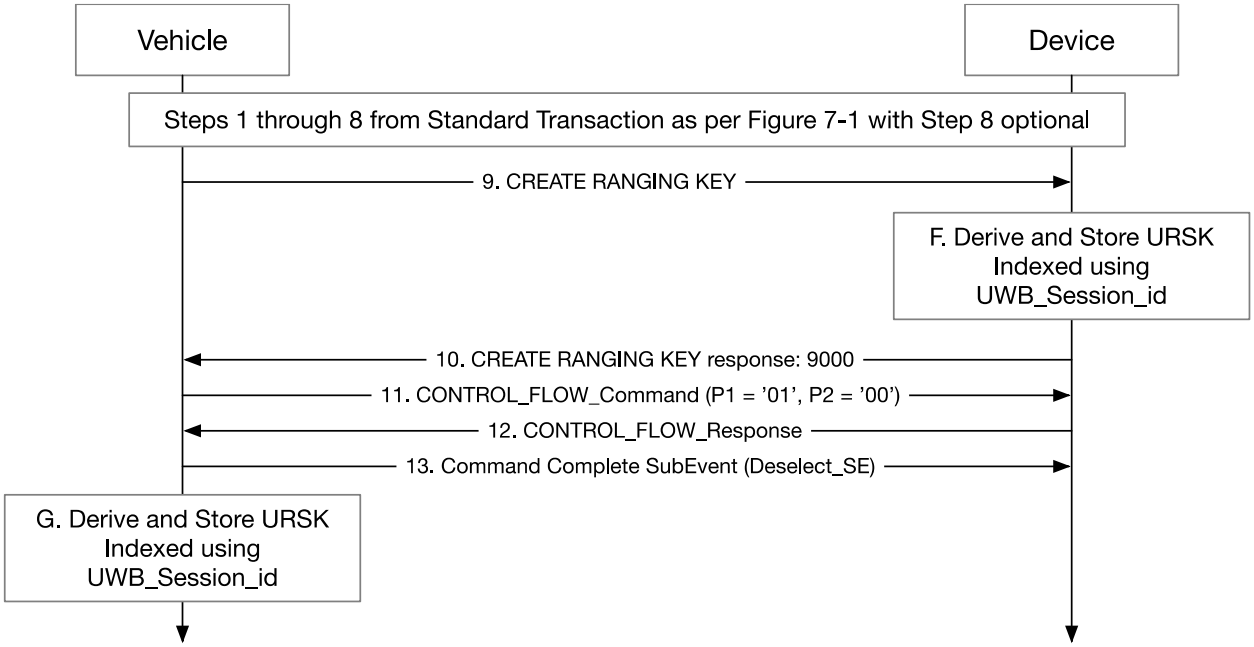
19.5.6.1 Standard Transaction with URSK derivation via Bluetooth LE

Optionally, a URSK can also be derived within a standard transaction via Bluetooth LE that was mainly intended for other purposes like start engine, i. e., regardless of the AUTH0 Command Transaction Type Coding. The necessary steps for the URSK derivation are performed after an otherwise successful standard transaction with (optional) EXCHANGE commands. This is depicted in Figure 19-30.

Note: Steps 1 through 8 in Figure 19-30are identical to Steps 1 through 8 from Figure 7-1.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-30: URSK Derivation Flow in a Standard Transaction intended for another purpose



If an error occurs within steps 1 to 8 of the standard transaction, URSK derivation is skipped and the appropriate error is signaled to the device. Vehicle then sends Command complete SubEvent (Deselect SE) to Device as per step 11 from Figure 19-30.

In case an error occurs during URSK derivation, only the action triggered with the preceding standard transaction is (already) accepted by the vehicle. If necessary, step 11: Command Complete SubEvent (Deselect SE) shall be sent to the device.

If a URSK derivation is still needed, a dedicated standard transaction for URSK derivation as described in section 19.5.4 should be performed.

19.5.7 *Engine Start*

Engine start shall be authorized for a limited time, only after completion of a standard transaction (which may also be used to retrieve an Immobilizer Token) and a UWB ranging.

The vehicle OEM may choose to setup a new ranging session using a different URSK with short TTL for engine start authorization. The usage of a short TTL (e.g. 30s) for the UWB ranging used for engine start increases overall security by reducing the general exposure of the URSK used for enabling engine start.

19.5.8 *Friend - Sharing & First Approach*

19.5.8.1 Friend Sharing

Digital Key solution supports sharing feature which allows vehicle owners to offer managed digital access of their vehicle with other friends. For a Bluetooth LE/UWB capable vehicle, regardless of owner device's 'Wireless Capabilities', sharing ensures that a friend device who may have UWB capabilities can enjoy Bluetooth LE/UWB based experience even when owner's device may not have Bluetooth LE/UWB capability.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

To enable passive entry experience for friend devices, vehicle provides its 'Wireless Capabilities' (7F49 template) and both device and vehicle derive Kble_oob_master as part of 'Owner Pairing' flow as described in Section 19.5.1.

During sharing, owner's device make use of Kble_oob_master and DK_Identifier to derive Kble_oob as described below.

Friend Bluetooth LE Pairing: Friend OOB Pairing Data Derivation

- Kble_oob = HKDF-SHA256(16, Kble_oob_master, DK_Identifier)
  // (output_len, input_keying_material, info)

Where DK_Identifier is an 8-byte padded slot_identifier of the friend. For example, if the slot_identifier is 6 bytes, it will have 0x0000 added as prefixed to make it 8 bytes.

Owner device shall send an updated 7F49 template (see Table 19-86) as part of the Import Request. This template is shared by the owner device to the friend device as described in Section 11.8.2.1.

19.5.8.2 Friend First Approach

For a friend with NFC/Bluetooth LE/UWB capable device which has successfully received Digital Key access for owner's vehicle (Bluetooth LE/UWB capable), the friend device can enjoy authorized RKE features using Bluetooth LE and passive entry feature without any further manual input or intent.

Note that the first friend transaction over Bluetooth LE is unrelated to any vehicle action such as lock or unlock. There is no requirement for user authentication for this transaction type.

After a successful Bluetooth LE Pairing Procedure, the vehicle shall initiate a first friend transaction. If the first friend transaction was successful, the vehicle shall end the Standard Transaction as defined in Figure 7-1 by a CONTROL FLOW command indicating First approach successful.

The Bluetooth LE Pairing Data shall be persisted:

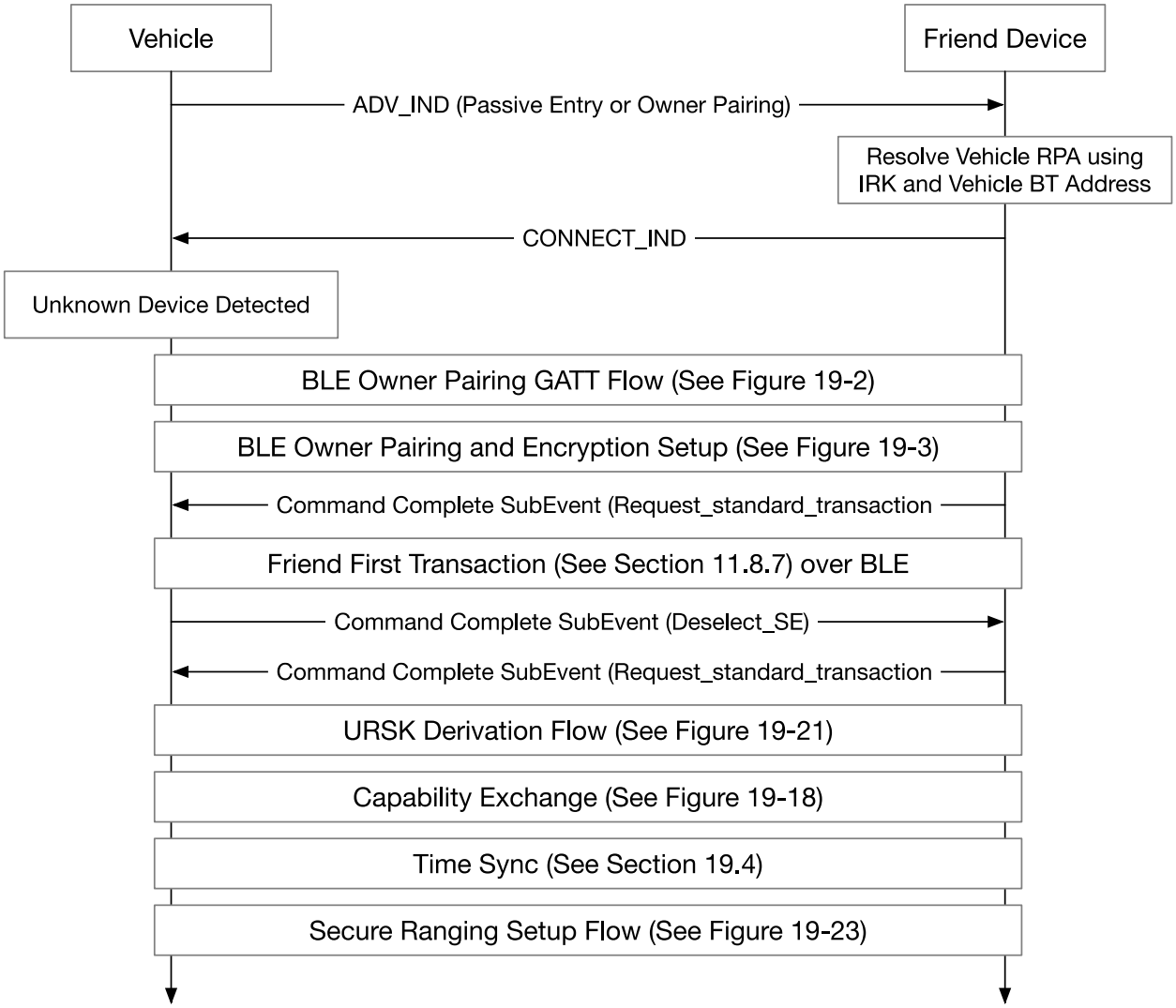
1. For the vehicle, when the CONTROL FLOW command indicating First approach successful was sent,
2. For the device, when the CONTROL FLOW command indicating First approach successful was received.

Any error prior to this point shall lead to rolling back the Bluetooth LE pairing data. The device shall then retry the first approach flow on reconnecting to the vehicle.

Figure 19-31 illustrates Friend First Approach flow.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-31: Friend First Approach Flow.



Bluetooth LE Link Layer Connection Establishment and L2CAP Setup Connection-Oriented Channel Setup

Upon receiving an ADV_IND from the vehicle, a friend device shall first resolve the vehicle RPA using the IRK, and then shall send a CONNECT_IND to establish Bluetooth LE link layer connection. Once the Bluetooth LE link layer connection is established, the friend device and vehicle shall establish L2CAP Connection-Oriented channel.

Bluetooth LE Pairing and Encryption Procedure

Both device and vehicle shall perform Bluetooth LE Secure OOB Pairing Prep flow, except for Kble_oob derivation (see Figure 19-17, except for box D).

Once vehicle has successfully derived Kble_oob, device shall initiate Bluetooth LE Pairing procedure by sending pairing request as described above in Section 19.2.1.4. Post successful pairing, device shall setup Bluetooth LE encryption before moving to Friend First Transaction and URSK Derivation flow as described in Sections 11.8.7 and 19.5.4, respectively.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Any Bluetooth LE messages being exchanged over-the-air as part of this flow shall not make use of DK message defined in Section 19.3.

Capability Exchange

For Friend First Approach, the Capability Exchange shall be performed as described in Figure 19-18.

Time Sync

If the Device UWB Clock is “Not in sync” while Bluetooth LE connected with the device, the vehicle shall trigger the Bluetooth LE Timesync procedure 1 if all conditions are met (see Section 19.4.2)

19.5.9 *RKE Function Flow*

Remote transactions allow device to initiate on-demand features (e.g. lock, unlock and alarm) from within Bluetooth LE range. The protocol supports single event-triggered actions (e.g., locking on single tap) and enduring actions (e.g., holding a slider to adjust a window).

RKE Functions shall be supported by vehicles and devices with Wireless Capability Combinations WCC2 or WCC3 (see Table 19-78 for list of functions)

Before triggering a desired action, the device shall perform user authentication (see Section 9). If the user is successfully authenticated, the device shall request a challenge from the vehicle by sending RKE Request SubEvent. The device shall use the RKE_Challenge received in RKE_AUTH_RQ as arbitrary data to create a device signature using its private key for the requesting vehicle.

The device shall provide the user with the option to enable implicit UA, to enable explicit UA, or to disable use of RKE according to one of the allowed configurations defined in Table 19-88. RKE policies may be selected individually for each Digital Key.

Table 19-88: RKE allowed user authentication configuration.

	Improved UX Policies	High Security Policies	
Configuration	Enable Implicit UA	Enable Explicit UA	Disable RKE
1	Supported	Supported	Supported
2	Supported	Supported	Not Supported
3	Supported	Not Supported	Supported
4	Not Supported	Supported	Supported
5	Not Supported	Supported	Not Supported

The implicit UA policy is described in Section 9.2.

The explicit UA policy shall require an additional UA to perform an RKE function (see Section 9.1). This additional UA shall be performed after the intent to perform the RKE function has been registered and before the device signature for the RKE_AUTH_RS is generated for a specific key in the SE.

If usage of RKE is enabled, explicit UA should be selected by default.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

To exchange RKE_Challenge and challenge response over BLE, it shall be encoded as below.

Message Type: Supplementary Service message

Message Type:

Message: RKE_Auth_RQ (see Section 19.3.5.1)

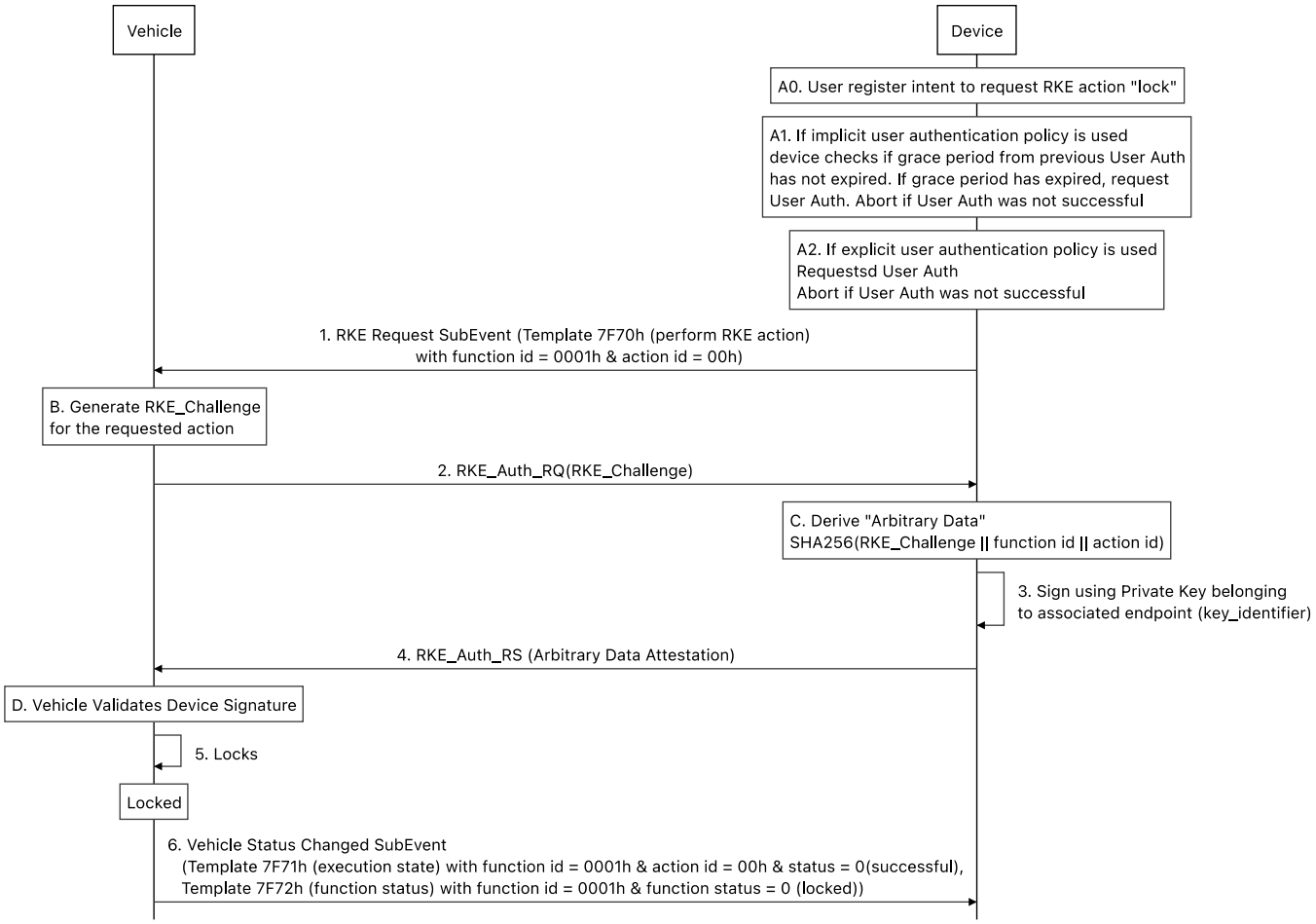
Message: RKE_Auth_RS (see Section 19.3.5.2)

DK Event Notification (see Section 19.3.8)

19.5.9.1 Event-based RKE Action

Figure 19-32 provides an example flow for the event-based RKE flow with function status exchange.

Figure 19-32: RKE Flow for an Event-based RKE Action.



In box A, the user triggers the action and performs user authentication (see Section 9).

In box C, the device shall derive arbitrary data by performing SHA256 of the concatenation of the RKE_Challenge and the corresponding requested RKE Request SubEvent function id and action id (see Section 19.3.8.2).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

In box D, vehicle shall derive arbitrary data using the same procedure as the device and shall perform signature verification (see Section 15), before the action is executed.

In step 6, the successful execution together with a Vehicle function status change is communicated via the Vehicle Status Changed SubEvent.

19.5.9.2 Enduring RKE Action

Enduring RKE allows an action to be performed as long as the user confirms that the action should be continued (e.g. by holding a slider to close a window). The vehicle may stop the function when an end state was reached or an error occurred.

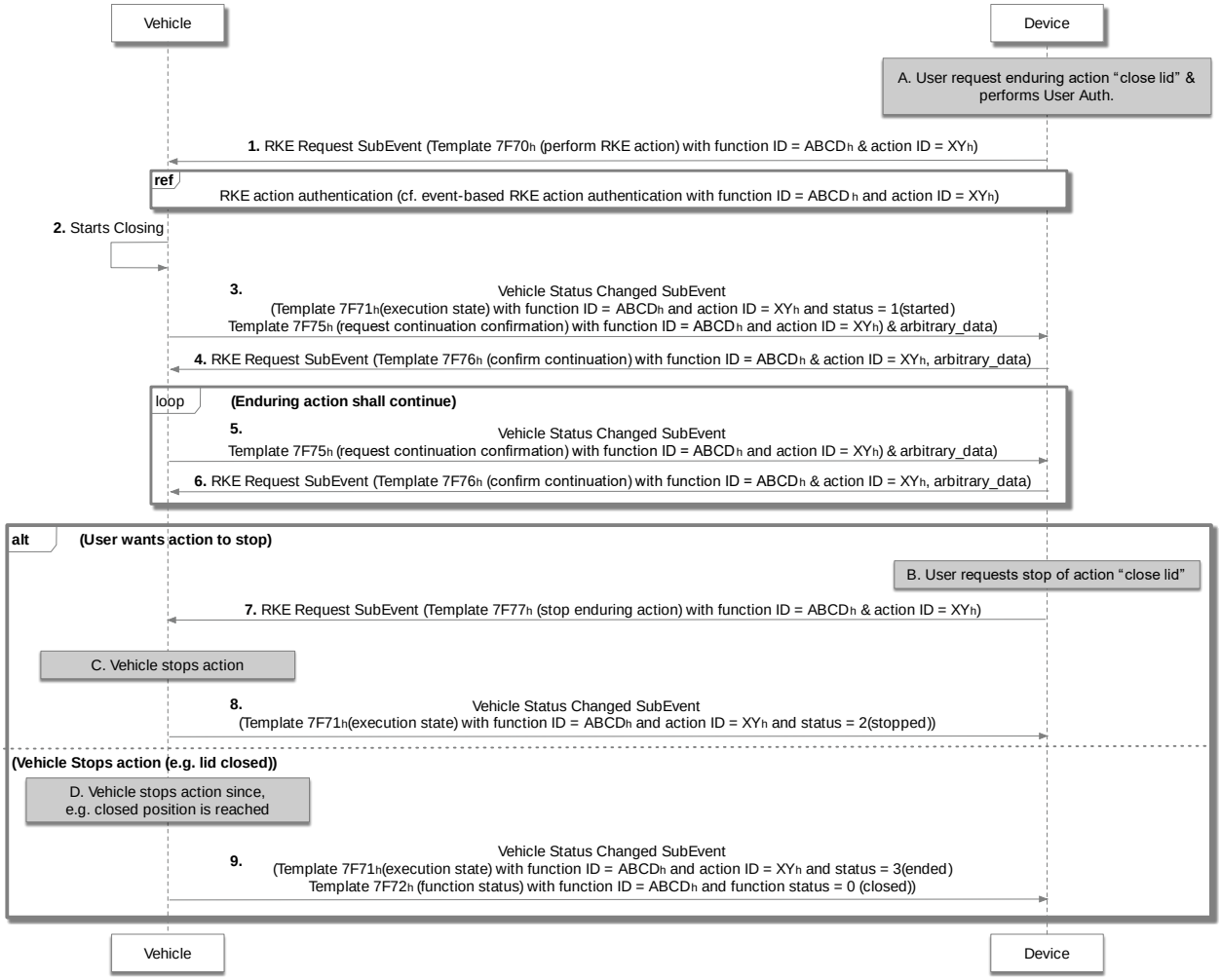
Enduring RKE offers two modes:

1. With Continuous Confirmation: (see loop with “Enduring action shall continue”) is required. In this case, the continuation confirmation shall only be sent if the user confirms the action on the user interface.
2. Without Continuous Confirmation: a certain action takes place until the device stops the execution (loop with continuation request is not present).

Figure 19-33 provides an example of an Enduring RKE action with Function id ABCD and action id XY are used for illustration only.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-33: RKE Flow for an Enduring RKE action.



In box A, the user starts the Enduring RKE action.

After performing the RKE authentication as defined in Section 19.5.9.1, the Enduring RKE flow starts. The confirmation interval is controlled by the vehicle. The arbitrary data as payload in the Vehicle Status Changed SubEvent with "Request Continuation Confirmation" and, in the RKE Request SubEvent with "Confirm Continuation" can be the basis of a vehicle OEM-specific mechanism to ensure that the app on the device is still reachable or to perform round-trip time measurements. If a "Request Continuation Confirmation" Vehicle Status Changed SubEvent contains arbitrary data, the device shall return it unmodified in the corresponding "Confirm Continuation" RKE Request SubEvent. This should happen within the next connection event (30ms recommended). It is up to the Device OEM implementation to call the callback before or after sending the "Confirm Continuation" RKE Request SubEvent.

While the user is actively using an Enduring RKE function with continuous confirmation on the device for the given vehicle, and if the vehicle does not receive a continuation confirmation within 100ms after a continuation confirmation request was sent, it may stop the function. This time measurement is between the over-the-air packet sent by vehicle and the over-the-air packet received by the vehicle.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

In box B, the user wants to stop the Enduring RKE action. This is indicated by the device via the "stop enduring action" payload.

In box D, the vehicle initiates the stop of the Enduring RKE action. The device can be notified when an enduring function reaches a certain stop state or when an error occurs.

The Vehicle Status Changed SubEvent may contain one or more function status updates along with the Continuation Confirmation Request.

19.5.9.3 Vehicle Function Status Update

Figure 19-34 shows the possible flows for requesting a vehicle function's status and being informed about status changes. The subscription mechanism allows the device to subscribe to vehicle OEM-specific function ids, the vehicle shall send function status updates when the status of those function ids changes. With each subscription message, the vehicle shall delete the current subscription ranges and take over the new ranges. To unsubscribe from updates, a single function range with "from function id" = 0 and "to function id" = 0 shall be sent.

Besides subscription, the status of all supported functions or the status of specific functions can be requested. When all supported functions are requested by the device, the vehicle shall only return the function ids with status that are supported by the vehicle. The function status report can be split in multiple messages depending on the maximum supported L2CAP SDU size.

For the standardized functions, no subscription is necessary. The vehicle shall send an initial status on connection establishment, after owner pairing, after Friend First Approach and always send status updates to connected devices when the vehicle status has changed. The explicit status request shall only be executed on explicit user intent or for device error recovery. Afterward, changes shall be tracked via the function status updates send out on status change.

When entering low power mode, the vehicle shall send Vehicle Status Changed SubEvent indicating it is in low power mode to all connected devices. If the device has received a Vehicle Status Changed SubEvent indicating the vehicle is in a low power mode, the device shall not send a RKE Request SubEvent for status request updates except for device error recovery. After exiting lower power mode, the vehicle shall send a status update for all connected devices.

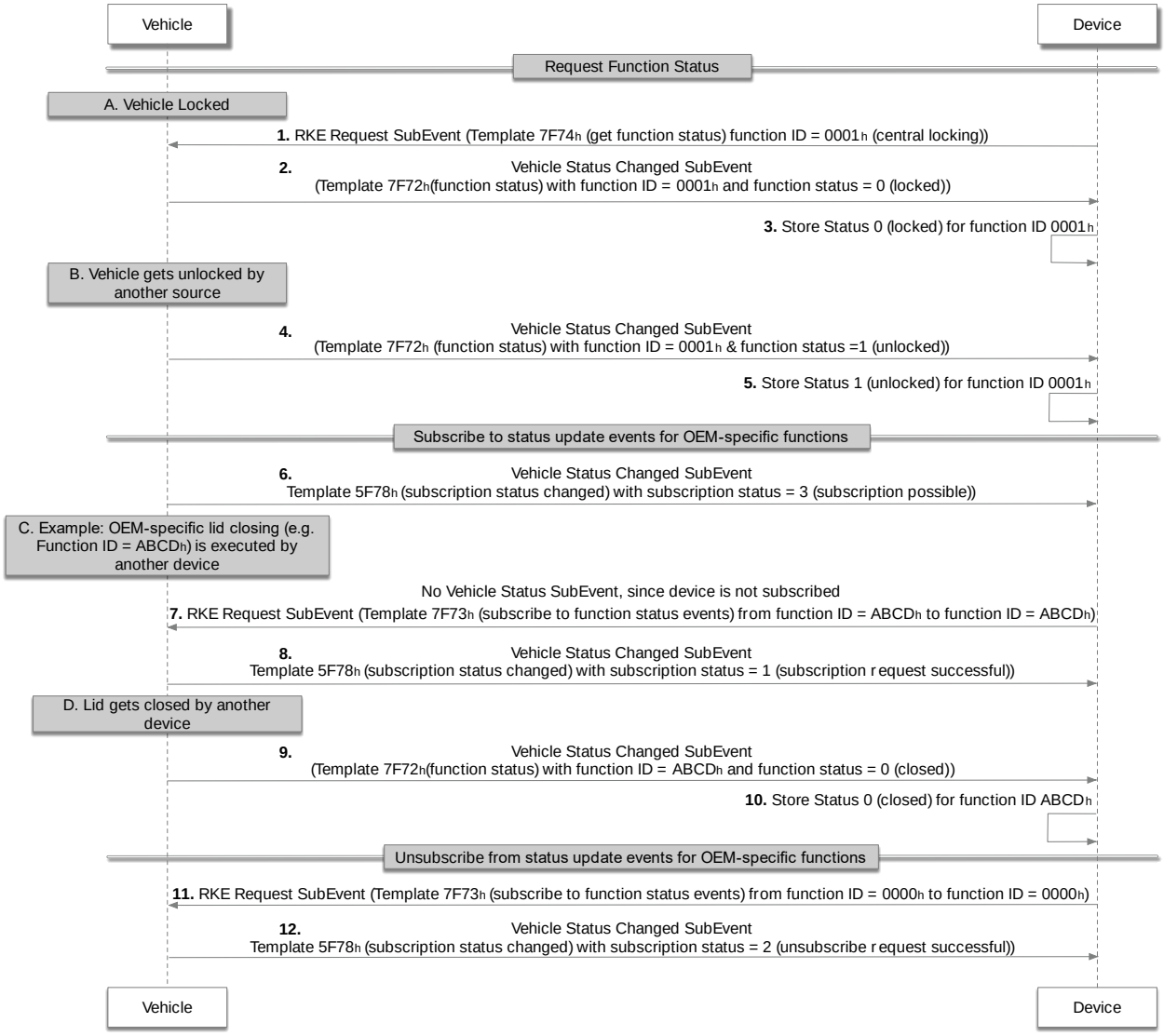
If the vehicle is in low power mode, the status request shall not be sent except for device error recovery. After establishing the Bluetooth LE connection, the device may subscribe to status updates, only when the vehicle indicates first the availability of the subscription mechanism (see message in step 6 of Figure 19-34). The vehicle shall indicate when it stops the automatic status updates for the subscribed ranges (e.g., due to entering low power mode) and when a resubscription is possible again.

When a function is not supported by the vehicle, the vehicle shall only send a status update informing about the unavailability (cf. Table 19-80, status $F0_h$), when the status is explicitly requested by the device ("get function status").

If the Bluetooth LE connection between the device and the vehicle is available and the subscription to a certain function id range is active, the framework shall store the latest received function status values for this range of function ids. The device shall provide an API for eligible vehicle OEM apps for subscribing to a certain function id range, requesting a certain function status value from the vehicle and reading the stored status data.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-34: Vehicle Function Status Retrieval and Event-based Update.



19.5.10 *Bluetooth LE Activation Flow*

If a user goes through NFC based owner pairing or friend first approach even though both device and vehicle support Bluetooth LE/UWB capability, DK solution allows the user to migrate NFC provisioned key to Bluetooth LE/UWB based passive entry experience without user having to perform owner pairing or key sharing again.

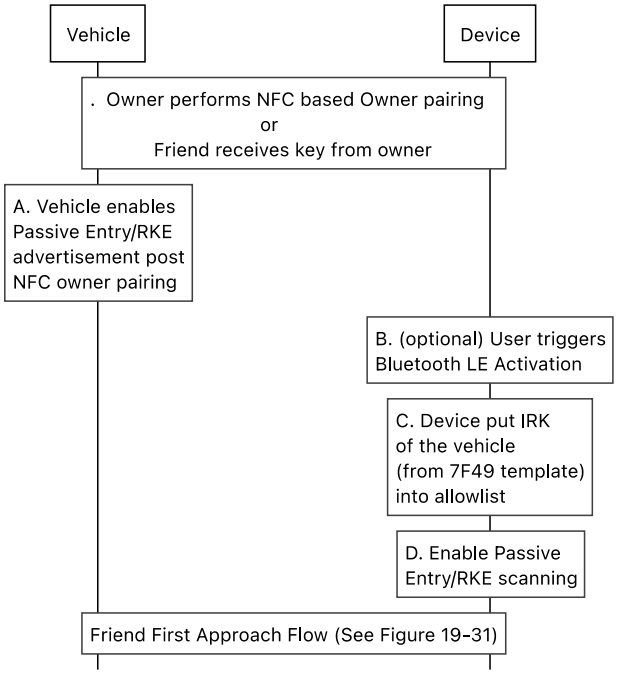
If a user goes through NFC based owner pairing or friend first approach for a DK system which supports Bluetooth LE without UWB, DK solution allows the user to migrate NFC provisioned key to Bluetooth LE based RKE experience without user having to perform owner pairing or key sharing again

This NFC to Bluetooth LE migration may be triggered by user anytime even when the vehicle is not within the Bluetooth LE proximity. Upon first approach after user triggered migration, device and vehicle shall establish Bluetooth LE bonding and offer passive entry or RKE experience for that approach.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-35 illustrates Bluetooth LE Activation.

Figure 19-35: First Approach Activation.



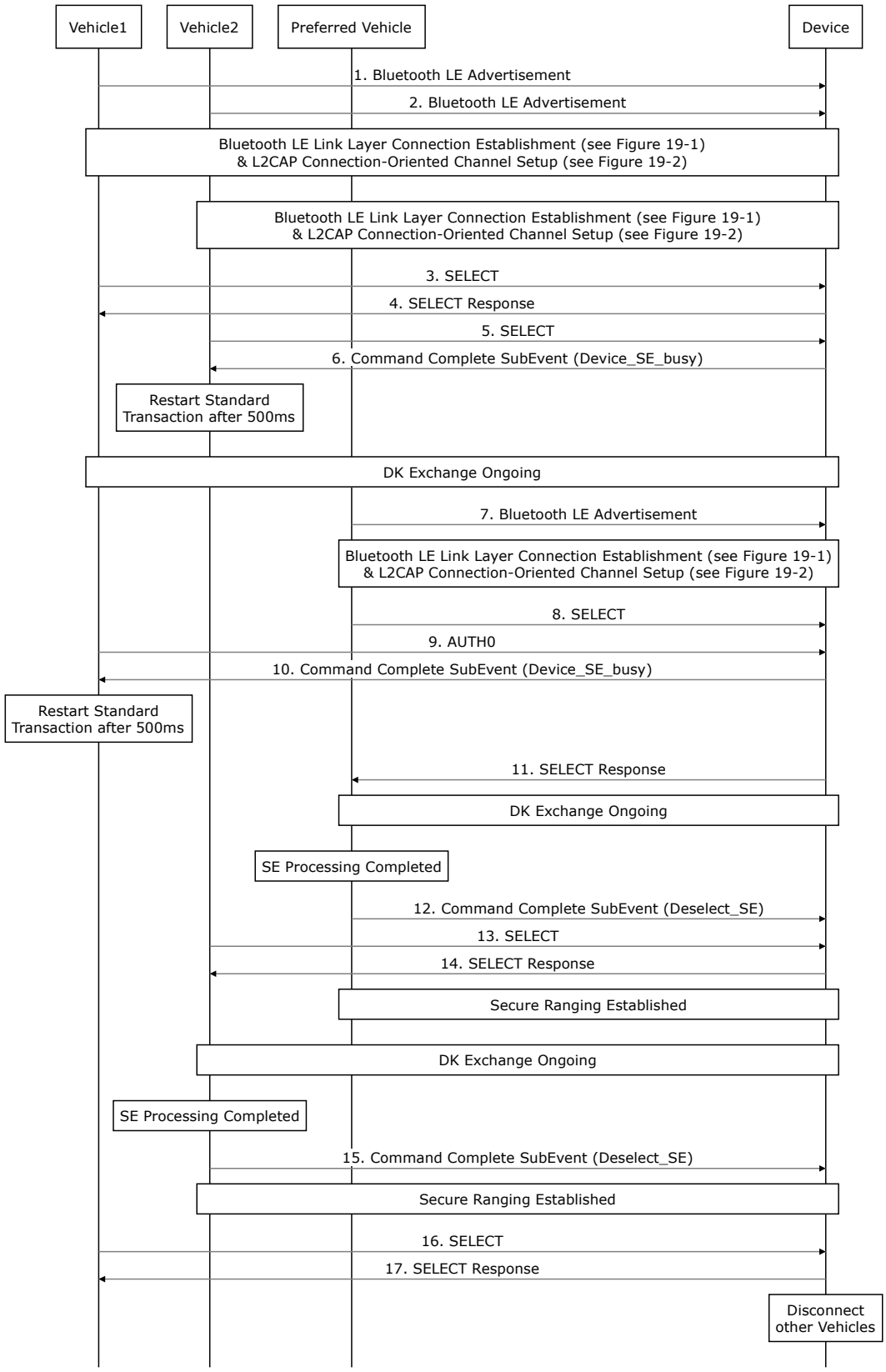
19.6 Digital Key - Preference Management

19.6.1 *Preference Management: Connected Vehicles < Connection Limit*

A user with multiple Digital Keys can mark any one of the Digital Keys as preferred or default. When such user approaches multiple valid vehicles, the device shall prioritize the SE access for vehicle with the default Digital Key and put the other vehicle's SE request on hold by sending ‘Device_SE_busy’, if they compete for resources with the preferred Digital Key. When vehicle receives ‘Device_SE_busy’ SubEvent, it shall wait for a defined wait time (e.g. two seconds) before restarting the standard transaction again. Once the device has completed SE processing with the preferred vehicle, it shall continue with the rest of the vehicles. Figure 19-36 provides an example flow for device preference management.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-36: Device Preference Management.



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.6.2 *Preference Management: Connected Vehicles >= Connection Limit*

When a device is already connected to its max limit Bluetooth LE connections and user approaches (within Bluetooth LE range) additional vehicle for which it has a valid Digital Key with the intent to access it, user may go through one of the following two scenarios:

1. New Vehicle is Preferred
2. New Vehicle is Non-Preferred

For scenario #1, when the device detects advertisement from a new vehicle which is configured as preferred vehicle for the device, it may drop one of the connected vehicles to prioritize the Bluetooth LE connection for the preferred vehicle.

For scenario #2, when the device detects advertisement from a new non-preferred vehicle, it shall drop one of the non-preferred connected vehicles (which connection to drop is device OEM's implementation) to prioritize the Bluetooth LE connection for the new vehicle

When there are more vehicles than device's max Bluetooth LE connection limit, user shall be able to exercise passive entry experience with any of the connected vehicle. However, for non-connected, non-preferred vehicle, user may have to fallback to RKE feature.

To exercise RKE feature in this scenario, user selects the right vehicle and requests RKE feature. This shall prioritize the non-connected vehicle for Bluetooth LE connection for user to gain access. Once connected and unlocked, vehicle should try to establish secure ranging with newly connected device to move the user to passive entry experience and for device in/out detection for engine start

19.7 SubEvent Handling

19.7.1 *Command Complete SubEvent Code Handling*

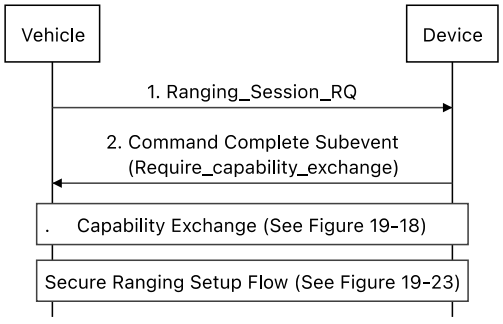
This section defines how vehicle shall recover from a SubEvent defined in Section 19.3.8.

19.7.1.1 Require Capability Exchange

Ranging capabilities are first negotiated between device and vehicle as part of the owner pairing flow (See Section 19.5.1) using Ranging_Capability_RQ and Ranging_Capability_RS.

Figure 19-37 provides an example flow for handling Require Capability Exchange SubEvent.

Figure 19-37: Required Capability Exchanged.



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.7.2 *Ranging Session Status Changed SubEvent*

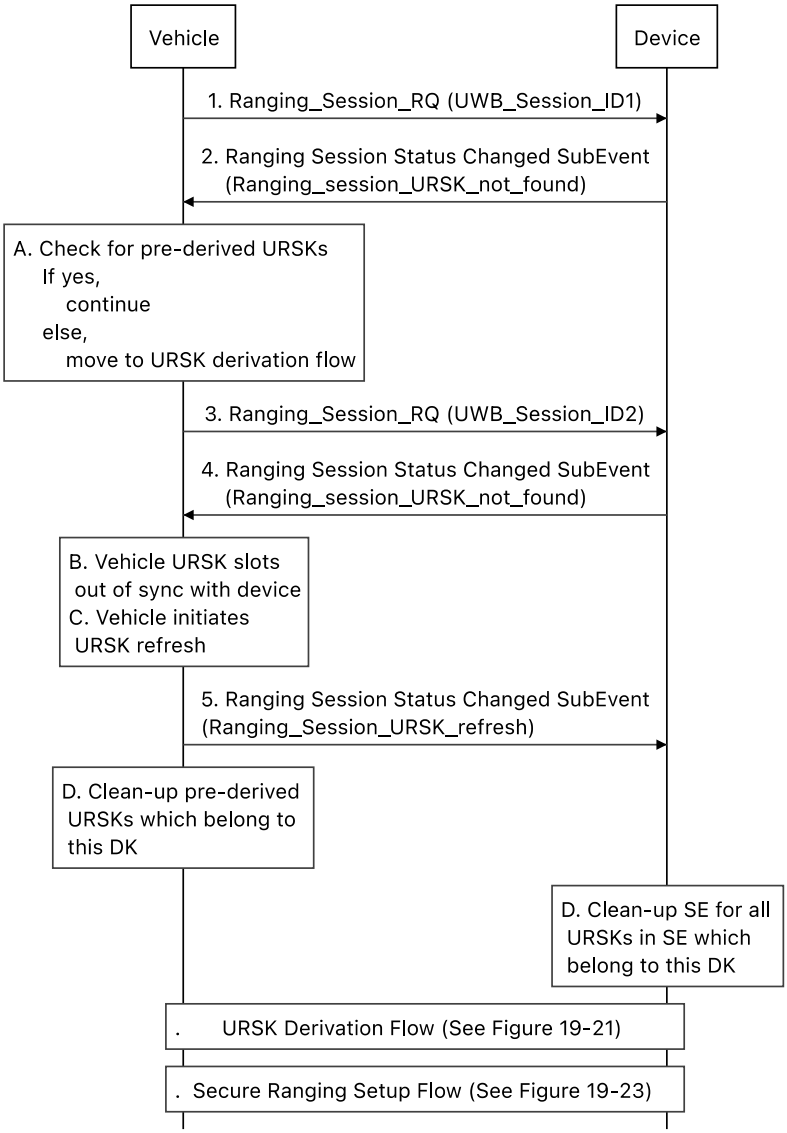
19.7.2.1 URSK Not Found

When a device fails to retrieve the associated URSK for a requested UWB_Session_Id, it shall respond with a ‘Ranging Session Status Changed SubEvent’ with parameter value “URSK_not_found”.

The vehicle should clean up the pre-derived URSKs on the device if it fails to retrieve URSKs repeatedly.

Figure 19-38 provides an example flow for URSK Retrieval Failure handling.

Figure 19-38: URSK Not Found.



For box A, vehicle shall query for stored URSKs.

For box B, it is vehicle’s decision on when to initiate URSK Refresh flow.

For box D, vehicle should delete URSKs associated with the digital key.

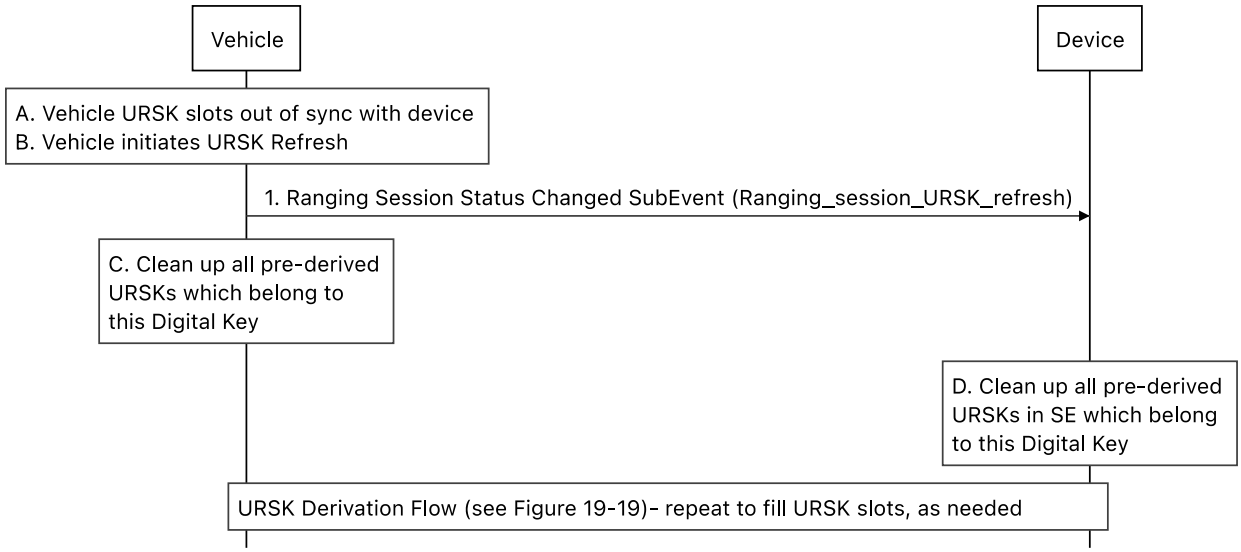
Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

19.7.2.2 URSK Refresh

The vehicle may trigger a deletion of all pre-derived URSKs in the device's SE at any time by sending a Ranging Session Status Changed SubEvent Notification with Session_Status "Ranging_session_URSK_refresh". This method shall not affect the URSK used in the active ranging session.

Figure 19-39 provides an example flow for URSK Refresh flow.

Figure 19-39 URSK Refresh Flow



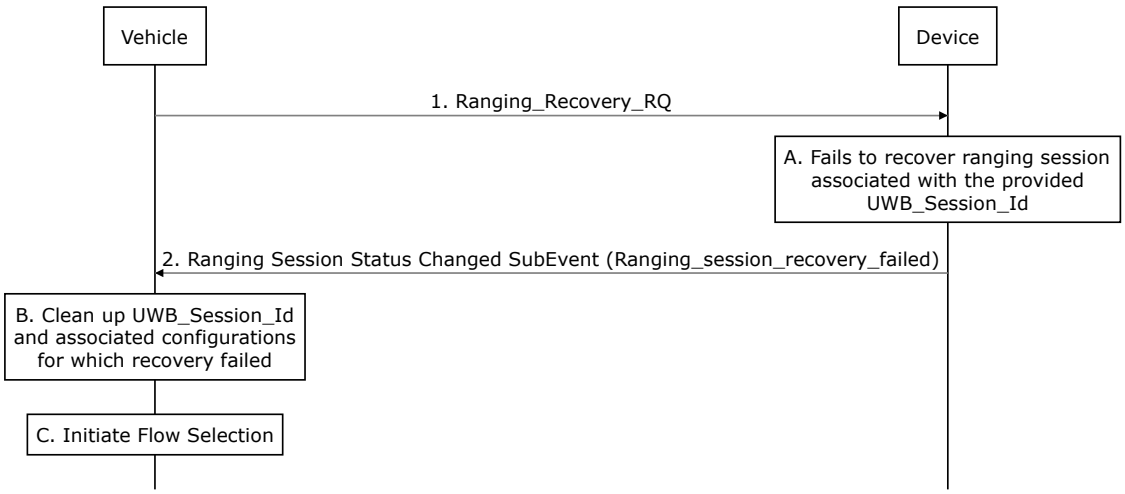
19.7.2.3 Recovery Failed

When the device fails to recover a ranging session, which was requested by vehicle, the device shall respond to the vehicle with Ranging Session Status Changed SubEvent Notification with Session_Status "Ranging_session_recovery_failed". Upon receiving this notification, the vehicle shall discard the corresponding URSK. A new ranging session should then be established according to the flow selection (see Figure 19-22).

Figure 19-40 provides an example flow for "Recovery Failed" handling.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-40: Recovery Failed Flow.



For box B, process the received message, the vehicle removes the URSK, configurations and any other metadata associated with UWB_Session_Id for which recovery failed.

For box C, refer to Section 19.5.5.

19.8 Error Handling

19.8.1 *SE Transaction Recovery*

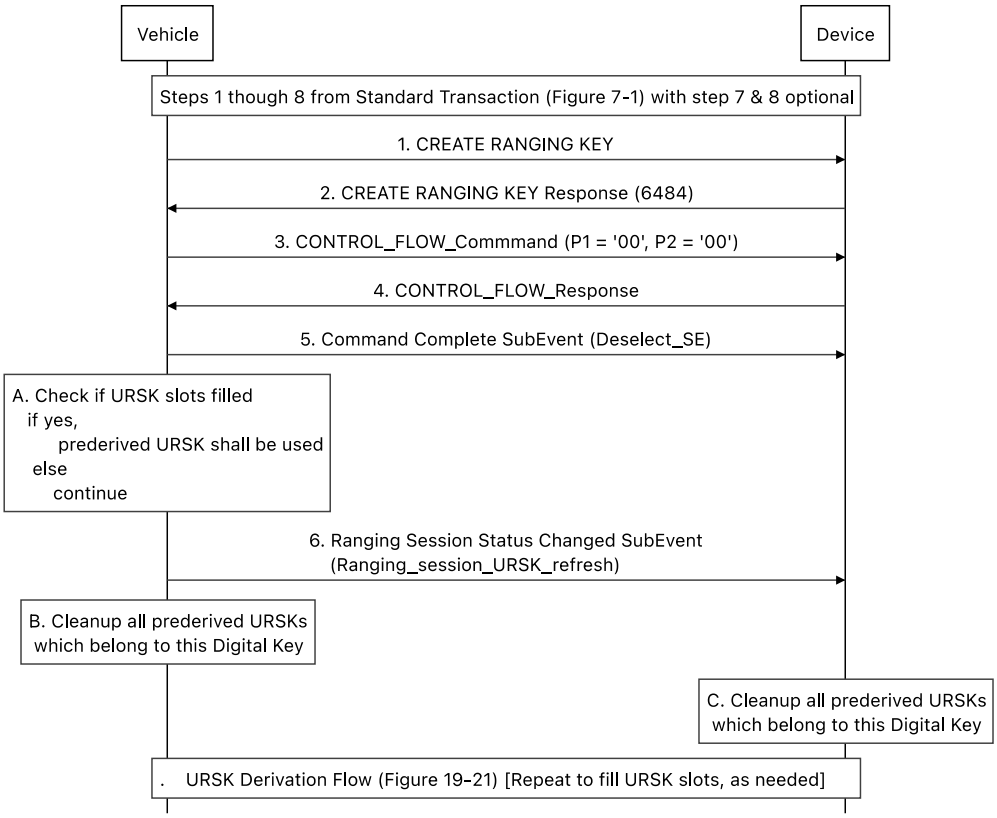
19.8.1.1 CREATE RANGING KEY Failure

If device URSK slots for a given Digital Key are filled, it shall fail to store any new key. In this event, the device shall respond to CREATE RANGING KEY command with Status Word 6484_h, which means ‘URSK slots have been exhausted’. Upon receiving status word 6484_h, vehicle shall not request new URSK derivation unless it requests “URSK_refresh” as outlined in Figure 19-41 first.

If vehicle identifies that device and vehicle are out of sync with pre-derived URSKs, vehicle may go through URSK Refresh flow (see Figure 19-39). Figure 19-41 provides an example flow for Create Ranging Key Failure.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 19-41: URSK Refresh due to Status Word 6484_h in CREATE RANGING KEY Response.



For box A, vehicle shall query for pre-derived URSKs.

For box B, vehicle shall delete all pre-derived URSKs associated to the Digital Key.

For box C, device shall delete all pre-derived URSKs associated to the Digital Key.

19.8.1.2 Others

Error handling for rest of the applet commands shall be done based on the Status Words and flows defined in Section 15.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

20 UWB MAC AND CHANNEL ACCESS [WCC3]

This section defines the MAC layer and channel access protocols for UWB ranging for Digital Key.

20.1 UWB MAC Architecture

20.1.1 *Overview*

This subsection describes the concepts that are used in the UWB MAC (see [31] and [33]).

20.1.1.1 Ranging Roles Definition

In the DK UWB ranging service, the ranging device roles are based on which device starts the ranging procedure and is responsible for setting the ranging exchange. The following role definitions apply to UWB layer only:

1. An entity that starts the UWB ranging packet exchange by sending a first UWB POLL packet is called an “initiator.” In the case of DK this is the device.
2. An entity that responds to a UWB POLL packet is called a “responder”. In the case of DK, these are the anchors on the vehicle.
3. An entity that includes a number of responders is called a “responder-device”. In the case of DK, this is the vehicle.
4. An entity that controls the ranging procedure by sending a Pre-POLL packet is called the controller. In the case of DK, this is the device, i.e. the device is the initiator and the controller

Note that the roles definition above may not apply to the Bluetooth LE layer.

20.1.1.2 Logical and Physical Responders

1. A responder can be a logical responder as well as a physical one.
2. A physical responder shall have one UWB module and one or more physical antennas.
3. A logical responder corresponds to one responder role which, for example, could be assigned to a specific UWB module and one specific physical antenna. Hence, a physical responder may include one or more logical responders.
4. The responder-device shall coordinate which logical responders transmit and in which order such that there is no interference between transmissions from two logical responders belonging to the same responder-device.

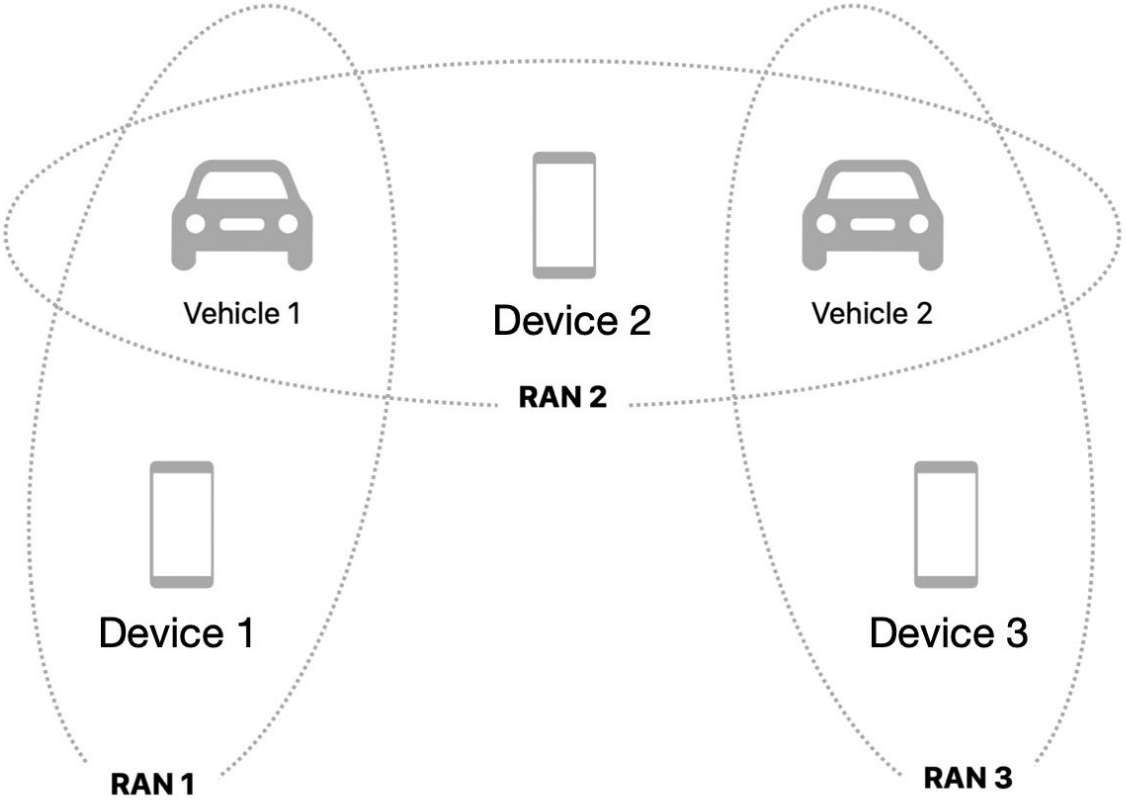
20.1.1.3 DK Ranging Area Network (RAN)

1. An initiator and a responder-device engaged in a continuous ranging procedure that is characterized by a specific set of parameters is called a ranging session.
2. An initiator and its (one or more) ranging sessions are called a “ranging area network (RAN).” Each RAN is characterized by a timing reference that is established by the initiator. All responder-devices in the same RAN approximate the initiator timeline (there is no assumption of global synchronization here, see Section 20.3).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

3. A single RAN shall have one initiator only and may have more than one responder-device (e.g. a device ranging with two vehicles). In the example shown in Figure 20-1, this is represented by Device 2, Vehicle 1, and Vehicle 2 which all belong to RAN 2.
4. Each responder-device may have a different number of logical responders (e.g. one vehicle may have 7 responders and another vehicle in the same RAN may have 5 responders).
5. A single responder-device may belong to multiple RANs, e.g. a single vehicle ranging with two different devices. In the example shown in Figure 20-1, this is represented by Vehicle 2 which belongs to RAN 2 controlled by Device 2 and RAN 3 controlled by Device 3.

Figure 20-1: Digital Key RAN Concept.



20.1.1.4 Inter-/Intra- RAN Interference and Resource Management

Each responder on a responder-device can reliably predict its allowed transmission window and there will be no interference between responders on the same responder-device. However, given the above definitions, three possible performance degradation scenarios can be identified:

1. Inter-RAN Interference:

- This may be caused by actual over-the-air collisions of packets from different RANs.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- This is the normal mode of operation and should be expected as there is no assumption of coordination between different RANs. The impact of these collisions may be mitigated by employing the ranging round hopping strategy defined below (see Section 20.4).

2. Intra-RAN Resource Conflict:

- A resource conflict may occur when the initiator would have to serve two different ranging exchanges (to two different responder-devices) at the same time.
- This scenario may be resolved at the initiator by prioritizing one of the ranging sessions involved. The target ranging round may be used for the ranging session with the highest priority and the initiator shall skip the round for the other ranging sessions. The impact of skipped ranging round appears like a failed ranging round and may be mitigated by employing the ranging round hopping strategy explained in Section 20.4.

3. Inter-RAN Resource Conflict:

- A resource conflict occurs when a responder would have to serve ranging rounds from two (or more) different RANs at the same time.
- This scenario may be resolved by prioritizing one RAN and skipping the round for the other RANs. The prioritization is determined by the involved responder-device.
- For the initiators of the skipped RANs, this appears like a failed reception of the responses for the current ranging exchange and the responder-device hops to a different round (see Section 20.4).

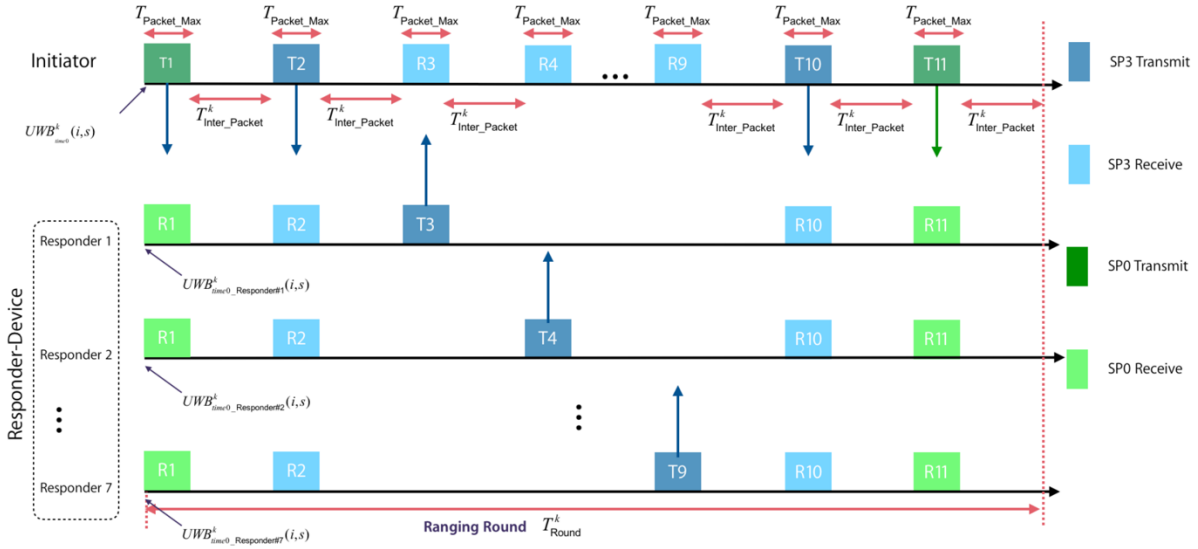
The criteria for determining the priority is left to the device and the vehicle manufacturer. In all subsequent sections and text below, unless otherwise stated, a responder is understood to mean a logical responder.

20.2 MAC Time Grid

The DK UWB ranging protocol is a one-to-many (O2M) ranging protocol between the initiator and the responder-device. Figure 20-2 shows an example for the packet exchange in the O2M ranging protocol for an initiator ranging with a responder-device with seven responders. The symbols used in the figure are defined in subsequent text.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 20-2: Digital Key One-to-Many Ranging Protocol.



The DK UWB ranging protocol is a block-based ranging approach as described in [33]. The protocol uses a ranging block structure where each ranging block is T^k_{Block} in time duration (it is explained later in this section how the block timing is established within the RAN). Each ranging block is divided into N^k_{Round} ranging rounds that are long enough to carry out one complete ranging exchange. The length T^k_{Round} of the ranging rounds is session specific.

- First, the following global parameters are defined:
  - T_{Chap}: Unit of time for the MAC protocol. All durations (except for $T_{\text{Packet_Max}}$ defined below) of the MAC protocol are integer multiples of T_{Chap}.

$$T_{\text{Chap}} = \frac{1}{3}\text{ms} = 400 \text{ RSTU}$$

    where the RSTU is defined in [33] as $416/(499.2\ \text{MHz}) \approx 833.33$ ns.

  - $T_{\text{Block_Min}}$: Minimum ranging block duration, which is set to 96ms, is an integer multiple of T_{Chap}

$$T(\text{Block_Min}) = 288 \times T_\text{Chap} = 96\text{ms}$$

- One RAN consists of one initiator and *k = 1, …, K* responder-devices.
- Each responder-device is assigned at most one active ranging session in the RAN.
- The *k*-th ranging session (which is associated with the *k*-th responder-device) is identified with a unique $UWB_Session_ID^k$ that is assigned during the negotiation phase.
- The *k*-th ranging session is defined relative to a specific time reference UWB^k_{time0}. This time reference is defined by the initiator with respect to its clock base. This time reference defines the beginning of a series of consecutive ranging blocks. Given that

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

there is no assumption of global synchronization, Section 20.3 describes how that time reference is established at the responders.

- The length of each ranging block for the *k*-th session is given by:

$$T^k_{\text{Block}} = N^k_{\text{RAN}} \times T_{\text{Block_Min}}$$

-

where N^k_{RAN} is the RAN Multiplier. It is a session specific parameter that is used to control the maximum ranging frequency of the corresponding session in a given RAN (see Section 20.5.2 and G.1). It is set during the negotiation phase between the initiator and the responder-device. Every ranging session shall have a ranging block duration that is greater than or equal to $T_{\text{Block_Min}}$.

- The (*i*+1)-th ranging block of the *k*-th ranging session starts at:

$$UWB^k_{time0} + i \times T^k_{\text{Block}} \qquad i = 0,1,2,\ldots$$

- Each block is divided into a number of slots. The length of each slot T^k_{Slot} shall be greater than or equal to the sum of the maximum UWB packet transmission time and the maximum processing time required by the UWB receiver to process such a packet (these are indicated by $T_{\text{Packet_MAX}}$ and $T^k_{\text{Inter_Packet}}$ in Figure 20-2)[6]. The duration of the slot in terms of T_{Chap} is expressed as:

$$T^k_{\text{Slot}} = N^k_{\text{Chap_per_Slot}} \times T_{\text{Chap}}$$

where $N^k_{\text{Chap_per_Slot}}$ is determined during the negotiation phase (see Section 20.5.2).

- A sequence of $N^k_{\text{Slot_per_Round}}$ consecutive slots constitutes a ranging round of length T^k_{Round}. The number of slots per ranging round shall be greater than or equal to the number of packet exchanges required for one complete ranging exchange between the initiator and the responder-device.

$$T^k_{\text{Round}} = N^k_{\text{Slot_per_round}} \times T^k_{\text{Slot}} = N^k_{\text{Slot_per_round}} \times N^k_{\text{Chap_per_Slot}} \times T_{\text{Chap}}$$

- Each block of the *k*-th ranging session is divided into N^k_{Round} consecutive ranging rounds, where

$$N^k_{\text{Round}} = \frac{T^k_{\text{Block}}}{T^k_{\text{Round}}}$$

[6] Given that the initiator and the responder-device may be using two different UWB radios with different processing capabilities, the two radios may support different values of slot durations. The duration of the appropriate slot is selected from the set of common values during the negotiation phase between the initiator and the responder-device according to the protocol described in Section 20.5.2.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- The (s+1)-th ranging round in the (i+1)-th ranging block of the k-th ranging session shall start at

$$UWB_{time0}^{k} + i \times T_{\text{Block}}^{k} + s \times T_{\text{Round}}^{k} \quad i = 0,1,2,3 \ldots \quad s = 0,1,2, \ldots, (N_{\text{Round}}^{k} - 1)$$

- Each of the ranging sessions (k=1,2, …, K) is assigned a pseudo-random hopping sequence

$$H^{k} = \left\{ S_0^k, S_1^k, \cdots, S_i^k \right\} \quad \text{with} \quad 0 \le S_i^k \le \left(N_{\text{Round}}^{k} - 1 \right)$$

where S_i^k denotes the index of the ranging round in ranging block i that starts at

$$UWB_{time0}^{k} + i \times T_{\text{Block}}^{k} + S_i^k \times T_{\text{Round}}^{k} \quad i = 0,1,2,3 \ldots \; and \; 0 \le S_i^k \le (N_{\text{Round}}^{k} - 1)$$

S_i^k also referred to as $Round_Idx^k(i)$ in later sections of this document to denote the ranging round index of (i+1)-th ranging block selected pursuant to the hopping configuration. The selection of the hopping configuration, i.e. adaptive or continuous, and hopping sequence will be made according to the selection made during the ranging session setup described below. Additionally, Appendix G.2 shows the computation for the hopping sequence H^k. The hopping sequence shall be used, along with the hopping mode and the hopping flag to compute the ranging round index in the current ranging block. See Section 20.4 below for details on the hopping flag and ranging round index determination.

For details on the ranging packet exchange initiation process and the hopping configuration selection, see Section 20.5.2.

- Given the above definitions, the average ranging frequency of the k-th ranging session is

$$f_{\text{Ranging}}^{k} = \frac{1}{T_{\text{Block}}^{k}} \text{£} \frac{1}{96\text{ms}} \text{@} 10.4166667\,\text{Hz}$$

The achievable ranging block durations $T_{\text{Block}}^{\text{k}}$ for a given N_{RAN}^{k} is $N_{\text{RAN}}^{k} \times 96ms$.

- For the (i+1)-th ranging measurement in the k-th ranging session, the ranging round with index $Round_Idx^k(i)$ is divided into $N_{\text{Slot_per_Round}}^{k}$ consecutive slots of equal length

$$T_{\text{Slot}}^{k} = N_{\text{Chap_per_Slot}}^{k} \times T_{\text{Chap}}.$$

The slot with index m shall start at time

$$UWB_{time}^{k} + i \times T_{\text{Block}}^{k} + Round_Idx^{k}(i) \times T_{\text{Round}}^{k} + m \times T_{\text{Slot}}^{k}$$

$$i = 0,1,2,3, \ldots \quad 0 \le \text{Round}_\text{I}\text{dx}^{k}(i) \le (N_{\text{Round}}^{k} - 1) \quad m = 0,1,2, \ldots, (N_{\text{Slot_per_Round}}^{k} - 1)$$

- The first N_{Packets}^{k} slots (out of $N_{\text{Slot_per_Round}}^{k}$) of the ranging round with index $Round_Idx^k(i)$ (slots $0,1,2,\cdots,N_{\text{Packets}}^{k}-1$) shall be mapped by the initiator and

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

responder-device to transmit and receive UWB ranging packets (see the example in Table 20-2). The remaining $N^k_{\text{Slot_per_Round}}$ - N^k_{Packets} slots shall be left unmapped.

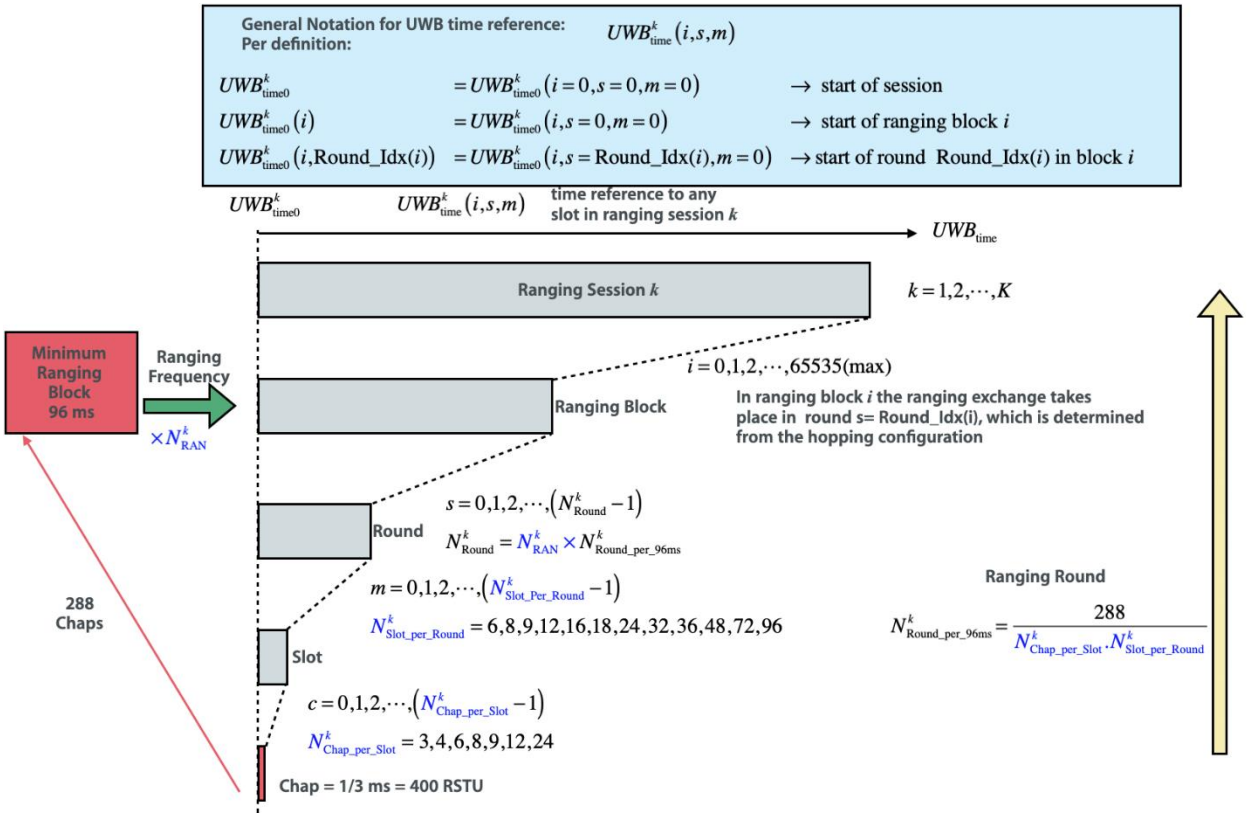
- The initiator and the responder-device shall send the ranging packets as close as technically possible to the beginning of the mapped slots.
- Neither the initiator nor the responder-device shall start transmission before the beginning of a slot.
- If a resource conflict between two (or more) ranging sessions in the same RAN occurs, the initiator shall resolve the conflict by prioritization.

The MAC grid described above is illustrated in Figure 20-3. In this figure and throughout this section, the following notation is used:

$UWB^k_{time0}(i,s,m)$ is the UWB time reference for the (m+1)-th slot of the (s+1)-th ranging round of the (i+1)-th ranging block in the k-th ranging session.

$$UWB^k_{time0} = UWB^k_{time0}(i=0,s=0,m=0) \rightarrow \text{start of session}$$
$$UWB^k_{time0}(i) = UWB^k_{time0}(i,s=0,m=0) \rightarrow \text{start of ranging block } i$$
$$UWB^k_{time0}(i,s) = UWB^k_{time0}(i,s,m=0) \rightarrow \text{start of ranging round } s \text{ in block } i$$

Figure 20-3: Overview of MAC Grid (See Table G- 1).



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 20-1: Example MAC Configuration.

Parameter	Vehicle 1 (k=1)	Vehicle 2 (k=2)
N^k_{RAN}	3	2
$N^k_{\text{Chap_per_Slot}}$	6	3
$N^k_{\text{Responder}}$	2	3
T^k_{Block}	288 ms	192 ms
T^k_{Round}	12 ms	8 ms
N^k_{Round}	24	24
f^k_{Ranging}	3.47 Hz	5.21 Hz

20.3 MAC Time Grid Synchronization

As stated above, each ranging session is defined relative to a specific time reference UWB^k_{time0}, which is defined relative to the initiator's internal clock. The time reference UWB^k_{time0} establishes the MAC time grid for the *k*-th ranging session.

Given that there is no assumption of tight clock-level synchronization between the initiator and the responder-device, the MAC time grid shall be estimated, with reasonable accuracy, at each responder (at the responder-device). This allows each responder (at the responder-device) to determine when to expect to receive the Pre-Poll message from the initiator, when to send its response message back to the initiator, and when to expect to receive the Final and Final_Data messages from the initiator.

The approximate MAC time grid is established at each responder as follows:

- During the ranging session negotiation, the initiator shall send the value of UWB^k_{time0} to the responder-device that is to be associated with the *k*-th ranging session.
- Each responder may approximate the MAC time grid of the initiator by:
  - The responder uses a dedicated out-of-band (OOB) clock synchronization (e.g. via the Bluetooth LE control channel). By using an OOB clock synchronization method, the initiator can inform the responder-device when to expect the ranging session to start, i.e. the approximate MAC grid time reference $UWB^k_{time0_\text{Responder\#}l}$ for the session for the (l+1)-th responder, l=0,1,2, …, $N^k_{\text{Responder}} - 1$. The responders then turn on their UWB receivers and start looking for the first UWB packet.
  - Upon receiving UWB packets, the responders further use the incoming UWB packets from the initiator to continuously refine the estimation of the MAC time grid under the assumption that each incoming packet transmission was not started by the initiator before the beginning of its dedicated slot.
  - The time-of-flight (ToF) calculations shall remain unaffected from the MAC grid timings.

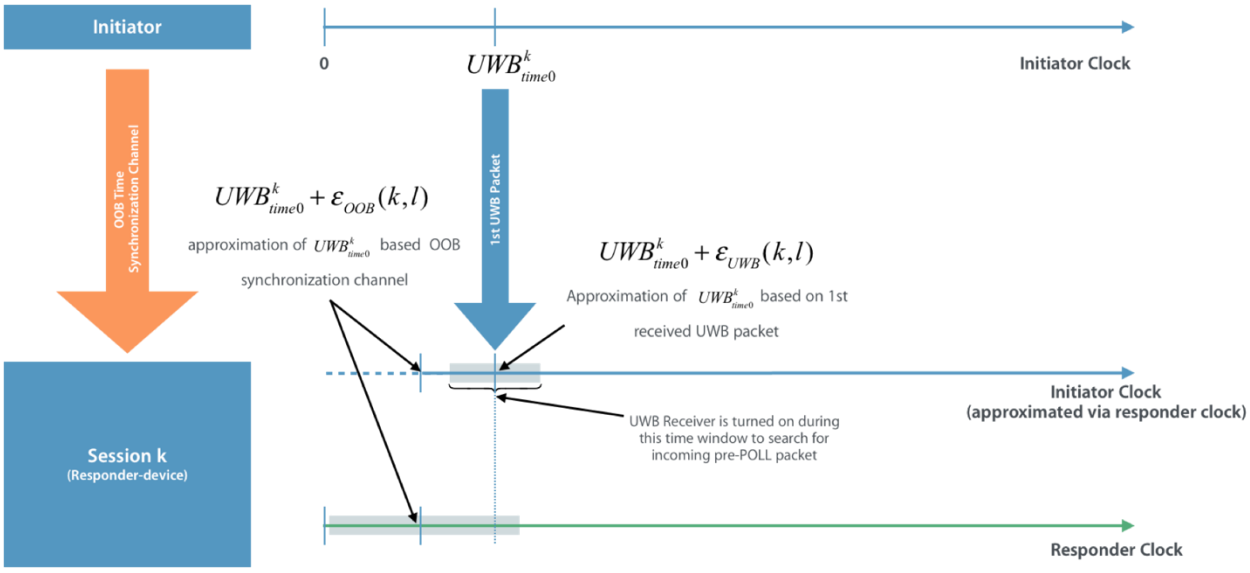
Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

For the *k*-th ranging session, the approximate MAC grid and time reference established at the *l*-th responder is denoted by $UWB^{k}_{time0_Responder\#l}$ which is related to the initiator time reference by

$$UWB^{k}_{time0_Responder\#l} = UWB^{k}_{time0} + e(k,l).$$

The error $e(k,l)$ depends on whether an incoming UWB packet ($e(k,l) = e_{UWB}(k,l)$) or an OOB clock synchronization protocol ($e(k,l) = e_{OOB}(k,l)$) is used to approximate the MAC time grid. Characterization of this error and techniques to minimize it are out of scope for this specification. A high-level description of the block timing synchronization process is shown in Figure 20-4.

Figure 20-4: Overview of Block Synchronization Approach.



20.4 Hopping Flag and Round Index Determination

As stated above, the initiator in the *k*-th ranging session in any given RAN shall start the UWB ranging procedure in the first ranging round (ranging round 0) in the first ranging block (ranging block 0) by default. This assumes that the responder-device has either achieved block synchronization by OOB method or, if not, is permanently listening for Pre-Poll messages.

At the initiator, and assuming no resource conflict occurs, the $Hop_Flag^{k}(i+1)$ and $Round_Idx^{k}(i+1)$ for the next ranging block (ranging block *i*+1) will depend on the hopping mode selected during ranging session setup as follows:

- If the hopping mode is set to "no hopping," then the initiator shall continue to use the same ranging round in ranging block *i* and $Round_Idx^{k}(i+1)$ is set as:

$$Round_Idx^{k}(i+1) = Round_Idx^{k}(i) = 0$$

  and $Hop_Flag^{k}(i+1)$ is set to 0. Note that in this case, $Hop_Flag^{k}(i+1)$ is irrelevant to the receiver and shall be ignored.
- If the hopping mode is set to "continuous hopping," the initiator uses the S^{k}_{i+1}-th ranging round

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

$$Round_Idx^{k}(i+1) = S_{i+1}^{k}$$

and $Hop_Flag^{k}(i+1)$ is set to 1. Again, $Hop_Flag^{k}(i+1)$ is irrelevant to the receiver and shall be ignored.

- If the hopping mode is set to "adaptive hopping," then at the initiator, the $Hop_Flag^{k}(i+1)$ and $Round_Idx^{k}(i+1)$ for the next ranging block (ranging block i+1) shall be set as follows:
  - If the initiator determines that the round is clean, i.e. no interference and ranging in the current round is successful, the initiator shall stay in the current round and set as:

    $$Hop_Flag^{k}(i+1) = 0 \quad \text{and} \quad Round_Idx^{k}(i+1) = Round_Idx^{k}(i)$$

  - If the initiator determines that there is some interference on the current round or if the ranging is not successful, the initiator shall hop to a different round:

    $$Hop_Flag^{k}(i+1) = 1 \quad \text{and} \quad Round_Idx^{k}(i+1) = S_{i+1}^{k}$$

The initiator shall send the $Hop_Flag^{k}(i+1)$ and $Round_Idx^{k}(i+1)$ for next ranging block to the responder-device as part of the payload packet carrying time stamps Final_Data at the end of the ranging sequence.

At the responder-device, the $Hop_Flag^{k}(i+1)$ and $Round_Idx^{k}(i+1)$ for subsequent ranging rounds shall be set as follows:

if *Final_Data* packet is received
{
$Hop_Flag^{k}(i+1) = Final_Data.Hop_Flag$
if ($Hop_Flag^{k}(i+1) = 0$)
{
set $Round_Idx^{k}(i+1) = Round_Idx^{k}(i)$
}
elseif ($Hop_Flag^{k}(i+1) = 1$)
{
set $Round_Idx^{k}(i+1) = S_{i+1}^{k}$
}
}
else
{
set $Round_Idx^{k}(i+1) = S_{i+1}^{k}$.
}

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The initiator may decide to trigger the hopping (i.e. setting Hop_Flag=1) based on the interference level or packet collision rate that it measures over the current exchange and/or the number of responses that it receives back from the responder-device. In general, the exact criteria for triggering hopping at the initiator is out of scope of this specification and is left to each device manufacturer.

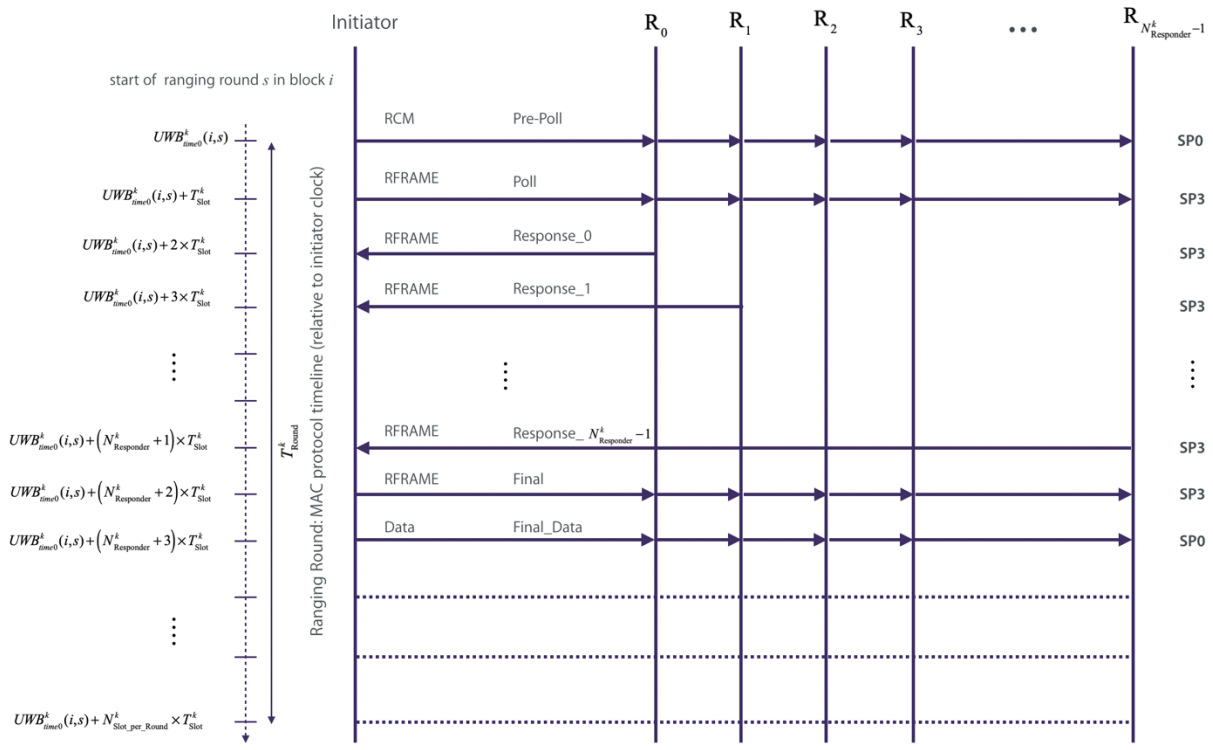
Note: The only requirement in any such criteria is that if the device does not receive any responses from the responder-device, the device shall trigger hopping.

20.5 MAC Protocol

20.5.1 *Ranging Exchange Sequence*

This section details the DK MAC protocol for a double-sided, two-way ranging with a three-packet exchange between the initiator and each responder of the responder-device. Figure 20-5 shows the UWB message flow for the MAC protocol between an initiator and $N^k_{\text{Responder}}$ responders in the (s+1)-th ranging round in the (i+1)-th ranging block with $N^k_{\text{Slot_per_Round}}$ slots. While the figure shows the MAC protocol according to the MAC timeline relative to the initiator clock and time reference, this protocol shall be carried out according to the respective responder-device's time reference as described in Section 20.3. Table 20-2 shows the mapping of the $N^k_{\text{Responder}}$ UWB ranging packets to the $N^k_{\text{Slot_per_Round}}$ slots.

Figure 20-5: Example of UWB Message Flow for MAC Protocol.



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 20-2: Mapping of Slots to UWB Ranging Packets.

Slot Index	Mnemonic	STS Packet Format	Sender
0	Pre-POLL	SP0 : Data Packet	Initiator
1	POLL	SP3 : RFRAME	Initiator
1+1	Response_0	SP3 : RFRAME	Responder
1+2	Response_1	SP3 : RFRAME	Responder
…	…	SP3 : RFRAME	Responder
$1+ N^k_{\text{Responder}}$	Response_ $N^k_{\text{Responder}}$ - 1	SP3 : RFRAME	Responder
$1+ N^k_{\text{Responder}}$ +1	Final	SP3 : RFRAME	Initiator
$1+ N^k_{\text{Responder}}$ +2= N^k_{Packets}	Final_Data	SP0 : Data Packet	Initiator
$1+ N^k_{\text{Responder}}$ +3	Slot not used	N/A	N/A
…	Slot not used	N/A	N/A
$N^k_{\text{Slot_per_Round}}$ -1	Slot not used	N/A	N/A

The MAC protocol sequence under the normal mode of operation (i.e. both initiator and responder-device are engaged in an active ranging session) is as follows:

1. The first packet sent by the initiator is a data packet called Pre-POLL message at UWB^k_{time0}. Note that, as mentioned above, the ranging starts in ranging round 0 in ranging block 0 by default when a ranging session is started or recovered. This first packet is a SP0 packet. Its payload shall include:
    a. $UWB_Session_ID^k$: ID of the current ranging session.
    b. $STS_Index^k\left(i, Round_Idx^k(i), POLL\right)$: STS index of the succeeding POLL message.
    c. i: ranging session current block index
    d. $Hop_Flag^k(i)$: Hopping flag as set from the previous ranging exchange. If current ranging round is the first ranging round after start or recovery of the ranging session, then $Hop_Flag^k(i=0)$ is set to '0.' Note that this field is only analyzed by the receiver if the hopping mode is set to "adaptive hopping."
    e. $Round_Idx^k(i)$: Round index for the (i+1)-th ranging block as set in Final_Data packet of the previous ranging exchange (i-th ranging block). If the current ranging round is the first in the started or recovered ranging session, then $Round_Idx^k(i=0)$ is set to '0.'

The Pre-POLL message and its parameters are defined in Table 20-3 and Table 20-4. All values of the parameters listed in Table 20-4 shall be encoded in little endian format.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 20-3: Pre-Poll Request Message and its parameters.

UWB MAC Message	UWB MAC Message ID	Parameters
Pre-POLL	1	UWB_Session_ID, Poll_STS_Index, Ranging_Block, Hop_Flag, Round_Index

Table 20-4: The definition of parameters in the Pre-Poll Message.

Parameters	Length (bytes)	Value	Description
UWB_Session_ID	4	0 - (2^{32}-1)	ID of the UWB ranging session
Poll_STS_Index	4	0 - (2^{31}-1)	STS index of the succeeding POLL message
Ranging_Block	2	0 - 65535	Session's Index of current ranging block
Hop_Flag	1	0:No Hopping 1:Hopping	Hop flag for current ranging block as set from the ranging exchange in the previous ranging block For no hopping configuration this field is always 0 For continuous hopping configuration this field is always 1 (except for the first ranging block with index 0 after start or recovery of a ranging session).
Round_Index	2	0-65535	The ranging round index for the current ranging block as set from the ranging exchange in the previous ranging block. This field is set to 0 in first ranging block where we always start in ranging round 0.

2. After a period of T^{k}_{Slot} measured from UWB^{k}_{time0}, the initiator shall start sending a POLL message. This shall be an SP3 type packet (see Section 21).
3. Each responder l ($l = 0,\ldots, N^{k}_{\text{Responder}} - 1$) shall send its response packet in its dedicated response slot.
    a. Each message shall be sent as a SP3 type packet. The index to the STS used is related to the POLL message sent in step 2 above (see Section 20.6).
    b. If any of the responders does not receive the POLL message in the current ranging round from the initiator, that responder shall not transmit during its dedicated response slot.
4. If at least one valid response has been received by the initiator:
    a. After $\left(N^{k}_{\text{Responder}} + 2\right)$ slots, the initiator shall send its Final message. This packet is an SP3 packet and it completes the time-of-flight measurements. The initiator may optionally send its Final message together with the following Final_Data packet, if no valid response has been received. In the case the Final message and Final_Data packet is skipped:
        - The hopping mode for both the initiator and the responder shall be triggered if the adaptive hopping mode is active.
        - The Frame Counter value shall not be incremented for the skipped Final Data packet.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

b. After $\left(N^{k}_{\text{Responder}}+3\right)$ slots, the initiator shall complete the ranging exchange by sending a Final_Data packet to the responder-device. This packet is an SP0 type packet and the payload shall include the following:

- $UWB_Session_ID^{k}$: id of the current ranging session.
- i: index of the current ranging block.
- $Hop_Flag^{k}(i+1)$: Hopping flag to be used in ranging block i+1. Note that this field is only relevant if the hopping configuration is set to adaptive hopping.
- $Round_Idx^{k}(i+1)$: ranging round index of the next ranging exchange in ranging block i+1.
- $STS_Index^{k}\left(i, Round_Idx^{k}(i), FINAL\right)$: STS index of the preceding FINAL message.
- $Final_Time_Stamp^{k}(i)$: the time stamp for the Final message. This time stamp shall be calculated as the difference between the RMARKER of the initiator POLL message and the RMARKER of the initiator Final message
- $N_{\text{Rx_Responses}}$: Number of received responses in the current ranging exchange.

  $[Time_Stamps^{k}(l)]$: All the ranging measurement time stamps for all responders, up to $N^{k}_{\text{Responder}}$, whose valid ranging responses have been received at the initiator. The initiator may additionally add the time stamp data of responders where no valid response has been received. The size of the payload depends on the number of added time stamp data. The time stamp data for each responder shall include the following fields:
  - Responder_Index l, $l = 0,1,\ldots,N^{k}_{\text{Responder}} - 1$, corresponding to responder slot l (slot index 2+l in the ranging round).
  - Ranging_Timestamp _Responder value for (l+1)-th responder. The time stamp value shall be calculated as the difference between the RMARKER of the initiator POLL message and the RMARKER of the (l+1)-th responder's response message (see Section 21.6 for details). In the case of no valid response was received from the responder but the time stamp data are added for this responder, the time stamp value shall be set to 0.
  - Ranging_Timestamp_Uncertainty_Responder parameter.
  - Ranging_Status_Responder parameter is set to 0x0 (success) in case of a valid response was received, otherwise this parameter is set to one of the values shown in Table 20-7 to show the reason for the error.

The Final_Data message and its parameters are defined in Table 20-5 and Table 20-6.

Table 20-5: Final_Data Message and its parameters.

UWB MAC Message	UWB MAC Message ID	Parameters
Final_Data	2	UWB_Session_ID, Ranging_Block, Hop_Flag, Round_Index, Final_STS_Index,

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

UWB MAC Message	UWB MAC Message ID	Parameters
		Ranging_Timestamp_FINAL_TX, Number_Ranging_Responders, Responder_Index, Ranging_Timestamp_Responder_*l*, Ranging_Timestamp_Uncertainty_Responder_*l*, Ranging_Status_Responder_*l*, …

1 *Table 20-6: The definition of parameters in the Final_Data Message.*

Parameters	Length (bytes)	Value	Description
UWB_Session_ID	4	0–(2^{32}-1)	ID of the UWB ranging session
Ranging_Block	2	0–65535	Index of current ranging Block
Hop_Flag	1	0: No Hopping 1: Hopping	Hop Flag for next ranging Block For no hopping configuration this field is always 0 For continuous hopping configuration this field is always 1
Round_Index	2	0 – 65535	The ranging round in which the ranging cycle will be executed in the next ranging block
Final_STS_Index	4	0 – (2^{31}-1)	STS index of the preceding Final message.
Ranging_Timestamp_FINAL_TX	4	Range = 0 to 0XFFFFFFFF Timestamp = N × 1/128 × 1/499.2e6 sec Time Range = 0 to 67.21 ms	Time difference between POLL and Final messages transmit times at the initiator.
Number_Ranging_Responders	1	0–255	Number of timestamps to follow in this message
Responder_Index	1	0–255	Index of Responder *l* whose timestamp data is referred to in the Ranging_Timestamp_Responder_*l*, Ranging_Timestamp_Uncertainty_Responder_-*l*, and Ranging_Status_Responder_*l* fields
Ranging_Timestamp_Responder_*l*	4	Range = 0 to 0XFFFFFFFF Timestamp=N × 1/128 × 1/499.2e6 sec Time Range = 0 to 67.21 ms	Time difference between POLL and RESPONSE of Responder *l*, as received by the initiator.
Ranging_Timestamp_Uncertainty_-Responder_*l*	1	See Section 6.9.1.7 in [33]	Range of values from 1.5 cm–3.6 m at various confidences
Ranging_Status_Responder_*l*	1	See Table 20-7	The status for the response frame from the responder. See the corresponding description for the ranging status in Table 20-7
…	…	…	Repeat parameters Responder_Index to Ranging_Status_Responder_*l* for each responder whose time stamp data has been added (up to Number_Ranging_Responders times)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

All values of the parameters listed in Table 20-6 shall be encoded in little endian format. Additionally, with the above definition of the Final_Data message and with a 127 maximum payload size, the maximum number of responders $N^{k}_{\text{Responder}}$ that can be involved in a single ranging round is 10 responders. In case $N^{k}_{\text{Chap_per_Slot}}$ = 24, then the maximum number of responders $N^{k}_{\text{Chap_per_Slot}}$ in a single ranging round is limited to 7 due to the maximum time stamp value for the Final message. A vehicle may have any number of responders Nv (physical and/or logical) and this number may be >10 responders. However, the vehicle shall set the value of the number of responders it wants to involve in each ranging round $N^{k}_{\text{Responder}}$ to a number that is less than or equal to10 in the ranging session setup procedure. It is up to the vehicle to determine which set of its responders it will involve in the ranging exchange in each ranging round.

The Ranging_Status_Responder_*l* parameter uses the values shown in Table 20-7.

Table 20-7: Ranging Status of a Responder.

Ranging Status	Value	Description
Success	0x0	Successful receipt and transmission of a packet
Transaction overflow	0x1	RESPONSE SP3 frame from this responder cannot be processed by the device
Transaction expired	0x2	No RESPONSE SP3 frame was received from this responder
Incorrect frame	0x03	The RESPONSE SP3 frame received from this responder was not correct
Reserved	0x04-0xFF	

20.5.2 *Ranging Session Setup*

The following procedure describes how the MAC parameter negotiation shall be carried out between the initiator and the responder-device. First, the responder-device and the initiator shall perform a ranging capability exchange step over Bluetooth LE link, where the responder-device sends a RC-RQ message to the initiator and the initiator replies with a RC-RS message, as defined in Sections 19.3.1.1 and 19.3.1.2, respectively. Once this exchange is successful, the following negotiation steps shall be carried out over the Bluetooth LE control channel to setup the ranging session:

1. Vehicle initiates the ranging session setup by sending the Ranging Session Request message defined in Section 19.3.1.3 with the following content:
   - *Selected_Protocol_Version*^{k}: selected protocol version from the ranging capability exchange.
   - *Selected_UWB_Config_ID*^{k}: selected UWB configuration from the ranging capability exchange.
   - *UWB_Session_ID*^{k}: id of the current ranging session.
   - *Selected_PulseShape_Combo*: device selected single pulse shape combination from list of common supported ones.
   - *Channel_BitMask*: $\{\text{CH_IDX}\}_{\text{Responder}}$ list of available UWB RF channels.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

2. Device responds by sending the Ranging Session Response message (see Section 19.3.1.4) with the following contents:
   - *RAN_Multiplier*: RAN multiplier N^k_{RAN} sets the minimum ranging block duration for the *k*-th ranging session that can be supported by the initiator (i.e. sets the maximum ranging frequency that can be supported by the initiator for this ranging session)

$$T^k_{\text{Block_RAN}} = N^k_{\text{RAN}} \times T_{\text{Block_Min}} = N^k_{\text{RAN}} \times 96\,ms$$

$$f^k_{\text{Ranging_RAN}} = \frac{1}{T^k_{\text{Block_RAN}}} = \frac{10.4166667}{N^k_{\text{RAN}}}\,\text{Hz}$$

   - *Slot_BitMask*: $\left\{N_{\text{Chap_per_Slot}}\right\}_{\text{Initiator}}$ list of slot durations supported by the initiator for the session expressed as specified in Section 19.3.1.4 and Table G-1. Note that initiator may prefer to use slot durations that are not necessarily the minimum (by excluding the minimum $N_{\text{Chap_per_Slot}}$ from the list) to accommodate other constraints it may have, such as co-existence.
   - *SYNC_Code_Index_BitMask*: $\{\text{SYC_IDX}\}_{\text{Initiator}}$ list of available UWB preamble sequences that the responder-device may choose from.
   - *Selected_UWB_Channel CH_IDX*: selected UWB RF channel.
   - *Hopping_Config_Mask*: $\{H_Config_Seq\}_{\text{Initiator}}$ bitmask indicating the hopping configuration (no hopping, continuous, adaptive) and hopping sequences supported by the device.

3. Responder-device calculates/selects:
   - *Session_RAN_Multiplier* $N^k_{\text{RAN_S}}$: the responder-device selects an N^k_{RAN} value that is greater than or equal to the value sent by the initiator to achieve a desired ranging interval of T^k_{Block} that is an integer multiple of $T_{\text{Block_Min}}$:

$$N^k_{\text{RAN_S}} = \frac{T^k_{\text{Block}}}{T_{\text{Block_Min}}} \quad \left(N^k_{\text{RAN_S}} \geq N^k_{\text{RAN}}\right) \quad \text{and} \quad f^k_{\text{Ranging}} = \frac{1}{T^k_{\text{Block}}} = \frac{10.4166667}{N^k_{\text{RAN_S}}}\,\text{Hz}$$

   - *Number_Chaps_per_Slot* $N^k_{\text{Chap_per_Slot}}$: shortest slot duration that is common between slot durations supported by initiator $\left\{N_{\text{Chap_per_Slot}}\right\}_{\text{Initiator}}$ and slot durations supported by the responder-device $\left\{N_{\text{Chap_per_Slot}}\right\}_{\text{Responder}}$:

$$N^k_{\text{Chap_per_Slot}} = \min\left(\left\{N_{\text{Chap_per_Slot}}\right\}_{\text{Initiator}} \cap \left\{N_{\text{Chap_per_Slot}}\right\}_{\text{Responder}}\right)$$

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- *Number_Slots_per_Round* $N^k_{\text{Slot_per_Round}}$: responder-device selects the number of slots that is greater than or equal to $\left(N^k_{\text{Responder}} + 4\right)$ out of all possible values of slots corresponding to $N^k_{\text{Chap_per_Slot}}$ (see Table G-1 for details).
- *SYNC_Code_Index* $\{\text{SYC_IDX}\}_{\text{Responder}}$: list of UWB preamble sequences that the responder-device selected to use. This set is a subset of the list sent by the initiator.
- *Hopping_Config_Mask*: $\{H_Config_Seq\}_{\text{Responder}}$ bitmask indicating the hopping configuration and hopping sequences selected by responder-device.

4. Responder-device uses the "Ranging Session Setup Request (RSS-RQ)" message as defined in Section 19.3.1.5 to send the following to the initiator:
   a. $N^k_{\text{RAN_S}}$, $N^k_{\text{Chap_per_Slot}}$, $N^k_{\text{Responder}}$, $N^k_{\text{Slot_per_Round}}$, $\{\text{SYC_IDX}\}_{\text{Responder}}$, $\{H_Config_Seq\}_{\text{Responder}}$
5. Initiator selects the number of rounds per block for the ranging session N^k_{Round} as

$$N^k_{Round} = \frac{288 \times N^k_{RAN_S}}{N^k_{Chap_per_Slot} \times N^k_{Slot_per_Round}}$$

Appendix G.1 shows a list of all valid/possible numbers of rounds for different combinations of $\left(N^k_{\text{Chap_per_Slot}}, N^k_{\text{Slot_per_Round}}\right)$ for each 96 ms of block duration.

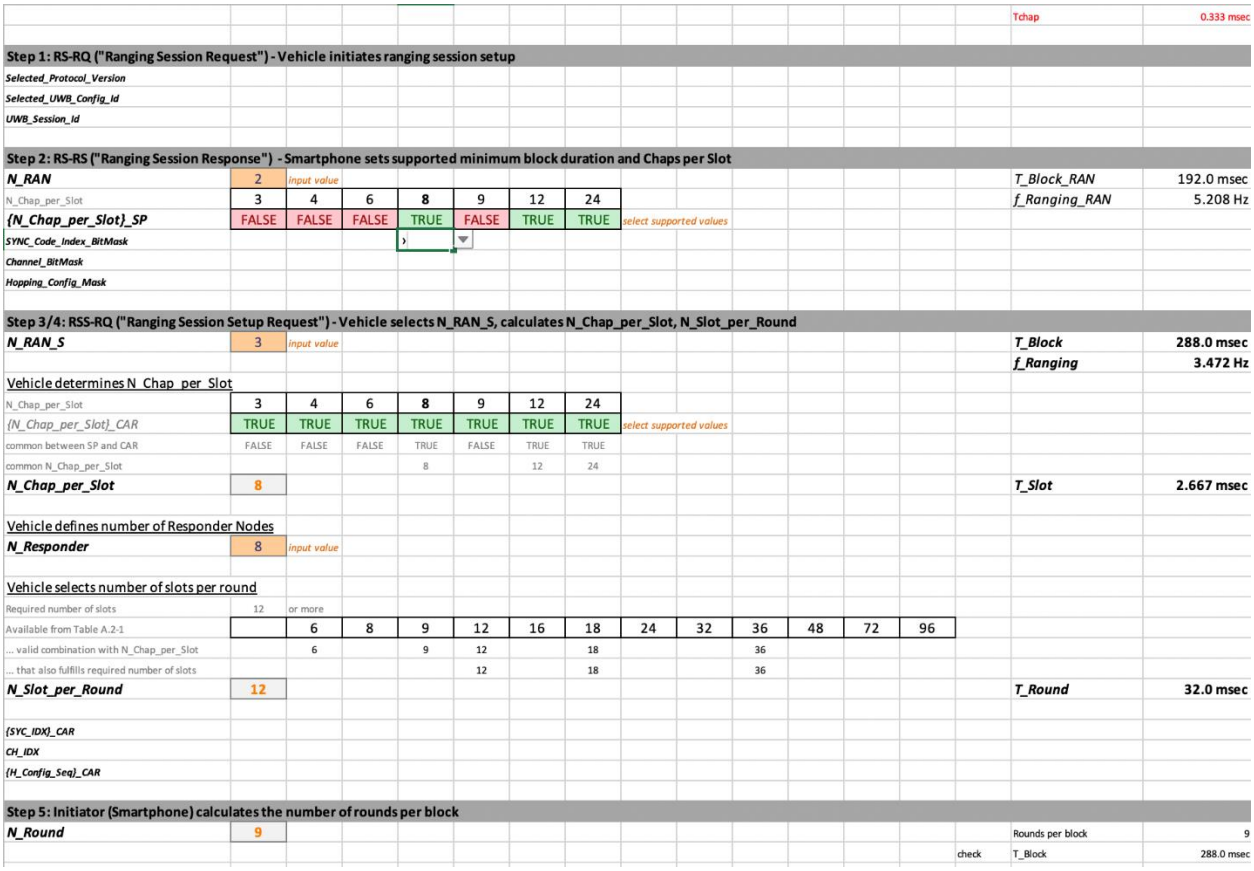
6. Initiator selects the preamble SYNC code index $SYNC_Code_Index^k$. See Section 21.4.1 for the definition of BPRF SYNC.
7. Initiator uses the "Ranging Session Setup - Response (RSS-RS)" message as defined in Section 19.3.1.6 to send the following back to the responder-device:
   a. $STS_Index0^k, UWB^k_{time0}, HOP_Key_RW^k$
   b. $SYNC_Code_Index^k$

Figure 20-6 shows an example of the negotiation handshake used to set the MAC parameters. The following applies to MAC parameters negotiation:

1. $N^k_{\text{Chap_per_Slot}} = 8$ is mandatory and shall be supported by all devices.
   - This avoids a deadlock situation due to a possibly disjoint configuration parameters capabilities between the initiator and the responder-device during negotiation.
2. Responder-device will accept any N^k_{RAN} chosen by the initiator, even if it does not allow it to realize the desired ranging frequency.
   - The initiator shall set N^k_{RAN} as small as possible to meet the quality of service (QoS) constraints to meet the functional requirement.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 20-6: Example of MAC Parameter Negotiation and Setting.



														Tchap	0.333 msec
Step 1: RS-RQ ("Ranging Session Request") - Vehicle initiates ranging session setup															
Selected_Protocol_Version															
Selected_UWB_Config_Id															
UWB_Session_Id															
Step 2: RS-RS ("Ranging Session Response") - Smartphone sets supported minimum block duration and Chaps per Slot															
N_RAN	2	*input value*												*T_Block_RAN*	192.0 msec
N_Chap_per_Slot	3	4	6	8	9	12	24							*f_Ranging_RAN*	5.208 Hz
{N_Chap_per_Slot}_SP	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	TRUE	*select supported values*							
SYNC_Code_Index_BitMask															
Channel_BitMask															
Hopping_Config_Mask															
Step 3/4: RSS-RQ ("Ranging Session Setup Request") - Vehicle selects N_RAN_S, calculates N_Chap_per_Slot, N_Slot_per_Round															
N_RAN_S	3	*input value*												*T_Block*	288.0 msec
														f_Ranging	3.472 Hz
Vehicle determines N_Chap_per_Slot															
N_Chap_per_Slot	3	4	6	8	9	12	24								
{N_Chap_per_Slot}_CAR	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	*select supported values*							
common between SP and CAR	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	TRUE								
common N_Chap_per_Slot				8		12	24								
N_Chap_per_Slot	8													*T_Slot*	2.667 msec
Vehicle defines number of Responder Nodes															
N_Responder	8	*input value*													
Vehicle selects number of slots per round															
Required number of slots	12	or more													
Available from Table A.2-1		6	8	9	12	16	18	24	32	36	48	72	96		
... valid combination with N_Chap_per_Slot		6		9	12		18			36					
... that also fulfills required number of slots					12		18			36					
N_Slot_per_Round	12													*T_Round*	32.0 msec
{SYC_IDX}_CAR															
CH_IDX															
{H_Config_Seq}_CAR															
Step 5: Initiator (Smartphone) calculates the number of rounds per block															
N_Round	9													Rounds per block	9
													check	T_Block	288.0 msec

20.5.3 *MAC Control Channel (Over Bluetooth LE)*

A MAC control channel shall be implemented with following requirements:

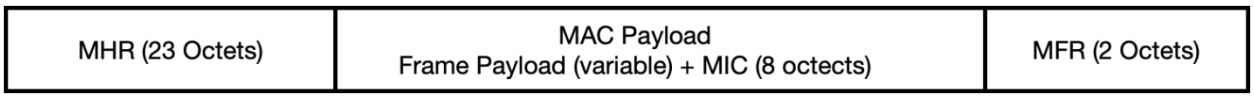
1. Control channel shall be available for negotiation or re-negotiation of UWB parameters (e.g. ranging session setup as in Section 20.5.2).
2. The control channel shall be session specific and shall be available before the start of, during the entire lifetime of, and after the termination of the ranging session.
3. A RAN will have as many UWB control channels as it has responder-devices.
4. The control channel shall be carried over the Bluetooth LE connection.
5. Control traffic with respect to the *k*-th ranging session shall be referenced with the $UWB_Session_ID^k$.
6. Timings during an active ranging session shall be referenced via the appropriate MAC time grid indices (e.g. block index, round index, slot index, or mapped STS indices) of this ranging session.
7. The control channel shall be triggered by a channel ID in the Bluetooth LE L2CAP channel. Initiator or responder-device may trigger the control channel and can request a change of UWB parameters.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

20.5.4 *UWB MAC Configuration*

All SP0 type packets sent according to the above MAC protocol shall include a MAC header (MHR), MAC payload and MAC Footer (MFR) fields. This section describes how the contents of the SP0 packets are constructed.

Figure 20-7: SP0 Packet Content.



20.5.4.1 MHR Field

The contents of the MHR field according to Section 7.2 of [31] are defined in Table 20-8. All values of the parameters listed in Table 20-8 shall be encoded in little endian format.

Table 20-8: The Contents of the MHR Field.

MAC parameter	Length (bytes)	Description
Frame Control	2	Describes how the MAC frame is configured
Sequence Number	0	Sequence number is not used. Sequence Number Suppression bit field in the Frame Control shall be set to 1
Destination PAN ID	0	Destination PAN ID not present
Destination Address	2	Destination Short Address
Source PAN ID	0	Source PAN ID not present
Source Address	0	Source Address not present
Auxiliary Security Header	10	Describes security configuration for data payload encryption and message identification authentication (MIC)
Vendor Specific Header IE	7	Used to signal the UWB message ID and the UWB message length
Header Termination IE	2	Header Termination IE of Type HT2. See 7.4.1 of [31]

The Frame Control field depends on the specific MAC header configuration. See [31] for more information about the individual parameters and their definitions. The Frame Control field is defined in Table 20-9.

Table 20-9: The Contents of the Frame Control Field.

Frame Control	Length (bits)	Value	Description
Frame Type	3	0b001 (data)	Data frame being transmitted
Security Enabled	1	0b1	Every packet has security enabled
Frame Pending	1	0b0	Frame Pending Flag is always set to 0.
AR	1	0b0	This flag indicates whether this frame requires acknowledgment (0b1) or not (0b0). Always set to 0 as neither Pre-Poll or Final Data packets require an acknowledgment back from the vehicle
PAN ID Compression	1	0b1	Source Addressing Mode set to 0, Destination Addressing Mode set to 2 (short address), and Destination PAN ID not present this corresponds to the 4th entry of Table 7-2 in [31].

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Frame Control	Length (bits)	Value	Description
Reserved	1	0b0	
Sequence Number Suppression	1	0b1	Sequence number is disabled
IE Present	1	0b1	Header IE Used.
Destination Addressing Mode	2	0b10	Destination Short Address
Frame Version	2	0b10	802.15.4 frames for data and acknowledgement.
Source Addressing Mode	2	0b0	Source PAN ID and Source Address field are not present

The Auxiliary Security Header defines the security configuration. The Auxiliary Security Header is defined in Table 20-10. Note that the Auxiliary header is not relevant to SP3 packets (Poll and Final messages) since SP3 packets do not get security processing due to the fact they do not carry MAC data frames (payload, MHR, and MFR). Therefore, The Auxiliary Security header is applicable to SP0 packets only (Pre-Poll and Final_Data). All values of the parameters listed in Table 20-10 shall be encoded in little endian format.

Table 20-10: The Contents of the Auxiliary Security Header.

Auxiliary Security Header	Length (bytes)	Value	Description
Security Control Field	See below		
Security Level	3 bits	0b110 (ENC-MIC-64)	Security level for authentication of MAC header and data payload
Key Identifier Mode	2 bits	0b10, Key is determined explicitly from the 4-byte Key Source field and Key Identifier field	Key identification mode
Frame Counter Suppression	1 bit	0b0, enable frame counter	Frame counter is suppressed or not
ASN in Nonce	1 bit	0b0, ASN in not in nonce	Is ASN in nonce or not
Reserved	1 bit	0b0	
Frame Counter	4 bytes	0x00000000-0xFFFFFFFF	Counter that increments for each transmitted packet from a source
Key Identifier Field	See below		
Key Source	4 bytes	0x00000000-0xFFFFFFFF	Key Source derived from UAD
Key Index	1 byte	0xAA	

The content of the Header IE defined the Vendor Specific Header IE which shall be used to signal the UWB message ID and the UWB message length carried by the corresponding UWB packet.

The Vendor Specific Header IE is defined in Table 20-11. All parameter values listed in Table 20-11 shall be encoded in little endian format.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 20-11: The Contents of the Vendor Specific Header IE.

Vendor Specific Header IE	Length (bytes)	Value	Description
Length	7 bits	5 (bit 0 - bit 6)	Length of the IE content in bytes
IE ID	8 bits	0 (bit 7 - bit 14)	Vendor Specific Header IE
Type	1 bit	0 (bit 15)	Header IE
Vendor OUI	3 bytes	0x04DF69	OUI of Car Connectivity Consortium per IEEE registration authority (See http://standards-oui.ieee.org/oui.txt).
Vendor Specific Information	2 Bytes	Byte 1: 0x00 - 0xFF Byte 2: 0x00 - 0xFF	Byte 1: UWB MAC message ID of message sent in this UWB packet 0x01: Pre-Poll UWB MAC message ID 0x02: Final_Data UWB MAC message ID 0x03-0xFF: Reserved for future use Byte 2: UWB MAC message payload length in bytes sent in this UWB packet excluding the MHR, MFR, and MIC fields.

20.5.4.2 MHR Source and Destination Address

The Source Addressing Mode shall be set to 0 in the Frame Control Field of the MHR. This indicates that the Source Address and the Source PAN ID shall not be present in the MHR. The Destination Addressing Mode shall be set to 2 and the PAN ID Compression shall all be set to 1 in the Frame Control field of the MHR. This indicates that the Destination Short address shall be present, and the Destination PAN ID shall not be present in the MHR. The Destination Short Address along with Source Long Address and Key Source fields shall be generated according to address rotation procedure -as described in Section 22.

20.5.4.3 MAC Payload

The MAC payload field is of variable length and includes the MAC frame payload plus the 64-bit message integrity check (MIC) field

20.5.4.4 MFR Field

MAC footer shall be according to Section 7.2.10 of [31], which states that the MAC footer shall contain a Frame Check Sequence (FCS) field. For the BPRF mode in the HRP UWB PHY, the FCS contains a 16-bit ITU-T CRC. The FCS is calculated over the MHR and MAC payload part of the frame; these parts together are referred to as the calculation fields. This 2-byte FCS is calculated using the following standard generator polynomial of degree 16:

$$G_{16}(x) = x^{16} + x^{12} + x^5 + 1$$

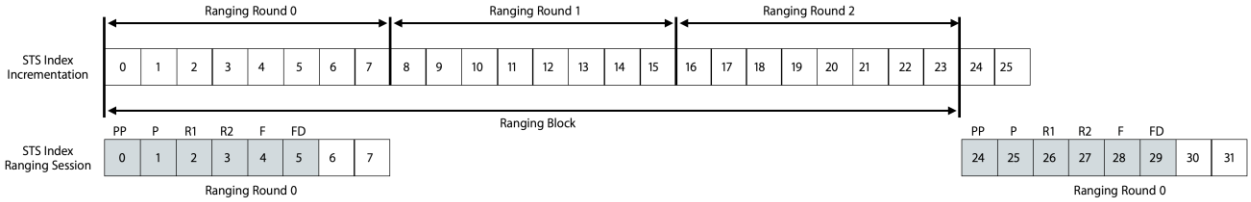
20.6 STS Index Incrementation

The STS index parameter (which needs to change for every slot in ranging block regardless of whether that slot is used or not) plays an important role for the secure ranging operation. This section defines the STS index incrementation and the mapping of the STS index onto the packets

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

of a ranging session within the MAC layer framework. The basic principle of the STS incrementation is shown in Figure 20-8.

Figure 20-8: Basic Principles of STS Incrementation.



(The example uses $N^k_{\text{Responder}} = 2$, $N^k_{\text{Slot_per_Round}} = 8$, $N^k_{\text{Round}} = 3$. Note that $N^k_{\text{Round}} = 3$ is not a supported number of rounds in the current design but is used in this example for illustration purpose only.)

20.6.1 *Rules for STS Index Incrementation*

The following rules shall be used for incrementing the STS index:

1. The STS index is managed separately for each ranging session of a given RAN.
2. A *k*-th ranging session starts at time UWB^k_{time0}.
3. The STS index of the k-th ranging session is referenced to the start of this ranging session:

   STS_Index0^k is mapped to ranging round 0 of ranging block 0. That is, for $i = 0$ and $Round_Idx^k(i = 0) = 0$

   $$STS_Index^k\left(i = 0, Round_Idx^k(i = 0) = 0\right) = STS_Index0^k$$

4. Within a ranging round, the STS index increments in every slot even if that slot is unmapped (i.e. not used).
5. The incrementation scheme is applied for every ranging round (even if it is not used). This results in:
   a. The STS index is incremented by $N^k_{\text{Slot_per_Round}}$ in every ranging round.
   b. The STS index is incremented by $N^k_{\text{Slot_per_Round}} \cdot N^k_{\text{Round}}$ in every ranging block.
6. The STS index is incremented for all slots in the round. However, data packets (Pre-POLL and Final_Data) reference the STS index in their payload as follows:
   a. Pre-POLL message shall carry the reference to the STS index that is used in the following POLL message.
   b. Final_Data message shall refer to the STS index of the preceding FINAL message.

20.6.2 *STS Incrementation and Packet Mapping within a Ranging Round*

Within any ranging round s in ranging block *i*, the STS index increments for every slot (relative to the first STS index in the round):

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- Pre_POLL $STS_Index^k(i,s)$
- POLL $STS_Index^k(i,s)+1$
- Response_0 $STS_Index^k(i,s)+2$
- Response_1 $STS_Index^k(i,s)+3$
- …
- Response $_N^k_{\text{Responder}}$ - 1 $STS_Index^k(i,s)+N^k_{\text{Responder}}+1$
- Final $STS_Index^k(i,s)+N^k_{\text{Responder}}+2$
- Final_Data $STS_Index^k(i,s)+N^k_{\text{Responder}}+3$

20.6.3 *Calculation of STS Index*

The following formulas are used to calculate the STS indices for ranging round $Round_Idx^k(i)$ in ranging block *i*:

$$STS_Index^k\left(i,Round_Idx^k(i),\text{Pre_POLL}\right)=STS_Index0^k+\left(i\times N^k_{\text{Round}}+Round_Idx^k(i)\right)\times N^k_{\text{Slot_per_Round}}$$

$$STS_Index^k\left(i,Round_Idx^k(i),\text{POLL}\right)=STS_Index0^k+\left(i\times N^k_{\text{Round}}+Round_Idx^k(i)\right)\times N^k_{\text{Slot_per_Round}}+1$$

$$STS_Index^k\left(i,Round_Idx^k(i),\text{R}_0\right)=STS_Index0^k+\left(i\times N^k_{\text{Round}}+Round_Idx^k(i)\right)\times N^k_{\text{Slot_per_Round}}+2$$

$$STS_Index^k\left(i,Round_Idx^k(i),\text{R}_1\right)=STS_Index0^k+\left(i\times N^k_{\text{Round}}+Round_Idx^k(i)\right)\times N^k_{\text{Slot_per_Round}}+3$$

$$\vdots$$

$$STS_Index^k\left(i,Round_Idx^k(i),\text{R}_{N^k_{\text{Responder}}-1}\right)=STS_Index0^k+\left(i\times N^k_{\text{Round}}+Round_Idx^k(i)\right)\times N^k_{\text{Slot_per_Round}}+N^k_{\text{Responder}}+1$$

$$STS_Index^k\left(i,Round_Idx^k(i),\text{Final}\right)=STS_Index0^k+\left(i\times N^k_{Round}+Round_Idx^k(i)\right)\times N^k_{\text{Slot_per_Round}}+N^k_{\text{Responder}}+2$$

$$STS_Index^k\left(i,Round_Idx^k(i),\text{Final_Data}\right)=STS_Index0^k+\left(i\times N^k_{\text{Round}}+Round_Idx^k(i)\right)\times N^k_{\text{Slot_per_Round}}+N^k_{\text{Responder}}+3$$

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

21 UWB PHY [WCC3]

21.1 Introduction

The UWB PHY uses a waveform based on an impulse radio signal using band-limited pulses. UWB PHY is primarily used for ranging but may also be used for data communication. UWB PHY described in this specification is based on HRP UWB PHY in [33].

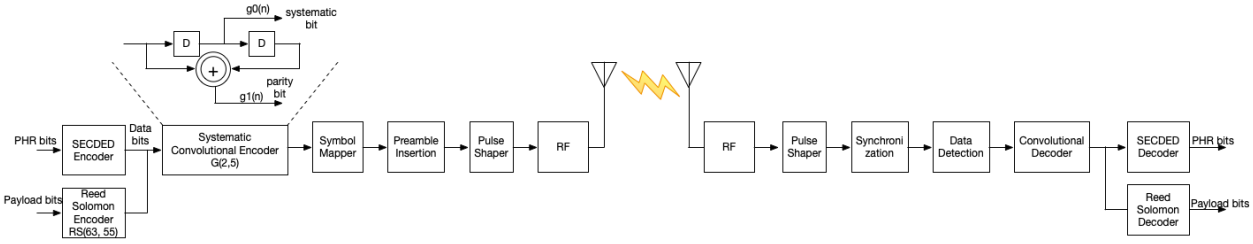
This section describes interoperable enhanced ranging devices (ERDEVs) featuring enhanced immunity to attack. The main enhancement for secure time of flight is the inclusion of a Scrambled Timestamp Sequence (STS) in the basic PPDU format. Note that the STS is included in the basic PPDU format in the BPRF mode as specified in [33].

The IEEE standard defines a very flexible UWB PHY. The flexibility in IEEE standard is offered by adaptation of parameters like preamble for synchronization (SYNC) lengths, preamble codes, Start of Frame Delimiter (SFD) lengths/codes, pulse repetition frequencies (PRFs), and data rates. This specification, however, does not require to implement all the parameters and modes that are specified in [33]. Please refer to appropriate sections on mandatory and optional PHY modes for secure ranging, ERDEVs on both sides of the link shall pre-negotiate the specific parameters that will be used for a secure ranging session. The negotiations of secure ranging parameters can be done at higher layers or using Bluetooth LE.

21.2 UWB PHY Block Diagram

Figure 21-1 shows the block diagram for IEEE 802.15.4z HRP UWB PHY.

Figure 21-1: Block diagram for IEEE 802.15.4z HRP UWB PHY.



21.3 UWB Packet Format

Figure 21-2 and Figure 21-3 illustrate the two, packet types for UWB configurations: STS packet type 3 (SP3) and STS packet type 0 (SP0).

For SP3, the Deterministic Random Bit Generator (DRBG) STS engine shall generate 4096 bits to create the STS waveform and there is no PHR or data field. Note that packet configurations to be used for UWB ranging (and also for UWB data communication) shall be known a-priori. The packet configurations to be used between two nodes shall be exchanged over Bluetooth LE link. However, the length of the data field in SP0 is flexible.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Figure 21-2: SP3

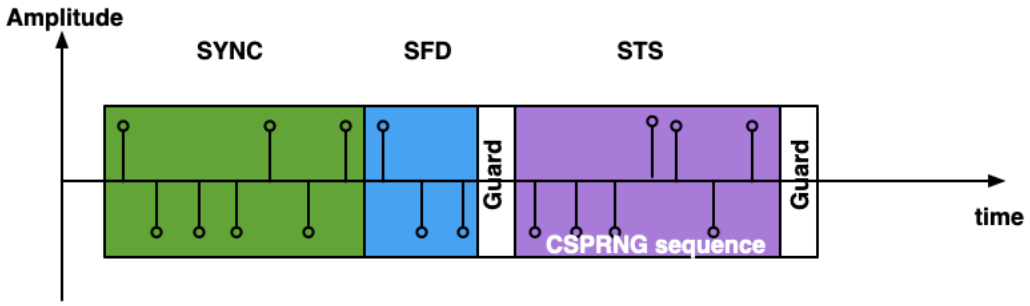
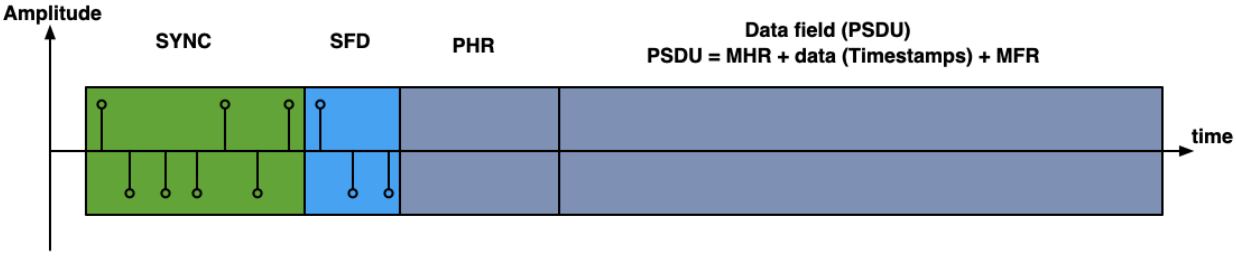


Figure 21-3: SP0



SP3 is intended for use cases where the participants in the secure ranging exchange are known to each other such that information about the source and/or the destination are implicit in the knowledge of what STS is used for transmission and reception between the connected devices, respectively.

A nominal mean PRF of at least 62.4 MHz shall be used for all packets during a ranging session. Nominal mean PRFs lower than 62.4 MHz are not required for ERDEVs.

21.4 UWB Frame Elements

Only IEEE 802.15.4z HRP BPRF UWB SP0 and SP3 packets are used in this specification. SP0 packet consists of preamble (SYNC), SFD, PHY header (PHR), and payload (PSDU or Data field). Supported configurations are referenced as UWB_Config as shown in Table 21-1. Each of the fields is described in detail in the subsections below.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table 21-1: Supported UWB Configurations.

DK protocol			M/O	STS Packet Type 3 (SP3) IEEE 802.15.4z parameters				STS Packet Type 0 (SP0) IEEE 802.15.4z parameters						
UWB configuration	UWB Channel (IEEE 802.15.4)	SYNC preamble code (IEEE 802.15.4)	Mandatory/ Optional	corresponding IEEE BPRF Mandatory Set	phyHrpUwbPsr	SFD (phyHrpUwbSfd-Selector)	STS (segment × symbols)	corresponding IEEE802.15.4z BPRF Set	phyHrpUwbPsr	SFD (phyHrpUwbSfd-Selector)	STS	phyHrpUwbPhr-DataRate	PHR bit rate	PSDU bit rate
0	5, 9	9,10,11,12 Optional: 13-16, 21-24	M	N/A	64	ternary (0)	1× 64	**BPRF #1**	64	ternary (0)	no	DRBM-_LP	0.85 Mbps	6.81 Mbps
1	5, 9	9,10,11,12 Optional: 13-16, 21-24	Device: M Vehicle: O	BPRF #4	64	binary (2)	1× 64	**BPRF #2**	64	binary (2)	no	DRBM-_LP	0.85 Mbps	6.81 Mbps

21.4.1 *BPRF SYNC*

SYNC is used to aid receiver algorithms related to AGC settling, timing acquisition, coarse and fine frequency recovery, packet and frame synchronization. [31] defines four mandatory preamble lengths for SYNC: a default preamble, a short preamble, a medium preamble, and a long preamble. The length of default preamble is 64 symbols, short preamble is 16 symbols, medium preamble is 1024 symbols, and long preamble is 4096 symbols. The PHY as specified in this specification shall support 64 symbols short preamble.

ERDEV in this specification, shall support ternary code with length 127, even though, ternary code with length 127 is optional in [31]. Each preamble code is a sequence of code symbols drawn from a ternary alphabet {–1,0,1} and is selected for use because of their perfect periodic autocorrelation properties.

In this specification, the ERDEV shall operate on channels 5 and 9, and hence the support for the four length-127 ternary code sequences shown in Table 21-2 are mandatory, though they are optional in [31].

Table 21-2: The Mandatory Length-127 Ternary Code Sequences.

Code index	Ternary Code Sequence	UWB channels
9	+00+000-0--00--+0+0+00-+-++0+0000++-000+00-00--0-+0+0--0-+++0++000+-0+00-0++-0+++00-+00+0+0-0++-+--+000000+00000-+0000-0-000--+	5,9
10	++00+0-+00+00+000000-000-00--000-0+-+0-0+-0-+00000+-00++0-0+00--+00++-+0+-0+0000-0-0-0-++-+0+00+0+000-+0+++000----+++0000+++0--	5,9
11	-+-0000+00--00000-0+0+0+-0+00+00+0-00-+++00+000-+0+0-0000+++++-+0+--0+-0++--0-000+0-+00+0+----000-000000-+00+-0++000++-00++-0-0	5,9
12	-+0++000000-0+0-+0---+-++00-+0++0+0+0+000-00-00-+00+-++000-+-0-++0-0++++0-00-0++00+0+00++-00+000+-000-0--+0000-0000--0+00000+--	5,9

The Code index 9, 10, 11, 12 shall be supported.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Support for code indexes 13-16 and 21-24 ternary code sequences as specified in [31] is optional in this specification.

21.4.2 *BPRF SFD*

SFD is added to the UWB frame to establish frame timing at the receiver. Only length 8 SFD is used in this specification:

The short ternary SFD shall be [0 +1 0 –1 +1 0 0 –1] spread by the preamble symbol Si, where the leftmost bit shall be transmitted first in time. This is as specified in [31]. To improve performance, the binary sequence: [-1 -1 -1 +1 -1 -1 +1 -1] should be supported per [33].

21.4.3 *Scrambled Timestamp Sequence (STS)*

UWB PHY supports Secure Ranging mechanisms to prevent different forms of attacks carried out by a "rogue" device injecting signal energies with the objective of making ranging receivers misread the distance between associated ranging peers. Scrambled Timestamp Sequence (STS) provides an AES-generated ranging waveform to protect against range measurement spoofing. DRBG based STS is resilient to various types of distance manipulation attacks like Brute force attack, Cicada attack, preamble injection attack and preamble EDLC. Details on how STS sequence is generated is specified in [33]. The number of bits used for STS is specified in Table 21-1. STS bits are mapped to pulses as follows:

1. Bit 1 is mapped to a negative pulse
2. Bit 0 is mapped to a positive pulse

In the BPRF (64 MHz PRF) mode, there is one pulse every 8 chips (spreading = 8)

21.4.4 *BPRF PHR*

A PHR, as shown in Figure 21-4, shall be added after the SHR preamble. The PHR consists of 19 bits and conveys information necessary for a successful decoding of the packet to the receiver. The PHR contains information about the data rate used to transmit the PSDU, the duration of the current frame's preamble, and the length of the frame payload. Additionally, six parity check bits are used to further protect the PHR against propagation channel errors.

Figure 21-4: PHR bit assignment.

Bits: 0–1	**2–8**	**9**	**10**	**11–12**	**13–18**
Data Rate	Frame Length	Ranging	Reserved	Preamble Duration	SECDED

It is mandatory for RDEVs and ERDEVs to support transmission and reception of PRF64 PHRs at 850 Kbit/s data rate. The BPRF PHR shall use Single Error Correct Dual Error Detect (SECDED) encoding followed by systematic convolutional encoder with generator polynomial G(2,5) as shown in Figure 21-1. The SECDED field is a set of six parity check bits that is used to protect the PHR from errors caused by noise and channel impairments. The SECDED bits are a simple Hamming block code that enables the correction of a single error and the detection of two

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

errors at the receiver. The SECDED bit values depend on PHR bits 0–12 and are computed as follows:

b18 = XOR(b1, b0, b8, b6, b4, b3, b10, b11)

b17 = XOR(b0, b6, b5, b3, b2, b9, b10, b12)

b16 = XOR(b1, b8, b7, b3, b2, b9, b10)

b15 = XOR(b8, b7, b6, b5, b4 ,b9, b10)

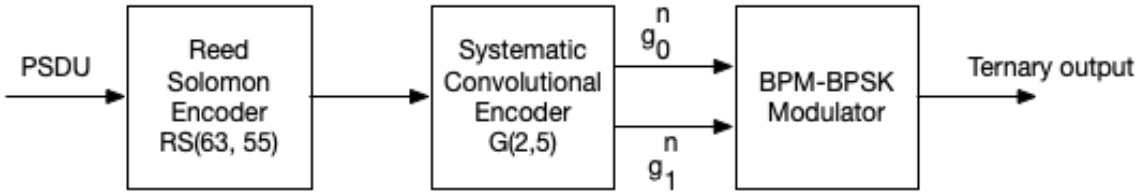
b14 = XOR(b12, b11)

b13 = XOR(b0, b1, b2, b3, b4, b5, b6, b7, b8, b9, b10, b11, b12, b14, b15, b16, b17, b18)

21.4.5 *Standard Compliant Data Field*

The Data field is transmitted using the 6.8 Mbps data rate at a mean PRF of 62.4 MHz and encoded as shown in Figure 21-5.

Figure 21-5: Data Field Encoding for Standard PHY.



The data field shall be formed as follows:

- Encode the PSDU using systematic Reed-Solomon block code, which adds 48 parity bits
- Encode the output of the Reed-Solomon block code using a systematic convolutional encoder. If the Viterbi rate for the modulation is defined as one, then the convolutional encoder is bypassed
- Spread and modulate the encoded block using BPM-BPSK modulation

21.5 Pulse Shape Combinations

In addition to the pulse shape requirements in Section 15.4.4 and 15.4.5 of [33], a compliant transmitter shall choose between a traditional symmetrical Root-Raised-Cosine Pulse (see Section 21.5.1) and a Precursor-Free Pulse (see Section 21.5.2).

21.5.1 *Symmetrical Root-Raised-Cosine Pulse – PulseShape 0x0*

For the symmetrical Root-Raised-Cosine Pulse, referred to as PulseShape 0x0, the RF envelope (or equivalent baseband pulse amplitude) is expressed by [31]:

$$r(t) = \frac{4\beta}{\pi\sqrt{T_p}} \frac{cos\big((1+\beta)\pi t/T_p\big) + \frac{sin\big((1-\beta)\pi t/T_p\big)}{4\beta(t/T_p)}}{1-(4\beta t/T_p)^2}$$

where:

β = roll-off factor with a value of 0.45 ±0.05

Tp = Pulse duration, which is inverse of the chip frequency (1/499.2MHz)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Finite impulse response (FIR) samples of PulseShape 0x0 using β of 0.45 with 32x oversampling per chip and total span of 8 chips are included in Appendix H as informative reference.

21.5.2 *Precursor-Free Pulse – PulseShape 0x1*

In general, “precursor-free” refers to the pulse’s property to monotonically raise its amplitude to the first peak that also represents its main lobe. PulseShape 0x1 represents a group of pulse shapes that meet the time-domain mask of Figure-89 in [33].

21.5.2.1 Precursor-Free Pulse – PulseShape 0x2

PulseShape 0x2 is a special case within PulseShape 0x1, by applying minimum-phase conversion of PulseShape 0x0 using method described in [31]. A Python implementation of the linear-to-minimum phase conversion is included in Appendix G, and FIR samples for a span of 8 chips at oversampling of 32x and β of 0.45 for PulseShape 0x2 are again included in Appendix H, both provided as informative references. It is recommended to use PulseShape 0x2 whenever possible if transmitter chooses to implement precursor-free pulse shape.

Table 21-3 summarizes the different pulse shapes that allowed in the specification. However, it is recommended to use PulseShape 0x2.

Table 21-3: Summary of pulse shapes.

PulseShape	Description	bitfield
Symmetrical Root-Raised-Cosine Pulse	Symmetrical Root-Raised-Cosine Pulse, referred to as PulseShape 0x0. The RF envelope or equivalent baseband pulse amplitude is expressed by [31].	0x0
Precursor-Free Pulse	Group of pulse shapes that meet the time-domain mask of Fig. 89 in [33].	0x1
	A special case within PulseShape 0x1, by applying minimum-phase conversion of PulseShape 0x0 using method described in [31].	0x2

21.5.3 *PulseShape Combinations*

Compliant receivers shall be able to receive and suitably process both symmetrical and precursor-free pulses. It is up to the implementer whether the same receiver configuration is used for PulseShape 0x2 and PulseShape 0x1, or whether reception of PulseShape 0x2 is explicitly optimized via a separate receiver configuration.

Possible transmitter PulseShape combinations (PulseShape_Combo) for the initiator and responder are shown in Table 21-4. The responder’s and initiator’s receive configurations shall be set according to the PulseShape selections made by the initiator’s and responder’s transmitters, respectively.

Table 21-4: Overview of PulseShape_Combo values and the associated transmit pulse shapes.

PulseShape_Combo	Transmit PulseShape	
	Initiator	Responder
0x00	0x0	0x0
0x01	0x0	0x1
0x02	0x0	0x2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

PulseShape_Combo	Transmit PulseShape	
	Initiator	Responder
0x10	0x1	0x0
0x11	0x1	0x1
0x12	0x1	0x2
0x20	0x2	0x0
0x21	0x2	0x1
0x22	0x2	0x2

21.6 Ranging Marker

The RMARKER as defined in [33] shall be supported for taking timestamp measurements.

21.7 UWB RF

21.7.1 *Operating frequency bands and Channel assignments*

The UWB technology as specified in [31] supports three independent bands of operation and a wide frequency range from hundreds of MHz to several GHz:

- The sub-gigahertz band, which consists of a single channel and occupies the spectrum from 249.6 MHz to 749.6 MHz.
- The low band, which consists of four channels and occupies the spectrum from 3.1 GHz to 4.8 GHz.
- The high band, which consists of 11 channels and occupies the spectrum from 6.0 GHz to 10.6 GHz.

In this specification, the ERDEV shall operate in the high band using channel 5 and channel 9. Channel 5 uses center frequency of 6489.6 MHz and channel 9 uses center frequency of 7987.2 MHz. ERDEV that only operates in specific geographic regions where regulatory restrictions prevent the use of specific band(s) may be exempted from this requirement.

21.7.2 *Frequency Source Requirements*

RF center frequency and the symbol clock frequency shall all be derived from the same reference clock (clk_ref), which is typically implemented as a crystal oscillator (XO). Clock and sampling rates for all transmitter and receiver activities in the PHY and the RF Carrier Frequency shall be tied to the same common clk_ref.

clk_ref tolerance

For the device, clk_ref tolerance should be regulated within ±10 ppm over operation conditions. For the vehicle, clk_ref tolerance should be regulated within ±15 ppm over operation conditions.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

22 UWB SECURITY [WCC3]

This section addresses the security requirements inherent (applicable and enforced) to the UWB-related functionality of the Digital Key feature. The device OEM shall use NFC as a fallback mechanism in case correct operation of the UWB and the BLE module is disturbed; Information about the fallback mechanism should be provided by the device OEM to the user.

22.1 Cryptography

This section details the various cryptographic functions identified in the various security functions covered earlier in the document.

Build STS based on cryptographic DRBG

Section 20 describes:

- MAC protocol (Ranging session and MAC control Channel)
- STS Index Incrementation (rules, incrementation and update)

22.1.1 *Deriving the STS Sequence from the DRBG*

The derivation of the STS follows the sequence described in Section 15.2.9.1 and Figure 15-7b of [33]. For the purpose of STS generation and in the context of this specification, the following parameters as defined in [33] are mapped to the corresponding parameters as defined in this specification as follows:

phyHrpUwbStsVCounter = SaltedHash[0:32]

phyHrpUwbStsVUpper96 = SaltedHash([64:64])|| ((SaltedHash[32:32]+STS_Index) & 0xffffffff)

phyHrpUwbStsKey = dURSK[0:128]

The 32-bit counter is initialized with phyHrpUwbStsVCounter for each STS generation, that means for each slot. The 32-bit counter is incremented by one for each 128-bit chunk of the STS, that means 31 increments for a 4096-bit long STS.

SaltedHash[0:128], STSIndex[0:32], and Counter[0:32] (the same one as the IEEE 32-bit counter, initialized at 0) are used to derive phyHrpUwbStsVUpper96 and phyHrpUwbStsVCounter.

22.1.2 *SP0 Frames Encryption*

SP0 shall be encrypted using the AES-CCM* algorithm as described in Section 9.3 of [31].

Unlike the NIST specification for CCM which requires encrypted data length > 0, the IEEE 802.15 CCM* implementation allows authentication-only CCM (as used in [31] with Security Level set to 1, 2, or 3).

STS_index encryption (SP0 Pre-POLL)

- *Algorithm*: AES-CCM* as described in Section 9.3 of [31]
- *Mode*: Encrypted and authenticated [31](Security Level 6 - ENC-MIC64)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- *K (Key)* = mUPSK1[0:128]
  - *N (Nonce)* = Source Long Address | Frame Counter | Nonce Security Level as described in Section 9.3.2.1 of [31]
  - Incrementing Frame Counter, STS_Index in payload

Note: The key mUPSK1 is static for the ranging session.

Timestamps encryption (SP0 Final_Data timestamp frame)

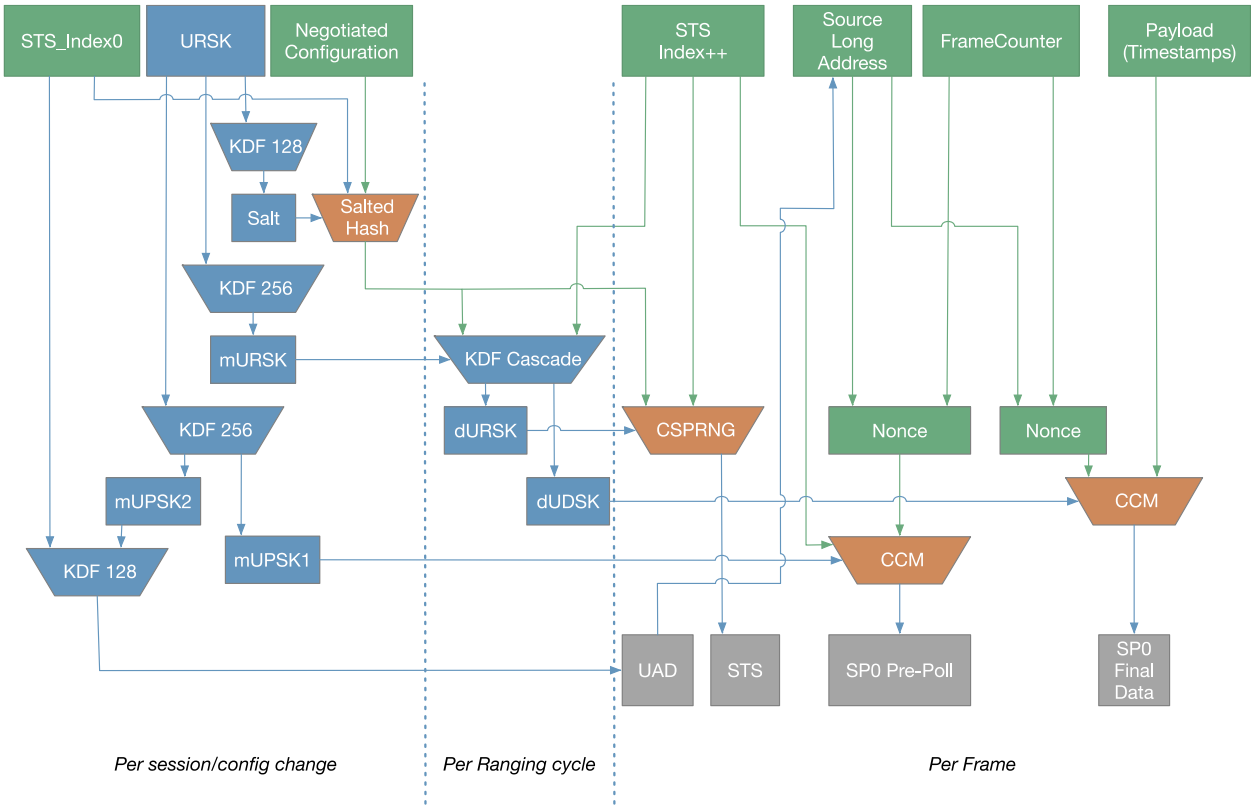
- Algorithm: AES-CCM* per as described in Section 9.3 of [31]
- Mode: Encrypted and authenticated [31](Security Level 6 - ENC-MIC64)
- K (Key) = dUDSK, where dUDSK is defined later in this subsection
- N (Nonce) = Source Address | Frame Counter | Nonce Security Level as described in Section 9.3.2.1 of [31]
- Incrementing Frame Counter

Note: The probability of IV/Key repetition is insignificant since the key dUDSK shall be updated for every ranging block.

22.1.3 *Key Derivation Functions (KDFs)*

The cryptographic foundations of the protocol shall be established with the following cascaded KDFs as shown in Figure 22-1. Information about specific key derivations is in the subsequent sections.

Figure 22-1: Overview of the KDF used and its relationship.



Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

22.1.3.1 mUPSKs Key Derivation

Purpose: Protection of STS_Index

Occurrence: Once per session

Cryptography:

mUPSK1[0:128] = CMAC(URSK, 0x00000001 || **0x5550534B** || 0x00 || 0x000000 || 0x00000180)

mUPSK2[128:128] = CMAC(URSK, 0x00000002 || **0x5550534B** || 0x00 || 0x000000 || 0x00000180)

mUPSK2[0:128] = CMAC(URSK, 0x00000003 || **0x5550534B** || 0x00 || 0x000000 || 0x00000180)

CMAC= AES256-CMAC as PRF, r = 32, h=128

Reference: See Section 5.1 of [4] KDF in Counter Mode

Note: ASCII (**0x5550534B**) = “UPSK”

mUPSK1 is used in order to encrypt the SP0 Pre-POLL.

mUPSK2 is used to derive the KeySource and UWB addresses.

22.1.3.2 SaltedHash Key Derivation

Purpose: Preserve integrity of ranging configuration

Occurrence: Once per ranging configuration negotiation

Cryptography:

Salt[0:128] = CMAC(URSK, 0x00000001 || **0x53414C54** || 0x00 || 0x000000 || 0x00000080)

CMAC= AES256-CMAC as PRF, r = 32, h=128

Reference: See Section 5.1 of [4] KDF in Counter Mode

PaddedSalt[0:256] = 0x000..0000 || Salt

RangingConfiguration = Selected_Protocol_Version || Selected_UWB_Config_Id || UWB_Session_Id || STS_Index0 || Number_Responder_Nodes || Session_RAN_Multiplier || Number_Slot_per_Round || Number_Chaps_per_Slot || Selected_PulseShape_Combo

SaltedHash = CMAC(PaddedSalt, RangingConfiguration)

CMAC = AES256-CMAC as PRF, r = 32, h=128

Note: ASCII (**0x53414C54**) = “SALT”

RangingConfiguration parameters are fully described in Section 20.5.2.

22.1.3.3 mURSK Key Derivation

Purpose: Ranging Keys Seed derivation

Occurrence: Once per session

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Cryptography:

mURSK[128:128]
= CMAC(URSK, 0x00000001 || **0x5552534B** || 0x00 || 0x000000 || 0x00000100)

mURSK[0:128]
= CMAC(URSK, 0x00000002 || **0x5552534B** || 0x00 || 0x000000 || 0x00000100)

CMAC = AES256-CMAC as PRF, r = 32, h=128

Reference: Section 5.1 of [4] KDF in Counter Mode

Note: ASCII (**0x5552534B**) = “URSK”

22.1.3.4 dURSK/dUDSK Keys Derivation (KDF_cascade definition)

Purpose: Ranging Keys derivation

Occurrence: KDF executed once per ranging cycle (before “POLL”)

Cryptography:

URSK_KT[128:128] = CMAC(mURSK, 0x00000001 || **0x5552534B5F4B54** || 0x00 ||
7’b0 || STS_Index[31] || .. 7’b0 || STS_Index[16] || 7’b0 || STS_Index[15] || ..
7’b0 || STS_Index[1] || 7’b0 || STS_Index[0] || 0x00000100)
where 7’b0 represents seven binary zeros and STS_Index = $STS_Index^k(I,Round_Idx^k(i),\text{Pre_POLL})$

URSK_KT[0:128] = CMAC(mURSK, 0x00000002 || **0x5552534B5F4B54** || 0x00 ||
7’b0 || STS_Index[31] || .. 7’b0 || STS_Index[16] || 7’b0 || STS_Index[15] || ..
7’b0 || STS_Index[1] || 7’b0 || STS_Index[0] || 0x00000100)

dURSK[0:128]
= CMAC(URSK_KT, 0x00000001 || **0x5552534B** || 0x00 || SaltedHash || 0x000000 || 0x00000080)

dUDSK[0:128]
= CMAC(URSK_KT, 0x00000001 || **0x5544534B** || 0x00 || SaltedHash || 0x000000 || 0x00000080)

CMAC = AES256-CMAC as PRF, r = 32, h=128

Reference: Section 5.1 of [4] KDF in Counter Mode

Note: ASCII (**0x5552534B5F4B54**) = “URSK_KT”
ASCII (**0x5552534B**) = “URSK”
ASCII (**0x5544534B**) = “UDSK”

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

22.1.4 *UWB Tracking Prevention Between Ranging Recoveries*

In order to protect against tracking of the same UWB session at session setup or upon recovery (i.e., upon receiving the RR-RS message), the following actions shall be performed:,

- The following values shall be derived:
  - KeySource (4 bytes)
  - Destination Short Address (2 bytes)
  - Source Long Address (8 bytes)
- The Frame Counter shall be reset to 0.

22.1.4.1 UWB Addresses KDF

Purpose: Deriving UWB Addresses (UAD)

Occurrence: At ranging setup or at ranging recovery

Cryptography:

UAD[0:128] = CMAC(mUPSK2, 0x00000001 || **0x554144** || 0x00 || STSIndex0[0:32] || 0x00000080)

Some addresses shall be reserved for broadcast use:

- For the KeySourceLow, KeySourceHigh and the destination short address: 0xFFFF and 0xFFFE are reserved values.
- For the source long address: 0xFFFFFFFFFFFFFFFF is a reserved value.

The following method shall be used to flip the upper bit if the calculated UAD matches the reserved value:

```
function SetUpperBitToZeroIfReserved(bytes [X…0]){
   if  (bytesAreReserved(bytes)){
      bytes[X…X-7]  =  bytes[X…X-7]  &  0x7f
    }
    return  bytes
}
```

Note: The indexing notation still refers to bit-indexes.

The SetUpperBitToZeroIfReserved remaps the following entries and leaves the others unmodified:

Reserved Address	Remapped Address
0xFFFF	0x7FFFF
0xFFFE	0x7FFE
0xFFFFFFFFFFFFFFFF	0x7FFFFFFFFFFFFFFF

KeySourceLow[0:16] = SetUpperBitToZeroIfReserved(UAD[112:16])

KeySourceHigh[0:16] = SetUpperBitToZeroIfReserved(UAD[96:16])

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

DestinationShortAddress[0:16] = SetUpperBitToZeroIfReserved(UAD[80:16])
SourceLongAddress[0:64] = SetUpperBitToZeroIfReserved(UAD[16:64])
CMAC = AES256-CMAC as PRF, r = 32, h=128

Reference: Section 5.1 of [4] KDF in Counter Mode

Note: ASCII (0x554144) = "UAD"
The latest STSIndex0 passed is the one negotiated during session setup or recovery.

22.1.4.2 Key Source for Auxiliary Security Header

The 4-byte Key Source value shall be obtained as follows:

KeySource [0:32] = KeySourceHigh || KeySourceLow

22.2 UWB Ranging

The following applies to the UWB ranging session:

22.2.1 *STS Index Management*

The STS Index Incrementation shall comply with the following requirements (see Section 20.6).

1. The initial value of STS_Index selected at the start of a given ranging session (referred here as initial STS_Index0). It shall be a random value in the [0..2^30] range, therefore the STS_Index may be incremented at least 2^30 times. The initial STS_Index0 is included in Ranging Session Setup Response (see Section 19.3.1.6).
2. The STS_Index shall be monotonically increasing. Within a given ranging session identified by UWB_Session_ID, the STS_Index shall never go back be decremented or reused between any two different frames.
3. The maximum value of STS_index in a given ranging session shall be 2^31-1. When this maximum value is reached, the ranging session shall be ended and a new session with a new URSK shall be used.
4. The first STS_Index of a recovered ranging session (referred here as recovery STS_Index0) is included in Ranging Recovery Response (See Section 19.3.1.10) or Configurable Ranging Recovery Response (see Section 19.3.1.12). The recovery STS_Index0 shall be greater than the last used STS_Index prior to Ranging Suspension which is within the same ranging session.
5. The STS_Index value included in the Poll_STS_Index parameter of the first Pre-POLL message (received by the vehicle) shall be larger than the initial STS_Index0 or recovery STS_Index0 for the case of a new ranging session or a recovery of a suspended ranging session, respectively.

Re-synchronization

The re-synchronization of the anchors happens on the vehicle side.

This mechanism involves the mUPSK1, the STS_index, and a nonce (calculated from data in the MAC header): the device performs the encryption of the frame (SP0) and the vehicle decrypts.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

22.3 UWB Module

The pre-derived URSKs are generated by the DK applet. Then, when it becomes active, the URSK or sub keys derived from this URSK (depending on the device architecture) are transferred to the UWB Module.

The UWB Module is notably in charge of the processing of some cryptography operation like PPDU ciphering or derivation of the STS. An implementation shall provide hardware and software protection against logical and certain physical attacks for the full lifecycle of all assets related to the UWB secure ranging.

The transport channel between a SE and the UWB module shall protect confidentiality and integrity. The SE shall provide a secure binding with the UWB module that shall be resistant to manipulation after production. It shall be prevented that any code running on the application processor has access to or can inject data on this channel. The SE shall have the ability to distinguish communication between the UWB chip interface, and other interfaces.

The potential associated with an effective attack on the UWB module and its interface with the SE should be expressed in terms of the total effort required to mount a successful attack as defined by the identified threats. The detailed semantics of such attack potential is to be determined based on definition and related documentation by the DKCTG (e.g. upon conducting a threat analysis exercise).

Appropriate security measures shall be taken to prevent an attacker with expert expertise and bespoke equipment from leveraging a vulnerability to mount a successful exploit on any of the CCC DK Release 3 solution components, that would defeat the security of the use case (passive keyless entry or start engine), given sensitive knowledge of the implemented solution and that a few months time is available for searching vulnerabilities.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

23 REFERENCES

The documents listed in this section are included in requirements made in the body of this specification. Knowledge of their contents is required for the understanding and implementation of this specification. If a listing includes a date or a version identifier, only that specific version of the document is required. If the listing includes neither a date nor a version identifier, the latest version of the document is required. External references referred herein should apply to the latest version provided they are backward compatible with the version at the time of writing and are not violating the implementation specified in this specification.

[1] ISO/IEC 7816-4: Fourth Edition May 2020

[2] ISO 8601-1:2019: Date and time – Representations for information interchange – Part 1: Basic rules

[3] RFC 5280 - PKIX Certificate and CRL Profile - May 2008

[4] NIST SP 800-108 - Recommendation for Key Derivation Using Pseudorandom Functions - May 2005 (Updated August 2022 as NIST SP 800-108r1)

[5] NIST SP 800-38B - Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication – May 2005 (Updated October 2016)

[6] BSI TR-03111 - Elliptic Curve Cryptography - Version 2.0

[7] GPC_SPE_014 - GlobalPlatform - Secure Channel Protocol 03 - Amendment D - Version 1.1.1 or later version

[8] FIPS PUB 186-4 - Digital Signature Standard - July 2013

[9] NIST SP 800-38A - Block Cipher Modes of Operation: Methods and Techniques - 2001

[10] Network Working Group Internet-Draft: SPAKE2+, an Augmented PAKE, draft-bar-cfrg-spake2plus-00, Mar 9, 2020

[11] Internet Engineering Task Force (IETF): The Scrypt Password-Based Key Derivation Function (RFC 7914), August 2016

[12] Internet Engineering Task Force (IETF): Randomness Requirements for Security (RFC 4086), June 2005

[13] RFC 5869 - HMAC-based Extract-and-Expand Key Derivation Function - May 2010

[14] RFC 5480 - ECC SubjectPublicKeyInfo Format - October 2009

[15] GlobalPlatform Card Technology – Confidential Card Content Management, Card Specification v2.3 Amendment A Version 1.2 (or higher)

[16] NFC Analog Technical Specification 2.1 (https://nfc-forum.org/our-work/specifications-and-application-documents/) or later version.

[17] NFC Digital Protocol Technical Specification 2.1 (https://nfc-forum.org/our-work/specifications-and-application-documents/) or later version.

[18] NFC Activity Technical Specification 2.0 (https://nfc-forum.org/our-work/specifications-and-application-documents/) or later version.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

[19] NFC Controller Interface Technical Specification 2.1 (https://nfc-forum.org/our-work/specifications-and-application-documents/) or later version.
[20] GlobalPlatform Card Technology Card Specification v2.2.1 (or higher)
[21] GlobalPlatform Card – Contactless Extension Version 1.0 (or higher)
[22] SEC 1: Elliptical Curve Cryptography Version 2.0, https://www.secg.org/sec1-v2.pdf
[23] ISO/IEC 10118-3 Hashfunctions - Part 3: Dedicated hash-functions
[24] FIPS PUB 180-4: Specifications for the Secure Hash Standard – 2015
[25] ETSI TS 102 705 Release 9.2.0 (or higher) Smart Cards; UICC Application Programming Interface for Java Card™ for Contactless Applications, https://www.etsi.org/standards
[26] ETSI TS 102 622 Release 9.4.0 (or higher) Smart Cards; Contactless Front End, HCI (Host Controller Interface), https://www.etsi.org/standards
[27] RFC 3986 – Uniform Resource Identifier (URI): Generic Syntax – January 2005
[28] GPC_SPE_093 - GlobalPlatform - Secure Channel Protocol '11' - Amendment F - Version 1.1
[29] Car Connectivity Consortium "Android Digital Key Framework API", Version 1.0, Aug 2020
[30] The Bluetooth Core specification, version 5.0, December 2016
[31] IEEE Standard for Low-Rate Wireless Networks, 802.15.4-2020, July 2020
[32] CCC Digital Key Release 2 Specification version 1.1
[33] IEEE 802.15-4z-2020 - IEEE Standard for Low-Rate Wireless Networks, Amendment 1: Enhanced Ultra Wideband (UWB) Physical Layers (PHYs) and Associated Ranging Techniques, June 2020
[34] N. Damera-Venkata, B. L. Evans, S. R. McCaslin, “Design of optimal minimal phase filter using discrete Hilbert transform,” IEEE Trans. Signal Proc., vol. 48, pp. 1491-1495, May 2000
[35] CCC Vehicle OEM Document, D1.1
[36] Description 15.4z HRP UWB PHY Test Vectors”, IEEE 802.15 document number 15-20-0003-02-004z, 2020.
[37] RFC 2330 - Framework for IP Performance Metrics, https://tools.ietf.org/html/rfc2330#section-10
[38] Internet Engineering Task Force (IETF): Secure Credential Transfer - draft-secure-credential-transfer-03, https://datatracker.ietf.org/doc/draft-secure-credential-transfer/
[39] NIST Special Publication 800-63B, Digital Identity Guidelines, June 2017 including updates of 03-02-2020
[40] ITU Telecommunication Standardization Sector - Recommendation ITU-T X.690 International Standard 8825-1

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Appendix A.

A.1. Standard Transaction Cryptography Flow

Listing A-1: Standard Transaction Cryptography Flow

```
Vehicle key pair:
vehicle public key X: F47EB42A771052580C086EFDAAA3084AA3FF7A67CE23393A0373C63487DF1A63
   vehicle public key Y: 7D1FB34B2D2E7D5C8F92097A0619B5C5CC6C5850AF74C019EBBEC4273358AA94
vehicle private key: 86B9E3843D949890FD50E49C8542DB575BAC41D344F17588DDAFE4535521CE55
Endpoint long term key pair:
   endpoint public key X: 07B857B9B7F1147E20F4DBE6723CE5F46EF8670CBA20F56297F515C8265E4E42
   endpoint public key Y: 5F1FC9B5DAFB62DAAFB5DC9AA6F8B2EDC1CDD43E20A614EF2F8703FA1459721C
   endpoint private key: 13849B3560961053D985DA5E0FA2AF05EAA99DD73EC50CC4A87029A24D7C331B

>>00A4040005AAAAAAAAAA00

endpoint: received SELECT command

<<5C0201009000

Generate ephemeral keys:
   ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
   ephemeral public key Y: CAD19D201062DD1C7CB0BB293BF16A4BEFB2ED500977E7197E01F26906E39B5F
   ephemeral private key: B0FBA5FB966FDD3BE4096FA65307AB0A7A3BB914625BBFD3CB57DAD9183E19CB

>>80800000635C020100874104F98CCA31651AD2E63266144B2450FD6081D8FEA8
CEB826E1FB10E8034E932446CAD19D201062DD1C7CB0BB293BF16A4BEFB2ED50
0977E7197E01F26906E39B5F4C10BF1C41268230AF76BFFE3E7C5D00CF4A4D08
888888888888888800

endpoint: received AUTH0 command
Generate ephemeral keys:
   ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
   ephemeral public key Y: 3F95D4C06AB8966D2B9A0D3C4BC446DB9343EBF27F9EF811F242A37118AD4F10
ephemeral private key: E585C9EE89075F795452879AC38261ED0667C6396A34914DEE0681E8DC22A182
 Key Derivation:
 K: ACEDD14246C16AAF4561E177E567192454C06B2AAEEDB7E278980C25DD7994A2
 endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 reader ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 interface byte: 5E
 flag: 0000
 HKDF output: 55AC05B56D3081AAEF7BD8EB53F9B5BAAE126251B6224775A72770B3E10BC65D
F663103140A7832C8AD155E7377E63303DE2D8811B60F1225CAD4FCBDE0BAA3C
 Kcmac: 55AC05B56D3081AAEF7BD8EB53F9B5BA
 Kenc: AE126251B6224775A72770B3E10BC65D
 Kmac: F663103140A7832C8AD155E7377E6330
 Krmac: 3DE2D8811B60F1225CAD4FCBDE0BAA3C

<<86410443D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337B
B17F203F95D4C06AB8966D2B9A0D3C4BC446DB9343EBF27F9EF811F242A37118
AD4F109000

generate authentication hash:
 sha256 hash input: 4D088888888888888888862043D605526999F032E08F314F22EBCE051D1DAE53
DC71F1C4D614B0337BB17F208720F98CCA31651AD2E63266144B2450FD6081D8
```

```
FEA8CEB826E1FB10E8034E9324464C10BF1C41268230AF76BFFE3E7C5D00CF4A
9304415D9569
 sha256 hash output: 9A2A933D4B90F5A9CFB0B5524E36B10D3669B91F2526F6C0FC2369BD98A327A0

>>80810000429E40CCE7447AC8D0112C24AE4A261AF63EBA7B585126FFA4CE4C06
1D11D97B98151CB7D85BDCCA539D152B544B97647DD5CD38DCBDBD82EF93F5B5
796FFF3C2C0FD700

endpoint: received AUTH1 command
generate authentication hash:
 sha256 hash input: 4D088888888888888888862043D605526999F032E08F314F22EBCE051D1DAE53
DC71F1C4D614B0337BB17F208720F98CCA31651AD2E63266144B2450FD6081D8
FEA8CEB826E1FB10E8034E9324464C10BF1C41268230AF76BFFE3E7C5D00CF4A
9304415D9569
 sha256 hash output: 9A2A933D4B90F5A9CFB0B5524E36B10D3669B91F2526F6C0FC2369BD98A327A0
endpoint: successful signature verification
Derive Kdh:
 ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
 ephemeral public key Y: CAD19D201062DD1C7CB0BB293BF16A4BEFB2ED500977E7197E01F26906E39B5F
 ephemeral private key: E585C9EE89075F795452879AC38261ED0667C6396A34914DEE0681E8DC22A182
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 ecdh shared secret: 5A2E155294C7D0074DAE3E133D20A6DA60F0A15A02A3A909268BC15D160323FA
 sha256 input: 5A2E155294C7D0074DAE3E133D20A6DA60F0A15A02A3A909268BC15D160323FA
00000001BF1C41268230AF76BFFE3E7C5D00CF4A
 derived kdh: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 Key Derivation:
 K: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 reader ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 interface byte: 5E
 flag: 0000
 HKDF output: 65B3C36092CC8B15878DC90E0C3A475D4DC72A2325377760B9B1E1774CBE7ED8
46BD16584973BEE37BA5732F3628411B
CAC033EE373EBFE533DB574C5C8477A7
 Kenc: 65B3C36092CC8B15878DC90E0C3A475D
 Kmac: 4DC72A2325377760B9B1E1774CBE7ED8
 Krmac: 46BD16584973BEE37BA5732F3628411B
 Key Derivation:
 K: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 reader ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 interface byte: 5E
 flag: 0000
 HKDF output: 0C0E989932DDE515E6D8409A4628DE5650D43135413724FD097EDFC3332CF0AC
 Kpersistent: 0C0E989932DDE515E6D8409A4628DE5650D43135413724FD097EDFC3332CF0AC
generate authentication hash:
 sha256 hash input: 4D088888888888888888862043D605526999F032E08F314F22EBCE051D1DAE53
DC71F1C4D614B0337BB17F208720F98CCA31651AD2E63266144B2450FD6081D8
FEA8CEB826E1FB10E8034E9324464C10BF1C41268230AF76BFFE3E7C5D00CF4A
93044E887B4C
 sha256 hash output: 48BCCE4843E4E87A01AEC830A1AAF6E7D1380D950C468F81BB5AD4CF40705040
 appending confidential mailbox content: 00000000000000000000000000000000
 appending private mailbox content: 00000000000000000000000000000000
wrap response:
 plaintext:4E04001122339E407D2E38CB7A01825EF105E5B53166BE8E85D24243A1A5D276
7737297E733AD2BA3FECD02D38128B2D42794BC13C03028D58DED27B298115D5
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
82BE41FC38D38F424A1000000000000000000000000000004B1000000000
00000000000000000000000
 ICV:4AEA75139218A8AEF7B3BC0770A60E0F
 cleartext payload:4E04001122339E407D2E38CB7A01825EF105E5B53166BE8E85D24243A1A5D276
7737297E733AD2BA3FECD02D38128B2D42794BC13C03028D58DED27B298115D5
82BE41FC38D38F424A1000000000000000000000000000004B1000000000
00000000000000000000000080000000
 MAC chaining value:00000000000000000000000000000000
 cipher|mac:605320BA70CC086F962A7D7F015815A88AC76A192C57B50B5070DAFC2FD4CD59
11ACC2D748620622BAE23AAAC8E6D88A2C0A4FFF5D2DFCEE88F2190EA861E709
053D72EA7C9FFEACEEDC0471EEFE6CF5624E5505374FE40BF8A63FF7DDD069B8
A45565897E697EABCCB2FA1C5CB4AAC733A5C18FA30112F1

<<605320BA70CC086F962A7D7F015815A88AC76A192C57B50B5070DAFC2FD4CD59
11ACC2D748620622BAE23AAAC8E6D88A2C0A4FFF5D2DFCEE88F2190EA861E709
053D72EA7C9FFEACEEDC0471EEFE6CF5624E5505374FE40BF8A63FF7DDD069B8
A45565897E697EABCCB2FA1C5CB4AAC733A5C18FA30112F19000

Derive Kdh:
 ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 ephemeral public key Y: 3F95D4C06AB8966D2B9A0D3C4BC446DB9343EBF27F9EF811F242A37118AD4F10
 ephemeral private key: B0FBA5FB966FDD3BE4096FA65307AB0A7A3BB914625BBFD3CB57DAD9183E19CB
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 ecdh shared secret: 5A2E155294C7D0074DAE3E133D20A6DA60F0A15A02A3A909268BC15D160323FA
 sha256 input: 5A2E155294C7D0074DAE3E133D20A6DA60F0A15A02A3A909268BC15D160323FA
00000001BF1C41268230AF76BFFE3E7C5D00CF4A
 derived kdh: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 Key Derivation:
 K: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 reader ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 interface byte: 5E
 flag: 0000
 HKDF output: 65B3C36092CC8B15878DC90E0C3A475D4DC72A2325377760B9B1E1774CBE7ED8
46BD16584973BEE37BA5732F3628411B
CAC033EE373EBFE533DB574C5C8477A7
 Kenc: 65B3C36092CC8B15878DC90E0C3A475D
 Kmac: 4DC72A2325377760B9B1E1774CBE7ED8
 Krmac: 46BD16584973BEE37BA5732F3628411B
 Key Derivation:
 K: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 reader ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 interface byte: 5E
 flag: 0000
 HKDF output: 0C0E989932DDE515E6D8409A4628DE5650D43135413724FD097EDFC3332CF0AC
 Kpersistent: 0C0E989932DDE515E6D8409A4628DE5650D43135413724FD097EDFC3332CF0AC
unwrap response:
 cipher|mac:605320BA70CC086F962A7D7F015815A88AC76A192C57B50B5070DAFC2FD4CD59
11ACC2D748620622BAE23AAAC8E6D88A2C0A4FFF5D2DFCEE88F2190EA861E709
053D72EA7C9FFEACEEDC0471EEFE6CF5624E5505374FE40BF8A63FF7DDD069B8
A45565897E697EABCCB2FA1C5CB4AAC733A5C18FA30112F1
 plaintext:4E04001122339E407D2E38CB7A01825EF105E5B53166BE8E85D24243A1A5D276
7737297E733AD2BA3FECD02D38128B2D42794BC13C03028D58DED27B298115D5
82BE41FC38D38F424A1000000000000000000000000000004B1000000000
00000000000000000000000
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
reader : mac verification successful
generate authentication hash:
 sha256 hash input: 4D088888888888888888862043D605526999F032E08F314F22EBCE051D1DAE53
DC71F1C4D614B0337BB17F208720F98CCA31651AD2E63266144B2450FD6081D8
FEA8CEB826E1FB10E8034E9324464C10BF1C41268230AF76BFFE3E7C5D00CF4A
93044E887B4C
 sha256 hash output: 48BCCE4843E4E87A01AEC830A1AAF6E7D1380D950C468F81BB5AD4CF40705040
reader: endpoint signature verification successful
request to read private mailbox offset:0 length:5
request to write private mailbox offset:0 length:5 data:FFEEEEDDBB
request to read confidential mailbox offset:0 length:5
request to write confidential mailbox offset:0 length:5 data:AAEEEE33CC
wrap command:
 plaintext:0088030000058A070000FFEEEEDDBB89030000058B070000AAEEEE33CC
 MAC chaining value:3B30EA2AB387B0FEB5D9D899B944E17B
 cipher|mac:F094F8445A84E178484E167B1FD08DBB2C30C61EE0CA41FCE4F6B6A139740988
3B30EA2AB387B0FE

>>84C9000028F094F8445A84E178484E167B1FD08DBB2C30C61EE0CA41FCE4F6B6
A1397409883B30EA2AB387B0FE00

endpoint: received EXCHANGE command
unwrap command:
 cipher|mac:F094F8445A84E178484E167B1FD08DBB2C30C61EE0CA41FCE4F6B6A139740988
3B30EA2AB387B0FE
 MAC chaining value:00000000000000000000000000000000

 plaintext:0088030000058A070000FFEEEEDDBB89030000058B070000AAEEEE33CC
wrap response:
 plaintext:05AAAAAAAAAA05BBBBBBBBBB
 ICV:925FE52E4E2793662479418FE2C8A3C0
 cleartext payload:05AAAAAAAAAA05BBBBBBBBBB80000000
 MAC chaining value:3B30EA2AB387B0FEB5D9D899B944E17B
 cipher|mac:73DFAE8DF93751D1169C4295565220F2B10F64BA83956435

<<73DFAE8DF93751D1169C4295565220F2B10F64BA839564359000

unwrap response:
 cipher|mac:73DFAE8DF93751D1169C4295565220F2B10F64BA83956435
 plaintext:05AAAAAAAAAA05BBBBBBBBBB
```

A.2. Fast Transaction Cryptography Flow

Listing A-2: Fast Transaction Cryptography Flow

```
 Vehicle key pair:
  vehicle public key X: F47EB42A771052580C086EFDAAA3084AA3FF7A67CE23393A0373C63487DF1A63
  vehicle public key Y: 7D1FB34B2D2E7D5C8F92097A0619B5C5CC6C5850AF74C019EBBEC4273358AA94
  vehicle private key: 86B9E3843D949890FD50E49C8542DB575BAC41D344F17588DDAFE4535521CE55
 Endpoint long term key pair:
  endpoint public key X: 07B857B9B7F1147E20F4DBE6723CE5F46EF8670CBA20F56297F515C8265E4E42
  endpoint public key Y: 5F1FC9B5DAFB62DAAFB5DC9AA6F8B2EDC1CDD43E20A614EF2F8703FA1459721C
  endpoint private key: 13849B3560961053D985DA5E0FA2AF05EAA99DD73EC50CC4A87029A24D7C331B

  >>00A4040005AAAAAAAAAA00

  endpoint: received SELECT command
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
13
14    <<5C0201009000
15
16    Generate ephemeral keys:
17     ephemeral public key X: 82BF9E948ECBDD73C10C7EB7D34D5BEB31CF2908B09ADAC701CB4B1F116F5467
18     ephemeral public key Y: C9187749054455AA1231FA6562D6D4198779FEC2F4F36DB8D9D6EF2082EEA75B
19     ephemeral private key: E82CED017293885DC5D157A6DAC87013A72B3182F94939BFAA92BB9A367E7966
20
21    >>80800100635C02010087410482BF9E948ECBDD73C10C7EB7D34D5BEB31CF2908
22    B09ADAC701CB4B1F116F5467C9187749054455AA1231FA6562D6D4198779FEC2
23    F4F36DB8D9D6EF2082EEA75B4C10F92F7260B588238C1E2A4825AD4D7D2E4D08
24    888888888888888800
25
26    endpoint: received AUTH0 command
27    Generate ephemeral keys:
28     ephemeral public key X: 0EA56A82A1AD7FC2C739FBB793C0BC3B8935C2ED46B672EFCB98F7DF124FA7FF
29     ephemeral public key Y: A4155A91F0FCBB007C61E6C574F0F87D3CAF1F41EDE0DF87F43DB664B2C81540
30     ephemeral private key: BCD0A7ECE3BD6A27A2E0597DAA647028E9058D415921A282C0B18DE90E6EFA94
31     Key Derivation:
32     K: B1E9126FBB4FFCA027AE116FC242A1F93093082DE8661B3CD1942078DEB384FD
33     endpoint ephemeral public key X:
     0EA56A82A1AD7FC2C739FBB793C0BC3B8935C2ED46B672EFCB98F7DF124FA7FF
34     vehicle ephemeral public key X: 82BF9E948ECBDD73C10C7EB7D34D5BEB31CF2908B09ADAC701CB4B1F116F5467
35     transaction identifier: F92F7260B588238C1E2A4825AD4D7D2E
36     interface byte: 5E
37     flag: 0100
38     HKDF output: 960BA7366865139D9EFAEFB53B8CB18713736C529481B67615811D4EA9F49650
39    AAA090CC271484E68BB5B6EE18655AAFA5B82D564DCD9961DEE2830C6550E1C1
40     Kcmac: 960BA7366865139D9EFAEFB53B8CB187
41     Kenc: 13736C529481B67615811D4EA9F49650
42     Kmac: AAA090CC271484E68BB5B6EE18655AAF
43     Krmac: A5B82D564DCD9961DEE2830C6550E1C1
44    generate cryptogram:
45     Kcmac: 960BA7366865139D9EFAEFB53B8CB187
46     endpoint public key X: 07B857B9B7F1147E20F4DBE6723CE5F46EF8670CBA20F56297F515C8265E4E42
47     vehicle public key X: F47EB42A771052580C086EFDAAA3084AA3FF7A67CE23393A0373C63487DF1A63
48     vehicle identifier: 8888888888888888
49     transaction identifier: F92F7260B588238C1E2A4825AD4D7D2E
50     input of AES-CMAC: 00000000000000000000000003200008001F47EB42A771052580C086EFDAAA3084A
51    A3FF7A67CE23393A0373C63487DF1A6307B857B9B7F1147E20F4DBE6723CE5F4
52    6EF8670CBA20F56297F515C8265E4E42F92F7260B588238C1E2A4825AD4D7D2E
53    8888888888888888
54     cryptogram: E5B79C3D703D1BE1B26C2A999DB2975B
55
56    <<8641040EA56A82A1AD7FC2C739FBB793C0BC3B8935C2ED46B672EFCB98F7DF12
57    4FA7FFA4155A91F0FCBB007C61E6C574F0F87D3CAF1F41EDE0DF87F43DB664B2
58    C815409D10E5B79C3D703D1BE1B26C2A999DB2975B9000
59
60     Key Derivation:
61     K: B1E9126FBB4FFCA027AE116FC242A1F93093082DE8661B3CD1942078DEB384FD
       endpoint ephemeral public key X:
62   0EA56A82A1AD7FC2C739FBB793C0BC3B8935C2ED46B672EFCB98F7DF124FA7FF
63     vehicle ephemeral public key X: 82BF9E948ECBDD73C10C7EB7D34D5BEB31CF2908B09ADAC701CB4B1F116F5467
64     transaction identifier: F92F7260B588238C1E2A4825AD4D7D2E
65     interface byte: 5E
66     flag: 0100
67     HKDF output: 960BA7366865139D9EFAEFB53B8CB18713736C529481B67615811D4EA9F49650
68    AAA090CC271484E68BB5B6EE18655AAFA5B82D564DCD9961DEE2830C6550E1C1
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
 Kcmac: 960BA7366865139D9EFAEFB53B8CB187
 Kenc: 13736C529481B67615811D4EA9F49650
 Kmac: AAA090CC271484E68BB5B6EE18655AAF
 Krmac: A5B82D564DCD9961DEE2830C6550E1C1
generate cryptogram:
 Kcmac: 960BA7366865139D9EFAEFB53B8CB187
 endpoint public key X: 07B857B9B7F1147E20F4DBE6723CE5F46EF8670CBA20F56297F515C8265E4E42
 vehicle public key X: F47EB42A771052580C086EFDAAA3084AA3FF7A67CE23393A0373C63487DF1A63
 vehicle identifier: 8888888888888888
 transaction identifier: F92F7260B588238C1E2A4825AD4D7D2E
 input of AES-CMAC: 00000000000000000000003200008001F47EB42A771052580C086EFDAAA3084A
A3FF7A67CE23393A0373C63487DF1A6307B857B9B7F1147E20F4DBE6723CE5F4
6EF8670CBA20F56297F515C8265E4E42F92F7260B588238C1E2A4825AD4D7D2E
8888888888888888
 cryptogram: E5B79C3D703D1BE1B26C2A999DB2975B
vehicle: matching cryptogram found
```

A.3. Standard Transaction Cryptography Flow (with Reader Intent for Fast Transaction)

Listing A-3: Standard Transaction Cryptography Flow (with Reader Intent for Fast Transaction)

```
Vehicle key pair:
 vehicle public key X: F47EB42A771052580C086EFDAAA3084AA3FF7A67CE23393A0373C63487DF1A63
 vehicle public key Y: 7D1FB34B2D2E7D5C8F92097A0619B5C5CC6C5850AF74C019EBBEC4273358AA94
 vehicle private key: 86B9E3843D949890FD50E49C8542DB575BAC41D344F17588DDAFE4535521CE55
Endpoint long term key pair:
 endpoint public key X: 07B857B9B7F1147E20F4DBE6723CE5F46EF8670CBA20F56297F515C8265E4E42
 endpoint public key Y: 5F1FC9B5DAFB62DAAFB5DC9AA6F8B2EDC1CDD43E20A614EF2F8703FA1459721C
 endpoint private key: 13849B3560961053D985DA5E0FA2AF05EAA99DD73EC50CC4A87029A24D7C331B

 >>00A4040005AAAAAAAAAA00

 endpoint: received SELECT command

 <<5C0201009000

 Generate ephemeral keys:
  ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
  ephemeral public key Y: CAD19D201062DD1C7CB0BB293BF16A4BEFB2ED500977E7197E01F26906E39B5F
  ephemeral private key: B0FBA5FB966FDD3BE4096FA65307AB0A7A3BB914625BBFD3CB57DAD9183E19CB

 >>80800100635C020100874104F98CCA31651AD2E63266144B2450FD6081D8FEA8
 CEB826E1FB10E8034E932446CAD19D201062DD1C7CB0BB293BF16A4BEFB2ED50
 0977E7197E01F26906E39B5F4C10BF1C41268230AF76BFFE3E7C5D00CF4A4D08
 888888888888888800

 endpoint: received AUTH0 command
 Generate ephemeral keys:
  ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
  ephemeral public key Y: 3F95D4C06AB8966D2B9A0D3C4BC446DB9343EBF27F9EF811F242A37118AD4F10
  ephemeral private key: E585C9EE89075F795452879AC38261ED0667C6396A34914DEE0681E8DC22A182
  Key Derivation:
  K: ACEDD14246C16AAF4561E177E567192454C06B2AAEEDB7E278980C25DD7994A2
  endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
  vehicle ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 interface byte: 5E
 flag: 0100
 HKDF output: C23F571D20E08CCC6AA08367D7800DC4A20BB6F231B3EE2F007E8B952C4B95B0
B0C367A2867DD0530C560C5FD167D18E017C2FA77D4C653C4794529B3F69B529
 Kcmac: C23F571D20E08CCC6AA08367D7800DC4
 Kenc: A20BB6F231B3EE2F007E8B952C4B95B0
 Kmac: B0C367A2867DD0530C560C5FD167D18E
 Krmac: 017C2FA77D4C653C4794529B3F69B529
generate cryptogram:
 Kcmac: C23F571D20E08CCC6AA08367D7800DC4
 endpoint public key X: 07B857B9B7F1147E20F4DBE6723CE5F46EF8670CBA20F56297F515C8265E4E42
 vehicle public key X: F47EB42A771052580C086EFDAAA3084AA3FF7A67CE23393A0373C63487DF1A63
 vehicle identifier: 8888888888888888
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 input of AES-CMAC: 00000000000000000000003200008001F47EB42A771052580C086EFDAAA3084A
A3FF7A67CE23393A0373C63487DF1A6307B857B9B7F1147E20F4DBE6723CE5F4
6EF8670CBA20F56297F515C8265E4E42BF1C41268230AF76BFFE3E7C5D00CF4A
8888888888888888
 cryptogram: BD75825ECE29A6B84ADA79D9BF719839

<<86410443D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337B
B17F203F95D4C06AB8966D2B9A0D3C4BC446DB9343EBF27F9EF811F242A37118
AD4F109D10BD75825ECE29A6B84ADA79D9BF7198399000

vehicle: no matching cryptogram found
generate authentication hash:
 sha256 hash input: 4D088888888888888888862043D605526999F032E08F314F22EBCE051D1DAE53
DC71F1C4D614B0337BB17F208720F98CCA31651AD2E63266144B2450FD6081D8
FEA8CEB826E1FB10E8034E9324464C10BF1C41268230AF76BFFE3E7C5D00CF4A
9304415D9569
 sha256 hash output: 9A2A933D4B90F5A9CFB0B5524E36B10D3669B91F2526F6C0FC2369BD98A327A0

>>80810000429E40CCE7447AC8D0112C24AE4A261AF63EBA7B585126FFA4CE4C06
1D11D97B98151CB7D85BDCCA539D152B544B97647DD5CD38DCBDBD82EF93F5B5
796FFF3C2C0FD700

endpoint: received AUTH1 command
generate authentication hash:
 sha256 hash input: 4D088888888888888888862043D605526999F032E08F314F22EBCE051D1DAE53
DC71F1C4D614B0337BB17F208720F98CCA31651AD2E63266144B2450FD6081D8
FEA8CEB826E1FB10E8034E9324464C10BF1C41268230AF76BFFE3E7C5D00CF4A
9304415D9569
 sha256 hash output: 9A2A933D4B90F5A9CFB0B5524E36B10D3669B91F2526F6C0FC2369BD98A327A0
endpoint: successful signature verification
Derive Kdh:
 ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
 ephemeral public key Y: CAD19D201062DD1C7CB0BB293BF16A4BEFB2ED500977E7197E01F26906E39B5F
 ephemeral private key: E585C9EE89075F795452879AC38261ED0667C6396A34914DEE0681E8DC22A182
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 ecdh shared secret: 5A2E155294C7D0074DAE3E133D20A6DA60F0A15A02A3A909268BC15D160323FA
 sha256 input: 5A2E155294C7D0074DAE3E133D20A6DA60F0A15A02A3A909268BC15D160323FA
00000001BF1C41268230AF76BFFE3E7C5D00CF4A
 derived Kdh: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 Key Derivation:
 Kdh: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 vehicle ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 interface byte: 5E
 flag: 0100
 HKDF output: 3606D26C0DEF1E40FC3AF389B6D473CEE1EA7AFECA71183B78173BA9D9425892
BB8B5856F73AC0976DA0E80B3AC38DED

 Kenc: 3606D26C0DEF1E40FC3AF389B6D473CE
 Kmac: E1EA7AFECA71183B78173BA9D9425892
 Krmac: BB8B5856F73AC0976DA0E80B3AC38DED

 Key Derivation:
 K: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 vehicle ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 interface byte: 5E
 flag: 0100
 HKDF output: B1E9126FBB4FFCA027AE116FC242A1F93093082DE8661B3CD1942078DEB384FD
 Kpersistent: B1E9126FBB4FFCA027AE116FC242A1F93093082DE8661B3CD1942078DEB384FD
generate authentication hash:
 sha256 hash input: 4D088888888888888888862043D605526999F032E08F314F22EBCE051D1DAE53
DC71F1C4D614B0337BB17F208720F98CCA31651AD2E63266144B2450FD6081D8
FEA8CEB826E1FB10E8034E9324464C10BF1C41268230AF76BFFE3E7C5D00CF4A
93044E887B4C
 sha256 hash output: 48BCCE4843E4E87A01AEC830A1AAF6E7D1380D950C468F81BB5AD4CF40705040
 appending confidential mailbox content: 00000000000000000000000000000000
 appending private mailbox content: 00000000000000000000000000000000
wrap response:
 plaintext:4E04001122339E407D2E38CB7A01825EF105E5B53166BE8E85D24243A1A5D276
7737297E733AD2BA3FECD02D38128B2D42794BC13C03028D58DED27B298115D5
82BE41FC38D38F424A1000000000000000000000000000004B1000000000
00000000000000000000
 ICV:C9A1CB185C77FC6CC5D176A4CFCE161A
 cleartext payload:4E04001122339E407D2E38CB7A01825EF105E5B53166BE8E85D24243A1A5D276
7737297E733AD2BA3FECD02D38128B2D42794BC13C03028D58DED27B298115D5
82BE41FC38D38F424A1000000000000000000000000000004B1000000000
00000000000000000000000080000000
 MAC chaining value:00000000000000000000000000000000
 cipher|mac: B3C5750D412C06971F28D5DF7DC977F8056637B03E70711B71CA96DC789495A2
94CE076350FB5DF2AC6CF3E9DA751B072E2C9368EFD476F6506B9F6B61FAC353
5534BCF84150D2FCDD1CC465EB07504FAFC9B65E04DDD3646FD2B576EF1AC3B1
451EEAC3E83DBB610E8031142AF4A783508C1D65EF0854A4

<<B3C5750D412C06971F28D5DF7DC977F8056637B03E70711B71CA96DC789495A2
94CE076350FB5DF2AC6CF3E9DA751B072E2C9368EFD476F6506B9F6B61FAC353
5534BCF84150D2FCDD1CC465EB07504FAFC9B65E04DDD3646FD2B576EF1AC3B1
451EEAC3E83DBB610E8031142AF4A783508C1D65EF0854A49000

Derive Kdh:
 ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
 ephemeral public key Y: 3F95D4C06AB8966D2B9A0D3C4BC446DB9343EBF27F9EF811F242A37118AD4F10
 ephemeral private key: B0FBA5FB966FDD3BE4096FA65307AB0A7A3BB914625BBFD3CB57DAD9183E19CB
 transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
 ecdh shared secret: 5A2E155294C7D0074DAE3E133D20A6DA60F0A15A02A3A909268BC15D160323FA
 sha256 input: 5A2E155294C7D0074DAE3E133D20A6DA60F0A15A02A3A909268BC15D160323FA
00000001BF1C41268230AF76BFFE3E7C5D00CF4A
 derived Kdh: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
 Key Derivation:
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
  Kdh: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
  endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
  vehicle ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
  transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
  interface byte: 5E
  flag: 0100
  HKDF output: 3606D26C0DEF1E40FC3AF389B6D473CEE1EA7AFECA71183B78173BA9D9425892
 BB8B5856F73AC0976DA0E80B3AC38DED

  Kenc: 3606D26C0DEF1E40FC3AF389B6D473CE
  Kmac: E1EA7AFECA71183B78173BA9D9425892
  Krmac: BB8B5856F73AC0976DA0E80B3AC38DED

  Key Derivation:
  K: 18B0CDC20B916B22D2E5D87FDA544D7BD809D171DA00E103A72DF0FF5E5CD185
  endpoint ephemeral public key X: 43D605526999F032E08F314F22EBCE051D1DAE53DC71F1C4D614B0337BB17F20
  vehicle ephemeral public key X: F98CCA31651AD2E63266144B2450FD6081D8FEA8CEB826E1FB10E8034E932446
  transaction identifier: BF1C41268230AF76BFFE3E7C5D00CF4A
  interface byte: 5E
  flag: 0100
  HKDF output: B1E9126FBB4FFCA027AE116FC242A1F93093082DE8661B3CD1942078DEB384FD
  Kpersistent: B1E9126FBB4FFCA027AE116FC242A1F93093082DE8661B3CD1942078DEB384FD
 unwrap response:
  cipher|mac: B3C5750D412C06971F28D5DF7DC977F8056637B03E70711B71CA96DC789495A2
 94CE076350FB5DF2AC6CF3E9DA751B072E2C9368EFD476F6506B9F6B61FAC353
 5534BCF84150D2FCDD1CC465EB07504FAFC9B65E04DDD3646FD2B576EF1AC3B1
 451EEAC3E83DBB610E8031142AF4A783508C1D65EF0854A4
  plaintext:4E04001122339E407D2E38CB7A01825EF105E5B53166BE8E85D24243A1A5D276
 7737297E733AD2BA3FECD02D38128B2D42794BC13C03028D58DED27B298115D5
 82BE41FC38D38F424A1000000000000000000000000000004B1000000000
 00000000000000000000
 vehicle: mac verification successful
 generate authentication hash:
  sha256 hash input: 4D088888888888888888862043D605526999F032E08F314F22EBCE051D1DAE53
 DC71F1C4D614B0337BB17F208720F98CCA31651AD2E63266144B2450FD6081D8
 FEA8CEB826E1FB10E8034E9324464C10BF1C41268230AF76BFFE3E7C5D00CF4A
 93044E887B4C
  sha256 hash output: 48BCCE4843E4E87A01AEC830A1AAF6E7D1380D950C468F81BB5AD4CF40705040
 vehicle: endpoint signature verification successful
 request to read private mailbox offset:0 length:5
 request to write private mailbox offset:0 length:5 data: FFEEEEDDBB
 request to read confidential mailbox offset:0 length:5
 request to write confidential mailbox offset:0 length:5 data: AAEEEE33CC
wrap command:
 plaintext:0088030000058A070000FFEEEEDDBB89030000058B070000AAEEEE33CC
 MAC chaining value:8B6EC24F1A0591AAF0C1AB48C49029B5
 cipher|mac:2D58D4699A72329D5E774CAAC4E60CF8AFA912B887053BC748F3CDA106DC9B67
8B6EC24F1A0591AA

>>84C90000282D58D4699A72329D5E774CAAC4E60CF8AFA912B887053BC748F3CD
A106DC9B678B6EC24F1A0591AA00

endpoint: received EXCHANGE command
unwrap command:
 cipher|mac:2D58D4699A72329D5E774CAAC4E60CF8AFA912B887053BC748F3CDA106DC9B67
8B6EC24F1A0591AA
 MAC chaining value:00000000000000000000000000000000
 plaintext:0088030000058A070000FFEEEEDDBB89030000058B070000AAEEEE33CC
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
wrap response:
 plaintext:05AAAAAAAAAA05BBBBBBBBBB
 ICV:8A8829B8A956A893BE53B4A0E050E8FC
 cleartext payload:05AAAAAAAAAA05BBBBBBBBBB80000000
 MAC chaining value:8B6EC24F1A0591AAF0C1AB48C49029B5
 cipher|mac:56C5CA7C53338B7927DA2B6E8113FDCD5189CB0DF19EDD4B

<<56C5CA7C53338B7927DA2B6E8113FDCD5189CB0DF19EDD4B9000

unwrap response:
 cipher|mac:56C5CA7C53338B7927DA2B6E8113FDCD5189CB0DF19EDD4B
 plaintext:05AAAAAAAAAA05BBBBBBBBBB
```

A.4. X.509 Schemes

A.4.1. X.509 Basic Certificate Schema

Listing A-4: X.509 v3 schema

```
Certificate  ::=  SEQUENCE
{
    tbsCertificate            TBSCertificate,
    signatureAlgorithm     AlgorithmIdentifier,
    signatureValue          BIT STRING
}

TBSCertificate  ::=  SEQUENCE
{
    version                [0]   EXPLICIT Version DEFAULT v1,
    serialNumber                 CertificateSerialNumber,
    signature                     AlgorithmIdentifier,
    issuer                         Name,
    validity                       Validity,
    subject                        Name,
    subjectPublicKeyInfo     SubjectPublicKeyInfo,
    issuerUniqueID    [1]     IMPLICIT UniqueIdentifier OPTIONAL,
                                  -- If present, version SHALL be v2 or v3
    subjectUniqueID   [2]    IMPLICIT UniqueIdentifier OPTIONAL,
                                  -- If present, version SHALL be v2 or v3
    extensions             [3]    EXPLICIT Extensions OPTIONAL
                                  -- If present, version SHALL be v3
}

Version ::= INTEGER  {  v1(0), v2(1), v3(2)  }

CertificateSerialNumber ::= INTEGER

Validity ::= SEQUENCE
{
    notBefore      Time,
    notAfter       Time
}

Time ::= CHOICE
{
    utcTime          UTCTime,
    generalTime    GeneralizedTime
}
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
UniqueIdentifier  ::=  BIT STRING

SubjectPublicKeyInfo  ::=  SEQUENCE
{
   algorithm               AlgorithmIdentifier,
   subjectPublicKey     BIT STRING
}

Extensions  ::=  SEQUENCE SIZE (1..MAX) OF Extension

Extension  ::=  SEQUENCE
{
   extnID        OBJECT IDENTIFIER,
   critical        BOOLEAN DEFAULT FALSE,
   extnValue   OCTET STRING
                     -- contains the DER encoding of an ASN.1 value
                     -- corresponding to the extension type identified
                     -- by extnID
}

AlgorithmIdentifier  ::=  SEQUENCE
{
    algorithm       OBJECT IDENTIFIER,
    parameters   ANY DEFINED BY algorithm OPTIONAL
}

Name ::= CHOICE { -- only one possibility for now --
    rdnSequence  RDNSequence }

RDNSequence ::= SEQUENCE OF RelativeDistinguishedName

RelativeDistinguishedName ::= SET SIZE (1..MAX) OF AttributeTypeAndValue

AttributeTypeAndValue ::= SEQUENCE
{
    type     AttributeType,
    value    AttributeValue
}

AttributeType   ::=  OBJECT IDENTIFIER

AttributeValue  ::=  ANY -- DEFINED BY AttributeType
```

A.4.2. X.509 Key Usage Extension

Listing A-5: X.509 Key Usage extension

```
id-ce-keyUsage OBJECT IDENTIFIER ::=  { id-ce 15 }

KeyUsage ::= BIT STRING
{
    digitalSignature          (0),
    nonRepudiation          (1), -- recent editions of X.509 have
                                 -- renamed this bit to contentCommitment
    keyEncipherment       (2),
    dataEncipherment      (3),
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    keyAgreement        (4),
    keyCertSign         (5),
    cRLSign             (6),
    encipherOnly        (7),
    decipherOnly        (8)
}
```

A.4.3. X.509 Basic Constraints Extension

Listing A-6: X.509 Basic Constraints extension

```
id-ce-basicConstraints OBJECT IDENTIFIER ::=  { id-ce 19 }

BasicConstraints ::= SEQUENCE
{
    cA                  BOOLEAN DEFAULT FALSE,
    pathLenConstraint   INTEGER (0..MAX) OPTIONAL
}
```

A.4.4. X.509 Authority Key Identifier Extension

Listing A-7: X.509 Authority Key Identifier extension

```
AuthorityKeyIdentifier ::= SEQUENCE
{
    keyIdentifier             [0] KeyIdentifier            OPTIONAL,
    authorityCertIssuer       [1] GeneralNames             OPTIONAL,
    authorityCertSerialNumber [2] CertificateSerialNumber  OPTIONAL
}

KeyIdentifier ::= OCTET STRING
```

A.4.5. X.509 Subject Key Identifier Extension

Listing A-8: X.509 Subject Key Identifier extension

```
SubjectKeyIdentifier ::= KeyIdentifier

KeyIdentifier ::= OCTET STRING
```

A.4.6. X.509 ECDSA Signature

Listing A-9: X.509 ECDSA Signature Format

```
Ecdsa-Sig-Value  ::=  SEQUENCE
{
    r    INTEGER,
    s    INTEGER
}
```

A.4.7. Issuer and Verifier Rules

The following rules supersede the issuance and verification rules defined in RFC 5280 [3]. If no specific rules are stated, the RFC 5280 [3] rules shall apply. In the following tables, “defined” values refer to content of Listing 15-5, Listing 15-13, and Listing 15-16.

The device OEM Server shall ensure consistency of endpoint issuer CN format and instance CA subject CN format (both either UTF8 or printableString).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The issuer CN format (either UTF8String or PrintableString) in the external CA certificate shall be the same as the subject CN format in the vehicle OEM root certificate.

The subject CN format (either UTF8String or PrintableString) in the external CA certificate shall be the same as the subject CN format in the device OEM root certificate.

Table A-1: Issuer and Verifier Rules Common to External CA, Instance CA and Endpoint

Item	Verifier Rule	Issuer Rule
tbsCertificate.version	Shall reject if not v3.	Shall issue only v3 certificates.
tbsCertificate.serialNumber	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Shall be compliant with RFC 5280 rules for conforming CAs.
tbsCertificate.signature.algorithm	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Shall contain defined OID in algorithm attribute. Shall not contain parameters attribute.
tbsCertificate.(issuer | subject) rdnSequence path validation and issuer/subject name chaining	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Shall issue certificates chain with valid issuer/subject name chaining as per RFC 5280.
tbsCertificate.(issuer | subject) rdnSequence	Shall accept multiple RelativeDistinguishedName.	Shall contain only 1 RelativeDistinguishedName.
tbsCertificate.(issuer | subject) rdnSequence.[].[].type	Shall accept multiple AttributeTypeAndValue per RelativeDistinguishedName.	The RelativeDistinguishedName shall contain only 1 AttributeTypeAndValue.
tbsCertificate.(issuer | subject) rdnSe-	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	OID shall be CommonName.
tbsCertificate.(issuer | subject) rdnSequence.[].[].value item type	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Shall be PrintableString or UTF8String.
tbsCertificate.(issuer | subject) rdnSequence.[].[].value item type	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Item type for issuer field in child certificate shall be the same type as subject of parent certificate.
tbsCertificate.(issuer | subject) rdnSequence.[].[].value item content	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Content length shall not exceed 30 bytes.
tbsCertificate.validitytime item type	Shall accept UTCTime or GeneralizedTime time types independently of the represented date. The expiration of the endpoint certificate shall not be checked.	Shall be compliant with RFC 5280 rules for conforming CAs.
tbsCertificate.validitytime item content	Should not verify unless verifier has a trusted source of time.	Shall be compliant with RFC 5280 rules for conforming CAs.
tbsCertificate.SubjectPublicKeyInfo.algorithm	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Shall contain defined OID in algorithm attribute. Shall contain defined OID in parameters attribute.
signatureAlgorithm.algorithm	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Shall contain defined OID in algorithm attribute. Shall not contain parameters attribute.
signatureValue	Shall be verified as per RFC 5280 rules.	Shall be compliant with RFC5280 rules.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table A-2: Issuer and Verifier Rules For Extensions Common to External CA, Instance CA and Endpoint

Item	Verifier Rule	Issuer Rule
tbsCertificate.extensions handling of unrecognized extensions	Unrecognized extensions shall be handled as per RFC 5280 rules.	Extensions not listed in this specification shall be marked as non-critical.
tbsCertificate.extensions OIDs of unrecognized extensions	Shall reject if OIDs are not compliant with RFC 5280 OID format rules.	Shall only issue OIDs compliant with RFC 5280 format rules.
tbsCertificate.extensions handling of duplicate extensions	Shall reject if 2 extensions or more have the same OID.	Shall not issue with 2 or more extensions having the same OID.
tbsCertificate.extensions handling of extension order	Shall accept any extension order.	Extensions can be ordered arbitrarily.
tbsCertificate.extensions.[].Authority-KeyIdentifier	Shall reject if AuthorityKeyIdentifier extension is not present. Shall accept if extension is marked as critical.	Shall contain a AuthorityKeyIdentifier extension with the defined extnValue. Shall be marked as non-critical.
tbsCertificate.extensions.[].KeyUsage	Shall reject if KeyUsage extension is not present. Shall reject if extnValue is different from the defined one. Shall reject if extension is not marked as critical.	Shall contain a KeyUsage extension with the defined extnValue. Shall be marked as critical.

Table A-3: Issuer and Verifier Rules For Extensions Specific to External CA

Item	Verifier Rule	Issuer Rule
tbsCertificate.extensions.[].externalCA	Shall reject if extension is not present. Shall reject if extnValue format or content is different from the defined one. Shall reject if extension is not marked as critical.	Shall contain an extension with the defined OID. Shall contain an extension with defined format and content. Shall be marked as critical.
tbsCertificate.extensions.[].SubjectKey-Identifier	Shall reject if SubjectKeyIdentifier extension is not present. Shall also accept if the extension is marked as critical.	Shall contain a SubjectKeyIdentifier extension with the defined extnValue. Shall be marked as non-critical.
tbsCertificate.extensions.[].Basic-Constraints	Shall reject if BasicConstraints extension is not present. Shall reject if extnValue format or content is not the defined one. Shall reject if extension is not marked as critical.	Shall contain a BasicConstraints extension. The extnValue attribute shall comply with the defined format and content. Shall be marked as critical.

Table A-4: Issuer and Verifier Rules For Extensions Specific to Instance CA

Item	Verifier Rule	Issuer Rule
tbsCertificate.extensions.[].instanceCA	Shall reject if extension is not present. Shall reject if extnValue format or content is different from the defined one. Shall reject if the extension is not marked as critical.	Shall contain an extension with the defined OID. Shall contain an extension with defined format and content. Shall be marked as critical.
tbsCertificate.extensions.[].SubjectKey-Identifier	Shall reject if SubjectKeyIdentifier extension is not present. Shall accept if the extension is marked as critical.	Shall contain a SubjectKeyIdentifier extension with the defined extnValue. Shall be marked as non-critical.
tbsCertificate.extensions.[].Basic-Constraints	Shall reject if BasicConstraints extension is not present. Shall reject if extnValue format or content is not the defined one. Shall reject if the extension is not marked as critical.	Shall contain a BasicConstraints extension. The extnValue attribute shall comply with the defined format and content. Shall be marked as critical.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table A-5: Issuer and Verifier Rules For Extensions Specific to Endpoint

Item	Verifier Rule	Issuer Rule
tbsCertificate.extensions.[].endpoint	Shall reject if extension is not present. Shall reject if extnValue format or content is different from the defined one. Shall reject if the extension is not marked as critical.	Shall contain an extension with the defined OID. Shall contain an extension with defined format and content. Shall be marked as critical.
tbsCertificate.extensions.[].Basic-Constraints	Shall accept if BasicConstraints extension is absent. Shall reject if extnValue format or content is not the defined one. Shall reject if the extension is not marked as critical.	Shall contain a BasicConstraints extension. The extnValue attribute shall comply with the defined format and content. Shall be marked as critical.

Vehicle Public Key Certificate [K], intermediate CA, Vehicle OEM CA Certificate (signed by Device OEM) [M], and vehicle OEM CA (sub)root certificates shall follow the same rules as the external CA certificate with the following exceptions:

For the Vehicle OEM Root CA certificate:

Table A-6: Issuer and Verifier Rules for Vehicle OEM Root CA certificate

Item	Verifier Rule	Issuer Rule
tbsCertificate.extensions.[].-AuthorityKeyIdentifier	Shall not verify presence or content of this field.	May contain a AuthorityKeyIdentifier extension with the defined extnValue. If present it shall be marked as non-critical.
tbsCertificate.extensions.[].Key-Usage	Shall not verify presence or content of this field.	May contain a KeyUsage extension with the defined extnValue. If present it shall be marked as critical.

For the Vehicle Public Key Certificate

Table A-7: Issuer and Verifier Rules for Vehicle Public Key Certificate

Item	Verifier Rule	Issuer Rule
tbsCertificate.(issuer | subject) rdnSequence.[].[].value item content	May verify only if the verifier has the capability to strictly follow RFC 5280 verification rules.	Content length shall not exceed 32 bytes.
tbsCertificate.extensions.[].Sub-jectKeyIdentifier	Shall accept if SubjectKeyIdentifier extension is not present If the extension is present, it shall also be accepted if marked as critical.	Should contain a SubjectKeyIdentifier extension with the defined extnValue. If present, it shall be marked as non-critical.

Appendix B.

B.1. Instance Configurations

instance AID = See Section 15.3.2.1

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

maximum number of endpoints = <Device OEM policy applies>
maximum number of Instance CAs = <Device OEM policy applies>
SHARING_PASSWORD_LENGTH = <vehicle OEM policy, length of 4 to 8 digits (including 4 and 8)>
The vehicle OEM should choose a suitable combination of password length and allowed number of attempts to enter the password in the vehicle UI, following the recommendations in [39].

B.2. Certificate Field Formats

All certificates, except [C] and [N] (which are attestations) as specified in Section 16, shall comply with X.509 formatting as per RFC 5280 [3].
All certificates shall use 256-bit ECC public keys.
All certificates shall append the server environment indicator to their subject CN ("-P", "-E", see below) in addition to specific naming requirements for the subject CN as listed below.

B.2.1. Key Identifier Calculation

The key identifier corresponds to the field Subject Key Identifier in Section 4.2.1.2, method (1) of RFC 5280 [3].

B.2.2. OID Assignment

Each certificate shall be identifiable by the following OID values:
Vehicle Public Key Certificates (leaf) [K]
1.3.6.1.4.1.41577.5.1
External CA Certificates (intermediate)[F]
1.3.6.1.4.1.41577.5.2
Instance CA Certificates (intermediate) [E]
1.3.6.1.4.1.41577.5.3
Endpoint Certificates (leaf)[H]
1.3.6.1.4.1.41577.5.4
Vehicle OEM Encryption Certificate (VehicleOEM.Enc.Cert)
1.3.6.1.4.1.41577.5.5
Vehicle OEM Signature Certificate (VehicleOEM.Sig.Cert)
1.3.6.1.4.1.41577.5.6
Device.Enc.Cert (optional as only the PK is sent in the trackKey request)
1.3.6.1.4.1.41577.5.7
Vehicle Intermediate Certificate
1.3.6.1.4.1.41577.5.8
Vehicle OEM CA Certificate [J]
1.3.6.1.4.1.41577.5.9
Vehicle OEM CA Certificate (Signed by Device OEM CA) [M] (optional as only used by some implemenation)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1.3.6.1.4.1.41577.5.10

Extensions with the above OIDs shall be marked as critical.

B.2.3. Endpoint Certificate [H]

The **endpoint certificate** shall be formatted as follows:

Subject (CN):

DIGK.[OWNR, FRND].[identifier (4-byte printable ASCII)]|[optional: free text]

The subject field holds the endpoint_identifier value provided at Digital Key creation. The endpoint_identifier and thus the subject shall be unique per Digital Key applet instance. The 4-byte identifier is chosen by the device to ensure uniqueness. The free text may be added for debugging or other identification purposes. It shall not contain dots (“.”) or equal-to (“=”) signs. The subject field free text is separated by an equal to [=] sign from the identifier. The equal-to (“=”) delimiter shall only be included if the optional free text is used.

Examples:

DIGK.OWNR.63K7=JOHN_APPLESEED

DIGK.FRND.Nd75=75924635

The issuer field equals the subject of Instance CA Certificate. The allowed character set per element is A–Z, a–z, 0–9, coded in ASCII.

B.2.4. Instance CA Certificate [E]

The **Instance CA Certificate** shall be formatted as follows:

Subject (CN):

<Vehicle OEM Identifier>

The vehicle OEM identifier is defined in [35]. It shall have a length of 4 characters.

The issuer CN of the Instance CA certificate shall be equal to the subject CN of the external Certificate. The format of text used in the issuer CN of the Instance CA certificate (either PrintableString or UTF8String) shall also match the format of the text used in the subject CN of the external Certificate.

B.2.5. External CA Certificate [F]

The external CA certificate issuer CN equals the subject CN of the vehicle OEM root certificate and the subject CN equals the subject CN of the device OEM root certificate. The format of the subject CN in the vehicle OEM root certificate and the device OEM root certificate shall follow the same rules as outlined in section A.4.7.
The CN fields for subject and issuer in the external CA certificate shall not exceed 30 bytes in DER coding.

B.2.6. Vehicle Public Key/Leaf Certificate [K]

The **Vehicle Public Key Certificate** [K] shall be formatted as follows:

The subject field of [K] shall contain the following information as a readable string:

V.[vehicle_oem].[datacenter/region].[brand].[vehicle_identifier]

All elements are separated by a “.” (dot).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The allowed character set per element is A–Z, a–z, 0–9, coded in ASCII.

[vehicle_oem] defines the Instance CA assignment. It shall match the Vehicle OEM Identifier in Table 2-1 in [35].

Datacenter/region field contains information required by the server, e.g., to know which endpoint/region and server environment to connect to in order to reach the vehicle. It shall be 3 bytes.

It is strongly recommended to have one worldwide endpoint per server environment with the following definition:

- WWP: region=WorldWide, environment=PROD
- WWE: region=WorldWide, environment=E2E (Testing)

If required, possible regional values are:

- EUE: region=EUR, environment=E2E
- EUP: region=EUR, environment=PROD
- USE: region=USA, environment=E2E
- USP: region=USA, environment=PROD
- CHE: region=CHN, environment=E2E
- CHP: region=CHN, environment=PROD

The [brand] field represents the Vehicle Brand Name and should have a granularity that is used to identify the app to be launched for this vehicle. It shall be 4 bytes.

The [vehicle_identifier] is used to associate vehicle and certificate, without using the VIN. The vehicle_identifier requires 16 bytes of ASCII to represent the 8-byte value.

The following part of the subject field is used as ROUTING_INFORMATION for key sharing:

vehicle_oem.datacenter/region.brand

The vehicle_identifier is not part of the routing information.

B.2.7. Intermediate Vehicle Certificate

An intermediate certificate may be used to attest to the vehicle public key. The Vehicle OEM CA attests to the intermediate certificate:

Vehicle OEM CA —> Intermediate Vehicle Certificate —> Vehicle Public Key

Subject CN is defined by the vehicle OEM. It shall terminate with an indication of the environment in which the certificate is valid.

Subject (CN):

<Vehicle OEM defined>-INTERMEDIATE-<region>-<environment>

Example:

Vehicle OEM CA-INTERMEDIATE-WW-P

B.2.8. Vehicle OEM CA Certificate [J]

The Vehicle OEM CA certificate is trusted by an offline process by each device OEM. It is also embedded in the vehicle.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Issuer CN and subject CN are defined by the vehicle OEM. Both shall terminate with an indication of the environment in which the certificate is valid.

Subject (CN):

< Vehicle OEM defined>-<environment>

Example:

Vehicle OEM CA-P

B.2.9. Vehicle OEM CA Certificate (signed by Device OEM) [M]

The Vehicle OEM CA certificate (signed by Device OEM) issuer CN equals subject CN of the Device OEM root certificate[D] and the subject CN equals the subject CN of the vehicle OEM root certificate [J]. The CN field for subject and issuer in the Vehicle OEM CA certificate (signed by Device) shall not exceed 30 bytes in DER coding.

B.2.10. Vehicle OEM Privacy Encryption Certificate

The Vehicle OEM privacy encryption certificate shall be formatted as follows:

Subject (CN):

<Vehicle OEM defined>-ENC-<region>-<environment>

Issuer:

Vehicle OEM-ENC-WW-P

B.2.11. Vehicle OEM Signature Verification Certificate

The Vehicle OEM signature certificate should be formatted as follows:

Subject (CN):

<Vehicle OEM defined>-TERM-<region>-<environment>

Example:

Vehicle OEM-TERM-WW-P

B.2.12. Validity Period

The fields “not before” and “not after” of the certificates shall be set to the following values:

External CA Certificate, issued by Vehicle OEM

not before = date and time of certificate creation

not after = <Vehicle OEM policy applies>

Instance CA Certificate [E]

not before = date and time of certificate creation

not after = <device and vehicle OEM policies apply>

The expiration of the Instance CA Certificate shall be checked by the vehicle, the owner device, and the key tracking server.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Endpoint Certificate

not before = issuance date and time

not after = 99991231235959Z (as defined in [3])

The expiration date of the endpoint certificate should not be checked anywhere in the system.

Note: The validity definition in the friend's attestation package determines the lifetime of the key in the system.

Vehicle OEM Privacy Encryption Certificate

not before = issuance date

not after = <vehicle OEM policy applies>

Vehicle OEM Signature Certificate

not before = issuance date

not after = <vehicle OEM policy applies>

The undefined expiration date value 99991231235959Z shall be accepted by the framework and applet implementation.

Appendix C.

C.1. External CA Certificate

The following examples of certificate for illustration purpose using a “mock” environment (“-M”). The certificates may vary within the RFC5280 limits, depending on vehicle OEM issuance policies.

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            42:76:cb:ad:b3:1a:8c:4a:72:e0:1c:5a:5c:ce:59:6f:67:42:d9:c5
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: CN=MOCK-CAR-ROOT-CA-E
        Validity
            Not Before: Jan  1 00:00:00 2019 GMT
            Not After : Jan  1 00:00:00 2049 GMT
        Subject: CN=Device OEM Root Certificate
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
                pub:
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
                    04:86:86:92:e0:41:b2:25:58:39:c7:85:62:03:b2:
                    d9:ae:fd:28:5c:f9:87:94:ec:72:69:37:cb:58:87:
                    63:28:91:d8:99:4a:85:c0:29:9a:d3:2d:f8:09:4d:
                    75:55:51:56:25:57:5e:74:1b:0d:82:42:cc:08:b2:
                    6a:c7:de:43:5e
                ASN1 OID: prime256v1
                NIST CURVE: P-256
        X509v3 extensions:
            X509v3 Authority Key Identifier:
                key-
id:23:18:E5:56:71:F0:8E:AE:21:21:42:A8:17:72:0F:B8:17:EE:93:BF

            X509v3 Subject Key Identifier:
                65:17:11:D3:7B:96:37:00:21:3F:F2:8C:9C:4F:55:02:E4:D8:51:76
            X509v3 Basic Constraints: critical
                CA:TRUE, pathlen:1
            1.3.6.1.4.1.41577.5.2: critical
                0....
            X509v3 Key Usage: critical
                Certificate Sign
    Signature Algorithm: ecdsa-with-SHA256
         30:44:02:20:0c:54:51:3f:a1:cd:04:82:dc:49:13:f5:37:4c:
         fa:5e:7e:4c:40:61:86:55:0b:12:3a:71:93:a1:22:22:2a:69:
         02:20:04:2c:2f:06:65:3d:68:a2:81:cf:65:bc:57:85:b0:ca:
         34:91:8b:51:18:83:59:8b:91:1e:58:89:a2:bd:cd:e1
```

C.2. Vehicle OEM Intermediate Certificate

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            6d:da:2b:a0:69:c7:55:5a:db:84:a3:ee:dd:a3:83:fc:8a:33:b3:97
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: CN=MOCK-CAR-ROOT-CA-E
        Validity
            Not Before: Jan  1 00:00:00 2019 GMT
            Not After : Jan  1 00:00:00 2049 GMT
        Subject: CN=MOCK-CAR-INTERMEDIATE-US-E
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
                pub:
                    04:84:22:42:f6:18:2b:a1:c1:13:8d:32:b7:7f:b9:
                    f7:f3:7b:70:03:4b:9f:04:44:3a:5b:ea:3c:18:8b:
                    ea:db:36:49:0a:7e:95:f9:1a:4c:16:2a:cf:c3:40:
                    1c:3a:4f:4e:5a:59:25:1d:45:24:3a:c8:54:4a:66:
                    5c:b9:51:42:2f
                ASN1 OID: prime256v1
                NIST CURVE: P-256
        X509v3 extensions:
            X509v3 Authority Key Identifier:
                key-
id:23:18:E5:56:71:F0:8E:AE:21:21:42:A8:17:72:0F:B8:17:EE:93:BF

            X509v3 Subject Key Identifier:
                E6:97:50:57:10:09:13:C3:AB:07:29:D4:53:3F:4E:11:29:9A:34:83
            X509v3 Basic Constraints: critical
                CA:TRUE, pathlen:0
            1.3.6.1.4.1.41577.5.8: critical
                0....
            X509v3 Key Usage: critical
                Certificate Sign
    Signature Algorithm: ecdsa-with-SHA256
         30:45:02:20:08:b4:7d:06:3f:c8:f0:27:95:5d:5d:99:62:ee:
         92:91:5e:79:5d:99:4a:4b:5a:a7:6a:f4:2b:6c:16:1d:8c:85:
         02:21:00:d8:34:99:b3:49:3c:5b:ee:55:39:98:0b:64:9f:58:
         f1:43:54:e6:04:de:a3:9d:22:b3:cc:01:ad:a7:e2:40:04
```

C.3. Vehicle OEM Key/Leaf Certificate

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            0d:7a:48:40:4c:a2:24:14:45:85:d1:64:51:94:80:7b:ea:69:79:22
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: CN=MOCK-CAR-INTERMEDIATE-US-E
        Validity
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
            Not Before: Jan  1 00:00:00 2019 GMT
            Not After : Jan  1 00:00:00 2049 GMT
        Subject: CN=V.MOCK.USE.BRND.1111111111111111
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
                pub:
                    04:1c:84:2b:af:21:cc:2b:28:3b:83:03:6b:a5:26:
                    dd:bc:e9:c2:01:f6:3e:80:dc:73:50:88:c4:f3:ca:
                    90:ff:f2:a4:08:ba:74:4f:78:b6:68:c8:8b:63:d0:
                    90:a3:e3:48:d2:4d:29:d1:53:f5:31:15:e0:8a:31:
                    4b:ff:d6:86:01
                ASN1 OID: prime256v1
                NIST CURVE: P-256
        X509v3 extensions:
            X509v3 Authority Key Identifier:
                key-
id:E6:97:50:57:10:09:13:C3:AB:07:29:D4:53:3F:4E:11:29:9A:34:83

            X509v3 Subject Key Identifier:
                57:11:1D:D5:E7:5E:24:75:AB:5F:07:BC:96:FB:69:89:FB:7D:0E:AA
            X509v3 Basic Constraints: critical
                CA:FALSE
            1.3.6.1.4.1.41577.5.1: critical
                0....
            X509v3 Key Usage: critical
                Digital Signature
    Signature Algorithm: ecdsa-with-SHA256
         30:44:02:20:48:95:c3:bb:16:d0:32:bb:15:c3:0e:2d:50:22:
         cd:f8:78:a1:4e:87:6c:6b:08:71:09:12:91:fe:48:ba:cf:12:
         02:20:43:17:ca:c5:d4:4c:67:53:52:7a:f9:d0:64:cb:fb:83:
         4e:01:78:a0:b5:53:1f:af:83:3d:38:9b:f9:64:ed:b3
```

C.4. Vehicle OEM Privacy Encryption Certificate

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            7f:86:4e:b6:84:72:d0:52:26:47:92:9d:1e:ab:ae:e4:ef:74:bd:f9
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: CN=MOCK-CAR-ROOT-CA-E
        Validity
            Not Before: Jan  1 00:00:00 2019 GMT
            Not After : Jan  1 00:00:00 2049 GMT
        Subject: CN=MOCK-CAR-ENC-US-E
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
                pub:
                    04:2b:1d:13:34:c9:d3:16:1c:ca:d0:f5:b9:22:6b:
                    ca:df:ad:5f:c9:b7:83:b2:7f:4b:51:34:84:b7:76:
                    2e:e6:35:33:4d:a4:dc:9b:c7:54:92:3e:21:cb:7b:
                    22:ae:f7:0b:ca:16:6a:12:f0:35:f3:f2:d5:77:aa:
                    a3:b4:8b:d5:f5
                ASN1 OID: prime256v1
                NIST CURVE: P-256
        X509v3 extensions:
            X509v3 Authority Key Identifier:
                key-
id:23:18:E5:56:71:F0:8E:AE:21:21:42:A8:17:72:0F:B8:17:EE:93:BF

            X509v3 Subject Key Identifier:
                C3:6C:36:D4:97:A1:31:5B:BB:F2:17:10:A3:E9:6D:12:4D:8D:07:5C
            X509v3 Basic Constraints: critical
                CA:FALSE
            1.3.6.1.4.1.41577.5.5: critical
                0....
            X509v3 Key Usage: critical
                Key Agreement, Encipher Only
    Signature Algorithm: ecdsa-with-SHA256
         30:46:02:21:00:f0:f2:bd:59:75:73:8b:60:3f:a9:a4:2d:7d:
         70:23:4e:73:5a:63:73:59:25:d4:fe:42:4b:33:68:a4:64:5a:
         e5:02:21:00:b0:df:b3:40:2f:7e:0f:2b:e3:81:43:f8:c5:b3:
         b1:56:33:4e:7a:87:30:7c:73:64:4d:ad:03:a6:23:01:87:a0
```

C.5. Vehicle OEM Signature Verification Certificate

```
Certificate:
    Data:
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        Version: 3 (0x2)
        Serial Number:
            0b:30:c4:1b:39:98:0a:44:41:4b:36:54:55:4f:fc:b5:28:8f:11:cf
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: CN=MOCK-CAR-ROOT-CA-E
        Validity
            Not Before: Jan  1 00:00:00 2019 GMT
            Not After : Jan  1 00:00:00 2049 GMT
        Subject: CN=MOCK-CAR-TERM-US-E
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
                pub:
                    04:62:9e:51:60:2d:98:4c:e3:ea:77:ae:93:6f:e9:
                    75:7a:34:a9:16:25:99:81:6a:12:df:17:f4:85:84:
                    ad:77:ee:81:c7:d4:7f:8e:2d:50:29:27:4e:58:32:
                    ac:69:0a:52:0d:20:22:fb:84:6a:75:c6:a1:fd:e4:
                    45:ef:d6:a9:b0
                ASN1 OID: prime256v1
                NIST CURVE: P-256
        X509v3 extensions:
            X509v3 Authority Key Identifier:
                key-
id:23:18:E5:56:71:F0:8E:AE:21:21:42:A8:17:72:0F:B8:17:EE:93:BF

            X509v3 Subject Key Identifier:
                29:F8:B9:AE:B8:4A:19:76:CF:90:A6:61:D3:0F:49:24:34:6E:31:EC
            X509v3 Basic Constraints: critical
                CA:FALSE
            1.3.6.1.4.1.41577.5.6: critical
                0....
            X509v3 Key Usage: critical
                Digital Signature
    Signature Algorithm: ecdsa-with-SHA256
         30:45:02:20:3d:d9:06:f8:f6:c0:be:c1:af:88:3c:7f:97:7d:
         ad:7c:ad:ee:e7:62:09:71:7f:8f:4d:7c:cd:1f:19:d1:85:70:
         02:21:00:89:bc:ad:9a:60:8a:b9:49:85:97:9e:d8:e1:5c:25:
         f7:c9:55:82:7e:9a:b7:18:9c:5a:2e:ed:d5:fe:d0:e8:c9
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Appendix D. Owner Pairing Test Vectors [To be updated]

D.1. Derivation of z0, z1 with Scrypt

Computed by the Vehicle OEM Server as well as the device.

Parameters

Password: "pleaseletmein" (ASCII, 13 bytes)
Salt: "yellowsubmarines" (ASCII, 16 bytes)
Nscrypt: 32768 (cost parameter)
r: 8 (block size)
p: 1 (parallelization parameter)
dkLen: 80 (output length)

Results

dk: (full 80 bytes)
6408161bbc64a41827cf072c62efdae535739e51
3ad3050e66a9f53eb69c15bb3c220f0acb5f7f02
dd008ce70d974431369ef8e569dee33977fa9fd2
e13a55c8c80c244a05559264b0498c1f5216f327
z0: (left 40 bytes)
6408161bbc64a41827cf072c62efdae535739e51
3ad3050e66a9f53eb69c15bb3c220f0acb5f7f02
z1: (right 40 bytes)
dd008ce70d974431369ef8e569dee33977fa9fd2
e13a55c8c80c244a05559264b0498c1f5216f327

D.2. Computation of w0, w1

w0 and w1 computed per FIPS 186-4, B.5.1 Per-Message Secret Number Generation Using Extra Random Bits.

$w0 = (z0 \bmod (n - 1)) + 1$

$w1 = (z1 \bmod (n - 1)) + 1$

With n being the order of base point G as defined for NIST P-256.

Parameters

n: (32 bytes)
ffffffff00000000ffffffffffffffff
bce6faada7179e84f3b9cac2fc632551
z0: (40 bytes)
6408161bbc64a41827cf072c62efdae535739e51
3ad3050e66a9f53eb69c15bb3c220f0acb5f7f02
z1: (40 bytes)
dd008ce70d974431369ef8e569dee33977fa9fd2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
e13a55c8c80c244a05559264b0498c1f5216f327
```

Results

```
w0: (32 bytes)
e433ab43428320b24fab82f915d1db11 4acd72f8a4bf4fbf3c712b94bcc2f013
w1: (32 bytes)
44363d157f471221b1e75e596ff4714a
712b9578301665d84ec17004952523a8
```

D.3. Computation of L

Only computed on the Vehicle OEM Server.

$L = w1 \times G$

L is the result of scalar multiplication on curve NIST P-256. G is the base point as defined for NIST P-256.

Parameters

```
w1: (32 bytes)
44363d157f471221b1e75e596ff4714a
712b9578301665d84ec17004952523a8
G.x: (32 bytes)
6b17d1f2e12c4247f8bce6e563a440f2 77037d812deb33a0f4a13945d898c296
G.y: (32 bytes)
4fe342e2fe1a7f9b8ee7eb4a7c0f9e16 2bce33576b315ececbb6406837bf51f5
```

Results

```
L.x: (32 bytes)
ff69eb6086938b3cce2c9e64dcacea1a 925918e75e8c17948d316322d370123f
L.y: (32 bytes)
69132aed7398919e6e6614f7627b0a54
060c5a8c0d93d2754166ab10fea6a8ff
```

D.4. Computation of X

Computed by the device.

$X = x \times G + w0 \times M$

x is a random scalar. G is the base point as defined for NIST P-256 and M a fixed point as defined by the spec.

Parameters

```
x: (32 bytes)
000102030405060708090a0b0c0d0e0f
101112131415161718191a1b1c1d1e1f
w0: (32 bytes)
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

e433ab43428320b24fab82f915d1db11
4acd72f8a4bf4fbf3c712b94bcc2f013
G.x: (32 bytes)
6b17d1f2e12c4247f8bce6e563a440f2
77037d812deb33a0f4a13945d898c296
G.y: (32 bytes)
4fe342e2fe1a7f9b8ee7eb4a7c0f9e16
2bce33576b315ececbb6406837bf51f5
M: (65 bytes)
04
886e2f97ace46e55ba9dd7242579f299
3b64e16ef3dcab95afd497333d8fa12f
5ff355163e43ce224e0b0e65ff02ac8e
5c7be09419c785e0ca547d55a12e2d20

Results

xG.x: (32 bytes)
7a593180860c4037c83c12749845c8ee
1424dd297fadcb895e358255d2c7d2b2
xG.y: (32 bytes)
a8ca25580f2626fe579062ff1b99ff91 c24a0da06fb32b5be20148c9249f5650
w0M.x: (32 bytes)
0d556afe7d4cf1b87a9baa429ab9fb57 5de4f36d412cbd7d0dc095779590f5aa
w0M.y: (32 bytes)
5f941aae1286700e4dab2ff38b4eeb2d 6ceb5dd3f7072bc84ee74360d0f7ad0c
X.x: (32 bytes)
f44555207a617fd90900dba5c8e6f81e ddbd87590873a63b9057dda9f138dbc1
X.y: (32 bytes)
6f453195f6452ce71d399052435952b8
9a10b927435574f5e3707eae031c40e0

D.5. Computation of Y

Computed by the vehicle.

$Y = y \times G + w0 \times N$

y is a random scalar. G is the base point as defined for NIST P-256 and N a fixed point as defined by the spec.

Parameters

y: (32 bytes)
f1e1d1c1b1a191817161514131211101
f0e0d0c0b0a090807060504030201000

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

w0: (32 bytes)
e433ab43428320b24fab82f915d1db11
4acd72f8a4bf4fbf3c712b94bcc2f013
G.x: (32 bytes)
6b17d1f2e12c4247f8bce6e563a440f2
77037d812deb33a0f4a13945d898c296
G.y: (32 bytes)
4fe342e2fe1a7f9b8ee7eb4a7c0f9e16
2bce33576b315ececbb6406837bf51f5
N: (65 bytes)
04
d8bbd6c639c62937b04d997f38c37707
19c629d7014d49a24b4f98baa1292b49
07d60aa6bfade45008a636337f5168c6
4d9bd36034808cd564490b1e656edbe7

Results

yG.x: (32 bytes)
d05d3f33d7a67710f70275600e2905e4
4f436e3331a1f479f7de0b650f679d12
yG.y: (32 bytes)
1c404aa84e6f75fb75d0526ff48e004b 4ea689d7fc619a05e0ba37dfbef340bf
w0N.x: (32 bytes)
4f8cd0a6e01386dac1b2e1af01e1e56a 65d7beccfcb7ed39437308f4bd99d3b6
w0N.y: (32 bytes)
e89a7da518b37a32f9acfe497da5fd16 05b80820b4ce92406569de793811d938
Y.x: (32 bytes)
b6fdaf3f6949869d68f667108b75e4ce 74847e8953d1e3c6aae21699e8027211
Y.y: (32 bytes)
c2d9b2b2a906cc7ea7020715dec44e95 659e3fc8994f635b95e7c9ea5c362cbe

D.6. Computation of Z, V (by device)

Z, V are keys shared by the device and vehicle.

$T = Y - w0 \times N$

$Z = x \times T$

$V = w1 \times T$

Parameters

x: (32 bytes)
000102030405060708090a0b0c0d0e0f

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
101112131415161718191a1b1c1d1e1f
w0: (32 bytes)
e433ab43428320b24fab82f915d1db11
4acd72f8a4bf4fbf3c712b94bcc2f013
w1: (32 bytes)
44363d157f471221b1e75e596ff4714a
712b9578301665d84ec17004952523a8
Y: (65 bytes)
04
b6fdaf3f6949869d68f667108b75e4ce
74847e8953d1e3c6aae21699e8027211
c2d9b2b2a906cc7ea7020715dec44e95
659e3fc8994f635b95e7c9ea5c362cbe
N: (65 bytes)
04
d8bbd6c639c62937b04d997f38c37707
19c629d7014d49a24b4f98baa1292b49
07d60aa6bfade45008a636337f5168c6
4d9bd36034808cd564490b1e656edbe7
```

Results

```
T.x: (32 bytes)
d05d3f33d7a67710f70275600e2905e4
4f436e3331a1f479f7de0b650f679d12
T.y: (32 bytes)
1c404aa84e6f75fb75d0526ff48e004b
4ea689d7fc619a05e0ba37dfbef340bf
Z.x: (32 bytes)
89af176e8122e67c00dbcea089bc4993 5634132b1b226030d2b14f16b3e73351
Z.y: (32 bytes)
c2254b1b62477fdc976379f5ae7c57c6 ac2b31ef9a032c33e4677c5c3acbb1d3
V.x: (32 bytes)
2f229f13e1ebfb6442a67eebdafb23b2 f6e656597384035a8a1e50ad95d24211
V.y: (32 bytes)
339e90a669dd8a56fb2524fb6e6c784b 89019c1130c2def98143fb46dcc507d2
```

D.7. Computation of Z, V (by vehicle)

Z, V are keys shared by the device and vehicle.

$Z = y \times (X - w0 \times M)$

$V = y \times L$

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Parameters

y: (32 bytes)
f1e1d1c1b1a191817161514131211101
f0e0d0c0b0a090807060504030201000
w0: (32 bytes)
e433ab43428320b24fab82f915d1db11
4acd72f8a4bf4fbf3c712b94bcc2f013
X: (65 bytes)
04
f44555207a617fd90900dba5c8e6f81e
ddbd87590873a63b9057dda9f138dbc1
6f453195f6452ce71d399052435952b8
9a10b927435574f5e3707eae031c40e0
M: (65 bytes)
04
886e2f97ace46e55ba9dd7242579f299
3b64e16ef3dcab95afd497333d8fa12f
5ff355163e43ce224e0b0e65ff02ac8e
5c7be09419c785e0ca547d55a12e2d20
L: (65 bytes)
04
ff69eb6086938b3cce2c9e64dcacea1a
925918e75e8c17948d316322d370123f
69132aed7398919e6e6614f7627b0a54
060c5a8c0d93d2754166ab10fea6a8ff

Results

Z.x: (32 bytes)
89af176e8122e67c00dbcea089bc4993
5634132b1b226030d2b14f16b3e73351
Z.y: (32 bytes)
c2254b1b62477fdc976379f5ae7c57c6
ac2b31ef9a032c33e4677c5c3acbb1d3
V.x: (32 bytes)
2f229f13e1ebfb6442a67eebdafb23b2
f6e656597384035a8a1e50ad95d24211
V.y: (32 bytes)
339e90a669dd8a56fb2524fb6e6c784b
89019c1130c2def98143fb46dcc507d2

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

D.8. Derivation of K, CK, SK

K is the secret shared by the device and vehicle.

K = SHA-256(len(X) || X || len(Y) || Y || len(Z) || Z || len(V) || V || len(w0) || w0)

len(x) denotes the length of a string in bytes, represented as an 8-byte little-endian number.

Parameters

w0: (32 bytes)
e433ab43428320b24fab82f915d1db11
4acd72f8a4bf4fbf3c712b94bcc2f013
len(w0): (8 bytes)
2000000000000000
X: (65 bytes)
04
f44555207a617fd90900dba5c8e6f81e
ddbd87590873a63b9057dda9f138dbc1
6f453195f6452ce71d399052435952b8
9a10b927435574f5e3707eae031c40e0
Y: (65 bytes)
04
b6fdaf3f6949869d68f667108b75e4ce
74847e8953d1e3c6aae21699e8027211
c2d9b2b2a906cc7ea7020715dec44e95
659e3fc8994f635b95e7c9ea5c362cbe
Z: (65 bytes)
04
89af176e8122e67c00dbcea089bc4993
5634132b1b226030d2b14f16b3e73351
c2254b1b62477fdc976379f5ae7c57c6
ac2b31ef9a032c33e4677c5c3acbb1d3
V: (65 bytes)
04
2f229f13e1ebfb6442a67eebdafb23b2
f6e656597384035a8a1e50ad95d24211
339e90a669dd8a56fb2524fb6e6c784b
89019c1130c2def98143fb46dcc507d2
len(X/Y/Z/V): (8 bytes)
4100000000000000

Results

K: (32 bytes)

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

381fc44894aedc0fd37257fdca763ee4
9f695017ba5e1df74a1b8bbf27fe1e0d
CK: (16 bytes)
381fc44894aedc0fd37257fdca763ee4
SK: (16 bytes)
9f695017ba5e1df74a1b8bbf27fe1e0d

D.9. Derivation of Evidence Keys K1, K2

K1 and K2 are the MAC keys used for key confirmation.

HKDF-SHA-256 is used as the KDF, defined by IETF RFC5869.

Parameters

L: 32
IKM/CK: (16 bytes)
381fc44894aedc0fd37257fdca763ee4
Info: "ConfirmationKeys" (ASCII, 16 bytes)
5b020101 (Agreed SPAKE2+ protocol version, 4 bytes)
5c0401010100 (Tag 5C of the SPAKE2+ REQUEST command, 6 bytes)

Results

K1: (16 bytes)
ab667caeffe27505265d0f2026e146ea
K2: (16 bytes)
af679bf88b734e1cb7cd0243fb21a589

Note that this calculation is an example of key derivation that uses an undefined protocol version 0101. Its intent is to show the key derivation algorithm in principle.

D.10. Computation of Evidences M1, M2

M1 and M2 are the MACs used for key confirmation.

M1 = CMAC(K1, X)

M2 = CMAC(K2, Y)

Parameters

K1: (16 bytes)
ab667caeffe27505265d0f2026e146ea
K2: (16 bytes)
af679bf88b734e1cb7cd0243fb21a589
X: (65 bytes)
04
f44555207a617fd90900dba5c8e6f81e
ddbd87590873a63b9057dda9f138dbc1

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

6f453195f6452ce71d399052435952b8
9a10b927435574f5e3707eae031c40e0
Y: (65 bytes)
04
b6fdaf3f6949869d68f667108b75e4ce
74847e8953d1e3c6aae21699e8027211
c2d9b2b2a906cc7ea7020715dec44e95
659e3fc8994f635b95e7c9ea5c362cbe

Results

M1: (16 bytes)
110d49f8c5a896e11d4dde4c3b9704d2
M2: (16 bytes)
23d1a618ad3acbfd7a9bd19fd1737107

D.11. Derivation of System Keys

HKDF-SHA-256 is used as the KDF, defined by IETF RFC 5869.

Parameters

L: 64
IKM/SK: (16 bytes)
9f695017ba5e1df74a1b8bbf27fe1e0d
Info: "SystemKeys" (ASCII, 10 bytes)

Results

Kenc: (16 bytes)
161886cb9ae7403d8dbccfe36b8a0426
Kmac: (16 bytes)
6387ba65479cb7eb9df97bd48ac33159
Krmac: (16 bytes)
41677fb6398459199f1e569760df91c1
LONG_TERM_SHARED_SECRET: (16 bytes)
5c4e19da553524e386fa1eca91e8ad0e

Appendix E. NFC-F Support [WCC1]

E.1. Introduction

This Appendix defines a protocol to perform the contactless transactions defined in this document over NFC-F.

Support for NFC-F is an optional feature of the Digital Key protocol. The requirements in this appendix apply only if an implementation supports this option.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Implementation may include support for NFC-F to be able to use NFC-F devices for holding Digital Keys or if the contactless transactions defined in this document are used as part of a larger NFC-F transaction.

The protocol in this appendix defines how the device exchanges APDUs with the vehicle over NFC-F. The APDU-based message exchanges defined in this document are not modified in any way when run over NFC-F. Support for NFC-F only affects the contactless interface; it has no impact on the device-internal interfaces to communicate between the Digital Key applet and Digital Key framework or on the communication with the Device OEM Server or Vehicle OEM Server.

The following definitions are specific to this appendix:

- **Protocol Error**: Receiving a response with a wrong syntax or receiving a correct response when it is not expected.
- **Timeout Error**: No response has been received within the defined waiting time.
- **Transmission Error**: Error caused by the receipt of an incorrect frame. This includes the case that the CRC in the NFC-F frame is not correct.

The following abbreviations are specific to this appendix:

RW_ACK	Acknowledge send from vehicle to device
DEVICE_ACK	Acknowledge send from device to vehicle
C-APDU	Command APDU as defined in [1]
fc	Carrier Frequency
NACK	Negative acknowledgement
NFC-F	NFC Type F RF Technology
NFCID2	NFC-F identifier as defined in [DIGITAL].
R-APDU	Response APDU as defined in [1]
RWT	Response waiting time
RWTI	Response waiting time integer

E.2. Device Requirement

To support NFC-F, the following requirements apply on the device side:

- The protocol defined in this appendix shall be implemented by the Digital Key applet.
- The SE shall be compliant with the GlobalPlatform card specification and shall implement GlobalPlatform Contactless Services – Amendment C [21], including support for “Type F”.
- The Digital Key applet shall register the System Code $ACCC_h$ as part of its contactless protocol parameters for Type F (Sub-tag 80_h of the Type F anti-collision parameters entry tag $A0_h$; see [21]).
- The device shall configure the NFC controller to route frames containing the NFCID2 of System Code $ACCC_h$ to the SE that hosts the Digital Key applet.
- The Digital Key applet shall register an NFCID2 (as defined in [DIGITAL]) as part of its contactless protocol parameters for Type F. The NFCID2 is registered as the “PICC

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Identifier" contained in sub-tag 81_h of the Type F anti-collision parameters entry tag $A0_h$ (see [21]).

- The following applies only if owner pairing over NFC-F has to be supported:
  - The protocol defined in this appendix shall be implemented by the Digital Key framework
  - The Digital Key framework shall use the System Code $4CCC_h$ for NFC-F.
  - The Digital Key framework shall use an NFCID2 in the range of $02FE000000000000_h$ to $02FEFFFFFFFFFFFF_h$. The PAD0 field in the SENSF_RES as defined in [17] shall be configured to be $01FE_h$.
  - The device shall configure the NFC controller to route frames containing the NFCID2 of System Code $4CCC_h$ to the host with the Digital Key framework.

To support NFC-F, the following requirements apply on the vehicle side:

- The vehicle shall implement the protocol defined in this appendix. The protocol shall be available for communication on the door NFC readers and console NFC reader.
- If configured to support NFC-F, the NFC readers shall include polling for NFC-F. The polling shall use the system code for the Digital Key applet except in case of owner pairing, in which case the system code for the Digital Key framework shall be used.

Note: As only devices that support the requested system code will respond to the polling, there is no risk that the NFC link is established using NFC-F technology without the Digital Key protocol being available.

On either side, the APDUs exchanged as part of the Digital Key protocol shall be encapsulated as defined in the following sections before sending and shall be extracted accordingly after receiving.

E.3. Preconditions

This specification is based on NFC-F technology, more specifically on the Type 3 Tag Platform as defined in [17] and [18].

The vehicle and the device shall be able to send and receive NFC-F frames as defined in [17]. The device shall be able to respond to a SENSF_REQ command as defined in [17].

E.4. Commands and responses

The formats for NFC-F commands and responses defined in this section shall be transported inside the Payload field of the NFC-F Data and Payload format as defined in [17].

The LEN byte and the CRC are not included in these format definitions. The LEN field has to be prepended and the CRC to be appended to obtain the content for the data field of an NFC-F frame.

The LEN field is supposed to be part of the command or response and therefore constitutes the first byte of each command and response.

E.4.1. Encapsulation Command (ENCAPSULATION_CMD)

The ENCAPSULATION_CMD is used to transfer APDU data from the vehicle to the device.

The ENCAPSULATION_CMD shall be formatted as defined in Table E-1.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table E-1: ENCAPSULATION_CMD format

Byte 1	Byte 2	Bytes 3–10	Bytes 11–n
$C2_h$	P1	NFCID2	PAYLOAD

P1

The P1 field shall be formatted as defined in Table E-2.

Table E-2: Format of the P1 field

b8	b7	b6	b5	b4	b3	b2	b1	Meaning
	Message Type			Message Features				
0	0	0	0	0	0	0		CAPDU_DATA
							x	Chaining, if set to 1_b
0	0	0	1	0	0	0	0	RW_ACK
0	0	1	0	0	0	0	0	NACK
Any other value								RFU

In the following sections, the name in the “Meaning” column is often used to refer to an ENCAPSULATION_CMD with a specific “Message Type” value.

Chaining set to 1_b indicates that the Payload field contains a segment of a larger data. If chaining is set to 1_b, the Command is referred to as “CAPDU_DATA indicating chaining.” If “indicating chaining” is not explicitly mentioned, the bit is set to 0_b.

The device shall ignore commands that have an RFU value.

NFCID2

The vehicle shall set the NFCID2 to the value included in the SENSF_RES response of the targeted device. The device shall compare the NFCID2 contained in this command with its own NFCID2. If they are not equal, the device shall not respond to this command.

PAYLOAD

For RW_ACK and NACK messages, the Payload field shall be empty.

The content of the Payload field for CAPDU_DATA is specified in Appendix E-5.

E.4.2. Encapsulation Response (ENCAPSULATION_RSP)

The ENCAPSULATION_RSP is used to transfer APDU data from the device to the vehicle.

The ENCAPSULATION_RSP shall be formatted as defined in Table E-3.

Table E-3: ENCAPSULATION_RSP format

Byte 1	Byte 2	Bytes 3–10	Bytes 11–n
$C3_h$	P1	NFCID2	PAYLOAD

P1

The P1 field shall be formatted as defined in Table E-4.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Table E-4: Format of the P1 field

b8	b7	b6	b5	b4	b3	b2	b1	Meaning
	Message Type			Message Features				
0	0	0	0	0	0	0		RAPDU_DATA
							x	Chaining, if set to 1_b
0	0	0	1	0	0	0	0	DEVICE_ACK
Any other value								RFU

In the following sections, the name in the "Meaning" column is often used to refer to an ENCAPSULATION_RSP with a specific "Message Type" value.

Chaining set to 1_b indicates that the Payload field contains a segment of a larger data. If chaining is set to 1_b, the response is referred to as "RAPDU_DATA indicating chaining.". If "indicating chaining" is not explicitly mentioned, the bit is set to 0_b.

NFCID2

The device shall set the NFCID2 to its own NFCID2. The vehicle shall verify that the NFCID2 in the response is the same as the one sent in the previous command. If the NFCID2 is different, the vehicle shall ignore the response.

PAYLOAD

For DEVICE_ACK messages, the Payload field shall be empty.

The content of the Payload field for RAPDU_DATA is specified in Appendix E.5.

E.5. Protocol Operation

E.5.1. Protocol activation

The protocol is implicitly activated if the device responds to a SENSF_REQ with a SENSF_RES that includes an NFCID2 that is associated with one of the system codes defined in Appendix E-2 (see [18] for more information on activation of a Type 3 Tag Platform and data exchange with a Type 3 Tag Platform).

The vehicle shall ignore the values of PAD1, $MRTI_{CHECK}$, and $MRTI_{UPDATE}$ as defined in [17].

E.5.2. General rules

After sending an ENCAPSULATION_CMD, the vehicle shall be ready to receive an ENCAPSULATION_RSP. After receiving an ENCAPSULATION_CMD without error, the device shall respond with an ENCAPSULATION_RSP.

If the length of the C-APDU is 244 bytes or less, the C-APDU shall be transported in the Payload field of a CAPDU_DATA message as defined in Appendix E.4.1.

If the length of the R-APDU is 244 bytes or less, the R-APDU shall be transported in the Payload field of an RAPDU_DATA message as defined in Appendix E.4.2.

For APDUs with a larger size, refer to Appendix E.5.3.

E.5.3. Chaining

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The chaining feature allows transmitting APDUs that do not fit in a single command or response by dividing it into segments. Each segment is transmitted in a separate command or response.

C-APDU

If the length of the C-APDU is above 244 bytes, the C-APDU shall be segmented and transferred in multiple CAPDU_DATA messages.

All but the last segment shall be transported in the Payload field of a CAPDU_DATA message indicating chaining as defined in Appendix E.4.1. C-APDU segments that are transported in a CAPDU_DATA message indicating chaining shall have a Payload field with a size of 244 bytes. The last segment shall be transported in the Payload field of a CAPDU_DATA message indicating no chaining.

The vehicle shall send the first segment of the C-APDU in a CAPDU_DATA message indicating chaining. After receiving a CAPDU_DATA message indicating chaining, the device shall acknowledge the receipt of the segment by sending a DEVICE_ACK as defined in Appendix E.4.2. After receiving a DEVICE_ACK, the vehicle shall send the next segment in a CAPDU_DATA message indicating chaining, or, if it is the last segment, in a CAPDU_DATA message indicating no chaining.

R-APDU

If the length of the R-APDU is greater than 244 bytes, the R-APDU shall be segmented and transferred in multiple RAPDU_DATA messages.

All but the last segment shall be transported in the Payload field of an RAPDU_DATA message indicating chaining as defined in Appendix E.4.2. R-APDU segments that are transported in an RAPDU_DATA message indicating chaining shall have a Payload field with a size of 244 bytes. The last segment shall be transported in the Payload field of an RAPDU_DATA message indicating no chaining.

The device shall send the first segment of the R-APDU in an RAPDU_DATA message indicating chaining. After receiving an RAPDU_DATA message indicating chaining, the vehicle shall acknowledge receipt of the segment by sending a RW_ACK as defined in Appendix E.4.1. After receiving a RW_ACK, the device shall send the next segment in an RAPDU_DATA message indicating chaining, or, if it is the last segment, in an RAPDU_DATA message indicating no chaining.

E.5.4. Timing requirements

The waiting time used by the protocol is announced in the SENSF_RES. The protocol uses one waiting time, referred to as RWT.

The SENSF_RES as defined in [17] has the following format:

Table E-5: SENSF_RES format

Byte 1	Bytes 2–9	Bytes 10–11	Bytes 12–14	Byte 15	Byte 16	Byte 17	Bytes 18–19
01_h	NFCID2	PAD0	Undefined Values (PAD1)	$MRTI_{CHECK}$	$MRTI_{UPDATE}$	Undefined Values (PAD2)	[RD]

In this specification, Byte 17 (PAD2) is referred to as RWTI. The device shall use a value for RWTI that reflects the maximum time it requires to respond to a CAPDU_DATA message.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

The Digital Key applet shall register its RWTI as part of its contactless protocol parameters for Type F. The RWTI corresponds to the last byte of the "Response Time Descriptor" contained in Sub-tag 81_h of the Type F anti-collision parameters entry tag $A0_h$ (see [21]. The recommended value for the first two bytes of the "Response Time Descriptor" (PAD0 of the SENSF_RES) is $01FF_h$.

The format of the RWTI byte shall be as defined in Table E-6:

Table E-6: Format of RWTI

b7	b6	b5	b4	b3	b2	b1	b0	Meaning
					x	x	x	Parameter A
		x	x	x				Parameter B
x	x							Parameter E

The RWT shall be calculated according to the following formula:

$$RWT = T \times ((A+1) + 8 \times (B+1)) \times 4^E$$

where T is a fixed duration with the following value: $T = 256 \times 16/f_c$ (~0.3020 ms).

After receiving the last byte of a ENCAPSULATION_CMD message, the device shall wait no longer than RWT before sending the first byte of an ENCAPSULATION_RSP (which is the LEN byte). If the vehicle does not receive the first byte of an ENCAPSULATION_RSP within RWT + ΔRWT, the vehicle shall treat this as a timeout error.

The value of ΔRWT shall be 20 ms.

E.5.5. Error handling

Device

The device shall not attempt any error recovery. The device shall stay in receive mode when it detects an error.

Vehicle

If a timeout error occurs, the vehicle shall resend the last command.

If the vehicle can detect transmission errors and if it receives a response with a transmission error, the vehicle shall send a NACK as defined in Appendix E.4.1.

After receiving a NACK, the device shall resend the last ENCAPSULATION_RSP.

The vehicle shall attempt sequential error recovery (without receiving a valid response in between) up to a maximum of two times.

If not specified otherwise, the handling of protocol errors by the vehicle is implementation-specific.

E.5.6. Protocol deactivation

The vehicle deactivates the protocol either by selecting another System Code via a SENSF_REQ or by switching off the operating field.

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Appendix F. Digital Key Framework API

This is a normative appendix, specifying API requirements.

F.1. Overview

The API requirements specified in this document enable mobile applications to access the Digital Key framework running the Android system in mobile devices. This specification provides interface definitions for an application developer to allow implementation on Android mobile platforms. The Android APIs are defined in Android Digital Key Framework API [29].

F.2. Functional Requirements

The Digital Key framework shall provide common Digital Key service functionality via a set of OS-specific APIs for Vehicle OEM apps. The following high-level services shall be provided to the Vehicle OEM app:

Owner pairing, key sharing, and key management. At a minimum, these APIs shall be able to be used to, respectively:

- Trigger owner pairing
- Trigger key sharing
- List Digital Keys

Additionally, the API should provide interfaces to enable the following functionalities:

- Owner pairing:
  - Trigger owner pairing and provide owner pairing password
  - Request/receive owner pairing status updates
- Key sharing:
  - For owner: Provide entitlements (including proprietary ones), friendly name, and retrieve sharing URL and sharing password from the framework.
  - For friend: Receive/send sharing URL and capture acceptance/consent.
  - Request/receive friend key status updates.
  - Send notification to Vehicle OEM app when a friend key is created on friend device.
- Key management:
  - Terminate Digital Key
  - List all owner/shared Digital Keys with status, entitlements, friendly names, vehicle identifier (vehicleIdentifier), and Digital Key identifier (keyID).
  - Deeplink to owner/shared keys to jump from Vehicle OEM app directly to the key in the device native representation.
  - Sign vehicle identifier (vehicleIdentifier) and Digital Key identifier (keyID) with attestable key from Digital Key applet instance on friend device to verify the identifiers in Vehicle OEM Server.
  - Sign data with attestable key from Digital Key applet instance on owner or friend device to verify signature in Vehicle OEM Server.
  - Read/write information related to Digital Key (e.g., information stored in private mailbox and Instance CA ID).

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

- RKE Functionality
  - Trigger event based RKE action
  - Trigger enduring RKE action
    - Callback providing the arbitrary data from request continuation confirmation while action endures
    - Sending of arbitrary data in confirm continuation message as answer to the request above
- Vehicle Management
  - Retrieve vehicle state
    - subscribing to a certain function id range
    - requesting a certain function status value from the vehicle
    - reading the stored status data.

Appendix G.

G.1. Ranging Session Parameter Tables [WCC3]

As part of the ranging session configuration, each ranging session is configured with the following values:

- $N^k_{\text{RAN}} \in \mathbb{N}^+$, set of all positive natural numbers.
- $N^k_{\text{RAN_S}} \in \mathbb{N}^+$, set of all positive natural numbers.
- $N^k_{\text{Chap_per_Slot}} \in \{3,4,6,8,9,12,24\}$
- $N^k_{\text{Slot_per_Round}} \in \{6,8,9,12,16,18,24,32,36,48,72,96\}$
- $N^k_{\text{Slot_per_Round}} \in \{6,8,9,12,16,18\}$ when the number of responder in each ranging round $N^k_{\text{Responder}} \leq 10$.

Table G-1 shows which combinations of $N^k_{\text{Chap_per_Slot}}$ and $N^k_{\text{Slot_per_Round}}$ result in valid ranging session configurations. During the handshake process for the *k*-th ranging session over the Bluetooth LE control channel (for details see Section 20.5.2), the Table G-1 is used to negotiate a valid ranging session configuration that maximizes the number of hopping rounds N^k_{Round} for a given set of constraint parameters $N^k_{\text{Chap_per_Slot}}$ and $N^k_{\text{Slot_per_Round}}$. With $N^k_{\text{Responder}} \leq 10$, the only the shaded part of the Table G-1 below will be used during this handshake process.

Table G-1: Number of Valid Ranging Rounds per 96 ms.

Number of Slots (per 96 ms)			$N^k_{\text{Slot_per_Round}}$											
$N^k_{\text{Chap_per_Slot}}$	Resulting T^k_{Slot}		6	8	9	12	16	18	24	32	36	48	72	96
	3	1.000 ms	16	12	N/A	8	6	N/A	4	3	N/A	2	N/A	1
	4	1.333 ms	12	9	8	6	N/A	4	3	N/A	2	N/A	1	N/A

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

Number of Slots (per 96 ms)			$N^k_{Slot_per_Round}$											
	6	2.000 ms	8	6	N/A	4	3	N/A	2	N/A	N/A	1	N/A	N/A
	8	2.667 ms	6	N/A	4	3	N/A	2	N/A	N/A	1	N/A	N/A	N/A
	9	3.000 ms	N/A	4	N/A	N/A	2	N/A	N/A	1	N/A	N/A	N/A	N/A
	12	4.000 ms	4	3	N/A	2	N/A	N/A	1	N/A	N/A	N/A	N/A	N/A
	24	8.000 ms	2	N/A	N/A	1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

G.2. Hopping Sequence

This section describes the method to generate the hopping sequence

G.3. Default Hopping Sequence:

This method is mandatory and shall be supported by any DK device.

For the *k*-th ranging session with N^k_{Round} ranging rounds per block and hoping key $HOP_Key_RW^k$, the following function is used to generate the hopping round Sk index for ranging block *i*:

$$S^k\left(i, HOP_Key_RW^k, N^k_{\text{Round}}\right) = \left(\left(\left(\left(i + HOP_Key_RW^k\right) \;\&\; 0\text{xFFFF}\right)^2 \bmod \left(2^{16} - 15\right)\right) N^k_{\text{Round}}\right) >> 16$$

The initiator shall generate and send the hopping key $HOP_Key_RW^k$ to the responder-device during the ranging session initiation process (see Section 20.5.2).

G.4. AES-Based Hopping Sequence:

This method is optional and may be supported by any DK device. It uses AES to generate the hopping sequence.

For the *k*-th ranging session with N^k_{Round} ranging rounds per block and hopping key $HOP_Key_RW^k$, the following function is used to generate the AES-based hopping round index for ranging block i:

$$S^k\left(i, HOP_Key_RW^k, N^k_{\text{Round}}\right) = \left(\left(\left(\text{AES}\left(i, HOP_Key_RW^k\right) \;\&\; 0\text{xFFFF}\right) N^k_{\text{Round}}\right) >> 16\right)$$

For the AES function, AES-128 shall be used. Here, the notation AES(plaintext, key) is used where the AES() operation pads each of the inputs to the left with 0x00 to reach the AES block size.

Appendix H. [WCC3]

The following Python code is contributed to IEEE Mentor as in [36] and implements a minimum-phase conversion from [34].

```
def min_phase(firseq, fft_pts):
    """A function to convert a FIR impulse response to min-phase
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
    A function that calculates the minimum-phase equivalent of a
FIR
    impulse response based on a Discrete Hilbert Transform
    applied in the frequency domain.
    """
    # Recommended fft_pts >= 32768
    #
    # For guidelines on the required number of FFT points, see:
    # N. Damera-Venkata, B. L. Evans and S. R. McCaslin,
    # "Design of optimal minimum-phase digital FIR filters using
    # discrete Hilbert transforms," in IEEE Transactions on
Signal
    # Processing, vol. 48, no. 5, pp. 1491-1495, May 2000.
    max_phase = np.real(np.fft.ifft( \
        np.exp(hilbert(np.real(np.log(np.fft.fft( \
        firseq, fft_pts)))))))
    tmp_min_phase = max_phase[::-1] # reverse max phase
    return tmp_min_phase[0:len(firseq)] # maintain length
```

Appendix I. [WCC3]

Pulse shapes using β of 0.45 with 32x oversampling per chip with total span of 8 chips are provided here for informative reference only. The samples are left to right, row by row from early to late samples. PulseShape 0x0 follows the mathematical equation in Section 21.5 and PulseShape 0x1 is converted from PulseShape 0x0 with the code shown in Appendix H. Implementation may choose to gradually smooth the edge samples toward zero.

I.1. PulseShape 0x0

-0.0089385	-0.0080918	-0.0070627	-0.0058655	-0.0045178	-0.0030411	-0.0014601
0.0001978	0.0019028	0.0036231	0.0053257	0.0069768	0.0085426	0.0099899
0.0112867	0.0124031	0.0133117	0.0139884	0.0144129	0.0145693	0.0144466
0.0140389	0.0133461	0.0123737	0.0111332	0.0096417	0.0079225	0.0060041
0.0039205	0.0017102	-0.0005840	-0.0029158	-0.0052361	-0.0074940	-0.0096374
-0.0116144	-0.0133739	-0.0148670	-0.0160477	-0.0168743	-0.0173102	-0.0173247
-0.0168943	-0.0160030	-0.0146437	-0.0128178	-0.0105365	-0.0078208	-0.0047014
-0.0012189	0.0025766	0.0066255	0.0108598	0.0152038	0.0195746	0.0238837
0.0280380	0.0319412	0.0354955	0.0386030	0.0411676	0.0430968	0.0443030

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

0.0447061 0.0442347 0.0428277 0.0404366 0.0370264 0.0325774 0.0270859
0.0205660 0.0130496 0.0045875 -0.0047503 -0.0148749 -0.0256783 -0.0370341
-0.0487983 -0.0608102 -0.0728941 -0.0848606 -0.0965085 -0.1076271 -0.1179985
-0.1273998 -0.1356065 -0.1423943 -0.1475430 -0.1508388 -0.1520776 -0.1510679
-0.1476337 -0.1416170 -0.1328811 -0.1213124 -0.1068231 -0.0893529 -0.0688710
-0.0453773 -0.0189032 0.0104872 0.0426970 0.0775967 0.1150248 0.1547883
0.1966642 0.2404012 0.2857218 0.3323244 0.3798865 0.4280672 0.4765109
0.5248505 0.5727115 0.6197154 0.6654839 0.7096429 0.7518264 0.7916805
0.8288673 0.8630684 0.8939885 0.9213591 0.9449406 0.9645258 0.9799418
0.9910521 0.9977580 1.0000000 0.9977580 0.9910521 0.9799418 0.9645258
0.9449406 0.9213591 0.8939885 0.8630684 0.8288673 0.7916805 0.7518264
0.7096429 0.6654839 0.6197154 0.5727115 0.5248505 0.4765109 0.4280672
0.3798865 0.3323244 0.2857218 0.2404012 0.1966642 0.1547883 0.1150248
0.0775967 0.0426970 0.0104872 -0.0189032 -0.0453773 -0.0688710 -0.0893529
-0.1068231 -0.1213124 -0.1328811 -0.1416170 -0.1476337 -0.1510679 -0.1520776
-0.1508388 -0.1475430 -0.1423943 -0.1356065 -0.1273998 -0.1179985 -0.1076271
-0.0965085 -0.0848606 -0.0728941 -0.0608102 -0.0487983 -0.0370341 -0.0256783
-0.0148749 -0.0047503 0.0045875 0.0130496 0.0205660 0.0270859 0.0325774
0.0370264 0.0404366 0.0428277 0.0442347 0.0447061 0.0443030 0.0430968
0.0411676 0.0386030 0.0354955 0.0319412 0.0280380 0.0238837 0.0195746
0.0152038 0.0108598 0.0066255 0.0025766 -0.0012189 -0.0047014 -0.0078208
-0.0105365 -0.0128178 -0.0146437 -0.0160030 -0.0168943 -0.0173247 -0.0173102
-0.0168743 -0.0160477 -0.0148670 -0.0133739 -0.0116144 -0.0096374 -0.0074940
-0.0052361 -0.0029158 -0.0005840 0.0017102 0.0039205 0.0060041 0.0079225
0.0096417 0.0111332 0.0123737 0.0133461 0.0140389 0.0144466 0.0145693
0.0144129 0.0139884 0.0133117 0.0124031 0.0112867 0.0099899 0.0085426
0.0069768 0.0053257 0.0036231 0.0019028 0.0001978 -0.0014601 -0.0030411
-0.0045178 -0.0058655 -0.0070627 -0.0080918

I.2. PulseShape 0x2

0.0193408 0.0273888 0.0369563 0.0481487 0.0610612 0.0757769 0.0923654
0.1108803 0.1313579 0.1538153 0.1782489 0.2046337 0.2329215 0.2630407

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

1	0.2948957	0.3283664	0.3633086	0.3995544	0.4369126	0.4751700	0.5140929
2	0.5534282	0.5929059	0.6322413	0.6711370	0.7092864	0.7463760	0.7820887
3	0.8161076	0.8481184	0.8778138	0.9048965	0.9290824	0.9501048	0.9677167
4	0.9816948	0.9918417	0.9979891	1.0000000	0.9977710	0.9912337	0.9803566
5	0.9651456	0.9456452	0.9219380	0.8941451	0.8624247	0.8269719	0.7880160
6	0.7458198	0.7006760	0.6529055	0.6028537	0.5508875	0.4973912	0.4427634
7	0.3874123	0.3317516	0.2761968	0.2211602	0.1670473	0.1142522	0.0631537
8	0.0141115	-0.0325377	-0.0764831	-0.1174435	-0.1551694	-0.1894459	-0.2200942
9	-0.2469734	-0.2699813	-0.2890554	-0.3041727	-0.3153495	-0.3226406	-0.3261383
10	-0.3259704	-0.3222984	-0.3153151	-0.3052419	-0.2923256	-0.2768351	-0.2590582
11	-0.2392974	-0.2178666	-0.1950870	-0.1712832	-0.1467796	-0.1218960	-0.0969449
12	-0.0722271	-0.0480290	-0.0246194	-0.0022470	0.0188624	0.0385076	0.0565141
13	0.0727348	0.0870515	0.0993751	0.1096460	0.1178335	0.1239357	0.1279780
14	0.1300121	0.1301142	0.1283833	0.1249388	0.1199185	0.1134756	0.1057763
15	0.0969971	0.0873217	0.0769383	0.0660371	0.0548067	0.0434326	0.0320936
16	0.0209603	0.0101925	-0.0000627	-0.0096722	-0.0185187	-0.0265018	-0.0335384
17	-0.0395639	-0.0445319	-0.0484147	-0.0512024	-0.0529029	-0.0535408	-0.0531563
18	-0.0518044	-0.0495529	-0.0464813	-0.0426788	-0.0382426	-0.0332763	-0.0278875
19	-0.0221866	-0.0162841	-0.0102898	-0.0043100	0.0015533	0.0072041	0.0125540
20	0.0175230	0.0220408	0.0260473	0.0294936	0.0323420	0.0345664	0.0361523
21	0.0370968	0.0374078	0.0371038	0.0362131	0.0347730	0.0328289	0.0304330
22	0.0276434	0.0245228	0.0211373	0.0175550	0.0138452	0.0100764	0.0063158
23	0.0026280	-0.0009263	-0.0042910	-0.0074151	-0.0102543	-0.0127706	-0.0149333
24	-0.0167193	-0.0181129	-0.0191063	-0.0196988	-0.0198975	-0.0197161	-0.0191746
25	-0.0182993	-0.0171212	-0.0156759	-0.0140028	-0.0121439	-0.0101431	-0.0080455
26	-0.0058961	-0.0037395	-0.0016186	0.0004259	0.0023562	0.0041382	0.0057420
27	0.0071423	0.0083185	0.0092556	0.0099438	0.0103786	0.0105611	0.0104975
28	0.0101990	0.0096813	0.0089643	0.0080718	0.0070304	0.0058694	0.0046200
29	0.0033145	0.0019857	0.0006661	-0.0006126	-0.0018203	-0.0029293	-0.0039144
30	-0.0047540	-0.0054301	-0.0059290	-0.0062416	-0.0063635	-0.0062950	-0.0060415
31	-0.0056132	-0.0050251	-0.0042966	-0.0034510	-0.0025154	-0.0015200	-0.0004973
32	0.00051848	0.00149222	0.00238888	0.00317447	0.00381706	0.00428794	0.00456281
33	0.00462307	0.00445708	0.00406159	0.00344317	0.00261972	0.00162203	0.00049542

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

-0.0006985 -0.0018802 -0.0029501 -0.0037867 -0.0042448 -0.0041529 -0.0033108
-0.0014877 0.00158115 0.00619674 0.0000000319

Appendix J. [WCC3]

J.1. UWB Test Vector

```
Values are Hexadecimal unless labeled Decimal.
STSIndex0_Decimal: 123456789
STSIndex0_Hex_BigEndian: 075bcd15
URSK: ed07a80d2beb00f785af2627c96ae7c118504243cb2c3226b3679daa0f7e616c
mUPSK1: 94ce7a78b643d0cac4e2e4f8b5f55a4d
mUPSK2: a93a052ba546f74aa1b771dfb8c89f503a199bcc7af64ad2881c94b320a1243e
UAD: 8b779d0709d9af2c324320afb5301a8f
Keysource-low: 8b77
Keysource-high: 9d07
DestinationShortAddress: 09d9
SourceLongAddress: af2c324320afb530
mURSK: 1cac2a6a5b04490c86089bb989c7d35987c47efc3fff71133bd98e6d7966de70
Salt: c56ee5c153daf28f917c6081c8bb63ca
PaddedSalt: 00000000000000000000000000000000c56ee5c153daf28f917c6081c8bb63ca
Selected_Protocol_Version: 0100
Selected_UWB_Config_Id: 0000
UWB_Session_Id: 12345678
STS_Index0: 075bcd15
Number_Responder_Nodes: 01
Session_RAN_Multiplier: 0a
Number_Slot_per_Round: 06
Number_Chaps_per_Slot: 06
Selected_PulseShape_Combo: 00
Ranging Configuration: 0100000012345678075bcd15010a060600
SaltedHash: 79e865186cbd864b955682c51cdd34f8
STSIndex_Decimal: 123456789
KDF Cascade Context:
0000000000010101000100010100010101010000010100010000000100010001
URSK_KT: d8c951ca6029a8ff5c89199b6ae07387301056ac3462f93db1afb2009cd89d2f
dURSK: 4578aa54d663cb5aa88070cd01c604a7
dUDSK: 932e0f6d9bb393096fe24cc79a704447
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
Second Block (Block index i=1), first Ranging Round (Round index s=0)
STSIndex_Decimal: 123457269
KDF Cascade Context:
00000000000101010001000101000101010100000101010001010101000010001
URSK_KT: ebd253fbf2c57294284de578fee5de984f3489d740e6fd5ee50af537c714af6e
dURSK: 9d40e2080db7d2cda114838189d7ae9f
dUDSK: 011adc74ba889e7e59a8ee61889d3fec
```

J.2. UWB Test Vector without hopping

Based on the UWB Test Vector of J.1 the following exchange of UWB PPDUs may occur:

```
Values are Hexadecimal.
First Ranging block:
    STS_Index for Pre Poll: 07 5B CD 15
    Frame Counter for Pre Poll: 00 00 00 00
        CMAC-Data(URSK_KT): 00 00 00 01 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 00
                            01 01 00 01 00 00 00 01 00 01 00 01 00 00 01 00
        CMAC-Data(URSK_KT): 00 00 00 02 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 00
                            01 01 00 01 00 00 00 01 00 01 00 01 00 00 01 00
    URSK_KT: D8 C9 51 CA 60 29 A8 FF 5C 89 19 9B 6A E0 73 87
             30 10 56 AC 34 62 F9 3D B1 AF B2 00 9C D8 9D 2F
        CMAC-Data(URSK): 00 00 00 01 55 52 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dURSK: 45 78 AA 54 D6 63 CB 5A A8 80 70 CD 01 C6 04 A7
        CMAC-Data(UDSK): 00 00 00 01 55 44 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dUDSK: 93 2E 0F 6D 9B B3 93 09 6F E2 4C C7 9A 70 44 47
    PrePoll MHR: 49 2B D9 09 16 00 00 00 00 77 8B 07 9D AA 05 00
                 69 DF 04 01 0D 80 3F
    PrePoll Payload: 78 56 34 12 16 CD 5B 07 00 00 00 00 00
        AES-CCM* key K: 94 CE 7A 78 B6 43 D0 CA C4 E2 E4 F8 B5 F5 5A 4D
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 00 06
        AddAuthData: 00 17 49 2B D9 09 16 00 00 00 00 77 8B 07 9D AA
                     05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 16 CD 5B 07 00 00 00 00 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 00 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 16 CD 5B 07 00 00 00 00 00 00 00 00
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 00 06 00 0D
        X: DD 38 D8 1E 45 5A 1B AD A3 58 8C 62 6C 90 EF E7
           A2 AB 2B AE DE C2 E5 B4 D1 42 9B AF 4D 48 2F 9B
           E5 2E A2 52 9C 05 9F DD 0F 78 B9 BB A5 54 4D B8
           CB 2A A9 D5 17 55 E2 40 00 93 E1 78 86 5F 63 D2
        T: CB 2A A9 D5 17 55 E2 40
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 00 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 00 06 00 01
        C: 4A 87 F6 F8 FD EC EF EA EA 19 34 00 43 4D CC 06
        S0: 11 43 A9 A2 0F F6 E6 C3 B6 CE 61 6C 47 85 65 96
        CipherText: 32 D1 C2 EA EB 21 B4 ED EA 19 34 00 43
        U: DA 69 00 77 18 A3 04 83
    PrePoll encrypted Payload: 32 D1 C2 EA EB 21 B4 ED EA 19 34 00 43
                               DA 69 00 77 18 A3 04 83
    STS-V for         POLL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 4F DB 1C DD 34 F8 -
                                 1C DD 35 17
      STS for         POLL PPDU: E8 AE 41 D3 4E 61 87 10 95 A6 2B 65 EF 1F 15 50 ..
                                 9D 5F 97 B1 EB 2B 09 55 40 B2 87 72 4B D6 88 76
    STS-V for  1. RESPONSE PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 4F DC 1C DD 34 F8 -
                                 1C DD 35 17
      STS for  1. RESPONSE PPDU: FA 26 6A E7 77 40 FA 4A E6 81 2A F3 E9 69 9D A2 ..
                                 1D 71 EA 7E 7D 0E A1 47 CC EB F3 18 DF 1D EC 9A
    STS-V for        FINAL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 4F DD 1C DD 34 F8 -
                                 1C DD 35 17
      STS for        FINAL PPDU: 51 76 79 08 78 9D 7D B7 49 1C 87 4B 8E 2E AF 7E ..
                                 87 8E C1 D3 3E D9 79 80 E5 E3 2D 7E 17 97 46 E0
    FinaData MHR: 49 2B D9 09 16 01 00 00 00 77 8B 07 9D AA 05 00
                  69 DF 04 02 19 80 3F
    FinaData Payload: 78 56 34 12 00 00 00 00 00 18 CD 5B 07 00 00 3C
                      0F 01 00 FF 04 9E 07 29 00
        AES-CCM* key K: 93 2E 0F 6D 9B B3 93 09 6F E2 4C C7 9A 70 44 47
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 01 06
        AddAuthData: 00 17 49 2B D9 09 16 01 00 00 00 77 8B 07 9D AA
                     05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 00 00 00 00 00 18 CD 5B 07 00 00 3C
                       0F 01 00 FF 04 9E 07 29 00 00 00 00 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 01 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 00 00 00 00 00 18 CD 5B 07 00 00 3C
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
                  0F 01 00 FF 04 9E 07 29 00 00 00 00 00 00 00 00
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 01 06 00 19
        X: 1A C5 15 F3 E1 77 DB A3 1D 20 DC 57 35 10 69 AD
           3A 51 46 95 AD A3 CB 01 EC 16 C4 BB E2 3F 28 D5
           BA CC 1B 33 47 D5 13 BB 39 40 56 13 78 AC C9 77
           8B EF 16 7F BC DA 85 BD AF B6 BB 8B 80 F0 4C E3
           B0 DA 6F BE 11 35 0C 2F 5C 80 50 DF 0B 7C CF D0
        T: B0 DA 6F BE 11 35 0C 2F
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 01 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 01 06 00 01
        A2: 01 AF 2C 32 43 20 AF B5 30 00 00 00 01 06 00 02
        C: E2 55 9D 5B EA 8C 3E 87 44 4C 08 F3 7B 2D 5F 10
           3C F2 DB 4A AB 57 E7 43 B7 78 D5 4D C2 AD C6 38
        S0: 2B 36 52 7B F4 57 8F FD CB C0 82 81 DD CD BB 6F
        CipherText: 9A 03 A9 49 EA 8C 3E 87 44 54 C5 A8 7C 2D 5F 2C
                    33 F3 DB B5 AF C9 E0 6A B7
        U: 9B EC 3D C5 E5 62 83 D2
    FinaData encrypted Payload: 9A 03 A9 49 EA 8C 3E 87 44 54 C5 A8 7C 2D 5F 2C
                                33 F3 DB B5 AF C9 E0 6A B7 9B EC 3D C5 E5 62 83 D2
Second Ranging block:
    STS_Index for Pre Poll: 07 5B CE F5
    Frame Counter for Pre Poll: 00 00 00 02
        CMAC-Data(URSK_KT): 00 00 00 01 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 00
                            01 01 01 00 01 01 01 01 00 01 00 01 00 00 01 00
        CMAC-Data(URSK_KT): 00 00 00 02 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 00
                            01 01 01 00 01 01 01 01 00 01 00 01 00 00 01 00
    URSK_KT: EB D2 53 FB F2 C5 72 94 28 4D E5 78 FE E5 DE 98
             4F 34 89 D7 40 E6 FD 5E E5 0A F5 37 C7 14 AF 6E
        CMAC-Data(URSK): 00 00 00 01 55 52 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dURSK: 9D 40 E2 08 0D B7 D2 CD A1 14 83 81 89 D7 AE 9F
        CMAC-Data(UDSK): 00 00 00 01 55 44 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dUDSK: 01 1A DC 74 BA 88 9E 7E 59 A8 EE 61 88 9D 3F EC
    PrePoll MHR: 49 2B D9 09 16 02 00 00 00 77 8B 07 9D AA 05 00
                 69 DF 04 01 0D 80 3F
    PrePoll Payload: 78 56 34 12 F6 CE 5B 07 01 00 00 00 00
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        AES-CCM* key K: 94 CE 7A 78 B6 43 D0 CA C4 E2 E4 F8 B5 F5 5A 4D
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 02 06
        AddAuthData: 00 17 49 2B D9 09 16 02 00 00 00 77 8B 07 9D AA
                     05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 F6 CE 5B 07 01 00 00 00 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 02 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 F6 CE 5B 07 01 00 00 00 00 00 00 00
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 02 06 00 0D
        X: 2F A9 D9 44 AA 74 A0 6F C7 CD AD 33 25 74 8A 5D
           7E 8C 86 E7 4D 2D EF 05 8F AC AE 3C 02 88 3C 85
           BC 7F 2D BB EE FE 33 9B 1B 77 F0 85 CE B1 0A AD
           94 EF E7 D4 A6 04 BE 54 1F 47 96 EE E7 23 BA 94
        T: 94 EF E7 D4 A6 04 BE 54
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 02 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 02 06 00 01
        C: BA 38 C2 2E 8C E7 D5 C8 DC 89 27 31 E7 77 D0 84
        S0: 6B 6D EE 0F 3E D9 56 7B C1 69 45 5B 33 14 8F 4D
        CipherText: C2 6E F6 3C 7A 29 8E CF DD 89 27 31 E7
        U: FF 82 09 DB 98 DD E8 2F
    PrePoll encrypted Payload: C2 6E F6 3C 7A 29 8E CF DD 89 27 31 E7
                               FF 82 09 DB 98 DD E8 2F
    STS-V for         POLL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 51 BB 1C DD 34 F8 -
                                 1C DD 35 17
      STS for         POLL PPDU: BE 4E 6F 9D 5B 1C 4B 76 51 AF 10 47 40 BA 22 C3 ..
                                 45 24 6A 04 FB AE 29 81 58 D2 70 E9 1D EB F6 C3
    STS-V for  1. RESPONSE PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 51 BC 1C DD 34 F8 -
                                 1C DD 35 17
      STS for  1. RESPONSE PPDU: 68 D3 38 09 A2 6A 26 08 47 0E D7 CC 14 67 2F D1 ..
                                 59 F4 F9 EE 2D EC DB CE 03 0D 76 4C 75 A1 7C FA
    STS-V for        FINAL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 51 BD 1C DD 34 F8 -
                                 1C DD 35 17
      STS for        FINAL PPDU: C0 B1 17 8D 50 60 C2 D6 70 0C C2 18 29 F6 BC 45 ..
                                 B7 26 0E C9 6E 2C DE BB 1E 2E 2D 42 D3 58 26 39
    FinaData MHR: 49 2B D9 09 16 03 00 00 00 77 8B 07 9D AA 05 00
                  69 DF 04 02 19 80 3F
    FinaData Payload: 78 56 34 12 01 00 00 00 00 F8 CE 5B 07 00 00 3C
                      0F 01 00 04 05 9E 07 29 00
        AES-CCM* key K: 01 1A DC 74 BA 88 9E 7E 59 A8 EE 61 88 9D 3F EC
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 03 06
        AddAuthData: 00 17 49 2B D9 09 16 03 00 00 00 77 8B 07 9D AA
                     05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 01 00 00 00 00 F8 CE 5B 07 00 00 3C
                       0F 01 00 04 05 9E 07 29 00 00 00 00 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 03 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 01 00 00 00 00 F8 CE 5B 07 00 00 3C
                  0F 01 00 04 05 9E 07 29 00 00 00 00 00 00 00 00
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 03 06 00 19
        X: AA 21 1A C9 71 A4 FA B8 39 20 22 11 DA 71 B8 B1
           44 44 62 2D E7 20 05 78 AF F1 86 A8 30 6C 5C B8
           A8 00 46 D8 0A 92 EB F0 3D CD F5 AD F3 E3 92 F2
           B0 77 A4 86 F9 E3 0D AE EB AC C8 67 BF 6A 45 32
           42 DF 7B 1E B6 60 4B 9A 95 DC D0 49 82 33 93 B3
        T: 42 DF 7B 1E B6 60 4B 9A
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 03 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 03 06 00 01
        A2: 01 AF 2C 32 43 20 AF B5 30 00 00 00 03 06 00 02
        C: 89 E7 B9 4F 63 EA 41 AC BA 5A BF 44 41 71 6C 01
           0C C4 F9 0B CC 09 FB 62 66 80 74 6F 8E 5A 76 F0
        S0: 09 7D 64 94 03 79 2F 64 E3 48 D8 DC BA 59 AC A5
        CipherText: F1 B1 8D 5D 62 EA 41 AC BA A2 71 1F 46 71 6C 3D
                    03 C5 F9 0F C9 97 FC 4B 66
        U: 4B A2 1F 8A B5 19 64 FE
    FinaData encrypted Payload: F1 B1 8D 5D 62 EA 41 AC BA A2 71 1F 46 71 6C 3D
03 C5 F9 0F C9 97 FC 4B 66 4B A2 1F 8A B5 19 64 FE
```

J.3. UWB Test Vector with continuous hopping

Based on the UWB Test Vector of J.1 the following exchange of UWB PPDUs may occur:

```
Values are Hexadecimal.
    Hop_Mode_Key for Default Hopping Sequence: CC 5D D7 9F
    Rounds per Block (Nround): 00 50
First Ranging block:
    STS_Index for Pre Poll: 07 5B CD 15
    Frame Counter for Pre Poll: 00 00 00 00
        CMAC-Data(URSK_KT): 00 00 00 01 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 00
                            01 01 00 01 00 00 00 01 00 01 00 01 00 00 01 00
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        CMAC-Data(URSK_KT): 00 00 00 02 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 00
                            01 01 00 01 00 00 00 01 00 01 00 01 00 00 01 00
    URSK_KT: D8 C9 51 CA 60 29 A8 FF 5C 89 19 9B 6A E0 73 87
             30 10 56 AC 34 62 F9 3D B1 AF B2 00 9C D8 9D 2F
        CMAC-Data(URSK): 00 00 00 01 55 52 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dURSK: 45 78 AA 54 D6 63 CB 5A A8 80 70 CD 01 C6 04 A7
        CMAC-Data(UDSK): 00 00 00 01 55 44 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dUDSK: 93 2E 0F 6D 9B B3 93 09 6F E2 4C C7 9A 70 44 47
    PrePoll MHR: 49 2B D9 09 16 00 00 00 00 77 8B 07 9D AA 05 00
                 69 DF 04 01 0D 80 3F
    PrePoll Payload: 78 56 34 12 16 CD 5B 07 00 00 00 00 00
        AES-CCM* key K: 94 CE 7A 78 B6 43 D0 CA C4 E2 E4 F8 B5 F5 5A 4D
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 00 06
        AddAuthData: 00 17 49 2B D9 09 16 00 00 00 00 77 8B 07 9D AA
                     05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 16 CD 5B 07 00 00 00 00 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 00 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 16 CD 5B 07 00 00 00 00 00 00 00 00
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 00 06 00 0D
        X: DD 38 D8 1E 45 5A 1B AD A3 58 8C 62 6C 90 EF E7
           A2 AB 2B AE DE C2 E5 B4 D1 42 9B AF 4D 48 2F 9B
           E5 2E A2 52 9C 05 9F DD 0F 78 B9 BB A5 54 4D B8
           CB 2A A9 D5 17 55 E2 40 00 93 E1 78 86 5F 63 D2
        T: CB 2A A9 D5 17 55 E2 40
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 00 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 00 06 00 01
        C: 4A 87 F6 F8 FD EC EF EA EA 19 34 00 43 4D CC 06
        S0: 11 43 A9 A2 0F F6 E6 C3 B6 CE 61 6C 47 85 65 96
        CipherText: 32 D1 C2 EA EB 21 B4 ED EA 19 34 00 43
        U: DA 69 00 77 18 A3 04 83
    PrePoll encrypted Payload: 32 D1 C2 EA EB 21 B4 ED EA 19 34 00 43
                               DA 69 00 77 18 A3 04 83
    STS-V for         POLL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 4F DB 1C DD 34 F8 -
                                 1C DD 35 17
      STS for         POLL PPDU: E8 AE 41 D3 4E 61 87 10 95 A6 2B 65 EF 1F 15 50 ..
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
                              9D 5F 97 B1 EB 2B 09 55 40 B2 87 72 4B D6 88 76
    STS-V for  1. RESPONSE PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 4F DC 1C DD 34 F8 -
                              1C DD 35 17
      STS for  1. RESPONSE PPDU: FA 26 6A E7 77 40 FA 4A E6 81 2A F3 E9 69 9D A2 ..
                              1D 71 EA 7E 7D 0E A1 47 CC EB F3 18 DF 1D EC 9A
    STS-V for        FINAL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 4F DD 1C DD 34 F8 -
                              1C DD 35 17
      STS for        FINAL PPDU: 51 76 79 08 78 9D 7D B7 49 1C 87 4B 8E 2E AF 7E ..
                              87 8E C1 D3 3E D9 79 80 E5 E3 2D 7E 17 97 46 E0
    Round Index of next Ranging Block (Default Hopping): 00 3E
    FinaData MHR: 49 2B D9 09 16 01 00 00 00 77 8B 07 9D AA 05 00
                 69 DF 04 02 19 80 3F
    FinaData Payload: 78 56 34 12 00 00 01 3E 00 18 CD 5B 07 00 00 3C
                     0F 01 00 C1 04 9E 07 2C 00
        AES-CCM* key K: 93 2E 0F 6D 9B B3 93 09 6F E2 4C C7 9A 70 44 47
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 01 06
        AddAuthData: 00 17 49 2B D9 09 16 01 00 00 00 77 8B 07 9D AA
                    05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 00 00 01 3E 00 18 CD 5B 07 00 00 3C
                       0F 01 00 C1 04 9E 07 2C 00 00 00 00 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 01 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 00 00 01 3E 00 18 CD 5B 07 00 00 3C
                  0F 01 00 C1 04 9E 07 2C 00 00 00 00 00 00 00 00
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 01 06 00 19
        X: 1A C5 15 F3 E1 77 DB A3 1D 20 DC 57 35 10 69 AD
           3A 51 46 95 AD A3 CB 01 EC 16 C4 BB E2 3F 28 D5
           BA CC 1B 33 47 D5 13 BB 39 40 56 13 78 AC C9 77
           F0 6B A2 25 F6 49 B4 7C EF C1 6B F9 0C C7 82 04
           64 97 62 53 2E CE 36 69 95 42 5D A6 CE 40 95 54
        T: 64 97 62 53 2E CE 36 69
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 01 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 01 06 00 01
        A2: 01 AF 2C 32 43 20 AF B5 30 00 00 00 01 06 00 02
        C: E2 55 9D 5B EA 8C 3E 87 44 4C 08 F3 7B 2D 5F 10
           3C F2 DB 4A AB 57 E7 43 B7 78 D5 4D C2 AD C6 38
        S0: 2B 36 52 7B F4 57 8F FD CB C0 82 81 DD CD BB 6F
        CipherText: 9A 03 A9 49 EA 8C 3F B9 44 54 C5 A8 7C 2D 5F 2C
                    33 F3 DB 8B AF C9 E0 6F B7
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        U: 4F A1 30 28 DA 99 B9 94
    FinaData encrypted Payload: 9A 03 A9 49 EA 8C 3F B9 44 54 C5 A8 7C 2D 5F 2C
                                33 F3 DB 8B AF C9 E0 6F B7 4F A1 30 28 DA 99 B9 94
Second Ranging block:
    STS_Index for Pre Poll: 07 5B D0 69
    Frame Counter for Pre Poll: 00 00 00 02
        CMAC-Data(URSK_KT): 00 00 00 01 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 01
                            00 00 00 00 00 01 01 00 01 00 00 01 00 00 01 00
        CMAC-Data(URSK_KT): 00 00 00 02 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 01
                            00 00 00 00 00 01 01 00 01 00 00 01 00 00 01 00
    URSK_KT: 06 32 7C C5 A3 6C AF 38 6A E3 F1 3F C8 CC A4 40
             42 7A A6 D9 A4 71 A6 E8 9C E4 63 22 F5 79 FE 73
        CMAC-Data(URSK): 00 00 00 01 55 52 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dURSK: 74 36 F1 90 14 51 4D C9 6F 90 01 FF 55 EC 87 47
        CMAC-Data(UDSK): 00 00 00 01 55 44 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dUDSK: 42 C8 4F 42 03 CE 36 DF 09 58 FF 21 CB 39 F2 F0
    PrePoll MHR: 49 2B D9 09 16 02 00 00 00 77 8B 07 9D AA 05 00
                 69 DF 04 01 0D 80 3F
    PrePoll Payload: 78 56 34 12 6A D0 5B 07 01 00 01 3E 00
        AES-CCM* key K: 94 CE 7A 78 B6 43 D0 CA C4 E2 E4 F8 B5 F5 5A 4D
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 02 06
        AddAuthData: 00 17 49 2B D9 09 16 02 00 00 00 77 8B 07 9D AA
                     05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 6A D0 5B 07 01 00 01 3E 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 02 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 6A D0 5B 07 01 00 01 3E 00 00 00 00
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 02 06 00 0D
        X: 2F A9 D9 44 AA 74 A0 6F C7 CD AD 33 25 74 8A 5D
           7E 8C 86 E7 4D 2D EF 05 8F AC AE 3C 02 88 3C 85
           BC 7F 2D BB EE FE 33 9B 1B 77 F0 85 CE B1 0A AD
           4D 10 94 4C 36 D7 C3 B4 5A 3F D1 A6 51 8A B4 30
        T: 4D 10 94 4C 36 D7 C3 B4
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 02 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 02 06 00 01
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        C: BA 38 C2 2E 8C E7 D5 C8 DC 89 27 31 E7 77 D0 84
        S0: 6B 6D EE 0F 3E D9 56 7B C1 69 45 5B 33 14 8F 4D
        CipherText: C2 6E F6 3C E6 37 8E CF DD 89 26 0F E7
        U: 26 7D 7A 43 08 0E 95 CF
    PrePoll encrypted Payload: C2 6E F6 3C E6 37 8E CF DD 89 26 0F E7
                               26 7D 7A 43 08 0E 95 CF
    STS-V for         POLL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 53 2F 1C DD 34 F8 -
                                 1C DD 35 17
      STS for         POLL PPDU: 8F 33 3E 3D D2 5C 0F 57 EB 8B 25 9D 3D 73 DB BC ..
                                 F4 12 FA A3 01 20 83 02 FF E0 04 AC 43 60 D3 C5
    STS-V for  1. RESPONSE PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 53 30 1C DD 34 F8 -
                                 1C DD 35 17
      STS for  1. RESPONSE PPDU: C7 2B 1A 1B CE 0C 86 C8 E8 28 2A 0B 44 3F F5 FB ..
                                 B3 CE 69 49 6D FD 40 61 D8 DA B9 DA 6A FA 88 D5
    STS-V for        FINAL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 53 31 1C DD 34 F8 -
                                 1C DD 35 17
      STS for        FINAL PPDU: 41 B5 25 D1 2F DE 4A DC 74 02 F5 EB F8 15 28 AB ..
                                 B2 58 5D 96 82 ED 93 A8 E3 2C FE A4 37 EA 00 D5
    Round Index of next Ranging Block (Default Hopping): 00 25
    FinaData MHR: 49 2B D9 09 16 03 00 00 00 77 8B 07 9D AA 05 00
                  69 DF 04 02 19 80 3F
    FinaData Payload: 78 56 34 12 01 00 01 25 00 6C D0 5B 07 00 00 3C
                      0F 01 00 B1 04 9E 07 2C 00
        AES-CCM* key K: 42 C8 4F 42 03 CE 36 DF 09 58 FF 21 CB 39 F2 F0
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 03 06
        AddAuthData: 00 17 49 2B D9 09 16 03 00 00 00 77 8B 07 9D AA
                     05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 01 00 01 25 00 6C D0 5B 07 00 00 3C
                       0F 01 00 B1 04 9E 07 2C 00 00 00 00 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 03 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 01 00 01 25 00 6C D0 5B 07 00 00 3C
                  0F 01 00 B1 04 9E 07 2C 00 00 00 00 00 00 00 00
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 03 06 00 19
        X: F1 F4 F4 BB 98 C3 45 B1 A1 39 09 05 50 0C 89 B2
           BB 2C B1 43 80 48 6B E5 2B BD E6 E4 70 72 ED EA
           62 26 07 E2 92 0C B7 C0 C8 5A 3C 75 04 5F 2C D7
           C8 67 6D FA 41 A6 3A 92 B5 8B 9D 63 7F B1 9E C8
           4B BD B3 37 B3 B0 89 34 E2 15 B7 48 14 42 64 65
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
        T: 4B BD B3 37 B3 B0 89 34
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 03 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 03 06 00 01
        A2: 01 AF 2C 32 43 20 AF B5 30 00 00 00 03 06 00 02
        C: 5F 13 1C E8 FF C1 D0 C6 9B 92 4E 47 B5 27 09 4C
           2B C8 5C F0 57 B9 1D E5 9B 33 4E 9A 31 81 D2 46
        S0: C8 E7 95 98 8E 7B 1A 69 48 2A 3B EF CE 43 C5 A7
        CipherText: 27 45 28 FA FE C1 D1 E3 9B FE 9E 1C B2 27 09 70
                    24 C9 5C 41 53 27 1A C9 9B
        U: 83 5A 26 AF 3D CB 93 5D
    FinaData encrypted Payload: 27 45 28 FA FE C1 D1 E3 9B FE 9E 1C B2 27 09 70
                                24 C9 5C 41 53 27 1A C9 9B 83 5A 26 AF 3D CB 93 5D
Third Ranging block:
    STS_Index for Pre Poll: 07 5B D1 B3
    Frame Counter for Pre Poll: 00 00 00 04
        CMAC-Data(URSK_KT): 00 00 00 01 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 01
                            00 00 00 01 01 00 01 01 00 00 01 01 00 00 01 00
        CMAC-Data(URSK_KT): 00 00 00 02 55 52 53 4B 5F 4B 54 00 00 00 00 00
                            00 01 01 01 00 01 00 01 01 00 01 01 01 01 00 01
                            00 00 00 01 01 00 01 01 00 00 01 01 00 00 01 00
    URSK_KT: 2F B7 E8 39 2D E1 00 6E 6C CB 83 83 AB 12 B5 C0
             29 85 F4 87 65 F1 C0 B7 89 7F 61 1C B1 67 D8 0D
        CMAC-Data(URSK): 00 00 00 01 55 52 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dURSK: A2 B7 89 A9 27 0E 74 85 D5 D9 48 D1 73 1B 92 4B
        CMAC-Data(UDSK): 00 00 00 01 55 44 53 4B 00 79 E8 65 18 6C BD 86
                         4B 95 56 82 C5 1C DD 34 F8 00 00 00 00 00 00 80
    dUDSK: CF 27 C7 7E 15 B9 E6 B9 88 62 90 32 FF 9D EE C9
    PrePoll MHR: 49 2B D9 09 16 04 00 00 00 77 8B 07 9D AA 05 00
                 69 DF 04 01 0D 80 3F
    PrePoll Payload: 78 56 34 12 B4 D1 5B 07 02 00 01 25 00
        AES-CCM* key K: 94 CE 7A 78 B6 43 D0 CA C4 E2 E4 F8 B5 F5 5A 4D
        Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 04 06
        AddAuthData: 00 17 49 2B D9 09 16 04 00 00 00 77 8B 07 9D AA
                     05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
        PlainTextData: 78 56 34 12 B4 D1 5B 07 02 00 01 25 00 00 00 00
        AuthData: 00 17 49 2B D9 09 16 04 00 00 00 77 8B 07 9D AA
                  05 00 69 DF 04 01 0D 80 3F 00 00 00 00 00 00 00
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
                78 56 34 12 B4 D1 5B 07 02 00 01 25 00 00 00 00
      B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 04 06 00 0D
      X: 77 6D 21 76 CF AA D5 17 95 E7 C1 A0 D0 09 48 18
         D5 5F B6 8F 77 C3 87 E6 31 CD A1 0C 49 C9 97 87
         05 62 09 AD 05 80 F9 5B 25 76 7A A8 91 01 53 69
         88 8E 6E 6E 50 AD FA 4D 36 31 F1 A6 7B A4 7C CA
      T: 88 8E 6E 6E 50 AD FA 4D
      A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 04 06 00 00
      A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 04 06 00 01
      C: AA 3A 5F C4 60 B9 7D 9D 43 60 01 22 99 E7 DF 11
      S0: 49 68 F9 C5 37 C4 FD C6 D3 50 6F 48 65 D7 72 37
      CipherText: D2 6C 6B D6 D4 68 26 9A 41 60 00 07 99
      U: C1 E6 97 AB 67 69 07 8B
   PrePoll encrypted Payload: D2 6C 6B D6 D4 68 26 9A 41 60 00 07 99 C1 E6 97
                              AB 67 69 07 8B
   STS-V for         POLL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 54 79 1C DD 34 F8 -
                                1C DD 35 17
     STS for         POLL PPDU: D9 9B 10 0A C2 38 BB 61 3C 9E 26 94 57 9D C9 C6 ..
                                CF C4 28 0D E1 09 5E 22 80 54 64 38 B0 75 CC 30
   STS-V for  1. RESPONSE PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 54 7A 1C DD 34 F8 -
                                1C DD 35 17
     STS for  1. RESPONSE PPDU: DD 39 EC 72 E6 CD AC FF E1 17 3B AA 3E 77 3C B3 ..
                                4C F1 78 CB 1C 47 68 E8 9C 74 17 A0 25 5B 17 3D
   STS-V for        FINAL PPDU: 79 E8 65 18 6C BD 86 4B 9C B2 54 7B 1C DD 34 F8 -
                                1C DD 35 17
     STS for        FINAL PPDU: 2F FD 4E F7 3A 8B EC 9E 6D 62 FF F8 B1 34 A0 7A ..
                                53 D8 A3 FB 00 23 05 D7 AE 7D 5A DE 87 EB 87 C4
   Round Index of next Ranging Block (Default Hopping): 00 0C
   FinaData MHR: 49 2B D9 09 16 05 00 00 00 77 8B 07 9D AA 05 00
                 69 DF 04 02 19 80 3F
   FinaData Payload: 78 56 34 12 02 00 01 0C 00 B6 D1 5B 07 00 00 3C
                     0F 01 00 C0 04 9E 07 2C 00
      AES-CCM* key K: CF 27 C7 7E 15 B9 E6 B9 88 62 90 32 FF 9D EE C9
      Nonce N: AF 2C 32 43 20 AF B5 30 00 00 00 05 06
      AddAuthData: 00 17 49 2B D9 09 16 05 00 00 00 77 8B 07 9D AA
                   05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
      PlainTextData: 78 56 34 12 02 00 01 0C 00 B6 D1 5B 07 00 00 3C
                     0F 01 00 C0 04 9E 07 2C 00 00 00 00 00 00 00 00
      AuthData: 00 17 49 2B D9 09 16 05 00 00 00 77 8B 07 9D AA
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential

```
                  05 00 69 DF 04 02 19 80 3F 00 00 00 00 00 00 00
                  78 56 34 12 02 00 01 0C 00 B6 D1 5B 07 00 00 3C
                  0F 01 00 C0 04 9E 07 2C 00 00 00 00 00 00 00 00
        B0: 59 AF 2C 32 43 20 AF B5 30 00 00 00 05 06 00 19
        X: 70 B8 B0 47 F2 D5 2F CF 2C CB E6 9E 75 08 FE FE
           51 1C 21 74 E3 D8 D9 70 39 5D F6 6B 9E AB 49 7A
           B7 49 31 26 50 97 E6 06 5E 77 67 5F 33 9E 52 3F
           23 00 69 8F 0D EA 5A D7 82 72 86 09 D3 36 6A 96
           46 09 CE AE 51 E6 AF 42 7A 3D B2 C0 AD 68 E6 B1
        T: 46 09 CE AE 51 E6 AF 42
        A0: 01 AF 2C 32 43 20 AF B5 30 00 00 00 05 06 00 00
        A1: 01 AF 2C 32 43 20 AF B5 30 00 00 00 05 06 00 01
        A2: 01 AF 2C 32 43 20 AF B5 30 00 00 00 05 06 00 02
        C: 1E 33 E7 49 01 D8 64 E1 DB 6C 6B E0 6C 3B CF 1D
           41 77 C7 8E 21 A1 52 FE D5 67 DA 56 3A 1B B9 C4
        S0: E5 07 B5 67 1A A8 DB 47 51 92 17 AB 6B D7 AC 21
        CipherText: 66 65 D3 5B 03 D8 65 ED DB DA BA BB 6B 3B CF 21
                    4E 76 C7 4E 25 3F 55 D2 D5
        U: A3 0E 7B C9 4B 4E 74 05
    FinaData encrypted Payload: 66 65 D3 5B 03 D8 65 ED DB DA BA BB 6B 3B CF 21
                                4E 76 C7 4E 25 3F 55 D2 D5 A3 0E 7B C9 4B 4E 74 05
```

Copyright © 2023 Car Connectivity Consortium LLC. All rights reserved.
Confidential